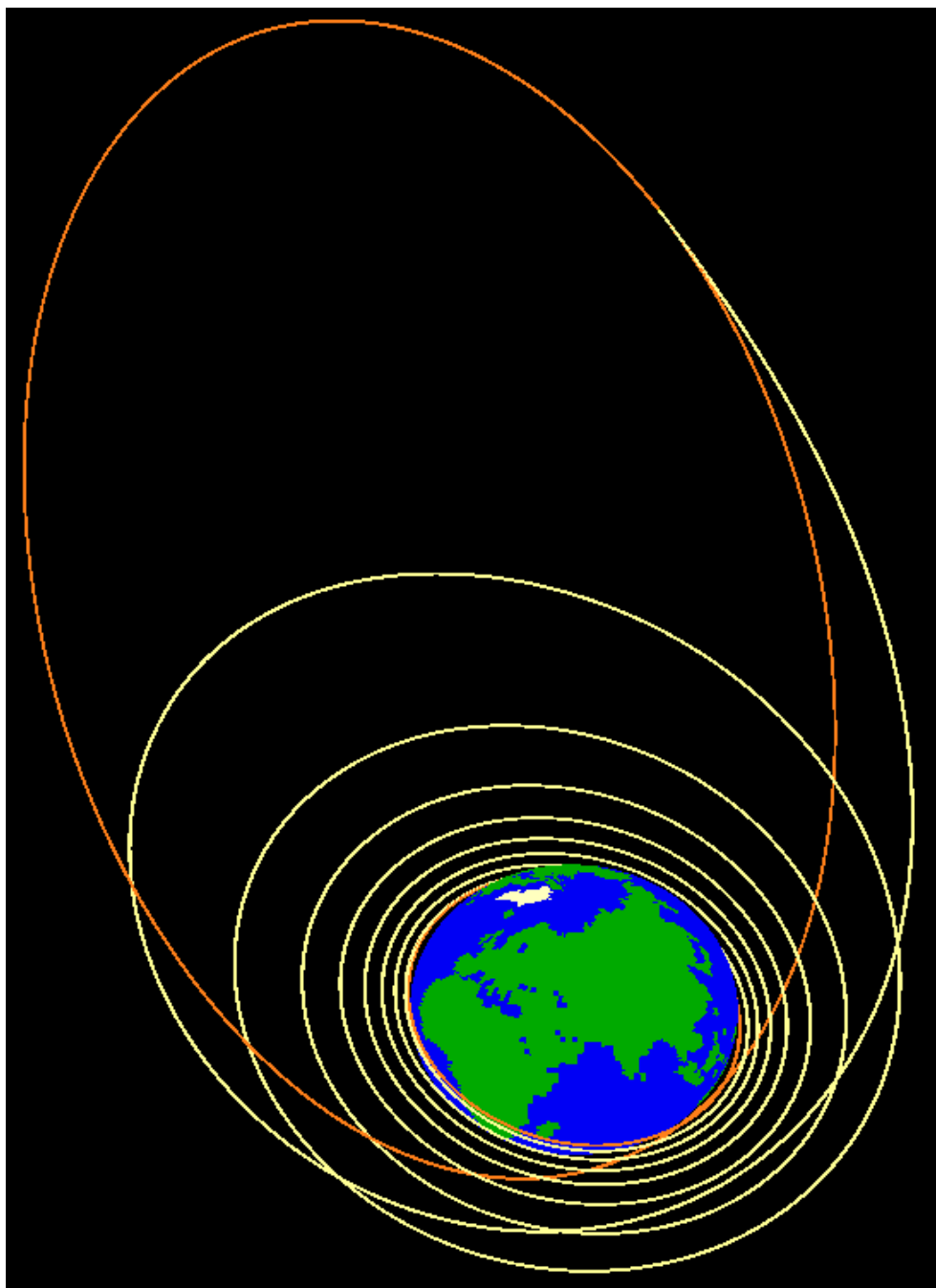


Sparse Optimization Suite

SOS

User's Guide
Release 2025.02



Contents

1	Introduction	1
2	Nonlinear Optimization	3
2.1	Optimization Background	3
2.1.1	Sequential Quadratic Programming	3
2.1.2	Interior Point Algorithm	7
2.2	Subprograms for Optimization	11
2.3	Optimization Iteration Output	246
2.3.1	SQP Algorithms	246
2.3.2	Barrier Algorithms	251
2.4	Reverse Communication Program Structure	254
3	Finite Difference Derivatives	257
3.1	Mathematical Background	257
3.2	Subprograms for Finite Differences	267
4	Advanced Usage Examples	329
4.1	Introduction—Advanced Application Examples	329
4.2	Reverse Communication Dense NLP with Finite Difference Derivatives	330
4.3	Sparse Finite Differences with Sparse NLP	338
4.4	Sparse Finite Differences with Sparse Least Squares	348
4.5	Diagnostic Line Search Output	358
5	Optimal Control Background	363
6	Optimal Control Software	375

6.1	Overview of Sparse Optimal Control Software	375
6.2	Subprograms for Optimal Control	380
6.3	User-Supplied Subprograms for Optimal Control	416
6.4	Utility Subprograms for Optimal Control	460
6.5	Optimal Control Error Messages	519
6.6	Optimal Control Iteration Output	527
6.6.1	Sparse NLP Iteration Output	527
6.6.2	Optimal Control Output	528
6.6.3	Differential Equation Output	544
6.6.4	Sparse Finite Difference Output	545
7	Usage Examples	547
7.1	Linear Tangent Steering Example	547
7.1.1	Problem Definition	547
7.1.2	SOSCTL Implementation	548
7.2	Generalized Linear Tangent Steering Example	569
7.2.1	Problem Definition	569
7.2.2	SOSCTL Implementation	569
7.3	Workshop Self-Instructional Examples	586
7.3.1	Problem 1: Simplified Usage Dense NLP–DNLPCFW	587
7.3.2	Problem 2: Reverse Communication Dense NLP–DNLPCRV	589
7.3.3	Problem 3: Sparse NLP–SNLPMN	589
7.3.4	Problem 4: Sparse Least Squares–SNLPLS	591
7.3.5	Problem 5: Reverse Communication NLP, Finite Difference Gradients	591
7.3.6	Problem 6: Sparse NLP with Sparse Finite Differences	592
7.3.7	Problem 7: One Phase Example	592
7.3.8	Problem 8: User Routines Example	593
7.3.9	Problem 9: Path Constraint Example	593
7.3.10	Problem 10: Quadrature Example	594
7.3.11	Problem 11: Point Function Example	594
7.3.12	Problem 12: Initial Guess Example	595

7.3.13	Problem 13: Auxilliary Output Example	595
7.3.14	Problem 14: Two Phase Example	595
7.4	SOS Demonstration/Test Examples	596
7.4.1	Zermelo’s Problem	596
7.4.2	Goddard Rocket Problem	596
7.4.3	Van der Pol Oscillator Problem	597
7.4.4	Maximum Crossrange Space Shuttle Reentry Problem	599
7.4.5	Minimum Time to Climb Problem	599
7.4.6	Low-Thrust Orbit Transfer	599
7.4.7	Two-Burn Orbit Transfer	600
7.4.8	Industrial Robot	600
7.4.9	Multibody Mechanism	600
7.4.10	Heat Flow–Parabolic PDE	600
7.4.11	Embedding (Homotopy) Example Problem	600
7.4.12	Orbit Determination (Parameter Estimation) Example Problem	601
7.4.13	First-Order Irreversible Chain Reaction Parameter Estimation Problem . . .	601
7.4.14	Initial Value and Shooting Method Example Problems	601
8	Spline Data Approximations	603
8.1	Overview of Data Fitting and Approximation	603
8.2	Subprograms for Tensor Product Spline Approximations of Multivariate Data	604
9	Getting Started	685
A	The Index Set Array	687
B	Standard Math and Machine Constants	691

List of Tables

6.1	Setting $\text{ITERM}(4,j)$ when $\text{ITERM}(3,j) = -1$	440
6.2	Setting $\text{ITERM}(4,j)$ when $\text{ITERM}(3,j) = 0$	440
6.3	Setting Path Domain	441

List of Subprograms

DNLPFW: Dense Nonlinear Programming	12
DNLPGF: Dense Nonlinear Programming—User Supplied Gradients	31
DNLPRV: Dense Nonlinear Programming—Reverse Communication Format	49
SNLPMN: Sparse Nonlinear Programming	67
SNLPLS: Sparse Constrained Nonlinear Least Squares	86
SHURQP: Sparse Quadratic Programming	107
DBNLPF: Dense Barrier Nonlinear Programming	121
DBNPGF: Dense Barrier Nonlinear Programming—User Supplied Gradients	137
DBNLPR: Dense Barrier Nonlinear Programming—Reverse Communication Format	153
SBRNLP: Sparse Barrier Nonlinear Programming	171
SBRLSQ: Sparse Barrier Constrained Nonlinear Least Squares	191
SBRLPQ: Sparse Barrier Quadratic Programming	211
INSNLP: Sparse Nonlinear Programming Input Procedure	226
RTSNLP: Sparse Nonlinear Programming Parameter Retrieval	244
HDDFDJ: Dense Finite Difference Jacobian—Reverse Communication Format	268
HDDFDH: Finite Difference Jacobian and Hessian—Reverse Communication Format	277
HJSFDI: Generate Sparse Finite Difference Index Sets	288
HDSFDJ: Sparse Finite Difference Jacobian—Reverse Communication Format	295
HDSFDH: Sparse Finite Difference Jacobian/Hessian—Reverse Communication Format	308
HDSFDC: Check Sparsity Pattern for Missing Elements—Reverse Communication Format	320
HDSFDP: Print Out Sparse Matrix	325
SOSCTL: Sparse Optimal Control	382
SOSDLY: Sparse Optimal Control of Delay Systems	388
SOSEST: Sparse Optimal Parameter Estimation	395

INPSOS: Sparse Optimization Suite Input Procedure	401
INSOCX: Sparse Optimal Control Input Procedure	403
DDLOAD: Discrete Data Load	416
EVALDL: Delay Equation Evaluation	419
ODEIGS: Optimal Control Initial Guess	422
ODEINP: Optimal Control Problem Definition	426
ODEPRT: Optimal Control Print	446
ODEPTF: Optimal Control Point Function Evaluation	450
ODERHS: Optimal Control Right Hand Side Evaluation	455
AUTOLK: Auto Link Variable Utility	460
AUTOPL: Auto Phase Length Constraint Utility	462
AUXOUT: Auxiliary Output Utility	464
BSPDEF: B-Spline Algebraic Variable Utility	468
CTLFIL: Optimal Control Input From a File	472
CTLSTA: Optimal Control Software Status Utility	474
FETCH: Variable Retrieval Utility	475
FYLFMT: Restart File Format Conversion Utility	477
ICSTPH: Optimal Control Solution Evaluation Utility	479
LINKST: Optimal Control Linkage Utility	481
NUMGRD: Gridpoint Evaluation Utility	483
OCDGRD: Optimal Control Solution and Derivative Grid Evaluation Utility	484
OCSEVL: Optimal Control Solution Evaluation Utility	487
OCSGRD: Optimal Control Solution Grid Evaluation Utility	491
OCSRNG: Optimal Control Solution Range Utility	493
OCSTAU: Optimal Control Solution Evaluation Utility	495
PHSLNG: Optimal Control Phase Duration Utility	498
PNTCON: Optimal Control Point Constraint Utility	500
PTH AUX: Path Constraint with Auxiliary Functions	503
PTHCON: Path Constraint Utility	506
RST2SP: Restart File to Spline Array Utility	509
SCTFIL: Save an Optimal Control Input File	510

SETMRF: Set Mesh Refinement Option	512
STMSEQ: Set Discretization Refinement Sequence	514
TRMAUX: Terminate Auxiliary Function Construction	516
WATCH: Variable Monitor Utility	518
HDMVGI: Interpolation of Multivariate Gridded Data	605
HDMONO: Piecewise Monotonic Cubic Spline Interpolation	612
HDMGDI: Tensor Product Spline Interpolation of Gridded Data	616
HDGDWH: Tensor Product Spline Approximation of Gridded Data with Holes	625
HDCMVS: Constrained Tensor Product Approximation of Multivariate Data	636
HDNPVL: Evaluation of a Tensor Product Spline	655
HDENVL: Spline Evaluation with Extrapolation	662
HDFNVL: Fast Spline Evaluation without Error Checking	670
HDFEVL: Fast Spline Evaluation with Extrapolation, but without Error Checking	676

Chapter 1

Introduction

This document describes software that is available to solve nonlinear optimization, optimal control, parameter estimation, and delay problems with equality and inequality constraints, as well as provide numerical estimates for the requisite function derivatives. The capability in the Sparse Optimization Suite (SOS) software library is available on most major hardware.

Chapter 2 describes a suite of algorithms capable of solving problems with nonlinear objective and constraint functions. Two fundamentally distinct methods are implemented in the nonlinear programming software suite. The first group of routines are based on a sequential quadratic programming method which is described in Section 2.1. Two distinct methods for solving the quadratic programming subproblem are available, one suitable for large sparse applications, the other for small dense problems. Section 2.2 describes software which implements the method for different levels of sophistication. The first subroutine DNLPCFW (p. 12) is a forward communication algorithm which is appropriate for small dense applications with simplified usage requirements. The second routine DNLPCRV (p. 49) is a reverse communication implementation which permits user supplied gradient information to be incorporated for dense applications. The third routine SNLPMN (p. 67) is a reverse communication subroutine which accommodates large sparse nonlinear programming applications. This general purpose algorithm is more sophisticated to use, as one might expect for very large sparse applications. The fourth routine SNLPLS (p. 86) is a reverse communication subroutine which accommodates large sparse constrained problems with least squares objective functions. Subroutine SHURQP (p. 107) is the underlying sparse quadratic programming algorithm used by the higher level nonlinear programming routines.

The second group of routines are based on an interior point method which is described in Section 2.1. Section 2.2 describes software that implements the method for different levels of sophistication. Section 2.2 also describes the procedure for setting optional parameters for any algorithm in the suite. The routine INSNLP (p. 226) used for optional input of algorithm parameters is described. Section 2.3 describes the options available for obtaining iteration output from the suite of subroutines.

Chapter 3 describes a collection of support algorithms for computing finite difference derivatives. Section 3.1 provides an overview of the methods and introduces the relevant terminology. Software for constructing finite difference derivative approximations when matrix sparsity is not exploited are documented in Section 3.2. First derivatives of a set of nonlinear functions can be computed using subroutine HDDFDJ (p. 268). Subroutine HDDFDH (p. 277) can be used to construct finite

difference approximations to the second derivative (Hessian) matrix. Subprograms which implement sparse finite differencing techniques are also described in Section 3.2. The first is HJSFDI (p. 288), which determines the partitioning of the variables into index sets that may be used by the other subroutines in the package. HDSFDJ (p. 295) is the subroutine for computing the Jacobian matrix of the set of functions, i.e., the matrix of first partial derivatives of the functions with respect to the variables. When in addition the second partial derivatives of the functions are needed, HDSFDH (p. 308) will form the Hessian matrix for an arbitrary linear combination of the input functions. For diagnostics, the subroutine HDSFDP (p. 325) is provided to print out either the full matrices or subsets of them, while HDSFDC (p. 320) can be used to check the input sparsity pattern for missing elements. Finally, Appendix A describes the data structure used to communicate information between the subroutines in the finite difference package.

Chapter 4 presents a collection of advanced usage examples which are designed to illustrate features of the optimization software suite commonly encountered in scientific applications.

Chapter 5 describes the mathematical background for optimal control problems, and presents a brief overview of the *direct transcription* method. The Sparse Optimization Suite (SOS) software package is a collection of FORTRAN 90 subroutines capable of solving optimal control problems. The software suite is described in Chapter 6. The package implements the direct transcription or collocation method to convert the continuous control problem into a discrete approximation. The discretization yields a finite dimensional sparse nonlinear programming problem which can be solved using one of the sparse optimization algorithms discussed in Chapter 2. Numerical procedures to improve the accuracy of the discretization by mesh refinement are implemented in the package. Software implementations of the direct transcription method are generically referred to using the acronym SOCX (sparse optimal control extended). In contrast the nonlinear programming implementations described in Chapter 2 are identified by the acronym “NLP” or “NP.”

Chapter 7 presents examples which are designed to illustrate features of the optimal control software suite commonly encountered in scientific applications, and includes a collection of typical application examples. Chapter 8 describes software for constructing spline approximations to tabular data. Chapter 9 presents guidelines for using the tool.

REFERENCES AND FURTHER READING

Each subprogram abstract in this manual assumes that the reader is familiar with the mathematics and computer science aspects of the functional area that the subprogram supports. Listed below are a few representative references that a user may consult for appropriate background material.

Additional information on nonlinear programming can be found in References [9], [8], [17], [18], and [19]. For more information on finite difference techniques consult References [13], and [14]. Background on optimal control methods can be found in References [5], [6], [10], [11], [4], [3], [15], [21], [22], and [16]. Information on the mathematical support libraries can be found in Appendix B and [1].

Chapter 2

Nonlinear Optimization

2.1 Optimization Background

The *Nonlinear Programming Problem* (NLP) requires computing the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the objective function

$$f(x)$$

subject to the nonlinear constraints

$$c_L \leq c(x) \leq c_U$$

and the simple bounds

$$x_L \leq x \leq x_U$$

where c is an m -vector. Equality constraints are imposed by setting $c_L = c_U$ and variables can be fixed by setting $x_L = x_U$. A point is said to be *feasible* if the bounds and constraints are both satisfied. It is assumed that the objective and constraint functions are twice continuously differentiable. Two different algorithms for solving this problem have been implemented and are described in the following subsections.

2.1.1 Sequential Quadratic Programming

A *sequential quadratic programming* (SQP) method is one approach used to solve the nonlinear programming (NLP) problem. The SQP algorithm requires an initial guess x . A new iterate is formed according to the formula

$$\bar{x} = x + \alpha p,$$

where the vector p is the search direction, and $\alpha > 0$ is a scalar step length. The search direction p is found by solving a *quadratic programming* (QP) subproblem defined at the current point x ; minimize the quadratic

$$g^T p + \frac{1}{2} p^T H p$$

subject to the linear constraints

$$\ell \leq \begin{bmatrix} Gp \\ p \end{bmatrix} \leq u,$$

where $\nabla_x f(x) = g(x) = g$ is the n -dimensional gradient vector, G is the $m \times n$ Jacobian matrix of constraint gradients and H is a symmetric $n \times n$ approximation to the Hessian matrix of the Lagrangian $L = f - \lambda^T c - \nu^T x$. The upper bound vector is defined by

$$u = \begin{bmatrix} c_U - c \\ x_U - x \end{bmatrix},$$

with a similar definition for the lower bound vector ℓ , where $c(x) = c$. Two techniques are available for solving the quadratic programming subproblem. When many elements in the Jacobian and Hessian matrices are zero the quadratic programming subproblem is efficiently solved using the sparse Schur-Complement Method. For small to moderate size problems with dense Jacobian and Hessian matrices, a null-space quadratic programming algorithm is employed.

The Hessian matrix used in the QP subproblem is

$$H = H_L + \tau(|\sigma| + 1)I,$$

where the Hessian of the Lagrangian is

$$H_L = \nabla_x^2 f - \sum_{i=1}^m \lambda_i \nabla_x^2 c_i.$$

Since H_L is not necessarily positive definite in the nullspace of the active constraints for a given QP subproblem, simply using $H = H_L$ may cause the QP subproblem to be ill-posed. The strategy is to use $H = H_L$ when possible, but to modify H_L if necessary. The Levenberg parameter τ is chosen such that $0 \leq \tau \leq 1$ and is normalized using the bound σ for the most negative eigenvalue of H_L . Since quadratic convergence of the algorithm can only occur when $\tau = 0$, the parameter is adjusted at every iteration and the rate of decrease is accelerated by monitoring the change in the norm of the projected gradient.

The software provides four options for constructing an approximation B to the Hessian H_L . The first is a *symmetric rank-one* (SR1) approximation given by

$$\overline{B} = B + \frac{(w - Bv)(w - Bv)^T}{(w - Bv)^T v}$$

where the differences $w = \nabla_x L(x) - \nabla_x L(x_-)$ and $v = x - x_-$ involve the gradients at the previous point x_- . A second option, the BFGS or Broyden-Fletcher-Goldfarb-Shanno update given by

$$\overline{B} = B + \frac{ww^T}{w^T v} - \frac{Bvv^T B}{v^T Bv}$$

is a symmetric rank-two positive definite approximation. To insure that the approximation remains positive definite it is necessary to adjust the steplength α such that $w^T v > 0$ and in some cases introduce a modified update. A third option, the SSQN or Self-scaling Quasi-Newton update is given by

$$\overline{B} = \tau B + \frac{ww^T}{w^T v} - \tau \frac{Bvv^T B}{v^T Bv}$$

where $\tau = \min \left[1, \frac{w^T v}{v^T Bv} \right]$. As a final option the Hessian of the Lagrangian may be approximated using finite differences, which is achieved using the finite difference methods described in Chapter 3. The simplified usage subprogram DNLFW constructs all first derivative information using the

finite difference procedures HDDFDJ and/or HDDFDH. Finite difference perturbation sizes are selected automatically to balance truncation and roundoff error. Forward difference estimates are utilized when possible for efficiency, and central difference estimates are invoked when necessary to improve accuracy. The user must supply first and second derivatives for the reverse communication routines DNLPRV, SNLPMN and SNLPLS, and thus it is possible to incorporate analytic and/or sparse difference estimates as desired.

The scalar step length α is adjusted using a line search algorithm to produce a sufficient reduction in an augmented Lagrangian merit function,

$$M = f - \lambda^T(c - s) - \nu^T(x - t) + \frac{1}{2}(c - s)^T Q(c - s) + \frac{1}{2}(x - t)^T R(x - t),$$

where s and t are implicit slack variables for the constraints and bounds, with corresponding Lagrange multipliers λ and ν . The diagonal matrices Q and R consist of a set of penalty weights for each constraint and bound that are adjusted at each iteration. The decrease in the merit function is determined to be “sufficient” if

$$M(\alpha) - M(0) < \mu\alpha M'(0),$$

where $M'(0)$ is the slope of the merit function in the search direction evaluated at $\alpha = 0$ and $\mu = 10^{-4}$. Furthermore a reduction in the slope according to the expression

$$|M'(\alpha)| < -\delta_s M'(0)$$

is required, where $\delta_s \in [0, 1)$ is a user specified slope tolerance.

Algorithm Strategy

The iterative process begins at the initial point x^0 , and terminates at the solution (or final point) x^* . The path followed by the iterates is determined by the algorithm strategy. Four different strategies for locating the solution are implemented.

<i>M</i>	<i>Minimize.</i> Beginning at x^0 solve a sequence of quadratic programs until the solution x^* is found. For well formulated problems this option may be the most efficient. Unfortunately since intermediate iterations are neither feasible nor optimal, premature termination of the algorithm because of formulation difficulties may not be helpful in debugging.
<i>FM</i>	<i>Feasible Minimize.</i> Find a feasible point then minimize. Beginning at x^0 solve a sequence of quadratic programs to locate a feasible point x^f , and then beginning from x^f solve a sequence of quadratic programs until the solution x^* is found. Locating a feasible point is often useful since formulation difficulties are quickly identified. Although feasibility is not maintained, subsequent iterates usually remain “nearly” feasible and often this enhances performance. This is the default option.
<i>FME</i>	<i>Feasible Minimize Equality.</i> Find a feasible point then minimize subject to equalities. Beginning at x^0 solve a sequence of quadratic programs to locate a feasible point x^f , and then beginning from x^f solve a sequence of quadratic programs while maintaining feasible equalities until the solution x^* is found.
<i>F</i>	<i>Feasible.</i> Find a feasible point. Beginning at x^0 solve a sequence of quadratic programs to locate a (possibly non-unique) feasible point $x^f = x^*$.

The first strategy is probably the most aggressive, and is normally used when the initial guess is “good” and/or the quadratic/linear model for the objective and constraints is “good”. The second strategy is the default option, while the *FME* strategy is probably the most conservative. The final strategy *F*, is useful when formulating a new problem or when optimization is not required.

Convergence Criteria

The solution point x^* must satisfy the Kuhn-Tucker necessary conditions for a local minimum:

- (1) x^* is *feasible*, i.e. the constraints and bounds are satisfied;
- (2) there exist Lagrange multipliers λ and ν such that

$$g = G^T \lambda + \nu;$$

- (3) the Lagrange multiplier for a constraint or variable active at its lower bound must be non-negative;
- (4) the Lagrange multiplier for a constraint or variable active at its upper bound must be non-positive;
- (5) the Lagrange multiplier for a strictly feasible constraint or free variable must be zero.

In general it may not be possible or desirable to satisfy the Kuhn-Tucker conditions exactly because of limited precision in the computed quantities. Instead, the algorithm is terminated at a point $\bar{x} \approx x^*$. For a given set of tolerances $(\delta_c, \delta_o, \delta_p)$ the point $\bar{x}$ is considered an estimate of the true solution x^* when it satisfies the following conditions:

- (a) it must be feasible, that is,

$$\begin{aligned} c_L - \delta_c &\leq c(\bar{x}) \leq c_U + \delta_c, \\ x_L - \delta_c &\leq \bar{x} \leq x_U + \delta_c; \end{aligned}$$

- (b) it must satisfy the optimality conditions,

$$\|g - G^T \lambda - \nu\|_\infty < \delta_p \max[1, \|g\|_\infty];$$

- (c) the current objective f and predicted minimum must be within tolerance,

$$|f - f^*| = |g^T p + \frac{1}{2} p^T H p| < \delta_o;$$

- (d) the steplength must be within tolerance,

$$\max_i \left[\frac{\alpha |p_i|}{1 + |\bar{x}_i|} \right] < \left[\frac{\delta_o}{1 + |f|} \right]^{\frac{1}{2}};$$

- (e) the Lagrange multipliers for the inequalities must have the correct sign,

$$\begin{aligned} \nu_i &\geq 0 && \text{for } \bar{x}_i = x_{L_i}; \\ \nu_i &\leq 0 && \text{for } \bar{x}_i = x_{U_i}; \\ \lambda_i &\geq 0 && \text{for } c_i = c_{L_i}; \\ \lambda_i &\leq 0 && \text{for } c_i = c_{U_i}. \end{aligned}$$

2.1.2 Interior Point Algorithm

An *interior point* or *barrier* method is another approach used to solve the nonlinear programming (NLP) problem. First the NLP as originally posed is recast in the following *internal format*:

Objective: minimize

$$f(y) \equiv f(x)$$

Variables:

$$y \equiv \begin{pmatrix} x \\ s \end{pmatrix}$$

(Nonlinear) Equality constraints:

$$c(y) \equiv \begin{pmatrix} c_{\mathcal{E}}(x) - c_{\mathcal{E}L} \\ c_{\mathcal{I}}(x) - s \end{pmatrix} = 0$$

Bounds:

$$b(y) \equiv \begin{pmatrix} (x - x_L)_{\mathcal{B}_1} \\ (x_U - x)_{\mathcal{B}_2} \\ (s - c_{\mathcal{I}L})_{\mathcal{B}_3} \\ (c_{\mathcal{I}U} - s)_{\mathcal{B}_4} \end{pmatrix} \geq 0.$$

The internal format includes the original variables that are free as well as slack variables s . The slack variables are used to convert the original nonlinear inequality constraints denoted by $c_{\mathcal{I}}(x)$ into equalities. In so doing, simple bounds must be introduced for the slack variables as well. Furthermore it is not necessary to include infinite bound values when the problem is recast. Thus we denote the variables with finite lower bounds by $(x - x_L)_{\mathcal{B}_1}$, with a similar treatment for the other bounds. The net result is that the original problem is recast in terms of nonlinear equality constraints, and simple linear bounds.

The fundamental method is stated in terms of the *logarithmic barrier function*

$$\beta(y, \mu) = f(y) - \mu \sum_{k=1}^{m_B} \ln b_k(y)$$

where the parameter μ is called the *barrier parameter*. The basic idea of a *primal barrier method* is to solve a sequence of equality constrained problems as $\mu \rightarrow 0$, namely minimize:

$$\beta(y, \mu),$$

subject to the equalities

$$c(y) = 0.$$

The solution of each equality constrained subproblem is characterized by the *modified primal-dual necessary conditions*:

$$F_\mu \equiv \begin{pmatrix} g - C^T \eta - B^T \lambda \\ c \\ D_b(\lambda - \pi_b) \end{pmatrix} = 0.$$

where C is the Jacobian of the equalities, B is the Jacobian of the bounds, and the matrix $D_b = \text{Diag}(b_1, b_2, \dots, b_{m_B})$ and $(\pi_b)_k = \mu/b_k$. The Lagrange multipliers for the equalities and inequalities are denoted by η and λ respectively. The final set of equations, referred to as “*approximate complementarity*” conditions are often written as

$$b_k(y)\lambda_k - \mu = 0$$

for $k = 1, \dots, m_B$, and the quantities $(\pi_b)_k \approx \lambda_k$. It can be shown that the solution to this sequence of equality constrained problems converges to the solution of the original inequality constrained NLP.

Application of Newton’s Method to the necessary conditions requires the solution of the following *symmetric primal-dual KKT system*:

$$\begin{pmatrix} W & C^T & B^T D_v \\ C & 0 & 0 \\ D_r D_\lambda B & 0 & -D_r D_b D_v \end{pmatrix} \begin{pmatrix} \Delta y \\ -\Delta \eta \\ -D_v^{-1} \Delta \lambda \end{pmatrix} = - \begin{pmatrix} g - C^T \eta - B^T \lambda \\ c \\ D_r D_b (\lambda - \pi_b) \end{pmatrix}$$

The following row and column scaling

$$\begin{aligned} D_r &= \text{Diag}(r_1, r_2, \dots, r_{m_B}) & r_k &= \frac{1}{\sqrt{\lambda_k}}. \\ D_v &= \text{Diag}(v_1, v_2, \dots, v_{m_B}) & v_k &= \sqrt{\lambda_k}. \end{aligned}$$

is used to make $D_r D_b D_v$ well-behaved as $\mu \rightarrow 0$. The Hessian of the Lagrangian is defined by

$$W = \begin{pmatrix} H & 0 \\ 0 & 0 \end{pmatrix}$$

with H defined as before in the SQP method.

When the row and column scaling matrices $D_r = D_v = I$, the step can be computed by solving the following *condensed primal-dual KKT system*:

$$\begin{pmatrix} W + B^T D_b^{-1} D_\lambda B & C^T \\ C & 0 \end{pmatrix} \begin{pmatrix} \Delta y \\ -\Delta \eta \end{pmatrix} = - \begin{pmatrix} g - C^T \eta - B^T \pi_b \\ c \end{pmatrix}$$

where

$$\Delta \lambda = -D_b^{-1} D_\lambda B \Delta y - \lambda + \pi_b.$$

This is the default method used by the software.

After solving the KKT system an improved estimate for the variables is constructed using the *primal-dual step*:

$$\begin{pmatrix} \bar{y} \\ \bar{\eta} \\ \bar{\lambda} \end{pmatrix} = \begin{pmatrix} y \\ \eta \\ \lambda \end{pmatrix} + \alpha \begin{pmatrix} \Delta y \\ \gamma \Delta \eta \\ \gamma \Delta \lambda \end{pmatrix}.$$

where the primal step length α is limited by $b(y) \geq 0$ and the dual step length $\alpha\gamma$ by $\lambda \geq 0$.

In order to ensure convergence of the barrier algorithm when it is applied to nonlinear non-convex problems it is necessary to adjust the steplength α based on some *globalization criteria*. In particular, for a fixed barrier parameter μ , we would like to

- minimize the log-barrier function $\beta(y, \mu)$ and,

- minimize the constraint violation

$$v^{(k)} = \left\| \begin{array}{c} c(y) \\ D_b(\lambda - \pi_b) \end{array} \right\|_{\infty}.$$

In contrast to the merit function used for globalization of the SQP algorithm, we incorporate a *nonlinear filter* proposed by Fletcher and Leyffer. Essentially, the strategy is to accept an iterate $\{\beta^{(\ell)}, v^{(\ell)}\}$ if there is improvement in either

- the objective $\beta(y, \mu)$ or,
- the violation $v^{(k)}$

compared to previous filter entries. Thus, the *globalization strategy* can be summarized as follows:

- If a point is accepted, update the filter and continue,
- If the point is rejected and the primal and dual steps are unequal ($\gamma \neq 1$), then set $\gamma = 1$ and repeat, otherwise,
- If the point is rejected and the primal and dual steps are equal ($\gamma = 1$), reduce the steplength α using a linesearch.

The interior point algorithm requires that $\mu \rightarrow 0$. The software implements the following barrier parameter update strategy:

- if $(\alpha \geq .1)$ and $\|F_{\mu}\| < \min[\kappa\mu, \epsilon_c]$ then

$$\hat{\mu} = \begin{cases} 10\mu^2 & \text{if } \mu < 10^{-4}, \\ \mu/10 & \text{if } \mu \geq 10^{-4}, \end{cases}$$

- elseif μ unchanged for N_u iterations

$$\hat{\mu} = .9\mu.$$

The interior point iterations must begin at a point that is strictly feasible with respect to the inequality constraints. To ensure feasibility the user supplied guess $y^{(0)}$ is modified such that $b(y^{(0)}) \geq 0$. Furthermore the software implements three different techniques for multiplier initialization, namely:

- Central Path Estimate:

$$\lambda = \mu D_b^{-1} e$$

- Minimum Norm Estimate:

Compute $\tilde{\mu}$, $\tilde{\eta}$ and $\tilde{\lambda}$ to minimize

$$\|g - C^T \tilde{\eta} - B^T \tilde{\lambda}\| \quad \text{or} \quad \left\| \begin{array}{c} g - C^T \tilde{\eta} - B^T \tilde{\lambda} \\ D_b \tilde{\lambda} - \tilde{\mu} e \end{array} \right\|$$

To ensure the multipliers are feasible the initial values $\tilde{\lambda}$ are modified such that $\tilde{\lambda} \geq 0$.

The primary algorithm, as described is appropriate for most nonlinear programming applications. In practice, it is not uncommon to encounter applications that are not amenable to solution using this primary formulation. As a result, we consider alternate methods of solution. One common situation is infeasible linear constraints. This can be a local phenomenon due to poor linearization of the nonlinear constraints. It can also be caused by a fundamental inconsistency in the problem. Regardless of the cause, the computed estimates for the Lagrange multipliers may become very large, ultimately leading to failure of the iterative process. In order to treat these difficulties we also consider a *relaxation formulation* that utilizes an *elastic programming* formulation. The basic idea is to replace the original problem with a relaxed (hopefully equivalent) problem that is more amenable to solution.

The key notion is to augment the variables x with an additional set of nonnegative slack variables that allow the constraints to be satisfied, and then try to minimize the sum of their values. That is,

$$y \equiv \begin{pmatrix} x \\ s \\ t \\ u \\ v \\ w \end{pmatrix}.$$

The *composite objective* to be minimized is

$$F(y) = f(y) + \rho e_t^T (t + u) + \rho e_v^T v + \rho e_w^T w,$$

where $\rho > 0$ is a penalty weight and e_t , e_v , and e_w are vectors of ones. Furthermore, the original equality constraints are modified to become

$$c(y) \equiv \begin{pmatrix} c_{\mathcal{E}}(x) - c_{\mathcal{E}L} - t + u \\ c_{\mathcal{I}}(x) - s \end{pmatrix} = 0,$$

subject to the modified bound inequalities

$$b(y) \equiv \begin{pmatrix} (x - x_L)_{\mathcal{B}_1} \\ (x_U - x)_{\mathcal{B}_2} \\ (s + v - c_{\mathcal{I}L})_{\mathcal{B}_3} \\ (c_{\mathcal{I}U} - s + w)_{\mathcal{B}_4} \\ t \\ u \\ v \\ w \end{pmatrix} \geq 0.$$

It can be demonstrated that the relaxed problem has the same solution as the original problem, provided the value of the relaxation penalty parameter ρ is large enough.

2.2 Subprograms for Optimization

SQP Subprograms

DNLFW: Dense Nonlinear Programming	12
DNLPG: Dense Nonlinear Programming–User Supplied Gradients	31
DNLPRV: Dense Nonlinear Programming–Reverse Communication Format	49
SNLPMN: Sparse Nonlinear Programming	67
SNLPLS: Sparse Constrained Nonlinear Least Squares	86
SHURQP: Sparse Quadratic Programming	107

Barrier Subprograms

DBNLFW: Dense Barrier Nonlinear Programming	121
DBNPG: Dense Barrier Nonlinear Programming–User Supplied Gradients	137
DBNLPR: Dense Barrier Nonlinear Programming–Reverse Communication Format	153
SBRNLP: Sparse Barrier Nonlinear Programming	171
SBRLSQ: Sparse Barrier Constrained Nonlinear Least Squares	191
SBRLPQ: Sparse Barrier Quadratic Programming	211

Optional Algorithm Parameters

INSNLP: Sparse Nonlinear Programming Input Procedure	226
RTSNLP: Sparse Nonlinear Programming Parameter Retrieval	244

DNLFW: Dense Nonlinear Programming

PURPOSE

DNLFW computes the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the objective function

$$f(x)$$

subject to the m nonlinear constraints

$$c_L \leq c(x) \leq c_U$$

and the simple bounds

$$x_L \leq x \leq x_U$$

Equality constraints are imposed by setting $c_{Li} = c_{Ui}$ and variables can be fixed by setting $x_{Li} = x_{Ui}$. DNLFW works under the assumption that the objective and constraint functions are twice continuously differentiable, and if this is not true algorithm performance is unpredictable. DNLFW uses a forward communication format and the user must supply a subroutine to compute the values of the objective and constraint functions. First and second derivatives of the user supplied functions are computed numerically. The matrix of second derivatives (Hessian of the Lagrangian) can be constructed using a quasi-Newton recursive estimate or by central differencing. Subroutine DNLFW calls the sparse nonlinear programming algorithm SNLPMN, and is intended for simplified usage as appropriate for small dense problems.

RELATED SUBPROGRAMS

HDDFDJ	Finite Difference Jacobian
HDDFDH	Finite Difference Hessian
DBNLFP	Dense Barrier Nonlinear Programming
SNLPMN	Sparse Nonlinear Programming
INSNLP	Sparse Nonlinear Programming Input Procedure

METHOD

A *sequential quadratic programming* (SQP) approach is used to solve the *nonlinear programming* (NLP) problem. Although the dense null-space method is the default, the Schur-complement method can be selected to solve the quadratic programming subproblem. DNLFW provides four alternatives for constructing an approximation to the Hessian matrix which are described in the background section of this document. The default is a *symmetric rank-one* (SR1) approximation, with options for using a BFGS, SSQN, or finite difference estimate instead. DNLFW constructs all first derivative information using the finite difference procedures HDDFDJ and/or HDDFDH. Finite difference perturbation sizes are selected automatically to balance truncation and roundoff error. Forward difference estimates are utilized when possible for efficiency, and central difference estimates are invoked when necessary to improve accuracy. Although default values are set for all algorithm parameters, INSNLP can be utilized to override the defaults.

WARNING

The objective function and constraints supplied by the user are assumed to be continuous and have continuous first and second derivatives. Nondifferentiable functions can cause unpredictable performance in the optimization algorithm. It should be emphasized that discontinuities in the Hessian matrices can significantly degrade the speed of convergence without producing any other obvious difficulties. The following common sources of error *should be avoided* when evaluating the objective and constraint functions:

- (1) nonsmooth interpolation of data;
- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;
- (3) discontinuous behavior caused by branching for IF tests;
- (4) nondifferentiable functions such as ABS, MAX, and MIN.

The user must provide a procedure to evaluate the objective and constraint functions. If it is not possible to evaluate the functions (for example, $c(x) = 1/x$ at $x = 0$) then set IFERR = 1 and exit the function evaluation process. Otherwise, set IFERR = 0 when the functions are evaluated. The function error flag should be used to avoid computational difficulties, but never in lieu of an explicit constraint.

The DNLPPFW software uses internal common blocks to communicate information. To avoid conflict with these internal commons the user should not have commons with the following names: NPSRR, NPSRI, NSPRC, STATIS, KONSTN, NPSALG, NPSDFL, NLPERR, PERCOM, ITEREF, and CLKCOM.

USAGE

```

INTEGER      IER, NDIM, MCON, NHOLD, NIHOLD, NEEDED
INTEGER      ISTATV(NDIM), ISTATC(MCON), IHOLD(NIHOLD)

DOUBLE PRECISION      FBAR, XBAR(NDIM), XLWR(NDIM), XUPR(NDIM), VECNU(NDIM)
DOUBLE PRECISION      CBAR(MCON), CLWR(MCON), CUPR(MCON)
DOUBLE PRECISION      VECLAM(MCON), HOLD(NHOLD)

```

```

EXTERNAL      FUNBOX

```

```

CALL DNLPPFW(FUNBOX, XBAR, XLWR, XUPR, ISTATV, VECNU, NDIM,
$           FBAR, CBAR, CLWR, CUPR, ISTATC, VECLAM, MCON,
$           HOLD, NHOLD, IHOLD, NIHOLD, NEEDED, IER)

```

ARGUMENTS

FUNCTION EVALUATION PROCEDURE

FUNBOX [INPUT, SUBROUTINE]

Name of user-supplied subroutine to evaluate objective and constraint functions;
see USER-SUPPLIED SUBPROGRAM

INDEPENDENT VARIABLE DATA

XBAR [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM of current variable values x . An initial guess must be input. When IER = 0, +101, the final point corresponds to a local optimum. When IER = +104, ..., +111, +114, +116, +117, +119, the final point is either the best feasible point if one has been found, or the last iterate.

XLWR [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM containing the variable lower bounds x_L . For variables that have no lower bound set $x_{Li} \leq -.01/\text{HDMCON}(5)$. See Appendix B for HDMCON.

XUPR [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM containing the variable upper bounds x_{Ui} . For variables that have no upper bound set $x_{Ui} \geq .01/\text{HDMCON}(5)$.

ISTATV [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length NDIM indicating variable status. On input ISTATV defines an estimate of the active variable bounds, and on output ISTATV defines the bounds active at the solution.

ISTATV(i)= 0 Variable is strictly feasible, i.e. $x_{Li} < x_i < x_{Ui}$.

ISTATV(i)= 1 Variable lower bound is active, i.e. $x_{Li} = x_i$.

ISTATV(i)= 2 Variable upper bound is active, i.e. $x_i = x_{Ui}$.

ISTATV(i)= 3 Fixed permanently.

ISTATV(i)= 10 Output only: Violated variable lower bound, i.e. $x_{Li} - \delta_c > x_i$.

ISTATV(i)= 20 Output only: Violated variable upper bound, i.e. $x_i > x_{Ui} + \delta_c$.

ISTATV(i)= 30 Output only: Violated fixed variable. Either $x_i < x_{Li} - \delta_c$ or $x_i > x_{Ui} + \delta_c$.

When ISTATV(i) = 3 it is necessary that $x_{Li} = x_{Ui}$ where $x_{Li} = \text{XLWR}(i)$ and $x_{Ui} = \text{XUPR}(i)$. When ISTATV(i) $\neq$ 3, it is necessary that $x_{Li} < x_{Ui}$ with $|x_{Li} - x_{Ui}| > \delta_c$.

VECNU [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
An array of length NDIM of bound multipliers ν .

NDIM [INPUT, INTEGER]
Number of variables, where NDIM $\geq$ 1.

OBJECTIVE FUNCTION DATA

FBAR	[OUTPUT, DOUBLE PRECISION] The objective function evaluated at XBAR, $f(\text{XBAR})$.
CONSTRAINT DATA	
CBAR	[OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON containing the constraints $c(x)$ evaluated at XBAR. Not referenced when MCON = 0.
CLWR	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON containing the constraint lower bounds c_L . For constraints that have no lower bound set $c_{Li} \leq -.01/\text{HDMCON}(5)$. Not referenced when MCON = 0.
CUPR	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON containing the constraint upper bounds c_U . For constraints that have no upper bound set $c_{Ui} \geq .01/\text{HDMCON}(5)$. Not referenced when MCON = 0.
ISTATC	[INPUT/OUTPUT, INTEGER, ARRAY] An integer array of length MCON indicating the constraint status. On input ISTATC defines an estimate of the active constraints, and on output ISTATC defines the constraints active at the solution. ISTATC(i)= 0 Free (inactive) inequality, i.e. $c_{Li} < c_i(x) < c_{Ui}$. ISTATC(i)= 1 Constraint lower bound is active, i.e. $c_{Li} = c_i(x)$. ISTATC(i)= 2 Constraint upper bound is active, i.e. $c_i(x) = c_{Ui}$. ISTATC(i)= 3 Equality. ISTATC(i)= 4 Ignored constraint. ISTATC(i)= 10 Output only: Violated constraint lower bound, i.e. $c_{Li} - \delta_c > c_i$. ISTATC(i)= 20 Output only: Violated constraint upper bound, i.e. $c_i > c_{Ui} + \delta_c$. ISTATC(i)= 30 Output only: Violated equality constraint. Either $c_i < c_{Li} - \delta_c$ or $c_i > c_{Ui} + \delta_c$. When ISTATC(i)=3 it is necessary that $c_{Li} = c_{Ui}$ where $c_{Li} = \text{CLWR}(i)$ and $c_{Ui} = \text{CUPR}(i)$. When ISTATC(i) $\neq$ 3, it is necessary that $c_{Li} < c_{Ui}$ with $ c_{Li} - c_{Ui} > \delta_c$. Not referenced when MCON = 0.
VECLAM	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON of Lagrange multipliers for constraints λ . Not referenced when MCON = 0.
MCON	[INPUT, INTEGER] Number of constraints, where $\text{MCON} \geq 0$.

WORKING STORAGE

HOLD	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length NHOLD containing the hold array. The contents of HOLD must not be changed during the optimization iterations, i.e. between successive calls to FUNBOX.
NHOLD	[INPUT, INTEGER] Dimension of HOLD array. When IER = -127, -131, or -701, NEEDED contains the required storage.
IHOLD	[INPUT/OUTPUT, INTEGER, ARRAY] An integer array of length NIHOLD containing the integer hold array. The contents of IHOLD must not be changed during the optimization iterations, i.e. between successive calls to FUNBOX.
NIHOLD	[INPUT, INTEGER] Dimension of IHOLD array. When IER = -128, -132, or -702, NEEDED contains the required storage.

OUTPUT ARGUMENTS SET BY ALGORITHM BEFORE TERMINATION

NEEDED	[OUTPUT, INTEGER] When IER = -127, -131, or -701, NEEDED contains the required length of HOLD. When IER = -128, -132, or -702, NEEDED contains the required length for IHOLD.
IER	[OUTPUT, INTEGER] Success/error code. Results are not computed for $IER < 0$. For $IER > 0$, results are computed but may be suspect; user inspection is recommended. Note many of the error returns involve optional input quantities as described in INSNLP.
IER=0	Normal termination.
IER=-101	(MCON < 0); the number of constraints is negative.
IER=-102	(NDIM < 1); the number of variables is less than one.
IER=-104	IHESHN > 3.
IER=-105	(NITMAX < max(1,NITMIN)); the maximum number of iterations is either less than 1 or less than the minimum number of iterations.
IER=-106	(SLPTOL ≥ 1) or (SLPTOL $\leq 10^{-5}$); the slope tolerance for the line search during optimization is too large or too small.
IER=-107	(IT1MAX < 1); the number of line search steps is less than 1.

IER=-108	(IOFLAG < 0) or (IOFLAG > 30); invalid input for the output control flag.
IER=-109	Invalid input for ALGOPT.
IER=-110	(OBJTOL $\leq$ 10.*HDMCON(5)); objective function tolerance is too small.
IER=-111	(PGDTOL $\leq$ [HDMCON(5)] ^{1/2}) or (PGDTOL > 10 ⁻²); the projected gradient tolerance is too small or too large.
IER=-112	CONTOL < [HDMCON(5)] ^{1/2} ; the constraint tolerance is too small.
IER=-113	(ISTATC(i) < 0) or (ISTATC(i) > 4); invalid input for constraint status.
IER=-117	(CUPR(i) < CLWR(i)); constraint upper bound is less than lower bound.
IER=-118	Either (CUPR(i) = CLWR(i)) and (ISTATC(i) $\neq$ 3) or (CUPR(i) $\neq$ CLWR(i)) and (ISTATC(i) = 3); constraint status array is not consistent with bounds.
IER=-119	(CUPR(i) $\neq$ CLWR(i)) and (CUPR(i)-CLWR(i) < CONTOL); constraint bounds are not equal, but differ by less than the constraint tolerance.
IER=-120	(ISTATV(i) < 0) or (ISTATV(i) > 3); invalid input for variable status array.
IER=-124	(XUPR(i) < XLWR(i)); the variable upper bound is less than the lower bound.
IER=-125	Either (XUPR(i) = XLWR(i)) and (ISTATV(i) $\neq$ 3) or (XUPR(i) $\neq$ XLWR(i)) and (ISTATV(i) = 3); the variable status array is not consistent with the bound values.
IER=-126	(XUPR(i) $\neq$ XLWR(i)) and (XUPR(i)-XLWR(i) < CONTOL); variable bounds are not equal but differ by less than the constraint tolerance.
IER=-127	Real hold array too small; insufficient storage detected in SNLPMN interface. The required storage is specified in NEEDED.
IER=-128	Integer hold array too small; insufficient storage detected in SNLPMN interface. The required storage is specified in NEEDED.
IER=-129	Function error at initial point or during gradient evaluation.
IER=-130	(The number of active constraints > NDIM); check ISTATC(i) and ISTATV(i).

IER=-131	NHOLD too small; insufficient real storage detected in algorithm. The required storage is specified in NEEDED.
IER=-132	NIHOLD too small; insufficient integer storage detected in algorithm. The required storage is specified in NEEDED.
IER=-133	Rank deficient Jacobian detected on successive iterations.
IER=-134	INSNLP input error; invalid character string displayed.
IER=-135	(IFERR < 0) or (IFERR > 1); invalid value for function error flag.
IER=-136	(SFZTOL ≥ 1) or (SFZTOL $\leq 10^{-5}$); the slope tolerance for the line search during the feasibility phase is too large or too small.
IER=-137	Conflict between user and multifrontal file number; check IPUMF1,...,IPUMF6 IPUDRF, IPUFZF, IPUSTF.
IER=-138	(NEWTON < 0) or (NEWTON > 1); invalid input for Newton method option flag.
IER=-147	Unexpected error; check storage allocation.
IER=-149	Incorrect value for QPOPTN.
IER=-150	QPOPTN = DENSE and either ALGOPT = 'FME' or NRES>0.
IER=-152	Incorrect value for KTOPTN.
IER=-153	I/O error, probably caused by insufficient disk space.
IER=-701	Real hold array too small; insufficient storage detected in dense interface. The required storage is specified in NEEDED.
IER=-702	Integer hold array too small; insufficient storage detected in dense interface. The required storage is specified in NEEDED.
IER=-703	$ \text{EPSRLF} < [\text{HDMCON}(5)]^{\frac{1}{2}}$.

IER=+101	Weak solution found (multipliers near zero).
IER=+102	Number of equality constraints = NDIM and ALGOPT $\neq$ 'F'.
IER=+103	Maximum number of consecutive function errors.
IER=+104	Maximum number of function evaluations.

IER=+105	Small step termination in optimization phase; suboptimal feasible point found.
IER=+106	Maximum number of iterations in optimization phase.
IER=+107	Maximum number of iterations in feasibility phase.
IER=+108	Feasible point not found.
IER=+109	Maximum number of interval halves in line search.
IER=+110	QP algorithm terminated because either the Hessian diagonal reached its maximum value or the merit function slope condition was violated; suboptimal feasible point found.
IER=+111	Projected gradient calculation failed; constraints may be degenerate.
IER=+112	Calculation of first order multiplier estimates failed; constraints may be degenerate.
IER=+113	Suboptimal feasible point found.
IER=+114	Schur-complement QP algorithm (subroutine SHURQP) failed with unexpected error.
IER=+115	CONTOL > OBJTOL; convergence tolerances may be inappropriate.
IER=+116	Uphill direction detected in line search.
IER=+117	Reduced objective function is linear.
IER=+118	Both $c_{Li} \leq -.01/\text{HDMCON}(5)$ and $c_{Ui} \geq .01/\text{HDMCON}(5)$; constraints ignored.
IER=+119	Terminate after diagnostic line search.
IER=+120	(IHESHN > 0) and (NEWTON = 0); recursive Hessian updates may be skipped causing poor algorithm performance.
IER=+703	Sparse parameter defaults may cause poor algorithm performance.

USER-SUPPLIED SUBPROGRAM

The user must provide a real subroutine, FUNBOX to compute the objective and constraint functions. Subroutine FUNBOX must have the following format:

```

SUBROUTINE FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
C
DOUBLE PRECISION XBAR,QUANT
INTEGER NDIM,MCON,IFERR
DIMENSION XBAR(NDIM),QUANT(MCON+1)
C
C      IF POSSIBLE EVALUATE OBJECTIVE AND CONSTRAINTS, OTHERWISE
C      SET FUNCTION ERROR FLAG.
C
IF( Functions can be evaluated at XBAR) THEN
C
C      SET FUNCTION ERROR FLAG
C
IFERR = 0
C
C      CONSTRAINTS
C
QUANT(1) = User supplied function evaluation process
.
.
.
QUANT(MCON) = User supplied function evaluation process
C
C      OBJECTIVE FUNCTION
C
QUANT(MCON+1) = User supplied function evaluation process
C
ELSE
C
C      SET FUNCTION ERROR FLAG
C
IFERR = 1
C
END IF
C
RETURN
END
```

The arguments of subroutine FUNBOX are defined as follows:

ARGUMENTS

XBAR [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM containing the current variable values x .

NDIM [INPUT, INTEGER]
Number of variables n , where $\text{NDIM} \geq 1$.

MCON	[INPUT, INTEGER] Number of constraints m , where $MCON \geq 0$.
QUANT	[OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length $MCON+1$ containing the values of the computed functions at the current value of x . The first $MCON$ elements of QUANT contain the constraints $c(x)$, and the objective function $f(x)$ is stored in location $MCON+1$. When the function error flag $IFERR = 1$, the QUANT array will not be referenced.
IFERR	[OUTPUT, INTEGER] Function error flag. Set $IFERR = 0$ when functions are successfully computed. Set $IFERR = 1$ when functions cannot be computed. Appendix A presents a discussion of the similarity between the input requirements for DNLFW and those of the NPSOL optimization algorithm.

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine DNLFW. All optional inputs are specified by calls to subroutine INSNLP. It should be emphasized that it is *not necessary* to call INSNLP if the default values are appropriate. If optional inputs are specified, it is recommended that the user first set the defaults for a dense problem, that is `CALL INSNLP('DENSE DEFAULT')`, and then specify the desired input parameter.

OPTIONAL PRINTOUT

The level of printed output from DNLFW is controlled by the variables IOFLAG and IPUNLP which are set by calls to INSNLP. The standard level of output which corresponds to $IOFLAG = 10$ is the *default* for DNLFW. A complete description of the iteration output options is found in Section 2.3.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2)$ to minimize the objective function

$$f(x) = x_1^2 + x_2^2 + \log(x_1 x_2)$$

subject to the constraint

$$x_1 x_2 \geq 1$$

and the bounds

$$0 \leq x_k \leq 10$$

for $k = 1, 2$ beginning at the initial point $x = (.5, 2)$. The solution is at $x^* = (1, 1)$.

SAMPLE PROGRAM (dnlpfw.cd1)¹

¹sosexample.tar

```

PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE NONLINEAR
C      PROGRAMMING ALGORITHM DNLPFW
C
C-----
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C      PARAMETER ( NHOLD=10000, NIHOLD=1000 )
C      PARAMETER ( ZERO=0.D0, ONE=1.D0, POINT5 = .5D0, TWO = 2.D0)
C      PARAMETER (MAXDIM = 5, MAXCON = 10)
C
C      NLP CALLING ARGUMENTS
C
C      DIMENSION XBAR(MAXDIM),XLWR(MAXDIM),XUPR(MAXDIM),
C      $      ISTATV(MAXDIM),VECNU(MAXDIM),
C      $      CBAR(MAXCON),CLWR(MAXCON),
C      $      CUPR(MAXCON),ISTATC(MAXCON),VECLAM(MAXCON)
C      DIMENSION HOLD(NHOLD),IHOLD(NIHOLD)
C      EXTERNAL FUNBOX
C
C      DEFINE INDEPENDENT VARIABLE DATA
C
C      NDIM = 2
C      XBAR(1) = POINT5
C      XBAR(2) = TWO
C      xlwr(1:ndim) = zero
C      xupr(1:ndim) = 10.d0
C      istatv(1:ndim) = 0
C
C      DEFINE CONSTRAINT DATA
C
C      BIGBND = 1.D0/HDMCON(5)
C      MCON = 1
C      CLWR(1) = ONE
C      CUPR(1) = BIGBND
C      ISTATC(1) = 0
C
C-----
C
C      CALL NLP TO SOLVE THE PROBLEM
C
C      CALL DNLPFW(FUNBOX, XBAR, XLWR, XUPR, ISTATV, VECNU, NDIM,
C      $      FBAR, CBAR, CLWR, CUPR, ISTATC, VECLAM, MCON,
C      $      HOLD, NHOLD, IHOLD, NIHOLD, NEEDED, IERNLP)
C
C      STOP
C      END

```

```

      SUBROUTINE FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
      SUBROUTINE TO EVALUATE THE PROBLEM FUNCTIONS FOR
      THE VARIOUS PASS/FAIL TEST PROBLEMS
C
      DIMENSION XBAR(NDIM),QUANT(MCON+1)
C
      NONLINEAR CONSTRAINTS
C
      QUANT(1) = XBAR(1)*XBAR(2)
C
      CHECK THAT ARGUMENT OF LOGARITHM IS POSITIVE
      BEFORE ATTEMPTING TO EVALUATE OBJECTIVE
C
      IF(QUANT(1).LT.HDMCON(5)) THEN
C
        FUNCTION ERROR
C
        IFERR = 1
C
      ELSE
C
        IFERR = 0
C
        OBJECTIVE FUNCTION
C
        QUANT(MCON+1) = XBAR(1)**2 + XBAR(2)**2
        $              + LOG(QUANT(1))
C
      ENDIF
C
      RETURN
      END

```

OUTPUT FROM SAMPLE PROGRAM

```

***** INITIAL POINT *****
*****
*
* Objective Function =      4.25000          IERNLP =      0
*
* -----
*
* Variable   Status   Value      Lower bound      Upper bound      Lagrange Mlt.      Slack
*
*      1     FR      0.500000      0.             10.0000          0.             0.5000
*      2     FR      2.00000      0.             10.0000          0.             2.000
*

```

```

* -----
*
* Constraint Status Value Lower bound Upper bound Lagrange Mlt. Slack
*
* 1 FR 1.00000 1.00000 4.503600E+15 0. 0.
*
*****
*****
*
* .....OPTIMIZATION OPERATOR SPRNLP.....
*
*****
*
* -----
* Constraints = 1 | Variables = 2
* Equalities = 0 | Equalities = 0
* Inequalities = 1 | Bounds = 2
* Inactive = 1 | Free = 2
* Fixed on Lower Bound = 0 | Fixed on Lower Bound = 0
* Fixed on Upper Bound = 0 | Fixed on Upper Bound = 0
* -----
* Number of Active Constraints = 0 | Number of Degrees of Freedom = 2
* -----
*
* ALGORITHM CONTROL PARAMETERS (DENSE DEFAULTS)
*
* CONTOL 1.490E-08 Constraint Tolerance
* OBJTOL 1.000E-07 Objective Function Tolerance
* PGDTOL 1.000E-05 Projected Gradient Tolerance
* SLPTOL 0.900 Slope Tolerance For SQP Line Search
* IOFLAG 10 Output Level
* MAXNFE 10000 Maximum Number of Function Evaluations
* NITMAX 100 Maximum Number of Iterations
* ALGOPT FM Algorithm Control Option
*
*****
*
* CONSTRAINED OPTIMIZATION
*
* Number of Active Constraints: 2
* Number of Degrees of Freedom: 0
*
* Constraint Multipliers
*
* Smallest 1 Largest 1
*
* ( 1: 1.7499999 ) ( 1: 1.7499999 )
*
* Constraint Multiplier Magnitudes
*
* Smallest 1 Largest 1
*
* ( 1: 1.7499999 ) ( 1: 1.7499999 )
*
* Bound Multipliers
*
* Smallest 2 Largest 2

```

```

*
*      (      1:      0.      )      (      2:      1.6250003      )
*      (      2:      1.6250003      )      (      1:      0.      )
*
*      Bound Multiplier Magnitudes
*
*      Smallest  1                      Largest  1
*
*      (      2:      1.6250003      )      (      2:      1.6250003      )
*
* -----
*
*      ----- Iteration 1 -----
*      Projected Gradient.....+2.00000 | Objective Function.....+4.25000
*      Equality Error.....+0. | Inequality Error.....+0.
*      Merit Function.....+0.999999 | Penalty Weight Norm.....+1.490116E-08
*      Levenberg Parameter.....+0. | Cond(G).....+1.00000
*      Min. Eigenvalue.....+1.00000 | Max. Eigenvalue.....+1.00000
*      Active Constraints.....2 | Degrees of Freedom.....0
*      QP Iterations.....1 | Matrix Factorizations.....1
*
*      Quadratic Line Search .....Step:  1.....ALFA =   1.0000000
*
*      OBJECTIVE =      4.25000          NFEVAL =      4
*
*      VARIABLES
*
*      1.00000          0.
*
*      CONSTRAINTS
*
*      1.00000
*
*
*      .....Function Error
*
*      Backtrack Line Search .....Step:  1.....ALFA =   9.04543273E-02
*
*      OBJECTIVE =      3.59815          NFEVAL =      5
*
*      VARIABLES
*
*      0.545227          1.81909
*
*      CONSTRAINTS
*
*      0.991818
*
*
*      .....Index =   0.65644490          Change = -0.34355458
*
*      ----- Iteration 2 -----
*      Projected Gradient.....+1.60876 | Objective Function.....+3.59815
*      Equality Error.....+0. | Inequality Error.....+8.181985E-03
*      Merit Function.....+3.03637 | Penalty Weight Norm.....+1.43841
*      Levenberg Parameter.....+2.980232E-08 | Cond(Z'HZ).....+1.00000
*      Min. Eigenvalue.....+0.939795 | Max. Eigenvalue.....+2.82144

```

```

*      Active Constraints.....1 | Degrees of Freedom.....1 *
*      QP Iterations.....2 | Matrix Factorizations.....1 *
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000 *
*
*      OBJECTIVE = 0.525432 NFEVAL = 8
*
*      VARIABLES
*
*      0.925950 0.563858
*
*      CONSTRAINTS
*
*      0.522104
*
*
*      .....Index = 1.7652685 Change = -1.2710978 *
*
*      ----- Iteration 3 ----- *
*      Projected Gradient.....+1.46902 | Objective Function.....+0.525432 *
*      Equality Error.....+0. | Inequality Error.....+0.477896 *
*      Merit Function.....+4.37434 | Penalty Weight Norm.....+22.8481 *
*      Levenberg Parameter.....+1.00000 | Cond(Z'HZ).....+1.00000 *
*      Min. Eigenvalue.....-42.1924 | Max. Eigenvalue.....+10.0428 *
*      Active Constraints.....1 | Degrees of Freedom.....1 *
*      QP Iterations.....6 | Matrix Factorizations.....1 *
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000 *
*
*      OBJECTIVE = 2.59763 NFEVAL = 11
*
*      VARIABLES
*
*      1.34769 0.823151
*
*      CONSTRAINTS
*
*      1.10935
*
*
*      .....Index = 1.7116329 Change = -2.6627057 *
*
*      ----- Iteration 4 ----- *
*      Projected Gradient.....+9.74156 | Objective Function.....+2.59763 *
*      Equality Error.....+0. | Inequality Error.....+0. *
*      Merit Function.....+2.22992 | Penalty Weight Norm.....+109.528 *
*      Levenberg Parameter.....+0.500000 | Cond(Z'HZ).....+1.00000 *
*      Min. Eigenvalue.....-9.99793 | Max. Eigenvalue.....+9.28164 *
*      Active Constraints.....1 | Degrees of Freedom.....1 *
*      QP Iterations.....2 | Matrix Factorizations.....1 *
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000 *
*
*      OBJECTIVE = 2.13949 NFEVAL = 14
*
*      VARIABLES

```

1.20555 0.828830

CONSTRAINTS

0.999193

```

*
* .....Index =      2.1419856          Change = -8.79296203E-02
*
* ----- Iteration 5 -----
* Projected Gradient.....+0.805032 | Objective Function.....+2.13949
* Equality Error.....+0. | Inequality Error.....+8.071959E-04
* Merit Function.....+2.14195 | Penalty Weight Norm.....+1.490116E-08
* Levenberg Parameter.....+4.131946E-02 | Cond(Z'HZ).....+1.00000
* Min. Eigenvalue.....-0.423728 | Max. Eigenvalue.....+129.049
* Active Constraints.....1 | Degrees of Freedom.....1
* QP Iterations.....2 | Matrix Factorizations.....1
*
* Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
* OBJECTIVE =      2.11556          NFEVAL = 17

```

VARIABLES

1.18495 0.843659

CONSTRAINTS

0.999695

```

*
* .....Index =      2.1167558          Change = -2.51940767E-02
*
* ----- Iteration 6 -----
* Projected Gradient.....+1.75451 | Objective Function.....+2.11556
* Equality Error.....+0. | Inequality Error.....+3.054235E-04
* Merit Function.....+2.11676 | Penalty Weight Norm.....+1.490116E-08
* Levenberg Parameter.....+2.065973E-02 | Cond(Z'HZ).....+1.00000
* Min. Eigenvalue.....-3.29794 | Max. Eigenvalue.....+5.20055
* Active Constraints.....1 | Degrees of Freedom.....1
* QP Iterations.....2 | Matrix Factorizations.....1
*
* Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
* OBJECTIVE =      1.94926          NFEVAL = 20

```

VARIABLES

1.02466 0.958043

CONSTRAINTS

0.981665

```

*
* .....Index =      2.0045617          Change = -0.11219409
*

```

```

*
* ----- Iteration 7 -----
* Projected Gradient.....+0.135729 | Objective Function.....+1.94926
* Equality Error.....+0. | Inequality Error.....+1.833500E-02
* Merit Function.....+2.00456 | Penalty Weight Norm.....+1.490116E-08
* Levenberg Parameter.....+1.598242E-03 | Cond(Z'HZ).....+1.00000
* Min. Eigenvalue.....-3.29395 | Max. Eigenvalue.....+4.31658
* Active Constraints.....1 | Degrees of Freedom.....1
* QP Iterations.....2 | Matrix Factorizations.....1
*
* Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
* OBJECTIVE = 1.99673 NFEVAL = 23
*
* VARIABLES
*
* 0.998760 1.00015
*
* CONSTRAINTS
*
* 0.998910
*
* .....Index = 1.9999823 Change = -4.57946574E-03
*
* ----- Iteration 8 -----
* Projected Gradient.....+2.134223E-02 | Objective Function.....+1.99673
* Equality Error.....+0. | Inequality Error.....+1.090436E-03
* Merit Function.....+2.00000 | Penalty Weight Norm.....+31.4657
* Levenberg Parameter.....+2.513098E-04 | Cond(Z'HZ).....+1.00000
* Min. Eigenvalue.....-1.70144 | Max. Eigenvalue.....+4.13121
* Active Constraints.....1 | Degrees of Freedom.....1
* QP Iterations.....2 | Matrix Factorizations.....1
*
* Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
* OBJECTIVE = 2.00000 NFEVAL = 26
*
* VARIABLES
*
* 0.999982 1.00002
*
* CONSTRAINTS
*
* 1.00000
*
* .....Index = 2.0000000 Change = -9.69668600E-07
*
* ----- Iteration 9 -----
* Projected Gradient.....+4.009368E-04 | Objective Function.....+2.00000
* Equality Error.....+0. | Inequality Error.....+1.615718E-07
* Merit Function.....+2.00000 | Penalty Weight Norm.....+1.490116E-08
* Levenberg Parameter.....+4.721125E-06 | Cond(Z'HZ).....+1.00000
* Min. Eigenvalue.....-2.07247 | Max. Eigenvalue.....+4.00536
* Active Constraints.....1 | Degrees of Freedom.....1
* QP Iterations.....2 | Matrix Factorizations.....1

```



```

*
*      Quadratic Line Search .....Step:  1.....ALFA =   1.0000000
*
*      OBJECTIVE =      2.00000      NFEVAL =   29
*
*      VARIABLES
*
*      1.00000      1.00000
*
*      CONSTRAINTS
*
*      1.00000
*
*
*      .....Index =   2.0000000      Change = -1.34865186E-09
*
*      ----- Iteration (10) -----
*      Projected Gradient.....+1.177169E-06 | Objective Function.....+2.00000
*      Equality Error.....+0. | Inequality Error.....+3.284605E-10
*      Merit Function.....+2.00000 | Penalty Weight Norm.....+1.490116E-08
*      Levenberg Parameter.....+4.721125E-06 | Cond(Z'HZ).....+1.00000
*      Min. Eigenvalue.....-2.07247 | Max. Eigenvalue.....+4.00536
*      Active Constraints.....1 | Degrees of Freedom.....1
*      QP Iterations.....0 | Matrix Factorizations.....0
*
*      CONSTRAINED OPTIMIZATION COMPLETED
*
*****
*
*      CONVERGENCE TESTS
*
*      -----
*      Constraints          =   1      | Variables          =   2
*      Equalities          =   0      | Equalities          =   0
*      Inequalities        =   1      | Bounds              =   2
*      Inactive             =   0      | Free                =   2
*      Fixed on Lower Bound =   1      | Fixed on Lower Bound =   0
*      Fixed on Upper Bound =   0      | Fixed on Upper Bound =   0
*
*      -----
*      Number of Active Constraints =   1      | Number of Degrees of Freedom =   1
*      -----
*
*      Predicted Optimum Objective Function =   2.0000000
*
*      Gradient Norm =   3.0000001
*
*      Projected Gradient Vector
*
*      Smallest  2      Largest  2
*
*      (   2: -1.17716857E-06) (   1: -1.65263582E-07)
*      (   1: -1.65263582E-07) (   2: -1.17716857E-06)
*
*      Constraint Multipliers
*
*      Smallest  1      Largest  1
*
*      (   1:  3.0000007   ) (   1:  3.0000007   )

```

```

*
*      Constraint Multiplier Magnitudes
*
*      Smallest  1              Largest  1
*
*      (    1:   3.0000007   )   (    1:   3.0000007   )
*
*      Bound Multipliers
*
*      Smallest  2              Largest  2
*
*      (    1:         0.   )   (    2:         0.   )
*      (    2:         0.   )   (    1:         0.   )
*
*****
***** CONVERGENCE *****
*****
|
|      SPRNLP ALGORITHM PERFORMANCE STATISTICS
|
|      Total CPU Time.....+1.929229E-02
|
|      Number of Function Calls.....11
|      Number of Gradient Calls.....10
|      Number of Hessian Calls.....9
|      Total Number of Function Evaluations.....33
|
|      Storage Needed in HOLD Array.....9960
|      Storage Needed in IHOLD Array.....800
|
*****
***** SOLUTION *****
*****
*
*      Objective Function =      2.00000      IERNLP =      0
*
*      -----
*
*      Variable   Status   Value           Lower bound       Upper bound   Lagrange Mlt.   Slack
*
*      1          FR       1.00000           0.             10.0000       0.             1.000
*      2          FR       1.00000           0.             10.0000       0.             1.000
*
*      -----
*
*      Constraint  Status   Value           Lower bound       Upper bound   Lagrange Mlt.   Slack
*
*      1          LB       1.00000           1.00000          4.503600E+15   3.00000        -3.2846E-10
*
*****

```

DNLPG: Dense Nonlinear Programming–User Supplied Gradients

PURPOSE

DNLPG computes the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the objective function

$$f(x)$$

subject to the m nonlinear constraints

$$c_L \leq c(x) \leq c_U$$

and the simple bounds

$$x_L \leq x \leq x_U$$

Equality constraints are imposed by setting $c_{Li} = c_{Ui}$ and variables can be fixed by setting $x_{Li} = x_{Ui}$. DNLPG works under the assumption that the objective and constraint functions are twice continuously differentiable, and if this is not true algorithm performance is unpredictable. DNLPG uses a forward communication format and the user must supply a subroutine to compute the values of the objective and constraint functions. First derivatives of the user defined functions must also be supplied. Second derivatives of the user supplied functions are computed numerically. The matrix of second derivatives (Hessian of the Lagrangian) is constructed by finite differencing user supplied gradients. Subroutine DNLPG calls the sparse nonlinear programming algorithm SNLPMN, and is intended for simplified usage as appropriate for small dense problems.

RELATED SUBPROGRAMS

HDDFDJ	Finite Difference Jacobian
HDDFDH	Finite Difference Hessian
DNLPGW	Dense Nonlinear Programming
DBNLPG	Dense Barrier Nonlinear Programming
SNLPMN	Sparse Nonlinear Programming
INSNLP	Sparse Nonlinear Programming Input Procedure

METHOD

A *sequential quadratic programming* (SQP) approach is used to solve the *nonlinear programming* (NLP) problem. Although the dense null-space method is the default, the Schur-complement method can be selected to solve the quadratic programming subproblem. DNLPG constructs an approximation to the Hessian matrix using gradient differences and the graph coloring algorithm of Coleman, Garbow and More. Forward difference estimates are utilized by default for efficiency, and central difference estimates can be selected by input. Although default values are set for all algorithm parameters, INSNLP can be utilized to override the defaults.

WARNING

The objective function and constraints supplied by the user are assumed to be continuous and have continuous first and second derivatives. Nondifferentiable functions can cause unpredictable performance in the optimization algorithm. It should be emphasized that discontinuities in the Hessian matrices can significantly degrade the speed of convergence without producing any other obvious difficulties. The following common sources of error *should be avoided* when evaluating the objective and constraint functions:

- (1) nonsmooth interpolation of data;
- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;
- (3) discontinuous behavior caused by branching for IF tests;
- (4) nondifferentiable functions such as ABS, MAX, and MIN.

The user must provide a procedure to evaluate the objective and constraint functions as well as their first derivatives. If it is not possible to evaluate the functions (for example, $c(x) = 1/x$ at $x = 0$) then set IFERR = 1 and exit the function evaluation process. Otherwise, set IFERR = 0 when the functions are evaluated. The function error flag should be used to avoid computational difficulties, but never in lieu of an explicit constraint.

The DNLPGF software uses internal common blocks to communicate information. To avoid conflict with these internal commons the user should not have commons with the following names: NPSPRR, NPSPRI, NPSPRC, STATIS, KONSTN, NPSALG, NPSDFL, NLPERR, PERCOM, ITEREF, and CLKCOM.

USAGE

INTEGER IER, NDIM, MCON

INTEGER ISTATV(NDIM), ISTATC(MCON)

DOUBLE PRECISION FBAR, XBAR(NDIM), XLWR(NDIM), XUPR(NDIM), VECNU(NDIM)

DOUBLE PRECISION CBAR(MCON), CLWR(MCON), CUPR(MCON), VECLAM(MCON)

EXTERNAL FNGBOX

CALL DNLPGF(FNGBOX, XBAR, XLWR, XUPR, ISTATV, VECNU, NDIM,
\$ FBAR, CBAR, CLWR, CUPR, ISTATC, VECLAM, MCON, IER)

ARGUMENTS

FUNCTION EVALUATION PROCEDURE

FNGBOX [INPUT, SUBROUTINE]

Name of user-supplied subroutine to evaluate objective and constraint functions and gradients; see USER-SUPPLIED SUBPROGRAM

INDEPENDENT VARIABLE DATA

XBAR	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM of current variable values x . An initial guess must be input. When IER = 0, +101, the final point corresponds to a local optimum. When IER = +104, ..., +111, +114, +116, +117, +119, the final point is either the best feasible point if one has been found, or the last iterate.
XLWR	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the variable lower bounds x_L . For variables that have no lower bound set $x_{Li} \leq -.01/\text{HDMCON}(5)$. See Appendix B for HDMCON.
XUPR	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the variable upper bounds x_{Ui} . For variables that have no upper bound set $x_{Ui} \geq .01/\text{HDMCON}(5)$.
ISTATV	[INPUT/OUTPUT, INTEGER, ARRAY] An integer array of length NDIM indicating variable status. On input ISTATV defines an estimate of the active variable bounds, and on output ISTATV defines the bounds active at the solution. ISTATV(i)= 0 Variable is strictly feasible, i.e. $x_{Li} < x_i < x_{Ui}$. ISTATV(i)= 1 Variable lower bound is active, i.e. $x_{Li} = x_i$. ISTATV(i)= 2 Variable upper bound is active, i.e. $x_i = x_{Ui}$. ISTATV(i)= 3 Fixed permanently. ISTATV(i)= 10 Output only: Violated variable lower bound, i.e. $x_{Li} - \delta_c > x_i$. ISTATV(i)= 20 Output only: Violated variable upper bound, i.e. $x_i > x_{Ui} + \delta_c$. ISTATV(i)= 30 Output only: Violated fixed variable. Either $x_i < x_{Li} - \delta_c$ or $x_i > x_{Ui} + \delta_c$. When ISTATV(i) = 3 it is necessary that $x_{Li} = x_{Ui}$ where $x_{Li} = \text{XLWR}(i)$ and $x_{Ui} = \text{XUPR}(i)$. When ISTATV(i) $\neq$ 3, it is necessary that $x_{Li} < x_{Ui}$ with $ x_{Li} - x_{Ui} > \delta_c$.
VECNU	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] An array of length NDIM of bound multipliers ν .
NDIM	[INPUT, INTEGER] Number of variables, where $\text{NDIM} \geq 1$.
OBJECTIVE FUNCTION DATA	
FBAR	[OUTPUT, DOUBLE PRECISION] The objective function evaluated at XBAR, $f(\text{XBAR})$.
CONSTRAINT DATA	

- CBAR** [OUTPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraints $c(x)$ evaluated at XBAR. Not referenced when MCON = 0.
- CLWR** [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraint lower bounds c_L . For constraints that have no lower bound set $c_{Li} \leq -.01/\text{HDMCON}(5)$. Not referenced when MCON = 0.
- CUPR** [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraint upper bounds c_U . For constraints that have no upper bound set $c_{Ui} \geq .01/\text{HDMCON}(5)$. Not referenced when MCON = 0.
- ISTATC** [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length MCON indicating the constraint status. On input ISTATC defines an estimate of the active constraints, and on output ISTATC defines the constraints active at the solution.
- ISTATC(i)= 0 Free (inactive) inequality, i.e. $c_{Li} < c_i(x) < c_{Ui}$.
- ISTATC(i)= 1 Constraint lower bound is active, i.e. $c_{Li} = c_i(x)$.
- ISTATC(i)= 2 Constraint upper bound is active, i.e. $c_i(x) = c_{Ui}$.
- ISTATC(i)= 3 Equality.
- ISTATC(i)= 4 Ignored constraint.
- ISTATC(i)= 10 Output only: Violated constraint lower bound, i.e. $c_{Li} - \delta_c > c_i$.
- ISTATC(i)= 20 Output only: Violated constraint upper bound, i.e. $c_i > c_{Ui} + \delta_c$.
- ISTATC(i)= 30 Output only: Violated equality constraint. Either $c_i < c_{Li} - \delta_c$ or $c_i > c_{Ui} + \delta_c$.
- When ISTATC(i)=3 it is necessary that $c_{Li} = c_{Ui}$ where $c_{Li} = \text{CLWR}(i)$ and $c_{Ui} = \text{CUPR}(i)$. When ISTATC(i) $\neq 3$, it is necessary that $c_{Li} < c_{Ui}$ with $|c_{Li} - c_{Ui}| > \delta_c$. Not referenced when MCON = 0.
- VECLAM** [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON of Lagrange multipliers for constraints λ . Not referenced when MCON = 0.
- MCON** [INPUT, INTEGER]
Number of constraints, where $\text{MCON} \geq 0$.

OUTPUT ARGUMENTS SET BY ALGORITHM BEFORE TERMINATION

IER

[OUTPUT, INTEGER]

Success/error code. Results are not computed for $IER < 0$. For $IER > 0$, results are computed but may be suspect; user inspection is recommended. Note many of the error returns involve optional input quantities as described in INSNLP.

IER=0	Normal termination.
IER=-101	(MCON < 0); the number of constraints is negative.
IER=-102	(NDIM < 1); the number of variables is less than one.
IER=-104	IHESHN > 3.
IER=-105	(NITMAX < max(1,NITMIN)); the maximum number of iterations is either less than 1 or less than the minimum number of iterations.
IER=-106	(SLPTOL ≥ 1) or (SLPTOL $\leq 10^{-5}$); the slope tolerance for the line search during optimization is too large or too small.
IER=-107	(IT1MAX < 1); the number of line search steps is less than 1.
IER=-108	(IOFLAG < 0) or (IOFLAG > 30); invalid input for the output control flag.
IER=-109	Invalid input for ALGOPT.
IER=-110	(OBJTOL $\leq 10 \cdot \text{HDMCON}(5)$); objective function tolerance is too small.
IER=-111	(PGDTOL $\leq [\text{HDMCON}(5)]^{\frac{1}{2}}$) or (PGDTOL $> 10^{-2}$); the projected gradient tolerance is too small or too large.
IER=-112	CONTOL < $[\text{HDMCON}(5)]^{\frac{1}{2}}$; the constraint tolerance is too small.
IER=-113	(ISTATC(i) < 0) or (ISTATC(i) > 4); invalid input for constraint status.
IER=-117	(CUPR(i) < CLWR(i)); constraint upper bound is less than lower bound.
IER=-118	Either (CUPR(i) = CLWR(i)) and (ISTATC(i) $\neq 3$) or (CUPR(i) $\neq$ CLWR(i)) and (ISTATC(i) = 3); constraint status array is not consistent with bounds.
IER=-119	(CUPR(i) $\neq$ CLWR(i)) and (CUPR(i)-CLWR(i) < CONTOL); constraint bounds are not equal, but differ by less than the constraint tolerance.
IER=-120	(ISTATV(i) < 0) or (ISTATV(i) > 3); invalid input for variable status array.

IER=-124	(XUPR(i) < XLWR(i)); the variable upper bound is less than the lower bound.
IER=-125	Either (XUPR(i) = XLWR(i)) and (ISTATV(i) ≠ 3) or (XUPR(i) ≠ XLWR(i)) and (ISTATV(i) = 3); the variable status array is not consistent with the bound values.
IER=-126	(XUPR(i) ≠ XLWR(i)) and (XUPR(i)-XLWR(i) < CONTOL); variable bounds are not equal but differ by less than the constraint tolerance.
IER=-127	Real hold array too small; insufficient storage detected in SNLPMN interface. The required storage is specified in NEEDED.
IER=-128	Integer hold array too small; insufficient storage detected in SNLPMN interface. The required storage is specified in NEEDED.
IER=-129	Function error at initial point or during gradient evaluation.
IER=-130	(The number of active constraints > NDIM); check ISTATC(i) and ISTATV(i).
IER=-133	Rank deficient Jacobian detected on successive iterations.
IER=-134	INSNLP input error; invalid character string displayed.
IER=-135	(IFERR < 0) or (IFERR > 1); invalid value for function error flag.
IER=-136	(SFZTOL ≥ 1) or (SFZTOL ≤ 10 ⁻⁵); the slope tolerance for the line search during the feasibility phase is too large or too small.
IER=-137	Conflict between user and multifrontal file number; check IPUMF1,...,IPUMF6 IPUDRF, IPUFZF, IPUSTF.
IER=-138	(NEWTON < 0) or (NEWTON > 1); invalid input for Newton method option flag.
IER=-147	Unexpected error; check storage allocation.
IER=-149	Incorrect value for QPOPTN.
IER=-150	QPOPTN = DENSE and either ALGOPT = 'FME' or NRES>0.
IER=-152	Incorrect value for KTOPTN.
IER=-153	I/O error, probably caused by insufficient disk space.
IER=-703	EPSRLF < [HDMCON(5)] ^{1/2} .

IER=+101	Weak solution found (multipliers near zero).
IER=+102	Number of equality constraints = NDIM and ALGOPT $\neq$ 'F'.
IER=+103	Maximum number of consecutive function errors.
IER=+104	Maximum number of function evaluations.
IER=+105	Small step termination in optimization phase; suboptimal feasible point found.
IER=+106	Maximum number of iterations in optimization phase.
IER=+107	Maximum number of iterations in feasibility phase.
IER=+108	Feasible point not found.
IER=+109	Maximum number of interval halves in line search.
IER=+110	QP algorithm terminated because either the Hessian diagonal reached its maximum value or the merit function slope condition was violated; suboptimal feasible point found.
IER=+111	Projected gradient calculation failed; constraints may be degenerate.
IER=+112	Calculation of first order multiplier estimates failed; constraints may be degenerate.
IER=+113	Suboptimal feasible point found.
IER=+114	Schur-complement QP algorithm (subroutine SHURQP) failed with unexpected error.
IER=+115	CONTOL > OBJTOL; convergence tolerances may be inappropriate.
IER=+116	Uphill direction detected in line search.
IER=+117	Reduced objective function is linear.
IER=+118	Both $c_{Li} \leq -.01/\text{HDMCON}(5)$ and $c_{Ui} \geq .01/\text{HDMCON}(5)$; constraints ignored.
IER=+119	Terminate after diagnostic line search.
IER=+703	Sparse parameter defaults may cause poor algorithm performance.

USER-SUPPLIED SUBPROGRAM

The user must provide a real subroutine, FNGBOX to compute the objective and constraint functions as well as first derivatives. Subroutine FNGBOX must have the following format:

```

SUBROUTINE FNGBOX(XBAR,NDIM,MCON,QUANT,DQUANT,IFERR)
C
DOUBLE PRECISION XBAR,QUANT,DQUANT
INTEGER NDIM,MCON,IFERR
DIMENSION XBAR(NDIM),QUANT(MCON+1),DQUANT(MCON+1,NDIM)
C
C      IF POSSIBLE EVALUATE OBJECTIVE AND CONSTRAINTS, OTHERWISE
C      SET FUNCTION ERROR FLAG.
C
IF( Functions can be evaluated at XBAR) THEN
C
C      SET FUNCTION ERROR FLAG
C
IFERR = 0
C
C      CONSTRAINTS
C
QUANT(1) = User supplied function evaluation process
.
.
.
QUANT(MCON) = User supplied function evaluation process
C
C      OBJECTIVE FUNCTION
C
QUANT(MCON+1) = User supplied function evaluation process
C
C      JACOBIAN MATRIX
C
DQUANT(1,1) = User supplied gradient
C
DQUANT(2,1) = User supplied gradient
.
.
.
C
DQUANT(MCON+1,1) = User supplied gradient
.
.
.
```

```

C      DQUANT(1,NDIM) = User supplied gradient
C
C      DQUANT(2,NDIM) = User supplied gradient
C      .
C      .
C      .
C
C      DQUANT(MCON+1,NDIM) = User supplied gradient
C
C
C      ELSE
C
C      SET FUNCTION ERROR FLAG
C
C      IFERR = 1
C
C      END IF
C
C      RETURN
C      END

```

The arguments of subroutine FNGBOX are defined as follows:

ARGUMENTS

XBAR	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the current variable values x .
NDIM	[INPUT, INTEGER] Number of variables n , where $NDIM \geq 1$.
MCON	[INPUT, INTEGER] Number of constraints m , where $MCON \geq 0$.
QUANT	[OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON+1 containing the values of the computed functions at the current value of x . The first MCON elements of QUANT contain the constraints $c(x)$, and the objective function $f(x)$ is stored in location MCON+1. When the function error flag IFERR = 1, the QUANT array will not be referenced.
DQUANT	[OUTPUT, DOUBLE PRECISION, ARRAY] A real two-dimensional array with MCON+1 rows, and NDIM columns. Element

$$DQUANT(i,j) = \frac{dq_i}{dx_j}$$

for $i = 1, \dots, (m+1)$, and $j = 1, \dots, n$ evaluated at the current value of x . The first MCON rows of DQUANT contain the constraint Jacobian matrix $G(x)$, and the

objective function gradient $\nabla f(x)$ is stored in row MCON+1. When the function error flag IFERR = 1, the DQUANT array will not be referenced.

IFERR [OUTPUT, INTEGER]

Function error flag. Set IFERR = 0 when functions are successfully computed. Set IFERR = 1 when functions cannot be computed.

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine DNLPGF. All optional inputs are specified by calls to subroutine INSNLP. It should be emphasized that it is *not necessary* to call INSNLP if the default values are appropriate. If optional inputs are specified, it is recommended that the user first set the defaults for a dense problem, that is CALL INSNLP('DENSE DEFAULT'), and then specify the desired input parameter.

OPTIONAL PRINTOUT

The level of printed output from DNLPGF is controlled by the variables IOFLAG and IPUNLP which are set by calls to INSNLP. The standard level of output which corresponds to IOFLAG = 10 is the *default* for DNLPGF. A complete description of the iteration output options is found in Section 2.3.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2)$ to minimize the objective function

$$f(x) = x_1^2 + x_2^2 + \log(x_1 x_2)$$

subject to the constraint

$$x_1 x_2 \geq 1$$

and the bounds

$$0 \leq x_k \leq 10$$

for $k = 1, 2$ beginning at the initial point $x = (.5, 2)$. The solution is at $x^* = (1, 1)$.

SAMPLE PROGRAM (dnlpgf.cd1)²

```
PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE NONLINEAR
C      PROGRAMMING ALGORITHM DNLPGF
C
C-----
C
```

²sosexample.tar

```

      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
      PARAMETER ( ZERO=0.D0, ONE=1.D0, POINT5 = .5D0, TWO = 2.D0)
      PARAMETER (MAXDIM = 5, MAXCON = 10)
C
C      NLP CALLING ARGUMENTS
C
      DIMENSION XBAR(MAXDIM),XLWR(MAXDIM),XUPR(MAXDIM),
$      ISTATV(MAXDIM),VECNU(MAXDIM),
$      CBAR(MAXCON),CLWR(MAXCON),
$      CUPR(MAXCON),ISTATC(MAXCON),VECLAM(MAXCON)
      EXTERNAL FNGBOX
C
C      DEFINE INDEPENDENT VARIABLE DATA
C
      NDIM = 2
      XBAR(1) = POINT5
      XBAR(2) = TWO
      xlwr(1:ndim) = zero
      xupr(1:ndim) = 10.d0
      istatv(1:ndim) = 0
C
C      DEFINE CONSTRAINT DATA
C
      BIGBND = 1.D0/HDMCON(5)
      MCON = 1
      CLWR(1) = ONE
      CUPR(1) = BIGBND
      ISTATC(1) = 0
C
C -----
C
C      CALL NLP TO SOLVE THE PROBLEM
C
      CALL DNLPGF(FNGBOX, XBAR, XLWR, XUPR, ISTATV, VECNU, NDIM,
$      FBAR, CBAR, CLWR, CUPR, ISTATC, VECLAM, MCON, IERNLP)
C
      STOP
      END
C
      SUBROUTINE FNGBOX(XBAR,NDIM,MCON,QUANT,DQUANT,IFERR)
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
      SUBROUTINE TO EVALUATE THE PROBLEM FUNCTIONS
C
      DIMENSION XBAR(NDIM),QUANT(MCON+1),DQUANT(MCON+1,NDIM)
C
C      NONLINEAR CONSTRAINTS

```

```
C
      QUANT(1) = XBAR(1)*XBAR(2)
C
      CHECK THAT ARGUMENT OF LOGARITHM IS POSITIVE
C      BEFORE ATTEMPTING TO EVALUATE OBJECTIVE
C
      IF(QUANT(1).LT.HDMCON(5)) THEN
C
C      FUNCTION ERROR
C
      IFERR = 1
      RETURN
C
      ELSE
C
      IFERR = 0
C
      OBJECTIVE FUNCTION
C
      QUANT(MCON+1) = XBAR(1)**2 + XBAR(2)**2
      $      + LOG(QUANT(1))
C
      ENDIF
C
      COMPUTE GRADIENT INFORMATION
C
      DQUANT(1,1) = XBAR(2)
      DQUANT(1,2) = XBAR(1)
C
      DQUANT(MCON+1,1) = 2.DO*XBAR(1) + DQUANT(1,1)/QUANT(1)
      DQUANT(MCON+1,2) = 2.DO*XBAR(2) + DQUANT(1,2)/QUANT(1)
C
      RETURN
      END
```

OUTPUT FROM SAMPLE PROGRAM

```
***** INITIAL POINT *****
*****
*
* Objective Function =      4.25000      IERNLP =      0
*
* -----
*
* Variable   Status   Value      Lower bound      Upper bound      Lagrange Mlt.      Slack
*
*      1      FR      0.500000      0.00000      10.0000      0.00000      0.5000
*      2      FR      2.00000      0.00000      10.0000      0.00000      2.000
*
* -----
*
```

```

*
* Constraint Status Value Lower bound Upper bound Lagrange Mlt. Slack
*
* 1 FR 1.00000 1.00000 4.503600E+15 0.00000 0.000
*
*****
*****
*
* .....OPTIMIZATION OPERATOR SPRNLP.....
*
*****
*
* -----
* Constraints = 1 | Variables = 2
* Equalities = 0 | Equalities = 0
* Inequalities = 1 | Bounds = 2
* Inactive = 1 | Free = 2
* Fixed on Lower Bound = 0 | Fixed on Lower Bound = 0
* Fixed on Upper Bound = 0 | Fixed on Upper Bound = 0
* -----
* Number of Active Constraints = 0 | Number of Degrees of Freedom = 2
* -----
*
* ALGORITHM CONTROL PARAMETERS (DENSE DEFAULTS)
*
* CONTOL 1.490E-08 Constraint Tolerance
* OBJTOL 1.000E-07 Objective Function Tolerance
* PGDTOL 1.000E-05 Projected Gradient Tolerance
* SLPTOL 0.900 Slope Tolerance For SQP Line Search
* IOFLAG 10 Output Level
* MAXNFE 10000 Maximum Number of Function Evaluations
* NITMAX 100 Maximum Number of Iterations
* ALGOPT FM Algorithm Control Option
*
*****
*
* CONSTRAINED OPTIMIZATION
*
* Number of Active Constraints: 2
* Number of Degrees of Freedom: 0
*
* Constraint Multipliers
*
* Smallest 1 Largest 1
*
* ( 1: 1.7500000 ) ( 1: 1.7500000 )
*
* Constraint Multiplier Magnitudes
*
* Smallest 1 Largest 1
*
* ( 1: 1.7500000 ) ( 1: 1.7500000 )
*
* Bound Multipliers
*
* Smallest 2 Largest 2
*
* ( 1: 0.0000000 ) ( 2: 1.6250000 )

```

```

*      (      2:   1.6250000   )      (      1:   0.0000000   )
*
*      Bound Multiplier Magnitudes
*
*      Smallest  1                      Largest  1
*
*      (      2:   1.6250000   )      (      2:   1.6250000   )
*
* -----
*
*      ----- Iteration 1 -----
*      Projected Gradient.....+2.00000   |   Objective Function.....+4.25000
*      Equality Error.....+0.00000   |   Inequality Error.....+0.00000
*      Merit Function.....+4.25000   |   Penalty Weight Norm.....+1.490116E-08
*      Levenberg Parameter.....+0.00000   |   Cond(Z'HZ).....+1.00000
*      Min. Eigenvalue.....-3.74976   |   Max. Eigenvalue.....+3.50001
*      Active Constraints.....1   |   Degrees of Freedom.....1
*      QP Iterations.....2   |   Matrix Factorizations.....1
*
*      Quadratic Line Search .....Step:  1.....ALFA =   1.0000000
*
*      OBJECTIVE =      0.188978          NFEVAL =      2
*
*      VARIABLES
*
*      0.874997          0.500014
*
*      CONSTRAINTS
*
*      0.437510
*
*
*      .....Index =      1.5600673          Change =   -2.6899327
*
*      ----- Iteration 2 -----
*      Projected Gradient.....+1.67405   |   Objective Function.....+0.188978
*      Equality Error.....+0.00000   |   Inequality Error.....+0.562490
*      Merit Function.....+2.88233   |   Penalty Weight Norm.....+8.35829
*      Levenberg Parameter.....+1.00000   |   Cond(Z'HZ).....+1.00000
*      Min. Eigenvalue.....-4.43707   |   Max. Eigenvalue.....+3.13146
*      Active Constraints.....1   |   Degrees of Freedom.....1
*      QP Iterations.....19   |   Matrix Factorizations.....1
*
*      Quadratic Line Search .....Step:  1.....ALFA =   1.0000000
*
*      OBJECTIVE =      2.26706          NFEVAL =      3
*
*      VARIABLES
*
*      1.03947          1.04887
*
*      CONSTRAINTS
*
*      1.09027
*
*
*      .....Index =      1.8383222          Change =   -1.0440038

```



```

*
* ----- Iteration 3 -----
* Projected Gradient.....+2.33621 | Objective Function.....+2.26706
* Equality Error.....+0.00000 | Inequality Error.....+0.00000
* Merit Function.....+2.00370 | Penalty Weight Norm.....+48.9457
* Levenberg Parameter.....+0.800429 | Cond(Z'HZ).....+1.00000
* Min. Eigenvalue.....-4.05209 | Max. Eigenvalue.....+6.21769
* Active Constraints.....1 | Degrees of Freedom.....1
* QP Iterations.....3 | Matrix Factorizations.....2
*
* Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
* OBJECTIVE = 2.00563 NFEVAL = 4
*
* VARIABLES
*
* 0.998035 1.00384
*
* CONSTRAINTS
*
* 1.00187
*
* .....Index = 2.0002717 Change = -3.43040747E-03
*
* ----- Iteration 4 -----
* Projected Gradient.....+9.239882E-02 | Objective Function.....+2.00563
* Equality Error.....+0.00000 | Inequality Error.....+0.00000
* Merit Function.....+2.00019 | Penalty Weight Norm.....+1.490116E-08
* Levenberg Parameter.....+0.655683 | Cond(Z'HZ).....+1.00000
* Min. Eigenvalue.....-1.92109 | Max. Eigenvalue.....+3.92486
* Active Constraints.....1 | Degrees of Freedom.....1
* QP Iterations.....3 | Matrix Factorizations.....2
*
* Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
* OBJECTIVE = 1.99999 NFEVAL = 5
*
* VARIABLES
*
* 0.999086 1.00091
*
* CONSTRAINTS
*
* 0.999997
*
* .....Index = 2.0000033 Change = -1.83146488E-04
*
* ----- Iteration 5 -----
* Projected Gradient.....+5.528521E-03 | Objective Function.....+1.99999
* Equality Error.....+0.00000 | Inequality Error.....+3.074314E-06
* Merit Function.....+2.00000 | Penalty Weight Norm.....+1.490116E-08
* Levenberg Parameter.....+0.655683 | Cond(Z'HZ).....+1.00000
* Min. Eigenvalue.....-1.99992 | Max. Eigenvalue.....+3.99999
* Active Constraints.....1 | Degrees of Freedom.....1
* QP Iterations.....3 | Matrix Factorizations.....1

```

```

*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      OBJECTIVE =      2.00000      NFEVAL =      6
*
*      VARIABLES
*
0.999699      1.00030
*
*      CONSTRAINTS
*
1.00000
*
*
*      .....Index = 2.0000004      Change = -2.96838191E-06
*
*      ----- Iteration 6 -----
*      Projected Gradient.....+1.205014E-03 | Objective Function.....+2.00000
*      Equality Error.....+0.00000 | Inequality Error.....+3.750348E-07
*      Merit Function.....+2.00000 | Penalty Weight Norm.....+1.490116E-08
*      Levenberg Parameter.....+0.142915 | Cond(Z'HZ).....+1.00000
*      Min. Eigenvalue.....-2.00056 | Max. Eigenvalue.....+4.00064
*      Active Constraints.....1 | Degrees of Freedom.....1
*      QP Iterations.....3 | Matrix Factorizations.....1
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      OBJECTIVE =      2.00000      NFEVAL =      7
*
*      VARIABLES
*
0.999971      1.00003
*
*      CONSTRAINTS
*
1.00000
*
*
*      .....Index = 2.0000000      Change = -3.58686401E-07
*
*      ----- Iteration 7 -----
*      Projected Gradient.....+1.165651E-04 | Objective Function.....+2.00000
*      Equality Error.....+0.00000 | Inequality Error.....+7.383808E-08
*      Merit Function.....+2.00000 | Penalty Weight Norm.....+1.490116E-08
*      Levenberg Parameter.....+1.382463E-02 | Cond(Z'HZ).....+1.00000
*      Min. Eigenvalue.....-2.00002 | Max. Eigenvalue.....+4.00010
*      Active Constraints.....1 | Degrees of Freedom.....1
*      QP Iterations.....3 | Matrix Factorizations.....1
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      OBJECTIVE =      2.00000      NFEVAL =      8
*
*      VARIABLES
*
1.00000      1.00000

```

CONSTRAINTS

1.00000

```

*
* .....Index =      2.0000000      Change = -3.39484130E-09
*
* ----- Iteration (8) -----
* Projected Gradient.....+1.197741E-06 | Objective Function.....+2.00000
* Equality Error.....+0.00000 | Inequality Error.....+8.314502E-10
* Merit Function.....+2.00000 | Penalty Weight Norm.....+1.490116E-08
* Levenberg Parameter.....+1.382463E-02 | Cond(Z'HZ).....+1.00000
* Min. Eigenvalue.....-2.00002 | Max. Eigenvalue.....+4.00010
* Active Constraints.....1 | Degrees of Freedom.....1
* QP Iterations.....0 | Matrix Factorizations.....0
*
* CONSTRAINED OPTIMIZATION COMPLETED
*
*****
*
* CONVERGENCE TESTS
*
* -----
* Constraints = 1 | Variables = 2
* Equalities = 0 | Equalities = 0
* Inequalities = 1 | Bounds = 2
* Inactive = 0 | Free = 2
* Fixed on Lower Bound = 1 | Fixed on Lower Bound = 0
* Fixed on Upper Bound = 0 | Fixed on Upper Bound = 0
*
* -----
* Number of Active Constraints = 1 | Number of Degrees of Freedom = 1
*
* -----
*
* Predicted Optimum Objective Function = 2.0000000
*
* Gradient Norm = 3.0000003
*
* Projected Gradient Vector
*
* Smallest 2 Largest 2
*
* ( 1: -1.19774139E-06) ( 2: 1.19640850E-06)
* ( 2: 1.19640850E-06) ( 1: -1.19774139E-06)
*
* Constraint Multipliers
*
* Smallest 1 Largest 1
*
* ( 1: 3.0000000 ) ( 1: 3.0000000 )
*
* Constraint Multiplier Magnitudes
*
* Smallest 1 Largest 1
*
* ( 1: 3.0000000 ) ( 1: 3.0000000 )
*
* Bound Multipliers
*

```

```

*      Smallest  2              Largest  2
*
*      (      1:  0.0000000   )   (      2:  0.0000000   )
*      (      2:  0.0000000   )   (      1:  0.0000000   )
*
*****
***** CONVERGENCE *****
*****
|
|              SPRNLP ALGORITHM PERFORMANCE STATISTICS
|
| Total CPU Time.....+0.00000
|
| Number of Function Calls.....8
| Number of Gradient Calls.....8
| Number of Hessian Calls.....7
| Total Number of Function Evaluations.....8
|
| Storage Needed in HOLD Array.....130
| Storage Needed in IHOLD Array.....35
|
*****
***** SOLUTION *****
*****
*
* Objective Function =      2.00000          IERNLP =      0
*
* -----
*
* Variable   Status   Value      Lower bound      Upper bound      Lagrange Mlt.      Slack
*
*      1      FR      1.00000      0.00000      10.0000      0.00000      1.000
*      2      FR      1.00000      0.00000      10.0000      0.00000      1.000
*
* -----
*
* Constraint  Status   Value      Lower bound      Upper bound      Lagrange Mlt.      Slack
*
*      1      LB      1.00000      1.00000      4.503600E+15      3.00000      -8.3145E-10
*
*****

```

DNLPRV: Dense Nonlinear Programming–Reverse Communication Format

PURPOSE

DNLPRV computes the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the objective function

$$f(x)$$

subject to the m nonlinear constraints

$$c_L \leq c(x) \leq c_U$$

and the simple bounds

$$x_L \leq x \leq x_U$$

Equality constraints are imposed by setting $c_{Li} = c_{Ui}$ and variables can be fixed by setting $x_{Li} = x_{Ui}$. DNLPRV works under the assumption that the objective and constraint functions are twice continuously differentiable, and if this is not true algorithm performance is unpredictable. DNLPRV uses a reverse communication format and upon request the user must supply the values of the functions, and their first derivatives. The matrix of second derivatives (Hessian of the Lagrangian) can be constructed using a quasi-Newton recursive estimate or supplied by the user. Subroutine DNLPRV calls the sparse nonlinear programming algorithm SNLPMN, and is intended for simplified usage as appropriate for small dense problems.

RELATED SUBPROGRAMS

DBNLPR	Dense Barrier Nonlinear Programming–Reverse Communication Format
SNLPMN	Sparse Nonlinear Programming
INSNLP	Sparse Nonlinear Programming Input Procedure

METHOD

A *sequential quadratic programming* (SQP) approach is used to solve the *nonlinear programming* (NLP) problem. Although the dense null-space method is the default, the Schur-complement method can be selected to solve the quadratic programming subproblem. DNLPRV provides four alternatives for constructing an approximation to the Hessian matrix. The default is a *symmetric rank-one* (SR1) approximation, with options for using a BFGS, or PSB. As a final option the Hessian of the Lagrangian may be supplied by the user. If the user cannot provide this information analytically, the finite difference techniques described in Chapter 3 should be utilized.

WARNING

The objective function and constraints supplied by the user are assumed to be continuous and have continuous first and second derivatives. Nondifferentiable functions can cause unpredictable performance in the optimization algorithm. It should be emphasized that discontinuities in the Hessian matrices can significantly degrade the speed of convergence without producing any other obvious difficulties. The following common sources of error *should be avoided* when evaluating the objective and constraint functions:

- (1) nonsmooth interpolation of data;
- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;
- (3) discontinuous behavior caused by branching for IF tests;
- (4) nondifferentiable functions such as ABS, MAX, and MIN.

The user must provide a procedure to evaluate the objective and constraint functions in addition to first and (optionally) second derivatives of these quantities. If it is not possible to evaluate the functions (for example, $c(x) = 1/x$ at $x = 0$) then set IFERR = 1 and exit the function evaluation process. Otherwise, set IFERR = 0 when the functions are evaluated. The function error flag should be used to avoid computational difficulties, but never in lieu of an explicit constraint.

The DNLPRV software uses internal common blocks to communicate information. To avoid conflict with these internal commons the user should not have commons with the following names: NPSPRR, NPSPRI, NPSPRC, STATIS, KONSTN, NPSALG, NPSDFL, NLPERR, PERCOM, ITEREF, and CLKCOM.

USAGE

```

INTEGER      IRVCOM, IFERR, NFEVAL,      IER, NEEDED
INTEGER      NDIM, MCON, NONZH, NONZG, NHOLD, NIHOLD
INTEGER      IREVRS(5), ISTATV(NDIM), ISTATC(MCON), IHOLD(NIHOLD)
PARAMETER    (NONZG=NDIM*MCON, NONZH=NDIM*(NDIM+1)/2 )

DOUBLE PRECISION      FBAR, XBAR(NDIM), XLWR(NDIM), XUPR(NDIM), VECNU(NDIM)
DOUBLE PRECISION      DELF(NDIM), HMAT(NONZH), CBAR(MCON), CLWR(MCON)
DOUBLE PRECISION      CUPR(MCON), VECLAM(MCON), GMAT(NONZG), HOLD(NHOLD)

CALL DNLPRV( IRVCOM, IREVRS, XBAR, XLWR, XUPR, ISTATV,
$           VECNU, NDIM, FBAR, DELF, HMAT, CBAR,
$           CLWR, CUPR, ISTATC, MCON, VECLAM, GMAT,
$           IFERR, NFEVAL, HOLD, NHOLD, IHOLD, NIHOLD,
$           NEEDED, IER )

```

ARGUMENTS

ARGUMENTS TO CONTROL REVERSE COMMUNICATION

IRVCOM [INPUT/OUTPUT, INTEGER]
Continuation control flag. This argument must be initialized. Thereafter it is set by the algorithm and must not be changed by the calling program.

IRVCOM = -1 Initialization pass.

IRVCOM = 0 Termination (normal or abnormal).

IRVCOM = +1 Perform an evaluation and then reenter DNLPRV. Operations to be performed are defined by the output variables IREVRS(1), IREVRS(2), IREVRS(3), and IREVRS(4) below. IREVRS(5) contains information which may be valuable to the interface but does not require any action on the part of the user.

IREVRS [OUTPUT, INTEGER, ARRAY]
 An integer array of length 5, containing reverse communication control information.

IREVRS(1) Function evaluation request
 = 0 Function evaluation not requested; *Warning: do not change the current function and constraint values.*
 = 1 Function evaluation requested; evaluate objective function and all constraints at current point XBAR.

IREVRS(2) Gradient evaluation request
 = 0 Gradient evaluation not requested; *Warning: do not change the current gradient and Jacobian.*
 = 1 Approximate (e.g. forward difference) gradient requested; evaluate gradient vector g and Jacobian matrix G at current point XBAR. (If available, accurate or analytic gradients may be supplied)
 = 2 Accurate (e.g. central difference or analytic) gradient requested; evaluate gradient vector g and Jacobian matrix G at current point XBAR.

IREVRS(3)	Hessian evaluation request	
	= 0	Hessian evaluation not requested; <i>Warning: do not change the current Hessian.</i>
	= 1	Hessian diagonal requested; get ready for full Hessian evaluation with (IREVRS(3)=2) at current point (XBAR,VECLAM) and optionally evaluate diagonal elements of Hessian matrix H .
	= 2	Full Hessian evaluation requested; evaluate Hessian matrix H of the function $L(x, \lambda) = f - \sum_{k=1}^m \lambda_k c_k$ at current point (XBAR,VECLAM). A full evaluation request is always preceded by a call with IREVRS(3)=1 at the same point XBAR, and consequently it may not be necessary to recompute the diagonal elements.
IREVRS(4)	System print (output) request	
	= 0	System print not requested.
	= 1	System print requested; perform any output operations (print, write files, etc.) desired to display problem specific information at the current point XBAR. The frequency of print is defined by the output control flag IOFLAG (see Optional Printout).
IREVRS(5)	Algorithm information flag	
	= 0	Call was made from the interface portion of the algorithm.
	= 1	Call was made from the optimization portion of the algorithm.
	= 2	Call was made from the optimization portion of the algorithm requesting Hessian reset.
	= 3	Call was made from the optimization portion of the algorithm requesting optimal perturbation size adjustment.
	= 4	Call was made from the feasibility portion of the algorithm.

INDEPENDENT VARIABLE DATA

XBAR[†] [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
 A real array of length NDIM of current variable values x .

IRVCOM=-1 Initial guess must be input.

IRVCOM=0 Final point is output. When IER = 0, +101, the final point corresponds to a local optimum. When IER = +104, ..., +111, +114, +116, +117, +119, the final point is either the best feasible point if one has been found, or the last iterate.

IRVCOM=1 Intermediate iteration points are output.

XLWR[†] [INPUT, DOUBLE PRECISION, ARRAY]
 A real array of length NDIM containing the variable lower bounds x_L . For variables that have no lower bound set $x_{Li} \leq -.01/\text{HDMCON}(5)$. See Appendix B for HDMCON.

XUPR[†] [INPUT, DOUBLE PRECISION, ARRAY]
 A real array of length NDIM containing the variable upper bounds x_U . For variables that have no upper bound set $x_{Ui} \geq .01/\text{HDMCON}(5)$.

ISTATV [INPUT/OUTPUT, INTEGER, ARRAY]
 An integer array of length NDIM indicating variable status. On input ISTATV defines an estimate of the active variable bounds, and on output ISTATV defines the bounds active at the solution.

ISTATV(i)= 0 Variable is strictly feasible, i.e. $x_{Li} < x_i < x_{Ui}$.

ISTATV(i)= 1 Variable lower bound is active, i.e. $x_{Li} = x_i$.

ISTATV(i)= 2 Variable upper bound is active, i.e. $x_i = x_{Ui}$.

ISTATV(i)= 3 Fixed permanently.

ISTATV(i)= 10 Output only: Violated variable lower bound, i.e. $x_{Li} - \delta_c > x_i$.

ISTATV(i)= 20 Output only: Violated variable upper bound, i.e. $x_i > x_{Ui} + \delta_c$.

ISTATV(i)= 30 Output only: Violated fixed variable. Either $x_i < x_{Li} - \delta_c$ or $x_i > x_{Ui} + \delta_c$.

When ISTATV(i) = 3 it is necessary that $x_{Li} = x_{Ui}$, where $x_{Li} = \text{XLWR}(i)$ and $x_{Ui} = \text{XUPR}(i)$. When ISTATV(i) $\neq$ 3, it is necessary that $x_{Li} < x_{Ui}$ with $|x_{Li} - x_{Ui}| > \delta_c$.

VECNU[#] [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
 An array of length NDIM of bound multipliers ν .

NDIM[†] [INPUT, INTEGER]
 Number of variables n , where $1 \leq \text{NDIM}$.

OBJECTIVE FUNCTION DATA

FBAR[‡] [INPUT, DOUBLE PRECISION]
 The objective function evaluated at XBAR, $f(\text{XBAR})$.

DELF[‡] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM containing the gradient g of FBAR at XBAR.

HMAT[‡] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length $n(n+1)/2$ containing the elements of Hessian matrix $H_L = \nabla_x^2 f - \sum_{i=1}^m \lambda_i \nabla_x^2 c_i$ of the Lagrangian function. Since the Hessian is symmetric, only the lower triangular part is input (see USAGE REMARKS).

CONSTRAINT DATA

CBAR[‡] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraints $c(x)$ evaluated at XBAR. Not referenced when MCON = 0.

CLWR[†] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraint lower bounds c_L . For constraints that have no lower bound set $c_{Li} \leq -.01/\text{HDMCON}(5)$. Not referenced when MCON = 0.

CUPR[†] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraint upper bounds c_U . For constraints that have no upper bound set $c_{Ui} \geq .01/\text{HDMCON}(5)$. Not referenced when MCON = 0.

ISTATC [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length MCON indicating the constraint status. On input ISTATC defines an estimate of the active constraints, and on output ISTATC defines the constraints active at the solution.

ISTATC(i)= 0 Free (inactive) inequality, i.e. $c_{Li} < c_i(x) < c_{Ui}$.

ISTATC(i)= 1 Constraint lower bound is active, i.e. $c_{Li} = c_i(x)$.

ISTATC(i)= 2 Constraint upper bound is active, i.e. $c_i(x) = c_{Ui}$.

ISTATC(i)= 3 Equality.

ISTATC(i)= 4 Ignored constraint.

ISTATC(i)= 10 Output only: Violated constraint lower bound, i.e. $c_{Li} - \delta_c > c_i$.

ISTATC(i)= 20 Output only: Violated constraint upper bound, i.e. $c_i > c_{Ui} + \delta_c$.

ISTATC(i)= 30 Output only: Violated equality constraint. Either $c_i < c_{Li} - \delta_c$ or $c_i > c_{Ui} + \delta_c$.

When ISTATC(i)=3 it is necessary that $c_{Li} = c_{Ui}$, where $c_{Li} = \text{CLWR}(i)$ and $c_{Ui} = \text{CUPR}(i)$. When ISTATC(i) $\neq$ 3, it is necessary that $c_{Li} < c_{Ui}$ with $|c_{Li} - c_{Ui}| > \delta_c$. Not referenced when MCON = 0.

MCON [†]	[INPUT, INTEGER] Number of constraints m , where $0 \leq \text{MCON}$.
VECLAM [#]	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON of Lagrange multipliers, λ , for constraints. The user must evaluate the Hessian of $L(x, \lambda) = f - \sum_{k=1}^m \lambda_k c_k$ (see IREVRS(3)). Not referenced when MCON = 0.
GMAT [‡]	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM*MCON containing the elements of the Jacobian matrix G at XBAR (see USAGE REMARKS). Not referenced when MCON = 0.

ALGORITHM CONTROL DATA

IFERR [‡]	[INPUT, INTEGER] Function evaluation error flag. IFERR= 1 When FBAR and/or CBAR cannot be evaluated. IFERR= 0 Otherwise.
---------------------------	---

NFEVAL [‡]	[INPUT, INTEGER] Number of function evaluations. Normally NFEVAL is initialized prior to the first call when IRVCOM = -1, and incremented whenever IREVRS(1), IREVRS(2), or IREVRS(3) are nonzero. It may be appropriate to increment NFEVAL for each perturbation when finite difference gradients are used, or not at all when analytic gradients are used.
----------------------------	--

WORKING STORAGE

HOLD	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length NHOLD containing the hold array (working storage that must <u>not</u> be destroyed by the user).
NHOLD [†]	[INPUT/OUTPUT, INTEGER] Dimension of HOLD array. When IER = -127, -131, or -701 NEEDED contains the required storage.
IHOLD	[INPUT/OUTPUT, INTEGER, ARRAY] An integer array of length NIHOLD containing the integer hold array (working storage that must <u>not</u> be destroyed by the user).
NIHOLD [†]	[INPUT/OUTPUT, INTEGER] Dimension of IHOLD array. When IER = -128, -132, or -702, NEEDED contains the required storage.

OUTPUT ARGUMENTS SET BY ALGORITHM BEFORE TERMINATION (IRVCOM = 0)

NEEDED	[OUTPUT, INTEGER] When IER = -127, -131, or -701, NEEDED contains the required length of HOLD. When IER = -128, -132, or -702, NEEDED contains the required length for IHOLD.
---------------	--

IER

[OUTPUT, INTEGER]

Success/error code. Results are not computed for $IER < 0$. For $IER > 0$, results are computed but may be suspect; user inspection is recommended. Note many of the error returns involve optional input quantities as described in INSNLP.

IER=0	Normal termination.
IER=-101	(MCON < 0); the number of constraints is negative.
IER=-102	(NDIM < 1); the number of variables is less than one.
IER=-104	IHESHN > 3 .
IER=-105	(NITMAX $< \max(1, \text{NITMIN})$); the maximum number of iterations is either less than 1 or less than the minimum number of iterations.
IER=-106	(SLPTOL ≥ 1) or (SLPTOL $\leq 10^{-5}$); the slope tolerance for the line search during optimization is too large or too small.
IER=-107	(IT1MAX < 1); the number of line search steps is less than 1.
IER=-108	(IOFLAG < 0) or (IOFLAG > 30); invalid input for the output control flag.
IER=-109	Invalid input for ALGOPT.
IER=-110	(OBJTOL $\leq 10 \cdot \text{HDMCON}(5)$); objective function tolerance is too small.
IER=-111	(PGDTOL $\leq [\text{HDMCON}(5)]^{\frac{1}{2}}$) or (PGDTOL $> 10^{-2}$); the projected gradient tolerance is too small or too large.
IER=-112	CONTOL $< [\text{HDMCON}(5)]^{\frac{1}{2}}$; the constraint tolerance is too small.
IER=-113	(ISTATC(i) < 0) or (ISTATC(i) > 4); invalid input for constraint status.
IER=-117	(CUPR(i) $< \text{CLWR}(i)$); constraint upper bound is less than lower bound.
IER=-118	Either (CUPR(i) = CLWR(i)) and (ISTATC(i) $\neq 3$) or (CUPR(i) $\neq \text{CLWR}(i)$) and (ISTATC(i) = 3); constraint status array is not consistent with bounds.
IER=-119	(CUPR(i) $\neq \text{CLWR}(i)$) and ($ \text{CUPR}(i) - \text{CLWR}(i) < \text{CONTOL}$); constraint bounds are not equal, but differ by less than the constraint tolerance.
IER=-120	(ISTATV(i) < 0) or (ISTATV(i) > 3); invalid input for variable status array.

IER=-124	(XUPR(i) < XLWR(i)); the variable upper bound is less than the lower bound.
IER=-125	Either (XUPR(i) = XLWR(i)) and (ISTATV(i) ≠ 3) or (XUPR(i) ≠ XLWR(i)) and (ISTATV(i) = 3); the variable status array is not consistent with the bound values.
IER=-126	(XUPR(i) ≠ XLWR(i)) and (XUPR(i)-XLWR(i) < CONTOL); variable bounds are not equal but differ by less than the constraint tolerance.
IER=-127	Real hold array too small; insufficient storage detected in SNLPMN interface. The required storage is specified in NEEDED.
IER=-128	Integer hold array too small; insufficient storage detected in SNLPMN interface. The required storage is specified in NEEDED.
IER=-129	Function error at initial point or during gradient evaluation.
IER=-130	(The number of active constraints > NDIM); check ISTATC(i) and ISTATV(i).
IER=-131	NHOLD too small; insufficient real storage detected in algorithm. The required storage is specified in NEEDED.
IER=-132	NIHOLD too small; insufficient integer storage detected in algorithm. The required storage is specified in NEEDED.
IER=-133	Rank deficient Jacobian detected on successive iterations.
IER=-134	INSNLP input error; invalid character string displayed.
IER=-135	(IFERR < 0) or (IFERR > 1); invalid value for function error flag.
IER=-136	(SFZTOL ≥ 1) or (SFZTOL ≤ 10 ⁻⁵); the slope tolerance for the line search during the feasibility phase is too large or too small.
IER=-137	Conflict between user and multifrontal file number; check IPUMF1,...,IPUMF6, IPUDRF, IPUFZF, IPUSTF.
IER=-138	(NEWTON < 0) or (NEWTON > 1); invalid input for Newton method option flag.
IER=-147	Unexpected error; check storage allocation and verify that the HOLD and IHOLD arrays are unchanged during reverse communication calls.
IER=-148	IRVCOM > 1.
IER=-149	Incorrect value for QPOPTN.

IER=-150	QPOPTN = DENSE and either ALGOPT = 'FME' or NRES>0.
IER=-152	Incorrect value for KTOPTN.
IER=-153	I/O error, probably caused by insufficient disk space.
IER=-701	Real hold array too small; insufficient storage detected in dense interface. The required storage is specified in NEEDED.
IER=-702	Integer hold array too small; insufficient storage detected in dense interface. The required storage is specified in NEEDED.

IER=+101	Weak solution found (multipliers near zero).
IER=+102	Number of equality constraints = NDIM and ALGOPT $\neq$ 'F'.
IER=+103	Maximum number of consecutive function errors.
IER=+104	Maximum number of function evaluations.
IER=+105	Small step termination in optimization phase; suboptimal feasible point found.
IER=+106	Maximum number of iterations in optimization phase.
IER=+107	Maximum number of iterations in feasibility phase.
IER=+108	Feasible point not found.
IER=+109	Maximum number of interval halves in line search.
IER=+110	QP algorithm terminated because either the Hessian diagonal reached its maximum value or the merit function slope condition was violated; suboptimal feasible point found.
IER=+111	Projected gradient calculation failed; constraints may be degenerate.
IER=+112	Calculation of first order multiplier estimates failed; constraints may be degenerate.
IER=+113	Suboptimal feasible point found.
IER=+114	Schur-complement QP algorithm (subroutine SHURQP) failed with unexpected error.

IER=+115	CONTROL > OBJTOL; convergence tolerances may be inappropriate.
IER=+116	Uphill direction detected in line search.
IER=+117	Reduced objective function is linear.
IER=+118	Both $c_{Li} \leq -.01/\text{HDMCON}(5)$ and $c_{Ui} \geq .01/\text{HDMCON}(5)$; constraints ignored.
IER=+119	Terminate after diagnostic line search.
IER=+120	(IHESHN > 0) and (NEWTON = 0); recursive Hessian updates may be skipped causing poor algorithm performance.
IER=+703	Sparse parameter defaults may cause poor algorithm performance.

†	Variable set when IRVCOM=-1, and not changed by user thereafter
‡	Variable set when IRVCOM=+1, as required by IREVRS(*)
#	Variable optionally set when IRVCOM=-1, and not changed by user thereafter

USAGE REMARKS

When IHESHN = 0 and IREVRS(3) > 0, the user must supply the Hessian matrix of the following Lagrangian function:

$$L(x, \lambda) = f - \sum_{k=1}^m \lambda_k c_k$$

that is,

$$L(\text{XBAR}, \text{VECLAM}) = \text{FBAR} - \text{VECLAM}(1) * \text{CBAR}(1) - \dots - \text{VECLAM}(\text{MCON}) * \text{CBAR}(\text{MCON}) .$$

Consider the following example in which $n = 3$:

$$H = \begin{pmatrix} 1.1 & 2.1 & 3.1 \\ 2.1 & 2.2 & 3.2 \\ 3.1 & 3.2 & 3.3 \end{pmatrix}$$

Since the Hessian is a symmetric matrix only the lower triangular portion must be specified. For this example NDIM = 3 and the values are specified as HMAT(*) = (1.1, 2.1, 3.1, 2.2, 3.2, 3.3). Notice that the columns of the lower triangular portion are stored sequentially and the elements in the upper triangular portion are not specified.

The Jacobian matrices input to the optimization software are reordered. Consequently if these quantities are to be reused by the calling program the user can insure they are returned in the original (unaltered) order by setting JACPRM = 1, as described in INSNLP (p. 226).

When the problem is unconstrained the user must set MCON = 0. The following constraint related quantities will not be referenced: CBAR, CLWR, CUPR, ISTATC, VECLAM, and GMAT. Since these quantities must appear as arguments in the call to SNLPMN, dummy values may be utilized.

A computational flow chart of a reverse communication optimization algorithm is found in the documentation of SNLPMN (p. 67).

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine DNLPRV. All optional inputs are specified by calls to subroutine INSNLP. It should be emphasized that it is *not necessary* to call INSNLP if the default values are appropriate.

OPTIONAL PRINTOUT

The level of printed output from DNLPRV is controlled by the variables IOFLAG and IPUNLP which are set by calls to INSNLP. The standard level of output which corresponds to IOFLAG = 10 is the *default* for DNLPRV. A complete description of the iteration output options is found in Section 2.3.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2, x_3, x_4)$ to minimize the objective function

$$f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2 + 100(x_4 - x_3^2)^2 + (1 - x_3)^2$$

subject to the two constraints

$$\begin{aligned} x_1 x_2 &\leq \frac{1}{2} \\ -\frac{1}{2} &\leq x_1 + x_3 \leq \frac{1}{2} \end{aligned}$$

and the bounds

$$-2 \leq x_k \leq 2$$

for $k = 1, 2, 3, 4$ beginning at the initial point $x = (1, 2, 1, 2)$. For this example the gradient vector is

$$g = \nabla_x f = \begin{bmatrix} -400x_1(x_2 - x_1^2) - 2(1 - x_1) \\ 200(x_2 - x_1^2) \\ -400x_3(x_4 - x_3^2) - 2(1 - x_3) \\ 200(x_4 - x_3^2) \end{bmatrix}$$

and the Jacobian matrix is

$$G = \begin{bmatrix} \nabla_x c_1^T \\ \nabla_x c_2^T \end{bmatrix} = \begin{bmatrix} x_2 & x_1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}.$$

When IHESHN = 0 and IREVR(3) > 0 it is also necessary to supply the Hessian matrix

$$\nabla_x^2 L = \nabla_x^2 f - \lambda_1 \nabla_x^2 c_1 - \lambda_2 \nabla_x^2 c_2$$

where

$$\nabla_x^2 f = \begin{bmatrix} 1200x_1^2 - 400x_2 + 2 & -400x_1 & 0 & 0 \\ -400x_1 & 200 & 0 & 0 \\ 0 & 0 & 1200x_3^2 - 400x_4 + 2 & -400x_3 \\ 0 & 0 & -400x_3 & 200 \end{bmatrix}$$

with

$$\nabla_x^2 c_1 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

and $\nabla_x^2 c_2 = 0$.

Note: This example also illustrates the use of the “Terse Output” option, by the call to INSNLP with “IOFLAG = 1”.

SAMPLE PROGRAM (dnlprv.cd1)³

```

      PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE NONLINEAR
C      PROGRAMMING ALGORITHM DNLPRV
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      PARAMETER (NDIM=4, MCON=2)
C      PARAMETER (NONZH=NDIM*(NDIM+1)/2, NONZG=NDIM*MCON)
C      PARAMETER (NHOLD=1500,NIHOLD=500)
C
C      DIMENSION IREVRS(5),XBAR(NDIM),XLWR(NDIM),XUPR(NDIM),
$      ISTATV(NDIM),VECNU(NDIM),DELF(NDIM),HMAT(NONZH),
$      CBAR(MCON),CLWR(MCON),
$      CUPR(MCON),ISTATC(MCON),VECLAM(MCON),
$      GMAT(NONZG)
C      DIMENSION HOLD(NHOLD),IHOLD(NIHOLD)
C
C      INITIALIZE REVERSE COMMUNICATION CONTROL FLAG
C
C      ICC = -1
C
C -----
C
C      INDEPENDENT VARIABLE DATA
C
C      ----INITIAL GUESS
C      XBAR(1) = 1.0D0
C      XBAR(2) = 2.0D0
C      XBAR(3) = 1.0D0
C      XBAR(4) = 2.0D0
C
C      ----BOUNDS
C      xlwr(1:ndim) = -2.d0
C      xupr(1:ndim) = 2.d0

```

³sosexample.tar

```
C      ----VARIABLE STATUS
C      istatv(1:ndim) = 0
C
C      -----
C
C      CONSTRAINT DATA
C
C      ----BOUNDS
C      CLWR(1) = -1.DO/HDMCON(5)
C      CUPR(1) = .5D0
C      CLWR(2) = -.5D0
C      CUPR(2) = .5D0
C      ----CONSTRAINT STATUS
C      istatc(1:mcon) = 0
C
C      -----
C
C      INITIALIZE FUNCTION EVALUATION COUNT
C
C      NFEVAL = 0
C
C      SET OPTIONAL INPUTS
C
C      CALL INSNLP('DENSE DEFAULT')
C      CALL INSNLP('IOFLAG=1')
C
C      -----
C
C      1000 CONTINUE
C
C      CALL DNLPRV( ICC ,IREVRS ,XBAR   ,XLWR   ,XUPR   ,ISTATV
C      $          ,VECNU  ,NDIM   ,FBAR   ,DELF   ,HMAT   ,CBAR
C      $          ,CLWR   ,CUPR   ,ISTATC ,MCON   ,VECLAM ,GMAT
C      $          ,IFERR  ,NFEVAL ,HOLD   ,NHOLD  ,IHOLD  ,NIHOLD
C      $          ,NEEDED ,IERNLP )
C
C      REVERSE COMMUNICATION TERMINATION TEST
C
C      IF(ICC.EQ.0) GO TO 10000
C
C      -----
C
C      SET FUNCTION EVALUATION ERROR FLAG
C
C      IFERR = 0
C
C      >>>>>FUNCTION EVALUATION REQUEST
C
```

```

      IF(IREVRS(1).EQ.1) THEN
C
      FBAR = 100.DO*(XBAR(2) - XBAR(1)**2)**2
$          + (1.DO - XBAR(1))**2
$          + 100.DO*(XBAR(4) - XBAR(3)**2)**2
$          + (1.DO - XBAR(3))**2
C
      CBAR(1) = XBAR(1)*XBAR(2)
      CBAR(2) = XBAR(1) + XBAR(3)
      NFEVAL = NFEVAL + 1
C
      ENDIF
C
      >>>>GRADIENT EVALUATION REQUEST
C
      IF(IREVRS(2).EQ.1) THEN
C
      DELF(1) = - 400.DO*XBAR(1)*(XBAR(2) - XBAR(1)**2)
$          - 2.DO*(1.DO - XBAR(1))
      DELF(2) = 200.DO*(XBAR(2) - XBAR(1)**2)
      DELF(3) = - 400.DO*XBAR(3)*(XBAR(4) - XBAR(3)**2)
$          - 2.DO*(1.DO - XBAR(3))
      DELF(4) = 200.DO*(XBAR(4) - XBAR(3)**2)
C
      GMAT(1) = XBAR(2)
      GMAT(2) = 1.DO
      GMAT(3) = XBAR(1)
      GMAT(4) = 0.DO
      GMAT(5) = 0.DO
      GMAT(6) = 1.DO
      GMAT(7) = 0.DO
      GMAT(8) = 0.DO
C
      ELSEIF(IREVRS(2).EQ.2) THEN
C
      DELF(1) = - 400.DO*XBAR(1)*(XBAR(2) - XBAR(1)**2)
$          - 2.DO*(1.DO - XBAR(1))
      DELF(2) = 200.DO*(XBAR(2) - XBAR(1)**2)
      DELF(3) = - 400.DO*XBAR(3)*(XBAR(4) - XBAR(3)**2)
$          - 2.DO*(1.DO - XBAR(3))
      DELF(4) = 200.DO*(XBAR(4) - XBAR(3)**2)
C
      GMAT(1) = XBAR(2)
      GMAT(2) = 1.DO
      GMAT(3) = XBAR(1)
      GMAT(4) = 0.DO
      GMAT(5) = 0.DO
      GMAT(6) = 1.DO

```

```
      GMAT(7) = 0.DO
      GMAT(8) = 0.DO
C
      ENDIF
C
C      >>>>HESSIAN EVALUATION REQUEST
C
C      (NOTE: THE HESSIAN EVALUATIONS BELOW ARE ONLY
C              NEEDED WHEN IHESHN=0)
C
      IF(IREVRS(3).EQ.1) THEN
C
C          DIAGONAL ELEMENTS
C
          HMAT(1) = 1200.DO*XBAR(1)**2
          $      - 400.DO*XBAR(2) + 2.DO
          HMAT(5) = 200.DO
          HMAT(8) = 1200.DO*XBAR(3)**2
          $      - 400.DO*XBAR(4) + 2.DO
          HMAT(10) = 200.DO
C
      ELSEIF(IREVRS(3).EQ.2) THEN
C
C          OFF-DIAGONAL ELEMENTS
C
          HMAT(2) = -VECLAM(1)*(1.DO) - 400.DO*XBAR(1)
          HMAT(3) = 0.DO
          HMAT(4) = 0.DO
          HMAT(6) = 0.DO
          HMAT(7) = 0.DO
          HMAT(9) = - 400.DO*XBAR(3)
C
      ENDIF
C
C      >>>>SYSTEM PRINT (OUTPUT) REQUEST
C
      IF(IREVRS(4).EQ.1) THEN
C
          WRITE(6,1001) 'OBJECTIVE  ',FBAR
          WRITE(6,1001) 'VARIABLES  ',(XBAR(II),II=1,NDIM)
          WRITE(6,1001) 'CONSTRAINTS',(CBAR(II),II=1,MCON)
C
      ENDIF
C
C -----
C
C      END OF REVERSE COMMUNICATION PROCESSING.  RETURN TO DNLPRV
C
```

```

      GO TO 1000
C
10000 CONTINUE
C
      TERMINATE PROCESSING
C
      IF(IERNLP.EQ.0) THEN
C
      NORMAL TERMINATION---SOLUTION FOUND
C
      WRITE(6,1001) 'SOLUTION    '
C
      WRITE(6,1001) 'OBJECTIVE    ',FBAR
      WRITE(6,1001) 'VARIABLES    ',(XBAR(II),II=1,NDIM)
      WRITE(6,1001) 'CONSTRAINTS',(CBAR(II),II=1,MCON)
C
      ELSEIF(IERNLP.LT.0) THEN
C
      ABNORMAL TERMINATION---FATAL ERROR
C
      PRINT *, 'FATAL ERROR; IERNLP =', IERNLP
C
      ELSE
C
      ABNORMAL TERMINATION---WARNING.  EVALUATION OF RESULTS RECOMMENDED!
C
      PRINT *, 'WARNING MESSAGE; IERNLP =', IERNLP
C
      ENDIF
C
1001 FORMAT(5X,A11,5X,4G17.8)
C
      STOP
      END

```

OUTPUT FROM SAMPLE PROGRAM

Feasible Point							
It	Qit	Nkt	Ndof	Cond(G)	Step	Norm p	Violtn
1	3	1	3	1.0E+00	1.0E+00	1.1E+00	2.1E+00
2	0	0	3	1.0E+00	1.0E+00	1.1E+00	0.0E+00

Optimization											
It	Qit	Nkt	Ndof	PH Cond	Step	Norm p	Violtn	Levnbrg	Penalty	Norm Pg	Merit Function
1	1	1	1	1.0E+00	1.0E+00	5.7E+00	4.4E-16	0.0E+00	1.5E-08	4.0E+00	-2.316094E+03
2	4	1	3	2.0E+02	5.0E-03	2.7E+00	0.0E+00	3.0E-08	2.2E+02	2.0E+00	1.159803E+03
(3)	2	1	3	2.0E+02	1.0E+00	4.0E-08	0.0E+00	3.0E-08	1.5E-08	5.7E-06	1.125000E+00
(4)	0	0	3	2.0E+02	1.0E+00	4.0E-08	0.0E+00	3.0E-08	1.5E-08	3.1E-08	1.125000E+00

SPRNLP ALGORITHM PERFORMANCE STATISTICS				
Total CPU Time.....+5.753541E-03				
Number of Function Calls.....7				
Number of Gradient Calls.....6				
Number of Hessian Calls.....3				
Total Number of Function Evaluations.....7				
Storage Needed in HOLD Array.....1457				
Storage Needed in IHOLD Array.....299				

SOLUTION				
OBJECTIVE	1.1250000			
VARIABLES	0.25000000	0.62500000E-01	0.25000000	0.62500000E-01
CONSTRAINTS	0.15625000E-01	0.50000000		

SNLPMN: Sparse Nonlinear Programming

PURPOSE

SNLPMN computes the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the objective function

$$f(x)$$

subject to the m nonlinear constraints

$$c_L \leq c(x) \leq c_U$$

and the simple bounds

$$x_L \leq x \leq x_U.$$

Equality constraints are imposed by setting $c_{Li} = c_{Ui}$ and variables can be fixed by setting $x_{Li} = x_{Ui}$. SNLPMN works under the assumption that the objective and constraint functions are twice continuously differentiable, and if this is not true algorithm performance is unpredictable. SNLPMN uses a reverse communication format and upon request the user must supply the values of the functions, and their first and second derivatives. The matrix of first derivatives (the Jacobian) and the matrix of second derivatives (Hessian of the Lagrangian) are represented in a sparse format consistent with that used in BCSLIB-EXT [1].

RELATED SUBPROGRAMS

SBRNLP Sparse Barrier Nonlinear Programming

INSNLP Sparse Nonlinear Programming Input Procedure

METHOD

A *sequential quadratic programming* (SQP) approach is used to solve the *nonlinear programming* (NLP) problem. The quadratic programming algorithm requires an estimate for the Hessian matrix H_L . If the user cannot provide this information analytically, the sparse finite difference techniques described in Chapter 3 should be utilized. For sparse applications the quadratic programming subproblem is efficiently solved using the sparse Schur-Complement Method. If problem sparsity is not exploited the QP subproblem can be solved efficiently using a dense null-space algorithm.

WARNING

The objective function and constraints supplied by the user are assumed to be continuous and have continuous first and second derivatives. Non-differentiable functions can cause unpredictable performance in the optimization algorithm. It should be emphasized that discontinuities in the Hessian matrices can significantly degrade the speed of convergence without producing any other obvious difficulties. The following common sources of error *should be avoided* when evaluating the objective and constraint functions:

- (1) non-smooth interpolation of data;
- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;

- (3) discontinuous behavior caused by branching for IF tests;
- (4) non-differentiable functions such as ABS, MAX, and MIN.

The user must provide a procedure to evaluate the objective and constraint functions in addition to first and second derivatives of these quantities. If it is not possible to evaluate the functions (for example, $c(x) = 1/x$ at $x = 0$) then set IFERR = 1 and exit the function evaluation process. Otherwise, set IFERR = 0 when the functions are evaluated. The function error flag should be used to avoid computational difficulties, but never in lieu of an explicit constraint.

The SNLPMN software uses internal common blocks to communicate information. To avoid conflict with these internal commons the user should not have commons with the following names: NPSPRR, NPSPRI, NPSPRC, STATIS, KONSTN, NPSALG, NPSDFL, NLPERR, PERCOM, ITEREF, and CLKCOM.

USAGE

```

INTEGER      IRVCOM, IFERR, NFEVAL, IER, NEEDED
INTEGER      NDIM, MCON, NONZH, NONZG, NHOLD, NIHOLD
INTEGER      IREVRS(5), ISTATV(NDIM), IROWH(NONZH), JSTRH(NDIM+1)
INTEGER      ISTATC(MCON), IROWG(NONZG), JCOLG(NONZG), IHOLD(NIHOLD)

DOUBLE PRECISION      FBAR, XBAR(NDIM), XLWR(NDIM), XUPR(NDIM), VECNU(NDIM)
DOUBLE PRECISION      DELF(NDIM), HMAT(NONZH), CBAR(MCON), CLWR(MCON)
DOUBLE PRECISION      CUPR(MCON), VECLAM(MCON), GMAT(NONZG), HOLD(NHOLD)

CALL SNLPMN( IRVCOM ,IREVRS ,XBAR      ,XLWR      ,XUPR      ,ISTATV
$           ,VECNU  ,NDIM      ,FBAR      ,DELF      ,HMAT      ,IROWH
$           ,JSTRH  ,NONZH    ,CBAR      ,CLWR      ,CUPR      ,ISTATC
$           ,MCON   ,VECLAM  ,GMAT      ,IROWG      ,JCOLG      ,NONZG
$           ,IFERR  ,NFEVAL  ,HOLD      ,NHOLD      ,IHOLD      ,NIHOLD
$           ,NEEDED ,IER  )

```

ARGUMENTS

ARGUMENTS TO CONTROL REVERSE COMMUNICATION

IRVCOM [INPUT/OUTPUT, INTEGER]
Continuation control flag. This argument must be initialized. Thereafter it is set by the algorithm and must not be changed by the calling program.

IRVCOM = -1 Initialization pass.

IRVCOM = 0 Termination (normal or abnormal).

IRVCOM = +1 Perform an evaluation and then reenter SNLPMN. Operations to be performed are defined by the output variables IREVRS(1), IREVRS(2), IREVRS(3), and IREVRS(4) below. IREVRS(5) contains information which may be valuable to the interface but does not require any action on the part of the user.

IREVRS [OUTPUT, INTEGER, ARRAY]
 An integer array of length 5, containing reverse communication control information.

IREVRS(1) Function evaluation request
 = 0 Function evaluation not requested; *Warning: do not change the current function and constraint values.*
 = 1 Function evaluation requested; evaluate objective function and all constraints at current point XBAR.

IREVRS(2) Gradient evaluation request
 = 0 Gradient evaluation not requested; *Warning: do not change the current gradient and Jacobian.*
 = 1 Approximate (e.g. forward difference) gradient requested; evaluate gradient vector g and Jacobian matrix G at current point XBAR. (If available, accurate or analytic gradients may be supplied)
 = 2 Accurate (e.g. central difference or analytic) gradient requested; evaluate gradient vector g and Jacobian matrix G at current point XBAR.

IREVRS(3)	Hessian evaluation request	
	= 0	Hessian evaluation not requested; <i>Warning: do not change the current Hessian.</i>
	= 1	Hessian diagonal requested; get ready for full Hessian evaluation with (IREVRS(3)=2) at current point (XBAR,VECLAM) and optionally evaluate diagonal elements of Hessian matrix H .
	= 2	Full Hessian evaluation requested; evaluate Hessian matrix H of the function $L(x, \lambda) = f - \sum_{k=1}^m \lambda_k c_k$ at current point (XBAR,VECLAM). A full evaluation request is always preceded by a call with IREVRS(3)=1 at the same point XBAR, and consequently it may not be necessary to recompute the diagonal elements.
IREVRS(4)	System print (output) request	
	= 0	System print not requested.
	= 1	System print requested; perform any output operations (print, write files, etc.) desired to display problem specific information at the current point XBAR. The frequency of print is defined by the output control flag IOFLAG (see Optional Printout).
IREVRS(5)	Algorithm information flag	
	= 0	Call was made from the interface portion of the algorithm.
	= 1	Call was made from the optimization portion of the algorithm.
	= 2	Call was made from the optimization portion of the algorithm requesting Hessian reset.
	= 3	Call was made from the optimization portion of the algorithm requesting optimal perturbation size adjustment.
	= 4	Call was made from the feasibility portion of the algorithm.

INDEPENDENT VARIABLE DATA

XBAR[†] [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM of current variable values x .

IRVCOM=-1 Initial guess must be input.

IRVCOM=0 Final point is output. When IER = 0, +101, the final point corresponds to a local optimum. When IER = +104, ..., +111, +114, +116, +117, +119, the final point is either the best feasible point if one has been found, or the last iterate.

IRVCOM=1 Intermediate iteration points are output.

XLWR[†] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM containing the variable lower bounds x_L . For variables that have no lower bound set $x_{Li} \leq -.01/\text{HDMCON}(5)$. See Appendix B for HDMCON.

XUPR[†] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM containing the variable upper bounds x_U . For variables that have no upper bound set $x_{Ui} \geq .01/\text{HDMCON}(5)$.

ISTATV[†] [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length NDIM indicating variable status. On input ISTATV defines an estimate of the active variable bounds, and on output ISTATV defines the bounds active at the solution.

ISTATV(i)= 0 Variable is strictly feasible, i.e. $x_{Li} < x_i < x_{Ui}$.

ISTATV(i)= 1 Variable lower bound is active, i.e. $x_{Li} = x_i$.

ISTATV(i)= 2 Variable upper bound is active, i.e. $x_i = x_{Ui}$.

ISTATV(i)= 3 Fixed permanently.

ISTATV(i)= 10 Output only: Violated variable lower bound, i.e. $x_{Li} - \delta_c > x_i$.

ISTATV(i)= 20 Output only: Violated variable upper bound, i.e. $x_i > x_{Ui} + \delta_c$.

ISTATV(i)= 30 Output only: Violated fixed variable. Either $x_i < x_{Li} - \delta_c$ or $x_i > x_{Ui} + \delta_c$.

When ISTATV(i) = 3 it is necessary that $x_{Li} = x_{Ui}$, where $x_{Li} = \text{XLWR}(i)$ and $x_{Ui} = \text{XUPR}(i)$. When ISTATV(i) $\neq$ 3, it is necessary that $x_{Li} < x_{Ui}$ with $|x_{Li} - x_{Ui}| > \delta_c$.

VECNU[#] [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
An array of length NDIM of bound multipliers ν .

NDIM[†] [INPUT, INTEGER]
Number of variables n , where $1 \leq \text{NDIM}$.

OBJECTIVE FUNCTION DATA

FBAR [‡]	[INPUT, DOUBLE PRECISION] The objective function evaluated at XBAR, $f(\text{XBAR})$.
DELF [‡]	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the gradient g of FBAR at XBAR.
HMAT [‡]	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NONZH containing the nonzero elements of Hessian matrix $H_L = \nabla_x^2 f - \sum_{i=1}^m \lambda_i \nabla_x^2 c_i$ of the Lagrangian function. Since the Hessian is symmetric, only the lower triangular part is input (see USAGE REMARKS). Storage for the diagonal elements must be provided even if they are zero.
IROWH [†]	[INPUT, INTEGER, ARRAY] An integer array of length NONZH containing the row indices of the Hessian nonzeros, where $0 < \text{IROWH}(k) \leq \text{NDIM}$ for $k = 1, \dots, \text{NONZH}$. The diagonal elements must be specified, i.e. $\text{IROWH}(\text{JSTRH}(k)) = k$ for $k = 1, \dots, \text{NDIM}$.
JSTRH [†]	[INPUT, INTEGER, ARRAY] An integer array of length NDIM+1 containing the column start indices of nonzeros, where $\text{JSTRH}(1) = 1$, $\text{JSTRH}(\text{NDIM}+1) = \text{NONZH} + 1$, and $\text{JSTRH}(k) < \text{JSTRH}(k+1)$ for $k = 1, \dots, \text{NDIM}$.
NONZH [†]	[INPUT, INTEGER] Number of nonzero Hessian elements, where $n \leq \text{NONZH} \leq n(n+1)/2$.

CONSTRAINT DATA

CBAR [‡]	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON containing the constraints $c(x)$ evaluated at XBAR. Not referenced when MCON = 0.
CLWR [†]	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON containing the constraint lower bounds c_L . For constraints that have no lower bound set $c_{Li} \leq -.01/\text{HDMCON}(5)$. Not referenced when MCON = 0.
CUPR [†]	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON containing the constraint upper bounds c_U . For constraints that have no upper bound set $c_{Ui} \geq .01/\text{HDMCON}(5)$. Not referenced when MCON = 0.
ISTATC [†]	[INPUT/OUTPUT, INTEGER, ARRAY] An integer array of length MCON indicating the constraint status. On input ISTATC defines an estimate of the active constraints, and on output ISTATC defines the constraints active at the solution. ISTATC(i)= 0 Free (inactive) inequality, i.e. $c_{Li} < c_i(x) < c_{Ui}$. ISTATC(i)= 1 Constraint lower bound is active, i.e. $c_{Li} = c_i(x)$. ISTATC(i)= 2 Constraint upper bound is active, i.e. $c_i(x) = c_{Ui}$.

ISTATC(i)= 3	Equality.
ISTATC(i)= 4	Ignored constraint.
ISTATC(i)= 10	Output only: Violated constraint lower bound, i.e. $c_{Li} - \delta_c > c_i$.
ISTATC(i)= 20	Output only: Violated constraint upper bound , i.e. $c_i > c_{Ui} + \delta_c$.
ISTATC(i)= 30	Output only: Violated equality constraint. Either $c_i < c_{Li} - \delta_c$ or $c_i > c_{Ui} + \delta_c$.

When ISTATC(i) =3 it is necessary that $c_{Li} = c_{Ui}$, where $c_{Li} = \text{CLWR}(i)$ and $c_{Ui} = \text{CUPR}(i)$. When ISTATC(i) $\neq 3$, it is necessary that $c_{Li} < c_{Ui}$ with $|c_{Li} - c_{Ui}| > \delta_c$.
Not referenced when MCON = 0.

MCON [†]	[INPUT, INTEGER] Number of constraints m , where $0 \leq \text{MCON}$.
VECLAM [#]	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON of Lagrange multipliers, λ , for constraints. The user must evaluate the Hessian of $L(x, \lambda) = f - \sum_{k=1}^m \lambda_k c_k$ (see IREVRS(3)). Not referenced when MCON = 0.
GMAT [‡]	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NONZG containing the nonzero elements of the Jacobian matrix G at XBAR (see USAGE REMARKS). Not referenced when MCON = 0.
IROWG [†]	[INPUT, INTEGER, ARRAY] An integer array of length NONZG containing the row indices of the Jacobian nonzeros, where $0 < \text{IROWG}(k) \leq \text{MCON}$ for $k = 1, \dots, \text{NONZG}$. Not referenced when MCON = 0.
JCOLG [†]	[INPUT, INTEGER, ARRAY] An integer array of length NONZG containing the column indices of the Jacobian nonzeros, where $0 < \text{JCOLG}(k) \leq \text{NDIM}$ for $k = 1, \dots, \text{NONZG}$. Not referenced when MCON = 0.
NONZG [†]	[INPUT, INTEGER] Number of Jacobian nonzeros, where $1 \leq \text{NONZG} \leq \text{NDIM} * \text{MCON}$.

ALGORITHM CONTROL DATA

IFERR [‡]	[INPUT, INTEGER] Function evaluation error flag. IFERR= 1 When FBAR and/or CBAR cannot be evaluated. IFERR= 0 Otherwise.
NFEVAL [‡]	[INPUT, INTEGER] Number of function evaluations. Normally NFEVAL is initialized prior to the first

call when $IRVCOM = -1$, and incremented whenever $IREVRS(1)$, $IREVRS(2)$, or $IREVRS(3)$ are nonzero. It may be appropriate to increment $NFEVAL$ for each perturbation when finite difference gradients are used, or not at all when analytic gradients are used.

WORKING STORAGE

HOLD	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length $NHOLD$ containing the hold array (working storage that must <u>not</u> be destroyed by the user).
NHOLD [†]	[INPUT/OUTPUT, INTEGER] Dimension of $HOLD$ array. When $IER = -127$, or -131 , $NEEDED$ contains the storage needed.
IHOLD	[INPUT/OUTPUT, INTEGER, ARRAY] An integer array of length $NIHOLD$ containing the integer hold array (working storage that must <u>not</u> be destroyed by the user).
NIHOLD [†]	[INPUT/OUTPUT, INTEGER] Dimension of $IHOLD$ array. When $IER = -128$, or -132 , $NEEDED$ contains the storage needed.

OUTPUT ARGUMENTS SET BY ALGORITHM BEFORE TERMINATION ($IRVCOM = 0$)

NEEDED	[OUTPUT, INTEGER] When $IER = -127$, or -131 , $NEEDED$ contains the required length of $HOLD$. When $IER = -128$, or -132 , $NEEDED$ contains the required length for $IHOLD$.
IER	[OUTPUT, INTEGER] Success/error code. Results are not computed for $IER < 0$. For $IER > 0$, results are computed but may be suspect; user inspection is recommended. Note many of the error returns involve optional input quantities as described in <i>INSNLP</i> . IER=0 Normal termination. IER=-101 ($MCON < 0$); the number of constraints is negative. IER=-102 ($NDIM < 1$); the number of variables is less than one. IER=-104 $ IHESHN > 3$. IER=-105 ($NITMAX < \max(1, NITMIN)$); the maximum number of iterations is either less than 1 or less than the minimum number of iterations. IER=-106 ($SLPTOL \geq 1$) or ($SLPTOL \leq 10^{-5}$); the slope tolerance for the line search during optimization is too large or too small. IER=-107 ($IT1MAX < 1$); the number of line search steps is less than 1.

IER=-108	(IOFLAG < 0) or (IOFLAG > 30); invalid input for the output control flag.
IER=-109	Invalid input for ALGOPT.
IER=-110	(OBJTOL $\leq$ 10.*HDMCON(5)); objective function tolerance is too small.
IER=-111	(PGDTOL $\leq$ [HDMCON(5)] ^{1/2}) or (PGDTOL > 10 ⁻²); the projected gradient tolerance is too small or too large.
IER=-112	CONTOL < [HDMCON(5)] ^{1/2} ; the constraint tolerance is too small.
IER=-113	(ISTATC(i) < 0) or (ISTATC(i) > 4); invalid input for constraint status.
IER=-114	(NONZG $\leq$ 0) or (NONZG > NDIM*MCON); the number of Jacobian nonzeros is either less than one, or exceeds the size of a dense matrix.
IER=-115	(JCOLG(i) $\leq$ 0) or (JCOLG(i) > NDIM); invalid input for Jacobian column index array.
IER=-116	(IROWG(i) $\leq$ 0) or (IROWG(i) > MCON); invalid input for Jacobian row index.
IER=-117	(CUPR(i) < CLWR(i)); constraint upper bound is less than lower bound.
IER=-118	Either (CUPR(i) = CLWR(i)) and (ISTATC(i) $\neq$ 3) or (CUPR(i) $\neq$ CLWR(i)) and (ISTATC(i) = 3); constraint status array is not consistent with bounds.
IER=-119	(CUPR(i) $\neq$ CLWR(i)) and (CUPR(i)-CLWR(i) < CONTOL); constraint bounds are not equal, but differ by less than the constraint tolerance.
IER=-120	(ISTATV(i) < 0) or (ISTATV(i) > 3); invalid input for variable status array.
IER=-121	(JSTRH(1) $\neq$ 1) or (JSTRH(k) $\geq$ JSTRH(k+1)); invalid input for Hessian column start array.
IER=-122	(NZHDIM < NDIM) or (NZHDIM > NDNSH) or (NZHDIM $\neq$ NONZH) where NZHDIM = JSTRH(NDIM+1)-1, and NDNSH = NDIM*(NDIM+1)/2; the number of Hessian nonzeros is less than one, exceeds the size of a dense Hessian, or is not consistent with the input value NONZH.
IER=-123	(IROWH(i) $\leq$ 0) or (IROWH(i) > NDIM); invalid value for Hessian row index.

IER=-124	(XUPR(i) < XLWR(i)); the variable upper bound is less than the lower bound.
IER=-125	Either (XUPR(i) = XLWR(i)) and (ISTATV(i) $\neq$ 3) or (XUPR(i) $\neq$ XLWR(i)) and (ISTATV(i) = 3); the variable status array is not consistent with the bound values.
IER=-126	(XUPR(i) $\neq$ XLWR(i)) and (XUPR(i)-XLWR(i) < CONTOL); variable bounds are not equal but differ by less than the constraint tolerance.
IER=-127	Real hold array too small; insufficient storage detected in SNLPMN interface. The required storage is specified in NEEDED.
IER=-128	Integer hold array too small; insufficient storage detected in SNLPMN interface. The required storage is specified in NEEDED.
IER=-129	Function error at initial point or during gradient evaluation.
IER=-130	(The number of active constraints > NDIM); check ISTATC(i) and ISTATV(i).
IER=-131	NHOLD too small; insufficient real storage detected in algorithm. The required storage is specified in NEEDED.
IER=-132	NIHOLD too small; insufficient integer storage detected in algorithm. The required storage is specified in NEEDED.
IER=-133	Rank deficient Jacobian detected on successive iterations.
IER=-134	INSNLP input error; invalid character string displayed.
IER=-135	(IFERR < 0) or (IFERR > 1); invalid value for function error flag.
IER=-136	(SFZTOL $\geq$ 1) or (SFZTOL $\leq 10^{-5}$); the slope tolerance for the line search during the feasibility phase is too large or too small.
IER=-137	Conflict between user and multifrontal file number; check IPUMF1,...,IPUMF6 IPUDRF, IPUFZF, IPUSTF.
IER=-138	(NEWTON < 0) or (NEWTON > 1); invalid input for Newton method option flag.
IER=-145	(IROWH(JSTRH(i)) $\neq$ i) for some value of i; Hessian diagonal elements are incorrect.
IER=-147	Unexpected error; check storage allocation and verify that the HOLD and IHOLD arrays are unchanged during reverse communication calls.

IER=-148	$ \text{IRVCOM} > 1$.
IER=-149	Incorrect value for QPOPTN.
IER=-150	QPOPTN = DENSE and either ALGOPT = 'FME' or NRES>0.
IER=-151	A constraint is inconsistent (either $c_i(x) < c_{Li}$ or $c_{Ui} < c_i(x)$) and cannot be changed because the corresponding row of the Jacobian is zero.
IER=-152	Incorrect value for KTOPTN.
IER=-153	I/O error, probably caused by insufficient disk space.

IER=+101	Weak solution found (multipliers near zero).
IER=+102	Number of equality constraints = NDIM and ALGOPT $\neq$ 'F'.
IER=+103	Maximum number of consecutive function errors.
IER=+104	Maximum number of function evaluations.
IER=+105	Small step termination in optimization phase; suboptimal feasible point found.
IER=+106	Maximum number of iterations in optimization phase.
IER=+107	Maximum number of iterations in feasibility phase.
IER=+108	Feasible point not found.
IER=+109	Maximum number of interval halves in line search.
IER=+110	QP algorithm terminated because either the Hessian diagonal reached its maximum value or the merit function slope condition was violated; suboptimal feasible point found.
IER=+111	Projected gradient calculation failed; constraints may be degenerate.
IER=+112	Calculation of first order multiplier estimates failed; constraints may be degenerate.
IER=+113	Suboptimal feasible point found.
IER=+114	Schur-complement QP algorithm (subroutine SHURQP) failed with unexpected error.

IER=+115	CONTROL > OBJTOL; convergence tolerances may be inappropriate.
IER=+116	Uphill direction detected in line search.
IER=+117	Reduced objective function is linear.
IER=+118	Both $c_{Li} \leq -.01/\text{HDMCON}(5)$ and $c_{Ui} \geq .01/\text{HDMCON}(5)$; constraints ignored.
IER=+119	Terminate after diagnostic line search.
IER=+120	(IHESHN > 0) and (NEWTON = 0); recursive Hessian updates may be skipped causing poor algorithm performance.

†	Variable set when IRVCOM=-1, and not changed by user thereafter
‡	Variable set when IRVCOM=+1, as required by IREVRS(*)
#	Variable optionally set when IRVCOM=-1, and not changed by user thereafter

USAGE REMARKS

Sparse Matrix Input

Hessian Matrix. The user must supply the Hessian matrix of the following Lagrangian function:

$$L(x, \lambda) = f - \sum_{k=1}^m \lambda_k c_k$$

that is,

$$L(\text{XBAR}, \text{VECLAM}) = \text{FBAR} - \text{VECLAM}(1) * \text{CBAR}(1) - \dots - \text{VECLAM}(\text{MCON}) * \text{CBAR}(\text{MCON}) .$$

Consider the following example in which $n = 3$:

$$H = \begin{pmatrix} 1.1 & 0 & 3.1 \\ 0 & 2.2 & 0 \\ 3.1 & 0 & 3.3 \end{pmatrix}$$

Since the Hessian is a symmetric matrix only the lower triangular portion must be specified. For this example $\text{NDIM} = 3$ and $\text{NONZH} = 4 = [\text{JSTRH}(\text{NDIM}+1)-1]$. The nonzero values are specified as $\text{HMAT}(\text{*}) = (1.1, 3.1, 2.2, 3.3)$. The row indices are $\text{IROWH}(\text{*}) = (1, 3, 2, 3)$, and the column start indices are $\text{JSTRH}(\text{*}) = (1, 3, 4, 5)$. Notice that the element in row 1, column 3 is not specified. Also observe that the diagonal elements must always be included in the Hessian sparsity pattern *even if the elements are zero*. Additional information on sparse matrix formats is found in BCSLIB-EXT [1].

Jacobian Matrix. The user must specify the nonzero elements in the Jacobian matrix when required.

Consider the following example in five variables $n = 5$ and three constraints $m = 3$:

$$G = \begin{pmatrix} 0 & 0 & 1.3 & 0 & 1.5 \\ 2.1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 3.3 & 0 & 0 \end{pmatrix}$$

For this example $\text{NDIM} = 5$, $\text{MCON} = 3$, and the number of nonzeros $\text{NONZG} = 4$. The nonzero array is $\text{GMAT}^* = (2.1, 1.3, 3.3, 1.5)$. The row indices are $\text{IROWG}^* = (2, 1, 3, 1)$, and the column indices are $\text{JCOLG}^* = (1, 3, 3, 5)$. Additional information on sparse matrix formats is found in BCSLIB-EXT [1].

Unconstrained Problems

When the problem is unconstrained the user must set $\text{MCON} = 0$. The following constraint related quantities will not be referenced: CBAR , CLWR , CUPR , ISTATC , VECLAM , GMAT , IROWG , JCOLG , and NONZG . Since these quantities must appear as arguments in the call to SNLPMN , dummy values may be utilized.

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine SNLPMN . All optional inputs are specified by calls to subroutine INSNLP . It should be emphasized that it is *not necessary* to call INSNLP if the default values are appropriate.

OPTIONAL PRINTOUT

The level of printed output from HSSNLP is controlled by the variables IOFLAG and IPUNLP which are set by calls to INSNLP . The standard level of output which corresponds to $\text{IOFLAG} = 10$ is the *default* for HSSNLP . A complete description of the iteration output options is found in Section 2.3.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2, x_3, x_4)$ to minimize the objective function

$$f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2 + 100(x_4 - x_3^2)^2 + (1 - x_3)^2$$

subject to the two constraints

$$x_1 x_2 \leq \frac{1}{2} \qquad -\frac{1}{2} \leq x_1 + x_3 \leq \frac{1}{2}$$

and the bounds

$$-2 \leq x_k \leq 2$$

for $k = 1, 2, 3, 4$ beginning at the initial point $x = (-1, 2, -1, 2)$. For this example the gradient vector is

$$g = \nabla_x f = \begin{bmatrix} -400x_1(x_2 - x_1^2) - 2(1 - x_1) \\ 200(x_2 - x_1^2) \\ -400x_3(x_4 - x_3^2) - 2(1 - x_3) \\ 200(x_4 - x_3^2) \end{bmatrix}$$

and the Jacobian matrix is

$$G = \begin{bmatrix} \nabla_x c_1^T \\ \nabla_x c_2^T \end{bmatrix} = \begin{bmatrix} x_2 & x_1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}.$$

It is also necessary to supply the Hessian matrix

$$\nabla_x^2 L = \nabla_x^2 f - \lambda_1 \nabla_x^2 c_1 - \lambda_2 \nabla_x^2 c_2$$

where

$$\nabla_x^2 f = \begin{bmatrix} 1200x_1^2 - 400x_2 + 2 & -400x_1 & 0 & 0 \\ -400x_1 & 200 & 0 & 0 \\ 0 & 0 & 1200x_3^2 - 400x_4 + 2 & -400x_3 \\ 0 & 0 & -400x_3 & 200 \end{bmatrix}$$

with

$$\nabla_x^2 c_1 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

and $\nabla_x^2 c_2 = 0$. Note: this example also illustrates the use of the “Terse Output” option by the call to INSNLP with “IOFLAG = 1”.

SAMPLE PROGRAM (snlpmn.cd1)⁴

```

PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE SPARSE NONLINEAR
C      PROGRAMMING ALGORITHM SNLPMN
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      PARAMETER (NDIM=4, MCON=2)
C      PARAMETER (NONZH=6, NONZG=4)
C      PARAMETER (NHOLD=2000,NIHOLD=2000)
C
C      DIMENSION IREVRS(5),XBAR(NDIM),XLWR(NDIM),XUPR(NDIM),
C      $      ISTATV(NDIM),VECNU(NDIM),DELF(NDIM),HMAT(NONZH),
C      $      IROWH(NONZH),JSTRH(NDIM+1),CBAR(MCON),CLWR(MCON),
C      $      CUPR(MCON),ISTATC(MCON),VECLAM(MCON),
C      $      GMAT(NONZG),IROWG(NONZG),JCOLG(NONZG)
C      DIMENSION HOLD(NHOLD),IHOLD(NIHOLD)
C
C      INITIALIZE REVERSE COMMUNICATION CONTROL FLAG
C
C      ICC = -1
C -----

```

⁴sosexample.tar

```
C
C      INDEPENDENT VARIABLE DATA
C
C      ----INITIAL GUESS
C      XBAR(1) = -1.0D0
C      XBAR(2) =  2.0D0
C      XBAR(3) = -1.0D0
C      XBAR(4) =  2.0D0
C      ----BOUNDS
C      XLWR(1:NDIM) = -2.D0
C      XUPR(1:NDIM) =  2.D0
C      ----VARIABLE STATUS
C      ISTATV(1:NDIM) = 0
C
C -----
C
C      OBJECTIVE FUNCTION DATA
C
C      ----HESSIAN SPARSITY PATTERN (ROW INDICES)
C      IROWH(1) = 1
C      IROWH(2) = 2
C      IROWH(3) = 2
C      IROWH(4) = 3
C      IROWH(5) = 4
C      IROWH(6) = 4
C      ----HESSIAN SPARSITY PATTERN (COLUMN START INDICES)
C      JSTRH(1) = 1
C      JSTRH(2) = 3
C      JSTRH(3) = 4
C      JSTRH(4) = 6
C      JSTRH(5) = 7
C
C -----
C
C      CONSTRAINT DATA
C
C      ----BOUNDS
C      CLWR(1) = -1.D0/HDMCON(5)
C      CUPR(1) = .5D0
C      CLWR(2) = -.5D0
C      CUPR(2) = .5D0
C      ----CONSTRAINT STATUS
C      ISTATC(1:MCON) = 0
C      ----JACOBIAN SPARSITY PATTERN (ROW INDICES)
C      IROWG(1) = 1
C      IROWG(2) = 2
C      IROWG(3) = 1
C      IROWG(4) = 2
```

```
C      ----JACOBIAN SPARSITY PATTERN (COLUMN INDICES)
      JCOLG(1) = 1
      JCOLG(2) = 1
      JCOLG(3) = 2
      JCOLG(4) = 3
C
C -----
C
C      INITIALIZE FUNCTION EVALUATION COUNT
C
      NFEVAL = 0
C
      SET OPTIONAL INPUTS
C
      CALL INSNLP('IOFLAG=1')
C
C -----
C
1000 CONTINUE
C
      CALL SNLPMN( ICC ,IREVRS ,XBAR ,XLWR ,XUPR ,ISTATV
$              ,VEGNU ,NDIM ,FBAR ,DELF ,HMAT ,IROWH
$              ,JSTRH ,NONZH ,CBAR ,CLWR ,CUPR ,ISTATC
$              ,MCON ,VECLAM ,GMAT ,IROWG ,JCOLG ,NONZG
$              ,IFERR ,NFEVAL ,HOLD ,NHOLD ,IHOLD ,NIHOLD
$              ,NEEDED ,IERNLP )
C
      REVERSE COMMUNICATION TERMINATION TEST
C
      IF(ICC.EQ.0) GO TO 10000
C
C -----
C
      SET FUNCTION EVALUATION ERROR FLAG
C
      IFERR = 0
C
      >>>>FUNCTION EVALUATION REQUEST
C
      IF(IREVRS(1).EQ.1) THEN
C
          FBAR = 100.DO*(XBAR(2) - XBAR(1)**2)**2
$              + (1.DO - XBAR(1))**2
$              + 100.DO*(XBAR(4) - XBAR(3)**2)**2
$              + (1.DO - XBAR(3))**2
C
          CBAR(1) = XBAR(1)*XBAR(2)
          CBAR(2) = XBAR(1) + XBAR(3)
```

```

      NFEVAL = NFEVAL + 1
C
      ENDIF
C
      >>>>GRADIENT EVALUATION REQUEST
C
      IF(IREVRS(2).EQ.1) THEN
C
          DELF(1) = - 400.DO*XBAR(1)*(XBAR(2) - XBAR(1)**2)
$          - 2.DO*(1.DO - XBAR(1))
          DELF(2) = 200.DO*(XBAR(2) - XBAR(1)**2)
          DELF(3) = - 400.DO*XBAR(3)*(XBAR(4) - XBAR(3)**2)
$          - 2.DO*(1.DO - XBAR(3))
          DELF(4) = 200.DO*(XBAR(4) - XBAR(3)**2)
C
          GMAT(1) = XBAR(2)
          GMAT(2) = 1.DO
          GMAT(3) = XBAR(1)
          GMAT(4) = 1.DO
C
      ELSEIF(IREVRS(2).EQ.2) THEN
C
          DELF(1) = - 400.DO*XBAR(1)*(XBAR(2) - XBAR(1)**2)
$          - 2.DO*(1.DO - XBAR(1))
          DELF(2) = 200.DO*(XBAR(2) - XBAR(1)**2)
          DELF(3) = - 400.DO*XBAR(3)*(XBAR(4) - XBAR(3)**2)
$          - 2.DO*(1.DO - XBAR(3))
          DELF(4) = 200.DO*(XBAR(4) - XBAR(3)**2)
C
          GMAT(1) = XBAR(2)
          GMAT(2) = 1.DO
          GMAT(3) = XBAR(1)
          GMAT(4) = 1.DO
C
      ENDIF
C
      >>>>HESSIAN EVALUATION REQUEST
C
      IF(IREVRS(3).EQ.1) THEN
C
          DIAGONAL ELEMENTS
C
          HMAT(1) = 1200.DO*XBAR(1)**2
$          - 400.DO*XBAR(2) + 2.DO
          HMAT(3) = 200.DO
          HMAT(4) = 1200.DO*XBAR(3)**2
$          - 400.DO*XBAR(4) + 2.DO
          HMAT(6) = 200.DO

```

```
C
      ELSEIF(IREVRS(3).EQ.2) THEN
C
C          OFF-DIAGONAL ELEMENTS
C
C          HMAT(2) = -VECLAM(1)*(1.DO) - 400.DO*XBAR(1)
C          HMAT(5) = - 400.DO*XBAR(3)
C
C      ENDIF
C
C      >>>>SYSTEM PRINT (OUTPUT) REQUEST
C
C      IF(IREVRS(4).EQ.1) THEN
C
C          WRITE(6,1001) 'OBJECTIVE  ',FBAR
C          WRITE(6,1001) 'VARIABLES  ',(XBAR(II),II=1,NDIM)
C          WRITE(6,1001) 'CONSTRAINTS',(CBAR(II),II=1,MCON)
C
C      ENDIF
C
C      -----
C
C      2000 CONTINUE
C
C          END OF REVERSE COMMUNICATION PROCESSING.  RETURN TO SNLPMN
C
C      GO TO 1000
C
C      10000 CONTINUE
C
C          TERMINATE PROCESSING
C
C      IF(IERNLP.EQ.0) THEN
C
C          NORMAL TERMINATION---SOLUTION FOUND
C
C          WRITE(6,1001) 'SOLUTION  '
C
C          WRITE(6,1001) 'OBJECTIVE  ',FBAR
C          WRITE(6,1001) 'VARIABLES  ',(XBAR(II),II=1,NDIM)
C          WRITE(6,1001) 'CONSTRAINTS',(CBAR(II),II=1,MCON)
C
C      ELSEIF(IERNLP.LT.0) THEN
C
C          ABNORMAL TERMINATION---FATAL ERROR
C
C          PRINT *, 'FATAL ERROR; IERNLP =', IERNLP
C
```



```

ELSE
C
C      ABNORMAL TERMINATION---WARNING.  EVALUATION OF RESULTS RECOMMENDED!
C
C      PRINT *, 'WARNING MESSAGE; IERNLP =', IERNLP
C
C      ENDIF
C
1001 FORMAT(5X,A11,5X,4G17.8)
C
STOP
END

```

OUTPUT FROM SAMPLE PROGRAM

Feasible Point							
It	Qit	Nkt	Ndof	KT Cond	Step	Norm p	Violtn
1	4	1	1	1.0E+00	1.0E+00	1.1E+00	1.5E+00
2	0	0	1	1.0E+00	1.0E+00	1.1E+00	0.0E+00

Optimization											
It	Qit	Nkt	Ndof	KT Cond	Step	Norm p	Violtn	Levnbrg	Penalty	Norm Pg	Merit Function
(1)	4	9	3	1.0E+00	1.0E+00	5.4E-01	0.0E+00	1.0E+00	1.5E-08	4.0E+00	7.539062E+02
(2)	2	3	3	1.2E+01	1.0E+00	5.1E-01	5.6E-16	1.0E+00	1.5E-08	3.1E+02	4.892724E+02
(3)	2	3	3	1.1E+01	1.0E+00	4.7E-01	2.2E-16	1.0E+00	1.5E-08	2.4E+02	2.919674E+02
(4)	2	2	3	9.6E+00	1.0E+00	4.1E-01	3.3E-16	1.0E+00	1.5E-08	1.7E+02	1.553314E+02
(5)	2	3	3	8.7E+00	1.0E+00	3.4E-01	1.1E-16	1.0E+00	1.5E-08	1.2E+02	7.054040E+01
(6)	2	3	3	8.1E+00	1.0E+00	2.5E-01	1.1E-16	1.0E+00	1.5E-08	6.8E+01	2.614903E+01
(7)	1	1	3	5.8E+02	1.0E+00	1.8E-01	2.2E-16	4.8E-01	1.5E-08	3.3E+01	8.432785E+00
[8]	1	2	4	7.9E+00	1.0E+00	6.4E-02	2.2E-16	1.0E+00	1.5E-08	7.0E+00	3.370612E+00
(9)	3	1	4	2.2E+01	1.0E+00	2.2E-01	0.0E+00	3.1E-01	1.5E-08	2.2E+00	3.018158E+00
(10)	3	1	4	2.3E+01	1.0E+00	2.8E-01	0.0E+00	1.5E-01	1.5E-08	2.8E+00	2.390020E+00
(11)	1	1	4	1.3E+01	1.0E+00	1.6E-01	0.0E+00	7.7E-02	1.5E-08	7.8E+00	1.874379E+00
[12]	4	2	3	5.1E+01	1.0E+00	7.3E-02	0.0E+00	1.8E-02	1.5E-08	1.8E+00	1.276652E+00
(13)	3	1	3	6.5E+02	1.0E+00	2.9E-03	0.0E+00	4.0E-03	1.5E-08	4.1E-01	1.125832E+00
(14)	1	1	3	5.2E+02	1.0E+00	2.8E-06	7.1E-15	3.9E-06	1.5E-08	4.0E-04	1.125000E+00
(15)	0	0	3	5.2E+02	1.0E+00	2.8E-06	1.2E-14	3.9E-06	1.5E-08	3.8E-10	1.125000E+00

```

-----
|
|               SPRNLP ALGORITHM PERFORMANCE STATISTICS
|
| Total CPU Time.....+4.701413E-02
|
| Number of Function Calls.....16
| Number of Gradient Calls.....17
| Number of Hessian Calls.....14
| Total Number of Function Evaluations.....16
|
| Storage Needed in HOLD Array.....374
| Storage Needed in IHOLD Array.....27
|
-----

```

```

SOLUTION
OBJECTIVE          1.1250000
VARIABLES          0.25000000    0.62500000E-01    0.25000000    0.62500000E-01
CONSTRAINTS        0.15625000E-01    0.50000000

```

SNLPLS: Sparse Constrained Nonlinear Least Squares

PURPOSE

SNLPLS computes the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the least squares objective function

$$f(x) = \frac{1}{2} r^T(x) r(x) = \frac{1}{2} \sum_{i=1}^{\ell} r_i^2$$

where $r(x)$ is an ℓ -vector of residuals, subject to the m nonlinear constraints

$$c_L \leq c(x) \leq c_U$$

and the simple bounds

$$x_L \leq x \leq x_U$$

Equality constraints are imposed by setting $c_{Li} = c_{Ui}$ and variables can be fixed by setting $x_{Li} = x_{Ui}$. SNLPLS works under the assumption that the residual and constraint functions are twice continuously differentiable, and if this is not true algorithm performance is unpredictable. SNLPLS uses a reverse communication format and upon request the user must supply the values of the functions, and their first and second derivatives. The matrix of first derivatives (the Jacobian) and the matrix of second derivatives (residual Hessian) are represented in a sparse format consistent with that used in BCSLIB-EXT [1]. An optional input to the software permits efficient solution of the problem when the residual and constraint functions are linear, i.e. the *linear least squares problem*.

RELATED SUBPROGRAMS

SNLPMN Sparse Nonlinear Programming

INSNLP Sparse Nonlinear Programming Input Procedure

METHOD

A *sequential quadratic programming* (SQP) approach is used to solve the *nonlinear programming* (NLP) problem. It is necessary to compute the residual Hessian matrix

$$V = \sum_{i=1}^{\ell} r_i \nabla^2 r_i - \sum_{i=1}^m \lambda_i \nabla^2 c_i.$$

If the user cannot provide this information analytically, the sparse finite difference techniques described in Chapter 3 should be utilized. Section 4.4 illustrates use of the sparse finite difference technique in conjunction with the least squares algorithm. Notice that the Hessian of the Lagrangian H_L , which is required input for the sparse nonlinear program SNLPMN is related to the residual Hessian by

$$H_L = V + R^T R$$

where R is the $\ell \times n$ residual Jacobian matrix. A sparse tableau form for the quadratic programming subproblem is utilized to avoid formation of the normal matrix $R^T R$. This QP subproblem is solved efficiently using the sparse Schur-Complement Method.

WARNING

The residual and constraint functions supplied by the user are assumed to be continuous and have continuous first and second derivatives. Nondifferentiable functions can cause unpredictable performance in the optimization algorithm. It should be emphasized that discontinuities in the Hessian matrices can significantly degrade the speed of convergence without producing any other obvious difficulties. The following common sources of error *should be avoided* when evaluating the residual and constraint functions:

- (1) nonsmooth interpolation of data;
- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;
- (3) discontinuous behavior caused by branching for IF tests;
- (4) nondifferentiable functions such as ABS, MAX, and MIN.

The user must provide a procedure to evaluate the residual and constraint functions in addition to first and second derivatives of these quantities. If it is not possible to evaluate the functions (for example, $c(x) = 1/x$ at $x = 0$) then set IFERR = 1 and exit the function evaluation process. Otherwise, set IFERR = 0 when the functions are evaluated. The function error flag should be used to avoid computational difficulties, but never in lieu of an explicit constraint.

The SNLPLS software uses internal common blocks to communicate information. To avoid conflict with these internal commons the user should not have commons with the following names: NPSPRR, NPSPRI, NPSPRC, STATIS, KONSTN, NPSALG, NPSDFL, NLPERR, PERCOM, ITEREF, and CLKCOM.

USAGE

INTEGER	IRVCOM, IFERR, NFEVAL, IER, NEEDED, NDIM, MCON
INTEGER	NRES, NONZH, NONZG, NHOLD, NIHOLD, NONZR
INTEGER	IREVRS(5), ISTATV(NDIM), IROWR(NONZR), JCOLR(NONZR)
INTEGER	IROWH(NONZH), JSTRH(NDIM+1)
INTEGER	ISTATC(MCON), IROWG(NONZG), JCOLG(NONZG), IHOLD(NIHOLD)
DOUBLE PRECISION	XBAR(NDIM), XLWR(NDIM), XUPR(NDIM), VECNU(NDIM)
DOUBLE PRECISION	RESVEC(NRES), RMAT(NONZR)
DOUBLE PRECISION	HMAT(NONZH), CBAR(MCON), CLWR(MCON)
DOUBLE PRECISION	CUPR(MCON), VECLAM(MCON), GMAT(NONZG), HOLD(NHOLD)

```

      CALL  SNLPLS( IRVCOM ,IREVRS ,XBAR  ,XLWR  ,XUPR
$          ,ISTATV ,VECNU  ,NDIM  ,RESVEC ,NRES  ,RMAT
$          ,IROWR  ,JCOLR  ,NONZR  ,HMAT   ,IROWH ,JSTRH
$          ,NONZH  ,CBAR   ,CLWR   ,CUPR   ,ISTATC ,MCON
$          ,VECLAM ,GMAT   ,IROWG  ,JCOLG  ,NONZG  ,IFERR
$          ,NFEVAL ,HOLD   ,NHOLD  ,IHOLD  ,NIHOLD ,NEEDED
$          ,IER   )

```

ARGUMENTS

ARGUMENTS TO CONTROL REVERSE COMMUNICATION

IRVCOM [INPUT/OUTPUT, INTEGER]

Continuation control flag. This argument must be initialized. Thereafter it is set by the algorithm and must not be changed by the calling program.

IRVCOM = -1 Initialization pass.

IRVCOM = 0 Termination (normal or abnormal).

IRVCOM = +1 Perform an evaluation and then reenter SNLPLS. Operations to be performed are defined by the output variables IREVRS(1), IREVRS(2), IREVRS(3), and IREVRS(4) below. IREVRS(5) contains information which may be valuable to the interface but does not require any action on the part of the user.

IREVRS [OUTPUT, INTEGER, ARRAY]

An integer array of length 5, containing reverse communication control information.

IREVRS(1) Function evaluation request

= 0 Function evaluation not requested; *Warning: do not change the current function and constraint values.*

= 1 Function evaluation requested; evaluate residual functions and all constraints at current point XBAR.

IREVRS(2) Gradient evaluation request

= 0 Gradient evaluation not requested; *Warning: do not change the current residual and constraint and Jacobians.*

= 1 Approximate (e.g. forward difference) gradient requested; evaluate residual Jacobian matrix R and Jacobian matrix G at current point XBAR. (If available, accurate or analytic gradients may be supplied)

= 2 Accurate (e.g. central difference or analytic) gradient requested; evaluate residual Jacobian matrix R and Jacobian matrix G at current point XBAR.

IREVRS(3)	Hessian evaluation request	
	= 0	Hessian evaluation not requested; <i>Warning: do not change the current Hessian.</i>
	= 1	Hessian diagonal requested; get ready for full Hessian evaluation with (IREVRS(3)=2) at current point (XBAR,VECLAM) and optionally evaluate diagonal elements of residual Hessian matrix V . When ALGOPT = 'LLSQ', this return will not occur (see Optional Input).
	= 2	Full Hessian evaluation requested; evaluate residual Hessian matrix $V = \sum_{i=1}^{\ell} r_i \nabla^2 r_i - \sum_{i=1}^m \lambda_i \nabla^2 c_i$ at current point (XBAR,VECLAM). A full evaluation request is always preceded by a call with IREVRS(3)=1 at the same point XBAR, and consequently it may not be necessary to recompute the diagonal elements. When ALGOPT = 'LLSQ', this return will not occur (see Optional Input).
IREVRS(4)	System print (output) request	
	= 0	System print not requested.
	= 1	System print requested; perform any output operations (print, write files, etc.) desired to display problem specific information at the current point XBAR. The frequency of print is defined by the output control flag IOFLAG (see Optional Printout).
IREVRS(5)	Algorithm information flag	
	= 0	Call was made from the interface portion of the algorithm.
	= 1	Call was made from the optimization portion of the algorithm.
	= 2	Call was made from the optimization portion of the algorithm requesting Hessian reset.
	= 3	Call was made from the optimization portion of the algorithm requesting optimal perturbation size adjustment.
	= 4	Call was made from the feasibility portion of the algorithm.

INDEPENDENT VARIABLE DATA

XBAR [†]	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM of current variable values x . IRVCOM=-1 Initial guess must be input. IRVCOM=0 Final point is output. When IER = 0, +101, the final point corresponds to a local optimum. When IER = +104, ..., +111, +114, +116, +117, +119, the final point is either the best feasible point if one has been found, or the last iterate. IRVCOM=1 Intermediate iteration points are output.
XLWR [†]	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the variable lower bounds x_L . For variables that have no lower bound set $x_{Li} \leq -.01/\text{HDMCON}(5)$. See Appendix B for HDMCON.
XUPR [†]	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the variable upper bounds x_U . For variables that have no upper bound set $x_{Ui} \geq .01/\text{HDMCON}(5)$.
ISTATV [†]	[INPUT/OUTPUT, INTEGER, ARRAY] An integer array of length NDIM indicating variable status. On input ISTATV defines an estimate of the active variable bounds, and on output ISTATV defines the bounds active at the solution. ISTATV(i)= 0 Variable is strictly feasible, i.e. $x_{Li} < x_i < x_{Ui}$. ISTATV(i)= 1 Variable lower bound is active, i.e. $x_{Li} = x_i$. ISTATV(i)= 2 Variable upper bound is active, i.e. $x_i = x_{Ui}$. ISTATV(i)= 3 Fixed permanently. ISTATV(i)= 10 Output only: Violated variable lower bound, i.e. $x_{Li} - \delta_c > x_i$. ISTATV(i)= 20 Output only: Violated variable upper bound, i.e. $x_i > x_{Ui} + \delta_c$. ISTATV(i)= 30 Output only: Violated fixed variable. Either $x_i < x_{Li} - \delta_c$ or $x_i > x_{Ui} + \delta_c$. When ISTATV(i) = 3 it is necessary that $x_{Li} = x_{Ui}$, where $x_{Li} = \text{XLWR}(i)$ and $x_{Ui} = \text{XUPR}(i)$. When ISTATV(i) $\neq$ 3, it is necessary that $x_{Li} < x_{Ui}$ with $ x_{Li} - x_{Ui} > \delta_c$.
VECNU [#]	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] An array of length NDIM of bound multipliers ν .
NDIM [†]	[INPUT, INTEGER] Number of variables n , where $1 \leq \text{NDIM}$.

OBJECTIVE FUNCTION DATA

RESVEC [‡]	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NRES containing the residuals r at XBAR.
NRES [†]	[INPUT, INTEGER] Number of residuals ℓ , where $0 \leq \text{NRES}$.
RMAT [‡]	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NONZR containing the nonzero elements of the residual Jacobian matrix R at XBAR (see USAGE REMARKS).
IROWR [†]	[INPUT, INTEGER, ARRAY] An integer array of length NONZR containing the row indices of the residual Jacobian nonzeros, where $0 < \text{IROWR}(k) \leq \text{NRES}$ for $k = 1, \dots, \text{NONZR}$.
JCOLR [†]	[INPUT, INTEGER, ARRAY] An integer array of length NONZR containing the column indices of the residual Jacobian nonzeros, where $0 < \text{JCOLR}(k) \leq \text{NDIM}$ for $k = 1, \dots, \text{NONZR}$.
NONZR [†]	[INPUT, INTEGER] Number of residual Jacobian nonzeros, where $0 \leq \text{NONZR} \leq \text{NRES} * \text{NDIM}$.
HMAT [‡]	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NONZH containing the nonzero elements of the residual Hessian matrix $V = \sum_{i=1}^{\ell} r_i \nabla^2 r_i - \sum_{i=1}^m \lambda_i \nabla^2 c_i$. Since the Hessian is symmetric, only the lower triangular part is input (see USAGE REMARKS). It is necessary to provide storage for diagonal elements even if $V = 0$. When $\text{ALGOPT} = \text{'LLSQ'}$, this matrix does not need to be computed (see Optional Input)
IROWH [†]	[INPUT, INTEGER, ARRAY] An integer array of length NONZH containing the row indices of the residual Hessian nonzeros, where $0 < \text{IROWH}(k) \leq \text{NDIM}$ for $k = 1, \dots, \text{NONZH}$. The diagonal elements must be specified, i.e. $\text{IROWH}(\text{JSTRH}(k)) = k$ for $k = 1, \dots, \text{NDIM}$.
JSTRH [†]	[INPUT, INTEGER, ARRAY] An integer array of length $\text{NDIM} + 1$ containing the column start indices of nonzeros, where $\text{JSTRH}(1) = 1$, $\text{JSTRH}(\text{NDIM} + 1) = \text{NONZH} + 1$, and $\text{JSTRH}(k) < \text{JSTRH}(k + 1)$ for $k = 1, \dots, \text{NDIM}$.
NONZH [†]	[INPUT, INTEGER] Number of nonzero residual Hessian elements, where $n \leq \text{NONZH} \leq n(n + 1)/2$. When $\text{ALGOPT} = \text{'LLSQ'}$, storage for diagonal elements must be provided and $\text{NONZH} = \text{NDIM}$ (see Optional Input)

CONSTRAINT DATA

CBAR [‡]	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON containing the constraints $c(x)$ evaluated at XBAR. Not referenced when $\text{MCON} = 0$.
--------------------------	--

- CLWR**[†] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraint lower bounds c_{Li} . For constraints that have no lower bound set $c_{Li} \leq -.01/\text{HDMCON}(5)$. Not referenced when MCON = 0.
- CUPR**[†] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraint upper bounds c_{Ui} . For constraints that have no upper bound set $c_{Ui} \geq .01/\text{HDMCON}(5)$. Not referenced when MCON = 0.
- ISTATC**[†] [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length MCON indicating the constraint status. On input ISTATC defines an estimate of the active constraints, and on output ISTATC defines the constraints active at the solution.
- ISTATC(i)= 0 Free (inactive) inequality, i.e. $c_{Li} < c_i(x) < c_{Ui}$.
- ISTATC(i)= 1 Constraint lower bound is active, i.e. $c_{Li} = c_i(x)$.
- ISTATC(i)= 2 Constraint upper bound is active, i.e. $c_i(x) = c_{Ui}$.
- ISTATC(i)= 3 Equality.
- ISTATC(i)= 4 Ignored constraint.
- ISTATC(i)= 10 Output only: Violated constraint lower bound, i.e. $c_{Li} - \delta_c > c_i$.
- ISTATC(i)= 20 Output only: Violated constraint upper bound, i.e. $c_i > c_{Ui} + \delta_c$.
- ISTATC(i)= 30 Output only: Violated equality constraint. Either $c_i < c_{Li} - \delta_c$ or $c_i > c_{Ui} + \delta_c$.
- When ISTATC(i)=3 it is necessary that $c_{Li} = c_{Ui}$, where $c_{Li} = \text{CLWR}(i)$ and $c_{Ui} = \text{CUPR}(i)$. When ISTATC(i) $\neq$ 3, it is necessary that $c_{Li} < c_{Ui}$ with $|c_{Li} - c_{Ui}| > \delta_c$. Not referenced when MCON = 0.
- MCON**[†] [INPUT, INTEGER]
Number of constraints m , where $0 \leq \text{MCON}$.
- VECLAM**[#] [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON of Lagrange multipliers, λ , for constraints. Not referenced when MCON = 0.
- GMAT**[‡] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NONZG containing the nonzero elements of the Jacobian matrix G at XBAR (see USAGE REMARKS). Not referenced when MCON = 0.
- IROWG**[†] [INPUT, INTEGER, ARRAY]
An integer array of length NONZG containing the row indices of the Jacobian

nonzeros, where $0 < \text{IROWG}(k) \leq \text{MCON}$ for $k = 1, \dots, \text{NONZG}$. Not referenced when $\text{MCON} = 0$.

JCOLG[†] [INPUT, INTEGER, ARRAY]
An integer array of length NONZG containing the column indices of the Jacobian nonzeros, where $0 < \text{JCOLG}(k) \leq \text{NDIM}$ for $k = 1, \dots, \text{NONZG}$. Not referenced when $\text{MCON} = 0$.

NONZG[†] [INPUT, INTEGER]
Number of Jacobian nonzeros, where $1 \leq \text{NONZG} \leq \text{NDIM} * \text{MCON}$.

ALGORITHM CONTROL DATA

IFERR[‡] [INPUT, INTEGER]
Function evaluation error flag.

IFERR= 1 When RESVEC and/or CBAR cannot be evaluated.
IFERR= 0 Otherwise.

NFEVAL[‡] [INPUT, INTEGER]
Number of function evaluations. Normally NFEVAL is initialized prior to the first call when $\text{IRVCOM} = -1$, and incremented whenever $\text{IREVRS}(1)$, $\text{IREVRS}(2)$, or $\text{IREVRS}(3)$ are nonzero. It may be appropriate to increment NFEVAL for each perturbation when finite difference gradients are used, or not at all when analytic gradients are used.

WORKING STORAGE

HOLD [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
A real array of length NHOLD containing the hold array (working storage that must not be destroyed by the user).

NHOLD[†] [INPUT/OUTPUT, INTEGER]
Dimension of HOLD array. When $\text{IER} = -127$, or -131 , NEEDED contains the storage needed.

IHOLD [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length NIHOLD containing the integer hold array (working storage that must not be destroyed by the user).

NIHOLD[†] [INPUT/OUTPUT, INTEGER]
Dimension of IHOLD array. When $\text{IER} = -128$, or -132 , NEEDED contains the storage needed.

OUTPUT ARGUMENTS SET BY ALGORITHM BEFORE TERMINATION ($\text{IRVCOM} = 0$)

NEEDED [OUTPUT, INTEGER]
When $\text{IER} = -127$, or -131 , NEEDED contains the required length of HOLD.
When $\text{IER} = -128$, or -132 , NEEDED contains the required length for IHOLD.

IER

[OUTPUT, INTEGER]

Success/error code. Results are not computed for $IER < 0$. For $IER > 0$, results are computed but may be suspect; user inspection is recommended. Note many of the error returns involve optional input quantities as described in INSNLP.

IER=0 Normal termination.

IER=-101 (MCON < 0); the number of constraints is negative.

IER=-102 (NDIM < 1); the number of variables is less than one.

IER=-105 (NITMAX $< \max(1, \text{NITMIN})$); the maximum number of iterations is either less than 1 or less than the minimum number of iterations.

IER=-106 (SLPTOL ≥ 1) or (SLPTOL $\leq 10^{-5}$); the slope tolerance for the line search during optimization is too large or too small.

IER=-107 (IT1MAX < 1); the number of line search steps is less than 1.

IER=-108 (IOFLAG < 0) or (IOFLAG > 30); invalid input for the output control flag.

IER=-109 Invalid input for ALGOPT.

IER=-110 (OBJTOL $\leq 10 \cdot \text{HDMCON}(5)$); objective function tolerance is too small.

IER=-111 (PGDTOL $\leq [\text{HDMCON}(5)]^{\frac{1}{2}}$) or (PGDTOL $> 10^{-2}$); the projected gradient tolerance is too small or too large.

IER=-112 CONTOL $< [\text{HDMCON}(5)]^{\frac{1}{2}}$; the constraint tolerance is too small.

IER=-113 (ISTATC(i) < 0) or (ISTATC(i) > 4); invalid input for constraint status.

IER=-114 (NONZG ≤ 0) or (NONZG $> \text{NDIM} \cdot \text{MCON}$); the number of Jacobian nonzeros is either less than one, or exceeds the size of a dense matrix.

IER=-115 (JCOLG(i) ≤ 0) or (JCOLG(i) $> \text{NDIM}$); invalid input for Jacobian column index array.

IER=-116 (IROWG(i) ≤ 0) or (IROWG(i) $> \text{MCON}$); invalid input for Jacobian row index.

IER=-117 (CUPR(i) $< \text{CLWR}(i)$); constraint upper bound is less than lower bound.

IER=-118 Either (CUPR(i) = CLWR(i)) and (ISTATC(i) $\neq 3$) or (CUPR(i) $\neq \text{CLWR}(i)$) and (ISTATC(i) = 3); constraint status array is not consistent with bounds.

IER=-119	(CUPR(i) $\neq$ CLWR(i)) and (CUPR(i)-CLWR(i) < CON- TOL); constraint bounds are not equal, but differ by less than the constraint tolerance.
IER=-120	(ISTATV(i) < 0) or (ISTATV(i) > 3); invalid input for variable status array.
IER=-121	(JSTRH(1) $\neq$ 1) or (JSTRH(k) $\geq$ JSTRH(k+1)); invalid input for Hessian column start array.
IER=-122	(NZHDIM < NDIM) or (NZHDIM > NDNSH) or (NZHDIM $\neq$ NONZH) where NZHDIM = JSTRH(NDIM+1)-1, and NDNSH = NDIM*(NDIM+1)/2; the number of Hessian nonzeros is less than one, exceeds the size of a dense Hessian, or is not consistent with the input value NONZH.
IER=-123	(IROWH(i) $\leq$ 0) or (IROWH(i) > NDIM); invalid value for Hessian row index.
IER=-124	(XUPR(i) < XLWR(i)); the variable upper bound is less than the lower bound.
IER=-125	Either (XUPR(i) = XLWR(i)) and (ISTATV(i) $\neq$ 3) or (XUPR(i) $\neq$ XLWR(i)) and (ISTATV(i) = 3); the variable status array is not consistent with the bound values.
IER=-126	(XUPR(i) $\neq$ XLWR(i)) and (XUPR(i)-XLWR(i) < CON- TOL); variable bounds are not equal but differ by less than the constraint tolerance.
IER=-127	Real hold array too small; insufficient storage detected in SNLPMN interface. The required storage is specified in NEEDED.
IER=-128	Integer hold array too small; insufficient storage detected in SNLPMN interface. The required storage is specified in NEEDED.
IER=-129	Function error at initial point or during gradient evaluation.
IER=-130	(The number of active constraints > NDIM); check ISTATC(i) and ISTATV(i).
IER=-131	NHOLD too small; insufficient real storage detected in algo- rithm. The required storage is specified in NEEDED.
IER=-132	NIHOLD too small; insufficient integer storage detected in al- gorithm. The required storage is specified in NEEDED.
IER=-133	Rank deficient Jacobian detected on successive iterations.
IER=-134	INSNLP input error; invalid character string displayed.

IER=-135	(IFERR < 0) or (IFERR > 1); invalid value for function error flag.
IER=-136	(SFZTOL ≥ 1) or (SFZTOL $\leq 10^{-5}$); the slope tolerance for the line search during the feasibility phase is too large or too small.
IER=-137	Conflict between user and multifrontal file number; check IPUMF1,...,IPUMF6, IPUDRF, IPUFZF, IPUSTF.
IER=-138	(NEWTON < 0) or (NEWTON > 1); invalid input for Newton method option flag.
IER=-139	(NRES < 0); the number of residuals is negative.
IER=-141	(NONZR ≤ 0) or (NONZR > NDIM*NRES); the number of residual Jacobian nonzeros is either less than one, or exceeds the size of a dense matrix.
IER=-142	(JCOLR(i) ≤ 0) or (JCOLR(i) > NDIM); invalid input for residual Jacobian column index array.
IER=-143	(IROWR(i) ≤ 0) or (IROWR(i) > NRES); invalid input for residual Jacobian row index.
IER=-144	A row of the residual Jacobian is identically (structurally) zero. If a residual does not depend on any variable, the sum of squares can be reformulated omitting the constant contribution.
IER=-145	(IROWH(JSTRH(i)) $\neq i$) for some value of i; Hessian diagonal elements are incorrect.
IER=-146	ALGOPT = 'LLSQ' and NRES ≤ 0 ; the linear least squares option has been specified with no least squares residuals.
IER=-147	Unexpected error; check storage allocation and verify that the HOLD and IHOLD arrays are unchanged during reverse communication calls.
IER=-148	IRVCOM > 1.
IER=-149	Incorrect value for QPOPTN.
IER=-150	QPOPTN = DENSE; invalid option for least squares problems.
IER=-151	A constraint is inconsistent (either $c_i(x) < c_{Li}$ or $c_{Ui} < c_i(x)$) and cannot be changed because the corresponding row of the Jacobian is zero.
IER=-152	Incorrect value for KTOPTN.
IER=-153	I/O error, probably caused by insufficient disk space.

IER=+101	Weak solution found (multipliers near zero).
IER=+102	Number of equality constraints = NDIM and ALGOPT $\neq$ 'F'.
IER=+103	Maximum number of consecutive function errors.
IER=+104	Maximum number of function evaluations.
IER=+105	Small step termination in optimization phase; suboptimal feasible point found.
IER=+106	Maximum number of iterations in optimization phase.
IER=+107	Maximum number of iterations in feasibility phase.
IER=+108	Feasible point not found.
IER=+109	Maximum number of interval halves in line search.
IER=+110	QP algorithm terminated because either the Hessian diagonal reached its maximum value or the merit function slope condition was violated; suboptimal feasible point found.
IER=+111	Projected gradient calculation failed; constraints may be degenerate.
IER=+112	Calculation of first order multiplier estimates failed; constraints may be degenerate.
IER=+113	Suboptimal feasible point found.
IER=+114	Schur-complement QP algorithm (subroutine SHURQP) failed with unexpected error.
IER=+115	CONTOL > OBJTOL; convergence tolerances may be inappropriate.
IER=+116	Uphill direction detected in line search.
IER=+117	Reduced objective function is linear.
IER=+118	Both $c_{Li} \leq -.01/\text{HDMCON}(5)$ and $c_{Ui} \geq .01/\text{HDMCON}(5)$; constraints ignored.
IER=+119	Terminate after diagnostic line search.

IER=+120 (IHESHN > 0) and (NEWTON = 0); recursive Hessian updates may be skipped causing poor algorithm performance.

†	Variable set when IRVCOM=-1, and not changed by user thereafter
‡	Variable set when IRVCOM=+1, as required by IREVRS(*)
#	Variable optionally set when IRVCOM=-1, and not changed by user thereafter

USAGE REMARKS

A *linear least squares problem* is a special form of the general problem. In particular, it is necessary to compute the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the least squares objective function

$$f(x) = \frac{1}{2}(Rx - d)^T(Rx - d) = \frac{1}{2}\| (Rx - d) \|^2$$

where d is an ℓ -vector of data, subject to the m linear constraints

$$c_L \leq Gx \leq c_U$$

and the simple bounds

$$x_L \leq x \leq x_U.$$

The $\ell \times n$ residual Jacobian matrix R and the $m \times n$ Jacobian matrix G are constant. The residual Hessian matrix $V = 0$ for this case. For this application the user should set ALGOPT = 'LLSQ'. Since the linear least squares problem can be solved with a single QP iteration;

- (1) the Jacobian matrices R and G need only be computed once and,
- (2) the residual Hessian matrix V does not need to be computed at all.

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine SNLPLS. All optional inputs are specified by calls to subroutine INSNLP. It should be emphasized that it is *not necessary* to call INSNLP if the default values are appropriate.

OPTIONAL PRINTOUT

The level of printed output from SNLPLS is controlled by the variables IOFLAG and IPUNLP which are set by calls to INSNLP. The standard level of output which corresponds to IOFLAG = 10 is the *default* for SNLPLS. A complete description of the iteration output options is found in Section 2.3.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2, x_3, x_4)$ to minimize the least squares objective function

$$f(x) = \frac{1}{2}r^T(x)r(x)$$

where the residual vector is

$$r = \begin{bmatrix} 10(x_2 - x_1^2) \\ (1 - x_1) \\ 10(x_4 - x_3^2) \\ (1 - x_3) \end{bmatrix}$$

subject to the two constraints

$$x_1 x_2 \leq \frac{1}{2} \qquad -\frac{1}{2} \leq x_1 + x_3 \leq \frac{1}{2}$$

and the bounds

$$-2 \leq x_k \leq 2$$

for $k = 1, 2, 3, 4$ beginning at the initial point $x = (-1, 2, -1, 2)$. For this example the residual Jacobian matrix is

$$R = \begin{bmatrix} \nabla_x r_1^T \\ \nabla_x r_2^T \\ \nabla_x r_3^T \\ \nabla_x r_4^T \end{bmatrix} = \begin{bmatrix} -20x_1 & 10 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & -20x_3 & 10 \\ 0 & 0 & -1 & 0 \end{bmatrix}.$$

and the constraint Jacobian matrix is

$$G = \begin{bmatrix} \nabla_x c_1^T \\ \nabla_x c_2^T \end{bmatrix} = \begin{bmatrix} x_2 & x_1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}.$$

It is also necessary to supply the residual Hessian matrix

$$V = r_1 \nabla_x^2 r_1 + r_2 \nabla_x^2 r_2 + r_3 \nabla_x^2 r_3 + r_4 \nabla_x^2 r_4 - \lambda_1 \nabla_x^2 c_1 - \lambda_2 \nabla_x^2 c_2$$

where

$$\nabla_x^2 r_1 = \begin{bmatrix} -20 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

and

$$\nabla_x^2 r_3 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -20 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

with

$$\nabla_x^2 c_1 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

and $\nabla_x^2 r_2 = \nabla_x^2 r_4 = \nabla_x^2 c_2 = 0$. Collecting results yields the expression

$$V = \begin{bmatrix} -20r_1 & -\lambda_1 & 0 & 0 \\ -\lambda_1 & 0 & 0 & 0 \\ 0 & 0 & -20r_3 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Note: this example also illustrates the use of the “Terse Output” option by the call to INSNLP with “IOFLAG = 1”.

SAMPLE PROGRAM (snlpls.cd1)⁵

```
      PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE SPARSE NONLINEAR
C      PROGRAMMING ALGORITHM SNLPLS
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      PARAMETER (NDIM=4, MCON=2)
C      PARAMETER (NONZR=6, NRES=4)
C      PARAMETER (NONZH=6, NONZG=4)
C      PARAMETER (NHOLD=2000,NIHOLD=2000)
C
C      DIMENSION IREVRS(5),XBAR(NDIM),XLWR(NDIM),XUPR(NDIM),
$      ISTATV(NDIM),VEGNU(NDIM),RESVEC(NRES),RMAT(NONZR),
$      IROWR(NONZR),JCOLR(NONZR),HMAT(NONZH),
$      IROWH(NONZH),JSTRH(NDIM+1),CBAR(MCON),CLWR(MCON),
$      CUPR(MCON),ISTATC(MCON),VECLAM(MCON),
$      GMAT(NONZG),IROWG(NONZG),JCOLG(NONZG)
C      DIMENSION HOLD(NHOLD),IHOLD(NIHOLD)
C
C      INITIALIZE REVERSE COMMUNICATION CONTROL FLAG
C
C      IRVCOM = -1
C
C      -----
C
C      INDEPENDENT VARIABLE DATA
C
C      ----INITIAL GUESS
C      XBAR(1) = -1.0D0
C      XBAR(2) =  2.0D0
C      XBAR(3) = -1.0D0
C      XBAR(4) =  2.0D0
C
C      ----BOUNDS
C      XLWR(1:NDIM) = -2.D0
C      XUPR(1:NDIM) =  2.D0
C
C      ----VARIABLE STATUS
C      ISTATV(1:NDIM) = 0
C
C      -----
```

⁵sosexample.tar


```

C          OBJECTIVE FUNCTION DATA
C
C      ----RESIDUAL JACOBIAN SPARSITY PATTERN (ROW INDICES)
C      IROWR(1) = 1
C      IROWR(2) = 2
C      IROWR(3) = 1
C      IROWR(4) = 3
C      IROWR(5) = 4
C      IROWR(6) = 3
C      ----RESIDUAL JACOBIAN SPARSITY PATTERN (COLUMN INDICES)
C      JCOLR(1) = 1
C      JCOLR(2) = 1
C      JCOLR(3) = 2
C      JCOLR(4) = 3
C      JCOLR(5) = 3
C      JCOLR(6) = 4
C      ----HESSIAN SPARSITY PATTERN (ROW INDICES)
C      IROWH(1) = 1
C      IROWH(2) = 2
C      IROWH(3) = 2
C      IROWH(4) = 3
C      IROWH(5) = 4
C      IROWH(6) = 4
C      ----HESSIAN SPARSITY PATTERN (COLUMN START INDICES)
C      JSTRH(1) = 1
C      JSTRH(2) = 3
C      JSTRH(3) = 4
C      JSTRH(4) = 6
C      JSTRH(5) = 7
C
C      -----
C
C          CONSTRAINT DATA
C
C      ----BOUNDS
C      CLWR(1) = -1.D0/HDMCON(5)
C      CUPR(1) = .5D0
C      CLWR(2) = -.5D0
C      CUPR(2) = .5D0
C      ----CONSTRAINT STATUS
C      ISTATC(1:MCON) = 0
C      ----JACOBIAN SPARSITY PATTERN (ROW INDICES)
C      IROWG(1) = 1
C      IROWG(2) = 2
C      IROWG(3) = 1
C      IROWG(4) = 2
C      ----JACOBIAN SPARSITY PATTERN (COLUMN INDICES)
C      JCOLG(1) = 1

```

```
      JCOLG(2) = 1
      JCOLG(3) = 2
      JCOLG(4) = 3
C
C -----
C
C      INITIALIZE FUNCTION EVALUATION COUNT
C
      NFEVAL = 0
C
C      SET OPTIONAL INPUTS
C
      CALL INSNLP('IOFLAG=1')
C
C -----
C
1000 CONTINUE
C
      CALL SNLPLS( IRVCOM ,IREVRS ,XBAR   ,XLWR   ,XUPR   ,ISTATV
$                ,VECNU  ,NDIM   ,RESVEC ,NRES   ,RMAT   ,IROWR
$                ,JCOLR  ,NONZR  ,HMAT   ,IROWH  ,JSTRH  ,NONZH
$                ,CBAR   ,CLWR   ,CUPR   ,ISTATC ,MCON   ,VECLAM
$                ,GMAT   ,IROWG  ,JCOLG  ,NONZG  ,IFERR  ,NFEVAL
$                ,HOLD   ,NHOLD  ,IHOLD  ,NIHOLD ,NEEDED ,IERNLP )
C
C      REVERSE COMMUNICATION TERMINATION TEST
C
      IF(IRVCOM.EQ.0) GO TO 10000
C
C -----
C
C      SET FUNCTION EVALUATION ERROR FLAG
C
      IFERR = 0
C
C      >>>>>FUNCTION EVALUATION REQUEST
C
      IF(IREVRS(1).EQ.1) THEN
C
          RESVEC(1) = 10.D0*(XBAR(2) - XBAR(1)**2)
          RESVEC(2) = (1.D0 - XBAR(1))
          RESVEC(3) = 10.D0*(XBAR(4) - XBAR(3)**2)
          RESVEC(4) = (1.D0 - XBAR(3))
C
          CBAR(1) = XBAR(1)*XBAR(2)
          CBAR(2) = XBAR(1) + XBAR(3)
          NFEVAL = NFEVAL + 1
C
```

```
      ENDIF
C
C      >>>>>GRADIENT EVALUATION REQUEST
C
      IF(IREVRS(2).EQ.1) THEN
C
      RMAT(1) = -20.D0*XBAR(1)
      RMAT(2) = -1.D0
      RMAT(3) = 10.D0
      RMAT(4) = -20.D0*XBAR(3)
      RMAT(5) = -1.D0
      RMAT(6) = 10.D0
C
      GMAT(1) = XBAR(2)
      GMAT(2) = 1.D0
      GMAT(3) = XBAR(1)
      GMAT(4) = 1.D0
C
      ELSEIF(IREVRS(2).EQ.2) THEN
C
      RMAT(1) = -20.D0*XBAR(1)
      RMAT(2) = -1.D0
      RMAT(3) = 10.D0
      RMAT(4) = -20.D0*XBAR(3)
      RMAT(5) = -1.D0
      RMAT(6) = 10.D0
C
      GMAT(1) = XBAR(2)
      GMAT(2) = 1.D0
      GMAT(3) = XBAR(1)
      GMAT(4) = 1.D0
C
      ENDIF
C
C      >>>>>HESSIAN EVALUATION REQUEST
C
      IF(IREVRS(3).EQ.1) THEN
C
C      DIAGONAL ELEMENTS
C
      HMAT(1) = -20.D0*RESVEC(1)
      HMAT(3) = 0.D0
      HMAT(4) = -20.D0*RESVEC(3)
      HMAT(6) = 0.D0
C
      ELSEIF(IREVRS(3).EQ.2) THEN
C
C      OFF-DIAGONAL ELEMENTS
```

```
C
      HMAT(2) = -VECLAM(1)
      HMAT(5) = 0.D0
C
      ENDIF
C
      >>>>SYSTEM PRINT (OUTPUT) REQUEST
C
      IF(IREVRS(4).EQ.1) THEN
C
        FOBJ = 0.D0
        DO I=1,NRES
          FOBJ = FOBJ + RESVEC(I)**2
        ENDDO
        WRITE(6,1001) 'OBJECTIVE  ',.5D0*FOBJ
        WRITE(6,1001) 'RESIDUALS  ',(RESVEC(II),II=1,NRES)
        WRITE(6,1001) 'VARIABLES  ',(XBAR(II),II=1,NDIM)
        WRITE(6,1001) 'CONSTRAINTS',(CBAR(II),II=1,MCON)
C
      ENDIF
C
      -----
C
      2000 CONTINUE
C
      END OF REVERSE COMMUNICATION PROCESSING.  RETURN TO SNLPLS
C
      GO TO 1000
C
      10000 CONTINUE
C
      TERMINATE PROCESSING
C
      IF(IERNLP.EQ.0) THEN
C
        NORMAL TERMINATION---SOLUTION FOUND
C
        WRITE(6,1001) 'SOLUTION  '
C
        FOBJ = 0.D0
        DO I=1,NRES
          FOBJ = FOBJ + RESVEC(I)**2
        ENDDO
        WRITE(6,1001) 'OBJECTIVE  ',.5D0*FOBJ
        WRITE(6,1001) 'RESIDUALS  ',(RESVEC(II),II=1,NRES)
        WRITE(6,1001) 'VARIABLES  ',(XBAR(II),II=1,NDIM)
        WRITE(6,1001) 'CONSTRAINTS',(CBAR(II),II=1,MCON)
C
```

```

      ELSEIF(IERNLP.LT.0) THEN
C
C      ABNORMAL TERMINATION---FATAL ERROR
C
      PRINT *, 'FATAL ERROR; IERNLP =', IERNLP
C
      ELSE
C
C      ABNORMAL TERMINATION---WARNING.  EVALUATION OF RESULTS RECOMMENDED!
C
      PRINT *, 'WARNING MESSAGE; IERNLP =', IERNLP
C
      ENDIF
C
1001 FORMAT(5X,A11,5X,4G17.8)
C
      STOP
      END

```

OUTPUT FROM SAMPLE PROGRAM

Feasible Point							
It	Qit	Nkt	Ndof	KT Cond	Step	Norm p	Violtn
1	4	1	1	1.0E+00	1.0E+00	1.1E+00	1.5E+00
2	0	0	1	1.0E+00	1.0E+00	1.1E+00	0.0E+00

Optimization											
It	Qit	Nkt	Ndof	KT Cond	Step	Norm p	Violtn	Levnbrg	Penalty	Norm Pg	Merit Function
(1)	4	9	3	7.0E+00	1.0E+00	5.6E-01	0.0E+00	1.0E+00	1.5E-08	4.0E+00	3.769531E+02
(2)	2	3	3	2.2E+02	1.0E+00	5.3E-01	0.0E+00	1.0E+00	1.5E-08	1.5E+02	2.389929E+02
(3)	2	3	3	1.6E+02	1.0E+00	4.9E-01	8.9E-16	1.0E+00	1.5E-08	1.2E+02	1.371212E+02
(4)	2	3	3	1.1E+02	1.0E+00	4.4E-01	8.9E-16	1.0E+00	1.5E-08	8.2E+01	6.807597E+01
(5)	2	3	3	6.9E+01	1.0E+00	3.5E-01	1.1E-15	1.0E+00	1.5E-08	5.1E+01	2.724357E+01
(6)	2	3	3	3.4E+01	1.0E+00	2.4E-01	1.1E-15	1.0E+00	1.5E-08	2.6E+01	8.132949E+00
(7)	1	3	4	1.2E+01	1.0E+00	2.1E-01	1.3E-15	1.0E+00	1.5E-08	8.8E+00	2.336650E+00
(8)	2	3	3	1.2E+01	1.0E+00	6.0E-01	0.0E+00	1.0E+00	1.5E-08	1.2E+00	1.349951E+00
(9)	3	1	3	1.5E+01	1.8E-01	4.8E-01	0.0E+00	1.0E+00	1.5E-08	9.5E-01	1.155189E+00
(10)	3	1	3	1.4E+01	3.0E-01	3.3E-01	0.0E+00	1.0E+00	1.5E-08	8.2E-01	9.036581E-01
(11)	3	1	3	1.3E+02	1.0E+00	7.7E-02	0.0E+00	1.0E+00	1.5E-08	5.5E+00	8.614925E-01
(12)	1	1	3	1.1E+01	1.0E+00	7.7E-04	0.0E+00	9.9E-03	1.5E-08	5.4E-02	5.625293E-01
(13)	1	1	3	9.7E+00	1.0E+00	7.6E-08	0.0E+00	9.8E-07	1.5E-08	5.4E-06	5.625000E-01
(14)	0	0	3	9.7E+00	1.0E+00	7.6E-08	0.0E+00	9.8E-07	1.5E-08	5.3E-14	5.625000E-01

```

-----
|
|               SPRNLP ALGORITHM PERFORMANCE STATISTICS
|
| Total CPU Time.....+5.521286E-02
|
| Number of Function Calls.....17
| Number of Gradient Calls.....16
| Number of Hessian Calls.....13
| Total Number of Function Evaluations.....17
|
| Storage Needed in HOLD Array.....428
|

```

	Storage Needed in IHOLD Array.....34				

SOLUTION					
OBJECTIVE	0.56250000				
RESIDUALS	-0.52735594E-14	0.75000000	-0.52735594E-14	0.75000000	
VARIABLES	0.25000000	0.62500000E-01	0.25000000	0.62500000E-01	
CONSTRAINTS	0.15625000E-01	0.50000000			

SHURQP: Sparse Quadratic Programming

PURPOSE

SHURQP computes the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the quadratic function

$$c^T x + \frac{1}{2} x^T H x$$

subject to the m linear constraints

$$b_L \leq Ax \leq b_U,$$

and the simple bounds

$$x_L \leq x \leq x_U$$

Equality constraints are imposed by setting $b_{Li} = b_{Ui}$ and variables can be fixed by setting $x_{Li} = x_{Ui}$. SHURQP works under the assumption that the quadratic programming problem has a unique, finite solution. Thus, it is assumed that the projected Hessian matrix is positive definite at the solution. The matrix of first derivatives (the Jacobian) and the matrix of second derivatives (Hessian) are represented in a sparse format consistent with that used in BCSLIB-EXT [1].

RELATED SUBPROGRAMS

INSNLP Sparse Nonlinear Programming Input Procedure

SBRLPQ Sparse Barrier Quadratic Programming

METHOD

A *sparse Schur-Complement* Method is used to solve the *quadratic programming* (QP) problem. The quadratic programming algorithm requires the Hessian matrix H and the Jacobian matrix A , both which must be specified in a sparse format. The Schur-complement method is efficient because it is necessary to factor the large sparse symmetric indefinite KT matrix only once. Subsequent changes to the active set of constraints can be computed using a “solve” with the original large sparse system and a solve with the small dense Schur-complement. The multifrontal algorithm described in BCSLIB-EXT [1] is used to factor and solve this large sparse linear system. The original Schur-complement algorithm proposed by Gill, Murray, Saunders, and Wright requires factorization of a linear system including active and inactive inequality constraints and can be exercised by setting the algorithm input KTOPTN to “large.” The default method (corresponding to KTOPTN “small”) factors a linear system which includes only the active constraints.

WARNING

The projected Hessian matrix is assumed to be positive definite at the solution, i.e. with the correct active constraints. If this is not true, the software will detect an incorrect value for the matrix inertia. When this occurs the algorithm will terminate and no attempt is made to alter the problem definition. The software will not solve indefinite quadratic programming problems.

The SHURQP software uses internal common blocks to communicate information. To avoid conflict with these internal commons the user should not have commons with the following names:

NPSPRR, NPSPRI, NPSPRC, STATIS, KONSTN, NPSALG, NPSDFL, NLPERR, PERCOM, ITEREF, and CLKCOM.

USAGE

```

INTEGER      NDIM, MCON, NZHDIM, NZADIM, NHOLD, NIHOLD, NEEDED, IPU
INTEGER      IPC, ISTART, IERSQP
INTEGER      ISTATV(NDIM), IROWH(NZHDIM), JSTRH(NDIM+1)
INTEGER      ISTATC(MCON), IROWA(NZADIM), JCOLA(NZADIM), IHOLD(NIHOLD)

DOUBLE PRECISION      XVEC(NDIM), XLWR(NDIM), XUPR(NDIM), VARMLT(NDIM)
DOUBLE PRECISION      CVCT(NDIM), GZERO(NDIM), HMAT(NZHDIM), BUPR(MCON)
DOUBLE PRECISION      BLWR(MCON), CONMLT(MCON), AMAT(NZADIM), HOLD(NHOLD)
DOUBLE PRECISION      QUAD, BIGBND, CNDNUM

CALL SHURQP(NDIM,MCON,HMAT,IROWH,JSTRH,NZHDIM,CVCT,GZERO,
$    QUAD,AMAT,IROWA,JCOLA,NZADIM,BUPR,BLWR,CONMLT,ISTATC,
$    XVEC,XUPR,XLWR,VARMLT,ISTATV,HOLD,NHOLD,IHOLD,NIHOLD,
$    NEEDED,BIGBND,IPU,IPC,ISTART,CNDNUM,IERSQP)

```

ARGUMENTS

PROBLEM DATA

NDIM [INPUT, INTEGER]
Number of variables n , where $1 \leq \text{NDIM}$.

MCON [INPUT, INTEGER]
Number of constraints m , where $0 \leq \text{MCON}$.

OBJECTIVE FUNCTION DATA

HMAT [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NONZH containing the nonzero elements of Hessian matrix H of the quadratic function. Since the Hessian is symmetric, only the lower triangular part is input (see USAGE REMARKS). Storage for the diagonal elements must be provided even if they are zero.

IROWH [INPUT, INTEGER, ARRAY]
An integer array of length NONZH containing the row indices of the Hessian nonzeros, where $0 < \text{IROWH}(k) \leq \text{NDIM}$ for $k = 1, \dots, \text{NONZH}$. The diagonal elements must be specified, i.e. $\text{IROWH}(\text{JSTRH}(k)) = k$ for $k = 1, \dots, \text{NDIM}$.

JSTRH [INPUT, INTEGER, ARRAY]
An integer array of length NDIM+1 containing the column start indices of nonzeros, where $\text{JSTRH}(1) = 1$, $\text{JSTRH}(\text{NDIM}+1) = \text{NONZH} + 1$, and $\text{JSTRH}(k) < \text{JSTRH}(k+1)$ for $k = 1, \dots, \text{NDIM}$.

NZHDIM	[INPUT, INTEGER] Dimension of Hessian nonzero arrays, where $n \leq \text{NONZH} \leq n(n+1)/2$, and $\text{NZHDIM} \geq \text{NONZH} = \text{JSTRH}(\text{NDIM}+1) - 1$.
CVCT	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the linear term c in the objective function.
GZERO	[OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the gradient of the objective function at the initial point XVEC.
QUAD	[OUTPUT, DOUBLE PRECISION] The objective function evaluated at the solution XVEC.

CONSTRAINT DATA

AMAT	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NONZA containing the nonzero elements of the Jacobian matrix A . Not referenced when MCON = 0.
IROWA	[INPUT, INTEGER, ARRAY] An integer array of length NONZA containing the row indices of the Jacobian nonzeros, where $0 < \text{IROWA}(k) \leq \text{MCON}$ for $k = 1, \dots, \text{NONZA}$. Not referenced when MCON = 0.
JCOLA	[INPUT/OUTPUT, INTEGER, ARRAY] When ISTART = 2, an integer array of length NONZA containing the column indices of the Jacobian nonzeros, where $0 < \text{JCOLA}(k) \leq \text{NDIM}$ for $k = 1, \dots, \text{NONZA}$. When ISTART < 2, an integer array of length NDIM+1 containing the column start indices of nonzeros, where $\text{JCOLA}(1) = 1$, $\text{JCOLA}(\text{NDIM}+1) = \text{NONZA} + 1$, and $\text{JCOLA}(k) < \text{JCOLA}(k+1)$ for $k = 1, \dots, \text{NDIM}$. When ISTART = 2, the output array JCOLA is in column format. Not referenced when MCON = 0.
NZADIM	[INPUT, INTEGER] Dimension of Jacobian nonzeros, where $\max(\text{NDIM}+1, \text{NONZA}) \leq \text{NZADIM} \leq \text{NDIM} * \text{MCON}$.
BUPR	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON containing the constraint upper bounds b_U . For constraints that have no upper bound set $b_{U_i} \geq \text{BIGBND}$. Not referenced when MCON = 0.
BLWR	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON containing the constraint lower bounds b_L . For constraints that have no lower bound set $b_{L_i} \leq -\text{BIGBND}$. Not referenced when MCON = 0.
CONMLT	[OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON of Lagrange multipliers, λ , for the constraints. Not referenced when MCON = 0.

ISTATC	[INPUT/OUTPUT, INTEGER, ARRAY] An integer array of length MCON indicating the constraint status. On input ISTATC defines an estimate of the active constraints, and on output ISTATC defines the constraints active at the solution.
ISTATC(i)= 0	Free (inactive) inequality, i.e. $b_{Li} < b_i(x) < b_{Ui}$.
ISTATC(i)= 1	Constraint lower bound is active, i.e. $b_{Li} = b_i(x)$.
ISTATC(i)= 2	Constraint upper bound is active, i.e. $b_i(x) = b_{Ui}$.
ISTATC(i)= 3	Equality.
ISTATC(i)= 4	Ignored constraint.
ISTATC(i)= -1	Constraint lower bound is violated, i.e. $b_i(x) < b_{Li}$.
ISTATC(i)= -2	Constraint upper bound is violated, i.e. $b_{Ui} < b_i(x)$.

When ISTATC(i) = 3 it is necessary that $b_{Li} = b_{Ui}$, where $b_{Li} = \text{BLWR}(i)$ and $b_{Ui} = \text{BUPR}(i)$. When ISTATC(i) $\neq 3$, it is recommended that $b_{Li} < b_{Ui}$ with $|b_{Li} - b_{Ui}| > [\text{HDMCON}(5)]^{\frac{1}{2}}$. Not referenced when MCON = 0.

INDEPENDENT VARIABLE DATA

XVEC	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM of variable values x . An initial guess must be input. On output, XVEC contains the solution, provided IER = 0.
XUPR	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the variable upper bounds x_U . For variables that have no upper bound set $x_{Ui} \geq \text{BIGBND}$.
XLWR	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the variable lower bounds x_L . For variables that have no lower bound set $x_{Li} \leq -\text{BIGBND}$.
ISTATV	[INPUT/OUTPUT, INTEGER, ARRAY] An integer array of length NDIM indicating variable status. On input ISTATV defines an estimate of the active variable bounds, and on output ISTATV defines the bounds active at the solution.
ISTATV(i)= 0	Variable is strictly feasible, i.e. $x_{Li} < x_i < x_{Ui}$.
ISTATV(i)= 1	Variable lower bound is active, i.e. $x_{Li} = x_i$.
ISTATV(i)= 2	Variable upper bound is active, i.e. $x_i = x_{Ui}$.
ISTATV(i)= 3	Fixed permanently.
ISTATV(i)= -1	Variable lower bound is violated, i.e. $x < x_{Li}$.

ISTATV(i) = -2 Variable upper bound is violated, i.e. $x_{Ui} < x$.

When ISTATV(i) = 3 it is necessary that $x_{Li} = x_{Ui}$, where $x_{Li} = \text{XLWR}(i)$ and $x_{Ui} = \text{XUPR}(i)$. When ISTATV(i) $\neq$ 3, it is recommended that $x_{Li} < x_{Ui}$ with $|x_{Li} - x_{Ui}| > [\text{HDMCON}(5)]^{\frac{1}{2}}$.

VARMLT [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
An array of length NDIM of bound multipliers ν .

ALGORITHM CONTROL DATA

HOLD [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
A real array of length NHOLD containing the hold array. This working storage must not be destroyed by the user between a cold and a warm start.

NHOLD [INPUT/OUTPUT, INTEGER]
Dimension of HOLD array. When IER = -1014 NEEDED contains the storage needed.

IHOLD [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length NIHOLD containing the integer hold array (working storage that must not be destroyed by the user).

NIHOLD [INPUT/OUTPUT, INTEGER]
Dimension of IHOLD array. When IER = -1019, NEEDED contains the storage needed.

NEEDED [OUTPUT, INTEGER]
When IER = -1014, NEEDED contains the required length of HOLD. When IER = -1019, NEEDED contains the required length for IHOLD.

BIGBND [INPUT, DOUBLE PRECISION]
Big bound value. A suggested value for BIGBND = .01/HDMCON(5). See Appendix B for HDMCON.

IPU [INPUT, INTEGER]
Output Unit Number.

IPC [INPUT, INTEGER]
Output Control Flag.

IPC = 0 No output.

IPC > 100 Debug output

CNDNUM [OUTPUT, DOUBLE PRECISION]
Condition number of KKT system.

IER

[OUTPUT, INTEGER]

Success/error code. Results are not computed for $IER < 0$. For $IER > 0$, results are computed but may be suspect; user inspection is recommended. Note some of the error returns involve optional input quantities specified using INSNLP.

IER=0 Normal termination.

IER=-1001 (NDIM < 1); the number of variables is less than one.

IER=-1002 (MCON < 0); the number of constraints is negative.

IER=-1003 Warm or Hot Start not preceded by a cold start. ISTART must be positive on first call.

IER=-1004 (JSTRH(1) $\leq$ 0) or (JSTRH(k) $\geq$ JSTRH($k+1$)); invalid input for Hessian column start array.

IER=-1005 (NONZH $\leq$ 0) or (NONZH > NDNSH) or (NONZH > NZHDIM) where NONZH = JSTRH(NDIM+1)-1, and NDNSH = NDIM*(NDIM+1)/2; the number of Hessian nonzeros is less than one, exceeds the size of a dense Hessian, or is not consistent with the input dimension NZHDIM.

IER=-1006 (IROWH(i) $\leq$ IDIAG) or (IROWH(i) > NDIM) where IDIAG = row index of diagonal; invalid value for Hessian row index. Hessian must be lower triangular.

IER=-1007 Invalid input for Jacobian column index array. When ISTART = 1, valid input requires (JCOLA(1) = 1) and (JCOLA(i) $\leq$ JCOLA(i+1)). When ISTART = 2, valid input requires (JCOLA(i) > 0) and (JCOLA(i) $\leq$ NDIM).

IER=-1008 (NONZA $\leq$ 0) or (NONZA > NDIM*MCON) or (NONZA > NZADIM) where NONZA = JCOLA(NDIM+1)-1; the number of Jacobian nonzeros is either less than one, exceeds the size of a dense matrix, or is inconsistent with the input dimension NZADIM.

IER=-1009 (IROWA(i) $\leq$ 0) or (IROWA(i) > MCON); invalid input for Jacobian row index.

IER=-1010 (BUPR(i) < BLWR(i)) or (BUPR(i) $\geq$ BIGBND) and (BLWR(i) $\leq$ -BIGBND); constraint upper bound is less than lower bound, or both bounds are infinite.

IER=-1011 (XUPR(i) < XLWR(i)); the variable upper bound is less than the lower bound.

IER=-1012	(MEQUAL + NFIXVR > NDIM) where MEQUAL is the number of equality constraints, and NFIXVR is the number of fixed variables; there are no degrees of freedom.
IER=-1013	(IPC < 0) and (IPU > 0); invalid output control flag.
IER=-1014	NHOLD too small; insufficient real storage detected in SHURQP. The required storage is specified in NEEDED.
IER=-1016	A row of the constraint matrix is structurally zero. When IPC > 100, the indices of the zero rows are printed.
IER=-1017	Unexpected error in CNVRTR. Check storage allocation and verify that the HOLD and IHOLD arrays are unchanged between calls with warm start.
IER=-1018	Unexpected error in DCNVRT. Check storage allocation and verify that the HOLD and IHOLD arrays are unchanged between calls with warm start.
IER=-1019	NIHOLD too small; insufficient integer storage detected in SHURQP. The required storage is specified in NEEDED.
IER=-1020	Inconsistent linear constraints.
IER=-1021	Incorrect value for KTOPTN. Defaults may need to be set.
IER=-1103	Singular KT matrix detected by multifrontal algorithm (IER = -503).
IER=-1104	Incorrect inertia of KT matrix.
IER=-1105	Unexpected error from multifrontal algorithm.
IER=-1106	Condition number of KT matrix is too large (IRETCD = 12 or 14).
IER=-1107	Incorrect inertia of KT matrix after active set change with warm start (IRETCD = 11).
IER=-1108	Excessive fill during numeric factorization of KT matrix presumably because of ill-conditioning. A larger value of TOLFIL may be required (see subroutine INSNLP).
IER=-1109	Incorrect inertia for Schur-complement with relaxed feasibility tolerance.
IER=-1111	I/O error, probably caused by insufficient disk space.
IER=+1	Weak solution found (multipliers near zero).

IER=+2	Solution found with relaxed constraint tolerance.
IER=+3	Feasible point not found. Penalty parameter $\rho > \rho_{max}$ where $\rho_{max} = .0001/\text{HDMCON}(6)$.
IER=+1017	Maximum number of KT refactorizations. Solution not found.

USAGE REMARKS

Sparse Matrix Input

Hessian Matrix. The user must supply the Hessian matrix H of the quadratic function

$$c^T x + \frac{1}{2} x^T H x.$$

Consider the following example in which $n = 3$:

$$H = \begin{pmatrix} 1.1 & 0 & 3.1 \\ 0 & 2.2 & 0 \\ 3.1 & 0 & 3.3 \end{pmatrix}$$

Since the Hessian is a symmetric matrix only the lower triangular portion must be specified. For this example $\text{NDIM} = 3$ and $\text{NONZH} = 4 = [\text{JSTRH}(\text{NDIM}+1)-1]$. The nonzero values are specified as $\text{HMAT}^* = (1.1, 3.1, 2.2, 3.3)$. The row indices are $\text{IROWH}^* = (1, 3, 2, 3)$, and the column start indices are $\text{JSTRH}^* = (1, 3, 4, 5)$. Notice that the element in row 1, column 3 is not specified. Also observe that the diagonal elements must always be included in the Hessian sparsity pattern *even if the elements are zero*. Additional information on sparse matrix formats is found in BCSLIB-EXT [1].

Jacobian Matrix. The user must specify the nonzero elements in the Jacobian matrix when required. Consider the following example in five variables $n = 5$ and three constraints $m = 3$:

$$A = \begin{pmatrix} 0 & 0 & 1.3 & 0 & 1.5 \\ 2.1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 3.3 & 0 & 0 \end{pmatrix}$$

For this example $\text{NDIM} = 5$, $\text{MCON} = 3$, and the number of nonzeros $\text{NONZA} = 4$. The matrix can be specified in two formats. When $\text{ISTART} = 2$, the Jacobian can be specified in triple format. The nonzero array is $\text{AMAT}^* = (2.1, 1.3, 3.3, 1.5)$. The row indices are $\text{IROWA}^* = (2, 1, 3, 1)$, and the column indices are $\text{JCOLA}^* = (1, 3, 3, 5)$. When $\text{ISTART} = 1$, the Jacobian is defined in column format. Specifically, $\text{JCOLA}^* = (1, 2, 2, 4, 4, 5)$ and both AMAT and IROWA are the same as before. Note that sufficient storage must be provided in the array JCOLA to accommodate at least $\text{NDIM}+1$ values. Additional information on sparse matrix formats is found in BCSLIB-EXT [1].

Unconstrained Problems

When the problem is unconstrained the user must set $\text{MCON} = 0$. The following constraint related quantities will not be referenced: BLWR , BUPR , ISTATC , CONMLT , AMAT , IROWA , JCOLA ,

and NONZA. Since these quantities must appear as arguments in the call to SHURQP, dummy values may be utilized.

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine SHURQP. All optional inputs are specified by calls to subroutine INSNLP. It should be emphasized that it is *necessary* to call INSNLP to set default values however it is *not necessary* to alter the default values if they are appropriate.

OPTIONAL PRINTOUT

The level of printed output from SHURQP is controlled by the variables IPC and IPU. The *default* for SHURQP is to have no output which corresponds to $IPC = 0$.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2, x_3, x_4)$ to minimize the objective function

$$f(x) = c^T x + \frac{1}{2} x^T H x$$

subject to the constraints

$$b_L \leq Ax \leq b_U,$$

where $b_L^T = (-\infty, -.5)$ and $b_U^T = (.5, .5)$. The solution must also satisfy the bounds

$$-2 \leq x_k \leq 2$$

for $k = 1, 2, 3, 4$ and as an initial guess select $x = (-1, 2, -1, 2)$. For this example the vector

$$c^T = [400, 200, 400, 200]$$

and the Jacobian matrix is

$$A = \begin{bmatrix} 2 & -1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}.$$

It is also necessary to supply the Hessian matrix

$$H = \begin{bmatrix} 400 & 1 & 0 & 0 \\ 1 & 200 & 0 & 0 \\ 0 & 0 & 2 & -1 \\ 0 & 0 & -1 & 20 \end{bmatrix}.$$

SAMPLE PROGRAM (shurqp.cd1)⁶

⁶sosexample.tar

```
      PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE SPARSE QUADRATIC
C      PROGRAMMING ALGORITHM SHURQP
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
      PARAMETER (NDIM=4, MCON=2)
      PARAMETER (NONZH=6, NONZA=4, LNJCLA=MAX(NDIM+1, NONZA))
      PARAMETER (NHOLD=2000, NIHOLD=2000)
C
      DIMENSION XVEC(NDIM), XLWR(NDIM), XUPR(NDIM), CVCT(NDIM),
$      GZERO(NDIM), ISTATV(NDIM), VARMLT(NDIM), HMAT(NONZH),
$      IROWH(NONZH), JSTRH(NDIM+1), BLWR(MCON),
$      BUPR(MCON), ISTATC(MCON), CONMLT(MCON), CVEC(MCON),
$      AMAT(NONZA), IROWA(NONZA), JCOLA(LNJCLA)
      DIMENSION HOLD(NHOLD), IHOLD(NIHOLD)
C
      CHARACTER(LEN=20) CHRVAR
C
C -----
C
C      INDEPENDENT VARIABLE DATA
C
C      ----INITIAL GUESS
      XVEC(1) = -1.0D0
      XVEC(2) =  2.0D0
      XVEC(3) = -1.0D0
      XVEC(4) =  2.0D0
C
C      ----BOUNDS
      XLWR(1:NDIM) = -2.D0
      XUPR(1:NDIM) =  2.D0
C
C      ----VARIABLE STATUS
      ISTATV(1:NDIM) = 0
C
C -----
C
C      OBJECTIVE FUNCTION DATA
C
C      ----HESSIAN SPARSITY PATTERN (ROW INDICES)
      IROWH(1) = 1
      IROWH(2) = 2
      IROWH(3) = 2
      IROWH(4) = 3
      IROWH(5) = 4
      IROWH(6) = 4
C
C      ----HESSIAN SPARSITY PATTERN (COLUMN START INDICES)
      JSTRH(1) = 1
```



```

      JSTRH(2) = 3
      JSTRH(3) = 4
      JSTRH(4) = 6
      JSTRH(5) = 7
C     ----DIAGONAL ELEMENTS
      HMAT(1) = 400.DO
      HMAT(3) = 200.DO
      HMAT(4) = 2.DO
      HMAT(6) = 20.DO
C     ----OFF-DIAGONAL ELEMENTS
      HMAT(2) = 1.DO
      HMAT(5) = -1.DO
C
C     ----LINEAR TERM FOR OBJECTIVE FUNCTION
C
      CVCT(1) = 400.DO
      CVCT(2) = 200.DO
      CVCT(3) = 400.DO
      CVCT(4) = 200.DO
C
C -----
C
C     CONSTRAINT DATA
C
C     ----BOUNDS
      BIGBND = 1.DO/HDMCON(5)
C
      BLWR(1) = -BIGBND
      BUPR(1) = .5D0
      BLWR(2) = -.5D0
      BUPR(2) = .5D0
C     ----CONSTRAINT STATUS
      ISTATC(1:MCON) = 0
C     ----JACOBIAN SPARSITY PATTERN (ROW INDICES)
      IROWA(1) = 1
      IROWA(2) = 2
      IROWA(3) = 1
      IROWA(4) = 2
C     ----JACOBIAN SPARSITY PATTERN (COLUMN INDICES)
      JCOLA(1) = 1
      JCOLA(2) = 1
      JCOLA(3) = 2
      JCOLA(4) = 3
C
      AMAT(1) = 2.DO
      AMAT(2) = 1.DO
      AMAT(3) = -1.DO
      AMAT(4) = 1.DO

```

```
C
C
C -----
C
C      SET DEFAULTS:  THIS IS REQUIRED FOR A "STANDALONE" QP CALL
C
C      CALL INSNLP('SPARSE DEFAULT')
C
C      ----QP OUTPUT CONTROL PARAMETERS
C      IPU = 6
C      IPC = 0
C
C      SET OPTIONAL INPUTS
C
C      COLD START; JACOBIAN IN TRIPLE FORMAT
C
C      ISTART = 2
C
C -----
C
C      CALL SHURQP(NDIM,MCON,HMAT,IROWH,JSTRH,NONZH,CVCT,GZERO,
$      QUAD,AMAT,IROWA,JCOLA,NONZA,BUPR,BLWR,CONMLT,ISTATC,
$      XVEC,XUPR,XLWR,VARMLT,ISTATV,HOLD,NHOLD,IHOLD,NIHOLD,
$      NEEDED,BIGBND,IPU,IPC,ISTART,CNDNUM,IERSQP)
C
C -----
C
C      TERMINATE PROCESSING
C
C      IF(IERSQP.EQ.0) THEN
C
C          NORMAL TERMINATION---SOLUTION FOUND
C
C          WRITE(IPU,1001) 'SOLUTION      '
C
C          WRITE(IPU,1001) 'OBJECTIVE    ',QUAD
C
C          COMPUTE AMAT*XVEC = CVEC
C
C          CALL MVPSPR(1,MCON,NDIM,AMAT,IROWA,JCOLA,XVEC,CVEC)
C
C          FINAL STATUS
C
C          WRITE(IPU,1003)
C          DO I = 1,MCON
C              IF(ISTATC(I).EQ.0) THEN
C                  CHRVAR = 'FREE (INACTIVE) INEQUALITY'
C              ELSEIF(ISTATC(I).EQ.1) THEN
```

```

        CHRVAR = 'FIXED ON LOWER BOUND'
    ELSEIF(ISTATC(I).EQ.2) THEN
        CHRVAR = 'FIXED ON UPPER BOUND'
    ELSEIF(ISTATC(I).EQ.3) THEN
        CHRVAR = 'EQUALITY'
    ELSEIF(ISTATC(I).EQ.4) THEN
        CHRVAR = 'IGNORED CONSTRAINT'
    ELSEIF(ISTATC(I).EQ.-1) THEN
        CHRVAR = 'VIOLATES LOWER BOUND'
    ELSEIF(ISTATC(I).EQ.-2) THEN
        CHRVAR = 'VIOLATES UPPER BOUND'
    ENDIF
    WRITE(IPU,1002) I,CVEC(I),CONMLT(I),CHRVAR
ENDDO
C
WRITE(IPU,1004)
DO I = 1,NDIM
    IF(ISTATV(I).EQ.0) THEN
        CHRVAR = 'FREE VARIABLE'
    ELSEIF(ISTATV(I).EQ.1) THEN
        CHRVAR = 'FIXED ON LOWER BOUND'
    ELSEIF(ISTATV(I).EQ.2) THEN
        CHRVAR = 'FIXED ON UPPER BOUND'
    ELSEIF(ISTATV(I).EQ.3) THEN
        CHRVAR = 'FIXED PERMANENTLY'
    ELSEIF(ISTATV(I).EQ.4) THEN
        CHRVAR = 'IGNORED BOUND'
    ELSEIF(ISTATV(I).EQ.-1) THEN
        CHRVAR = 'VIOLATES LOWER BOUND'
    ELSEIF(ISTATV(I).EQ.-2) THEN
        CHRVAR = 'VIOLATES UPPER BOUND'
    ENDIF
    WRITE(IPU,1002) I,XVEC(I),VARMLT(I),CHRVAR
ENDDO
C
ELSEIF(IERSQP.LT.0) THEN
C
C     ABNORMAL TERMINATION---FATAL ERROR
C
    PRINT *, 'FATAL ERROR; IERSQP =', IERSQP
C
ELSE
C
C     ABNORMAL TERMINATION---WARNING.  EVALUATION OF RESULTS RECOMMENDED!
C
    PRINT *, 'WARNING MESSAGE; IERSQP =', IERSQP
C
ENDIF

```

C

```
1001 FORMAT(//,5X,A,T30,4G17.8)
1002 FORMAT(5X,I2,T10,G17.8,T30,G17.8,A)
1003 FORMAT(//,T13,'CONSTRAINT',T33,'MULTIPLIER',T50,'STATUS',/)
1004 FORMAT(//,T15,'VARIABLE',T33,'MULTIPLIER',T50,'STATUS',/)
```

C

```
STOP
END
```

OUTPUT FROM SAMPLE PROGRAM

SOLUTION

OBJECTIVE -652.08592

	CONSTRAINT	MULTIPLIER	STATUS
1	0.50000000	-33.997927	FIXED ON UPPER BOUND
2	-0.50000000	401.32919	FIXED ON LOWER BOUND

	VARIABLE	MULTIPLIER	STATUS
1	-0.16459370	0.	FREE VARIABLE
2	-0.82918740	0.	FREE VARIABLE
3	-0.33540630	0.	FREE VARIABLE
4	-2.00000000	160.33541	FIXED ON LOWER BOUND

DBNLPF: Dense Barrier Nonlinear Programming

PURPOSE

DBNLPF computes the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the objective function

$$f(x)$$

subject to the m nonlinear constraints

$$c_L \leq c(x) \leq c_U$$

and the simple bounds

$$x_L \leq x \leq x_U$$

Equality constraints are imposed by setting $c_{Li} = c_{Ui}$ and variables can be fixed by setting $x_{Li} = x_{Ui}$. DBNLPF works under the assumption that the objective and constraint functions are twice continuously differentiable, and if this is not true algorithm performance is unpredictable. DBNLPF uses a forward communication format and the user must supply a subroutine to compute the values of the objective and constraint functions. First and second derivatives of the user supplied functions are computed numerically. The matrix of second derivatives (Hessian of the Lagrangian) can be constructed using a quasi-Newton recursive estimate or by central differencing. Subroutine DBNLPF calls the sparse barrier nonlinear programming algorithm SBRNLP, and is intended for simplified usage as appropriate for small dense problems.

RELATED SUBPROGRAMS

HDDFDJ	Finite Difference Jacobian
HDDFDH	Finite Difference Hessian
DNLPFW	Dense Nonlinear Programming
SBRNLP	Sparse Barrier Nonlinear Programming
INSNLP	Sparse Nonlinear Programming Input Procedure

METHOD

An *interior point or barrier* method is used to solve the *nonlinear programming* (NLP) problem. DBNLPF provides four alternatives for constructing an approximation to the Hessian matrix which are described in the background section of this document. The default is a *symmetric rank-one* (SR1) approximation, with options for using a BFGS, SSQN, or finite difference estimate instead. DBNLPF constructs all first derivative information using the finite difference procedures HDDFDJ and/or HDDFDH. Finite difference perturbation sizes are selected automatically to balance truncation and roundoff error. Forward difference estimates are utilized when possible for efficiency, and central difference estimates are invoked when necessary to improve accuracy. Although default values are set for all algorithm parameters, INSNLP can be utilized to override the defaults.

WARNING

The objective function and constraints supplied by the user are assumed to be continuous and have continuous first and second derivatives. Nondifferentiable functions can cause unpredictable performance in the optimization algorithm. It should be emphasized that discontinuities in the Hessian matrices can significantly degrade the speed of convergence without producing any other obvious difficulties. The following common sources of error *should be avoided* when evaluating the objective and constraint functions:

- (1) nonsmooth interpolation of data;
- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;
- (3) discontinuous behavior caused by branching for IF tests;
- (4) nondifferentiable functions such as ABS, MAX, and MIN.

The user must provide a procedure to evaluate the objective and constraint functions. If it is not possible to evaluate the functions (for example, $c(x) = 1/x$ at $x = 0$) then set IFERR = 1 and exit the function evaluation process. Otherwise, set IFERR = 0 when the functions are evaluated. The function error flag should be used to avoid computational difficulties, but never in lieu of an explicit constraint.

The DBNLPF software uses internal common blocks to communicate information. To avoid conflict with these internal commons the user should not have commons with the following names: NPSPRR, NPSPRI, NPSPRC, STATIS, KONSTN, NPSALG, NPSDFL, NLPERR, PERCOM, ITEREF, and CLKCOM.

USAGE

```

INTEGER      IER, NDIM, MCON, NHOLD, NIHOLD, NEEDED
INTEGER      ISTATV(NDIM), ISTATC(MCON), IHOLD(NIHOLD)

DOUBLE PRECISION      FBAR, XBAR(NDIM), XLWR(NDIM), XUPR(NDIM), VECNU(NDIM)
DOUBLE PRECISION      CBAR(MCON), CLWR(MCON), CUPR(MCON)
DOUBLE PRECISION      VECLAM(MCON), HOLD(NHOLD)

EXTERNAL      FUNBOX

CALL INSNLP('BARRIER DEFAULT')
CALL DBNLPF(FUNBOX, XBAR, XLWR, XUPR, ISTATV, VECNU, NDIM,
$          FBAR, CBAR, CLWR, CUPR, ISTATC, VECLAM, MCON,
$          HOLD, NHOLD, IHOLD, NIHOLD, NEEDED, IER)

```

ARGUMENTS

FUNCTION EVALUATION PROCEDURE

FUNBOX [INPUT, SUBROUTINE]
 Name of user-supplied subroutine to evaluate objective and constraint functions;
 see USER-SUPPLIED SUBPROGRAM

INDEPENDENT VARIABLE DATA

XBAR	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] A double precision array of length NDIM of current variable values x . An initial guess must be input. The final point is the output.
XLWR	[INPUT, DOUBLE PRECISION, ARRAY] A double precision array of length NDIM containing the variable lower bounds x_L . For variables that have no lower bound set $x_{Li} \leq -.01/\text{HDMCON}(5)$. See Appendix B for HDMCON.
XUPR	[INPUT, DOUBLE PRECISION, ARRAY] A double precision array of length NDIM containing the variable upper bounds x_{Ui} . For variables that have no upper bound set $x_{Ui} \geq .01/\text{HDMCON}(5)$.
ISTATV	[INPUT/OUTPUT, INTEGER, ARRAY] An integer array of length NDIM indicating variable status. On input ISTATV defines an estimate of the active variable bounds, and on output ISTATV defines the bounds active at the solution. ISTATV(i)= 0 Variable is strictly feasible, i.e. $x_{Li} < x_i < x_{Ui}$. ISTATV(i)= 1 Variable lower bound is active, i.e. $x_{Li} = x_i$. ISTATV(i)= 2 Variable upper bound is active, i.e. $x_i = x_{Ui}$. ISTATV(i)= 3 Fixed permanently. ISTATV(i)= 10 Output only: Violated variable lower bound, i.e. $x_{Li} - \delta_c > x_i$. ISTATV(i)= 20 Output only: Violated variable upper bound, i.e. $x_i > x_{Ui} + \delta_c$. ISTATV(i)= 30 Output only: Violated fixed variable. Either $x_i < x_{Li} - \delta_c$ or $x_i > x_{Ui} + \delta_c$. When ISTATV(i) = 3 it is necessary that $x_{Li} = x_{Ui}$ where $x_{Li} = \text{XLWR}(i)$ and $x_{Ui} = \text{XUPR}(i)$. When ISTATV(i) $\neq$ 3, it is necessary that $x_{Li} < x_{Ui}$ with $ x_{Li} - x_{Ui} > \delta_c$.
VECNU	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] An array of length NDIM of bound multipliers ν .
NDIM	[INPUT, INTEGER] Number of variables, where $\text{NDIM} \geq 1$.

OBJECTIVE FUNCTION DATA

FBAR	[OUTPUT, DOUBLE PRECISION] The objective function evaluated at XBAR, $f(\text{XBAR})$.
-------------	--

CONSTRAINT DATA

- CBAR** [OUTPUT, DOUBLE PRECISION, ARRAY]
A double precision array of length MCON containing the constraints $c(x)$ evaluated at XBAR. Not referenced when MCON = 0.
- CLWR** [INPUT, DOUBLE PRECISION, ARRAY]
A double precision array of length MCON containing the constraint lower bounds c_L . For constraints that have no lower bound set $c_{Li} \leq -.01/\text{HDMCON}(5)$. Not referenced when MCON = 0.
- CUPR** [INPUT, DOUBLE PRECISION, ARRAY]
A double precision array of length MCON containing the constraint upper bounds c_U . For constraints that have no upper bound set $c_{Ui} \geq .01/\text{HDMCON}(5)$. Not referenced when MCON = 0.
- ISTATC** [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length MCON indicating the constraint status. On input ISTATC defines an estimate of the active constraints, and on output ISTATC defines the constraints active at the solution.
- ISTATC(i)= 0 Free (inactive) inequality, i.e. $c_{Li} < c_i(x) < c_{Ui}$.
- ISTATC(i)= 1 Constraint lower bound is active, i.e. $c_{Li} = c_i(x)$.
- ISTATC(i)= 2 Constraint upper bound is active, i.e. $c_i(x) = c_{Ui}$.
- ISTATC(i)= 3 Equality.
- ISTATC(i)= 4 Ignored constraint.
- ISTATC(i)= 10 Output only: Violated constraint lower bound, i.e. $c_{Li} - \delta_c > c_i$.
- ISTATC(i)= 20 Output only: Violated constraint upper bound , i.e. $c_i > c_{Ui} + \delta_c$.
- ISTATC(i)= 30 Output only: Violated equality constraint. Either $c_i < c_{Li} - \delta_c$ or $c_i > c_{Ui} + \delta_c$.
- When ISTATC(i)=3 it is necessary that $c_{Li} = c_{Ui}$ where $c_{Li} = \text{CLWR}(i)$ and $c_{Ui} = \text{CUPR}(i)$. When ISTATC(i) $\neq 3$, it is necessary that $c_{Li} < c_{Ui}$ with $|c_{Li} - c_{Ui}| > \delta_c$. Not referenced when MCON = 0.
- VECLAM** [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
A double precision array of length MCON of Lagrange multipliers for constraints λ . Not referenced when MCON = 0.
- MCON** [INPUT, INTEGER]
Number of constraints, where MCON ≥ 0 .
- WORKING STORAGE**
- HOLD** [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
A double precision array of length NHOLD containing the hold array. The contents

of HOLD must not be changed during the optimization iterations, i.e. between successive calls to FUNBOX.

NHOLD [INPUT, INTEGER]
Dimension of HOLD array. When IER = -127, -131, or -701, NEEDED contains the required storage.

IHOLD [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length NIHOLD containing the integer hold array. The contents of IHOLD must not be changed during the optimization iterations, i.e. between successive calls to FUNBOX.

NIHOLD [INPUT, INTEGER]
Dimension of IHOLD array. When IER = -128, -132, or -702, NEEDED contains the required storage.

OUTPUT ARGUMENTS SET BY ALGORITHM BEFORE TERMINATION

NEEDED [OUTPUT, INTEGER]
When IER = -127, -131, or -701, NEEDED contains the required length of HOLD. When IER = -128, -132, or -702, NEEDED contains the required length for IHOLD.

IER [OUTPUT, INTEGER]
Success/error code. Results are not computed for IER < 0. For IER > 0, results are computed but may be suspect; user inspection is recommended. Note many of the error returns involve optional input quantities as described in INSNLP.

IER=0 Normal termination.

IER=-101 (MCON < 0); the number of constraints is negative.

IER=-102 (NDIM < 1); the number of variables is less than one.

IER=-103 Incorrect value for QPOPTN.

IER=-104 |IHESHN| > 3.

IER=-105 (NITMAX < max(1,NITMIN)); the maximum number of iterations is either less than 1 or less than the minimum number of iterations.

IER=-107 (IT1MAX < 1); the number of line search steps is less than 1.

IER=-108 (IOFLAG < 0) or (IOFLAG > 30); invalid input for the output control flag.

IER=-109 Invalid input for ALGOPT.

IER=-110	(OBJTOL $\leq 10 \cdot \text{HDMCON}(5)$); objective function tolerance is too small.
IER=-111	(PGDTOL $\leq [\text{HDMCON}(5)]^{\frac{1}{2}}$) or (PGDTOL $> 10^{-2}$); the projected gradient tolerance is too small or too large.
IER=-112	CONTOL $< [\text{HDMCON}(5)]^{\frac{1}{2}}$; the constraint tolerance is too small.
IER=-113	(ISTATC(i) < 0) or (ISTATC(i) > 4); invalid input for constraint status.
IER=-117	(CUPR(i) $< \text{CLWR}(i)$); constraint upper bound is less than lower bound.
IER=-118	Either (CUPR(i) = CLWR(i)) and (ISTATC(i) $\neq 3$) or (CUPR(i) $\neq \text{CLWR}(i)$) and (ISTATC(i) = 3); constraint status array is not consistent with bounds.
IER=-119	(CUPR(i) $\neq \text{CLWR}(i)$) and ($ \text{CUPR}(i) - \text{CLWR}(i) < \text{CONTOL}$); constraint bounds are not equal, but differ by less than the constraint tolerance.
IER=-120	(ISTATV(i) < 0) or (ISTATV(i) > 3); invalid input for variable status array.
IER=-124	(XUPR(i) $< \text{XLWR}(i)$); the variable upper bound is less than the lower bound.
IER=-125	Either (XUPR(i) = XLWR(i)) and (ISTATV(i) $\neq 3$) or (XUPR(i) $\neq \text{XLWR}(i)$) and (ISTATV(i) = 3); the variable status array is not consistent with the bound values.
IER=-126	(XUPR(i) $\neq \text{XLWR}(i)$) and ($ \text{XUPR}(i) - \text{XLWR}(i) < \text{CONTOL}$); variable bounds are not equal but differ by less than the constraint tolerance.
IER=-127	Double precision hold array too small; insufficient storage detected in SBRNLP interface. The required storage is specified in NEEDED.
IER=-128	Integer hold array too small; insufficient storage detected in SBRNLP interface. The required storage is specified in NEEDED.
IER=-129	Function error at initial point or during gradient evaluation.
IER=-130	BIGCON $< \text{CONTOL}$.
IER=-131	NHOLD too small; insufficient double precision storage detected in algorithm. The required storage is specified in NEEDED.
IER=-132	NIHOLD too small; insufficient integer storage detected in algorithm. The required storage is specified in NEEDED.

IER=-133	Rank deficient Jacobian detected on successive iterations.
IER=-134	INSNLP input error; invalid character string displayed.
IER=-135	(IFERR < 0) or (IFERR > 1); invalid value for function error flag.
IER=-136	FEATOL < CONTOL; the initial feasibility tolerance is less than the constraint tolerance.
IER=-137	Conflict between user and multifrontal file number; check IPUMF1,...,IPUMF6 IPUDRF, IPUFZF, IPUSTF.
IER=-138	NEWTON $\neq$ 0, 1, 2; invalid input for Newton method option flag.
IER=-146	PMULWR < HDMCON(5).
IER=-147	Unexpected error; check storage allocation.
IER=-149	PTHTOL < [HDMCON(5)] ^{1/2} .
IER=-150	RHOLWR < HDMCON(5).
IER=-151	A constraint is inconsistent (either $c_i(x) < c_{Li}$ or $c_{Ui} < c_i(x)$) and cannot be changed because the corresponding row of the Jacobian is zero.
IER=-152	IMAXMU < 1.
IER=-153	I/O error, probably caused by insufficient disk space.
IER=-154	TOLKTC < 1.
IER=-155	TOLPVT < 0 or TOLPVT > .5.
IER=-156	IRELAX < 0 or IRELAX > 2.
IER=-157	MUCALC = 0 or MUCALC > 3.
IER=-158	MXQPIT < 1.
IER=-701	Double Precision hold array too small; insufficient storage detected in dense interface. The required storage is specified in NEEDED.
IER=-702	Integer hold array too small; insufficient storage detected in dense interface. The required storage is specified in NEEDED.
IER=-703	EPSRLF < [HDMCON(5)] ^{1/2} .

IER=+101	Weak solution found (relaxation required and/or multipliers near zero).
IER=+102	Number of equality constraints = number of free variables and ALGOPT $\neq$ 'F'.
IER=+103	Maximum number of consecutive function errors.
IER=+104	Maximum number of function evaluations.
IER=+105	Small step termination in optimization phase; suboptimal feasible point found.
IER=+106	Maximum number of iterations.
IER=+108	Feasible point not found.
IER=+109	Maximum number of interval halves in line search.
IER=+110	Algorithm terminated because the Hessian diagonal reached its maximum value; suboptimal feasible point found.
IER=+111	Projected gradient calculation failed; constraints may be degenerate.
IER=+112	Calculation of first order multiplier estimates failed; constraints may be degenerate.
IER=+113	Suboptimal feasible point found.
IER=+114	Barrier NLP failed with unexpected error.
IER=+115	CONTOL > OBJTOL; convergence tolerances may be inappropriate.
IER=+116	Uphill direction detected in line search.
IER=+117	Reduced objective function is linear.
IER=+118	Both $c_{Li} \leq -.01/\text{HDMCON}(5)$ and $c_{Ui} \geq .01/\text{HDMCON}(5)$; constraints ignored.
IER=+119	Terminate after diagnostic line search.
IER=+120	(IHESHN > 0) and (NEWTON = 0); recursive Hessian updates may be skipped causing poor algorithm performance.

USER-SUPPLIED SUBPROGRAM

The user must provide a real subroutine, FUNBOX to compute the objective and constraint functions. Subroutine FUNBOX must have the following format:

```

SUBROUTINE FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
C
DOUBLE PRECISION XBAR,QUANT
INTEGER NDIM,MCON,IFERR
DIMENSION XBAR(NDIM),QUANT(MCON+1)
C
C      IF POSSIBLE EVALUATE OBJECTIVE AND CONSTRAINTS, OTHERWISE
C      SET FUNCTION ERROR FLAG.
C
IF( Functions can be evaluated at XBAR) THEN
C
C      SET FUNCTION ERROR FLAG
C
IFERR = 0
C
C      CONSTRAINTS
C
QUANT(1) = User supplied function evaluation process
.
.
.
QUANT(MCON) = User supplied function evaluation process
C
C      OBJECTIVE FUNCTION
C
QUANT(MCON+1) = User supplied function evaluation process
C
ELSE
C
C      SET FUNCTION ERROR FLAG
C
IFERR = 1
C
END IF
C
RETURN
END

```

The arguments of subroutine FUNBOX are defined as follows:

ARGUMENTS

XBAR	[INPUT, DOUBLE PRECISION, ARRAY] A double precision array of length NDIM containing the current variable values x .
NDIM	[INPUT, INTEGER] Number of variables n , where $\text{NDIM} \geq 1$.
MCON	[INPUT, INTEGER] Number of constraints m , where $\text{MCON} \geq 0$.
QUANT	[OUTPUT, DOUBLE PRECISION, ARRAY] A double precision array of length $\text{MCON}+1$ containing the values of the computed functions at the current value of x . The first MCON elements of QUANT contain the constraints $c(x)$, and the objective function $f(x)$ is stored in location $\text{MCON}+1$. When the function error flag $\text{IFERR} = 1$, the QUANT array will not be referenced.
IFERR	[OUTPUT, INTEGER] Function error flag. Set $\text{IFERR} = 0$ when functions are successfully computed. Set $\text{IFERR} = 1$ when functions cannot be computed. Appendix A presents a discussion of the similarity between the input requirements for DBNLPF and those of the NPSOL optimization algorithm.

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine DBNLPF. All optional inputs are specified by calls to subroutine INSNLP. It should be emphasized that it is *necessary* to call INSNLP to set default values for the barrier algorithm, that is `CALL INSNLP('BARRIER DEFAULT')`. If optional inputs are desired, they must be set by successive calls to INSNLP for each input parameter.

OPTIONAL PRINTOUT

The level of printed output from DBNLPF is controlled by the variables IOFLAG and IPUNLP which are set by calls to INSNLP. The standard level of output which corresponds to $\text{IOFLAG} = 10$ is the *default* for DBNLPF. A complete description of the iteration output options is found in Section 2.3.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2)$ to minimize the objective function

$$f(x) = x_1^2 + x_2^2 + \log(x_1 x_2)$$

subject to the constraint

$$x_1 x_2 \geq 1$$

and the bounds

$$0 \leq x_k \leq 10$$

for $k = 1, 2$ beginning at the initial point $x = (.5, 2)$. The solution is at $x^* = (1, 1)$

SAMPLE PROGRAM (dbnlpf.cd1)⁷

```
      PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE NONLINEAR
C      PROGRAMMING ALGORITHM DBNLPF
C
C-----
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
      PARAMETER ( NHOLD=10000, NIHOLD=1000 )
      PARAMETER ( ZERO=0.DO, ONE=1.DO, POINT5 = .5DO, TWO = 2.DO)
      PARAMETER (MAXDIM = 5, MAXCON = 10)
C
C      NLP CALLING ARGUMENTS
C
      DIMENSION XBAR(MAXDIM),XLWR(MAXDIM),XUPR(MAXDIM),
$      ISTATV(MAXDIM),VECNU(MAXDIM),
$      CBAR(MAXCON),CLWR(MAXCON),
$      CUPR(MAXCON),ISTATC(MAXCON),VECLAM(MAXCON)
      DIMENSION HOLD(NHOLD),IHOLD(NIHOLD)
      EXTERNAL FUNBOX
C
C      DEFINE INDEPENDENT VARIABLE DATA
C
      NDIM = 2
      XBAR(1) = POINT5
      XBAR(2) = TWO
      xlwr(1:ndim) = zero
      xupr(1:ndim) = 10.d0
      istatv(1:ndim) = 0
C
C      DEFINE CONSTRAINT DATA
C
      BIGBND = 1.DO/HDMCON(5)
      MCON = 1
      CLWR(1) = ONE
      CUPR(1) = BIGBND
      ISTATC(1) = 0
C
C-----
C
C      CALL NLP TO SOLVE THE PROBLEM
C
      CALL INSNLP('BARRIER DEFAULT')
C
```

⁷sosexample.tar

```
      CALL DBNLPF(FUNBOX, XBAR, XLWR, XUPR, ISTATV, VECNU, NDIM,
$      FBAR, CBAR, CLWR, CUPR, ISTATC, VECLAM, MCON,
$      HOLD, NHOLD, IHOLD, NIHOLD, NEEDED, IERNLP)
C
      STOP
      END

      SUBROUTINE FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
      SUBROUTINE TO EVALUATE THE PROBLEM FUNCTIONS FOR
      THE VARIOUS PASS/FAIL TEST PROBLEMS
C
      DIMENSION XBAR(NDIM),QUANT(MCON+1)
C
      NONLINEAR CONSTRAINTS
C
      QUANT(1) = XBAR(1)*XBAR(2)
C
      CHECK THAT ARGUMENT OF LOGARITHM IS POSITIVE
      BEFORE ATTEMPTING TO EVALUATE OBJECTIVE
C
      IF(QUANT(1).LT.HDMCON(5)) THEN
C
      FUNCTION ERROR
C
      IFERR = 1
C
      ELSE
C
      IFERR = 0
C
      OBJECTIVE FUNCTION
C
      QUANT(MCON+1) = XBAR(1)**2 + XBAR(2)**2
$      + LOG(QUANT(1))
C
      ENDIF
C
      RETURN
      END
```

OUTPUT FROM SAMPLE PROGRAM

```
***** INITIAL POINT *****
*****
*
```



```

*      Objective Function.....+4.25000   |      Gradient Of Lagrangian.....+6.59956
*      Log-Barrier Function.....+4.27744   |      Equality Error.....+1.000000E-02
*      Barrier Parameter.....+0.100000    |      Complementarity.....+1.387779E-17
*
*      -----
*      Filter: B(y)                |c(y)|
*      -----
*      | 1 | +4.277444E+00 | +1.000000E-02 |
*      -----
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      ..... (B,|c|) = ( +1.061710E+00, +3.151201E-01 )
*
*      ----- Iteration (2) -----
*      Objective Function.....+1.06148   |      Gradient Of Lagrangian.....+0.439618
*      Log-Barrier Function.....+1.06151   |      Equality Error.....+0.315120
*      Barrier Parameter.....+1.000000E-02 |      Complementarity.....+8.991003E-02
*
*      -----
*      Filter: B(y)                |c(y)|
*      -----
*      | 1 | +1.061507E+00 | +3.151201E-01 |
*      -----
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      ..... (B,|c|) = ( +2.085415E+00, +2.056006E-02 )
*
*      ----- Iteration (3) -----
*      Objective Function.....+2.07983   |      Gradient Of Lagrangian.....+0.776446
*      Log-Barrier Function.....+2.08541   |      Equality Error.....+9.165028E-03
*      Barrier Parameter.....+1.000000E-02 |      Complementarity.....+2.056006E-02
*
*      -----
*      Filter: B(y)                |c(y)|
*      -----
*      | 1 | +2.085415E+00 | +2.056006E-02 |
*      | 2 | +1.061507E+00 | +3.151201E-01 |
*      -----
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      ..... (B,|c|) = ( +1.983079E+00, +1.568306E-02 )
*
*      ----- Iteration (4) -----
*      Objective Function.....+1.96426   |      Gradient Of Lagrangian.....+0.427564
*      Log-Barrier Function.....+1.98308   |      Equality Error.....+1.568306E-02
*      Barrier Parameter.....+1.000000E-02 |      Complementarity.....+3.768136E-03
*
*      -----
*      Filter: B(y)                |c(y)|
*      -----
*      | 1 | +1.983079E+00 | +1.568306E-02 |
*      | 2 | +1.061507E+00 | +3.151201E-01 |
*      -----
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      ..... (B,|c|) = ( +2.019512E+00, +1.196678E-03 )
*
*      ----- Iteration (5) -----
*      Objective Function.....+2.00610   |      Gradient Of Lagrangian.....+1.302216E-02

```

```

*      Log-Barrier Function.....+2.00744   |   Equality Error.....+1.196678E-03   *
*      Barrier Parameter.....+1.000000E-03   |   Complementarity.....+9.036266E-03   *
*      -----
*      Filter: B(y)                |c(y)|
*      -----
*      | 1 | +2.007438E+00 | +9.036266E-03 |
*      -----
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      ..... (B,|c|) = ( +2.004597E+00, +3.798821E-05 )
*
*      ----- Iteration (6) -----
*      Objective Function.....+2.00095   |   Gradient Of Lagrangian....+4.481412E-05   *
*      Log-Barrier Function.....+2.00131   |   Equality Error.....+4.960401E-06   *
*      Barrier Parameter.....+1.000000E-04   |   Complementarity.....+9.288235E-04   *
*      -----
*      Filter: B(y)                |c(y)|
*      -----
*      | 1 | +2.001312E+00 | +9.288235E-04 |
*      -----
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      ..... (B,|c|) = ( +2.000692E+00, +3.177682E-07 )
*
*      ----- Iteration (7) -----
*      Objective Function.....+2.00010   |   Gradient Of Lagrangian....+2.949501E-06   *
*      Log-Barrier Function.....+2.00016   |   Equality Error.....+1.995874E-08   *
*      Barrier Parameter.....+1.000000E-05   |   Complementarity.....+9.012973E-05   *
*      -----
*      Filter: B(y)                |c(y)|
*      -----
*      | 1 | +2.000159E+00 | +9.012973E-05 |
*      -----
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      ..... (B,|c|) = ( +2.000092E+00, +3.350023E-09 )
*
*      ----- Iteration (8) -----
*      Objective Function.....+2.00001   |   Gradient Of Lagrangian....+3.138209E-07   *
*      Log-Barrier Function.....+2.00001   |   Equality Error.....+2.237179E-10   *
*      Barrier Parameter.....+1.000000E-09   |   Complementarity.....+1.000036E-05   *
*      -----
*      Filter: B(y)                |c(y)|
*      -----
*      | 1 | +2.000010E+00 | +1.000036E-05 |
*      -----
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      ..... (B,|c|) = ( +2.000000E+00, +4.144477E-11 )
*
*      ----- Iteration (9) -----
*      Objective Function.....+2.00000   |   Gradient Of Lagrangian....+3.502136E-08   *
*      Log-Barrier Function.....+2.00000   |   Equality Error.....+2.775558E-12   *
*      Barrier Parameter.....+1.000000E-09   |   Complementarity.....+4.144477E-11   *
*      -----

```

```

*                                     Filter: B(y)                                |c(y)|
*                                     -----
*                                     |  1  | +2.000000E+00 | +4.144477E-11 |
*                                     -----
*
*      CONSTRAINED OPTIMIZATION COMPLETED
*
*****
***** CONVERGENCE *****
*****
-----
|                                     |
|      BARNLP ALGORITHM PERFORMANCE STATISTICS      |
|                                     |
|      Total CPU Time.....+2.145502E-02      |
|                                     |
|      Number of Function Calls.....9      |
|      Number of Gradient Calls.....10      |
|      Number of Hessian Calls.....3      |
|      Total Number of Function Evaluations.....50      |
|                                     |
|      Storage Needed in HOLD Array.....480      |
|      Storage Needed in IHOLD Array.....69      |
|                                     |
-----
***** SOLUTION *****
*****
*
*      Objective Function =      2.00000      IERNLP =      0
*
*      -----
*
*      Variable      Status      Value      Lower bound      Upper bound      Lagrange Mlt.      Slack
*
*          1      FR      1.00000      0.      10.0000      2.046189E-08      1.000
*          2      FR      1.00000      0.      10.0000      3.592727E-08      1.000
*
*      -----
*
*      Constraint      Status      Value      Lower bound      Upper bound      Lagrange Mlt.      Slack
*
*          1      LB      1.00000      1.00000      4.503600E+15      3.00000      3.2229E-10
*
*****

```

DBNPFG: Dense Barrier Nonlinear Programming–User Supplied Gradients

PURPOSE

DBNPFG computes the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the objective function

$$f(x)$$

subject to the m nonlinear constraints

$$c_L \leq c(x) \leq c_U$$

and the simple bounds

$$x_L \leq x \leq x_U$$

Equality constraints are imposed by setting $c_{Li} = c_{Ui}$ and variables can be fixed by setting $x_{Li} = x_{Ui}$. DBNPFG works under the assumption that the objective and constraint functions are twice continuously differentiable, and if this is not true algorithm performance is unpredictable. DBNPFG uses a forward communication format and the user must supply a subroutine to compute the values of the objective and constraint functions. First derivatives of the user defined functions must also be supplied. Second derivatives of the user supplied functions are computed numerically. The matrix of second derivatives (Hessian of the Lagrangian) is constructed by finite differencing user supplied gradients. Subroutine DBNPFG calls the sparse nonlinear programming algorithm SBRNLP, and is intended for simplified usage as appropriate for small dense problems.

RELATED SUBPROGRAMS

HDDFDJ	Finite Difference Jacobian
HDDFDH	Finite Difference Hessian
DNLPFW	Dense Nonlinear Programming
DBNLPF	Dense Barrier Nonlinear Programming
SBRNLP	Sparse Barrier Nonlinear Programming
INSNLP	Sparse Nonlinear Programming Input Procedure

METHOD

A *primal-dual interior point or barrier* approach is used to solve the *nonlinear programming* (NLP) problem. DBNPFG constructs an approximation to the Hessian matrix using gradient differences and the graph coloring algorithm of Coleman, Garbow and More. Forward difference estimates are utilized by default for efficiency, and central difference estimates can be selected by input. Although default values are set for all algorithm parameters, INSNLP can be utilized to override the defaults.

WARNING

The objective function and constraints supplied by the user are assumed to be continuous and have continuous first and second derivatives. Nondifferentiable functions can cause unpredictable

performance in the optimization algorithm. It should be emphasized that discontinuities in the Hessian matrices can significantly degrade the speed of convergence without producing any other obvious difficulties. The following common sources of error *should be avoided* when evaluating the objective and constraint functions:

- (1) nonsmooth interpolation of data;
- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;
- (3) discontinuous behavior caused by branching for IF tests;
- (4) nondifferentiable functions such as ABS, MAX, and MIN.

The user must provide a procedure to evaluate the objective and constraint functions as well as their first derivatives. If it is not possible to evaluate the functions (for example, $c(x) = 1/x$ at $x = 0$) then set IFERR = 1 and exit the function evaluation process. Otherwise, set IFERR = 0 when the functions are evaluated. The function error flag should be used to avoid computational difficulties, but never in lieu of an explicit constraint.

The DBNPFG software uses internal common blocks to communicate information. To avoid conflict with these internal commons the user should not have commons with the following names: NPSPRR, NPSPRI, NPSPRC, STATIS, KONSTN, NPSALG, NPSDFL, NLPERR, PERCOM, ITEREF, and CLKCOM.

USAGE

```
INTEGER      IER, NDIM, MCON
INTEGER      ISTATV(NDIM), ISTATC(MCON)

DOUBLE PRECISION      FBAR, XBAR(NDIM), XLWR(NDIM), XUPR(NDIM), VECNU(NDIM)
DOUBLE PRECISION      CBAR(MCON), CLWR(MCON), CUPR(MCON), VECLAM(MCON)

EXTERNAL      FNGBOX

CALL DBNPFG(FNGBOX, XBAR, XLWR, XUPR, ISTATV, VECNU, NDIM,
$           FBAR, CBAR, CLWR, CUPR, ISTATC, VECLAM, MCON, IER)
```

ARGUMENTS

FUNCTION EVALUATION PROCEDURE

FNGBOX [INPUT, SUBROUTINE]
Name of user-supplied subroutine to evaluate objective and constraint functions and gradients; see USER-SUPPLIED SUBPROGRAM

INDEPENDENT VARIABLE DATA

XBAR [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM of current variable values x . An initial guess must

be input. When $IER = 0, +101$, the final point corresponds to a local optimum. When $IER = +104, \dots, +111, +114, +116, +117, +119$, the final point is either the best feasible point if one has been found, or the last iterate.

XLWR	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the variable lower bounds x_L . For variables that have no lower bound set $x_{Li} \leq -.01/\text{HDMCON}(5)$. See Appendix B for HDMCON.
XUPR	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the variable upper bounds x_U . For variables that have no upper bound set $x_{Ui} \geq .01/\text{HDMCON}(5)$.
ISTATV	[INPUT/OUTPUT, INTEGER, ARRAY] An integer array of length NDIM indicating variable status. On input ISTATV defines an estimate of the active variable bounds, and on output ISTATV defines the bounds active at the solution. ISTATV(i)= 0 Variable is strictly feasible, i.e. $x_{Li} < x_i < x_{Ui}$. ISTATV(i)= 1 Variable lower bound is active, i.e. $x_{Li} = x_i$. ISTATV(i)= 2 Variable upper bound is active, i.e. $x_i = x_{Ui}$. ISTATV(i)= 3 Fixed permanently. ISTATV(i)= 10 Output only: Violated variable lower bound, i.e. $x_{Li} - \delta_c > x_i$. ISTATV(i)= 20 Output only: Violated variable upper bound, i.e. $x_i > x_{Ui} + \delta_c$. ISTATV(i)= 30 Output only: Violated fixed variable. Either $x_i < x_{Li} - \delta_c$ or $x_i > x_{Ui} + \delta_c$. When ISTATV(i) = 3 it is necessary that $x_{Li} = x_{Ui}$ where $x_{Li} = \text{XLWR}(i)$ and $x_{Ui} = \text{XUPR}(i)$. When ISTATV(i) $\neq$ 3, it is necessary that $x_{Li} < x_{Ui}$ with $ x_{Li} - x_{Ui} > \delta_c$.
VECNU	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] An array of length NDIM of bound multipliers ν .
NDIM	[INPUT, INTEGER] Number of variables, where $\text{NDIM} \geq 1$.
OBJECTIVE FUNCTION DATA	
FBAR	[OUTPUT, DOUBLE PRECISION] The objective function evaluated at XBAR, $f(\text{XBAR})$.
CONSTRAINT DATA	
CBAR	[OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON containing the constraints $c(x)$ evaluated at XBAR. Not referenced when $\text{MCON} = 0$.

- CLWR** [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraint lower bounds c_L . For constraints that have no lower bound set $c_{Li} \leq -.01/\text{HDMCON}(5)$. Not referenced when MCON = 0.
- CUPR** [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraint upper bounds c_U . For constraints that have no upper bound set $c_{Ui} \geq .01/\text{HDMCON}(5)$. Not referenced when MCON = 0.
- ISTATC** [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length MCON indicating the constraint status. On input ISTATC defines an estimate of the active constraints, and on output ISTATC defines the constraints active at the solution.
- ISTATC(i)= 0 Free (inactive) inequality, i.e. $c_{Li} < c_i(x) < c_{Ui}$.
- ISTATC(i)= 1 Constraint lower bound is active, i.e. $c_{Li} = c_i(x)$.
- ISTATC(i)= 2 Constraint upper bound is active, i.e. $c_i(x) = c_{Ui}$.
- ISTATC(i)= 3 Equality.
- ISTATC(i)= 4 Ignored constraint.
- ISTATC(i)= 10 Output only: Violated constraint lower bound, i.e. $c_{Li} - \delta_c > c_i$.
- ISTATC(i)= 20 Output only: Violated constraint upper bound, i.e. $c_i > c_{Ui} + \delta_c$.
- ISTATC(i)= 30 Output only: Violated equality constraint. Either $c_i < c_{Li} - \delta_c$ or $c_i > c_{Ui} + \delta_c$.
- When ISTATC(i)=3 it is necessary that $c_{Li} = c_{Ui}$ where $c_{Li} = \text{CLWR}(i)$ and $c_{Ui} = \text{CUPR}(i)$. When ISTATC(i) $\neq 3$, it is necessary that $c_{Li} < c_{Ui}$ with $|c_{Li} - c_{Ui}| > \delta_c$. Not referenced when MCON = 0.
- VECLAM** [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON of Lagrange multipliers for constraints λ . Not referenced when MCON = 0.
- MCON** [INPUT, INTEGER]
Number of constraints, where MCON ≥ 0 .
- OUTPUT ARGUMENTS SET BY ALGORITHM BEFORE TERMINATION**
- IER** [OUTPUT, INTEGER]
Success/error code. Results are not computed for IER < 0. For IER > 0, results are computed but may be suspect; user inspection is recommended. Note many of the error returns involve optional input quantities as described in INSNLP.

IER=0	Normal termination.
IER=-101	(MCON < 0); the number of constraints is negative.
IER=-102	(NDIM < 1); the number of variables is less than one.
IER=-104	IHESHN > 3.
IER=-105	(NITMAX < max(1,NITMIN)); the maximum number of iterations is either less than 1 or less than the minimum number of iterations.
IER=-106	(SLPTOL ≥ 1) or (SLPTOL ≤ 10 ⁻⁵); the slope tolerance for the line search during optimization is too large or too small.
IER=-107	(IT1MAX < 1); the number of line search steps is less than 1.
IER=-108	(IOFLAG < 0) or (IOFLAG > 30); invalid input for the output control flag.
IER=-109	Invalid input for ALGOPT.
IER=-110	(OBJTOL ≤ 10.*HDMCON(5)); objective function tolerance is too small.
IER=-111	(PGDTOL ≤ [HDMCON(5)] ^{1/2}) or (PGDTOL > 10 ⁻²); the projected gradient tolerance is too small or too large.
IER=-112	CONTOL < [HDMCON(5)] ^{1/2} ; the constraint tolerance is too small.
IER=-113	(ISTATC(i) < 0) or (ISTATC(i) > 4); invalid input for constraint status.
IER=-117	(CUPR(i) < CLWR(i)); constraint upper bound is less than lower bound.
IER=-118	Either (CUPR(i) = CLWR(i)) and (ISTATC(i) ≠ 3) or (CUPR(i) ≠ CLWR(i)) and (ISTATC(i) = 3); constraint status array is not consistent with bounds.
IER=-119	(CUPR(i) ≠ CLWR(i)) and (CUPR(i)-CLWR(i) < CONTOL); constraint bounds are not equal, but differ by less than the constraint tolerance.
IER=-120	(ISTATV(i) < 0) or (ISTATV(i) > 3); invalid input for variable status array.
IER=-124	(XUPR(i) < XLWR(i)); the variable upper bound is less than the lower bound.

IER=-125	Either $(XUPR(i) = XLWR(i))$ and $(ISTATV(i) \neq 3)$ or $(XUPR(i) \neq XLWR(i))$ and $(ISTATV(i) = 3)$; the variable status array is not consistent with the bound values.
IER=-126	$(XUPR(i) \neq XLWR(i))$ and $(XUPR(i) - XLWR(i) < CONTOL)$; variable bounds are not equal but differ by less than the constraint tolerance.
IER=-127	Real hold array too small; insufficient storage detected in SBRNLP interface. The required storage is specified in NEEDED.
IER=-128	Integer hold array too small; insufficient storage detected in SBRNLP interface. The required storage is specified in NEEDED.
IER=-129	Function error at initial point or during gradient evaluation.
IER=-130	(The number of active constraints $> NDIM$); check $ISTATC(i)$ and $ISTATV(i)$.
IER=-133	Rank deficient Jacobian detected on successive iterations.
IER=-134	INSNLP input error; invalid character string displayed.
IER=-135	$(IFERR < 0)$ or $(IFERR > 1)$; invalid value for function error flag.
IER=-136	$(SFZTOL \geq 1)$ or $(SFZTOL \leq 10^{-5})$; the slope tolerance for the line search during the feasibility phase is too large or too small.
IER=-137	Conflict between user and multifrontal file number; check $IPUMF1, \dots, IPUMF6$ $IPUDRF$, $IPUFZF$, $IPUSTF$.
IER=-138	$(NEWTON < 0)$ or $(NEWTON > 1)$; invalid input for Newton method option flag.
IER=-147	Unexpected error; check storage allocation.
IER=-149	Incorrect value for QPOPTN.
IER=-150	$QPOPTN = DENSE$ and either $ALGOPT = 'FME'$ or $NRES > 0$.
IER=-152	Incorrect value for KTOPTN.
IER=-153	I/O error, probably caused by insufficient disk space.
IER=-703	$ EPSRLF < [HDMCON(5)]^{\frac{1}{2}}$.

IER=+101	Weak solution found (multipliers near zero).
IER=+102	Number of equality constraints = NDIM and ALGOPT $\neq$ 'F'.
IER=+103	Maximum number of consecutive function errors.
IER=+104	Maximum number of function evaluations.
IER=+105	Small step termination in optimization phase; suboptimal feasible point found.
IER=+106	Maximum number of iterations in optimization phase.
IER=+107	Maximum number of iterations in feasibility phase.
IER=+108	Feasible point not found.
IER=+109	Maximum number of interval halves in line search.
IER=+110	QP algorithm terminated because either the Hessian diagonal reached its maximum value or the merit function slope condition was violated; suboptimal feasible point found.
IER=+111	Projected gradient calculation failed; constraints may be degenerate.
IER=+112	Calculation of first order multiplier estimates failed; constraints may be degenerate.
IER=+113	Suboptimal feasible point found.
IER=+114	Schur-complement QP algorithm (subroutine SHURQP) failed with unexpected error.
IER=+115	CONTOL > OBJTOL; convergence tolerances may be inappropriate.
IER=+116	Uphill direction detected in line search.
IER=+117	Reduced objective function is linear.
IER=+118	Both $c_{Li} \leq -.01/\text{HDMCON}(5)$ and $c_{Ui} \geq .01/\text{HDMCON}(5)$; constraints ignored.
IER=+119	Terminate after diagnostic line search.
IER=+703	Sparse parameter defaults may cause poor algorithm performance.

USER-SUPPLIED SUBPROGRAM

The user must provide a real subroutine, FNGBOX to compute the objective and constraint functions as well as first derivatives. Subroutine FNGBOX must have the following format:

```

SUBROUTINE FNGBOX(XBAR,NDIM,MCON,QUANT,DQUANT,IFERR)
C
DOUBLE PRECISION XBAR,QUANT,DQUANT
INTEGER NDIM,MCON,IFERR
DIMENSION XBAR(NDIM),QUANT(MCON+1),DQUANT(MCON+1,NDIM)
C
C      IF POSSIBLE EVALUATE OBJECTIVE AND CONSTRAINTS, OTHERWISE
C      SET FUNCTION ERROR FLAG.
C
IF( Functions can be evaluated at XBAR) THEN
C
C      SET FUNCTION ERROR FLAG
C
IFERR = 0
C
C      CONSTRAINTS
C
QUANT(1) = User supplied function evaluation process
.
.
.
QUANT(MCON) = User supplied function evaluation process
C
C      OBJECTIVE FUNCTION
C
QUANT(MCON+1) = User supplied function evaluation process
C
C      JACOBIAN MATRIX
C
DQUANT(1,1) = User supplied gradient
C
DQUANT(2,1) = User supplied gradient
.
.
.
C
DQUANT(MCON+1,1) = User supplied gradient
.
.
.
C
DQUANT(1,NDIM) = User supplied gradient
```

```

C      DQUANT(2,NDIM) = User supplied gradient
C      .
C      .
C      .
C      DQUANT(MCON+1,NDIM) = User supplied gradient
C
C      ELSE
C
C      SET FUNCTION ERROR FLAG
C
C      IFERR = 1
C
C      END IF
C
C      RETURN
C      END

```

The arguments of subroutine FNGBOX are defined as follows:

ARGUMENTS

XBAR	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the current variable values x .
NDIM	[INPUT, INTEGER] Number of variables n , where $\text{NDIM} \geq 1$.
MCON	[INPUT, INTEGER] Number of constraints m , where $\text{MCON} \geq 0$.
QUANT	[OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON+1 containing the values of the computed functions at the current value of x . The first MCON elements of QUANT contain the constraints $c(x)$, and the objective function $f(x)$ is stored in location MCON+1. When the function error flag IFERR = 1, the QUANT array will not be referenced.
DQUANT	[OUTPUT, DOUBLE PRECISION, ARRAY] A real two-dimensional array with MCON+1 rows, and NDIM columns. Element

$$DQUANT(i, j) = \frac{dq_i}{dx_j}$$

for $i = 1, \dots, (m+1)$, and $j = 1, \dots, n$ evaluated at the current value of x . The first MCON rows of DQUANT contain the constraint Jacobian matrix $G(x)$, and the objective function gradient $\nabla f(x)$ is stored in row MCON+1. When the function error flag IFERR = 1, the DQUANT array will not be referenced.

IFERR [OUTPUT, INTEGER]

Function error flag. Set IFERR = 0 when functions are successfully computed. Set IFERR = 1 when functions cannot be computed.

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine DBNPFG. All optional inputs are specified by calls to subroutine INSNLP. It should be emphasized that it is *not necessary* to call INSNLP if the default values are appropriate. If optional inputs are specified, it is recommended that the user first set the defaults for a dense problem, that is CALL INSNLP('DENSE DEFAULT'), and then specify the desired input parameter.

OPTIONAL PRINTOUT

The level of printed output from DBNPFG is controlled by the variables IOFLAG and IPUNLP which are set by calls to INSNLP. The standard level of output which corresponds to IOFLAG = 10 is the *default* for DBNPFG. A complete description of the iteration output options is found in Section 2.3.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2)$ to minimize the objective function

$$f(x) = x_1^2 + x_2^2 + \log(x_1 x_2)$$

subject to the constraint

$$x_1 x_2 \geq 1$$

and the bounds

$$0 \leq x_k \leq 10$$

for $k = 1, 2$ beginning at the initial point $x = (.5, 2)$. The solution is at $x^* = (1, 1)$.

SAMPLE PROGRAM (dbnpfg.cd1)⁸

```
PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE NONLINEAR
C      PROGRAMMING ALGORITHM DBNPFG
C
C-----
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C      PARAMETER ( ZERO=0.D0, ONE=1.D0, POINT5 = .5D0, TWO = 2.D0)
```

⁸sosexample.tar

```

C      PARAMETER (MAXDIM = 5, MAXCON = 10)
C
C      NLP CALLING ARGUMENTS
C
C      DIMENSION XBAR(MAXDIM),XLWR(MAXDIM),XUPR(MAXDIM),
$      ISTATV(MAXDIM),VECNU(MAXDIM),
$      CBAR(MAXCON),CLWR(MAXCON),
$      CUPR(MAXCON),ISTATC(MAXCON),VECLAM(MAXCON)
C      EXTERNAL FNGBOX
C
C      DEFINE INDEPENDENT VARIABLE DATA
C
C      NDIM = 2
C      XBAR(1) = POINT5
C      XBAR(2) = TWO
C      xlwr(1:ndim) = zero
C      xupr(1:ndim) = 10.d0
C      istatv(1:ndim) = 0
C
C      DEFINE CONSTRAINT DATA
C
C      BIGBND = 1.DO/HDMCON(5)
C      MCON = 1
C      CLWR(1) = ONE
C      CUPR(1) = BIGBND
C      ISTATC(1) = 0
C
C      -----
C
C      CALL NLP TO SOLVE THE PROBLEM
C
C      CALL DBNPFPG(FNGBOX, XBAR, XLWR, XUPR, ISTATV, VECNU, NDIM,
$      FBAR, CBAR, CLWR, CUPR, ISTATC, VECLAM, MCON, IERNLP)
C
C      STOP
C      END
C
C      SUBROUTINE FNGBOX(XBAR,NDIM,MCON,QUANT,DQUANT,IFERR)
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      SUBROUTINE TO EVALUATE THE PROBLEM FUNCTIONS
C
C      DIMENSION XBAR(NDIM),QUANT(MCON+1),DQUANT(MCON+1,NDIM)
C
C      NONLINEAR CONSTRAINTS
C
C      QUANT(1) = XBAR(1)*XBAR(2)

```

```

***** INITIAL POINT *****
*****
*
* Objective Function =      4.25000      IERNLP =      0
*
* -----
*
* Variable   Status   Value      Lower bound      Upper bound      Lagrange Mlt.      Slack
*
*          1    FR    0.500000      0.00000      10.0000      0.00000      0.5000
*          2    FR    2.00000      0.00000      10.0000      0.00000      2.000
*
* -----
*
* Constraint  Status   Value      Lower bound      Upper bound      Lagrange Mlt.      Slack
*

```



```

*      Objective Function.....+1.06150   |      Gradient Of Lagrangian.....+0.439611
*      Log-Barrier Function.....+1.06152   |      Equality Error.....+0.315116
*      Barrier Parameter.....+1.000000E-02   |      Complementarity.....+8.991004E-02
*
*      -----
*      Filter: B(y)                |c(y)|
*      -----
*      | 1 | +1.061524E+00 | +3.151159E-01 |
*      -----
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      ..... (B,|c|) = ( +2.085408E+00, +2.055933E-02 )
*
*      ----- Iteration (3) -----
*      Objective Function.....+2.07982   |      Gradient Of Lagrangian.....+0.776347
*      Log-Barrier Function.....+2.08541   |      Equality Error.....+9.166065E-03
*      Barrier Parameter.....+1.000000E-02   |      Complementarity.....+2.055933E-02
*
*      -----
*      Filter: B(y)                |c(y)|
*      -----
*      | 1 | +2.085408E+00 | +2.055933E-02 |
*      | 2 | +1.061524E+00 | +3.151159E-01 |
*      -----
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      ..... (B,|c|) = ( +1.983094E+00, +1.567656E-02 )
*
*      ----- Iteration (4) -----
*      Objective Function.....+1.96427   |      Gradient Of Lagrangian.....+0.427432
*      Log-Barrier Function.....+1.98309   |      Equality Error.....+1.567656E-02
*      Barrier Parameter.....+1.000000E-02   |      Complementarity.....+3.766869E-03
*
*      -----
*      Filter: B(y)                |c(y)|
*      -----
*      | 1 | +1.983094E+00 | +1.567656E-02 |
*      | 2 | +1.061524E+00 | +3.151159E-01 |
*      -----
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      ..... (B,|c|) = ( +2.019514E+00, +1.195832E-03 )
*
*      ----- Iteration (5) -----
*      Objective Function.....+2.00610   |      Gradient Of Lagrangian.....+1.301492E-02
*      Log-Barrier Function.....+2.00744   |      Equality Error.....+1.195832E-03
*      Barrier Parameter.....+1.000000E-03   |      Complementarity.....+9.036248E-03
*
*      -----
*      Filter: B(y)                |c(y)|
*      -----
*      | 1 | +2.007440E+00 | +9.036248E-03 |
*      -----
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      ..... (B,|c|) = ( +2.004597E+00, +3.798363E-05 )
*
*      ----- Iteration (6) -----
*      Objective Function.....+2.00095   |      Gradient Of Lagrangian.....+4.473370E-05

```

```

*      Log-Barrier Function.....+2.00131 | Equality Error.....+4.954164E-06 *
*      Barrier Parameter.....+1.000000E-04 | Complementarity.....+9.288175E-04 *
*      ----- *
*      Filter: B(y) |c(y)| *
*      ----- *
*      | 1 | +2.001312E+00 | +9.288175E-04 | *
*      ----- *
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000 *
*
*      ..... (B,|c|) = ( +2.000692E+00, +3.177813E-07 ) *
*
*      ----- Iteration (7) ----- *
*      Objective Function.....+2.00010 | Gradient Of Lagrangian....+2.954553E-06 *
*      Log-Barrier Function.....+2.00016 | Equality Error.....+1.995992E-08 *
*      Barrier Parameter.....+1.000000E-05 | Complementarity.....+9.012976E-05 *
*      ----- *
*      Filter: B(y) |c(y)| *
*      ----- *
*      | 1 | +2.000159E+00 | +9.012976E-05 | *
*      ----- *
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000 *
*
*      ..... (B,|c|) = ( +2.000092E+00, +3.350177E-09 ) *
*
*      ----- Iteration (8) ----- *
*      Objective Function.....+2.00001 | Gradient Of Lagrangian....+3.143777E-07 *
*      Log-Barrier Function.....+2.00001 | Equality Error.....+2.237177E-10 *
*      Barrier Parameter.....+1.000000E-09 | Complementarity.....+1.000036E-05 *
*      ----- *
*      Filter: B(y) |c(y)| *
*      ----- *
*      | 1 | +2.000010E+00 | +1.000036E-05 | *
*      ----- *
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000 *
*
*      ..... (B,|c|) = ( +2.000000E+00, +4.144677E-11 ) *
*
*      ----- Iteration (9) ----- *
*      Objective Function.....+2.00000 | Gradient Of Lagrangian....+3.507998E-08 *
*      Log-Barrier Function.....+2.00000 | Equality Error.....+2.775558E-12 *
*      Barrier Parameter.....+1.000000E-09 | Complementarity.....+4.144677E-11 *
*      ----- *
*      Filter: B(y) |c(y)| *
*      ----- *
*      | 1 | +2.000000E+00 | +4.144677E-11 | *
*      ----- *
*
*      CONSTRAINED OPTIMIZATION COMPLETED *
*
*      ***** CONVERGENCE *****
*
*      -----
*      |
*      | BARNLP ALGORITHM PERFORMANCE STATISTICS |
*      |
*      | Total CPU Time.....+2.999000E-03 |
*      |

```

Number of Function Calls.....	9
Number of Gradient Calls.....	10
Number of Hessian Calls.....	3
Total Number of Function Evaluations.....	9
Storage Needed in HOLD Array.....	580
Storage Needed in IHOLD Array.....	69

```
***** SOLUTION *****
*****
*
* Objective Function =      2.00000          IERNLP =      0
*
* -----
*
* Variable   Status   Value      Lower bound      Upper bound      Lagrange Mlt.      Slack
*
*      1     FR      1.00000      0.00000      10.0000      2.052498E-08      1.000
*      2     FR      1.00000      0.00000      10.0000      3.598589E-08      1.000
*
* -----
*
* Constraint  Status   Value      Lower bound      Upper bound      Lagrange Mlt.      Slack
*
*      1     LB      1.00000      1.00000      4.503600E+15      3.00000      3.2229E-10
*
*****
```

DBNLPR: Dense Barrier Nonlinear Programming–Reverse Communication Format

PURPOSE

DBNLPR computes the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the objective function

$$f(x)$$

subject to the m nonlinear constraints

$$c_L \leq c(x) \leq c_U$$

and the simple bounds

$$x_L \leq x \leq x_U$$

Equality constraints are imposed by setting $c_{Li} = c_{Ui}$ and variables can be fixed by setting $x_{Li} = x_{Ui}$. DBNLPR works under the assumption that the objective and constraint functions are twice continuously differentiable, and if this is not true algorithm performance is unpredictable. DBNLPR uses a reverse communication format and upon request the user must supply the values of the functions, and their first derivatives. The matrix of second derivatives (Hessian of the Lagrangian) can be constructed using a quasi-Newton recursive estimate or supplied by the user. Subroutine DBNLPR calls the sparse nonlinear programming algorithm SBRNLP, and is intended for simplified usage as appropriate for small dense problems.

RELATED SUBPROGRAMS

DNLPRV	Dense Nonlinear Programming–Reverse Communication Format
SBRNLP	Sparse Barrier Nonlinear Programming
INSNLP	Sparse Nonlinear Programming Input Procedure

METHOD

An *interior point or barrier* method is used to solve the *nonlinear programming* (NLP) problem. DBNLPR provides four alternatives for constructing an approximation to the Hessian matrix which are described in the background section of this document. The default is a *symmetric rank-one* (SR1) approximation, with options for using a BFGS, SSQN, or finite difference estimate instead. If the user cannot provide this information analytically, the finite difference techniques described in Chapter 3 should be utilized.

WARNING

The objective function and constraints supplied by the user are assumed to be continuous and have continuous first and second derivatives. Nondifferentiable functions can cause unpredictable performance in the optimization algorithm. It should be emphasized that discontinuities in the Hessian matrices can significantly degrade the speed of convergence without producing any other obvious difficulties. The following common sources of error *should be avoided* when evaluating the objective and constraint functions:

- (1) nonsmooth interpolation of data;

- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;
- (3) discontinuous behavior caused by branching for IF tests;
- (4) nondifferentiable functions such as ABS, MAX, and MIN.

The user must provide a procedure to evaluate the objective and constraint functions in addition to first and (optionally) second derivatives of these quantities. If it is not possible to evaluate the functions (for example, $c(x) = 1/x$ at $x = 0$) then set IFERR = 1 and exit the function evaluation process. Otherwise, set IFERR = 0 when the functions are evaluated. The function error flag should be used to avoid computational difficulties, but never in lieu of an explicit constraint.

The DBNLPR software uses internal common blocks to communicate information. To avoid conflict with these internal commons the user should not have commons with the following names: NPSPRR, NPSPRI, NPSPRC, STATIS, KONSTN, NPSALG, NPSDFL, NLPERR, PERCOM, ITEREF, and CLKCOM.

USAGE

```

INTEGER      IRVCOM, IFERR, NFEVAL,      IER, NEEDED
INTEGER      NDIM, MCON, NONZH, NONZG, NHOLD, NIHOLD
INTEGER      IREVRS(5), ISTATV(NDIM), ISTATC(MCON), IHOLD(NIHOLD)
PARAMETER    (NONZG=NDIM*MCON, NONZH=NDIM*(NDIM+1)/2 )

DOUBLE PRECISION      FBAR, XBAR(NDIM), XLWR(NDIM), XUPR(NDIM), VECNU(NDIM)
DOUBLE PRECISION      DELF(NDIM), HMAT(NONZH), CBAR(MCON), CLWR(MCON)
DOUBLE PRECISION      CUPR(MCON), VECLAM(MCON), GMAT(NONZG), HOLD(NHOLD)

CALL INSNLP('BARRIER DEFAULT')
CALL DBNLPR( IRVCOM, IREVRS,  XBAR,  XLWR,  XUPR, ISTATV,
$           VECNU,  NDIM,  FBAR,  DELF,  HMAT,  CBAR,
$           CLWR,  CUPR, ISTATC,  MCON, VECLAM,  GMAT,
$           IFERR, NFEVAL,  HOLD,  NHOLD,  IHOLD, NIHOLD,
$           NEEDED,  IER )

```

ARGUMENTS

ARGUMENTS TO CONTROL REVERSE COMMUNICATION

IRVCOM [INPUT/OUTPUT, INTEGER]
Continuation control flag. This argument must be initialized. Thereafter it is set by the algorithm and must not be changed by the calling program.

IRVCOM = -1 Initialization pass.

IRVCOM = 0 Termination (normal or abnormal).

IRVCOM = +1 Perform an evaluation and then reenter DBNLPR. Operations to be performed are defined by the output variables IREVRS(1), IREVRS(2), IREVRS(3), and IREVRS(4) below. IREVRS(5) contains information which may be valuable to the interface but does not require any action on the part of the user.

IREVRS [OUTPUT, INTEGER, ARRAY]
 An integer array of length 5, containing reverse communication control information.

IREVRS(1) Function evaluation request
 = 0 Function evaluation not requested; *Warning: do not change the current function and constraint values.*
 = 1 Function evaluation requested; evaluate objective function and all constraints at current point XBAR.

IREVRS(2) Gradient evaluation request
 = 0 Gradient evaluation not requested; *Warning: do not change the current gradient and Jacobian.*
 = 1 Approximate (e.g. forward difference) gradient requested; evaluate gradient vector g and Jacobian matrix G at current point XBAR. (If available, accurate or analytic gradients may be supplied)
 = 2 Accurate (e.g. central difference or analytic) gradient requested; evaluate gradient vector g and Jacobian matrix G at current point XBAR.

IREVRS(3)	Hessian evaluation request	
	= 0	Hessian evaluation not requested; <i>Warning: do not change the current Hessian.</i>
	= 1	Hessian diagonal requested; get ready for full Hessian evaluation with (IREVRS(3)=2) at current point (XBAR,VECLAM) and optionally evaluate diagonal elements of Hessian matrix H .
	= 2	Full Hessian evaluation requested; evaluate Hessian matrix H of the function $L(x, \lambda) = f - \sum_{k=1}^m \lambda_k c_k$ at current point (XBAR,VECLAM). A full evaluation request is always preceded by a call with IREVRS(3)=1 at the same point XBAR, and consequently it may not be necessary to recompute the diagonal elements.
IREVRS(4)	System print (output) request	
	= 0	System print not requested.
	= 1	System print requested; perform any output operations (print, write files, etc.) desired to display problem specific information at the current point XBAR. The frequency of print is defined by the output control flag IOFLAG (see Optional Printout).
IREVRS(5)	Algorithm information flag	
	= 0	Call was made from the interface portion of the algorithm.
	= 1	Call was made from the optimization portion of the algorithm.
	= 2	Call was made from the optimization portion of the algorithm requesting Hessian reset.
	= 3	Call was made from the optimization portion of the algorithm requesting optimal perturbation size adjustment.
	= 4	Call was made from the feasibility portion of the algorithm.

INDEPENDENT VARIABLE DATA

XBAR[†] [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM of current variable values x .

IRVCOM=-1 Initial guess must be input.

IRVCOM=0 Final point is output.

IRVCOM=1 Intermediate iteration points are output.

XLWR[†] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM containing the variable lower bounds x_L . For variables that have no lower bound set $x_{Li} \leq -.01/\text{HDMCON}(5)$. See Appendix B for HDMCON.

XUPR[†] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM containing the variable upper bounds x_U . For variables that have no upper bound set $x_{Ui} \geq .01/\text{HDMCON}(5)$.

ISTATV [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length NDIM indicating variable status. On input ISTATV defines an estimate of the active variable bounds, and on output ISTATV defines the bounds active at the solution.

ISTATV(i)= 0 Variable is strictly feasible, i.e. $x_{Li} < x_i < x_{Ui}$.

ISTATV(i)= 1 Variable lower bound is active, i.e. $x_{Li} = x_i$.

ISTATV(i)= 2 Variable upper bound is active, i.e. $x_i = x_{Ui}$.

ISTATV(i)= 3 Fixed permanently.

ISTATV(i)= 10 Output only: Violated variable lower bound, i.e. $x_{Li} - \delta_c > x_i$.

ISTATV(i)= 20 Output only: Violated variable upper bound, i.e. $x_i > x_{Ui} + \delta_c$.

ISTATV(i)= 30 Output only: Violated fixed variable. Either $x_i < x_{Li} - \delta_c$ or $x_i > x_{Ui} + \delta_c$.

When ISTATV(i) = 3 it is necessary that $x_{Li} = x_{Ui}$, where $x_{Li} = \text{XLWR}(i)$ and $x_{Ui} = \text{XUPR}(i)$. When ISTATV(i) $\neq$ 3, it is necessary that $x_{Li} < x_{Ui}$ with $|x_{Li} - x_{Ui}| > \delta_c$.

VECNU[#] [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
An array of length NDIM of bound multipliers ν .

NDIM[†] [INPUT, INTEGER]
Number of variables n , where $1 \leq \text{NDIM}$.

OBJECTIVE FUNCTION DATA

FBAR[‡] [INPUT, DOUBLE PRECISION]
The objective function evaluated at XBAR, $f(\text{XBAR})$.

DELF[‡] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM containing the gradient g of FBAR at XBAR.

HMAT[‡] [INPUT, DOUBLE PRECISION, ARRAY]
 A real array of length $n(n+1)/2$ containing the elements of Hessian matrix $H_L = \nabla_x^2 f - \sum_{i=1}^m \lambda_i \nabla_x^2 c_i$ of the Lagrangian function. Since the Hessian is symmetric, only the lower triangular part is input (see USAGE REMARKS).

CONSTRAINT DATA

CBAR[‡] [INPUT, DOUBLE PRECISION, ARRAY]
 A real array of length MCON containing the constraints $c(x)$ evaluated at XBAR. Not referenced when MCON = 0.

CLWR[‡] [INPUT, DOUBLE PRECISION, ARRAY]
 A real array of length MCON containing the constraint lower bounds c_L . For constraints that have no lower bound set $c_{Li} \leq -.01/\text{HDMCON}(5)$. Not referenced when MCON = 0.

CUPR[‡] [INPUT, DOUBLE PRECISION, ARRAY]
 A real array of length MCON containing the constraint upper bounds c_U . For constraints that have no upper bound set $c_{Ui} \geq .01/\text{HDMCON}(5)$. Not referenced when MCON = 0.

ISTATC [INPUT/OUTPUT, INTEGER, ARRAY]
 An integer array of length MCON indicating the constraint status. On input ISTATC defines an estimate of the active constraints, and on output ISTATC defines the constraints active at the solution.

- | | |
|---------------|--|
| ISTATC(i)= 0 | Free (inactive) inequality, i.e. $c_{Li} < c_i(x) < c_{Ui}$. |
| ISTATC(i)= 1 | Constraint lower bound is active, i.e. $c_{Li} = c_i(x)$. |
| ISTATC(i)= 2 | Constraint upper bound is active, i.e. $c_i(x) = c_{Ui}$. |
| ISTATC(i)= 3 | Equality. |
| ISTATC(i)= 4 | Ignored constraint. |
| ISTATC(i)= 10 | Output only: Violated constraint lower bound, i.e. $c_{Li} - \delta_c > c_i$. |
| ISTATC(i)= 20 | Output only: Violated constraint upper bound, i.e. $c_i > c_{Ui} + \delta_c$. |
| ISTATC(i)= 30 | Output only: Violated equality constraint. Either $c_i < c_{Li} - \delta_c$ or $c_i > c_{Ui} + \delta_c$. |

When ISTATC(i)=3 it is necessary that $c_{Li} = c_{Ui}$, where $c_{Li} = \text{CLWR}(i)$ and $c_{Ui} = \text{CUPR}(i)$. When ISTATC(i) $\neq$ 3, it is necessary that $c_{Li} < c_{Ui}$ with $|c_{Li} - c_{Ui}| > \delta_c$. Not referenced when MCON = 0.

MCON[‡] [INPUT, INTEGER]
 Number of constraints m , where $0 \leq \text{MCON}$.

VECLAM[#] [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
 A real array of length MCON of Lagrange multipliers, λ , for constraints. The user must evaluate the Hessian of $L(x, \lambda) = f - \sum_{k=1}^m \lambda_k c_k$ (see IREVRS(3)). Not referenced when MCON = 0.

GMAT[‡] [INPUT, DOUBLE PRECISION, ARRAY]
 A real array of length NDIM*MCON containing the elements of the Jacobian matrix G at XBAR (see USAGE REMARKS). Not referenced when MCON = 0.

ALGORITHM CONTROL DATA

IFERR[‡] [INPUT, INTEGER]
 Function evaluation error flag.
 IFERR= 1 When FBAR and/or CBAR cannot be evaluated.
 IFERR= 0 Otherwise.

NFEVAL[‡] [INPUT, INTEGER]
 Number of function evaluations. Normally NFEVAL is initialized prior to the first call when IRVCOM = -1, and incremented whenever IREVRS(1), IREVRS(2), or IREVRS(3) are nonzero. It may be appropriate to increment NFEVAL for each perturbation when finite difference gradients are used, or not at all when analytic gradients are used.

WORKING STORAGE

HOLD [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
 A real array of length NHOLD containing the hold array (working storage that must not be destroyed by the user).

NHOLD[†] [INPUT/OUTPUT, INTEGER]
 Dimension of HOLD array. When IER = -127, -131, or -701 NEEDED contains the required storage.

IHOLD [INPUT/OUTPUT, INTEGER, ARRAY]
 An integer array of length NIHOLD containing the integer hold array (working storage that must not be destroyed by the user).

NIHOLD[†] [INPUT/OUTPUT, INTEGER]
 Dimension of IHOLD array. When IER = -128, -132, or -702, NEEDED contains the required storage.

OUTPUT ARGUMENTS SET BY ALGORITHM BEFORE TERMINATION (IRVCOM = 0)

NEEDED [OUTPUT, INTEGER]
 When IER = -127, -131, or -701, NEEDED contains the required length of HOLD. When IER = -128, -132, or -702, NEEDED contains the required length for IHOLD.

IER

[OUTPUT, INTEGER]

Success/error code. Results are not computed for $IER < 0$. For $IER > 0$, results are computed but may be suspect; user inspection is recommended. Note many of the error returns involve optional input quantities as described in INSNLP.

IER=0 Normal termination.

IER=-101 (MCON < 0); the number of constraints is negative.

IER=-102 (NDIM < 1); the number of variables is less than one.

IER=-103 Incorrect value for QPOPTN.

IER=-104 |IHESHN| > 3 .

IER=-105 (NITMAX $< \max(1, \text{NITMIN})$); the maximum number of iterations is either less than 1 or less than the minimum number of iterations.

IER=-107 (IT1MAX < 1); the number of line search steps is less than 1.

IER=-108 (IOFLAG < 0) or (IOFLAG > 30); invalid input for the output control flag.

IER=-109 Invalid input for ALGOPT.

IER=-110 (OBJTOL $\leq 10 \cdot \text{HDMCON}(5)$); objective function tolerance is too small.

IER=-111 (PGDTOL $\leq [\text{HDMCON}(5)]^{\frac{1}{2}}$) or (PGDTOL $> 10^{-2}$); the projected gradient tolerance is too small or too large.

IER=-112 CONTOL $< [\text{HDMCON}(5)]^{\frac{1}{2}}$; the constraint tolerance is too small.

IER=-113 (ISTATC(i) < 0) or (ISTATC(i) > 4); invalid input for constraint status.

IER=-117 (CUPR(i) $< \text{CLWR}(i)$); constraint upper bound is less than lower bound.

IER=-118 Either (CUPR(i) = CLWR(i)) and (ISTATC(i) $\neq 3$) or (CUPR(i) $\neq \text{CLWR}(i)$) and (ISTATC(i) = 3); constraint status array is not consistent with bounds.

IER=-119 (CUPR(i) $\neq \text{CLWR}(i)$) and ($|\text{CUPR}(i) - \text{CLWR}(i)| < \text{CONTOL}$); constraint bounds are not equal, but differ by less than the constraint tolerance.

IER=-120 (ISTATV(i) < 0) or (ISTATV(i) > 3); invalid input for variable status array.

IER=-124	(XUPR(i) < XLWR(i)); the variable upper bound is less than the lower bound.
IER=-125	Either (XUPR(i) = XLWR(i)) and (ISTATV(i) ≠ 3) or (XUPR(i) ≠ XLWR(i)) and (ISTATV(i) = 3); the variable status array is not consistent with the bound values.
IER=-126	(XUPR(i) ≠ XLWR(i)) and (XUPR(i)-XLWR(i) < CONTOL); variable bounds are not equal but differ by less than the constraint tolerance.
IER=-127	Double precision hold array too small; insufficient storage detected in SBRNLP interface. The required storage is specified in NEEDED.
IER=-128	Integer hold array too small; insufficient storage detected in SBRNLP interface. The required storage is specified in NEEDED.
IER=-129	Function error at initial point or during gradient evaluation.
IER=-130	BIGCON < CONTOL.
IER=-131	NHOLD too small; insufficient double precision storage detected in algorithm. The required storage is specified in NEEDED.
IER=-132	NIHOLD too small; insufficient integer storage detected in algorithm. The required storage is specified in NEEDED.
IER=-133	Rank deficient Jacobian detected on successive iterations.
IER=-134	INSNLP input error; invalid character string displayed.
IER=-135	(IFERR < 0) or (IFERR > 1); invalid value for function error flag.
IER=-136	FEATOL < CONTOL; the initial feasibility tolerance is less than the constraint tolerance.
IER=-137	Conflict between user and multifrontal file number; check IPUMF1,...,IPUMF6 IPUDRF, IPUFZF, IPUSTF.
IER=-138	NEWTON ≠ 0, 1, 2; invalid input for Newton method option flag.
IER=-146	PMULWR < HDMCON(5).
IER=-147	Unexpected error; check storage allocation.
IER=-148	IRVCOM > 1.
IER=-149	PTHOTOL < [HDMCON(5)] ^{1/2} .

IER=-150	RHOLWR < HDMCON(5).
IER=-151	A constraint is inconsistent (either $c_i(x) < c_{Li}$ or $c_{Ui} < c_i(x)$) and cannot be changed because the corresponding row of the Jacobian is zero.
IER=-152	IMAXMU < 1.
IER=-153	I/O error, probably caused by insufficient disk space.
IER=-154	TOLKTC < 1.
IER=-155	TOLPVT < 0 or TOLPVT > .5.
IER=-156	IRELAX < 0 or IRELAX > 2.
IER=-157	MUCALC = 0 or MUCALC > 3.
IER=-158	MXQPIT < 1.
IER=-701	Double Precision hold array too small; insufficient storage detected in dense interface. The required storage is specified in NEEDED.
IER=-702	Integer hold array too small; insufficient storage detected in dense interface. The required storage is specified in NEEDED.

IER=+101	Weak solution found (relaxation required and/or multipliers near zero).
IER=+102	Number of equality constraints = number of free variables and ALGOPT $\neq$ 'F'.
IER=+103	Maximum number of consecutive function errors.
IER=+104	Maximum number of function evaluations.
IER=+105	Small step termination in optimization phase; suboptimal feasible point found.
IER=+106	Maximum number of iterations.
IER=+108	Feasible point not found.
IER=+109	Maximum number of interval halves in line search.

IER=+110	Algorithm terminated because the Hessian diagonal reached its maximum value; suboptimal feasible point found.
IER=+111	Projected gradient calculation failed; constraints may be degenerate.
IER=+112	Calculation of first order multiplier estimates failed; constraints may be degenerate.
IER=+113	Suboptimal feasible point found.
IER=+114	Barrier NLP failed with unexpected error.
IER=+115	CONTOL > OBJTOL; convergence tolerances may be inappropriate.
IER=+116	Uphill direction detected in line search.
IER=+117	Reduced objective function is linear.
IER=+118	Both $c_{Li} \leq -.01/\text{HDMCON}(5)$ and $c_{Ui} \geq .01/\text{HDMCON}(5)$; constraints ignored.
IER=+119	Terminate after diagnostic line search.
IER=+120	(IHESHN > 0) and (NEWTON = 0); recursive Hessian updates may be skipped causing poor algorithm performance.

†	Variable set when IRVCOM=-1, and not changed by user thereafter
‡	Variable set when IRVCOM=+1, as required by IREVRS(*)
#	Variable optionally set when IRVCOM=-1, and not changed by user thereafter

USAGE REMARKS

When IHESHN = 0 and IREVRS(3) > 0, the user must supply the Hessian matrix of the following Lagrangian function:

$$L(x, \lambda) = f - \sum_{k=1}^m \lambda_k c_k$$

that is,

$$L(\text{XBAR}, \text{VECLAM}) = \text{FBAR} - \text{VECLAM}(1) * \text{CBAR}(1) - \dots - \text{VECLAM}(\text{MCON}) * \text{CBAR}(\text{MCON}) .$$

Consider the following example in which $n = 3$:

$$H = \begin{pmatrix} 1.1 & 2.1 & 3.1 \\ 2.1 & 2.2 & 3.2 \\ 3.1 & 3.2 & 3.3 \end{pmatrix}$$

Since the Hessian is a symmetric matrix only the lower triangular portion must be specified. For this example NDIM = 3 and the values are specified as HMAT(*) = (1.1, 2.1, 3.1, 2.2, 3.2, 3.3).

Notice that the columns of the lower triangular portion are stored sequentially and the elements in the upper triangular portion are not specified.

When the problem is unconstrained the user must set $MCON = 0$. The following constraint related quantities will not be referenced: CBAR, CLWR, CUPR, ISTATC, VECLAM, and GMAT. Since these quantities must appear as arguments in the call to SBRNLP, dummy values may be utilized.

A computational flow chart of a reverse communication optimization algorithm is found in the documentation of SBRNLP.

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine DBNLPR. All optional inputs are specified by calls to subroutine INSNLP. It should be emphasized that it is *not necessary* to call INSNLP if the default values are appropriate.

OPTIONAL PRINTOUT

The level of printed output from DBNLPR is controlled by the variables IOFLAG and IPUNLP which are set by calls to INSNLP. The standard level of output which corresponds to IOFLAG = 10 is the *default* for DBNLPR. A complete description of the iteration output options is found in Section 2.3.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2, x_3, x_4)$ to minimize the objective function

$$f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2 + 100(x_4 - x_3^2)^2 + (1 - x_3)^2$$

subject to the two constraints

$$x_1 x_2 \leq \frac{1}{2} \qquad -\frac{1}{2} \leq x_1 + x_3 \leq \frac{1}{2}$$

and the bounds

$$-2 \leq x_k \leq 2$$

for $k = 1, 2, 3, 4$ beginning at the initial point $x = (1, 2, 1, 2)$. For this example the gradient vector is

$$g = \nabla_x f = \begin{bmatrix} -400x_1(x_2 - x_1^2) - 2(1 - x_1) \\ 200(x_2 - x_1^2) \\ -400x_3(x_4 - x_3^2) - 2(1 - x_3) \\ 200(x_4 - x_3^2) \end{bmatrix}$$

and the Jacobian matrix is

$$G = \begin{bmatrix} \nabla_x c_1^T \\ \nabla_x c_2^T \end{bmatrix} = \begin{bmatrix} x_2 & x_1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}.$$

When IHESHN = 0 and IREVR(3) > 0 it is also necessary to supply the Hessian matrix

$$\nabla_x^2 L = \nabla_x^2 f - \lambda_1 \nabla_x^2 c_1 - \lambda_2 \nabla_x^2 c_2$$

where

$$\nabla_x^2 f = \begin{bmatrix} 1200x_1^2 - 400x_2 + 2 & -400x_1 & 0 & 0 \\ -400x_1 & 200 & 0 & 0 \\ 0 & 0 & 1200x_3^2 - 400x_4 + 2 & -400x_3 \\ 0 & 0 & -400x_3 & 200 \end{bmatrix}$$

with

$$\nabla_x^2 c_1 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

and $\nabla_x^2 c_2 = 0$.

Note: This example also illustrates the use of the “Terse Output” option, by the call to INSNLP with “IOFLAG = 1”.

SAMPLE PROGRAM (dbnlpr.cd1)⁹

```

PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE NONLINEAR
C      PROGRAMMING ALGORITHM DBNLPR
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      PARAMETER (NDIM=4, MCON=2)
C      PARAMETER (NONZH=NDIM*(NDIM+1)/2, NONZG=NDIM*MCON)
C      PARAMETER (NHOLD=5000,NIHOLD=5000)
C
C      DIMENSION IREVRS(5),XBAR(NDIM),XLWR(NDIM),XUPR(NDIM),
$      ISTATV(NDIM),VECNU(NDIM),DELF(NDIM),HMAT(NONZH),
$      CBAR(MCON),CLWR(MCON),
$      CUPR(MCON),ISTATC(MCON),VECLAM(MCON),
$      GMAT(NONZG)
C      DIMENSION HOLD(NHOLD),IHOLD(NIHOLD)
C
C      INITIALIZE REVERSE COMMUNICATION CONTROL FLAG
C
C      ICC = -1
C
C -----
C
C      INDEPENDENT VARIABLE DATA
C
C      ----INITIAL GUESS
C      XBAR(1) = 1.0D0
C      XBAR(2) = 2.0D0

```

⁹sosexample.tar

```
      XBAR(3) = 1.0D0
      XBAR(4) = 2.0D0
C     ----BOUNDS
      xlwr(1:ndim) = -2.d0
      xupr(1:ndim) = 2.d0
C     ----VARIABLE STATUS
      istatv(1:ndim) = 0
C
C -----
C
C     CONSTRAINT DATA
C
C     ----BOUNDS
      CLWR(1) = -1.D0/HDMCON(5)
      CUPR(1) = .5D0
      CLWR(2) = -.5D0
      CUPR(2) = .5D0
C     ----CONSTRAINT STATUS
      istatc(1:mcon) = 0
C
C -----
C
C     INITIALIZE FUNCTION EVALUATION COUNT
C
      NFEVAL = 0
C
C     SET OPTIONAL INPUTS
C
      CALL INSNLP('BARRIER DEFAULT')
      CALL INSNLP('IOFLAG=1')
C
C -----
C
      1000 CONTINUE
C
      CALL DBNLPR( ICC ,IREVRS ,XBAR ,XLWR ,XUPR ,ISTATV
$              ,VEGNU ,NDIM ,FBAR ,DELF ,HMAT ,CBAR
$              ,CLWR ,CUPR ,ISTATC ,MCON ,VECLAM ,GMAT
$              ,IFERR ,NFEVAL ,HOLD ,NHOLD ,IHOLD ,NIHOLD
$              ,NEEDED ,IERNLP )
C
C     REVERSE COMMUNICATION TERMINATION TEST
C
      IF(ICC.EQ.0) GO TO 10000
C
C -----
C
C     SET FUNCTION EVALUATION ERROR FLAG
```

```

C
    IFERR = 0
C
C    >>>>FUNCTION EVALUATION REQUEST
C
    IF(IREVRS(1).EQ.1) THEN
C
        FBAR = 100.DO*(XBAR(2) - XBAR(1)**2)**2
$           + (1.DO - XBAR(1))**2
$           + 100.DO*(XBAR(4) - XBAR(3)**2)**2
$           + (1.DO - XBAR(3))**2
C
        CBAR(1) = XBAR(1)*XBAR(2)
        CBAR(2) = XBAR(1) + XBAR(3)
        NFEVAL = NFEVAL + 1
C
    ENDIF
C
C    >>>>GRADIENT EVALUATION REQUEST
C
    IF(IREVRS(2).EQ.1) THEN
C
        DELF(1) = - 400.DO*XBAR(1)*(XBAR(2) - XBAR(1)**2)
$           - 2.DO*(1.DO - XBAR(1))
        DELF(2) = 200.DO*(XBAR(2) - XBAR(1)**2)
        DELF(3) = - 400.DO*XBAR(3)*(XBAR(4) - XBAR(3)**2)
$           - 2.DO*(1.DO - XBAR(3))
        DELF(4) = 200.DO*(XBAR(4) - XBAR(3)**2)
C
        GMAT(1) = XBAR(2)
        GMAT(2) = 1.DO
        GMAT(3) = XBAR(1)
        GMAT(4) = 0.DO
        GMAT(5) = 0.DO
        GMAT(6) = 1.DO
        GMAT(7) = 0.DO
        GMAT(8) = 0.DO
C
    ELSEIF(IREVRS(2).EQ.2) THEN
C
        DELF(1) = - 400.DO*XBAR(1)*(XBAR(2) - XBAR(1)**2)
$           - 2.DO*(1.DO - XBAR(1))
        DELF(2) = 200.DO*(XBAR(2) - XBAR(1)**2)
        DELF(3) = - 400.DO*XBAR(3)*(XBAR(4) - XBAR(3)**2)
$           - 2.DO*(1.DO - XBAR(3))
        DELF(4) = 200.DO*(XBAR(4) - XBAR(3)**2)
C
        GMAT(1) = XBAR(2)

```

```
      GMAT(2) = 1.DO
      GMAT(3) = XBAR(1)
      GMAT(4) = 0.DO
      GMAT(5) = 0.DO
      GMAT(6) = 1.DO
      GMAT(7) = 0.DO
      GMAT(8) = 0.DO
C
      ENDIF
C
C      >>>>HESSIAN EVALUATION REQUEST
C
C      (NOTE: THE HESSIAN EVALUATIONS BELOW ARE ONLY
C              NEEDED WHEN IHESHN=0)
C
      IF(IREVRS(3).EQ.1) THEN
C
C          DIAGONAL ELEMENTS
C
          HMAT(1) = 1200.DO*XBAR(1)**2
$          - 400.DO*XBAR(2) + 2.DO
          HMAT(5) = 200.DO
          HMAT(8) = 1200.DO*XBAR(3)**2
$          - 400.DO*XBAR(4) + 2.DO
          HMAT(10) = 200.DO
C
      ELSEIF(IREVRS(3).EQ.2) THEN
C
C          OFF-DIAGONAL ELEMENTS
C
          HMAT(2) = -VECLAM(1)*(1.DO) - 400.DO*XBAR(1)
          HMAT(3) = 0.DO
          HMAT(4) = 0.DO
          HMAT(6) = 0.DO
          HMAT(7) = 0.DO
          HMAT(9) = - 400.DO*XBAR(3)
C
      ENDIF
C
C      >>>>SYSTEM PRINT (OUTPUT) REQUEST
C
C      IF(IREVRS(4).EQ.1) THEN
C
          WRITE(6,1001) 'OBJECTIVE ',FBAR
          WRITE(6,1001) 'VARIABLES ',(XBAR(II),II=1,NDIM)
          WRITE(6,1001) 'CONSTRAINTS',(CBAR(II),II=1,MCON)
C
      ENDIF
```

```

C
C -----
C
C      END OF REVERSE COMMUNICATION PROCESSING.  RETURN TO DBNLPR
C
C      GO TO 1000
C
C 10000 CONTINUE
C
C      TERMINATE PROCESSING
C
C      IF(IERNLP.EQ.0) THEN
C
C          NORMAL TERMINATION---SOLUTION FOUND
C
C          WRITE(6,1001) 'SOLUTION    '
C
C          WRITE(6,1001) 'OBJECTIVE  ',FBAR
C          WRITE(6,1001) 'VARIABLES  ',(XBAR(II),II=1,NDIM)
C          WRITE(6,1001) 'CONSTRAINTS',(CBAR(II),II=1,MCON)
C
C      ELSEIF(IERNLP.LT.0) THEN
C
C          ABNORMAL TERMINATION---FATAL ERROR
C
C          PRINT *, 'FATAL ERROR; IERNLP =', IERNLP
C
C      ELSE
C
C          ABNORMAL TERMINATION---WARNING.  EVALUATION OF RESULTS RECOMMENDED!
C
C          PRINT *, 'WARNING MESSAGE; IERNLP =', IERNLP
C
C      ENDIF
C
C 1001 FORMAT(5X,A11,5X,4G17.8)
C
C      STOP
C      END

```

OUTPUT FROM SAMPLE PROGRAM

It	Lnfltr	KT	Cond	Step	Levnbrg	Optimization				Obj	Log-Barrier Fn
						Cmplmt	Barrier	Violtn	Grdl		
(1)	1	8.8E+04	2.5E-02	0.0E+00	0.0E+00	1.4E-01	1.5E+00	2.5E+02	1.9E+02	1.935611E+02	
(2)	2	3.1E+07	5.4E-02	0.0E+00	1.4E-01	1.4E-01	1.5E+00	2.4E+02	1.9E+02	1.948491E+02	
(3)	1	2.1E+07	9.9E-01	0.0E+00	1.4E-01	1.4E-01	1.4E+00	2.4E+02	1.7E+02	1.708350E+02	
(4)	1	6.6E+04	4.0E-01	0.0E+00	3.3E-01	1.4E-02	1.1E-01	5.9E+02	6.3E+02	6.287785E+02	
(5)	1	3.5E+06	1.0E+00	0.0E+00	3.3E-01	1.4E-02	7.3E-02	4.5E+02	2.4E+02	2.381381E+02	

(6)	1	3.1E+05	2.0E-01	0.0E+00	3.4E-02	1.4E-02	2.5E-02	6.5E+01	3.6E+00	3.690829E+00
(7)	1	1.5E+05	1.0E+00	0.0E+00	2.6E-02	1.4E-02	2.0E-02	4.7E+01	3.3E+00	3.405584E+00
(8)	2	2.7E+03	1.0E+00	0.0E+00	2.3E-01	1.4E-02	1.1E-03	3.1E+00	1.9E+00	1.933416E+00
(9)	2	3.7E+03	1.0E+00	0.0E+00	7.5E-02	1.4E-02	1.1E-02	5.4E+00	1.6E+00	1.630948E+00
(10)	2	2.1E+03	1.0E+00	3.0E-08	6.8E-02	1.4E-02	1.4E-02	7.6E+00	1.6E+00	1.610440E+00
(11)	1	7.6E+02	1.0E+00	0.0E+00	1.1E-02	1.4E-02	7.4E-03	1.9E+00	1.4E+00	1.385251E+00
(12)	2	8.1E+02	1.0E+00	0.0E+00	3.9E-03	1.4E-02	2.0E-02	4.3E+00	1.3E+00	1.294883E+00
(13)	2	4.6E+02	1.0E+00	3.0E-08	2.7E-03	1.4E-02	1.6E-02	5.8E-01	1.3E+00	1.264586E+00
(14)	2	5.0E+02	1.0E+00	3.0E-08	2.3E-03	1.4E-02	1.5E-02	6.6E-01	1.2E+00	1.227338E+00
(15)	1	6.4E+02	1.0E+00	2.7E-08	1.4E-03	1.2E-02	1.2E-02	1.1E+00	1.2E+00	1.185220E+00
(16)	1	5.7E+02	1.0E+00	0.0E+00	8.3E-05	1.2E-02	3.4E-03	9.9E-01	1.2E+00	1.153458E+00
(17)	1	4.1E+02	1.0E+00	0.0E+00	4.6E-06	1.2E-02	7.1E-04	2.8E-01	1.1E+00	1.140677E+00
(18)	1	4.0E+02	1.0E+00	0.0E+00	1.1E-02	1.2E-03	4.2E-04	1.6E-01	1.1E+00	1.137593E+00
(19)	1	1.7E+03	1.0E+00	0.0E+00	1.2E-04	1.2E-03	3.6E-06	6.7E-02	1.1E+00	1.129187E+00
(20)	1	1.9E+03	1.0E+00	0.0E+00	1.1E-03	1.2E-04	1.3E-05	5.2E-03	1.1E+00	1.126530E+00
(21)	1	1.8E+04	1.0E+00	0.0E+00	1.1E-04	1.2E-05	9.5E-08	7.8E-04	1.1E+00	1.125182E+00
(22)	1	1.8E+05	1.0E+00	0.0E+00	1.2E-05	1.5E-09	5.9E-12	1.5E-04	1.1E+00	1.125012E+00
(23)	1	1.8E+05	1.0E+00	0.0E+00	1.0E-10	1.5E-09	5.4E-12	1.1E-05	1.1E+00	1.125000E+00

BARNLP ALGORITHM PERFORMANCE STATISTICS

Total CPU Time.....	+3.842568E-02
Number of Function Calls.....	27
Number of Gradient Calls.....	24
Number of Hessian Calls.....	16
Total Number of Function Evaluations.....	27
Storage Needed in HOLD Array.....	770
Storage Needed in IHOLD Array.....	161

SOLUTION

OBJECTIVE	1.1250000			
VARIABLES	0.25000058	0.62500321E-01	0.24999942	0.62499654E-01
CONSTRAINTS	0.15625116E-01	0.50000000		

SBRNLP: Sparse Barrier Nonlinear Programming

PURPOSE

SBRNLP computes the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the objective function

$$f(x)$$

subject to the m nonlinear constraints

$$c_L \leq c(x) \leq c_U$$

and the simple bounds

$$x_L \leq x \leq x_U.$$

Equality constraints are imposed by setting $c_{Li} = c_{Ui}$ and variables can be fixed by setting $x_{Li} = x_{Ui}$. SBRNLP works under the assumption that the objective and constraint functions are twice continuously differentiable, and if this is not true algorithm performance is unpredictable. SBRNLP uses a reverse communication format and upon request the user must supply the values of the functions, and their first and second derivatives. The matrix of first derivatives (the Jacobian) and the matrix of second derivatives (Hessian of the Lagrangian) are represented in a sparse format consistent with that used in BCSLIB-EXT [\[1\]](#).

RELATED SUBPROGRAMS

SNLPMN	Sparse Nonlinear Programming
INSNLP	Sparse Nonlinear Programming Input Procedure

METHOD

An *interior point or barrier* method is used to solve the *nonlinear programming* (NLP) problem. The barrier algorithm requires an estimate for the Hessian matrix H_L . If the user cannot provide this information analytically, the sparse finite difference techniques described in Chapter [3](#) should be utilized.

WARNING

The objective function and constraints supplied by the user are assumed to be continuous and have continuous first and second derivatives. Non-differentiable functions can cause unpredictable performance in the optimization algorithm. It should be emphasized that discontinuities in the Hessian matrices can significantly degrade the speed of convergence without producing any other obvious difficulties. The following common sources of error *should be avoided* when evaluating the objective and constraint functions:

- (1) non-smooth interpolation of data;
- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;
- (3) discontinuous behavior caused by branching for IF tests;

(4) non-differentiable functions such as ABS, MAX, and MIN.

The user must provide a procedure to evaluate the objective and constraint functions in addition to first and second derivatives of these quantities. If it is not possible to evaluate the functions (for example, $c(x) = 1/x$ at $x = 0$) then set IFERR = 1 and exit the function evaluation process. Otherwise, set IFERR = 0 when the functions are evaluated. The function error flag should be used to avoid computational difficulties, but never in lieu of an explicit constraint.

The SBRNLP software uses internal common blocks to communicate information. To avoid conflict with these internal commons the user should not have commons with the following names: NPSPRR, NPSPRI, NPSPRC, STATIS, KONSTN, NPSALG, NPSDFL, NLPERR, PERCOM, ITEREF, and CLKCOM.

USAGE

```

INTEGER      IRVCOM, IFERR, NFEVAL, IER, NEEDED
INTEGER      NDIM, MCON, NONZH, NONZG, NHOLD, NIHOLD
INTEGER      IREVRS(5), ISTATV(NDIM), IROWH(NONZH), JSTRH(NDIM+1)
INTEGER      ISTATC(MCON), IROWG(NONZG), JCOLG(NONZG), IHOLD(NIHOLD)

DOUBLE PRECISION      FBAR, XBAR(NDIM), XLWR(NDIM), XUPR(NDIM), VECNU(NDIM)
DOUBLE PRECISION      DELF(NDIM), HMAT(NONZH), CBAR(MCON), CLWR(MCON)
DOUBLE PRECISION      CUPR(MCON), VECLAM(MCON), GMAT(NONZG), HOLD(NHOLD)

```

```

CALL INSNLP('BARRIER DEFAULT')
CALL SBRNLP( IRVCOM ,IREVRS ,XBAR      ,XLWR      ,XUPR      ,ISTATV
$           ,VECNU  ,NDIM    ,FBAR      ,DELF      ,HMAT      ,IROWH
$           ,JSTRH  ,NONZH   ,CBAR      ,CLWR      ,CUPR      ,ISTATC
$           ,MCON   ,VECLAM ,GMAT      ,IROWG      ,JCOLG      ,NONZG
$           ,IFERR  ,NFEVAL ,HOLD      ,NHOLD      ,IHOLD      ,NIHOLD
$           ,NEEDED ,IER    )

```

ARGUMENTS

ARGUMENTS TO CONTROL REVERSE COMMUNICATION

IRVCOM [INPUT/OUTPUT, INTEGER]
Continuation control flag. This argument must be initialized. Thereafter it is set by the algorithm and must not be changed by the calling program.

IRVCOM = -1 Initialization pass.

IRVCOM = 0 Termination (normal or abnormal).

IRVCOM = +1 Perform an evaluation and then reenter SBRNLP. Operations to be performed are defined by the output variables IREVRS(1), IREVRS(2), IREVRS(3), and IREVRS(4) below. IREVRS(5) contains information which may be valuable to the interface but does not require any action on the part of the user.

IREVRS [OUTPUT, INTEGER, ARRAY]
 An integer array of length 5, containing reverse communication control information.

IREVRS(1) Function evaluation request
 = 0 Function evaluation not requested; *Warning: do not change the current function and constraint values.*
 = 1 Function evaluation requested; evaluate objective function and all constraints at current point XBAR.

IREVRS(2) Gradient evaluation request
 = 0 Gradient evaluation not requested; *Warning: do not change the current gradient and Jacobian.*
 = 1 Approximate (e.g. forward difference) gradient requested; evaluate gradient vector g and Jacobian matrix G at current point XBAR. (If available, accurate or analytic gradients may be supplied)
 = 2 Accurate (e.g. central difference or analytic) gradient requested; evaluate gradient vector g and Jacobian matrix G at current point XBAR.

IREVRS(3)	Hessian evaluation request	
	= 0	Hessian evaluation not requested; <i>Warning: do not change the current Hessian.</i>
	= 1	Hessian diagonal requested; get ready for full Hessian evaluation with (IREVRS(3)=2) at current point (XBAR,VECLAM) and optionally evaluate diagonal elements of Hessian matrix H .
	= 2	Full Hessian evaluation requested; evaluate Hessian matrix H of the function $L(x, \lambda) = f - \sum_{k=1}^m \lambda_k c_k$ at current point (XBAR,VECLAM). A full evaluation request is always preceded by a call with IREVRS(3)=1 at the same point XBAR, and consequently it may not be necessary to recompute the diagonal elements.
IREVRS(4)	System print (output) request	
	= 0	System print not requested.
	= 1	System print requested; perform any output operations (print, write files, etc.) desired to display problem specific information at the current point XBAR. The frequency of print is defined by the output control flag IOFLAG (see Optional Printout).
IREVRS(5)	Algorithm information flag	
	= 0	Call was made from the interface portion of the algorithm.
	= 1	Call was made from the optimization portion of the algorithm.
	= 2	Call was made from the optimization portion of the algorithm requesting Hessian reset.
	= 3	Call was made from the optimization portion of the algorithm requesting optimal perturbation size adjustment.
	= 4	Call was made from the feasibility portion of the algorithm.

INDEPENDENT VARIABLE DATA

XBAR[†] [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
 A real array of length NDIM of current variable values x .

IRVCOM=-1 Initial guess must be input.

IRVCOM=0 Final point is output.

IRVCOM=1 Intermediate iteration points are output.

XLWR[†] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM containing the variable lower bounds x_L . For variables that have no lower bound set $x_{Li} \leq -.01/\text{HDMCON}(5)$. See Appendix B for HDMCON.

XUPR[†] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM containing the variable upper bounds x_U . For variables that have no upper bound set $x_{Ui} \geq .01/\text{HDMCON}(5)$.

ISTATV[†] [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length NDIM indicating variable status. On input ISTATV defines an estimate of the active variable bounds, and on output ISTATV defines the bounds active at the solution.

ISTATV(i)= 0 Variable is strictly feasible, i.e. $x_{Li} < x_i < x_{Ui}$.

ISTATV(i)= 1 Variable lower bound is active, i.e. $x_{Li} = x_i$.

ISTATV(i)= 2 Variable upper bound is active, i.e. $x_i = x_{Ui}$.

ISTATV(i)= 3 Fixed permanently.

ISTATV(i)= 10 Output only: Violated variable lower bound, i.e. $x_{Li} - \delta_c > x_i$.

ISTATV(i)= 20 Output only: Violated variable upper bound, i.e. $x_i > x_{Ui} + \delta_c$.

ISTATV(i)= 30 Output only: Violated fixed variable. Either $x_i < x_{Li} - \delta_c$ or $x_i > x_{Ui} + \delta_c$.

When ISTATV(i) = 3 it is necessary that $x_{Li} = x_{Ui}$, where $x_{Li} = \text{XLWR}(i)$ and $x_{Ui} = \text{XUPR}(i)$. When ISTATV(i) $\neq$ 3, it is necessary that $x_{Li} < x_{Ui}$ with $|x_{Li} - x_{Ui}| > \delta_c$.

VECNU[#] [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
An array of length NDIM of bound multipliers ν .

NDIM[†] [INPUT, INTEGER]
Number of variables n , where $1 \leq \text{NDIM}$.

OBJECTIVE FUNCTION DATA

FBAR[‡] [INPUT, DOUBLE PRECISION]
The objective function evaluated at XBAR, $f(\text{XBAR})$.

DELF[‡] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM containing the gradient g of FBAR at XBAR.

HMAT[‡] [INPUT, DOUBLE PRECISION, ARRAY]
 A real array of length NONZH containing the nonzero elements of Hessian matrix $H_L = \nabla_x^2 f - \sum_{i=1}^m \lambda_i \nabla_x^2 c_i$ of the Lagrangian function. Since the Hessian is symmetric, only the lower triangular part is input (see USAGE REMARKS). Storage for the diagonal elements must be provided even if they are zero.

IROWH[†] [INPUT, INTEGER, ARRAY]
 An integer array of length NONZH containing the row indices of the Hessian nonzeros, where $0 < \text{IROWH}(k) \leq \text{NDIM}$ for $k = 1, \dots, \text{NONZH}$. The diagonal elements must be specified, i.e. $\text{IROWH}(\text{JSTRH}(k)) = k$ for $k = 1, \dots, \text{NDIM}$.

JSTRH[†] [INPUT, INTEGER, ARRAY]
 An integer array of length NDIM+1 containing the column start indices of nonzeros, where $\text{JSTRH}(1) = 1$, $\text{JSTRH}(\text{NDIM}+1) = \text{NONZH} + 1$, and $\text{JSTRH}(k) < \text{JSTRH}(k+1)$ for $k = 1, \dots, \text{NDIM}$.

NONZH[†] [INPUT, INTEGER]
 Number of nonzero Hessian elements, where $n \leq \text{NONZH} \leq n(n+1)/2$.

CONSTRAINT DATA

CBAR[‡] [INPUT, DOUBLE PRECISION, ARRAY]
 A real array of length MCON containing the constraints $c(x)$ evaluated at XBAR. Not referenced when MCON = 0.

CLWR[†] [INPUT, DOUBLE PRECISION, ARRAY]
 A real array of length MCON containing the constraint lower bounds c_L . For constraints that have no lower bound set $c_{Li} \leq -.01/\text{HDMCON}(5)$. Not referenced when MCON = 0.

CUPR[†] [INPUT, DOUBLE PRECISION, ARRAY]
 A real array of length MCON containing the constraint upper bounds c_U . For constraints that have no upper bound set $c_{Ui} \geq .01/\text{HDMCON}(5)$. Not referenced when MCON = 0.

ISTATC[†] [INPUT/OUTPUT, INTEGER, ARRAY]
 An integer array of length MCON indicating the constraint status. On input ISTATC defines an estimate of the active constraints, and on output ISTATC defines the constraints active at the solution.

ISTATC(i)= 0 Free (inactive) inequality, i.e. $c_{Li} < c_i(x) < c_{Ui}$.

ISTATC(i)= 1 Constraint lower bound is active, i.e. $c_{Li} = c_i(x)$.

ISTATC(i)= 2 Constraint upper bound is active, i.e. $c_i(x) = c_{Ui}$.

ISTATC(i)= 3 Equality.

ISTATC(i)= 4 Ignored constraint.

ISTATC(i)= 10 Output only: Violated constraint lower bound, i.e. $c_{Li} - \delta_c > c_i$.

ISTATC(i)= 20 Output only: Violated constraint upper bound , i.e. $c_i > c_{Ui} + \delta_c$.

ISTATC(i)= 30 Output only: Violated equality constraint. Either $c_i < c_{Li} - \delta_c$ or $c_i > c_{Ui} + \delta_c$.

When ISTATC(i) =3 it is necessary that $c_{Li} = c_{Ui}$, where $c_{Li} = \text{CLWR}(i)$ and $c_{Ui} = \text{CUPR}(i)$. When ISTATC(i) $\neq 3$, it is necessary that $c_{Li} < c_{Ui}$ with $|c_{Li} - c_{Ui}| > \delta_c$. Not referenced when MCON = 0.

MCON[†] [INPUT, INTEGER]
Number of constraints m , where $0 \leq \text{MCON}$.

VECLAM[#] [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON of Lagrange multipliers, λ , for constraints. The user must evaluate the Hessian of $L(x, \lambda) = f - \sum_{k=1}^m \lambda_k c_k$ (see IREVRS(3)). Not referenced when MCON = 0.

GMAT[‡] [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NONZG containing the nonzero elements of the Jacobian matrix G at XBAR (see USAGE REMARKS). Not referenced when MCON = 0.

IROWG[†] [INPUT, INTEGER, ARRAY]
An integer array of length NONZG containing the row indices of the Jacobian nonzeros, where $0 < \text{IROWG}(k) \leq \text{MCON}$ for $k = 1, \dots, \text{NONZG}$. Not referenced when MCON = 0.

JCOLG[†] [INPUT, INTEGER, ARRAY]
An integer array of length NONZG containing the column indices of the Jacobian nonzeros, where $0 < \text{JCOLG}(k) \leq \text{NDIM}$ for $k = 1, \dots, \text{NONZG}$. Not referenced when MCON = 0.

NONZG[†] [INPUT, INTEGER]
Number of Jacobian nonzeros, where $1 \leq \text{NONZG} \leq \text{NDIM} * \text{MCON}$.

ALGORITHM CONTROL DATA

IFERR[‡] [INPUT, INTEGER]
Function evaluation error flag.

IFERR= 1 When FBAR and/or CBAR cannot be evaluated.

IFERR= 0 Otherwise.

NFEVAL[‡] [INPUT, INTEGER]
Number of function evaluations. Normally NFEVAL is initialized prior to the first call when IRVCOM = -1, and incremented whenever IREVRS(1), IREVRS(2), or IREVRS(3) are nonzero. It may be appropriate to increment NFEVAL for each perturbation when finite difference gradients are used, or not at all when analytic gradients are used.

WORKING STORAGE

HOLD	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length NHOLD containing the hold array (working storage that must <u>not</u> be destroyed by the user).
NHOLD [†]	[INPUT/OUTPUT, INTEGER] Dimension of HOLD array. When IER = -127, or -131, NEEDED contains the storage needed.
IHOLD	[INPUT/OUTPUT, INTEGER, ARRAY] An integer array of length NIHOLD containing the integer hold array (working storage that must <u>not</u> be destroyed by the user).
NIHOLD [†]	[INPUT/OUTPUT, INTEGER] Dimension of IHOLD array. When IER = -128, or -132, NEEDED contains the storage needed.

OUTPUT ARGUMENTS SET BY ALGORITHM BEFORE TERMINATION (IRVCOM = 0)

NEEDED	[OUTPUT, INTEGER] When IER = -127, or -131, NEEDED contains the required length of HOLD. When IER = -128, or -132, NEEDED contains the required length for IHOLD.
IER	[OUTPUT, INTEGER] Success/error code. Results are not computed for IER < 0. For IER > 0, results are computed but may be suspect; user inspection is recommended. Note many of the error returns involve optional input quantities as described in INSNLP.
IER=0	Normal termination.
IER=-101	(MCON < 0); the number of constraints is negative.
IER=-102	(NDIM < 1); the number of variables is less than one.
IER=-103	Incorrect value for QPOPTN.
IER=-104	IHESHN > 3.
IER=-105	(NITMAX < max(1,NITMIN)); the maximum number of iterations is either less than 1 or less than the minimum number of iterations.
IER=-107	(IT1MAX < 1); the number of line search steps is less than 1.
IER=-108	(IOFLAG < 0) or (IOFLAG > 30); invalid input for the output control flag.
IER=-109	Invalid input for ALGOPT.
IER=-110	(OBJTOL ≤ 10.*HDMCON(5)); objective function tolerance is too small.

IER=-111	(PGDTOL $\leq$ [HDMCON(5)] ^{1/2}) or (PGDTOL $> 10^{-2}$); the projected gradient tolerance is too small or too large.
IER=-112	CONTOL $<$ [HDMCON(5)] ^{1/2} ; the constraint tolerance is too small.
IER=-113	(ISTATC(i) < 0) or (ISTATC(i) > 4); invalid input for constraint status.
IER=-114	(NONZG ≤ 0) or (NONZG $>$ NDIM*MCON); the number of Jacobian nonzeros is either less than one, or exceeds the size of a dense matrix.
IER=-115	(JCOLG(i) ≤ 0) or (JCOLG(i) $>$ NDIM); invalid input for Jacobian column index array.
IER=-116	(IROWG(i) ≤ 0) or (IROWG(i) $>$ MCON); invalid input for Jacobian row index.
IER=-117	(CUPR(i) $<$ CLWR(i)); constraint upper bound is less than lower bound.
IER=-118	Either (CUPR(i) = CLWR(i)) and (ISTATC(i) $\neq 3$) or (CUPR(i) $\neq$ CLWR(i)) and (ISTATC(i) = 3); constraint status array is not consistent with bounds.
IER=-119	(CUPR(i) $\neq$ CLWR(i)) and (CUPR(i)-CLWR(i) $<$ CONTOL); constraint bounds are not equal, but differ by less than the constraint tolerance.
IER=-120	(ISTATV(i) < 0) or (ISTATV(i) > 3); invalid input for variable status array.
IER=-121	(JSTRH(1) $\neq 1$) or (JSTRH(k) $\geq$ JSTRH(k+1)); invalid input for Hessian column start array.
IER=-122	(NZHDIM $<$ NDIM) or (NZHDIM $>$ NDNSH) or (NZHDIM $\neq$ NONZH) where NZHDIM = JSTRH(NDIM+1)-1, and NDNSH = NDIM*(NDIM+1)/2; the number of Hessian nonzeros is less than one, exceeds the size of a dense Hessian, or is not consistent with the input value NONZH.
IER=-123	(IROWH(i) ≤ 0) or (IROWH(i) $>$ NDIM); invalid value for Hessian row index.
IER=-124	(XUPR(i) $<$ XLWR(i)); the variable upper bound is less than the lower bound.
IER=-125	Either (XUPR(i) = XLWR(i)) and (ISTATV(i) $\neq 3$) or (XUPR(i) $\neq$ XLWR(i)) and (ISTATV(i) = 3); the variable status array is not consistent with the bound values.

IER=-126	(XUPR(i) $\neq$ XLWR(i)) and ($ XUPR(i)-XLWR(i) < CONTOL$); variable bounds are not equal but differ by less than the constraint tolerance.
IER=-127	Double precision hold array too small; insufficient storage detected in SBRNLP interface. The required storage is specified in NEEDED.
IER=-128	Integer hold array too small; insufficient storage detected in SBRNLP interface. The required storage is specified in NEEDED.
IER=-129	Function error at initial point or during gradient evaluation.
IER=-130	BIGCON < CONTOL.
IER=-131	NHOLD too small; insufficient double precision storage detected in algorithm. The required storage is specified in NEEDED.
IER=-132	NIHOLD too small; insufficient integer storage detected in algorithm. The required storage is specified in NEEDED.
IER=-133	Rank deficient Jacobian detected on successive iterations.
IER=-134	INSNLP input error; invalid character string displayed.
IER=-135	(IFERR < 0) or (IFERR > 1); invalid value for function error flag.
IER=-136	FEATOL < CONTOL; the initial feasibility tolerance is less than the constraint tolerance.
IER=-137	Conflict between user and multifrontal file number; check IPUMF1,...,IPUMF6 IPUDRF, IPUFZF, IPUSTF.
IER=-138	NEWTON $\neq 0, 1, 2$; invalid input for Newton method option flag.
IER=-145	(IROWH(JSTRH(i)) $\neq$ i) for some value of i; Hessian diagonal elements are incorrect.
IER=-146	PMULWR < HDMCON(5).
IER=-147	Unexpected error; check storage allocation.
IER=-148	$ IRVCOM > 1$.
IER=-149	PTHTOL < $[HDMCON(5)]^{\frac{1}{2}}$.
IER=-150	RHOLWR < HDMCON(5).

IER=-151	A constraint is inconsistent (either $c_i(x) < c_{Li}$ or $c_{Ui} < c_i(x)$) and cannot be changed because the corresponding row of the Jacobian is zero.
IER=-152	IMAXMU < 1.
IER=-153	I/O error, probably caused by insufficient disk space.
IER=-154	TOLKTC < 1.
IER=-155	TOLPVT < 0 or TOLPVT > .5.
IER=-156	IRELAX < 0 or IRELAX > 2.
IER=-157	MUCALC = 0 or MUCALC > 4.
IER=-158	MXQPIT < 1.

IER=+101	Weak solution found (relaxation required and/or multipliers near zero).
IER=+102	Number of equality constraints = number of free variables and ALGOPT $\neq$ 'F'.
IER=+103	Maximum number of consecutive function errors.
IER=+104	Maximum number of function evaluations.
IER=+105	Small step termination in optimization phase; suboptimal feasible point found.
IER=+106	Maximum number of iterations.
IER=+108	Feasible point not found.
IER=+109	Maximum number of interval halves in line search.
IER=+110	Algorithm terminated because the Hessian diagonal reached its maximum value; suboptimal feasible point found.
IER=+111	Projected gradient calculation failed; constraints may be degenerate.
IER=+112	Calculation of first order multiplier estimates failed; constraints may be degenerate.
IER=+113	Suboptimal feasible point found.

IER=+114	Barrier NLP failed with unexpected error.
IER=+115	CONTOL > OBJTOL; convergence tolerances may be inappropriate.
IER=+116	Uphill direction detected in line search.
IER=+117	Reduced objective function is linear.
IER=+118	Both $c_{Li} \leq -.01/\text{HDMCON}(5)$ and $c_{Ui} \geq .01/\text{HDMCON}(5)$; constraints ignored.
IER=+119	Terminate after diagnostic line search.
IER=+704	Sparse or dense parameter defaults may cause poor algorithm performance.

†	Variable set when IRVCOM=-1, and not changed by user thereafter
‡	Variable set when IRVCOM=+1, as required by IREVRS(*)
#	Variable optionally set when IRVCOM=-1, and not changed by user thereafter

USAGE REMARKS

Sparse Matrix Input

Hessian Matrix. The user must supply the Hessian matrix of the following Lagrangian function:

$$L(x, \lambda) = f - \sum_{k=1}^m \lambda_k c_k$$

that is,

$$L(\text{XBAR}, \text{VECLAM}) = \text{FBAR} - \text{VECLAM}(1) * \text{CBAR}(1) - \dots - \text{VECLAM}(\text{MCON}) * \text{CBAR}(\text{MCON}) .$$

Consider the following example in which $n = 3$:

$$H = \begin{pmatrix} 1.1 & 0 & 3.1 \\ 0 & 2.2 & 0 \\ 3.1 & 0 & 3.3 \end{pmatrix}$$

Since the Hessian is a symmetric matrix only the lower triangular portion must be specified. For this example $\text{NDIM} = 3$ and $\text{NONZH} = 4 = [\text{JSTRH}(\text{NDIM}+1)-1]$. The nonzero values are specified as $\text{HMAT}^* = (1.1, 3.1, 2.2, 3.3)$. The row indices are $\text{IROWH}^* = (1, 3, 2, 3)$, and the column start indices are $\text{JSTRH}^* = (1, 3, 4, 5)$. Notice that the element in row 1, column 3 is not specified. Also observe that the diagonal elements must always be included in the Hessian sparsity pattern *even if the elements are zero*. Additional information on sparse matrix formats is found in BCSLIB-EXT [1].

Jacobian Matrix. The user must specify the nonzero elements in the Jacobian matrix when required. Consider the following example in five variables $n = 5$ and three constraints $m = 3$:

$$G = \begin{pmatrix} 0 & 0 & 1.3 & 0 & 1.5 \\ 2.1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 3.3 & 0 & 0 \end{pmatrix}$$

For this example $\text{NDIM} = 5$, $\text{MCON} = 3$, and the number of nonzeros $\text{NONZG} = 4$. The nonzero array is $\text{GMAT}^* = (2.1, 1.3, 3.3, 1.5)$. The row indices are $\text{IROWG}^* = (2, 1, 3, 1)$, and the column indices are $\text{JCOLG}^* = (1, 3, 3, 5)$. Additional information on sparse matrix formats is found in BCSLIB-EXT [1].

Unconstrained Problems

When the problem is unconstrained the user must set $\text{MCON} = 0$. The following constraint related quantities will not be referenced: CBAR , CLWR , CUPR , ISTATC , VECLAM , GMAT , IROWG , JCOLG , and NONZG . Since these quantities must appear as arguments in the call to SBRNLP , dummy values may be utilized.

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine SBRNLP . All optional inputs are specified by calls to subroutine INSNLP . It should be emphasized that it is *not necessary* to call INSNLP if the default values are appropriate.

OPTIONAL PRINTOUT

The level of printed output from HSBNLP is controlled by the variables IOFLAG and IPUNLP which are set by calls to INSNLP . The standard level of output which corresponds to $\text{IOFLAG} = 10$ is the *default* for HSBNLP . A complete description of the iteration output options is found in Section 2.2.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2, x_3, x_4)$ to minimize the objective function

$$f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2 + 100(x_4 - x_3^2)^2 + (1 - x_3)^2$$

subject to the two constraints

$$x_1 x_2 \leq \frac{1}{2} \qquad -\frac{1}{2} \leq x_1 + x_3 \leq \frac{1}{2}$$

and the bounds

$$-2 \leq x_k \leq 2$$

for $k = 1, 2, 3, 4$ beginning at the initial point $x = (-1, 2, -1, 2)$. For this example the gradient vector is

$$g = \nabla_x f = \begin{bmatrix} -400x_1(x_2 - x_1^2) - 2(1 - x_1) \\ 200(x_2 - x_1^2) \\ -400x_3(x_4 - x_3^2) - 2(1 - x_3) \\ 200(x_4 - x_3^2) \end{bmatrix}$$

and the Jacobian matrix is

$$G = \begin{bmatrix} \nabla_x c_1^T \\ \nabla_x c_2^T \end{bmatrix} = \begin{bmatrix} x_2 & x_1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}.$$

It is also necessary to supply the Hessian matrix

$$\nabla_x^2 L = \nabla_x^2 f - \lambda_1 \nabla_x^2 c_1 - \lambda_2 \nabla_x^2 c_2$$

where

$$\nabla_x^2 f = \begin{bmatrix} 1200x_1^2 - 400x_2 + 2 & -400x_1 & 0 & 0 \\ -400x_1 & 200 & 0 & 0 \\ 0 & 0 & 1200x_3^2 - 400x_4 + 2 & -400x_3 \\ 0 & 0 & -400x_3 & 200 \end{bmatrix}$$

with

$$\nabla_x^2 c_1 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

and $\nabla_x^2 c_2 = 0$. Note: this example also illustrates the use of the “Terse Output” option by the call to INSNLP with “IOFLAG = 1”.

SAMPLE PROGRAM (sbrnlp.cd1)¹⁰

```

PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE SPARSE NONLINEAR
C      PROGRAMMING ALGORITHM SBRNLP
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      PARAMETER (NDIM=4, MCON=2)
C      PARAMETER (NONZH=6, NONZG=4)
C      PARAMETER (NHOLD=5000,NIHOLD=5000)
C
C      DIMENSION IREVRS(5),XBAR(NDIM),XLWR(NDIM),XUPR(NDIM),
C      $      ISTATV(NDIM),VECNU(NDIM),DELF(NDIM),HMAT(NONZH),
C      $      IROWH(NONZH),JSTRH(NDIM+1),CBAR(MCON),CLWR(MCON),
C      $      CUPR(MCON),ISTATC(MCON),VECLAM(MCON),
C      $      GMAT(NONZG),IROWG(NONZG),JCOLG(NONZG)
C      DIMENSION HOLD(NHOLD),IHOLD(NIHOLD)
C
C      INITIALIZE REVERSE COMMUNICATION CONTROL FLAG
C
C      ICC = -1
C
C -----

```

¹⁰sosexample.tar

```

C
C      INDEPENDENT VARIABLE DATA
C
C      ----INITIAL GUESS
C      XBAR(1) = -1.0D0
C      XBAR(2) =  2.0D0
C      XBAR(3) = -1.0D0
C      XBAR(4) =  2.0D0
C      ----BOUNDS
C      XLWR(1:NDIM) = -2.D0
C      XUPR(1:NDIM) =  2.D0
C      ----VARIABLE STATUS
C      ISTATV(1:NDIM) = 0
C
C      -----
C
C      OBJECTIVE FUNCTION DATA
C
C      ----HESSIAN SPARSITY PATTERN (ROW INDICES)
C      IROWH(1) = 1
C      IROWH(2) = 2
C      IROWH(3) = 2
C      IROWH(4) = 3
C      IROWH(5) = 4
C      IROWH(6) = 4
C      ----HESSIAN SPARSITY PATTERN (COLUMN START INDICES)
C      JSTRH(1) = 1
C      JSTRH(2) = 3
C      JSTRH(3) = 4
C      JSTRH(4) = 6
C      JSTRH(5) = 7
C
C      -----
C
C      CONSTRAINT DATA
C
C      ----BOUNDS
C      CLWR(1) = -1.D0/HDMCON(5)
C      CUPR(1) = .5D0
C      CLWR(2) = -.5D0
C      CUPR(2) = .5D0
C      ----CONSTRAINT STATUS
C      ISTATC(1:MCON) = 0
C      ----JACOBIAN SPARSITY PATTERN (ROW INDICES)
C      IROWG(1) = 1
C      IROWG(2) = 2
C      IROWG(3) = 1
C      IROWG(4) = 2

```

```
C      ----JACOBIAN SPARSITY PATTERN (COLUMN INDICES)
      JCOLG(1) = 1
      JCOLG(2) = 1
      JCOLG(3) = 2
      JCOLG(4) = 3

C
C -----
C
C      INITIALIZE FUNCTION EVALUATION COUNT
C
      NFEVAL = 0
C
      SET OPTIONAL INPUTS
C
      CALL INSNLP('BARRIER DEFAULT')
      CALL INSNLP('IOFLAG=1')
C
C -----
C
      1000 CONTINUE
C
      CALL SBRNLP( ICC ,IREVRS ,XBAR  ,XLWR  ,XUPR  ,ISTATV
$              ,VEGNU  ,NDIM  ,FBAR  ,DELF  ,HMAT  ,IROWH
$              ,JSTRH  ,NONZH  ,CBAR  ,CLWR  ,CUPR  ,ISTATC
$              ,MCON   ,VECLAM ,GMAT  ,IROWG  ,JCOLG  ,NONZG
$              ,IFERR  ,NFEVAL ,HOLD  ,NHOLD  ,IHOLD  ,NIHOLD
$              ,NEEDED ,IERNLP )
C
      REVERSE COMMUNICATION TERMINATION TEST
C
      IF(ICC.EQ.0) GO TO 10000
C
C -----
C
      SET FUNCTION EVALUATION ERROR FLAG
C
      IFERR = 0
C
      >>>>>FUNCTION EVALUATION REQUEST
C
      IF(IREVRS(1).EQ.1) THEN
C
          FBAR = 100.DO*(XBAR(2) - XBAR(1)**2)**2
$              + (1.DO - XBAR(1))**2
$              + 100.DO*(XBAR(4) - XBAR(3)**2)**2
$              + (1.DO - XBAR(3))**2
C
          CBAR(1) = XBAR(1)*XBAR(2)
```

```

      CBAR(2) = XBAR(1) + XBAR(3)
      NFEVAL = NFEVAL + 1
C
      ENDIF
C
C      >>>>>GRADIENT EVALUATION REQUEST
C
      IF(IREVRS(2).EQ.1) THEN
C
      DELF(1) = - 400.DO*XBAR(1)*(XBAR(2) - XBAR(1)**2)
$          - 2.DO*(1.DO - XBAR(1))
      DELF(2) = 200.DO*(XBAR(2) - XBAR(1)**2)
      DELF(3) = - 400.DO*XBAR(3)*(XBAR(4) - XBAR(3)**2)
$          - 2.DO*(1.DO - XBAR(3))
      DELF(4) = 200.DO*(XBAR(4) - XBAR(3)**2)
C
      GMAT(1) = XBAR(2)
      GMAT(2) = 1.DO
      GMAT(3) = XBAR(1)
      GMAT(4) = 1.DO
C
      ELSEIF(IREVRS(2).EQ.2) THEN
C
      DELF(1) = - 400.DO*XBAR(1)*(XBAR(2) - XBAR(1)**2)
$          - 2.DO*(1.DO - XBAR(1))
      DELF(2) = 200.DO*(XBAR(2) - XBAR(1)**2)
      DELF(3) = - 400.DO*XBAR(3)*(XBAR(4) - XBAR(3)**2)
$          - 2.DO*(1.DO - XBAR(3))
      DELF(4) = 200.DO*(XBAR(4) - XBAR(3)**2)
C
      GMAT(1) = XBAR(2)
      GMAT(2) = 1.DO
      GMAT(3) = XBAR(1)
      GMAT(4) = 1.DO
C
      ENDIF
C
C      >>>>>HESSIAN EVALUATION REQUEST
C
      IF(IREVRS(3).EQ.1) THEN
C
      DIAGONAL ELEMENTS
C
      HMAT(1) = 1200.DO*XBAR(1)**2
$          - 400.DO*XBAR(2) + 2.DO
      HMAT(3) = 200.DO
      HMAT(4) = 1200.DO*XBAR(3)**2
$          - 400.DO*XBAR(4) + 2.DO

```

```
      HMAT(6) = 200.D0
C
      ELSEIF(IREVRS(3).EQ.2) THEN
C
C          OFF-DIAGONAL ELEMENTS
C
      HMAT(2) = -VECLAM(1)*(1.D0) - 400.D0*XBAR(1)
      HMAT(5) = - 400.D0*XBAR(3)
C
      ENDIF
C
C      >>>>SYSTEM PRINT (OUTPUT) REQUEST
C
      IF(IREVRS(4).EQ.1) THEN
C
      WRITE(6,1001) 'OBJECTIVE  ',FBAR
      WRITE(6,1001) 'VARIABLES  ',(XBAR(II),II=1,NDIM)
      WRITE(6,1001) 'CONSTRAINTS',(CBAR(II),II=1,MCON)
C
      ENDIF
C
C -----
C
      2000 CONTINUE
C
      END OF REVERSE COMMUNICATION PROCESSING.  RETURN TO SBRNLP
C
      GO TO 1000
C
      10000 CONTINUE
C
      TERMINATE PROCESSING
C
      IF(IERNLP.EQ.0) THEN
C
      NORMAL TERMINATION---SOLUTION FOUND
C
      WRITE(6,1001) 'SOLUTION  '
C
      WRITE(6,1001) 'OBJECTIVE  ',FBAR
      WRITE(6,1001) 'VARIABLES  ',(XBAR(II),II=1,NDIM)
      WRITE(6,1001) 'CONSTRAINTS',(CBAR(II),II=1,MCON)
C
      ELSEIF(IERNLP.LT.0) THEN
C
      ABNORMAL TERMINATION---FATAL ERROR
C
      PRINT *, 'FATAL ERROR; IERNLP =', IERNLP
```



```

C
      ELSE
C
C      ABNORMAL TERMINATION---WARNING.  EVALUATION OF RESULTS RECOMMENDED!
C
      PRINT *, 'WARNING MESSAGE; IERNLP =', IERNLP
C
      ENDIF
C
1001 FORMAT(5X,A11,5X,4G17.8)
C
      STOP
      END

```

OUTPUT FROM SAMPLE PROGRAM

It	Lnfltr	KT	Cond	Step	Levnbrg	Optimization			Grdl	Obj	Log-Barrier Fn
						Cmplmt	Barrier	Violtn			
(1)	1	5.0E+04	2.5E-02	0.0E+00	0.0E+00	1.4E-01	1.5E+00	2.5E+02	2.0E+02	2.008936E+02	
(2)	2	8.2E+06	1.0E+00	1.0E+00	1.4E-01	1.4E-01	1.5E+00	2.4E+02	2.0E+02	2.020735E+02	
(3)	1	3.1E+05	6.6E-01	1.0E+00	2.3E-01	1.4E-02	6.9E-01	2.3E+02	9.9E+01	9.872009E+01	
(4)	1	2.3E+06	1.0E+00	1.0E+00	2.0E-01	1.4E-02	2.1E-01	2.2E+02	4.8E+01	4.832725E+01	
(5)	1	1.1E+06	1.0E+00	5.0E-01	4.4E-02	1.4E-02	9.1E-03	1.3E+02	9.1E+00	9.167177E+00	
(6)	1	7.7E+03	1.0E+00	1.5E-01	3.0E-02	1.4E-02	4.4E-03	3.9E+01	4.2E+00	4.190492E+00	
(7)	2	3.6E+03	1.0E+00	1.5E-02	2.3E-01	1.4E-02	3.5E-03	3.9E+00	3.7E+00	3.674143E+00	
(8)	2	5.1E+02	1.0E+00	1.5E-02	8.9E-03	1.4E-02	8.3E-02	1.5E+01	3.2E+00	3.152611E+00	
(9)	1	5.7E+02	1.0E+00	7.7E-03	4.1E-03	1.4E-02	2.2E-02	1.2E+01	2.6E+00	2.583827E+00	
(10)	2	7.1E+02	3.8E-01	3.8E-03	1.0E-02	1.4E-02	5.4E-02	1.2E+01	2.0E+00	1.983696E+00	
(11)	2	2.1E+03	1.0E+00	1.9E-03	1.4E-02	1.4E-02	3.3E-02	7.7E+00	1.7E+00	1.735478E+00	
(12)	1	1.3E+03	8.1E-01	8.3E-04	1.8E-02	1.4E-02	2.7E-04	3.3E+00	1.5E+00	1.437484E+00	
(13)	1	1.4E+04	1.0E+00	4.1E-04	1.3E-02	1.4E-02	2.5E-03	2.9E+00	1.3E+00	1.335245E+00	
(14)	1	7.7E+02	1.0E+00	1.9E-04	1.4E-03	1.2E-02	9.4E-04	1.5E+00	1.2E+00	1.211659E+00	
(15)	1	7.0E+02	1.0E+00	9.3E-05	1.1E-04	1.2E-02	1.3E-03	1.3E+00	1.2E+00	1.166334E+00	
(16)	1	4.6E+02	1.0E+00	2.7E-05	6.1E-06	1.2E-02	8.4E-04	3.8E-01	1.1E+00	1.144744E+00	
(17)	1	4.2E+02	1.0E+00	1.4E-05	7.1E-06	1.2E-02	7.1E-04	3.3E-01	1.1E+00	1.138631E+00	
(18)	1	3.3E+02	1.0E+00	1.1E-07	1.1E-02	1.2E-03	8.1E-05	2.7E-02	1.1E+00	1.137412E+00	
(19)	1	1.8E+03	1.0E+00	0.0E+00	1.2E-03	1.2E-04	3.4E-05	1.4E-02	1.1E+00	1.126633E+00	
(20)	1	1.8E+04	1.0E+00	0.0E+00	1.1E-04	1.2E-05	3.2E-07	1.1E-04	1.1E+00	1.125183E+00	
(21)	1	1.8E+05	1.0E+00	0.0E+00	1.2E-05	1.5E-09	1.5E-09	6.0E-07	1.1E+00	1.125012E+00	
(22)	1	1.8E+05	1.0E+00	0.0E+00	1.4E-10	1.5E-09	1.5E-11	1.2E-07	1.1E+00	1.125000E+00	

BARNLP ALGORITHM PERFORMANCE STATISTICS

```

Total CPU Time.....+4.383216E-02
Number of Function Calls.....22
Number of Gradient Calls.....23
Number of Hessian Calls.....20
Total Number of Function Evaluations.....22

Storage Needed in HOLD Array.....762
Storage Needed in IHOLD Array.....141

```

SOLUTION				
OBJECTIVE	1.1250000			
VARIABLES	0.25000000	0.62499999E-01	0.25000000	0.62500000E-01
CONSTRAINTS	0.15625000E-01	0.50000000		

SBRLSQ: Sparse Barrier Constrained Nonlinear Least Squares

PURPOSE

SBRLSQ computes the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the least squares objective function

$$f(x) = \frac{1}{2} r^T(x) r(x) = \frac{1}{2} \sum_{i=1}^{\ell} r_i^2$$

where $r(x)$ is an ℓ -vector of residuals, subject to the m nonlinear constraints

$$c_L \leq c(x) \leq c_U$$

and the simple bounds

$$x_L \leq x \leq x_U$$

Equality constraints are imposed by setting $c_{Li} = c_{Ui}$ and variables can be fixed by setting $x_{Li} = x_{Ui}$. SBRLSQ works under the assumption that the residual and constraint functions are twice continuously differentiable, and if this is not true algorithm performance is unpredictable. SBRLSQ uses a reverse communication format and upon request the user must supply the values of the functions, and their first and second derivatives. The matrix of first derivatives (the Jacobian) and the matrix of second derivatives (residual Hessian) are represented in a sparse format consistent with that used in BCSLIB-EXT [1]. An optional input to the software permits efficient solution of the problem when the residual and constraint functions are linear, i.e. the *linear least squares problem*.

RELATED SUBPROGRAMS

SBRNLP Sparse Barrier Nonlinear Programming

INSNLP Sparse Nonlinear Programming Input Procedure

METHOD

An *interior point or barrier* method is used to solve the *nonlinear programming* (NLP) problem. It is necessary to compute the residual Hessian matrix

$$V = \sum_{i=1}^{\ell} r_i \nabla^2 r_i - \sum_{i=1}^m \lambda_i \nabla^2 c_i.$$

If the user cannot provide this information analytically, the sparse finite difference techniques described in Chapter 3 should be utilized. Section 4.4 illustrates use of the sparse finite difference technique in conjunction with the least squares algorithm. Notice that the Hessian of the Lagrangian H_L , which is required input for the sparse nonlinear program SBRNLP is related to the residual Hessian by

$$H_L = V + R^T R$$

where R is the $\ell \times n$ residual Jacobian matrix. A sparse tableau form for the KKT linear system is utilized to avoid formation of the normal matrix $R^T R$. This linear system is solved efficiently using the multifrontal method.

WARNING

The residual and constraint functions supplied by the user are assumed to be continuous and have continuous first and second derivatives. Nondifferentiable functions can cause unpredictable performance in the optimization algorithm. It should be emphasized that discontinuities in the Hessian matrices can significantly degrade the speed of convergence without producing any other obvious difficulties. The following common sources of error *should be avoided* when evaluating the residual and constraint functions:

- (1) nonsmooth interpolation of data;
- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;
- (3) discontinuous behavior caused by branching for IF tests;
- (4) nondifferentiable functions such as ABS, MAX, and MIN.

The user must provide a procedure to evaluate the residual and constraint functions in addition to first and second derivatives of these quantities. If it is not possible to evaluate the functions (for example, $c(x) = 1/x$ at $x = 0$) then set IFERR = 1 and exit the function evaluation process. Otherwise, set IFERR = 0 when the functions are evaluated. The function error flag should be used to avoid computational difficulties, but never in lieu of an explicit constraint.

The SBRLSQ software uses internal common blocks to communicate information. To avoid conflict with these internal commons the user should not have commons with the following names: NPSPRR, NPSPRI, NPSPRC, STATIS, KONSTN, NPSALG, NPSDFL, NLPERR, PERCOM, ITEREF, and CLKCOM.

USAGE

```

INTEGER      IRVCOM, IFERR, NFEVAL, IER, NEEDED, NDIM, MCON
INTEGER      NRES,  NONZH, NONZG, NHOLD, NIHOLD, NONZR
INTEGER      IREVRS(5), ISTATV(NDIM), IROWR(NONZR), JCOLR(NONZR)
INTEGER      IROWH(NONZH), JSTRH(NDIM+1)
INTEGER      ISTATC(MCON), IROWG(NONZG), JCOLG(NONZG), IHOLD(NIHOLD)

DOUBLE PRECISION  XBAR(NDIM), XLWR(NDIM), XUPR(NDIM), VECNU(NDIM)
DOUBLE PRECISION  RESVEC(NRES), RMAT(NONZR)
DOUBLE PRECISION  HMAT(NONZH), CBAR(MCON), CLWR(MCON)
DOUBLE PRECISION  CUPR(MCON), VECLAM(MCON), GMAT(NONZG), HOLD(NHOLD)

```

```

      CALL  SBRLSQ( IRVCOM ,IREVRS ,XBAR  ,XLWR  ,XUPR
$          ,ISTATV ,VECNU  ,NDIM  ,RESVEC ,NRES  ,RMAT
$          ,IROWR  ,JCOLR  ,NONZR  ,HMAT  ,IROWH ,JSTRH
$          ,NONZH  ,CBAR   ,CLWR   ,CUPR   ,ISTATC ,MCON
$          ,VECLAM ,GMAT   ,IROWG  ,JCOLG  ,NONZG  ,IFERR
$          ,NFEVAL ,HOLD   ,NHOLD  ,IHOLD  ,NIHOLD ,NEEDED
$          ,IER    )

```

ARGUMENTS

ARGUMENTS TO CONTROL REVERSE COMMUNICATION

IRVCOM	[INPUT/OUTPUT, INTEGER]
	Continuation control flag. This argument must be initialized. Thereafter it is set by the algorithm and must not be changed by the calling program.
$\text{IRVCOM} = -1$	Initialization pass.
$\text{IRVCOM} = 0$	Termination (normal or abnormal).
$\text{IRVCOM} = +1$	Perform an evaluation and then reenter SBRLSQ. Operations to be performed are defined by the output variables IREVRS(1), IREVRS(2), IREVRS(3), and IREVRS(4) below. IREVRS(5) contains information which may be valuable to the interface but does not require any action on the part of the user.
IREVRS	[OUTPUT, INTEGER, ARRAY]
	An integer array of length 5, containing reverse communication control information.
IREVRS(1)	Function evaluation request
$= 0$	Function evaluation not requested; <i>Warning: do not change the current function and constraint values.</i>
$= 1$	Function evaluation requested; evaluate residual functions and all constraints at current point XBAR.
IREVRS(2)	Gradient evaluation request
$= 0$	Gradient evaluation not requested; <i>Warning: do not change the current residual and constraint and Jacobians.</i>
$= 1$	Approximate (e.g. forward difference) gradient requested; evaluate residual Jacobian matrix R and Jacobian matrix G at current point XBAR. (If available, accurate or analytic gradients may be supplied)
$= 2$	Accurate (e.g. central difference or analytic) gradient requested; evaluate residual Jacobian matrix R and Jacobian matrix G at current point XBAR.

IREVRS(3)	Hessian evaluation request
	= 0 Hessian evaluation not requested; <i>Warning: do not change the current Hessian.</i>
	= 1 Hessian diagonal requested; get ready for full Hessian evaluation with (IREVRS(3)=2) at current point (XBAR,VECLAM) and optionally evaluate diagonal elements of residual Hessian matrix V .
	= 2 Full Hessian evaluation requested; evaluate residual Hessian matrix $V = \sum_{i=1}^{\ell} r_i \nabla^2 r_i - \sum_{i=1}^m \lambda_i \nabla^2 c_i$ at current point (XBAR,VECLAM). A full evaluation request is always preceded by a call with IREVRS(3)=1 at the same point XBAR, and consequently it may not be necessary to recompute the diagonal elements.
IREVRS(4)	System print (output) request
	= 0 System print not requested.
	= 1 System print requested; perform any output operations (print, write files, etc.) desired to display problem specific information at the current point XBAR. The frequency of print is defined by the output control flag IOFLAG (see Optional Printout).
IREVRS(5)	Algorithm information flag
	= 0 Call was made from the interface portion of the algorithm.
	= 1 Call was made from the optimization portion of the algorithm.
	= 2 Call was made from the optimization portion of the algorithm requesting Hessian reset.
	= 3 Call was made from the optimization portion of the algorithm requesting optimal perturbation size adjustment.
	= 4 Call was made from the feasibility portion of the algorithm.

INDEPENDENT VARIABLE DATA

XBAR[†] [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
 A real array of length NDIM of current variable values x .

IRVCOM=-1 Initial guess must be input.

IRVCOM=0 Final point is output.

IRVCOM=1 Intermediate iteration points are output.

XLWR[†] [INPUT, DOUBLE PRECISION, ARRAY]
 A real array of length NDIM containing the variable lower bounds x_L . For variables that have no lower bound set $x_{Li} \leq -.01/\text{HDMCON}(5)$. See Appendix B for HDMCON.

XUPR[†] [INPUT, DOUBLE PRECISION, ARRAY]
 A real array of length NDIM containing the variable upper bounds x_U . For variables that have no upper bound set $x_{Ui} \geq .01/\text{HDMCON}(5)$.

ISTATV[†] [INPUT/OUTPUT, INTEGER, ARRAY]
 An integer array of length NDIM indicating variable status. On input ISTATV defines an estimate of the active variable bounds, and on output ISTATV defines the bounds active at the solution.

ISTATV(i)= 0 Variable is strictly feasible, i.e. $x_{Li} < x_i < x_{Ui}$.

ISTATV(i)= 1 Variable lower bound is active, i.e. $x_{Li} = x_i$.

ISTATV(i)= 2 Variable upper bound is active, i.e. $x_i = x_{Ui}$.

ISTATV(i)= 3 Fixed permanently.

ISTATV(i)= 10 Output only: Violated variable lower bound, i.e. $x_{Li} - \delta_c > x_i$.

ISTATV(i)= 20 Output only: Violated variable upper bound, i.e. $x_i > x_{Ui} + \delta_c$.

ISTATV(i)= 30 Output only: Violated fixed variable. Either $x_i < x_{Li} - \delta_c$ or $x_i > x_{Ui} + \delta_c$.

When ISTATV(i) = 3 it is necessary that $x_{Li} = x_{Ui}$, where $x_{Li} = \text{XLWR}(i)$ and $x_{Ui} = \text{XUPR}(i)$. When ISTATV(i) $\neq$ 3, it is necessary that $x_{Li} < x_{Ui}$ with $|x_{Li} - x_{Ui}| > \delta_c$.

VECNU[#] [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
 An array of length NDIM of bound multipliers ν .

NDIM[†] [INPUT, INTEGER]
 Number of variables n , where $1 \leq \text{NDIM}$.

OBJECTIVE FUNCTION DATA

RESVEC[‡] [INPUT, DOUBLE PRECISION, ARRAY]
 A real array of length NRES containing the residuals r at XBAR.

NRES[†] [INPUT, INTEGER]
 Number of residuals ℓ , where $0 \leq \text{NRES}$.

- RMAT[‡]** [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NONZR containing the nonzero elements of the residual Jacobian matrix R at XBAR (see USAGE REMARKS).
- IROWR[†]** [INPUT, INTEGER, ARRAY]
An integer array of length NONZR containing the row indices of the residual Jacobian nonzeros, where $0 < \text{IROWR}(k) \leq \text{NRES}$ for $k = 1, \dots, \text{NONZR}$.
- JCOLR[†]** [INPUT, INTEGER, ARRAY]
An integer array of length NONZR containing the column indices of the residual Jacobian nonzeros, where $0 < \text{JCOLR}(k) \leq \text{NDIM}$ for $k = 1, \dots, \text{NONZR}$.
- NONZR[†]** [INPUT, INTEGER]
Number of residual Jacobian nonzeros, where $0 \leq \text{NONZR} \leq \text{NRES} * \text{NDIM}$.
- HMAT[‡]** [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NONZH containing the nonzero elements of the residual Hessian matrix $V = \sum_{i=1}^{\ell} r_i \nabla^2 r_i - \sum_{i=1}^m \lambda_i \nabla^2 c_i$. Since the Hessian is symmetric, only the lower triangular part is input (see USAGE REMARKS). It is necessary to provide storage for diagonal elements even if $V = 0$.
- IROWH[†]** [INPUT, INTEGER, ARRAY]
An integer array of length NONZH containing the row indices of the residual Hessian nonzeros, where $0 < \text{IROWH}(k) \leq \text{NDIM}$ for $k = 1, \dots, \text{NONZH}$. The diagonal elements must be specified, i.e. $\text{IROWH}(\text{JSTRH}(k)) = k$ for $k = 1, \dots, \text{NDIM}$.
- JSTRH[†]** [INPUT, INTEGER, ARRAY]
An integer array of length NDIM+1 containing the column start indices of nonzeros, where $\text{JSTRH}(1) = 1$, $\text{JSTRH}(\text{NDIM}+1) = \text{NONZH} + 1$, and $\text{JSTRH}(k) < \text{JSTRH}(k+1)$ for $k = 1, \dots, \text{NDIM}$.
- NONZH[†]** [INPUT, INTEGER]
Number of nonzero residual Hessian elements, where $n \leq \text{NONZH} \leq n(n+1)/2$.

CONSTRAINT DATA

- CBAR[‡]** [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraints $c(x)$ evaluated at XBAR. Not referenced when MCON = 0.
- CLWR[†]** [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraint lower bounds c_{Li} . For constraints that have no lower bound set $c_{Li} \leq -.01/\text{HDMCON}(5)$. Not referenced when MCON = 0.
- CUPR[†]** [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraint upper bounds c_{Ui} . For constraints that have no upper bound set $c_{Ui} \geq .01/\text{HDMCON}(5)$. Not referenced when MCON = 0.
- ISTATC[†]** [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length MCON indicating the constraint status. On input

ISTATC defines an estimate of the active constraints, and on output ISTATC defines the constraints active at the solution.

ISTATC(i)= 0	Free (inactive) inequality, i.e. $c_{Li} < c_i(x) < c_{Ui}$.
ISTATC(i)= 1	Constraint lower bound is active, i.e. $c_{Li} = c_i(x)$.
ISTATC(i)= 2	Constraint upper bound is active, i.e. $c_i(x) = c_{Ui}$.
ISTATC(i)= 3	Equality.
ISTATC(i)= 4	Ignored constraint.
ISTATC(i)= 10	Output only: Violated constraint lower bound, i.e. $c_{Li} - \delta_c > c_i$.
ISTATC(i)= 20	Output only: Violated constraint upper bound, i.e. $c_i > c_{Ui} + \delta_c$.
ISTATC(i)= 30	Output only: Violated equality constraint. Either $c_i < c_{Li} - \delta_c$ or $c_i > c_{Ui} + \delta_c$.

When ISTATC(i) = 3 it is necessary that $c_{Li} = c_{Ui}$, where $c_{Li} = \text{CLWR}(i)$ and $c_{Ui} = \text{CUPR}(i)$. When ISTATC(i) $\neq 3$, it is necessary that $c_{Li} < c_{Ui}$ with $|c_{Li} - c_{Ui}| > \delta_c$. Not referenced when MCON = 0.

MCON [†]	[INPUT, INTEGER] Number of constraints m , where $0 \leq \text{MCON}$.
VECLAM [#]	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length MCON of Lagrange multipliers, λ , for constraints. Not referenced when MCON = 0.
GMAT [‡]	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NONZG containing the nonzero elements of the Jacobian matrix G at XBAR (see USAGE REMARKS). Not referenced when MCON = 0.
IROWG [†]	[INPUT, INTEGER, ARRAY] An integer array of length NONZG containing the row indices of the Jacobian nonzeros, where $0 < \text{IROWG}(k) \leq \text{MCON}$ for $k = 1, \dots, \text{NONZG}$. Not referenced when MCON = 0.
JCOLG [†]	[INPUT, INTEGER, ARRAY] An integer array of length NONZG containing the column indices of the Jacobian nonzeros, where $0 < \text{JCOLG}(k) \leq \text{NDIM}$ for $k = 1, \dots, \text{NONZG}$. Not referenced when MCON = 0.
NONZG [†]	[INPUT, INTEGER] Number of Jacobian nonzeros, where $1 \leq \text{NONZG} \leq \text{NDIM} * \text{MCON}$.

ALGORITHM CONTROL DATA

IFERR[‡] [INPUT, INTEGER]
 Function evaluation error flag.
 IFERR= 1 When RESVEC and/or CBAR cannot be evaluated.
 IFERR= 0 Otherwise.

NFEVAL[‡] [INPUT, INTEGER]
 Number of function evaluations. Normally NFEVAL is initialized prior to the first call when IRVCOM = -1, and incremented whenever IREVRS(1), IREVRS(2), or IREVRS(3) are nonzero. It may be appropriate to increment NFEVAL for each perturbation when finite difference gradients are used, or not at all when analytic gradients are used.

WORKING STORAGE

HOLD [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
 A real array of length NHOLD containing the hold array (working storage that must not be destroyed by the user).

NHOLD[‡] [INPUT/OUTPUT, INTEGER]
 Dimension of HOLD array. When IER = -127, or -131, NEEDED contains the storage needed.

IHOLD [INPUT/OUTPUT, INTEGER, ARRAY]
 An integer array of length NIHOLD containing the integer hold array (working storage that must not be destroyed by the user).

NIHOLD[‡] [INPUT/OUTPUT, INTEGER]
 Dimension of IHOLD array. When IER = -128, or -132, NEEDED contains the storage needed.

OUTPUT ARGUMENTS SET BY ALGORITHM BEFORE TERMINATION (IRVCOM = 0)

NEEDED [OUTPUT, INTEGER]
 When IER = -127, or -131, NEEDED contains the required length of HOLD.
 When IER = -128, or -132, NEEDED contains the required length for IHOLD.

IER [OUTPUT, INTEGER]
 Success/error code. Results are not computed for IER < 0. For IER > 0, results are computed but may be suspect; user inspection is recommended. Note many of the error returns involve optional input quantities as described in INSNLP.

IER=0 Normal termination.

IER=-101 (MCON < 0); the number of constraints is negative.

IER=-102 (NDIM < 1); the number of variables is less than one.

IER=-103	Incorrect value for QPOPTN.
IER=-104	$ IHESHN > 3$.
IER=-105	(NITMAX < max(1,NITMIN)); the maximum number of iterations is either less than 1 or less than the minimum number of iterations.
IER=-107	(IT1MAX < 1); the number of line search steps is less than 1.
IER=-108	(IOFLAG < 0) or (IOFLAG > 30); invalid input for the output control flag.
IER=-109	Invalid input for ALGOPT.
IER=-110	(OBJTOL $\leq$ 10.*HDMCON(5)); objective function tolerance is too small.
IER=-111	(PGDTOL $\leq$ [HDMCON(5)] ^{1/2}) or (PGDTOL > 10 ⁻²); the projected gradient tolerance is too small or too large.
IER=-112	CONTOL < [HDMCON(5)] ^{1/2} ; the constraint tolerance is too small.
IER=-113	(ISTATC(i) < 0) or (ISTATC(i) > 4); invalid input for constraint status.
IER=-114	(NONZG $\leq$ 0) or (NONZG > NDIM*MCON); the number of Jacobian nonzeros is either less than one, or exceeds the size of a dense matrix.
IER=-115	(JCOLG(i) $\leq$ 0) or (JCOLG(i) > NDIM); invalid input for Jacobian column index array.
IER=-116	(IROWG(i) $\leq$ 0) or (IROWG(i) > MCON); invalid input for Jacobian row index.
IER=-117	(CUPR(i) < CLWR(i)); constraint upper bound is less than lower bound.
IER=-118	Either (CUPR(i) = CLWR(i)) and (ISTATC(i) $\neq$ 3) or (CUPR(i) $\neq$ CLWR(i)) and (ISTATC(i) = 3); constraint status array is not consistent with bounds.
IER=-119	(CUPR(i) $\neq$ CLWR(i)) and (CUPR(i)-CLWR(i) < CONTOL); constraint bounds are not equal, but differ by less than the constraint tolerance.
IER=-120	(ISTATV(i) < 0) or (ISTATV(i) > 3); invalid input for variable status array.
IER=-121	(JSTRH(1) $\neq$ 1) or (JSTRH(k) $\geq$ JSTRH(k+1)); invalid input for Hessian column start array.

IER=-122	(NZHDIM < NDIM) or (NZHDIM > NDNSH) or (NZHDIM $\neq$ NONZH) where NZHDIM = JSTRH(NDIM+1)-1, and NDNSH = NDIM*(NDIM+1)/2; the number of Hessian nonzeros is less than one, exceeds the size of a dense Hessian, or is not consistent with the input value NONZH.
IER=-123	(IROWH(i) $\leq$ 0) or (IROWH(i) > NDIM); invalid value for Hessian row index.
IER=-124	(XUPR(i) < XLWR(i)); the variable upper bound is less than the lower bound.
IER=-125	Either (XUPR(i) = XLWR(i)) and (ISTATV(i) $\neq$ 3) or (XUPR(i) $\neq$ XLWR(i)) and (ISTATV(i) = 3); the variable status array is not consistent with the bound values.
IER=-126	(XUPR(i) $\neq$ XLWR(i)) and (XUPR(i)-XLWR(i) < CONTOL); variable bounds are not equal but differ by less than the constraint tolerance.
IER=-127	Real hold array too small; insufficient storage detected in SBRNLP interface. The required storage is specified in NEEDED.
IER=-128	Integer hold array too small; insufficient storage detected in SBRNLP interface. The required storage is specified in NEEDED.
IER=-129	Function error at initial point or during gradient evaluation.
IER=-130	BIGCON < CONTOL.
IER=-131	NHOLD too small; insufficient real storage detected in algorithm. The required storage is specified in NEEDED.
IER=-132	NIHOLD too small; insufficient integer storage detected in algorithm. The required storage is specified in NEEDED.
IER=-133	Rank deficient Jacobian detected on successive iterations.
IER=-134	INSNLP input error; invalid character string displayed.
IER=-135	(IFERR < 0) or (IFERR > 1); invalid value for function error flag.
IER=-136	FEATOL < CONTOL; the initial feasibility tolerance is less than the constraint tolerance.
IER=-137	Conflict between user and multifrontal file number; check IPUMF1,...,IPUMF6, IPUDRF, IPUFZF, IPUSTF.
IER=-138	NEWTON $\neq$ 0, 1, 2; invalid input for Newton method option flag.

IER=-139	(NRES < 0); the number of residuals is negative.
IER=-141	(NONZR $\leq$ 0) or (NONZR > NDIM*NRES); the number of residual Jacobian nonzeros is either less than one, or exceeds the size of a dense matrix.
IER=-142	(JCOLR(i) $\leq$ 0) or (JCOLR(i) > NDIM); invalid input for residual Jacobian column index array.
IER=-143	(IROWR(i) $\leq$ 0) or (IROWR(i) > NRES); invalid input for residual Jacobian row index.
IER=-144	A row of the residual Jacobian is identically (structurally) zero. If a residual does not depend on any variable, the sum of squares can be reformulated omitting the constant contribution.
IER=-145	(IROWH(JSTRH(i)) $\neq$ i) for some value of i; Hessian diagonal elements are incorrect.
IER=-146	PMULWR < HDMCON(5).
IER=-147	Unexpected error; check storage allocation and verify that the HOLD and IHOLD arrays are unchanged during reverse communication calls.
IER=-148	IRVCOM > 1.
IER=-149	PTHTOL < [HDMCON(5)] ^{1/2} .
IER=-150	RHOLWR < HDMCON(5).
IER=-151	A constraint is inconsistent (either $c_i(x) < c_{Li}$ or $c_{Ui} < c_i(x)$) and cannot be changed because the corresponding row of the Jacobian is zero.
IER=-152	IMAXMU < 1.
IER=-153	I/O error, probably caused by insufficient disk space.
IER=-154	TOLKTC < 1.
IER=-155	TOLPVT < 0 or TOLPVT > .5.
IER=-156	IRELAX < 0 or IRELAX > 2.
IER=-157	MUCALC = 0 or MUCALC > 4.
IER=-158	MXQPIT < 1.

IER=+101	Weak solution found (multipliers near zero).
IER=+102	Number of equality constraints = NDIM and ALGOPT $\neq$ 'F'.
IER=+103	Maximum number of consecutive function errors.
IER=+104	Maximum number of function evaluations.
IER=+105	Small step termination in optimization phase; suboptimal feasible point found.
IER=+106	Maximum number of iterations in optimization phase.
IER=+108	Feasible point not found.
IER=+109	Maximum number of interval halves in line search.
IER=+110	Algorithm terminated because either the Hessian diagonal reached its maximum value or the merit function slope condition was violated; suboptimal feasible point found.
IER=+111	Projected gradient calculation failed; constraints may be degenerate.
IER=+112	Calculation of first order multiplier estimates failed; constraints may be degenerate.
IER=+113	Suboptimal feasible point found.
IER=+114	Barrier NLP failed with unexpected error.
IER=+115	CONTOL > OBJTOL; convergence tolerances may be inappropriate.
IER=+116	Uphill direction detected in line search.
IER=+117	Reduced objective function is linear.
IER=+118	Both $c_{Li} \leq -.01/\text{HDMCON}(5)$ and $c_{Ui} \geq .01/\text{HDMCON}(5)$; constraints ignored.
IER=+119	Terminate after diagnostic line search.
IER=+704	Sparse or dense parameter defaults may cause poor algorithm performance.

†	Variable set when IRVCOM=-1, and not changed by user thereafter
‡	Variable set when IRVCOM=+1, as required by IREVRS(*)
#	Variable optionally set when IRVCOM=-1, and not changed by user thereafter

USAGE REMARKS

A *linear least squares problem* is a special form of the general problem. In particular, it is necessary to compute the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the least squares objective function

$$f(x) = \frac{1}{2}(Rx - d)^T(Rx - d) = \frac{1}{2}\|Rx - d\|^2$$

where d is an ℓ -vector of data, subject to the m linear constraints

$$c_L \leq Gx \leq c_U$$

and the simple bounds

$$x_L \leq x \leq x_U.$$

The $\ell \times n$ residual Jacobian matrix R and the $m \times n$ Jacobian matrix G are constant. The residual Hessian matrix $V = 0$ for this case. Efficient solution of a linear least squares problem should exploit the fact that;

- (1) the Jacobian matrices R and G need only be computed once and,
- (2) the residual Hessian matrix V does not need to be computed at all.

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine SBRLSQ. All optional inputs are specified by calls to subroutine INSNLP. It should be emphasized that it is *not necessary* to call INSNLP if the default values are appropriate.

OPTIONAL PRINTOUT

The level of printed output from SBRLSQ is controlled by the variables IOFLAG and IPUNLP which are set by calls to INSNLP. The standard level of output which corresponds to IOFLAG = 10 is the *default* for SBRLSQ. A complete description of the iteration output options is found in Section 2.3.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2, x_3, x_4)$ to minimize the least squares objective function

$$f(x) = \frac{1}{2}r^T(x)r(x)$$

where the residual vector is

$$r = \begin{bmatrix} 10(x_2 - x_1^2) \\ (1 - x_1) \\ 10(x_4 - x_3^2) \\ (1 - x_3) \end{bmatrix}$$

subject to the two constraints

$$x_1 x_2 \leq \frac{1}{2} \qquad -\frac{1}{2} \leq x_1 + x_3 \leq \frac{1}{2}$$

and the bounds

$$-2 \leq x_k \leq 2$$

for $k = 1, 2, 3, 4$ beginning at the initial point $x = (-1, 2, -1, 2)$. For this example the residual Jacobian matrix is

$$R = \begin{bmatrix} \nabla_x r_1^T \\ \nabla_x r_2^T \\ \nabla_x r_3^T \\ \nabla_x r_4^T \end{bmatrix} = \begin{bmatrix} -20x_1 & 10 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & -20x_3 & 10 \\ 0 & 0 & -1 & 0 \end{bmatrix}.$$

and the constraint Jacobian matrix is

$$G = \begin{bmatrix} \nabla_x c_1^T \\ \nabla_x c_2^T \end{bmatrix} = \begin{bmatrix} x_2 & x_1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}.$$

It is also necessary to supply the residual Hessian matrix

$$V = r_1 \nabla_x^2 r_1 + r_2 \nabla_x^2 r_2 + r_3 \nabla_x^2 r_3 + r_4 \nabla_x^2 r_4 - \lambda_1 \nabla_x^2 c_1 - \lambda_2 \nabla_x^2 c_2$$

where

$$\nabla_x^2 r_1 = \begin{bmatrix} -20 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

and

$$\nabla_x^2 r_3 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & -20 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

with

$$\nabla_x^2 c_1 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

and $\nabla_x^2 r_2 = \nabla_x^2 r_4 = \nabla_x^2 c_2 = 0$. Collecting results yields the expression

$$V = \begin{bmatrix} -20r_1 & -\lambda_1 & 0 & 0 \\ -\lambda_1 & 0 & 0 & 0 \\ 0 & 0 & -20r_3 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Note: this example also illustrates the use of the “Terse Output” option by the call to INSNLP with “IOFLAG = 1”.

SAMPLE PROGRAM (sbrlsq.cd1)¹¹

¹¹sosexample.tar


```

PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE SPARSE NONLINEAR
C      PROGRAMMING ALGORITHM SBRLSQ
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      PARAMETER (NDIM=4, MCON=2)
C      PARAMETER (NONZR=6, NRES=4)
C      PARAMETER (NONZH=6, NONZG=4)
C      PARAMETER (NHOLD=5000,NIHOLD=5000)
C
C      DIMENSION IREVR(5),XBAR(NDIM),XLWR(NDIM),XUPR(NDIM),
$      ISTATV(NDIM),VECNU(NDIM),RESVEC(NRES),RMAT(NONZR),
$      IROWR(NONZR),JCOLR(NONZR),HMAT(NONZH),
$      IROWH(NONZH),JSTRH(NDIM+1),CBAR(MCON),CLWR(MCON),
$      CUPR(MCON),ISTATC(MCON),VECLAM(MCON),
$      GMAT(NONZG),IROWG(NONZG),JCOLG(NONZG)
C      DIMENSION HOLD(NHOLD),IHOLD(NIHOLD)
C
C      INITIALIZE REVERSE COMMUNICATION CONTROL FLAG
C
C      IRVCOM = -1
C
C      -----
C
C      INDEPENDENT VARIABLE DATA
C
C      ----INITIAL GUESS
C      XBAR(1) = -1.0D0
C      XBAR(2) =  2.0D0
C      XBAR(3) = -1.0D0
C      XBAR(4) =  2.0D0
C
C      ----BOUNDS
C      XLWR(1:NDIM) = -2.D0
C      XUPR(1:NDIM) =  2.D0
C
C      ----VARIABLE STATUS
C      ISTATV(1:NDIM) = 0
C
C      -----
C
C      OBJECTIVE FUNCTION DATA
C
C      ----RESIDUAL JACOBIAN SPARSITY PATTERN (ROW INDICES)
C      IROWR(1) = 1
C      IROWR(2) = 2
C      IROWR(3) = 1
C      IROWR(4) = 3

```

```

        IROWR(5) = 4
        IROWR(6) = 3
C      ----RESIDUAL JACOBIAN SPARSITY PATTERN (COLUMN INDICES)
        JCOLR(1) = 1
        JCOLR(2) = 1
        JCOLR(3) = 2
        JCOLR(4) = 3
        JCOLR(5) = 3
        JCOLR(6) = 4
C      ----HESSIAN SPARSITY PATTERN (ROW INDICES)
        IROWH(1) = 1
        IROWH(2) = 2
        IROWH(3) = 2
        IROWH(4) = 3
        IROWH(5) = 4
        IROWH(6) = 4
C      ----HESSIAN SPARSITY PATTERN (COLUMN START INDICES)
        JSTRH(1) = 1
        JSTRH(2) = 3
        JSTRH(3) = 4
        JSTRH(4) = 6
        JSTRH(5) = 7

C
C -----
C
C      CONSTRAINT DATA
C
C      ----BOUNDS
        CLWR(1) = -1.DO/HDMCON(5)
        CUPR(1) = .5D0
        CLWR(2) = -.5D0
        CUPR(2) = .5D0
C      ----CONSTRAINT STATUS
        ISTATC(1:MCON) = 0
C      ----JACOBIAN SPARSITY PATTERN (ROW INDICES)
        IROWG(1) = 1
        IROWG(2) = 2
        IROWG(3) = 1
        IROWG(4) = 2
C      ----JACOBIAN SPARSITY PATTERN (COLUMN INDICES)
        JCOLG(1) = 1
        JCOLG(2) = 1
        JCOLG(3) = 2
        JCOLG(4) = 3

C
C -----
C
C      INITIALIZE FUNCTION EVALUATION COUNT

```

```

C
C      NFEVAL = 0
C
C      SET OPTIONAL INPUTS
C
C      CALL INSNLP('BARRIER DEFAULT')
C      CALL INSNLP('IOFLAG=1')
C
C -----
C
C      1000 CONTINUE
C
C      CALL SBRLSQ( IRVCOM ,IREVRS ,XBAR  ,XLWR  ,XUPR  ,ISTATV
$                ,VECNU  ,NDIM  ,RESVEC ,NRES  ,RMAT  ,IROWR
$                ,JCOLR  ,NONZR  ,HMAT  ,IROWH ,JSTRH ,NONZH
$                ,CBAR   ,CLWR   ,CUPR   ,ISTATC ,MCON   ,VECLAM
$                ,GMAT   ,IROWG  ,JCOLG  ,NONZG  ,IFERR  ,NFEVAL
$                ,HOLD    ,NHOLD  ,IHOLD  ,NIHOLD ,NEEDED ,IERNLP )
C
C      REVERSE COMMUNICATION TERMINATION TEST
C
C      IF(IRVCOM.EQ.0) GO TO 10000
C
C -----
C
C      SET FUNCTION EVALUATION ERROR FLAG
C
C      IFERR = 0
C
C      >>>>>FUNCTION EVALUATION REQUEST
C
C      IF(IREVRS(1).EQ.1) THEN
C
C          RESVEC(1) = 10.DO*(XBAR(2) - XBAR(1)**2)
C          RESVEC(2) = (1.DO - XBAR(1))
C          RESVEC(3) = 10.DO*(XBAR(4) - XBAR(3)**2)
C          RESVEC(4) = (1.DO - XBAR(3))
C
C          CBAR(1) = XBAR(1)*XBAR(2)
C          CBAR(2) = XBAR(1) + XBAR(3)
C          NFEVAL = NFEVAL + 1
C
C      ENDIF
C
C      >>>>>GRADIENT EVALUATION REQUEST
C
C      IF(IREVRS(2).EQ.1) THEN

```

```
      RMAT(1) = -20.D0*XBAR(1)
      RMAT(2) = -1.D0
      RMAT(3) = 10.D0
      RMAT(4) = -20.D0*XBAR(3)
      RMAT(5) = -1.D0
      RMAT(6) = 10.D0
C
      GMAT(1) = XBAR(2)
      GMAT(2) = 1.D0
      GMAT(3) = XBAR(1)
      GMAT(4) = 1.D0
C
      ELSEIF(IREVRS(2).EQ.2) THEN
C
      RMAT(1) = -20.D0*XBAR(1)
      RMAT(2) = -1.D0
      RMAT(3) = 10.D0
      RMAT(4) = -20.D0*XBAR(3)
      RMAT(5) = -1.D0
      RMAT(6) = 10.D0
C
      GMAT(1) = XBAR(2)
      GMAT(2) = 1.D0
      GMAT(3) = XBAR(1)
      GMAT(4) = 1.D0
C
      ENDIF
C
      >>>>HESSIAN EVALUATION REQUEST
C
      IF(IREVRS(3).EQ.1) THEN
C
      C      DIAGONAL ELEMENTS
C
      HMAT(1) = -20.D0*RESVEC(1)
      HMAT(3) = 0.D0
      HMAT(4) = -20.D0*RESVEC(3)
      HMAT(6) = 0.D0
C
      ELSEIF(IREVRS(3).EQ.2) THEN
C
      C      OFF-DIAGONAL ELEMENTS
C
      HMAT(2) = -VECLAM(1)
      HMAT(5) = 0.D0
C
      ENDIF
C
```

```

C      >>>>SYSTEM PRINT (OUTPUT) REQUEST
C
C      IF(IREVRS(4).EQ.1) THEN
C
C          FOBJ = 0.DO
C          DO I=1,NRES
C              FOBJ = FOBJ + RESVEC(I)**2
C          ENDDO
C          WRITE(6,1001) 'OBJECTIVE  ',.5D0*FOBJ
C          WRITE(6,1001) 'RESIDUALS  ',(RESVEC(II),II=1,NRES)
C          WRITE(6,1001) 'VARIABLES  ',(XBAR(II),II=1,NDIM)
C          WRITE(6,1001) 'CONSTRAINTS',(CBAR(II),II=1,MCON)
C
C      ENDIF
C
C      -----
C
C      2000 CONTINUE
C
C          END OF REVERSE COMMUNICATION PROCESSING.  RETURN TO SBRLSQ
C
C      GO TO 1000
C
C      10000 CONTINUE
C
C          TERMINATE PROCESSING
C
C      IF(IERNLP.EQ.0) THEN
C
C          NORMAL TERMINATION---SOLUTION FOUND
C
C          WRITE(6,1001) 'SOLUTION  '
C
C          FOBJ = 0.DO
C          DO I=1,NRES
C              FOBJ = FOBJ + RESVEC(I)**2
C          ENDDO
C          WRITE(6,1001) 'OBJECTIVE  ',.5D0*FOBJ
C          WRITE(6,1001) 'RESIDUALS  ',(RESVEC(II),II=1,NRES)
C          WRITE(6,1001) 'VARIABLES  ',(XBAR(II),II=1,NDIM)
C          WRITE(6,1001) 'CONSTRAINTS',(CBAR(II),II=1,MCON)
C
C      ELSEIF(IERNLP.LT.0) THEN
C
C          ABNORMAL TERMINATION---FATAL ERROR
C
C          PRINT *, 'FATAL ERROR; IERNLP =', IERNLP
C

```

```

      ELSE
C
C      ABNORMAL TERMINATION---WARNING.  EVALUATION OF RESULTS RECOMMENDED!
C
      PRINT *, 'WARNING MESSAGE; IERNLP =', IERNLP
C
      ENDIF
C
1001 FORMAT(5X,A11,5X,4G17.8)
C
      STOP
      END

```

OUTPUT FROM SAMPLE PROGRAM

It	Lnfltr	KT	Cond	Step	Levnbrg	Optimization Cmplmt	Barrier	Violtn	Grdl	Obj	Log-Barrier Fn
(1)	1	3.8E+04	2.6E-02	0.0E+00	0.0E+00	1.4E-01	1.5E+00	1.2E+02	1.0E+02	1.008536E+02	
(2)	2	6.8E+05	6.8E-01	0.0E+00	1.4E-01	1.4E-01	1.5E+00	1.2E+02	1.0E+02	1.031788E+02	
(3)	1	6.6E+03	1.0E+00	0.0E+00	1.9E-01	1.4E-02	7.0E-01	1.3E+02	2.9E+00	2.988540E+00	
(4)	1	3.6E+03	1.0E+00	0.0E+00	1.4E-01	1.4E-02	1.7E-01	9.7E+01	2.3E+00	2.325695E+00	
(5)	1	2.6E+01	1.0E+00	0.0E+00	1.7E-02	1.4E-02	7.1E-03	9.1E+00	1.6E+00	1.553936E+00	
(6)	1	2.4E+01	3.6E-01	0.0E+00	1.6E-02	1.4E-02	3.1E-03	8.2E+00	1.3E+00	1.305925E+00	
(6)	1	2.4E+01	3.6E-01	0.0E+00	1.6E-02	1.4E-02	3.1E-03	8.2E+00	1.3E+00	1.305925E+00	
(7)	1	1.3E+01	1.0E+00	3.0E-08	1.5E-02	1.4E-02	4.3E-03	7.7E+00	1.2E+00	1.160561E+00	
(8)	1	1.3E+01	1.0E+00	3.0E-08	8.3E-03	1.4E-02	5.6E-03	4.5E+00	9.8E-01	9.316891E-01	
(9)	1	1.5E+01	5.3E-01	1.5E-08	7.7E-03	1.4E-02	5.9E-03	2.6E+00	7.0E-01	6.688495E-01	
(10)	2	4.1E+02	1.0E+00	0.0E+00	1.4E-02	1.4E-02	4.7E-03	1.5E+00	5.8E-01	6.113099E-01	
(11)	1	2.5E+01	1.0E+00	0.0E+00	4.7E-02	1.4E-03	3.6E-05	1.2E-01	6.0E-01	6.023745E-01	
(12)	1	1.3E+02	1.0E+00	0.0E+00	5.3E-03	1.4E-03	2.4E-04	5.9E-02	5.7E-01	5.691454E-01	
(13)	1	7.1E+02	1.0E+00	0.0E+00	1.2E-03	1.4E-04	9.9E-06	1.7E-03	5.6E-01	5.640864E-01	
(14)	1	7.0E+03	1.0E+00	0.0E+00	1.3E-04	1.4E-05	8.5E-07	1.7E-04	5.6E-01	5.626924E-01	
(15)	1	7.0E+04	1.0E+00	0.0E+00	1.4E-05	1.9E-09	6.6E-09	1.6E-06	5.6E-01	5.625139E-01	
(16)	1	7.0E+04	1.0E+00	0.0E+00	3.3E-10	1.9E-09	7.6E-11	7.1E-08	5.6E-01	5.625000E-01	

BARNLP ALGORITHM PERFORMANCE STATISTICS

```

Total CPU Time.....+3.012364E-02
Number of Function Calls.....19
Number of Gradient Calls.....17
Number of Hessian Calls.....11
Total Number of Function Evaluations.....19

Storage Needed in HOLD Array.....762
Storage Needed in IHOLD Array.....147

```

SOLUTION

```

OBJECTIVE      0.56250000
RESIDUALS     -0.51058852E-08   0.75000004   -0.39336229E-09   0.74999996
VARIABLES      0.24999996   0.62499978E-01   0.25000004   0.62500020E-01
CONSTRAINTS    0.15624992E-01   0.50000000

```

SBRLPQ: Sparse Barrier Quadratic Programming

PURPOSE

SBRLPQ computes the vector $x = (x_1, x_2, \dots, x_n)$ which minimizes the quadratic function

$$c^T x + \frac{1}{2} x^T H x$$

subject to the m linear constraints

$$b_L \leq Ax \leq b_U,$$

and the simple bounds

$$x_L \leq x \leq x_U$$

Equality constraints are imposed by setting $b_{Li} = b_{Ui}$ and variables can be fixed by setting $x_{Li} = x_{Ui}$. A *linear programming* problem can be solved by setting $H = 0$. SBRLPQ works under the assumption that the quadratic programming problem has a unique, finite solution. Thus, it is assumed that the projected Hessian matrix is positive definite at the solution. The matrix of first derivatives (the Jacobian) and the matrix of second derivatives (Hessian) are represented in a sparse format consistent with that used in BCSLIB-EXT [1].

RELATED SUBPROGRAMS

SHURQP Sparse Quadratic Programming

INSNLP Sparse Nonlinear Programming Input Procedure

METHOD

An *interior point or barrier* method is used to solve the *quadratic programming* (QP) problem. The quadratic programming algorithm requires the Hessian matrix H and the Jacobian matrix A , both which must be specified in a sparse format. The barrier method is efficient because it avoids the computational complexity of active set methods when there are many inequality constraints, by using a log-barrier transformation. The large sparse KKT system is solved efficiently using the multifrontal algorithm described in BCSLIB-EXT [1]. The algorithm incorporates a nonlinear filter for globalization.

WARNING

The projected Hessian matrix is assumed to be positive definite at the solution, i.e. with the correct active constraints. If this is not true, the software will detect an incorrect value for the matrix inertia and modify the Hessian matrix.

The SBRLPQ software uses internal common blocks to communicate information. To avoid conflict with these internal commons the user should not have commons with the following names: NPSPRR, NPSPRI, NPSPRC, STATIS, KONSTN, NPSALG, NPSDFL, NLPERR, PERCOM, ITEREF, and CLKCOM.

USAGE

```

INTEGER      NDIM, MCON, NZHDIM, NZADIM, NHOLD, NIHOLD, NEEDED, IPU
INTEGER      IPC, IERLPQ
INTEGER      ISTATV(NDIM), IROWH(NZHDIM), JSTRH(NDIM+1)
INTEGER      ISTATC(MCON), IROWA(NZADIM), JCOLA(NZADIM), IHOLD(NIHOLD)

DOUBLE PRECISION      XVEC(NDIM), XLWR(NDIM), XUPR(NDIM), VARMLT(NDIM)
DOUBLE PRECISION      CVCT(NDIM), GZERO(NDIM), HMAT(NZHDIM), BUPR(MCON)
DOUBLE PRECISION      BLWR(MCON), CONMLT(MCON), AMAT(NZADIM), HOLD(NHOLD)
DOUBLE PRECISION      QUAD, BIGBND, CNDNUM

CALL SBRLPQ(NDIM,MCON,HMAT,IROWH,JSTRH,NONZH,CVCT,GZERO,
$  QUAD,AMAT,IROWA,JCOLA,NONZA,BUPR,BLWR,CONMLT,ISTATC,
$  XVEC,XUPR,XLWR,VARMLT,ISTATV,HOLD,NHOLD,IHOLD,NIHOLD,
$  NEEDED,BIGBND,IPU,IPC,CNDNUM,IERLPQ)

```

ARGUMENTS

PROBLEM DATA

NDIM [INPUT, INTEGER]
Number of variables n , where $1 \leq \text{NDIM}$.

MCON [INPUT, INTEGER]
Number of constraints m , where $0 \leq \text{MCON}$.

OBJECTIVE FUNCTION DATA

HMAT [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NONZH containing the nonzero elements of Hessian matrix H of the quadratic function. Since the Hessian is symmetric, only the lower triangular part is input (see USAGE REMARKS). Storage for the diagonal elements must be provided even if they are zero.

IROWH [INPUT, INTEGER, ARRAY]
An integer array of length NONZH containing the row indices of the Hessian nonzeros, where $0 < \text{IROWH}(k) \leq \text{NDIM}$ for $k = 1, \dots, \text{NONZH}$. The diagonal elements must be specified, i.e. $\text{IROWH}(\text{JSTRH}(k)) = k$ for $k = 1, \dots, \text{NDIM}$.

JSTRH [INPUT, INTEGER, ARRAY]
An integer array of length NDIM+1 containing the column start indices of nonzeros, where $\text{JSTRH}(1) = 1$, $\text{JSTRH}(\text{NDIM}+1) = \text{NONZH} + 1$, and $\text{JSTRH}(k) < \text{JSTRH}(k+1)$ for $k = 1, \dots, \text{NDIM}$.

NONZH [INPUT, INTEGER]
Dimension of Hessian nonzero arrays, where $n \leq \text{NONZH} \leq n(n+1)/2$, and $\text{NONZH} = \text{JSTRH}(\text{NDIM}+1) - 1$.

CVCT [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM containing the linear term c in the objective function.

GZERO [OUTPUT, DOUBLE PRECISION, ARRAY]
A real array of length NDIM containing the gradient of the objective function at the solution XVEC.

QUAD [OUTPUT, DOUBLE PRECISION]
The objective function evaluated at the solution XVEC.

CONSTRAINT DATA

AMAT [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length NONZA containing the nonzero elements of the Jacobian matrix A . Not referenced when MCON = 0.

IROWA [INPUT, INTEGER, ARRAY]
An integer array of length NONZA containing the row indices of the Jacobian nonzeros, where $0 < \text{IROWA}(k) \leq \text{MCON}$ for $k = 1, \dots, \text{NONZA}$. Not referenced when MCON = 0.

JCOLA [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length NONZA containing the column indices of the Jacobian nonzeros, where $0 < \text{JCOLA}(k) \leq \text{NDIM}$ for $k = 1, \dots, \text{NONZA}$. Not referenced when MCON = 0.

NONZA [INPUT, INTEGER]
The dimension of the arrays AMAT, IROWA, and JCOLA – usually the number of nonzero Jacobian elements, where $\max(\text{NDIM}+1, \text{NONZA}) \leq \text{NDIM} * \text{MCON}$.

BUPR [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraint upper bounds b_U . For constraints that have no upper bound set $b_{U_i} \geq \text{BIGBND}$. Not referenced when MCON = 0.

BLWR [INPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON containing the constraint lower bounds b_L . For constraints that have no lower bound set $b_{L_i} \leq -\text{BIGBND}$. Not referenced when MCON = 0.

CONMLT [OUTPUT, DOUBLE PRECISION, ARRAY]
A real array of length MCON of Lagrange multipliers, λ , for the constraints. Not referenced when MCON = 0.

ISTATC [INPUT/OUTPUT, INTEGER, ARRAY]
An integer array of length MCON indicating the constraint status. On input ISTATC defines an estimate of the active constraints, and on output ISTATC defines the constraints active at the solution.

ISTATC(i)= 0 Free (inactive) inequality, i.e. $b_{L_i} < b_i(x) < b_{U_i}$.

ISTATC(i)= 1 Constraint lower bound is active, i.e. $b_{L_i} = b_i(x)$.

ISTATC(i)= 2 Constraint upper bound is active, i.e. $b_i(x) = b_{U_i}$.

ISTATC(i)= 3	Equality.
ISTATC(i)= 4	Ignored constraint.
ISTATC(i)= -1	Constraint lower bound is violated, i.e. $b_i(x) < b_{Li}$.
ISTATC(i)= -2	Constraint upper bound is violated, i.e. $b_{Ui} < b_i(x)$.

When ISTATC(i) = 3 it is necessary that $b_{Li} = b_{Ui}$, where $b_{Li} = \text{BLWR}(i)$ and $b_{Ui} = \text{BUPR}(i)$. When ISTATC(i) $\neq$ 3, it is recommended that $b_{Li} < b_{Ui}$ with $|b_{Li} - b_{Ui}| > [\text{HDMCON}(5)]^{\frac{1}{2}}$. Not referenced when MCON = 0.

INDEPENDENT VARIABLE DATA

XVEC	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM of variable values x . An initial guess must be input. On output, XVEC contains the solution, provided IER = 0.
XUPR	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the variable upper bounds x_U . For variables that have no upper bound set $x_{Ui} \geq \text{BIGBND}$.
XLWR	[INPUT, DOUBLE PRECISION, ARRAY] A real array of length NDIM containing the variable lower bounds x_L . For variables that have no lower bound set $x_{Li} \leq -\text{BIGBND}$.
VARMLT	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] An array of length NDIM of bound multipliers ν .
ISTATV	[INPUT/OUTPUT, INTEGER, ARRAY] An integer array of length NDIM indicating variable status. On input ISTATV defines an estimate of the active variable bounds, and on output ISTATV defines the bounds active at the solution.
ISTATV(i)= 0	Variable is strictly feasible, i.e. $x_{Li} < x_i < x_{Ui}$.
ISTATV(i)= 1	Variable lower bound is active, i.e. $x_{Li} = x_i$.
ISTATV(i)= 2	Variable upper bound is active, i.e. $x_i = x_{Ui}$.
ISTATV(i)= 3	Fixed permanently.
ISTATV(i)= -1	Variable lower bound is violated, i.e. $x < x_{Li}$.
ISTATV(i)= -2	Variable upper bound is violated, i.e. $x_{Ui} < x$.

When ISTATV(i) = 3 it is necessary that $x_{Li} = x_{Ui}$, where $x_{Li} = \text{XLWR}(i)$ and $x_{Ui} = \text{XUPR}(i)$. When ISTATV(i) $\neq$ 3, it is recommended that $x_{Li} < x_{Ui}$ with $|x_{Li} - x_{Ui}| > [\text{HDMCON}(5)]^{\frac{1}{2}}$.

ALGORITHM CONTROL DATA

HOLD	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] A real array of length NHOLD containing the hold array.
NHOLD	[INPUT/OUTPUT, INTEGER] Dimension of HOLD array. When IER = -127 or -131 NEEDED contains the storage needed.
IHOLD	[INPUT/OUTPUT, INTEGER, ARRAY] An integer array of length NIHOLD containing the integer hold array.
NIHOLD	[INPUT/OUTPUT, INTEGER] Dimension of IHOLD array. When IER = -128 or -132, NEEDED contains the storage needed.
NEEDED	[OUTPUT, INTEGER] When IER = -127 or -131, NEEDED contains the required length of HOLD. When IER = -128 or -132, NEEDED contains the required length for IHOLD.
BIGBND	[INPUT, DOUBLE PRECISION] Big bound value. A suggested value for BIGBND = .01/HDMCON(5). See Appendix B for HDMCON.
IPU	[INPUT, INTEGER] Output Unit Number.
IPC	[INPUT, INTEGER] Output Control Flag. IPC=0 No Output. 0<IPC<10 Terse Output. 10≤IPC<20 Standard Output. 20≤IPC<30 Interpretive Output. 30=IPC Diagnostic Output.
CNDNUM	[OUTPUT, DOUBLE PRECISION] Condition number of KKT system.
IER	[OUTPUT, INTEGER] Success/error code. Results are not computed for IER < 0. For IER > 0, results are computed but may be suspect; user inspection is recommended. Note some of the error returns involve optional input quantities specified using INSNLP. IER=0 Normal termination.

IER=-101	(MCON < 0); the number of constraints is negative.
IER=-102	(NDIM < 1); the number of variables is less than one.
IER=-103	Incorrect value for QPOPTN.
IER=-104	IHESHN > 3.
IER=-105	(NITMAX < max(1,NITMIN)); the maximum number of iterations is either less than 1 or less than the minimum number of iterations.
IER=-107	(IT1MAX < 1); the number of line search steps is less than 1.
IER=-108	(IPC < 0) or (IPC > 30); invalid input for the output control flag.
IER=-109	Invalid input for ALGOPT.
IER=-110	(OBJTOL ≤ 10.*HDMCON(5)); objective function tolerance is too small.
IER=-111	(PGDTOL ≤ [HDMCON(5)] ^{1/2}) or (PGDTOL > 10 ⁻²); the projected gradient tolerance is too small or too large.
IER=-112	CONTOL < [HDMCON(5)] ^{1/2} ; the constraint tolerance is too small.
IER=-113	(ISTATC(i) < 0) or (ISTATC(i) > 4); invalid input for constraint status.
IER=-114	(NONZA ≤ 0) or (NONZA > NDIM*MCON); the number of Jacobian nonzeros is either less than one, or exceeds the size of a dense matrix.
IER=-115	(JCOLA(i) ≤ 0) or (JCOLA(i) > NDIM); invalid input for Jacobian column index array.
IER=-116	(IROWA(i) ≤ 0) or (IROWA(i) > MCON); invalid input for Jacobian row index.
IER=-117	(BUPR(i) < BLWR(i)); constraint upper bound is less than lower bound.
IER=-118	Either (BUPR(i) = BLWR(i)) and (ISTATC(i) ≠ 3) or (BUPR(i) ≠ BLWR(i)) and (ISTATC(i) = 3); constraint status array is not consistent with bounds.
IER=-119	(BUPR(i) ≠ BLWR(i)) and (BUPR(i)-BLWR(i) < CONTOL); constraint bounds are not equal, but differ by less than the constraint tolerance.
IER=-120	(ISTATV(i) < 0) or (ISTATV(i) > 3); invalid input for variable status array.

IER=-121	(JSTRH(1) $\neq$ 1) or (JSTRH(k) $\geq$ JSTRH(k+1)); invalid input for Hessian column start array.
IER=-122	(NZHDIM < NDIM) or (NZHDIM > NDNSH) or (NZHDIM $\neq$ NONZH) where NZHDIM = JSTRH(NDIM+1)-1, and NDNSH = NDIM*(NDIM+1)/2; the number of Hessian nonzeros is less than one, exceeds the size of a dense Hessian, or is not consistent with the input value NONZH.
IER=-123	(IROWH(i) $\leq$ 0) or (IROWH(i) > NDIM); invalid value for Hessian row index.
IER=-124	(XUPR(i) < XLWR(i)); the variable upper bound is less than the lower bound.
IER=-125	Either (XUPR(i) = XLWR(i)) and (ISTATV(i) $\neq$ 3) or (XUPR(i) $\neq$ XLWR(i)) and (ISTATV(i) = 3); the variable status array is not consistent with the bound values.
IER=-126	(XUPR(i) $\neq$ XLWR(i)) and (XUPR(i)-XLWR(i) < CONTOL); variable bounds are not equal but differ by less than the constraint tolerance.
IER=-127	Double precision hold array too small; insufficient storage detected in SBRNLP interface. The required storage is specified in NEEDED.
IER=-128	Integer hold array too small; insufficient storage detected in SBRNLP interface. The required storage is specified in NEEDED.
IER=-129	Function error at initial point or during gradient evaluation.
IER=-130	BIGCON < CONTOL.
IER=-131	NHOLD too small; insufficient double precision storage detected in algorithm. The required storage is specified in NEEDED.
IER=-132	NIHOLD too small; insufficient integer storage detected in algorithm. The required storage is specified in NEEDED.
IER=-133	Rank deficient Jacobian detected on successive iterations.
IER=-134	INSNLP input error; invalid character string displayed.
IER=-135	(IFERR < 0) or (IFERR > 1); invalid value for function error flag.
IER=-136	FEATOL < CONTOL; the initial feasibility tolerance is less than the constraint tolerance.

IER=-137	Conflict between user and multifrontal file number; check IPUMF1,...,IPUMF6 IPUDRF, IPUFZF, IPUSTF.
IER=-138	NEWTON $\neq 0, 1, 2$; invalid input for Newton method option flag.
IER=-145	(IROWH(JSTRH(i)) $\neq i$) for some value of i; Hessian diagonal elements are incorrect.
IER=-146	PMULWR < HDMCON(5).
IER=-147	Unexpected error; check storage allocation.
IER=-148	IRVCOM > 1.
IER=-149	PTHTOL < [HDMCON(5)] ^{1/2} .
IER=-150	RHOLWR < HDMCON(5).
IER=-151	A constraint is inconsistent (either $b_i(x) < b_{Li}$ or $b_{Ui} < b_i(x)$) and cannot be changed because the corresponding row of the Jacobian is zero.
IER=-152	IMAXMU < 1.
IER=-153	I/O error, probably caused by insufficient disk space.
IER=-154	TOLKTC < 1.
IER=-155	TOLPVT < 0 or TOLPVT > .5.
IER=-156	IRELAX < 0 or IRELAX > 2.
IER=-157	MUCALC = 0 or MUCALC > 3.
IER=-158	MXQPIT < 1.

IER=+101	Weak solution found (relaxation required and/or multipliers near zero).
IER=+102	Number of equality constraints = number of free variables and ALGOPT $\neq$ 'F'.
IER=+104	Maximum number of function evaluations.
IER=+105	Small step termination in optimization phase; suboptimal feasible point found.

IER=+106	Maximum number of iterations.
IER=+108	Feasible point not found.
IER=+109	Maximum number of interval halves in line search.
IER=+110	Algorithm terminated because the Hessian diagonal reached its maximum value; suboptimal feasible point found.
IER=+111	Projected gradient calculation failed; constraints may be degenerate.
IER=+112	Calculation of first order multiplier estimates failed; constraints may be degenerate.
IER=+113	Suboptimal feasible point found.
IER=+114	Barrier NLP failed with unexpected error.
IER=+115	CONTOL > OBJTOL; convergence tolerances may be inappropriate.
IER=+116	Uphill direction detected in line search.
IER=+117	Reduced objective function is linear.
IER=+118	Both $b_{Li} \leq -.01/\text{HDMCON}(5)$ and $b_{Ui} \geq .01/\text{HDMCON}(5)$; constraints ignored.

USAGE REMARKS

Sparse Matrix Input

Hessian Matrix. The user must supply the Hessian matrix H of the quadratic function

$$c^T x + \frac{1}{2} x^T H x.$$

Consider the following example in which $n = 3$:

$$H = \begin{pmatrix} 1.1 & 0 & 3.1 \\ 0 & 2.2 & 0 \\ 3.1 & 0 & 3.3 \end{pmatrix}.$$

Since the Hessian is a symmetric matrix only the lower triangular portion must be specified. For this example NDIM = 3 and NONZH = 4 = [JSTRH(NDIM+1)-1]. The nonzero values are specified as HMAT(*) = (1.1, 3.1, 2.2, 3.3). The row indices are IROWH(*) = (1, 3, 2, 3), and the column start indices are JSTRH(*) = (1, 3, 4, 5). Notice that the element in row 1, column 3 is not specified. Also observe that the diagonal elements must always be included in the Hessian

sparsity pattern *even if the elements are zero*. Additional information on sparse matrix formats is found in BCSLIB-EXT [1].

Jacobian Matrix. The user must specify the nonzero elements in the Jacobian matrix when required. Consider the following example in five variables $n = 5$ and three constraints $m = 3$:

$$A = \begin{pmatrix} 0 & 0 & 1.3 & 0 & 1.5 \\ 2.1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 3.3 & 0 & 0 \end{pmatrix}$$

For this example $\text{NDIM} = 5$, $\text{MCON} = 3$, and the number of nonzeros $\text{NONZA} = 4$. The matrix can be specified in two formats. The nonzero array is $\text{AMAT}^* = (2.1, 1.3, 3.3, 1.5)$. The row indices are $\text{IROWA}^* = (2, 1, 3, 1)$, and the column indices are $\text{JCOLA}^* = (1, 3, 3, 5)$. Note that sufficient storage must be provided in the array JCOLA to accomodate at least $\text{NDIM}+1$ values. Additional information on sparse matrix formats is found in BCSLIB-EXT [1].

Unconstrained Problems

When the problem is unconstrained the user must set $\text{MCON} = 0$. The following constraint related quantities will not be referenced: BLWR, BUPR, ISTATC, CONMLT, AMAT, IROWA, JCOLA, and NONZA. Since these quantities must appear as arguments in the call to SBRLPQ, dummy values may be utilized.

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine SBRLPQ. All optional inputs are specified by calls to subroutine INSNLP. It should be emphasized that it is *not necessary* to call INSNLP to set default values.

OPTIONAL PRINTOUT

The level of printed output from SBRLPQ is controlled by the variables IPC and IPU. The *default* for SBRLPQ is to have no output which corresponds to $\text{IPC} = 0$.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2, x_3, x_4)$ to minimize the objective function

$$f(x) = c^T x + \frac{1}{2} x^T H x$$

subject to the constraints

$$b_L \leq Ax \leq b_U,$$

where $b_L^T = (-\infty, -.5)$ and $b_U^T = (.5, .5)$. The solution must also satisfy the bounds

$$-2 \leq x_k \leq 2$$

for $k = 1, 2, 3, 4$ and as an initial guess select $x = (-1, 2, -1, 2)$. For this example the vector

$$c^T = [400, 200, 400, 200]$$

and the Jacobian matrix is

$$A = \begin{bmatrix} 2 & -1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}.$$

It is also necessary to supply the Hessian matrix

$$H = \begin{bmatrix} 400 & 1 & 0 & 0 \\ 1 & 200 & 0 & 0 \\ 0 & 0 & 2 & -1 \\ 0 & 0 & -1 & 20 \end{bmatrix}.$$

SAMPLE PROGRAM (sbrlpq.cd1)¹²

```

      PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE SPARSE QUADRATIC
C      PROGRAMMING ALGORITHM SHURQP
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
      PARAMETER (NZHDIM=100,NZADIM=100)
      PARAMETER (NDIM=4, MCON=2)
      PARAMETER (NONZH=6, NONZA=4,LNJCLA=MAX(NDIM+1,NONZA))
      PARAMETER (NHOLD=5000,NIHOLD=5000)
C
      DIMENSION XVEC(NDIM),XLWR(NDIM),XUPR(NDIM),CVCT(NDIM),
$      GZERO(NDIM),ISTATV(NDIM),VARMLT(NDIM),HMAT(NZHDIM),
$      IROWH(NZHDIM),JSTRH(NDIM+1),BLWR(MCON),
$      BUPR(MCON),ISTATC(MCON),CONMLT(MCON),CVEC(MCON),
$      AMAT(NZADIM),IROWA(NZADIM),JCOLA(LNJCLA)
      DIMENSION HOLD(NHOLD),IHOLD(NIHOLD)
C
      CHARACTER(LEN=20) CHRVAR
C
C -----
C
C      INDEPENDENT VARIABLE DATA
C
C      ----INITIAL GUESS
      XVEC(1) = -1.0D0
      XVEC(2) =  2.0D0
      XVEC(3) = -1.0D0
      XVEC(4) =  2.0D0
C
C      ----BOUNDS
      XLWR(1:NDIM) = -2.D0
      XUPR(1:NDIM) =  2.D0

```

¹²sosexample.tar

```
C      ----VARIABLE STATUS
      ISTATV(1:NDIM) = 0
C
C -----
C
C      OBJECTIVE FUNCTION DATA
C
C      ----HESSIAN SPARSITY PATTERN (ROW INDICES)
      IROWH(1) = 1
      IROWH(2) = 2
      IROWH(3) = 2
      IROWH(4) = 3
      IROWH(5) = 4
      IROWH(6) = 4
C      ----HESSIAN SPARSITY PATTERN (COLUMN START INDICES)
      JSTRH(1) = 1
      JSTRH(2) = 3
      JSTRH(3) = 4
      JSTRH(4) = 6
      JSTRH(5) = 7
C      ----DIAGONAL ELEMENTS
      HMAT(1) = 400.DO
      HMAT(3) = 200.DO
      HMAT(4) = 2.DO
      HMAT(6) = 20.DO
C      ----OFF-DIAGONAL ELEMENTS
      HMAT(2) = 1.DO
      HMAT(5) = -1.DO
C
C      ----LINEAR TERM FOR OBJECTIVE FUNCTION
C
      CVCT(1) = 400.DO
      CVCT(2) = 200.DO
      CVCT(3) = 400.DO
      CVCT(4) = 200.DO
C
C -----
C
C      CONSTRAINT DATA
C
C      ----BOUNDS
      BIGBND = 1.DO/HDMCON(5)
C
      BLWR(1) = -BIGBND
      BUPR(1) = .5DO
      BLWR(2) = -.5DO
      BUPR(2) = .5DO
C      ----CONSTRAINT STATUS
```

```

      ISTATC(1:MCON) = 0
C      ----JACOBIAN SPARSITY PATTERN (ROW INDICES)
      IROWA(1) = 1
      IROWA(2) = 2
      IROWA(3) = 1
      IROWA(4) = 2
C      ----JACOBIAN SPARSITY PATTERN (COLUMN INDICES)
      JCOLA(1) = 1
      JCOLA(2) = 1
      JCOLA(3) = 2
      JCOLA(4) = 3
C
      AMAT(1) = 2.DO
      AMAT(2) = 1.DO
      AMAT(3) = -1.DO
      AMAT(4) = 1.DO
C
C
C -----
C
C      ----LP/QP OUTPUT CONTROL PARAMETERS
C
      IPU = 6
      IPC = 0
C
C -----
C
      CALL SBRLPQ(NDIM,MCON,HMAT,IROWH,JSTRH,NONZH,CVCT,GZERO,
$      QUAD,AMAT,IROWA,JCOLA,NONZA,BUPR,BLWR,CONMLT,ISTATC,
$      XVEC,XUPR,XLWR,VARMLT,ISTATV,HOLD,NHOLD,IHOLD,NIHOLD,
$      NEEDED,BIGBND,IPU,IPC,IERLQP)
C
C -----
C
      TERMINATE PROCESSING
C
      IF(IERLQP.EQ.0) THEN
C
      NORMAL TERMINATION---SOLUTION FOUND
C
      WRITE(IPU,1001) 'SOLUTION      '
C
      WRITE(IPU,1001) 'OBJECTIVE    ',QUAD
C
      COMPUTE AMAT*XVEC = CVEC
C
      CALL MVPSRC(1,MCON,NDIM,NONZA,AMAT,IROWA,JCOLA,XVEC,CVEC)
C

```

```
C      FINAL STATUS
C
      WRITE(IPU,1003)
      DO I = 1,MCON
        IF(ISTATC(I).EQ.0) THEN
          CHRVAR = 'FREE (INACTIVE) INEQUALITY'
        ELSEIF(ISTATC(I).EQ.1) THEN
          CHRVAR = 'FIXED ON LOWER BOUND'
        ELSEIF(ISTATC(I).EQ.2) THEN
          CHRVAR = 'FIXED ON UPPER BOUND'
        ELSEIF(ISTATC(I).EQ.3) THEN
          CHRVAR = 'EQUALITY'
        ELSEIF(ISTATC(I).EQ.4) THEN
          CHRVAR = 'IGNORED CONSTRAINT'
        ELSEIF(ISTATC(I).EQ.-1) THEN
          CHRVAR = 'VIOLATES LOWER BOUND'
        ELSEIF(ISTATC(I).EQ.-2) THEN
          CHRVAR = 'VIOLATES UPPER BOUND'
        ENDIF
        WRITE(IPU,1002) I,CVEC(I),CONMLT(I),CHRVAR
      ENDDO
C
      WRITE(IPU,1004)
      DO I = 1,NDIM
        IF(ISTATV(I).EQ.0) THEN
          CHRVAR = 'FREE VARIABLE'
        ELSEIF(ISTATV(I).EQ.1) THEN
          CHRVAR = 'FIXED ON LOWER BOUND'
        ELSEIF(ISTATV(I).EQ.2) THEN
          CHRVAR = 'FIXED ON UPPER BOUND'
        ELSEIF(ISTATV(I).EQ.3) THEN
          CHRVAR = 'FIXED PERMANENTLY'
        ELSEIF(ISTATV(I).EQ.4) THEN
          CHRVAR = 'IGNORED BOUND'
        ELSEIF(ISTATV(I).EQ.-1) THEN
          CHRVAR = 'VIOLATES LOWER BOUND'
        ELSEIF(ISTATV(I).EQ.-2) THEN
          CHRVAR = 'VIOLATES UPPER BOUND'
        ENDIF
        WRITE(IPU,1002) I,XVEC(I),VARMLT(I),CHRVAR
      ENDDO
C
      ELSEIF(IERLQP.LT.0) THEN
C
C      ABNORMAL TERMINATION---FATAL ERROR
C
      PRINT *, 'FATAL ERROR; IERLQP =', IERLQP
C
```

```

ELSE
C
C      ABNORMAL TERMINATION---WARNING.  EVALUATION OF RESULTS RECOMMENDED!
C
C      PRINT *, 'WARNING MESSAGE; IERLQP =', IERLQP
C
C      ENDIF
C
1001 FORMAT(//, 5X, A, T30, 4G17.8)
1002 FORMAT(5X, I2, T10, G17.8, T30, G17.8, T52, A)
1003 FORMAT(//, T13, 'CONSTRAINT', T33, 'MULTIPLIER', T52, 'STATUS', /)
1004 FORMAT(//, T15, 'VARIABLE', T33, 'MULTIPLIER', T52, 'STATUS', /)
C
      STOP
      END

```

OUTPUT FROM SAMPLE PROGRAM

SOLUTION

OBJECTIVE	-652.08592
-----------	------------

	CONSTRAINT	MULTIPLIER	STATUS
1	0.50000000	-33.997927	FIXED ON UPPER BOUND
2	-0.50000000	401.32919	FIXED ON LOWER BOUND

	VARIABLE	MULTIPLIER	STATUS
1	-0.16459370	0.	FREE VARIABLE
2	-0.82918740	-0.71054274E-14	FREE VARIABLE
3	-0.33540630	0.56843419E-13	FREE VARIABLE
4	-2.00000000	160.33541	FIXED ON LOWER BOUND

INSNLP: Sparse Nonlinear Programming Input Procedure

PURPOSE

INSNLP is a utility subroutine used to define optional inputs for the suite of nonlinear programming algorithms: for the SQP methods DNLPGW, DNLPRV, DNLPGF, SNLPMN, SNLPLS and SHURQP; or the barrier methods DBNLPG, DBNLPR, DBNPGF, SBRNLP, SBRLSQ and SBRLPG. Subroutine INPSOS (p. 401) can be used as an alternative to INSNLP.

RELATED SUBPROGRAMS

DNLPGW	Dense Nonlinear Programming
DNLPRV	Dense Nonlinear Programming–Reverse Communication Format
DNLPGF	Dense Nonlinear Programming–User Supplied Gradients
SNLPMN	Sparse Nonlinear Programming
SNLPLS	Sparse Constrained Nonlinear Least Squares
SHURQP	Sparse Quadratic Programming
DBNLPG	Dense Barrier Nonlinear Programming
DBNLPR	Dense Barrier Nonlinear Programming–Reverse Communication Format
DBNPGF	Dense Barrier Nonlinear Programming–User Supplied Gradients
SBRNLP	Sparse Barrier Nonlinear Programming
SBRLSQ	Sparse Barrier Constrained Nonlinear Least Squares
SBRNLP	Sparse Barrier Nonlinear Programming
INSOCX	Sparse Optimal Control Input Procedure
INPSOS	Sparse Optimization Suite Input Procedure

METHOD

Subroutine INSNLP must be called once for every optional parameter to be set. The input to INSNLP must be a single character variable of length not to exceed 80 characters. The string is case insensitive and is assumed to have the following format:

' symbol = value '

The symbol must be one of the valid mnemonics defined below; see ARGUMENTS. Information contained in the character string is communicated to the sparse nonlinear programming algorithms SNLPMN, SNLPLS, DNLPRV, DNLPGW, SHURQP, DBNLPG, DBNLPR, SBRNLP, and SBRLPG via one of five commons:

NPSRR	Real variable inputs
NPSRI	Integer variable inputs

NPSPRC	Character variable inputs.
BNPSPR	Real variable inputs
BNPSPI	Integer variable inputs

The character string must have an equals sign “=” to separate the *symbol* from the *value*. The only exceptions to this rule are when *symbol* = 'SUMMARY', 'SPARSE DEFAULT', 'DENSE DEFAULT', 'BARRIER DEFAULT', 'OPTION', or 'FULL OPTIONS'. The *value* portion of the string can contain integer, real (single or double precision), or character values.

All parameters not specified by the user are automatically set to their default values. Any optional parameters that are set by the user are not altered by algorithms SNLPMN, SNLPLS, DNLPRV, DNLPUF, SHURQP, DBNLPU, DBNLPR, SBRNLP, and SBRLPQ and hence changes to the options are cumulative. Therefore, any input changes made by the user will remain in effect until they are explicitly reset by another call to INSNLP. Many of the optional inputs are used both by the SQP algorithms (SNLPMN, SNLPLS, DNLPRV, DNLPUF, SHURQP) and the barrier algorithms (DBNLPU, DBNLPR, SBRNLP, SBRLPQ). Input parameters that apply only to the SQP algorithms are indicated as “SQP Only.” Similarly, input parameters that are applicable to the barrier algorithms are denoted by “Barrier Only.”

There are two possible sources for error when using INSNLP;

- (1) The character string input to INSNLP is invalid,
- (2) The character string input to INSNLP is valid, but the *value* is incorrect for algorithms SNLPMN, SNLPLS, DNLPRV, DNLPUF, SHURQP, DBNLPU, DBNLPR, SBRNLP, and SBRLPQ.

Errors encountered by INSNLP as a result of incorrect character strings (class 1) are communicated to the user by a call to the error handler HHERR, followed by a call to HHERRX to display the incorrect string. A list of the character string errors is given below (see ARGUMENTS). Input errors caused by an incorrect *value* (i.e. class 2) are detected by algorithms SNLPMN, SNLPLS, DNLPRV, DNLPUF, SHURQP, DBNLPU, DBNLPR, SBRNLP, and SBRLPQ when they are called and the information is communicated to the user by the appropriate error return flag.

USAGE

```
CHARACTER STRING*k
CALL INSNLP(STRING)
```

ARGUMENTS

STRING [INPUT, CHARACTER]
Character variable of length k where $k \leq 80$, of the form $\text{STRING}(1:k) = \text{'symbol = value'}$.

Convergence and Termination Tolerances

The following optional inputs can be used to control convergence and termination:

<u>Symbol</u>	<u>Default</u>	<u>Description</u>
CONTOL	$\epsilon^{\frac{1}{2}}$	Constraint tolerance, $\delta_c \geq \epsilon^{\frac{1}{2}}$, where $\epsilon = \text{HDMCON}(5)$.

OBJTOL	10^{-7}	Objective function tolerance, δ_o , where $\delta_o \geq \max(10\epsilon, \delta_c)$
PGDTOL	10^{-5}	Projected gradient tolerance, δ_p , where $10\epsilon < \delta_p \leq .01$
MAXNFE	10000	Maximum number of function evaluations.
NITMAX	100	Maximum number of iterations. $\text{NITMAX} \geq \max(1, \text{NITMIN})$.
NITMIN	0	Minimum number of iterations.

Line Search

The following inputs can be used to control the line search:

<u>Symbol</u>	<u>Default</u>	<u>Description</u>
SLPTOL	0.9	Slope tolerance δ_s for line search during optimization phase, where $10^{-5} < \delta_s < 1$. (SQP Only)
SFZTOL	.01	Slope tolerance δ_s for line search during feasibility phase, where $10^{-5} < \delta_s < 1$. Monotonic error reduction when $\delta_s < 0$; linear constraint satisfaction step when $\delta_s > 0$. (SQP Only)
IT1MAX	20	Maximum number of steps for line search. $\text{IT1MAX} \geq 1$.

Line Search Diagnostic Plot File

A file named 'LYNPLT.FIL' containing information suitable for a diagnostic plot can be created from data available during the line search. The following inputs can be used to control the diagnostic output written on the file:

<u>Symbol</u>	<u>Default</u>	<u>Description</u>
ALFLWR	0.0	Lower bound on steplength, i.e. the independent variable for the plot file.
ALFUPR	1.0	Upper bound on steplength, i.e. the independent variable for the plot file.
LYNFNC	0	Number of the dependent function to be plotted. When $\text{LYNFNC} = 0$, the file will contain the merit or log-barrier function and objective function. When $\text{LYNFNC} = i$, the file will contain constraint i , Lagrange multiplier i , and slack variable i .
LYNOUT	0	Output unit number for line search plot file.
LYNPLT	0	Iteration number to create line search plot file (and then stop).
LYNPNT	101	Number of equally spaced points between ALFLWR and ALFUPR to evaluate quantities.
LYNVAR	0	Number of the independent function to be plotted. When $\text{LYNVAR} = 0$, the file will contain the value of the steplength α . When $\text{LYNVAR} = i$, the file will contain the steplength α , variable i , and bound slack variable i .

The first line of the file produced contains the following four integers: LYNPNT, NCOLS, LYNVAR, LYNFNC. Subsequently the file contains LYNPNT rows of data, displayed in NCOLS columns.

Input Parameters

Output Line

LYNVAR=0 and LYNFNC=0	$\alpha_k, M(\alpha_k)$ or $\beta(\alpha_k), f(\alpha_k)$.
LYNVAR=0 and LYNFNC= i	$\alpha_k, c_i(\alpha_k), \lambda_i(\alpha_k), s_i(\alpha_k)$.
LYNVAR= i and LYNFNC=0	$\alpha_k, x_i(\alpha_k), t_i(\alpha_k), M(\alpha_k)$ or $\beta(\alpha_k), f(\alpha_k)$.
LYNVAR= i and LYNFNC= i	$\alpha_k, x_i(\alpha_k), t_i(\alpha_k), c_i(\alpha_k), \lambda_i(\alpha_k), s_i(\alpha_k)$.

where $1 \leq i \leq m$ and $1 \leq k \leq \text{LYNPNT}$. After the LYNPNT rows of line search data, three additional pieces of information are written. The number of variables NDIM, the vector XBAR containing the nominal value x (at $\alpha = 0$), and the search direction VECP, where $\bar{x} = x + \alpha p$. An example illustrating the line search diagnostic output is given in Section 4.5.

Dense Algorithm Inputs

The following inputs can be used to control the calculation of gradient and Hessian information for the dense algorithms DNLPCFW, DBNLPF, DNLPRV, DBNLPV and DNLPGF:

<u>Symbol</u>	<u>Default</u>	<u>Description</u>
EPSRLF	$\epsilon^{\frac{1}{2}}$	Relative perturbation size parameter, ϵ_R . The forward difference perturbation size is $\delta_F = (1 + x) \epsilon_R ^9$ and the central difference perturbation size is $\delta_C = (1 + x) \epsilon_R ^6$. For analytic functions it is appropriate to set $\epsilon_R = \epsilon^{\frac{1}{2}}$ where $\epsilon = \text{HDMCON}(5)$. Reverse communication algorithms DNLPRV and DBNLPV do not use EPSRLF. For algorithms DNLPCFW or DBNLPF: EPSRLF $\geq \epsilon^{\frac{1}{2}}$ finite difference perturbation sizes are fixed for all iterations. EPSRLF $\leq -\epsilon^{\frac{1}{2}}$ finite difference perturbation sizes are adjusted from iteration to iteration to minimize the finite difference error. For algorithm DNLPGF the Hessian is constructed using gradient differences: EPSRLF $\geq \epsilon^{\frac{1}{2}}$ forward gradient differences with $\delta_F = (1 + x) \epsilon_R ^9$ EPSRLF $\leq -\epsilon^{\frac{1}{2}}$ central gradient differences with $\delta_C = (1 + x) \epsilon_R ^6$.

IHESHN	0	Hessian matrix evaluation option; $0 \leq \text{IHESHN} \leq 3$. When $\text{IHESHN} \leq 0$, the Hessian diagonal elements are initialized to finite difference estimates.
	IHESHN=0	Finite difference or analytic.
	IHESHN=1	SR1 (Symmetric Rank One); the default for DNLPGW and DBNLPG. Not used by DNLPGF.
	IHESHN=2	BFGS (Symmetric Positive Definite).. Not used by DNLPGF.
	IHESHN=3	SSQN (Self-Scaling Quasi-Newton).. Not used by DNLPGF.

Barrier Algorithm Inputs

The following inputs can be used to control the behavior of the barrier algorithms DBNLPG, DBNLPR, SBRNLP and SBRLPG:

Symbol	Default	Description
BIGCON	100.	Upper Bound on Equality Constraint Error. All iterates must satisfy the bound $\ c(y^{(k)})\ _\infty \leq \max\{\ c(y^{(0)})\ _\infty, \text{BIGCON}\}$.
FEATOL	.001	Initial Variable Offset for Feasibility. The initial (feasible) point for the barrier algorithm $\hat{y}_k$ is constructed from the user supplied guess y_k^0 according to:

$$\begin{aligned}\alpha &= \max[\text{FEATOL}, \text{FEATOL}|y_k^0|] \\ \delta &= \min[\alpha, .5|y_{Uk} - y_{Lk}|] \\ \hat{y}_k &= \max\{y_{Lk} + \delta, \min[y_k^0, y_{Uk} - \delta]\}.\end{aligned}$$

PMULWR	.1	Lower Bound on Initial Barrier Parameter; $\text{PMULWR} > \epsilon$ where $\epsilon = \text{HDMCON}(5)$.
PTHTOL	10.	Central Path Convergence Tolerance; $\text{PTHTOL} > \sqrt{\epsilon}$ where $\epsilon = \text{HDMCON}(5)$. The barrier parameter μ is reduced when $\ F_\mu\ < \min[\kappa\mu, \text{PTHTOL}]$. Thus a small value for PTHTOL will force accurate location of the central path.
RHOLWR	100.	Lower Bound on Initial Relaxation Parameter ρ ; $\text{RHOLWR} > \epsilon$ where $\epsilon = \text{HDMCON}(5)$. For a well-posed problem $\rho \approx \ (\eta, \lambda)\ _\infty$ and $\text{RHOLWR} \leq \rho$.
IMAXMU	10	Maximum Number of Iterations N_u with Fixed Barrier Parameter; $\text{IMAXMU} \geq 1$.

MUCALC 3 Barrier/Multiplier Initialization Option; MUCALC 1, ± 2 , ± 3 . There are three basic options where the symbol $\sim$ denotes the minimum norm least squares solution.

Option 1 (MUCALC = 1). Solve

$$\begin{aligned}\mu &= \text{PMULWR} \\ \lambda &= \mu D_b^{-1} e \\ C^T \eta &\sim g - B^T \lambda.\end{aligned}$$

Option 2 (MUCALC = ± 2). Solve

$$\begin{pmatrix} C^T & B^T D_b^{-1} e \end{pmatrix} \begin{pmatrix} \eta \\ \mu \end{pmatrix} \sim g$$

$$\lambda = \max[\mu, \text{PMULWR}] D_b^{-1} e.$$

If MUCALC = 2 and $\mu < \text{PMULWR}$, recompute using Option 1.

Option 3 (MUCALC = ± 3). Solve

$$\begin{pmatrix} C^T & B^T & 0 \\ 0 & D_b & -e \end{pmatrix} \begin{pmatrix} \eta \\ \hat{\lambda} \\ \mu \end{pmatrix} \sim \begin{pmatrix} g \\ 0 \end{pmatrix}$$

$$\lambda = \max[\epsilon, \hat{\lambda}].$$

where $\epsilon = \text{HDMCON}(5)$. If MUCALC = 3 and $\mu < \text{PMULWR}$, recompute using Option 1.

MXQPIT 1 Maximum Number of Barrier QP Iterations.

Output Control

The basic iteration output levels can be specified using the input IOFLAG described below. Additional output control is available using one or more of the other flags listed.

<u>Symbol</u>	<u>Default</u>	<u>Description</u>
IOFLAG	10	Output level; $0 \leq \text{IOFLAG} \leq 30$.
		IOFLAG=0 No Output.
		$0 < \text{IOFLAG} < 10$ Terse Output.
		$10 \leq \text{IOFLAG} < 20$ Standard Output.
		$20 \leq \text{IOFLAG} < 30$ Interpretive Output.
		$30 = \text{IOFLAG}$ Diagnostic Output.

IPUNLP	6	Output unit number for iteration output from algorithms.
IOFLIN	-1	Line search output level. A value of IOFLIN > 0 overrides the default, which is to set IOFLIN=IOFLAG.
IOFMFR	0	Output level flag for multifrontal linear algebra software (cf BCSLIB-EXT [1]).
	IOFMFR ≤ 0	No Output.
	IOFMFR = 1	Error messages and summary statistics.
	IOFMFR = 2	More complete statistics.
	IOFMFR = 3	First stage of debugging output.
	IOFMFR ≥ 4	Complete debugging output.
IOFPAT	0	Sparsity pattern output flag . Sparsity output may be written on the standard output unit or to an external file. A value of IOFPAT ≥ 10 produces sparsity pattern output. No sparsity output is produced for IOFPAT < 10. Three possibilities exist:
	IOFLAG = 30	Sparsity output on standard unit IPUNLP (equivalent to IOFPAT = 10.)
	IOFPAT = L	Sparsity output on standard unit IPUNLP. IOFPAT ≥ 10 and equal to one of the values in the list $L = (10, \text{IPUDRF}, \text{IPUFZF}, \text{IPUMF1}, \text{IPUMF2}, \text{IPUMF3}, \text{IPUMF4}, \text{IPUMF5}, \text{IPUMF6}, \text{IPUMF7}, \text{IPUSTF})$.
	IOFPAT ≠ L	Sparsity output on file 'SPRPAT.FIL' on unit IOFPAT. IOFPAT ≥ 10 and different than all values in the list $L = (10, \text{IPUDRF}, \text{IPUFZF}, \text{IPUMF1}, \text{IPUMF2}, \text{IPUMF3}, \text{IPUMF4}, \text{IPUMF5}, \text{IPUMF6}, \text{IPUMF7}, \text{IPUSTF})$.
IOFSHR	0	The Schur-complement or Barrier QP output level. A value of IOFSHR ≥ 0 overrides the default, which is to set IOFSHR = 0, for IOFLAG < 30 and IOFSHR = IOFLAG otherwise.
IOFSRC	0	Feasibility search output level. A value of IOFSRC > 0 overrides the default, which is to set IOFSRC = IOFLAG. (SQP Only)
ITDRQP	-1	If ITDRQP > 0, the input to the Schur-complement QP will be dumped as file 'SRCHDRQP.FIL' on optimization iteration number ITDRQP and then the processing will terminate. (SQP Only)
IPUDRF	0	Output unit for the Schur-complement QP dump from SQP. (SQP Only)

ITFZQP	-1	If ITFZQP > 0, the input to the Schur-complement QP will be dumped as file 'SRCHFZQP.FIL' on feasibility iteration number ITFZQP and then the processing will terminate. (SQP Only)
IPUFZF	0	Output unit for the Schur-complement QP dump from feasibility search. (SQP Only)
IPUSTF	0	If IPUSTF > 0, the input to the Schur-complement QP will be dumped as file 'QPSTRTQP.FIL' on output unit number IPUSTF, during the QPSTRT Lagrange multiplier calculations and then the processing will terminate. (SQP Only)
IPUMF1	11	Multifrontal I/O unit (see XSSLIN in BCSLIB-EXT [1]).
IPUMF2	12	Multifrontal I/O unit (see XSSLIN in BCSLIB-EXT [1]).
IPUMF3	13	Multifrontal I/O unit (see XSSLIN in BCSLIB-EXT [1]).
IPUMF4	14	Multifrontal I/O unit (see XSSLIN in BCSLIB-EXT [1]).
IPUMF5	15	Multifrontal I/O unit (see XSSLIN in BCSLIB-EXT [1]).
IPUMF6	16	Multifrontal I/O unit (see XSSLIN in BCSLIB-EXT [1]).
IPUMF7	17	Multifrontal I/O unit (see XSSLIN in BCSLIB-EXT [1]).
MAXLYN	5	Maximum number of output lines for arrays.

<u>Symbol</u>	<u>Default</u>	<u>Description</u>
TOLFIL	2	Tolerance used to detect excessive fill in the multifrontal method; $0 \leq \text{TOLFIL}$. A larger value of TOLFIL will permit additional fill.
TOLKTC	$\epsilon^{-1.6}$	Tolerance for the condition number of the KKT matrix, where $\epsilon = \text{HDMCON}(5)$. Iterative refinement is initiated when the condition number exceeds TOLKTC. TOLKTC is also used by the RRQR algorithm when computing rank.
TOLPVT	10^{-3}	Multifrontal pivot tolerance used to balance between sparsity and numeric stability; $0 \leq \text{TOLPVT} \leq .5$. A small value reduces fill, and a large value improves stability.
IRELAX	1	Constraint relaxation strategy option flag; IRELAX=0,1. (IRELAX=2, Barrier Only). IRELAX=0 Do not switch to relaxation strategy. IRELAX=1 Switch to relaxation strategy if necessary to make progress. IRELAX=2 Always use relaxation strategy. (Barrier Only)

JACPRM	0	Jacobian permutation option flag; JACPRM=0,1. (SQP Only)
		JACPRM=0 Return Jacobian matrix in internal (permuted) order during function evaluations.
		JACPRM=1 Restore Jacobian matrix to original external order during function evaluations.
NEWTON	0	Newton method option flag; NEWTON=0,1. (NEWTON=2, Barrier Only).
		NEWTON=0 Evaluate Hessian as needed to make good progress (not necessarily each optimization iteration).
		NEWTON=1 Evaluate Hessian on each optimization iteration.
		NEWTON=2 Do not evaluate Hessian on any iteration. (Barrier Only)
NORMAL	0	Least Squares Normal Matrix Option; NORMAL=0,1. (Least Squares Only)
		NORMAL=0 Use sparse tableau KKT system; do not form normal matrix.
		NORMAL=1 Use normal matrix in KKT system.
ALGOPT	FM	Algorithm control option.
		ALGOPT=FM Find feasible point then minimize (SQP Default).
		ALGOPT=FME Find feasible point then minimize with equalities binding. (SQP Only)
		ALGOPT=M Minimize from the initial point (Barrier Default).
		ALGOPT=F Find feasible point only.
		ALGOPT=LLSQ Linear Least Squares (only valid for SNLPLS).

KTOPTN	SMALL	KKT Matrix Factorization option.	
		KTOPTN=SMALL	Factor KKT matrix consisting of active constraints (SQP Default). This is the default and is recommended for problems with many inequality constraints. Factor the condensed KKT matrix (Barrier Default).
		KTOPTN=LARGE	Factor the augmented KKT matrix with all constraints.
QPOPTN	SPARSE	Quadratic Programming Algorithm option .	
		QPOPTN=SPARSE	Solve QP subproblem using sparse Schur-complement method. This is the default for SNLPMN and SNLPLS.
		QPOPTN=DENSE	Solve QP subproblem using dense null-space method. This is the default for DNLPRV and DNLPRV. (SQP Only)

Special Options

A number of special options can be obtained from INSNLP by setting STRING(1:k) = '*symbol*'. The following inputs describe the special options available:

<u>Symbol</u>	<u>Description</u>
SPARSE DEFAULT	Set default values for all optional inputs.
DENSE DEFAULT	Set default values appropriate for dense applications for all optional inputs. The following dense default values are <u>different</u> from the sparse defaults:
<u>Symbol</u>	<u>Dense</u> <u>Sparse</u>
IHESHN	1 0
NEWTON	1 0
IPUMF1	−1 11
IPUMF2	−1 12
IPUMF3	−1 13
IPUMF4	−1 14
IPUMF5	−1 15
IPUMF6	−1 16
IPUMF7	−1 17
QPOPTN	DENSE SPARSE

BARRIER DEFAULT Set default values for the barrier algorithm optional inputs. The following barrier default values are different from the sparse defaults:

<u>Symbol</u>	<u>Barrier</u>	<u>Sparse</u>
ALGOPT	M	FM

FULL OPTION Write a full description of the optional parameters.

OPTION Write a description of the optional parameters.

SUMMARY Write a brief description of the important optional parameters, and the optional parameters that are not default values.

Character String Input Errors

When an invalid value is input for the character variable STRING(1:k) subroutine INSNLP calls the standard error handler HHERR with an error flag $IER < 0$, followed by a call to HHERRX which displays the invalid string. The nonstandard error returns are defined as follows:

IER= -1 String too long (more than 80 characters).

IER= -2 Replacement error during capitalization.

IER= -3 Equal sign “=” missing from string.

IER= -4 Invalid real value.

IER= -5 Invalid integer value.

IER= -6 Invalid character value.

IER= -7 Invalid symbol name.

SAMPLE PROGRAM (insnlp.cd1)¹³

PROGRAM EXAMPL

```
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE INPUT UTILITY INSNLP FOR THE
C      SPARSE NONLINEAR PROGRAMMING SOFTWARE SUITE.  NOTICE THAT THE
C      CHARACTER STRING INPUTS TO INSNLP HAVE BEEN CONSTRUCTED (E.G. WITH
C      UPPER AND LOWER CASE LETTERS, SPACES, ETC.) TO ILLUSTRATE FEATURES
C      OF INSNLP.  THIS IS NOT INTENDED TO DEMONSTRATE RECOMMENDED
C      PROGRAMMING STYLE!
C
```

¹³sosexample.tar

Subprograms for Optimization

*				0 < IOFLAG < 10 Terse Output	*
*				9 < IOFLAG < 20 Standard Output	*
*				19 < IOFLAG < 30 Interpretive Output	*
*				IOFLAG = 30 Diagnostic Output	*
*	IOFLIN	-1	-1	Line Search Output Level	*
*				(0,10,20) Overrides Default (IOFLIN=IOFLAG)	*
*	IOFMFR	0	0	Multifrontal Output Level (0,1,2,3,4)	*
*	IOFPAT	0	0	Output Sparsity Pattern	*
*				.GE. 10 Overrides Default	*
*	IOFSHR	0	0	QP Output Level	*
*	IOFSRC	0	0	Feasibility Search Output Level	*
*				(0,10,20,30) Overrides Default (IOFSRC=IOFLAG)	*
*	IPOSTO	0	0	Postoptimality Analysis Option	*
*	IPUDRF	0	0	Output Unit for QP Dump From Optimization	*
*	IPUFZF	0	0	Output Unit for QP Dump From Feasibility	*
*	IPUMF1	11	11	Multifrontal I/O Unit	*
*	IPUMF2	12	12	Multifrontal I/O Unit	*
*	IPUMF3	13	13	Multifrontal I/O Unit	*
*	IPUMF4	14	14	Multifrontal I/O Unit	*
*	IPUMF5	15	15	Multifrontal I/O Unit	*
*	IPUMF6	16	16	Multifrontal I/O Unit	*
*	IPUMF7	17	17	Multifrontal I/O Unit	*
*	IPUNLP	6	6	Output Unit Number	*
*	IPUSTF	0	0	Output Unit No. for QP Dump From QP START	*
*	IRELAX	1	1	Constraint Relaxation Strategy Option	*
*				= 0 No Relaxation	*
*				= 1 Relaxation	*
*	ITDRQP	-1	-1	Dump QP on SQP Iteration No.	*
*	ITFZQP	-1	-1	Dump QP on Feasible Search Iteration No.	*
*	IT1MAX	20	20	Maximum Number of Steps for SQP Line Search	*
*	JACPRM	0	0	Jacobian Permutation Option (External Order = 1)	*
*	LYNFNC	0	0	Function Number (Line Search Diagnostic Plot)	*
*	LYNOUT	0	0	Output Unit Number (Line Search Diagnostic Plot)	*
*	LYNPLT	0	0	Iteration Number For Line Search Diagnostic Plot	*
*	LYNPNT	101	101	No. of Plot Points (Line Search Diagnostic Plot)	*
*	LYNVAR	0	0	Variable Number (Line Search Diagnostic Plot)	*
*	MAXLYN	5	5	Maximum Line Limit for Array Output	*
*	MAXNFE	10000	10000	Maximum Number of Function Evaluations	*
*	MNSAME	2	2	Switch to Equality QP after MNSAME Steps	*
*	NEWTON	0	0	Newton Method Option (NEWTON = 1; 0 Otherwise)	*
*	NITMAX	100	100	Maximum Number of Iterations	*
*	NITMIN	0	0	Minimum Number of Iterations	*
-----					*
*					*
*	ALGOPT	FM	FM	Algorithm Control Option	*
*				= FM Find Feasible Point Then Minimize	*
*				= FME Find Feasible Point Then Minimize	*
*				With Equalities Binding	*
*				= M Minimize From The Initial Point	*
*				= MWRM Minimize From The Initial Point	*
*				(WARM Start)	*
*				= MHOT Minimize From The Initial Point	*
*				(HOT Start)	*
*				= F Find Feasible Point Only	*
*				= FWRM Find Feasible Point Only (WARM Start)	*
*				= FHOT Find Feasible Point Only (HOT Start)	*
*				= LLSQ Linear Least Squares	*
*	KTOPTN	SMALL	SMALL	KT Matrix Factorization Option	*

*				9 < IOFLAG < 20 Standard Output	*
*				19 < IOFLAG < 30 Interpretive Output	*
*				IOFLAG = 30 Diagnostic Output	*
*	IOFMFR	0	0	Multifrontal Output Level (0,1,2,3,4)	*
*	IOFPAT	0	0	Output Sparsity Pattern	*
*				.GE. 10 Overrides Default	*
*	IPUMF1	11	11	Multifrontal I/O Unit	*
*	IPUMF2	12	12	Multifrontal I/O Unit	*
*	IPUMF3	13	13	Multifrontal I/O Unit	*
*	IPUMF4	14	14	Multifrontal I/O Unit	*
*	IPUMF5	15	15	Multifrontal I/O Unit	*
*	IPUMF6	16	16	Multifrontal I/O Unit	*
*	IPUMF7	17	17	Multifrontal I/O Unit	*
*	IPUNLP	6	6	Output Unit Number	*
*	IRELAX	0	0	Constraint Relaxation Strategy Option	*
*				= 0 No Relaxation	*
*				= 1 Relaxation as Needed	*
*	IT1MAX	20	20	Dump QP on Feasible Search Iteration No.	*
*	JACPRM	0	0	Maximum Number of Steps for Filter Line Search	*
*	LYNFNC	0	0	Jacobian Permutation Option (External Order = 1)	*
*	LYNOUT	0	0	Function Number (Line Search Diagnostic Plot)	*
*	LYNPLT	0	0	Output Unit Number (Line Search Diagnostic Plot)	*
*	LYNPNT	101	101	Iteration Number For Line Search Diagnostic Plot	*
*	LYNVAR	0	0	No. of Plot Points (Line Search Diagnostic Plot)	*
*	MUCALC	3	3	Barrier/Multiplier Initialization Option	*
*	MXQPIT	1	1	Maximum Number of Barrier QP Iterations	*
*	MAXLYN	5	5	Variable Number (Line Search Diagnostic Plot)	*
*	MAXNFE	10000	10000	Maximum Line Limit for Array Output	*
*	NEWTON	0	0	Newton Option: G-N (0); Newton (1); Gauss (2)	*
*	NITMAX	100	100	Maximum Number of Iterations	*
*	NITMIN	0	0	Minimum Number of Iterations	*
*				-----	*
*					*
*	ALGOPT	M	M	Algorithm Control Option	*
*				= FM Find Feasible Point Then Minimize	*
*				= M Minimize From The Initial Point	*
*				= F Find Feasible Point Only	*
*	KTOPTN	SMALL	SMALL	KT Matrix Factorization Option	*
*				= SMALL Condensed KKT System	*
*				= LARGE Full KKT System	*

RTSNLP: Sparse Nonlinear Programming Parameter Retrieval

PURPOSE

RTSNLP is a utility subroutine used to retrieve algorithm control parameter values from the sparse nonlinear programming algorithms SNLPMN and SNLPLS or the dense algorithms DNLPRV and DNLPCFW.

RELATED SUBPROGRAMS

INSNLP	Sparse Nonlinear Programming Input Procedure
DNLPCFW	Dense Nonlinear Programming
DNLPRV	Dense Nonlinear Programming-Reverse Communication Format
SNLPMN	Sparse Nonlinear Programming
SNLPLS	Sparse Constrained Nonlinear Least Squares

METHOD

Subroutine RTSNLP can be used to retrieve the value for any optional parameter that can be set by the routine INSNLP. Subroutine INSNLP permits input in the following format:

' symbol = value '

Given the character string *'symbol'*, RTSNLP retrieves the quantity *'value'*. A single parameter value is retrieved with a single call to RTSNLP.

USAGE

```
CHARACTER STRING*6
INTEGER          IVAL
DOUBLE PRECISION          RVAL
CHARACTER*6 CVAL
CALL RTSNLP (STRING, IVAL, RVAL, CVAL)
```

ARGUMENTS

STRING	[INPUT, CHARACTER] Character variable where STRING(1:6) corresponds to the input <i>'symbol'</i> in the INSNLP routine.
IVAL	[OUTPUT, INTEGER] Integer value for the parameter with mnemonic STRING(1:6). If STRING(1:6) does not correspond to any integer parameter then IVAL = JHMCON(1).
RVAL	[OUTPUT, DOUBLE PRECISION] Real value for the parameter with mnemonic STRING(1:6). If STRING(1:6) does not correspond to any real parameter then RVAL = HDMCON(1).

Character*6 value for the parameter with mnemonic STRING(1:6). If STRING(1:6) does not correspond to any character parameter then CVAL = '_____'.

2.3 Optimization Iteration Output

The level of printed output from the suite of optimization programs is controlled by the variables IOFLAG and IPUNLP which are set by calls to INSNLP. When IOFLAG = 0 and/or IPUNLP ≤ 0 , no output is printed by the software. Four different levels of iteration output are provided, when IPUNLP > 0 and IOFLAG > 0 . These levels are referred to as Terse, Standard, Interpretive, and Diagnostic and are defined in the following subsections. The *default* level of output for all subprograms in the optimization suite is referred to as Standard Output. In addition to output produced by the optimization software, the user has direct access to information via iteration log commons.

2.3.1 SQP Algorithms

Terse Output

Terse Output is obtained by setting $0 < \text{IOFLAG} < 10$, and all values of IOFLAG in this range will produce the same result. The output is segregated into two blocks denoted by the headings “Feasible Point”, and “Optimization” which correspond to the phase being performed by the algorithm. The following information is printed in either phase:

It	The major iteration number. The iteration number is enclosed in parenthesis, e.g. “(13)” when accurate or central difference gradients are required.
Qit	The number of QP iterations. The first solution of the KT system is counted as iteration one, and subsequent changes to the QP active set count as additional iterations.
Nkt	The number of numeric factorizations of the symmetric indefinite KT system (cf reference 1) when QPOPTN=SPARSE. The number of Cholesky factorizations of the reduced Hessian when QPOPTN=DENSE.
Ndof	The number of degrees of freedom; $Ndof = n - m_a$ where n is the number of variables and m_a is the number of active constraints.
KT Cond	The condition number of the symmetric indefinite KT system when QPOPTN=SPARSE. If QPOPTN=DENSE, during the feasibility phase this column displays “Cond(G)” which indicates the condition of the Jacobian matrix, and during the optimization phase “PH Cond” which indicates the condition of the reduced Hessian matrix.
Step	The length of the step α taken in the current search direction. The step length should be one as the solution is approached.
Norm p	The norm of the search direction vector $\ p\ $.
Violtn	The norm of the error in the violated constraints.

The following additional information is printed during the optimization phase only:

Levnbrg	The value of the Levenberg parameter τ .
Penalty	The norm of the penalty vector (diagonal elements of Q and R) used in the merit function.
Norm Pg	The norm of the projected gradient $\ g - G^T \lambda - \nu\ _\infty$.
Merit Function	The value of the merit function M .

Standard Output

Standard Output is obtained by setting $10 \leq \text{IOFLAG} < 20$, and all values of IOFLAG in this range will produce the same result. All standard output is enclosed between the symbols “*” in columns 2 and 104. By convention any action or message which is “nonstandard” is preceded by “....” as in the message “.....THE FOLLOWING CONSTRAINTS MAY BE LINEARLY DEPENDENT.” Although no action may be necessary by the user, it is suggested that all such messages be reviewed. Unlike the terse output format, quantities displayed in the standard format are more self explanatory. The iteration history begins with a summary of the problem inputs and settings. All such quantities are displayed using the program mnemonics. Any nonstandard input option is preceded by a “*” in the algorithm summary. The progress of the algorithm is reported during the feasibility phase as follows:

Iteration	The major iteration number. The iteration number is enclosed in parenthesis, e.g. “(13)” when accurate or central difference gradients are required.
Constraint Error	The norm of the error in the violated constraints.
Cond(K) or Cond(G)	The condition number of the symmetric indefinite KT system when QPOPTN=SPARSE, or the condition number of the Jacobian when QPOPTN=DENSE.
Active Constraints	The number of active constraints.
Degrees of Freedom	The number of degrees of freedom; $Ndof = n - m_a$ where n is the number of variables and m_a is the number of active constraints.
QP Iterations	The number of QP iterations. For the sparse QP, the first solution of the KT system is counted as iteration one, and subsequent changes to the QP active set count as additional iterations. For the dense QP, changes to the active set during both the LP feasibility phase, and optimization phase are counted.
Matrix Factorizations	The number of numeric factorizations of the symmetric indefinite KT system when QPOPTN=SPARSE, or the number of Cholesky factorizations of the reduced Hessian when QPOPTN=DENSE.

During the optimization phase the following additional information is printed

Projected Gradient	The norm of the projected gradient $\ g - G^T \lambda - \nu\ _\infty$.
Objective Function	The value of the objective function $f(x)$.
Equality Error	The norm of the error in the equality constraints.
Inequality Error	The norm of the error in the violated inequality constraints.
Merit Function	The value of the merit function M .
Penalty Weight Norm	The norm of the penalty vector (diagonal elements of Q and R) used in the merit function.
Levenberg Parameter	The value of the Levenberg parameter τ .
Min. Eigenvalue	The Gerschgorin bound for the minimum eigenvalue of H_L .
Max. Eigenvalue	The Gerschgorin bound for the maximum eigenvalue of H_L .

A line search is used during both the optimization and feasibility phase to adjust the steplength α . A single line of the form

Model Line SearchStep: *iter*ALFA = α

is printed which contains the following information about the line search:

<i>Model</i>	The model used to predict the steplength. The models are:
Quadratic	quadratic polynomial to predict minimum;
Cubic	cubic polynomial to predict minimum;
Quadratic Cliff	quadratic polynomial to predict location of cliff;
Cubic Cliff	cubic polynomial to predict location of cliff;
Bisection	bisect the interval to predict minimum;
Backtrack	step reduction when a function error is encountered;
<i>iter</i>	the line search step number;
α	the length of the step in the search direction p .

After completing a step in the line search a single line of the form

```
.....Index = M( $\alpha$ )   Change =  $\Delta M$ 
```

is printed which contains the results of the step where

$M(\alpha)$ is the value of the merit function at α ,

ΔM is the change in the merit function, i.e. $\Delta M = M(\alpha) - M(0)$.

During the feasibility phase this line prints “Error” instead of “Index” and shows the change in the constraint violation instead of the merit function M .

Interpretive Output

Interpretive Output is obtained by setting $20 \leq \text{IOFLAG} < 30$, and all values of IOFLAG in this range will produce the same result. The interpretive output format prints everything described for standard output and augments this information. The algorithm summary print is given in a “full” format with more extensive explanation of the quantities and options. When using the “FME” option, all iterations performed by the constraint elimination process are printed using a format similar to the feasibility phase output. In addition more extensive output is displayed from the line search procedure. In particular a block of the form

Eval	alfa	f(alfa)-f(0)	
1	0.000000E+00	0.000000E+00	
4	1.780138E-01	-2.196746E-01 *	
3	3.999480E-01	7.561202E-03	
2	1.000000E+00	4.585334E-01	

f'(0.) =	-1.83294	f''(0.) =	4.69820
f2'(0.1780) =	-0.229049	f3'(0.1780) =	-0.454380

is displayed after every line search step. The first block tabulates the progress of the line search so far. Column one presents the function evaluation number. The second column presents the steplength in a monotonically increasing order. The third column presents the corresponding change in the merit function when compared to the reference value at $\alpha = 0$. An evaluation which is a local minimum is denoted by a “*”. In the illustration above, this occurs on evaluation number 4 (i.e. row 2), at the point $\alpha = 1.780138E - 01$. In this example the minimum is “bracketed” i.e. the minimum α^* satisfies $0 \leq \alpha^* \leq 3.999480E - 01$. If the best point is not bracketed this is denoted by a “<” rather than “*”. The last two lines present slope and curvature information for the function being minimized. The direction derivative evaluated at the reference point $f'(\alpha = 0)$ is denoted by “f'(0.)”. Similarly the curvature at the reference point is denoted by “f''(0.)”. The quantity denoted “f2'(α)” prints the slope of the quadratic model evaluated at the current best point α . Finally, the quantity denoted “f3'(α)” prints the slope of the cubic model evaluated at the current best point α . The quadratic and/or cubic slope estimates are used to verify the line search slope tolerance test.

Diagnostic Output

Diagnostic Output is obtained by setting $30 = \text{IOFLAG}$. The diagnostic output format prints everything described for the standard and interpretive output and augments this information. The

values of the gradient, Jacobian, and Hessian matrices are printed at each evaluation. The sparsity pattern of the Jacobian and Hessian matrices is printed at the initial point. Extensive diagnostic print is displayed at each iteration of the quadratic programming subproblem. *The diagnostic option produces a great deal of output and should not be used without a complete understanding of the software!*

Optimization Iteration Log Commons

Many of the quantities that appear in the iteration output can also be obtained from commons accessible to the user. The SQP algorithms save information in the following two commons:

```
COMMON /QITLGI/ ITP,KEYMP,MACTIP,NDOFP,NQPITP,NUMKTP
COMMON /QITLGR/ PPGNOR,PFNOM,PERREQ,PERRIN,PFMRT,PPENMA,PDIAGN,
$              PCNDNU,PEIGMI,PEIGMA
```

The integer quantities in the common QITLGI are defined as follows:

ITP	The major iteration number.
KEYMP	Algorithm Key Indicator; (0 — Feasibility phase), (1 — Optimization phase).
MACTIP	The number of active constraints.
NDOFP	The number of degrees of freedom.
NQPITP	The number of QP iterations.
NUMKTP	The number of numeric factorizations of the KT system.

The double precision (real) quantities in the common QITLGR are defined as follows:

PPGNOR	The norm of the projected gradient.
PFNOM	The value of the objective function.
PERREQ	The norm of the error in the equality constraints.
PERRIN	The norm of the error in the violated inequality constraints.
PFMRT	The value of the merit function.
PPENMA	The norm of the penalty vector.
PDIAGN	The value of the Levenberg parameter.
PCNDNU	The condition number of the KT system.
PEIGMI	The Gerschgorin bound for the minimum eigenvalue.

PEIGMA The Gerschgorin bound for the maximum eigenvalue.

2.3.2 Barrier Algorithms

Terse Output

Terse Output is obtained by setting $0 < \text{IOFLAG} < 10$, and all values of IOFLAG in this range will produce the same result. Usually the output is preceded by the heading “Optimization.” When the output is preceded by the heading “Feasible Point”, the objective being minimized is the constraint error $f = \frac{1}{2}c^T c$. The following information is printed:

It	The major iteration number. The iteration number is enclosed in parenthesis, e.g. ”(13)” when accurate or central difference gradients are required.
Lnfltr	The number of distinct entries in the globalization filter.
KT Cond	The condition number of the symmetric indefinite KT system.
Step	The length of the step α taken in the current search direction. The step length should be one as the solution is approached.
Levnbrg	The value of the Levenberg parameter τ .
Cmplmt	The error in the approximate complementarity conditions; $\max_k b_k(y)\lambda_k - \mu $.
Barrier	The barrier parameter, μ .
Violtn	The norm of the error in the equality constraints, $\ c\ _\infty$.
Grdl	The norm of the gradient of the Lagrangian, $\ g - C^T\eta - B^T\lambda\ _\infty$.
Obj	The value of the objective function f .
Log-Barrier Fn	The value of the log-barrier function $\beta(y, \mu)$.

Standard Output

Standard Output is obtained by setting $10 \leq \text{IOFLAG} < 20$, and all values of IOFLAG in this range will produce the same result. All standard output is enclosed between the symbols “*” in columns 2 and 104. By convention any action or message which is “nonstandard” is preceded by “.....” as in the message “.....THE FOLLOWING CONSTRAINTS MAY BE LINEARLY DEPENDENT.” Although no action may be necessary by the user, it is suggested that all such messages be reviewed. Unlike the terse output format, quantities displayed in the standard format are more self explanatory. The

iteration history begins with a summary of the problem inputs and settings. All such quantities are displayed using the program mnemonics. Any nonstandard input option is preceded by a “*” in the algorithm summary. The progress of the algorithm is reported as follows:

Iteration	The major iteration number. The iteration number is enclosed in parenthesis, e.g. ”(13)” when accurate or central difference gradients are required.
Objective Function	The value of the objective function $f(x)$.
Gradient Of Lagrangian	The norm of the gradient of the Lagrangian, $\ g - C^T\eta - B^T\lambda\ _\infty$.
Log-Barrier Function	The value of the log-barrier function $\beta(y, \mu)$.
Equality Error	The norm of the error in the equality constraints, $\ c\ _\infty$.
Barrier Parameter	The barrier parameter, μ .
Complementarity	The error in the approximate complementarity conditions; $\max_k b_k(y)\lambda_k - \mu $.

A line search is used in conjunction with a filter mechanism to globalize the algorithm. The current entries in the filter given in monotonically increasing order of constraint violation are summarized in output of the form:

Filter: B(y)		c(y)
1	+7.342311E+00	+6.797935E-01
2	+6.436914E+00	+6.898000E-01
3	+6.447988E+00	+4.110440E+00

The first column presents the number of the filter entry. The second column presents the value of the log-barrier function and the third column presents the corresponding constraint violation. A line search is used to adjust the steplength α . A single line of the form

Model Line Search**Step:** *iter***ALFA** = α

is printed which contains the following information about the line search:

<i>Model</i>	The model used to predict the steplength. The models are:	
	Quadratic	standard quadratic (Newton) model of Lagrangian;
	Backtrack	step reduction when a function error is encountered;
	Quadratic Barrier	quadratic/logarithmic model to predict minimum of barrier function;
	Linear Barrier	linear/logarithmic model to predict minimum of barrier function;
	Constraint Error	quadratic polynomial model to predict minimum of constraint violation $c^T c$;
	Contraction	minimum allowable contraction from previous step;
	Filter	safeguarded contraction when other models fail.

iter the line search step number;
 α the length of the step in the search direction.
 After completing a step in the line search a single line of the form

..... (B, |c|) = ($\beta(\alpha)$, |c|)

is printed which contains the results of the step where

$\beta(\alpha)$ is the value of the log-barrier function at α ,
 |c| is the constraint violation at α .

Interpretive Output

Interpretive Output is obtained by setting $20 \leq \text{IOFLAG} < 30$, and all values of IOFLAG in this range will produce the same result. The interpretive output format prints everything described for standard output and augments this information. The algorithm summary print is given in a “full” format with a more extensive explanation of the quantities and options.

The following additional information is also printed:

Levenberg Parameter	The value of the Levenberg parameter τ .
Cond(K)	The condition number of the symmetric indefinite KT system.
Min. Eigenvalue	The Gerschgorin bound for the minimum eigenvalue of H_L .
Max. Eigenvalue	The Gerschgorin bound for the maximum eigenvalue of H_L .

Diagnostic Output

Diagnostic Output is obtained by setting $30 = \text{IOFLAG}$. The diagnostic output format prints everything described for the standard and interpretive output and augments this information. The values of the gradient, Jacobian, and Hessian matrices are printed at each evaluation. The sparsity pattern of the Jacobian and Hessian matrices is printed at the initial point. Extensive diagnostic print is displayed at each iteration of the quadratic programming subproblem. *The diagnostic option produces a great deal of output and should not be used without a complete understanding of the software!*

2.4 Reverse Communication Program Structure

The reverse communication structure of subroutine SNLPMN (and the related routines DNLPRV and SNLPLS) permits the user a great deal of flexibility when implementing an application. Typically the optimization software is called directly from either a main program or high level executive routine. This format permits the user to supply gradient and Hessian information using a technique appropriate for the application. This format is also useful when the application is implemented in a multitasking environment, since the function and/or gradient evaluations can be executed on different platforms. To aid the user with a reverse communication implementation, a flow chart presenting the various operations is presented in Figure 2-1.

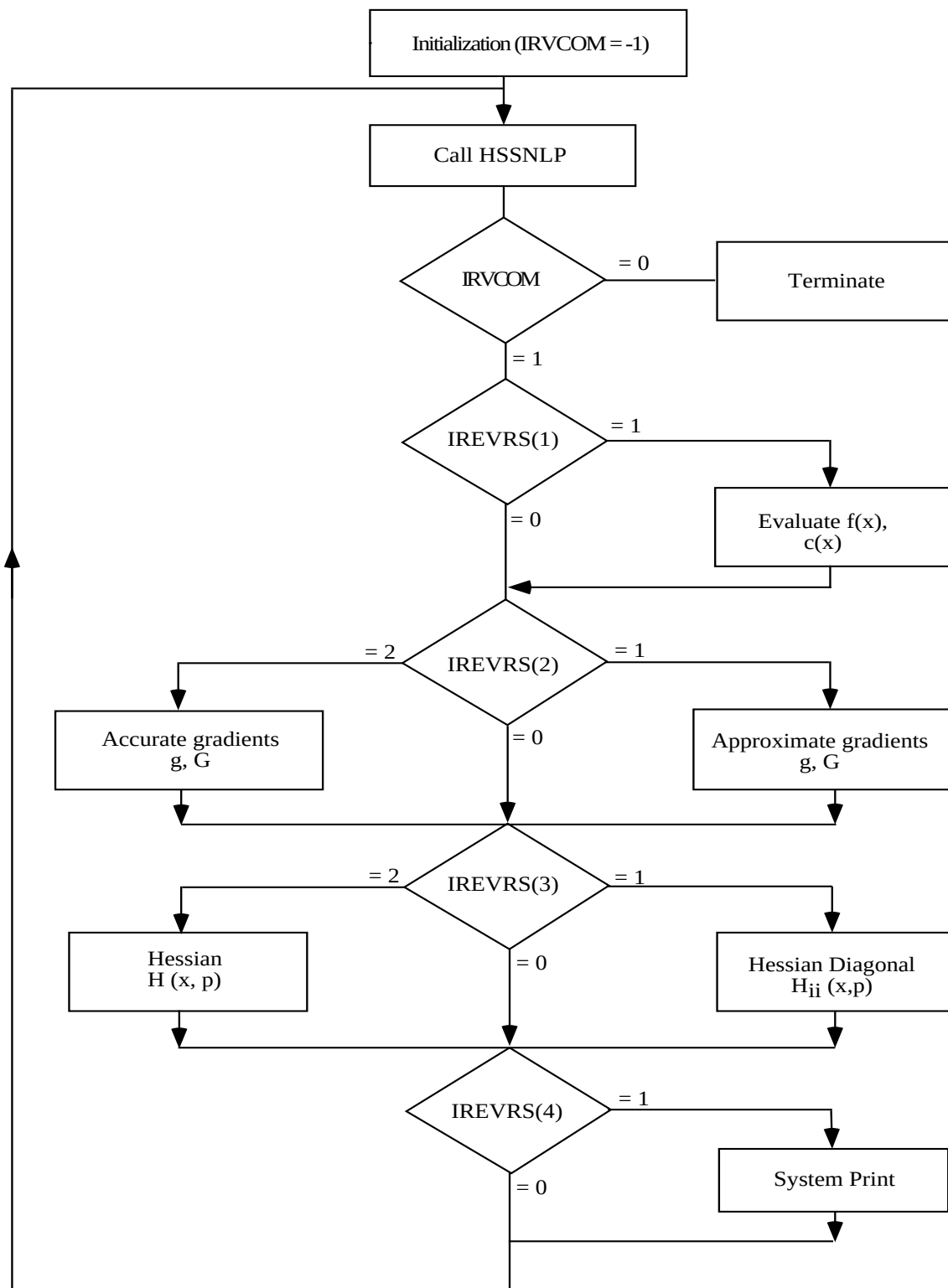


Figure 2.1: Reverse Communication Flow Chart

Chapter 3

Finite Difference Derivatives

3.1 Mathematical Background

Suppose it is necessary to evaluate the first partial derivatives of a set of M functions,

$$F_k(x_1, x_2, \dots, x_n)$$

for $k = 1, \dots, M$ with respect to a set of n variables. The *Jacobian* matrix is defined as the matrix, J , whose k^{th} row consists of the partial derivatives of function k with respect to all n variables:

$$J_{kj} = \frac{\partial F_k}{\partial x_j}$$

When used in conjunction with an optimization method there are two common definitions for the functions F_k . If we define

$$F(x) = [c_1(x), c_2(x), \dots, c_m(x), f(x)]$$

where $c_k(x)$ is a nonlinear constraint function, and $f(x)$ is the objective function, then the Jacobian matrix

$$J = \begin{bmatrix} \nabla_x^T c_1 \\ \vdots \\ \nabla_x^T c_m \\ \nabla_x^T f \end{bmatrix} = \begin{bmatrix} G \\ g^T \end{bmatrix}.$$

Notice that the first m rows contain the $m \times n$ constraint Jacobian G , and the last row contains the gradient vector g . This information is needed for the nonlinear programming algorithms described in Chapter 2. On the other hand if a least squares optimization problem is being solved, it is appropriate to define

$$F(x) = [c_1(x), c_2(x), \dots, c_m(x), r_1(x), r_2(x), \dots, r_\ell(x)]$$

where $c_k(x)$ is a nonlinear constraint function, $r_k(x)$ is a nonlinear residual function, and $M = \ell + m$. In this instance the Jacobian matrix

$$J = \begin{bmatrix} \nabla_x^T c_1 \\ \vdots \\ \nabla_x^T c_m \\ \nabla_x^T r_1 \\ \vdots \\ \nabla_x^T r_\ell \end{bmatrix} = \begin{bmatrix} G \\ R \end{bmatrix}.$$

As before the first m rows contain the $m \times n$ constraint Jacobian G , but the last ℓ rows contain the residual Jacobian R , which is required input for a least squares optimization.

Second derivative information can also be constructed using finite differencing. Define the function

$$L(x, \omega) = \sum_{i=1}^M \omega_i F_i(x)$$

which involves the n variables $x = (x_1, x_2, \dots, x_n)$ and the M multipliers $\omega = (\omega_1, \omega_2, \dots, \omega_M)$. The finite differencing software constructs the Hessian matrix of this function which is defined by

$$H = \nabla_x^2 L(x, \omega) = \sum_{i=1}^M \omega_i \nabla_x^2 F_i(x)$$

It is important to observe that the software does not explicitly save the second derivative matrices of the individual functions, i.e. $\nabla_x^2 F_i(x)$. For a general nonlinear programming application, the Hessian of the Lagrangian is readily obtained by defining the functions

$$F = (c_1, c_2, \dots, c_m, f)$$

and then setting $\omega = (-\lambda_1, -\lambda_2, \dots, -\lambda_m, 1)$. In this case $M = m + 1$, the objective function is $f(x)$, and λ_k is the Lagrange multiplier corresponding to the constraint c_k . Clearly, with these definitions the Hessian computed is

$$H_L = \nabla_x^2 f - \sum_{i=1}^m \lambda_i \nabla_x^2 c_i(x).$$

In contrast, for least squares applications it is necessary to compute the residual Hessian matrix defined by

$$V = \sum_{i=1}^{\ell} r_i \nabla_x^2 r_i(x) - \sum_{i=1}^m \lambda_i \nabla_x^2 c_i(x)$$

which involves both the nonlinear constraints $c_i(x)$ and the nonlinear residuals, $r_i(x)$. For this application, the software can be utilized by defining the functions

$$F = (c_1, c_2, \dots, c_m, r_1, r_2, \dots, r_\ell)$$

and then setting $\omega = (-\lambda_1, -\lambda_2, \dots, -\lambda_m, r_1, r_2, \dots, r_\ell)$. In this case $M = \ell + m$.

When the number of functions and variables is small, computing the finite difference approximation to the derivatives about a nominal point $\mathbf{x}$, simply involves perturbing each variable j by an amount δ_j and forming the 1 – sided finite difference quotient for the partial derivative:

$$\frac{\partial F_k}{\partial x_j} = \frac{F_k(\mathbf{x} + \delta_j \mathbf{e}_j) - F_k(\mathbf{x})}{\delta_j} + O(\delta_j)$$

or for 2 – sided finite difference derivatives:

$$\frac{\partial F_k}{\partial x_j} = \frac{F_k(\mathbf{x} + \delta_j \mathbf{e}_j) - F_k(\mathbf{x} - \delta_j \mathbf{e}_j)}{2\delta_j} + O(\delta_j^2)$$

for $k = 1, \dots, m$ and $j = 1, \dots, n$, where $\mathbf{x} = (x_1, x_2, \dots, x_n)$ and $\mathbf{e}_j$ is a unit vector in the j^{th} coordinate direction. The notation $O(r^p)$ means that the difference between the left side and the first term on the right side is proportional to r^p as $r \rightarrow 0$. For obvious reasons, the first expression is often referred to as a *forward difference* estimate and the second is called a *central difference* approximation. The trade-off between these formulae is that the second provides a more accurate approximation at the expense of an extra function evaluation per derivative.

For large problems with hundreds to thousands of variables, perturbing each variable individually becomes impractical because of the expense of evaluating the functions so many times. It is a fortunate occurrence that many large problems are also sparse, that is, the number of nonzero elements J_{kj} in the Jacobian matrix for the system is a relatively small percentage of the total number of elements in the matrix. In these cases derivatives can be computed more efficiently by grouping variables together and simultaneously perturbing all variables in the set, a technique called sparse differencing.

Definition: The Sparsity Pattern of a matrix is a list of the locations of the nonzero elements in the matrix.

One way to specify the sparsity pattern is to list the nonzero elements, a_{ij} , in a matrix by giving each location as a triple: (i, j, a_{ij}) . In general this list does not need to be ordered in any particular manner, although it could be permuted into row or column order. If the matrix has been ordered by columns (or rows), a second more storage efficient description is possible. The list now consists of only two entries: (i, a_{ij}) or (j, a_{ij}) with a separate vector pointing to the starting location of each column (row) in the list. An example of this format is given in the USAGE REMARKS for subroutine HJSFDI. Each representation can be convenient in certain situations.

In the following presentation it will be assumed that the nonzeros in the Jacobian matrix are described by a triple, where the row number is the number of the dependent function whose derivatives are being computed, the column number is the number of the independent variable, and the value is the corresponding value of the partial derivative. The Hessian matrix, $\mathbf{H}_k$, of a the function F_k is the matrix of second partial derivatives of the function with respect to all of the independent variables. Since it will be assumed that the functions are sufficiently differentiable that the mixed partial derivatives are equal, i.e.

$$\frac{\partial^2 F_k}{\partial x_i \partial x_j} = \frac{\partial^2 F_k}{\partial x_j \partial x_i} ,$$

the Hessian matrix is symmetric. Thus only the lower (upper) triangular portion of the matrix needs to be specified for the complete description. It will be convenient to express this matrix in

the second sparse form, where each element is specified as a row number and value with pointers to the beginning of each column.

Index Set Generation - HJSFDI

Definition: An Index Set is a subset of the problem variable indices such that no two variables belonging to the set affect the same dependent function. (No two columns in the matrix corresponding to variable numbers in the index set have a nonzero element in the same row.) Stated differently, an index set is a subset of the columns of the Jacobian, with the property that there is at most one nonzero element per row.

The definition of an index set allows all variables within it to be perturbed simultaneously, since there will be at most one variable in the set effecting each function. The partial derivative of the function with respect to that variable can be approximated by a difference quotient. As a simple example consider the set of 3 functions depending on 3 variables, $\mathbf{x} = (x_1, x_2, x_3)$:

$$\begin{aligned} F_1(\mathbf{x}) &= F_1(x_1) \\ F_2(\mathbf{x}) &= F_2(x_2) \\ F_3(\mathbf{x}) &= F_3(x_3) \end{aligned}$$

In this case the only index set is $I_1 = \{1, 2, 3\}$. To find all the partial derivatives we need only the single evaluation at the point, $\mathbf{x}_p = (x_1 + \delta_1, x_2 + \delta_2, x_3 + \delta_3)$. The partial derivatives may be obtained as:

$$\begin{aligned} \frac{\partial F_1}{\partial x_1} &\approx \frac{F_1(\mathbf{x}_p) - F_1(\mathbf{x})}{\delta_1} \\ \frac{\partial F_2}{\partial x_2} &\approx \frac{F_2(\mathbf{x}_p) - F_2(\mathbf{x})}{\delta_2} \\ \frac{\partial F_3}{\partial x_3} &\approx \frac{F_3(\mathbf{x}_p) - F_3(\mathbf{x})}{\delta_3} \end{aligned}$$

The example above shows that if the Jacobian matrix sparsity pattern is diagonal, then one perturbation is sufficient to generate the Jacobian matrix independent of how many variables are involved. For a general banded matrix the number of perturbations is no larger than the band width.

In general, a perturbation of index set, I_j , will mean that every variable in the set is perturbed by its corresponding perturbation size,

$$\Delta^j = \sum_{p \in I_j} \delta_p \mathbf{e}_p$$

The standard formulae for forward and central difference approximations still hold when the individual variable perturbations, $\delta_p \mathbf{e}_p$ are replaced with index set perturbations, Δ^j . Using index set perturbations to obtain the partial derivatives is called *Sparse Differencing*.

For a general sparse matrix the computation of the index sets can be done in a number of ways. The simplest method is one developed by Curtis, Powell, and Reid (CPR) (see references). It places the first column in the first index set and then sweeps over the rest of the matrix placing additional columns in the developing index set if there are no row conflicts with the columns already selected. Although the method is efficient, the drawback is that it is dependent on the ordering of the columns, and in general will not yield the minimum number of index sets required to generate the Jacobian. The CPR method is implemented in subroutine HJSFDI. A more sophisticated

method relates the problem to that of graph coloring, and is described by Coleman and More (see references).

It is relatively easy to obtain a lower bound for the number of index sets which will be required for any specific problem. One merely notes from the definition of an index set that the variables which affect any particular dependent function (row) must be in different sets to avoid a conflict. Thus a lower bound to the number of index sets is the maximum number of nonzero elements in any row.

The first observation obtained from this lower bound is that if a single function depends on all the variables, the number of index sets will equal the number of variables. In this case sparse differencing will reduce to the dense method of perturbing every variable, thereby nullifying any advantage of sparse differencing. The only recourse in this case is to see if the partial derivatives in the dense rows can be determined analytically or by some other method. Removing these rows from the problem will then allow the remaining rows to be determined by sparse differencing.

A second observation from the definition of an index set is that if a variable affects every dependent function (row), then that variable must be in its own index set to avoid row conflict with all other variables. This case is not as serious as the one above if the number of dense columns is relatively small, for the total number of index sets could still be substantially smaller than the total number of variables.

Note that the computation of the index sets is based solely on the sparsity pattern of the Jacobian matrix which must be input to subroutine HJSFDI in the row/column format for describing the location of the nonzero elements. Thus only the structure of the functional dependencies affects the index set computation. Determining the sparsity pattern for a particular problem may be a substantial task for the user of this subroutine.

The Jacobian Matrix - HDSFDJ and HDDFDJ

For dense applications subroutine HDDFDJ can be used to construct the Jacobian matrix. For sparse applications once the index sets have been determined, subroutine HDSFDJ can be used to compute the Jacobian matrix by sparse differences. To do this efficiently, a column ordering of the sparsity pattern is maintained internally. It is assumed that the perturbation sizes have been set appropriately prior to the call so that sufficiently accurate derivatives will be obtained. A central difference option is also included for the cases where accuracy is of paramount concern. The finite difference approximations to the derivatives for dense matrices can be generalized by replacing the computation with single variable perturbations by index set perturbations. Thus for each nonzero row i of a column $j \in \Gamma^k$, the forward difference approximation is

$$J_{ij} \approx \frac{1}{\delta_j} \left[F_i(\mathbf{x} + \Delta^k) - F_i(\mathbf{x}) \right]$$

and the central difference approximation is

$$J_{ij} \approx \frac{1}{2\delta_j} \left[F_i(\mathbf{x} + \Delta^k) - F_i(\mathbf{x} - \Delta^k) \right].$$

For dense applications the number of index sets is equal to the number of variables and $\Delta^k = \delta_k \mathbf{e}_k$.

To allow for maximum flexibility in the manner in which the functions are computed, subroutines HDSFDJ and HDDFDJ use a reverse communication technique. This means that the subroutine will not directly call a user subroutine (thus requiring a fixed argument list), but will instead return

to the calling program each time a new function evaluation is to be performed. After the evaluation, the finite difference routine must be called again with the new function values at the perturbed point. See the usage examples to obtain a more detailed look at how reverse communication is used.

Sparsity Pattern Check - HDSFDC

The sparsity pattern of the Jacobian defines the location of the nonzero derivatives, $J_{kj} = \frac{\partial F_k}{\partial x_j}$. The pattern can be incorrect in one of two ways:

1. A particular element J_{kj} can be included in the nonzero list even though the partial derivative is identically zero.
2. A particular element J_{kj} is not included in the nonzero list even though the partial derivative is nonzero.

The first case, namely specifying a nonzero location in the matrix when in fact there is no functional dependence, is not a serious error. Under these circumstances the index set determination will still be correct, with the computed value of the derivative being within truncation error of zero. The penalty for this error is computational efficiency. By incorrectly specifying extra elements, it is possible that more index sets will be needed than are necessary. This in turn will increase the number of function evaluations required to compute the relevant derivatives.

On the other hand, omitting a nonzero element location from the sparsity pattern may cause serious problems. In this case it is possible to obtain an incorrect partition of the variables into index sets, which will lead to the incorrect calculation of some of the partial derivatives. To illustrate this, consider the case where $F_7 = F_7(x_1, x_3)$ but the (7, 3) location is missing from the sparsity pattern. In this case, variables 1 and 3 might both be placed erroneously in the first index set if no other function in the problem depended simultaneously on these variables. Then the Jacobian computation would approximate the partial of function 7 with respect to variable 1 as:

$$\begin{aligned} \frac{\partial F_7}{\partial x_1} &\approx \frac{F_7(x_1 + \delta_1, x_3 + \delta_3) - F_7(x_1, x_3)}{\delta_1} \\ &= \frac{\partial F_7}{\partial x_1} + \frac{\partial F_7}{\partial x_3} \frac{\delta_3}{\delta_1} + O(\delta_1, \delta_3^2/\delta_1) \end{aligned}$$

In other words, the partial derivative $\frac{\partial F_7}{\partial x_3}$ would not be computed and a linear combination of the actual derivatives affecting F_7 would be mistaken for a single partial derivative. Unless the linear combination was so large that its size would catch the eye as being a potential error, the miscomputed derivatives would go unnoticed. However, if the matrix is being used by an iterative method, the error in the partial derivative will lead to a reduction in the rate of convergence of the method, and possibly to the inability to converge the problem. For these reasons, a subroutine which attempts to check for missing nonzero elements is included in the sparse difference package (HDSFDC).

In general, the brute force method of perturbing every variable independently to obtain the dense Jacobian and then inspecting for nonzero locations will be too expensive for all but the smallest problems. Instead, the assumption will be made that the number of missing elements is small, and that a consistency check of the index sets generated using the input sparsity pattern will reveal potential missing elements.

The search for missing elements uses two perturbations of each index set with different sets of coefficients. Consider the two perturbations of the first index set given by:

$$\begin{aligned}\mathbf{x}_{p1} &= \mathbf{x} + \sum_{i \in I_1} \delta_i \mathbf{e}_i \\ \mathbf{x}_{p2} &= \mathbf{x} + \sum_{i \in I_1} \alpha_i \delta_i \mathbf{e}_i\end{aligned}$$

Now suppose function F_k was specified as depending only on variable j in the first index set and form the difference,

$$[F_k(\mathbf{x}_{p2}) - F_k(\mathbf{x})] - \alpha_j[F_k(\mathbf{x}_{p1}) - F_k(\mathbf{x})] = \sum_{i \in I_1} (\alpha_i - \alpha_j) \frac{\partial F_k}{\partial x_i} \delta_i + O(\delta^2)$$

If in fact F_k depended only on element j in the index set, then the summation on the right hand side would be zero, and the difference would therefore be of second order. On the other hand, if F_k depended additionally on any other variables in the index set, the difference above would be of first order in the perturbation size. The size of the difference can therefore be used to estimate if there are elements in the sparsity pattern that are missing.

Under the assumption that at most one element is missing in row k for any particular index set, it is possible to estimate which variable in the set also affects the row by using one additional perturbation. To see this, assume that variable x_q is the only other variable in the first index set which also affects function k . Then the difference equation reduces to

$$[F_k(\mathbf{x}_{p2}) - F_k(\mathbf{x})] - \alpha_j[F_k(\mathbf{x}_{p1}) - F_k(\mathbf{x})] = (\alpha_q - \alpha_j) \frac{\partial F_k}{\partial x_q} \delta_q + O(\delta^2)$$

Let the third perturbation be given by

$$\mathbf{x}_{p3} = \mathbf{x} + \sum_{i \in I_1} \frac{1}{\alpha_i} \delta_i \mathbf{e}_i$$

Then the difference for perturbation 3 will be

$$[F_k(\mathbf{x}_{p3}) - F_k(\mathbf{x})] - \frac{1}{\alpha_j}[F_k(\mathbf{x}_{p1}) - F_k(\mathbf{x})] = \left(\frac{1}{\alpha_q} - \frac{1}{\alpha_j}\right) \frac{\partial F_k}{\partial x_q} \delta_q + O(\delta^2)$$

Now forming the ratio of the first perturbation with α_j times the second perturbation yields

$$r = -\alpha_q + O(\delta^2)$$

Thus the coefficient of the perturbation in the missing variable has been isolated, and if it is sufficiently different from the other coefficients, the missing variable can be inferred. The estimate will be incorrect if more than one element is missing in the row. The total time for the sparsity pattern check will therefore not exceed the cost of calculating three 1-sided Jacobian matrices.

The Hessian Matrix - HDSFDH and HDDFDH

Subroutine HDSFDH is provided to compute the Hessian matrix of any function, L , which can be written as a linear combination of the finite difference functions whose index sets have been computed by HJSFDI. Subroutine HDDFDH is a simplified version of HDSFDH which does not exploit sparsity. In either case,

$$L(x, \omega) = \sum_{i=1}^M \omega_i F_i(x)$$

HDSFDH and HDDFDH directly compute the Hessian of L , rather than the individual Hessians. This enables a substantial savings in storage. To directly compute the Hessian of one of the functions, F_p , the user needs only to set $\omega_p = 1$ and $\omega_k = 0, k \neq p$.

The computation is divided into two passes which have been found to be convenient for optimization applications. The first pass computes a central difference estimate of the Jacobian matrix of the functions along with the diagonal of the Hessian matrix. Working quantities are also stored on this pass which will be needed on the second pass when the off-diagonal elements of the Hessian are computed. It is important to note that the *same index sets are used for both Jacobian and Hessian* calculations. If it is not necessary to construct finite difference estimates for the Jacobian, more efficient sparse difference techniques exist which exploit the symmetry of the Hessian.

The Hessian of the function L can be formed by perturbing each index set in combination with every other index set. For a standard dense Hessian matrix, all second derivatives can be determined by pairwise perturbation of the variables according to the formula

$$\frac{\partial^2 F_k}{\partial x_i \partial x_j} \approx \frac{F_k(\mathbf{x} + \delta_i \mathbf{e}_i + \delta_j \mathbf{e}_j) + F_k(\mathbf{x}) - F_k(\mathbf{x} + \delta_i \mathbf{e}_i) - F_k(\mathbf{x} + \delta_j \mathbf{e}_j)}{\delta_i \delta_j}$$

for $i \neq j$ and,

$$\frac{\partial^2 F_k}{\partial x_i^2} \approx \frac{F_k(\mathbf{x} + \delta_i \mathbf{e}_i) + F_k(\mathbf{x} - \delta_i \mathbf{e}_i) - 2F_k(\mathbf{x})}{\delta_i^2}$$

for $i = j$ using the perturbations $+\delta_i \mathbf{e}_i$ and $-\delta_i \mathbf{e}_i$. This procedure is implemented in the dense software HDDFDH. In the sparse case (i.e. HDSFDH) these formulae also hold when the individual variable perturbations $\delta_j \mathbf{e}_j$ are replaced by full index set perturbations Δ^j .

The reason the above formula will hold when the index set containing x_i is perturbed simultaneously with the set containing x_j follows from the definition of index sets. For if the second partial $\frac{\partial^2 F_k}{\partial x_i \partial x_j}$ is not identically zero, then it must be the case that neither of the first partials of F_k with respect to x_i or x_j are identically zero. This implies that the sparsity pattern for the k^{th} row of the Jacobian contains nonzeros in columns i and j , which requires that i and j be in distinct index sets since $i \neq j$. By applying this argument to all dependent functions it is clear that when two index sets are perturbed, there can be at most one resulting second partial derivative generated for each F_k .

This relationship between nonzero locations in the Hessian matrix and those in the Jacobian matrix, J , can be used as a consistency check between the sparsity patterns of the two matrices. Consider the sparsity pattern of the normal matrix:

$$N = J^T J$$

$$N_{ij} = \sum_{k=1}^m \frac{\partial F_k}{\partial x_i} \frac{\partial F_k}{\partial x_j}$$

The matrix element N_{ij} is potentially nonzero if there is at least one function k which depends on both x_i and x_j . In general this function k will have a mixed second partial depending on these two variables and therefore the same element will occur as a nonzero in the Hessian of the Lagrangian. When HJSFDI is asked to generate the Hessian pattern, it will generate the most general pattern, that of N .

On the other hand, if the user inputs a Hessian pattern, it will be compared with N . There are then three possibilities in the comparison:

1. Input elements appearing in N will be accepted.
2. Input elements not appearing in N will be flagged as being ‘unreachable’.
3. Elements in N that are not included in the given Hessian sparsity pattern, will be flagged as ‘separable’.

Cases 2 and 3 are not necessarily indications that the Hessian sparsity pattern is incorrect. Perfectly legitimate second partial derivatives can be generated when there are elements corresponding to these cases.

If a Hessian element, H_{ij} , is flagged as unreachable, the computed value loaded at that location will always be zero. However, assuming the given Hessian pattern is correct, it could also be an indication of a Jacobian input error. This is due to the above argument showing that at least one function, F_k , will have nonzeros with respect to the two variables, i and j . Thus, both of the elements (k, i) and (k, j) should have appeared in the Jacobian pattern, and apparently did not, since the element N_{ij} was not found.

A location flagged as separable indicates that a function which was specified as depending simultaneously on two variables is really the sum of two terms, each term depending on only one of the variables. This presents a possibility for reducing the number of index sets by splitting the separable function into two parts, computing the finite difference derivatives of the parts, and then summing to get the whole.

Since the Hessian has been shown to consist of perturbing each pair of index sets, it can be generated with $n_I(n_I + 1)/2$ perturbations where n_I is the number of index sets. Although this number grows quadratically with the number of index sets, it is substantially less than the number for generating a dense Hessian, $n(n + 1)/2$ where n is the number of variables.

Sparse Matrix Print - HDSFDP

HDSFDP is a utility which has been included in the sparse difference package to print rows, columns, or the entire finite difference Jacobian and Hessian matrices. The usefulness of this subroutine lies mainly in the fact that the index set array for sparse differences contains both row and column representations of the matrices in most cases. This allows an efficient way to print these matrices by either row or column. It should be noted that if HJSFDP was called with the Jacobian only option, printing by row will require an exhaustive search in order to find the required elements for printing. It is therefore recommended that only selected rows, rather than the entire matrix, be printed in this case.

3.2 Subprograms for Finite Differences

Dense Differences

HDDFDJ: Dense Finite Difference Jacobian—Reverse Communication Format 268

HDDFDH: Finite Difference Jacobian and Hessian—Reverse Communication Format 277

Sparse Differences

HJSFDI: Generate Sparse Finite Difference Index Sets 288

HDSFDJ: Sparse Finite Difference Jacobian—Reverse Communication Format 295

HDSFDH: Sparse Finite Difference Jacobian/Hessian—Reverse Communication Format . . . 308

HDSFDC: Check Sparsity Pattern for Missing Elements—Reverse Communication Format . 320

HDSFDP: Print Out Sparse Matrix 325

HDDFDJ: Dense Finite Difference Jacobian—Reverse Communication Format**PURPOSE**

HDDFDJ computes a finite difference approximation to the dense Jacobian matrix of a set of dependent functions with respect to a set of independent variables. Forward or central differences can be computed using a fixed perturbation size.

RELATED SUBPROGRAMS**HDSFDJ**

Compute Sparse Finite Difference Jacobian—Reverse Communication Format

METHOD

The Jacobian is given by

$$J = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix}.$$

Each variable is perturbed one at a time from the given nominal values, and the corresponding function values are obtained. The finite differences can be computed using either forward (1-sided) differences:

$$\frac{\partial f_k}{\partial x_j} \approx \frac{f_k(\mathbf{x} + \vec{\delta}_j) - f_k(\mathbf{x})}{\delta_j}$$

or central (2-sided) differences:

$$\frac{\partial f_k}{\partial x_j} \approx \frac{f_k(\mathbf{x} + \vec{\delta}_j) - f_k(\mathbf{x} - \vec{\delta}_j)}{2\delta_j}$$

for $k = 1, \dots, m$ and $j = 1, \dots, n$, where $\mathbf{x} = (x_1, x_2, \dots, x_n)$ and $\vec{\delta}_j = \delta_j \hat{\mathbf{e}}_j$ with $\hat{\mathbf{e}}_j$ a unit vector in the j^{th} coordinate axis.

Note that the more accurate central difference approximations require twice as many function evaluations as the forward difference estimates.

USAGE

```

INTEGER ICC, NROW, NCOL, IFERR, ISTORE, MSGVLVL, IPU, IDTYPE, NHOLD, IER
DOUBLE PRECISION CJAC(NROW*NCOL), PRYSYZ(NCOL), XBAR(NCOL), FBAR(NROW), HOLD(NHOLD)
DOUBLE PRECISION PRTMIN
CALL HDDFDJ(ICC, NROW, NCOL, CJAC, PRYSYZ, PRTMIN, XBAR, FBAR,
1          IFERR, ISTORE, MSGVLVL, IPU, IDTYPE, HOLD, NHOLD, IER)

```

ARGUMENTS

ICC	[INPUT/OUTPUT, INTEGER] Initial/continue call code; see USAGE REMARKS for explanation. ICC is set to -1 by the user on the initialization call and must not be changed by the user subsequently. On output, ICC contains process status information and directions to the user: ICC=1 Continue flag; evaluate $\mathbf{f}(\mathbf{x})$ for the $\mathbf{x}$ variable values returned in XBAR and call HDDFDJ again with ICC = 1. ICC=0 Termination flag; check the value of IER for success/error status.
NROW	[INPUT, INTEGER] Number of rows in the finite difference Jacobian matrix; NROW ≥ 1 .
NCOL	[INPUT, INTEGER] Number of columns in the finite difference Jacobian matrix; NCOL ≥ 1 .
CJAC	[OUTPUT, DOUBLE PRECISION, ARRAY] If ICC = 0 and IER ≥ 0 , CJAC contains the finite difference approximation to the Jacobian matrix, stored as specified by the parameter ISTOR. CJAC must not be modified by the user during the Jacobian calculation process.
PRTSYZ	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Perturbation sizes, $\vec{\delta}$, for the $\mathbf{x}$ variable values. PRTSYZ(j) is the perturbation size for XBAR(j); $ \text{PRTSYZ}(j) \geq \text{PRTMIN}$. The perturbation sizes are only reduced when a function error, signalled by IFERR $\neq 0$, prevents a successful finite difference computation. PRTSYZ must be specified by the user on the initialization call and not subsequently changed by the user. A recommended initial value for PRTSYZ(j) is $10^{-5} \max[1, x(j)]$.
PRTMIN	[INPUT, DOUBLE PRECISION] Minimum absolute perturbation size for any variable x_j ; $10 * \text{HDMCON}(5) \leq \text{PRTMIN} \leq \min_{1 \leq j \leq \text{NCOL}} \text{PRTSYZ}(j) $. HDDFDJ will not reduce any $ \text{PRTSYZ}(j) $ below PRTMIN during the Jacobian computations. PRTMIN must be specified by the user on the initialization call and not subsequently changed. A recommended initial value for PRTMIN is $\sqrt{\text{HDMCON}(5)}$. Setting PRTMIN less than this value will result in a warning message.
XBAR	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Independent variable values $\mathbf{x}$. The current use of XBAR depends on the value of ICC: ICC= -1 XBAR must be initialized to the nominal point, that is, the $\mathbf{x}$ value for which the Jacobian is to be computed. ICC=1 XBAR contains a perturbed point at which a function evaluation is required. ICC=0 XBAR contains the nominal point, restored by HDDFDJ.

XBAR must be specified by the user on the initialization call and not subsequently changed by the user. See USAGE REMARKS.

FBAR

[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]

Dependent function values $\mathbf{f}(\mathbf{x})$. The current use of FBAR depends on the value of ICC (see USAGE REMARKS):

|ICC|=1 FBAR must be set by the user to the values of the function at the current value of XBAR.

ICC=0 FBAR contains the values of the function at the nominal point, restored by HDDFDJ.

IFERR

[INPUT, INTEGER]

Function error flag. This variable is used to communicate a function error that occurred when the user evaluated $\mathbf{f}(\mathbf{x})$.

IFERR=0 The evaluation of $\mathbf{f}(\mathbf{x})$ was successful.

IFERR $\neq$ 0 An error occurred in evaluating $\mathbf{f}(\mathbf{x})$.

IFERR must be 0 on the initialization call. This implies that the function must be valid at the nominal point. See USAGE REMARKS.

ISTOR

[INPUT, INTEGER]

Finite difference Jacobian storage scheme.

ISTOR=0 Finite difference Jacobian is stored as a single array of length NROW*NCOL. The value of the Jacobian in row i and column j is stored in CJAC(k) where $k = i + (j - 1)*NROW$.

ISTOR=1 Finite difference Jacobian is partitioned into two contiguous arrays. The function array FBAR is partitioned as $f = (c_1, c_2, \dots, c_m, F)$, where the constraints are stored in the first m elements, and the objective is stored as the last element. The value of the constraint Jacobian $\frac{\partial c_i}{\partial x_j}$ is stored in CJAC(k) where $k = i + (j - 1)*(NROW - 1)$. The objective gradient $\frac{\partial F}{\partial x_j}$ is stored in CJAC(k) where $k = NCOL*(NROW - 1) + j$.

MSGLVL

[INPUT, INTEGER]

Level of diagnostic print.

MSGLVL $\leq$ 0 Suppress all output.

MSGLVL=1 Print error messages and some diagnostic information about reduced perturbation sizes.

MSGLVL $\geq$ 2 Print complete diagnostic output, including additional perturbation information and the norm of the Jacobian.
Note: Matrix norms are computed for output purposes only and result in some extra computation.

IPU [INPUT, INTEGER]
 Fortran logical unit number to which diagnostic output will be written. It is assumed that this unit has been opened by the user prior to the call to HDDFDJ if MSGLVL > 0.

IDTYPE [INPUT, INTEGER]
 Derivative type; IDTYPE = 1 or 2. IDTYPE is only referenced on the initialization call.
 IDTYPE=1 Use forward finite differences.
 IDTYPE=2 Use central finite differences; this results in more accurate finite differences.

HOLD [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
 Hold array of length NHOLD. This array must not be changed by the user between successive calls to HDDFDJ when the Jacobian is being computed. After the Jacobian computation is complete (ICC = 0), this array can be used elsewhere as a work array.

NHOLD [INPUT, INTEGER]
 The length of the HOLD array. NHOLD $\geq$ (2*NROW+NCOL).

IER [OUTPUT, INTEGER]
 Success/error code. For IER < 0, ICC has been set to 0, and at least part of the CJAC array has been clobbered with HDMCON(1). See Appendix B for HDMCON.
 IER=0 Success, results computed.
 IER=1 $\text{PRTMIN} < \sqrt{\text{HDMCON}(5)}$.
 IER=-1 Invalid initial/continue call code; ICC $\neq$ -1 or 1.
 IER=-2 NROW < 1.
 IER=-3 IDTYPE $\leq$ 0 or IDTYPE > 2.
 IER=-4 NHOLD too small; NHOLD $\geq$ 2*NROW+NCOL.
 IER=-5 For ICC = -1, IFERR $\neq$ 0. There must not be a function error at the nominal point.

IER=-6	The minimum perturbation size is too small; PRTMIN < 10*HDMCON(5).
IER=-7	$\min_{1 \leq j \leq \text{NCOL}} \text{PRTSYZ}(j) < \text{PRTMIN}$.
IER=-8	NCOL < 1.
IER=-10	Incorrect use of ICC code; need to make initial call to HDDFDJ with ICC = -1. See USAGE REMARKS.
IER=-11	Smallest allowable perturbation caused a function error.

USAGE REMARKS

HDDFDJ uses reverse communication to compute the finite difference Jacobian. A user calls HDDFDJ repeatedly to formulate the finite difference values, where each call to HDDFDJ performs computations for a specific variable. Before the initial call to HDDFDJ, the nominal variable values must be stored in XBAR, and the corresponding nominal function values must be stored in FBAR. The function error flag IFERR must equal 0 at the nominal XBAR variable values. Also, the perturbation sizes to be used in perturbing the variable values must be stored in PRTSYZ. The user can then make an initial call to HDDFDJ with ICC = -1.

After each return from HDDFDJ, the user should examine the value of ICC to determine the course of action.

If ICC = 1, the user's program should evaluate the function using the $\mathbf{x}$ variable values returned in XBAR, that is, set $\text{FBAR} = \mathbf{f}(\mathbf{x})$. It then must call HDDFDJ again to continue the finite difference computations.

If ICC = 0 and IER = 0, HDDFDJ has successfully computed the finite difference Jacobian and loaded it in CJAC.

If ICC = 0 and IER < 0, HDDFDJ has detected a usage error.

Additionally, note the following:

1. The variables ICC, CJAC, PRTSYZ, PRTMIN, X, and HOLD must not be changed by the user between successive calls to HDDFDJ (when ICC = 1).
2. The variable IFERR can be used to communicate a function error that occurred when the user evaluated $\mathbf{f}(\mathbf{x})$. In this case, HDDFDJ will reduce the perturbation sizes, where possible, and request another function evaluation at new XBAR variable values.

EXAMPLE

The finite difference software described can be used to compute the Jacobian matrix $\mathbf{J}$ for a given

x. As an example, suppose $\mathbf{f}(\mathbf{x})$ is given by:

$$\begin{aligned} c_1 &= x_1^2 + x_2^2 - 25 \\ c_2 &= x_1 x_2 - 25 \\ F_1 &= \frac{1}{2}(c_1^2 + c_2^2) \end{aligned}$$

where

$$\mathbf{f} = \begin{pmatrix} c_1 \\ c_2 \\ F_1 \end{pmatrix} \quad \text{and} \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}.$$

The example illustrates how subroutine HDDFDJ can be used to construct the constraint Jacobian matrix $\mathbf{G}$ and the objective gradient $\mathbf{g}$ at the point $\mathbf{x} = (1, 2)$.

SAMPLE PROGRAM (hddfdj.cd1)¹

```

      PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE FINITE DIFFERENCE
C      SUBROUTINE HDDFDJ
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      PARAMETER (NROW = 3,NCOL = 2)
C      PARAMETER (MCON = NROW-1)
C      PARAMETER (NHOLD = 2*NROW+NCOL)
C      DIMENSION XBAR(NCOL),PRTSYZ(NCOL),FBAR(NROW),CJAC(NCOL*NROW)
C      DIMENSION TRUJAC(MCON,NCOL),TRUGRD(NCOL)
C      DIMENSION HOLD(NHOLD)
C
C-----
C
C      DEFINE NOMINAL VALUES FOR THE VARIABLES
C
C      XBAR(1) = 1.DO
C      XBAR(2) = 2.DO
C
C      COMPUTE NOMINAL FUNCTION VALUES
C
C      CALL FUNBOX(XBAR,NCOL,MCON,FBAR,IFERR)
C
C      COMPUTE THE ANALYTIC DERIVATIVES FOR COMPARISON
C
C      TRUJAC(1,1) = 2.DO*XBAR(1)
C      TRUJAC(1,2) = 2.DO*XBAR(2)

```

¹sosexample.tar

```
      TRUJAC(2,1) = XBAR(2)
      TRUJAC(2,2) = XBAR(1)
      TRUGRD(1) = FBAR(1)*TRUJAC(1,1) + FBAR(2)*TRUJAC(2,1)
      TRUGRD(2) = FBAR(1)*TRUJAC(1,2) + FBAR(2)*TRUJAC(2,2)
C
C      DEFINE PERTURBATION SIZES
C
      PRTMIN = SQRT(HDMCON(5))
      RELSIZ = 1.D-5
      DO I = 1,NCOL
         PRTSYZ(I) = RELSIZ*(1.DO + XBAR(I))
      ENDDO
C
      ICC = -1
      IPU = 6
      MSGGLVL = 1
C
C      CENTRAL DIFFERENCE ESTIMATES
C
      IDTYPE = 2
C
C      JACOBIAN AND GRADIENT AS SEPARATE ARRAYS
C
      ISTOR = 1
C
C-----
C
C      BEGIN REVERSE COMMUNICATION LOOP
C
120  CONTINUE
C
C      FINITE DIFFERENCE GRADIENTS
C
      CALL HDDFDJ(ICC,NROW,NCOL,CJAC,PRTSYZ,PRTMIN,XBAR,FBAR,
&               IFERR,ISTOR,MSGGLVL,IPU,IDTYPE,HOLD,NHOLD,IER)
C
      IF(ICC.NE.0) THEN
C
         CALL FUNBOX(XBAR,NCOL,MCON,FBAR,IFERR)
C
         GO TO 120
C
      ENDIF
C
C      END REVERSE COMMUNICATION LOOP
C
C-----
C
```

```

      IF(IER.NE.0) STOP
C
C      WRITE OUT THE JACOBIAN MATRIX
C
      WRITE(IPU,1004)
      WRITE(IPU,1003)
      DO J = 1,NCOL
        DO I = 1,MCON
          II = I + (J-1)*MCON
          WRITE(IPU,1001) I,J,CJAC(II),TRUJAC(I,J)
        ENDDO
      ENDDO
C
C      WRITE OUT THE GRADIENT VECTOR
C
      WRITE(IPU,1005)
      WRITE(IPU,1003)
      DO J = 1,NCOL
        II = MCON*NCOL + J
        WRITE(IPU,1002) J,CJAC(II),TRUGRD(J)
      ENDDO
C
1001 FORMAT(5X,'CJAC(',I1,',',I1,')',T25,2G20.12)
1002 FORMAT(5X,'GRAD(',I1,')',T25,2G20.12)
1003 FORMAT(T30,'FINITE DIFF.',T50,'ANALYTIC')
1004 FORMAT(//,T30,'JACOBIAN MATRIX',//)
1005 FORMAT(//,T30,'GRADIENT VECTOR',//)
      STOP
      END

      SUBROUTINE FUNBOX(XBAR,NCOL,MCON,QUANT,IFERR)
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
      DIMENSION XBAR(NCOL),QUANT(MCON+1)
C
      IFERR = 0
      QUANT(1) = XBAR(1)**2 + XBAR(2)**2 - 25.D0
      QUANT(2) = XBAR(1)*XBAR(2) - 25.D0
      QUANT(MCON+1) = (QUANT(1)**2 + QUANT(2)**2)/2.D0
C
      RETURN
      END

```

OUTPUT FROM SAMPLE PROGRAM

JACOBIAN MATRIX

	FINITE DIFF.	ANALYTIC
CJAC(1,1)	1.99999999992	2.00000000000
CJAC(2,1)	1.99999999992	2.00000000000
CJAC(1,2)	4.00000000003	4.00000000000
CJAC(2,2)	0.999999999962	1.00000000000

GRADIENT VECTOR

	FINITE DIFF.	ANALYTIC
GRAD(1)	-85.9999999975	-86.0000000000
GRAD(2)	-102.999999996	-103.000000000

HDDFDH: Finite Difference Jacobian and Hessian—Reverse Communication Format

PURPOSE

HDDFDH computes a finite difference approximation to the Jacobian matrix of a set of dependent functions with respect to a set of independent variables. In addition it constructs a finite difference approximation to the Hessian matrix of a linear combination of the functions.

RELATED SUBPROGRAMS

HDSFDH Compute Sparse Finite Difference Jacobian and the Hessian—Reverse Communication Format

METHOD

The Jacobian is given by

$$J = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix}.$$

Each variable is perturbed one at a time from the given nominal values, and the corresponding function values are obtained. The finite difference approximations are computed using central differences as defined by

$$\frac{\partial f_k}{\partial x_j} \approx \frac{f_k(\mathbf{x} + \vec{\delta}_j) - f_k(\mathbf{x} - \vec{\delta}_j)}{2\delta_j}$$

for $k = 1, \dots, m$ and $j = 1, \dots, n$, where $\mathbf{x} = (x_1, x_2, \dots, x_n)$ and $\vec{\delta}_j = \delta_j \hat{\mathbf{e}}_j$ with $\hat{\mathbf{e}}_j$ a unit vector in the j^{th} coordinate direction.

While the Jacobian is being computed, the diagonal of the Hessian matrix is also computed and quantities needed for the computation of off-diagonal Hessian elements are stored in the Hessian matrix. The Hessian matrix is formed from the second partial derivatives of the function $L = \sum_{k=1}^m \omega_k f_k$. The diagonal elements of the Hessian, h_{ii} , are computed using:

$$\begin{aligned} h_{ii} &= \frac{\partial^2 L}{\partial x_i^2} \\ &\approx \frac{L(\mathbf{x} + \vec{\delta}_i) + L(\mathbf{x} - \vec{\delta}_i) - 2L(\mathbf{x})}{\delta_i^2} \end{aligned}$$

where

$$L(\mathbf{x} + \vec{\delta}_i) = \sum_{k=1}^m \omega_k f_k(\mathbf{x} + \vec{\delta}_i)$$

with a similar definition for $L(\mathbf{x} - \vec{\delta}_i)$.

The number of perturbations which will be performed during this computation is $2n$, where n is the number of variables. Notice that the diagonal elements can be computed using the same evaluations needed to construct the central difference Jacobian. To complete the off-diagonal elements of the Hessian, an additional $n(n-1)/2$ perturbations are required. The off-diagonal Hessian elements are given by:

$$h_{ij} = \frac{\partial^2 L}{\partial x_i \partial x_j} \approx \frac{L(\mathbf{x} + \vec{\delta}_i + \vec{\delta}_j) + L(\mathbf{x}) - L(\mathbf{x} + \vec{\delta}_i) - L(\mathbf{x} + \vec{\delta}_j)}{\delta_i \delta_j}.$$

Notice that the Hessian matrices of the individual functions

$$\frac{\partial^2 f_k}{\partial x_i \partial x_j}$$

are not formed explicitly in order to save storage.

USAGE

```
INTEGER ICC, NROW, NCOL, IFERR, ISTER, MSGGLVL, IPU, NHOLD, IER
DOUBLE PRECISION CJAC(NROW*NCOL), HMAT((NCOL*(NCOL+1))/2), CMULT(NROW)
DOUBLE PRECISION PRTSYZ(NCOL), XBAR(NCOL), FBAR(NROW), HOLD(NHOLD)
DOUBLE PRECISION PRTMIN
CALL HDDFDH(ICC,NROW,NCOL,CJAC,HMAT,CMULT,PRTSYZ,PRTMIN,XBAR,FBAR,
1          IFERR,ISTOR,MSGGLVL,IPU,HOLD,NHOLD,IER)
```

ARGUMENTS

ICC	[INPUT/OUTPUT, INTEGER]
	Initial/continue call code; see USAGE REMARKS for explanation. ICC is set to -1 by the user on the initialization call and must not be changed by the user subsequently. On output, ICC contains process status information and directions to the user:
ICC=1	Continue flag; the Jacobian and the diagonal of the Hessian are being computed. Evaluate $\mathbf{f}(\mathbf{x})$ for the $\mathbf{x}$ variable values returned in XBAR and call HDDFDH again with ICC = 1.
ICC=-2	Continue flag; the Jacobian and the diagonal of the Hessian have been computed. Call HDDFDH again with ICC = -2 to continue the Hessian computations.
ICC=2	Continue flag; the off-diagonal Hessian elements are being computed. Evaluate $\mathbf{f}(\mathbf{x})$ for the $\mathbf{x}$ variable values returned in XBAR and call HDDFDH again with ICC = 2.
ICC=0	Termination flag; check the value of IER for success/error status.

NROW	[INPUT, INTEGER] Number of rows in the Jacobian, i.e. the number of dependent functions FBAR; $\text{NROW} \geq 1$.
NCOL	[INPUT, INTEGER] Number of columns in the Jacobian, i.e. the number of variables XBAR; $\text{NCOL} \geq 1$.
CJAC	[OUTPUT, DOUBLE PRECISION, ARRAY] If $ \text{ICC} \neq 1$ and $\text{IER} \geq 0$, CJAC contains the finite difference approximation to the Jacobian matrix, stored as specified by the parameter ISTOR. CJAC must not be modified by the user during the Jacobian calculation process.
HMAT	[OUTPUT, DOUBLE PRECISION, ARRAY] If $\text{ICC} = 0$ and $\text{IER} \geq 0$, HMAT contains the upper triangular portion of the Hessian matrix, stored as a packed array. HMAT must not be modified by the user during the Hessian calculation process.
CMULT	[INPUT, DOUBLE PRECISION, ARRAY] Array of multipliers, ω_k , used in forming the Hessian function. The Hessian matrix will be the second partial derivatives of the function $L = \sum_{k=1}^m \omega_k f_k$. At least one of the CMULT values must be nonzero. CMULT must be input by the user on the initialization call and not subsequently changed.
PRTSYZ	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Perturbation sizes, $\vec{\delta}$, for the $\mathbf{x}$ variable values. $\text{PRTSYZ}(j)$ is the perturbation size for $X(j)$; $ \text{PRTSYZ}(j) \geq \text{PRTMIN}$. The perturbation sizes are only reduced when a function error, signalled by $\text{IFERR} \neq 0$, prevents a successful Jacobian finite difference computation. PRTSYZ must be specified by the user on the initialization call and not subsequently changed by the user. A recommended initial value for $\text{PRTSYZ}(j)$ is $10^{-5} \max[1, x(j)]$.
PRTMIN	[INPUT, DOUBLE PRECISION] Minimum absolute perturbation size for any variable x_j ; $10 * \text{HDMCON}(5) \leq \text{PRTMIN} \leq \min_{1 \leq j \leq \text{NCOL}} \text{PRTSYZ}(j) $. HDDFDH will not reduce any $ \text{PRTSYZ}(j) $ below PRTMIN during the Jacobian computations. PRTMIN must be specified by the user on the initialization call and not subsequently changed. A recommended initial value for PRTMIN is $\sqrt{\text{HDMCON}(5)}$. Setting PRTMIN less than this value will result in a warning message.
XBAR	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Independent variable values $\mathbf{x}$. The current use of XBAR depends on the value of ICC: $\text{ICC} = -1$ XBAR must be initialized to the nominal point, i.e., the $\mathbf{x}$ value for which the Jacobian and Hessian are to be computed. $\text{ICC} > 0$ XBAR contains a perturbed point at which a function evaluation is required. $\text{ICC} = 0$ or -2XBAR contains the nominal point, restored by HDDFDH.

XBAR must be specified by the user on the initialization call and not subsequently changed by the user. See USAGE REMARKS.

FBAR

[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]

Dependent function values $\mathbf{f}(\mathbf{x})$. The current use of FBAR depends on the value of ICC (see USAGE REMARKS):

ICC=-1, FBAR must be set by the user to the values of the function at the
1, or 2 current value of XBAR.

ICC=0 or -2FBAR contains the values of the function at the nominal point,
restored by HDDFDH.

IFERR

[INPUT, INTEGER]

Function error flag. This variable is used to communicate a function error that occurred when the user evaluated $\mathbf{f}(\mathbf{x})$.

IFERR=0 The evaluation of $\mathbf{f}(\mathbf{x})$ was successful.

IFERR $\neq$ 0 An error occurred in evaluating $\mathbf{f}(\mathbf{x})$.

IFERR must be 0 on the initialization call. This implies that the function must be valid at the nominal point. See USAGE REMARKS.

ISTOR

[INPUT, INTEGER]

Finite difference Jacobian storage scheme.

ISTOR=0 Finite difference Jacobian is stored as a single array of length
NROW*NCOL. The value of the Jacobian in row i and column
 j is stored in CJAC(k) where $k = i + (j - 1)*\text{NROW}$.

ISTOR=1 Finite difference Jacobian is partitioned into two contiguous arrays.
The function array FBAR is partitioned as $f = (c_1, c_2, \dots, c_m, F)$,
where the constraints are stored in the first m elements, and the
objective is stored as the last element. The value of the con-
straint Jacobian $\frac{\partial c_i}{\partial x_j}$ is stored in CJAC(k) where $k = i + (j - 1)*(\text{NROW} - 1)$. The objective gradient $\frac{\partial F}{\partial x_j}$ is stored in CJAC(k)
where $k = \text{NCOL}*(\text{NROW} - 1) + j$.

MSGLVL

[INPUT, INTEGER]

Level of diagnostic print.

MSGLVL $\leq$ 0 Suppress all output.

MSGLVL=1 Error messages and some diagnostic information about reduced
perturbation sizes.

MSGLVL $\geq$ 2 Complete diagnostic output, including additional perturbation in-
formation and the norm of the Jacobian.

IPU	[INPUT, INTEGER] Fortran logical unit number to which diagnostic output will be written. It is assumed that this unit has been opened by the user prior to the call to HDDFDH if MSGVLV > 0.
HOLD	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Hold array of length NHOLD. This array must not be changed by the user between successive calls to HDDFDH when the Jacobian and Hessian matrices are being computed. After the Jacobian and Hessian computations are complete (ICC = 0), this array can be used elsewhere as a work array.
NHOLD	[INPUT, INTEGER] The length of the HOLD array. $NHOLD \geq 2*(NROW+NCOL)$.
IER	[OUTPUT, INTEGER] Success/error code. For $IER < 0$, ICC has been set to 0, and at least part of the CJAC and HMAT arrays have been clobbered with HDMCON(1). See Appendix B for HDMCON. Note, however, that for $IER = -21, -22$, and -23 , the CJAC array is not clobbered and contains the computed Jacobian matrix.
IER=0	Success, results computed.
IER=1	$PRTMIN < \sqrt{HDMCON(5)}$.
IER=4	The CMULT array is all zeros; no need to compute the Hessian matrix. If only the Jacobian matrix is needed, use HDDFDJ.
IER=-1	Invalid initial/continue call code; $ICC \neq -2, -1, 1$, or 2 .
IER=-2	$NROW < 1$.
IER=-4	NHOLD too small; the number of words needed is given by the printed error message.
IER=-5	For $ICC = -1$ or -2 , $IFERR \neq 0$. There must not be a function error at the nominal point.
IER=-6	The minimum perturbation size is too small; $PRTMIN < 10*HDMCON(5)$.
IER=-7	$\min_{1 \leq j \leq NCOL} PRTSYZ(j) < PRTMIN$.
IER=-8	$NCOL < 1$.
IER=-10	Incorrect use of ICC code; make sure that an initial call to HDDFDH with $ICC = -1$ is made prior to the Jacobian and Hessian diagonal computations ($ICC = 1$). See USAGE REMARKS.

IER=-11	Smallest allowable perturbation caused a function error.
IER=-20	Incorrect use of ICC code; make sure that a call to HDDFDH with $ICC = -2$ is made after the Jacobian and Hessian diagonal computations ($ICC = 1$) and prior to the off-diagonal Hessian computations ($ICC = 2$). See USAGE REMARKS.
IER=-21	Inconsistency in nominal variable values detected. The same nominal values must be used in XBAR for $ICC = -1$ and -2 , and the HOLD array must not be changed between successive calls to HDDFDH. In spite of this, the Jacobian matrix was successfully computed for the nominal values supplied when HDDFDH was called with $ICC = -1$.
IER=-22	Inconsistency in nominal function values detected. The same nominal values must be used in FBAR for $ICC = -1$ and -2 , and the HOLD array must not be changed between successive calls to HDDFDH. In spite of this, the Jacobian matrix was successfully computed for the nominal values supplied when HDDFDH was called with $ICC = -1$.
IER=-23	A function error occurred while computing the off-diagonal Hessian elements. The computation of the Hessian was not completed. The Jacobian matrix, on the other hand, was successfully computed and is stored in CJAC.

USAGE REMARKS

HDDFDH uses reverse communication to compute the Jacobian and Hessian matrices. A user calls HDDFDH repeatedly to formulate the Jacobian and Hessian values, where each call to HDDFDH performs computations for a particular variable. Before the initial call to HDDFDH, the nominal variable values must be stored in XBAR, and the corresponding nominal function values must be stored in FBAR. The function error flag IFERR must equal 0 at the nominal XBAR variable values. Also, the perturbation sizes used in perturbing the variable values must be stored in PRTSYZ. The user can then make an initial call to HDDFDH with $ICC = -1$.

After each return from HDDFDH, the user should examine the value of ICC to determine the course of action.

If $ICC = 1$ or 2 , the user's program should evaluate the function using the $\mathbf{x}$ variable values returned in XBAR, that is, set $FBAR = \mathbf{f}(\mathbf{x})$. It then must call HDDFDH again with the same value of ICC to continue the Jacobian and Hessian computations.

If $ICC = -2$, the Jacobian and the diagonal elements of the Hessian have been computed. To continue the Hessian computation, the user's program can simply call HDDFDH again with $ICC = -2$.

If $ICC = 0$ and $IER = 0$, HDDFDH has successfully computed the Jacobian and Hessian matrices and loaded them in CJAC and HMAT, respectively.

If $ICC = 0$ and $IER < 0$, HDDFDH has detected a usage error.

Additionally, note the following:

1. The variables ICC, CJAC, HMAT, CMULT, PRTSYZ, PRTMIN, XBAR, and HOLD, must not be changed by the user between successive calls to HDDDFDH (when ICC = 1, 2, or -2).
2. The variable IFERR can be used to communicate a function error that occurred when the user evaluated $\mathbf{f}(\mathbf{x})$. When ICC = 1 and IFERR $\neq$ 0, HDDDFDH will reduce the perturbation sizes, where possible, and request another function evaluation at new XBAR variable values. When ICC = 2 and IFERR $\neq$ 0, HDDDFDH cannot proceed with the Hessian computations and returns with IER = -23.

EXAMPLE

The finite difference software described can be used to compute the Jacobian matrix $\mathbf{J}$ for a given $\mathbf{x}$. As an example, suppose $\mathbf{f}(\mathbf{x})$ is given by:

$$\begin{aligned} c_1 &= x_1^2 + x_2^2 - 25 \\ c_2 &= x_1 x_2 - 25 \\ F_1 &= \frac{1}{2}(c_1^2 + c_2^2) \end{aligned}$$

where

$$\mathbf{f} = \begin{pmatrix} c_1 \\ c_2 \\ F_1 \end{pmatrix} \quad \text{and} \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}.$$

The example illustrates how subroutine HDDDFDH can be used to construct the constraint Jacobian matrix $\mathbf{G}$ and the objective gradient $\mathbf{g}$ and Hessian $\mathbf{H}$ at the point $\mathbf{x} = (1, 2)$ with $\omega = (-1, 2, 1)$.

SAMPLE PROGRAM (hddfdh.cd1)²

```

PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE FINITE DIFFERENCE
C      SUBROUTINE HDDDFDH
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      PARAMETER (NROW = 3,NCOL = 2)
C      PARAMETER (MCON = NROW-1,NFDHVL = ((NCOL+1)*NCOL)/2)
C      PARAMETER (NHOLD = 2*(NROW+NCOL))
C      DIMENSION XBAR(NCOL),PRTSYZ(NCOL),FBAR(NROW),CJAC(NCOL*NROW)
C      DIMENSION HMAT(NFDHVL),CMULT(NROW)
C      DIMENSION TRUJAC(MCON,NCOL),TRUGRD(NCOL)
C      DIMENSION TRUHC1(NFDHVL),TRUHC2(NFDHVL),TRUHF1(NFDHVL)
C      DIMENSION HOLD(NHOLD)

```

²sosexample.tar

```
C
C-----
C
C      DEFINE NOMINAL VALUES FOR THE VARIABLES
C
C      XBAR(1) = 1.DO
C      XBAR(2) = 2.DO
C
C      DEFINE NOMINAL VALUES FOR THE MULTIPLIERS
C
C      CMULT(1) = -1.DO
C      CMULT(2) = 2.DO
C      CMULT(3) = 1.DO
C
C      COMPUTE NOMINAL FUNCTION VALUES
C
C      CALL FUNBOX(XBAR,NCOL,MCON,FBAR,IFERR)
C
C      COMPUTE THE ANALYTIC DERIVATIVES FOR COMPARISON
C
C      TRUJAC(1,1) = 2.DO*XBAR(1)
C      TRUJAC(1,2) = 2.DO*XBAR(2)
C      TRUJAC(2,1) = XBAR(2)
C      TRUJAC(2,2) = XBAR(1)
C      TRUGRD(1) = FBAR(1)*TRUJAC(1,1) + FBAR(2)*TRUJAC(2,1)
C      TRUGRD(2) = FBAR(1)*TRUJAC(1,2) + FBAR(2)*TRUJAC(2,2)
C
C      TRUHC1(1) = 2.DO
C      TRUHC1(2) = 0.DO
C      TRUHC1(3) = 2.DO
C
C      TRUHC2(1) = 0.DO
C      TRUHC2(2) = 1.DO
C      TRUHC2(3) = 0.DO
C
C      TRUHF1(1) = TRUJAC(1,1)**2 + FBAR(1)*TRUHC1(1)
C      $          + TRUJAC(2,1)**2 + FBAR(2)*TRUHC2(1)
C      TRUHF1(2) = TRUJAC(1,1)*TRUJAC(1,2) + FBAR(1)*TRUHC1(2)
C      $          + TRUJAC(2,1)*TRUJAC(2,2) + FBAR(2)*TRUHC2(2)
C      TRUHF1(3) = TRUJAC(1,2)**2 + FBAR(1)*TRUHC1(3)
C      $          + TRUJAC(2,2)**2 + FBAR(2)*TRUHC2(3)
C
C      DEFINE PERTURBATION SIZES
C
C      PRTMIN = SQRT(HDMCON(5))
C      RELSIZ = 1.D-5
C      DO 110 I = 1,NCOL
C          PRTSYZ(I) = RELSIZ*(1.DO + XBAR(I))
```



```

110  CONTINUE
C
      IPU = 6
      MSGLVL = 1
C
      JACOBIAN AND GRADIENT AS SEPARATE ARRAYS
C
      Istor = 1
C
C-----
C
      BEGIN REVERSE COMMUNICATION LOOP
C
      ICC = -1
120  CONTINUE
C
      FINITE DIFFERENCE GRADIENTS
C
      CALL HDDFDH(ICC,NROW,NCOL,CJAC,HMAT,CMULT,PRTSYZ,PRTMIN,
&                XBAR,FBAR,IFERR,ISTOR,MSGLVL,IPU,HOLD,NHOLD,IER)
C
      IF(ICC.NE.0) THEN
C
          CALL FUNBOX(XBAR,NCOL,MCON,FBAR,IFERR)
C
          GO TO 120
C
      ENDIF
C
      END REVERSE COMMUNICATION LOOP
C
C-----
C
      IF(IER.NE.0) STOP
C
      WRITE OUT THE JACOBIAN MATRIX
C
      WRITE(IPU,1004)
      WRITE(IPU,1003)
      DO 130 J = 1,NCOL
          DO 130 I = 1,MCON
              II = I + (J-1)*MCON
              WRITE(IPU,1001) I,J,CJAC(II),TRUJAC(I,J)
130  CONTINUE
C
      WRITE OUT THE GRADIENT VECTOR
C
      WRITE(IPU,1005)

```

```
      WRITE(IPU,1003)
      DO 140 J = 1,NCOL
        II = MCON*NCOL + J
        WRITE(IPU,1002) J,CJAC(II),TRUGRD(J)
140    CONTINUE
C
C      WRITE OUT THE HESSIAN MATRIX (LOWER TRIANGULAR PORTION)
C
      WRITE(IPU,1006)
      WRITE(IPU,1003)
      H11 = CMULT(1)*TRUHC1(1) + CMULT(2)*TRUHC2(1) + CMULT(3)*TRUHF1(1)
      WRITE(IPU,1007) 'HMAT(1,1)',HMAT(1),H11
      H21 = CMULT(1)*TRUHC1(2) + CMULT(2)*TRUHC2(2) + CMULT(3)*TRUHF1(2)
      WRITE(IPU,1007) 'HMAT(2,1)',HMAT(2),H21
      H22 = CMULT(1)*TRUHC1(3) + CMULT(2)*TRUHC2(3) + CMULT(3)*TRUHF1(3)
      WRITE(IPU,1007) 'HMAT(2,2)',HMAT(3),H22
C
1001  FORMAT(5X,'CJAC(',I1,',',I1,')',T25,2G20.12)
1002  FORMAT(5X,'GRAD(',I1,')',T25,2G20.12)
1003  FORMAT(T30,'FINITE DIFF.',T50,'ANALYTIC')
1004  FORMAT(//,T30,'JACOBIAN MATRIX',//)
1005  FORMAT(//,T30,'GRADIENT VECTOR',//)
1006  FORMAT(//,T30,'HESSIAN MATRIX',//)
1007  FORMAT(5X,A9,T25,2G20.12)
      STOP
      END

      SUBROUTINE FUNBOX(XBAR,NCOL,MCON,QUANT,IFERR)
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
      DIMENSION XBAR(NCOL),QUANT(MCON+1)
C
      IFERR = 0
      QUANT(1) = XBAR(1)**2 + XBAR(2)**2 - 25.D0
      QUANT(2) = XBAR(1)*XBAR(2) - 25.D0
      QUANT(MCON+1) = (QUANT(1)**2 + QUANT(2)**2)/2.D0
C
      RETURN
      END
```

OUTPUT FROM SAMPLE PROGRAM

JACOBIAN MATRIX

	FINITE DIFF.	ANALYTIC
CJAC(1,1)	1.99999999992	2.00000000000
CJAC(2,1)	1.99999999992	2.00000000000
CJAC(1,2)	4.00000000003	4.00000000000
CJAC(2,2)	0.999999999962	1.00000000000

GRADIENT VECTOR

	FINITE DIFF.	ANALYTIC
GRAD(1)	-85.9999999975	-86.0000000000
GRAD(2)	-102.999999996	-103.000000000

HESSIAN MATRIX

	FINITE DIFF.	ANALYTIC
HMAT(1,1)	-34.0000028132	-34.0000000000
HMAT(2,1)	-10.9997699838	-11.0000000000
HMAT(2,2)	-24.9999882524	-25.0000000000

HJSFDI: Generate Sparse Finite Difference Index Sets

PURPOSE

HJSFDI defines the index sets for a sparse finite difference procedure. An array containing the ordering of the Jacobian nonzeros which is most favorable for computing finite differences is loaded for use by HDSFDJ and HDSFDH. Also, an option is provided to either check or generate a Hessian sparsity pattern based on the given Jacobian information.

RELATED SUBPROGRAMS

HDSFDJ	Compute Sparse Finite Difference Jacobian—Reverse Communication Format
HDSFDH	Compute Sparse Finite Difference Jacobian and the Hessian—Reverse Communication Format
HDSFDC	Check Sparsity Pattern for Missing Elements—Reverse Communication Format
HDSFDP	Print Out Sparse Matrix

METHOD

For a given Jacobian sparsity pattern, either the Curtis-Powell-Reid or the Coleman-Moré algorithm is used to generate the index sets. The Curtis-Powell-Reid method places the first column in the first index set and then sweeps over the rest of the matrix. Additional columns are placed in the developing index set if there are no row conflicts with the columns already selected. After sweeping through the matrix, the first index set is defined. Other index sets are similarly generated, using the remaining columns. The Coleman-Moré algorithm exploits graph coloring techniques to compute a near optimal number of index sets.

USAGE

```
INTEGER IER, IHESS, IPU, MAXHNZ, MSGVLV, NCOL, NCOLH, NFDHNZ,  
1      NFDJNZ, NINSET, NIWORK, NROW  
INTEGER INDSET(NINSET), IPCOLH(NCOLH), IROW(NFDJNZ),  
1      IROWH(MAXHNZ), IWORK(NIWORK), JCOL(NFDJNZ)  
CALL HJSFDI(IHESS,NCOL,NROW,NFDJNZ,IROW,JCOL,NFDHNZ,MAXHNZ,  
1          NCOLH,IPCOLH,IROWH,MSGVLV,IPU,  
2          INDSET,NINSET,IER)
```

ARGUMENTS

IHESS [INPUT, INTEGER]
Hessian option flag; $0 \leq \text{IHESS} \leq 2$ and $10 \leq \text{IHESS} \leq 12$.
IHESS=0 Compute only the index sets using Curtis-Powell-Reid (greedy) algorithm for the Jacobian.

IHESS=1	Compute the index sets using Curtis-Powell-Reid (greedy) algorithm for the Jacobian and check the given Hessian sparsity pattern against the expected sparsity pattern arising from the given Jacobian nonzero locations.
IHESS=2	Compute the index sets using Curtis-Powell-Reid (greedy) algorithm for the Jacobian and generate the Hessian sparsity pattern based on the given Jacobian nonzero locations.
IHESS=10	Compute only the index sets using Coleman-Moré (graph coloring) algorithm for the Jacobian.
IHESS=11	Compute the index sets using Coleman-Moré (graph coloring) algorithm for the Jacobian and check the given Hessian sparsity pattern against the expected sparsity pattern arising from the given Jacobian nonzero locations.
IHESS=12	Compute the index sets using Coleman-Moré (graph coloring) algorithm for the Jacobian and generate the Hessian sparsity pattern based on the given Jacobian nonzero locations.
NCOL	[INPUT, INTEGER] Number of independent variables. $\text{NCOL} \geq 1$.
NROW	[INPUT, INTEGER] Number of finite difference functions. $\text{NROW} \geq 1$.
NFDJNZ	[INPUT, INTEGER] Number of finite difference nonzeros in the Jacobian matrix; $1 \leq \text{NFDJNZ} \leq \text{NCOL} * \text{NROW}$.
IROW	[INPUT, INTEGER, ARRAY] Array of row numbers for each nonzero element of the Jacobian matrix. $\text{IROW}(k)$ gives the row number of the k^{th} nonzero element, for $k = 1, \dots, \text{NFDJNZ}$. Note that $1 \leq \text{IROW}(k) \leq \text{NROW}$.
JCOL	[INPUT, INTEGER, ARRAY] Array of column numbers for each nonzero element of the Jacobian matrix, in correspondence to the IROW array. $\text{JCOL}(k)$ gives the column number of the k^{th} nonzero element, for $k = 1, \dots, \text{NFDJNZ}$. Note that $1 \leq \text{JCOL}(k) \leq \text{NCOL}$.
NFDHNZ	[INPUT/OUTPUT, INTEGER] Number of finite difference Hessian nonzeros. NFDHNZ is the number of nonzeros in either the upper <u>or</u> lower triangle of the Hessian matrix, including the diagonal. Its use depends on the value of IHESS, as follows: IHESS=0,10 NFDHNZ is not referenced. IHESS=1,11 NFDHNZ is provided as input, along with the Hessian sparsity pattern.

	IHESS=2,12	NFDHZN is computed in generating the Hessian sparsity pattern; any input value is ignored.
MAXHNZ	[INPUT, INTEGER]	The length of array IROWH; MAXHNZ is only referenced when IHESS = 1, 2, 11, or 12. If IHESS = 1 or 11, $\text{MAXHNZ} \geq \text{NFDHZN} + \text{IPCOLH}(1) - 1$. Otherwise, MAXHNZ must be large enough to provide storage for all computed Hessian nonzeros. If MAXHNZ is less than the computed value of NFDHZN, an error will occur.
NCOLH	[INPUT, INTEGER]	The length of array IPCOLH; only referenced when IHESS = 1, 2, 11, or 12. $\text{NCOLH} \geq \text{NCOL} + 1$.
IPCOLH	[INPUT/OUTPUT, INTEGER, ARRAY]	Array containing pointers to locations in the IROWH array corresponding to the beginning of each column of the Hessian. $\text{IPCOLH}(j)$ gives the location in the IROWH array of the beginning of the j^{th} column, for $j = 1, \dots, \text{NCOL}$. Its use depends on the value of IHESS, as follows: IHESS=0,10 IPCOLH is not referenced. IHESS=1,11 IPCOLH is provided as part of the input Hessian sparsity pattern. See USAGE REMARKS. IHESS=2,12 IPCOLH is computed in generating the Hessian sparsity pattern; any input values are ignored.
IROWH	[INPUT/OUTPUT, INTEGER, ARRAY]	Array of row numbers of the Hessian nonzero locations. $\text{IROWH}(k)$ for $k = \text{IPCOLH}(j), \dots, \text{IPCOLH}(j + 1) - 1$ gives the row numbers in ascending order, starting with the diagonal, for the nonzero entries in column j of the Hessian lower triangular matrix. Its use depends on the value of IHESS, as follows: IHESS=0,10 IROWH is not referenced. IHESS=1,11 IROWH is provided as part of the input Hessian sparsity pattern. See USAGE REMARKS. IHESS=2,12 IROWH is computed in generating the Hessian sparsity pattern; any input values are ignored.
MSGVLV	[INPUT, INTEGER]	Level of diagnostic print. MSGVLV ≤ 0 Suppress all output. MSGVLV=1 Error messages and summary statistics. The error messages identify incorrect input array elements; the summary statistics are given for the index sets generated.

MSGLVL=2 More complete statistics. Incorrect array values are output along with the number of variables in each index set.

MSGLVL $\geq$ 3 Complete diagnostic output. All of the index set information is output along with any unreachable or separable elements found.

IPU [INPUT, INTEGER]

Fortran logical unit number to which diagnostic output will be written. It is assumed that this unit has been opened by the user prior to the call to HJSFDI if MSGLVL > 0.

INDSET [OUTPUT, INTEGER, ARRAY]

An array of length NINSET containing index set information and orderings used in the sparse finite difference routines HDSFDC, HDSFDH, HDSFDJ, and HDSFDP. INDSET must be passed, untouched by the user, to the other subroutines in this package. Those users who need detailed information about the contents of INDSET are referred to Appendix A.

NINSET [INPUT, INTEGER]

The length of the INDSET array. It depends on IHESS as follows:

IHESS=0,10 $NINSET \geq 21 + 3*(NCOL+1) + NFDJNZ$.

IHESS=1,11 $NINSET \geq 21 + (NROW+1) + 4*(NCOL+1) + 2*NFDJNZ + 2*NFDHNZ$.

IHESS=2,12 The minimum dimension of INDSET is the same as for IHESS = 1,11. However, NFDHNZ will not be known until the index sets for the Jacobian are generated and NFDHNZ is computed. If the user does not provide enough storage in INDSET, an error condition with IER = -6 is returned. A printed error message gives the required storage.

IER [OUTPUT, INTEGER]

Success/error code. For IER $\geq$ 0, the index set information has been computed. For IER < 0, part of the INDSET array has been clobbered so that it cannot be used by one or more of the routines HDSFDC, HDSFDH, HDSFDJ, and HDSFDP. More specifically, for $-20 < IER < 0$, the INDSET array cannot be used by any of the mentioned routines. If IER ≤ -20 , the INDSET array is valid for use by HDSFDC and HDSFDJ, but is valid for only the Jacobian part of the computations in HDSFDH and HDSFDP.

IER=0 Success, results computed.

IER=1 At least one row i of the Jacobian depends on all variables (IROW(k) = i for $k=k_1, k_2, \dots, k_{NCOL}$ and the corresponding JCOL(k) span the columns 1 to NCOL for $k=k_1, k_2, \dots, k_{NCOL}$).

IER=2	At least one row i of the Jacobian does not appear in the IROW values (IROW(k) $\neq i$ for $k = 1, \dots, \text{NFDJNZ}$).
IER=3	At least one column j of the Jacobian does not appear in the JCOL values (JCOL(k) $\neq j$ for $k = 1, \dots, \text{NFDJNZ}$).
IER=4	Unreachable elements were found in the Hessian sparsity pattern.
IER=5	Separable elements were found in the Hessian sparsity pattern.
IER=-1	IHESS must satisfy either $0 \leq \text{IHESS} \leq 2$ or $10 \leq \text{IHESS} \leq 12$.
IER=-2	NCOL < 1 or NROW < 1.
IER=-3	NFDJNZ < 1 or NFDJNZ > NCOL*NROW.
IER=-5	NINSET too small; the number of words needed is given by the printed error message.
IER=-11	Nonzero derivative input out of bounds (either JCOL(k) < 1 or JCOL(k) > NCOL or IROW(k) < 1 or IROW(k) > NROW for one or more k values in $1, \dots, \text{NFDJNZ}$).
IER=-12	Duplicate entries found in the Jacobian sparsity pattern (IROW(k_1) = IROW(k_2) and JCOL(k_1) = JCOL(k_2) for at least one pair of different k_1 and k_2 values in $1, \dots, \text{NFDJNZ}$). This condition is only checked when IHESS $\neq 0, 10$.
IER=-21	NCOLH $\leq$ NCOL.
IER=-22	MAXHNZ too small. If IHESS=1,11 MAXHNZ < NFDHNZ + IPCOLH(1) - 1. If IHESS=2,12 MAXHNZ < NFDHNZ.
IER=-23	Hessian column pointer(s) out of bounds (IPCOLH(j) < 1 or IPCOLH(j) > MAXHNZ + 1 for one or more j values in $1, \dots, \text{NCOL}+1$).
IER=-24	IPCOLH(NCOL+1) $\neq$ IPCOLH(1)+NFDHNZ. See USAGE REMARKS.
IER=-25	The elements of IPCOLH are not in ascending order (IPCOLH(j) $\geq$ IPCOLH($j+1$) for one or more j values in $1, \dots, \text{NCOL}$). See USAGE REMARKS.

- IER=-26 Nonzero Hessian row input out of bounds
(either IROWH(k) < 1 or IROWH(k) > NCOL for one or more k
values in IPCOLH(1),...,IPCOLH(1)+NFDHNZ-1).
- IER=-27 The Hessian row data is not ordered within each column of
data (IROWH(k) $\geq$ IROWH($k+1$) for some k in IPCOLH(j),
IPCOLH(j)+1,...,IPCOLH($j+1$)-2. Here, j is in 1,...,NCOL).
See USAGE REMARKS. The row data can be ordered by repeated
calls, one per column, to the subroutines HJSORT and HSSORT.
- IER=-28 Elements were missing from the diagonal in the Hessian sparsity
pattern. See USAGE REMARKS.

Note: For IER=1, 2, 3, 4, 5, -11, -12, -23, -25, -26, -27, and -28, a call to HJSFDI with MSGGLVL $\geq$ 1 will output the specific values for which the error condition is true. Additional diagnostic information can be obtained with a larger value of MSGGLVL.

WARNING

For accurate computation of finite difference derivatives, it is essential that all of the non-zero locations in the Jacobian be specified. If any are missed, then the index sets as specified in the INDSET array may be computed erroneously, which will cause the computed derivatives to be inaccurate. After a call to HJSFDI, the subroutine HDSFDC can be used to check the sparsity pattern for any potential missing elements.

USAGE REMARKS

For a given Jacobian sparsity pattern, HJSFDI only needs to be called once to define the index sets. The INDSET array computed by HJSFDI can be used repeatedly by HDSFDC, HDSFDH, HDSFDJ, and HDSFDP for computations based on the same sparsity pattern.

The locations of the Jacobian nonzero elements are contained in the arrays IROW and JCOL. The k^{th} nonzero element is located in row IROW(k) and column JCOL(k).

The locations of the Hessian nonzero elements are contained in the arrays IROWH and IPCOLH. When the Hessian sparsity pattern is provided as input (IHES=1,11), the IROWH and IPCOLH arrays must be provided in the following form:

1. Since the Hessian is symmetric, only the lower triangular portion of the matrix is given.
2. All of the diagonal elements of the Hessian matrix must be represented. IPCOLH(j) gives the location in IROWH of the beginning of the j^{th} column.
3. IPCOLH(NCOL+1) = IPCOLH(1)+NFDHNZ.
4. The nonzero locations must be grouped by columns in IROWH, with the groups in order (column 1, followed by column 2, ...). Therefore, IPCOLH(1) < IPCOLH(2) < ... < IPCOLH(NCOL+1) $\leq$ MAXHNZ+1.

5. Within each column of the Hessian represented in IROWH, the nonzero row locations must be ordered, starting with the diagonal element. This can be achieved by calling the subroutine HJSORT once for each column.

For example, suppose there are 10 independent variables so that the Hessian is a 10×10 symmetric matrix. The nonzero elements in the lower triangular Hessian matrix are known to be: (2,1), (10,3), (5,1), (5,2), (8,5), (6,5), and (9,7), in addition to the diagonal elements. This would give the following sparsity pattern:

$$\begin{pmatrix} * & * & & * & & & & & & \\ * & * & & * & & & & & & \\ & & * & & & & & & & * \\ & & & * & & & & & & \\ * & * & & * & * & & * & & & \\ & & & * & * & & & & & \\ & & & & * & & * & & & \\ & & & * & & * & & & & \\ & & & & * & & * & & & \\ & & * & & & & & & & * \end{pmatrix}.$$

Then, the IROWH and IPCOLH arrays would be as follows:

IPCOLH(1) = 1 →	IROWH(1) = 1	(1,1)
	IROWH(2) = 2	(2,1)
	IROWH(3) = 5	(5,1)
IPCOLH(2) = 4 →	IROWH(4) = 2	(2,2)
	IROWH(5) = 5	(5,2)
IPCOLH(3) = 6 →	IROWH(6) = 3	(3,3)
	IROWH(7) = 10	(10,3)
IPCOLH(4) = 8 →	IROWH(8) = 4	(4,4)
IPCOLH(5) = 9 →	IROWH(9) = 5	(5,5)
	IROWH(10) = 6	(6,5)
	IROWH(11) = 8	(8,5)
IPCOLH(6) = 12 →	IROWH(12) = 6	(6,6)
IPCOLH(7) = 13 →	IROWH(13) = 7	(7,7)
	IROWH(14) = 9	(9,7)
IPCOLH(8) = 15 →	IROWH(15) = 8	(8,8)
IPCOLH(9) = 16 →	IROWH(16) = 9	(9,9)
IPCOLH(10) = 17 →	IROWH(17) = 10	(10,10)
IPCOLH(11) = 18		

HDSFDJ: Sparse Finite Difference Jacobian—Reverse Communication Format

PURPOSE

HDSFDJ computes a finite difference approximation to the sparse Jacobian matrix of a set of dependent functions with respect to a set of independent variables. Forward or central differences can be computed using a fixed perturbation size.

RELATED SUBPROGRAMS

HJSFDI	Generate Sparse Finite Difference Index Sets
HDDFDJ	Compute Dense Finite Difference Jacobian—Reverse Communication Format
HDSFDH	Compute Sparse Finite Difference Jacobian and the Hessian—Reverse Communication Format
HDSFDC	Check Sparsity Pattern for Missing Elements—Reverse Communication Format
HDSFDP	Print Out Sparse Matrix

METHOD

The index sets generated by HJSFDI are used to efficiently compute the finite difference Jacobian, given by

$$J = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix}.$$

Each variable in an index set is perturbed simultaneously from the given nominal values, and the corresponding function values are obtained. The finite differences can be computed using either forward (1-sided) differences:

$$\frac{\partial f_k}{\partial x_j} \approx \frac{f_k(\mathbf{x} + \vec{\delta}_j) - f_k(\mathbf{x})}{\delta_j}$$

or central (2-sided) differences:

$$\frac{\partial f_k}{\partial x_j} \approx \frac{f_k(\mathbf{x} + \vec{\delta}_j) - f_k(\mathbf{x} - \vec{\delta}_j)}{2\delta_j}$$

for $k = 1, \dots, m$ and $j = 1, \dots, n$, where $\mathbf{x} = (x_1, x_2, \dots, x_n)$ and $\vec{\delta}_j = \delta_j \hat{\mathbf{e}}_j$ with $\hat{\mathbf{e}}_j$ a unit vector in the j^{th} coordinate axis.

Note that a second set of function evaluations is required for the more accurate central finite differences.

USAGE

```

INTEGER NCOL, NFDJNZ, NINSET, NROW
INTEGER ICC, IDTYPE, IER, IFERR, IPU, MSGLVL, NHOLD
INTEGER INDSET(NINSET), IROW(NFDJNZ), JCOL(NFDJNZ)
DOUBLE PRECISION    PRTMIN, TOLJAC
DOUBLE PRECISION    CJAC(NFDJNZ), F(NROW), HOLD(NHOLD), PRTSYZ(NCOL), X(NCOL)
CALL HDSFDJ(ICC, IROW, JCOL, CJAC, PRTSYZ, PRTMIN, X, F, IFERR,
1          MSGLVL, IPU, IDTYPE, TOLJAC, INDSET, HOLD, NHOLD, IER)

```

where NCOL, NFDJNZ, NINSET, and NROW have the same values as used in HJSFDI. NCOL is the number of columns, n , in the Jacobian matrix; NFDJNZ is the number of nonzeros in the Jacobian matrix; NINSET is a dimensional constant for INDSET; and NROW is the number of rows, m , in the Jacobian matrix. These values were stored in the INDSET array by HJSFDI and are therefore not repeated in the calling sequence to HDSFDJ.

ARGUMENTS

- ICC** [INPUT/OUTPUT, INTEGER]
Initial/continue call code; see USAGE REMARKS for explanation. ICC is set to -1 by the user on the initialization call and must not be changed by the user subsequently. On output, ICC contains process status information and directions to the user:
- ICC=1 Continue flag; evaluate $\mathbf{f}(\mathbf{x})$ for the $\mathbf{x}$ variable values returned in X and call HDSFDJ again with ICC = 1.
- ICC=0 Termination flag; check the value of IER for success/error status.
- IROW** [INPUT, INTEGER, ARRAY]
Array of row numbers for each nonzero element of the Jacobian matrix. IROW(k) gives the row number of the k^{th} nonzero element, for $k = 1, \dots, \text{NFDJNZ}$. IROW must be the same array as in HJSFDI and must not be changed by the user after the call to HJSFDI.
- JCOL** [INPUT, INTEGER, ARRAY]
Array of column numbers for each nonzero element of the Jacobian matrix, in correspondence to the IROW array. JCOL(k) gives the column number of the k^{th} nonzero element, for $k = 1, \dots, \text{NFDJNZ}$. JCOL must be the same array as in HJSFDI and must not be changed by the user after the call to HJSFDI.
- CJAC** [OUTPUT, DOUBLE PRECISION, ARRAY]
If ICC = 0 and IER ≥ 0 , CJAC contains the finite difference approximation to the Jacobian matrix, stored in correspondence to the IROW and JCOL arrays. CJAC(k) gives the finite difference value for row IROW(k) and column JCOL(k) of the Jacobian, $\partial f_{\text{IROW}(k)} / \partial x_{\text{JCOL}(k)}$. CJAC must not be modified by the user during the Jacobian iteration process.

- PRTSYZ** [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
 Perturbation sizes, $\vec{\delta}$, for the $\mathbf{x}$ variable values. $\text{PRTSYZ}(j)$ is the perturbation size for $X(j)$; $|\text{PRTSYZ}(j)| \geq \text{PRTMIN}$. The perturbation sizes are only reduced when a function error, signalled by $\text{IFERR} \neq 0$, prevents a successful finite difference computation. Furthermore, a reduction in perturbation sizes is done only for variables in the current index set. **PRTSYZ** must be specified by the user on the initialization call and not subsequently changed by the user. A recommended initial value for $\text{PRTSYZ}(j)$ is $10^{-5} \max[1, x(j)]$.
- PRTMIN** [INPUT, DOUBLE PRECISION]
 Minimum absolute perturbation size for any variable x_j ;
 $10 * \text{HDMCON}(5) \leq \text{PRTMIN} \leq \min_{1 \leq j \leq \text{NCOL}} |\text{PRTSYZ}(j)|$. **HDSFDJ** will not reduce any $|\text{PRTSYZ}(j)|$ below **PRTMIN** during the Jacobian computations. **PRTMIN** must be specified by the user on the initialization call and not subsequently changed. A recommended initial value for **PRTMIN** is $\sqrt{\text{HDMCON}(5)}$. Setting **PRTMIN** less than this value will result in a warning message.
- X** [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
 Independent variable values $\mathbf{x}$. The current use of **X** depends on the value of **ICC**:
- ICC**=-1 **X** must be initialized to the nominal point, that is, the $\mathbf{x}$ value for which the Jacobian is to be computed.
- ICC**=1 **X** contains a perturbed point at which a function evaluation is required.
- ICC**=0 **X** contains the nominal point, restored by **HDSFDJ**.
- X** must be specified by the user on the initialization call and not subsequently changed by the user. See **USAGE REMARKS**.
- F** [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
 Dependent function values $\mathbf{f}(\mathbf{x})$. The current use of **F** depends on the value of **ICC** (see **USAGE REMARKS**):
- $|\text{ICC}|=1$ **F** must be set by the user to the values of the function at the current value of **X**.
- ICC**=0 **F** contains the values of the function at the nominal point, restored by **HDSFDJ**.
- IFERR** [INPUT, INTEGER]
 Function error flag. This variable is used to communicate a function error that occurred when the user evaluated $\mathbf{f}(\mathbf{x})$.
- IFERR**=0 The evaluation of $\mathbf{f}(\mathbf{x})$ was successful.
- IFERR**≠0 An error occurred in evaluating $\mathbf{f}(\mathbf{x})$.
- IFERR** must be 0 on the initialization call. This implies that the function must be valid at the nominal point. See **USAGE REMARKS**.

MSGVLV	[INPUT, INTEGER] Level of diagnostic print. MSGVLV $\leq$ 0 Suppress all output. MSGVLV=1 Print error messages and some diagnostic information about reduced perturbation sizes. MSGVLV $\geq$ 2 Print complete diagnostic output, including additional perturbation information and the norm of the Jacobian. Note: Matrix norms are computed for output purposes only and result in some extra computation.
IPU	[INPUT, INTEGER] Fortran logical unit number to which diagnostic output will be written. It is assumed that this unit has been opened by the user prior to the call to HDSFDJ if MSGVLV > 0.
IDTYPE	[INPUT, INTEGER] Derivative type; IDTYPE = 1 or 2. IDTYPE is only referenced on the initialization call. IDTYPE=1 Use forward finite differences. IDTYPE=2 Use central finite differences; this results in more accurate finite differences.
TOLJAC	[INPUT, DOUBLE PRECISION] Tolerance for row and column norms; TOLJAC $\geq$ 0. If MSGVLV $\geq$ 2, any rows for which $\max_{1 \leq j \leq n} \frac{\partial f_k}{\partial x_j} \leq$ TOLJAC and any columns for which $\max_{1 \leq k \leq m} \frac{\partial f_k}{\partial x_j} \leq$ TOLJAC will be identified in the output file. If MSGVLV < 2, TOLJAC is not used by HDSFDJ.
INDSET	[INPUT/OUTPUT, INTEGER, ARRAY] An array containing the index set information generated by HJSFDI. This array must not be changed by the user between the call to HJSFDI and the initial call to HDSFDJ and also must not be changed by the user between successive calls to HDSFDJ.
HOLD	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Hold array of length NHOLD. This array must not be changed by the user between successive calls to HDSFDJ when the Jacobian is being computed. After the Jacobian computation is complete (ICC = 0), this array can be used elsewhere as a work array.
NHOLD	[INPUT, INTEGER] The length of the HOLD array. NHOLD $\geq$ (2*NROW+NCOL), where NROW and NCOL are the same values used in HJSFDI.
IER	[OUTPUT, INTEGER] Success/error code. For IER < 0, ICC has been set to 0, and at least part of the

CJAC array has been clobbered with HDMCON(1). See Appendix B for HDMCON.

IER=0	Success, results computed.
IER=1	$\text{PRTMIN} < \sqrt{\text{HDMCON}(5)}$.
IER=2	$\text{MSGLVL} \geq 2$ and $\text{TOLJAC} < 0$. No matrix norms were computed.
IER=3	$\text{MSGLVL} \geq 2$ and at least one row or column norm is $\leq \text{TOLJAC}$.
IER=-1	Invalid initial/continue call code; $\text{ICC} \neq -1$ or 1 .
IER=-2	Invalid INDSET array—the INDSET array must not be changed between the call to HJSFDI and the initial call to HDSFDJ and also must not be changed between successive calls to HDSFDJ.
IER=-3	$\text{IDTYPE} \leq 0$ or $\text{IDTYPE} > 2$.
IER=-4	NHOLD too small; the number of words needed is given by the printed error message.
IER=-5	For $\text{ICC} = -1$, $\text{IFERR} \neq 0$. There must not be a function error at the nominal point.
IER=-6	The minimum perturbation size is too small; $\text{PRTMIN} < 10 * \text{HDMCON}(5)$.
IER=-7	$\min_{1 \leq j \leq \text{NCOL}} \text{PRTSYZ}(j) < \text{PRTMIN}$.
IER=-10	Incorrect use of ICC code; need to make initial call to HDSFDJ with $\text{ICC} = -1$. See USAGE REMARKS.
IER=-11	Smallest allowable perturbation caused a function error.

USAGE REMARKS

HDSFDJ uses reverse communication to compute the finite difference Jacobian. A user calls HDSFDJ repeatedly to formulate the finite difference values, where each call to HDSFDJ performs computations based on a particular index set. Before the initial call to HDSFDJ, the nominal variable values must be stored in X, and the corresponding nominal function values must be stored in F. The function error flag IFERR must equal 0 at the nominal X variable values. Also, the perturbation sizes to be used in perturbing the variable values must be stored in PRTSYZ. The user can then make an initial call to HDSFDJ with $\text{ICC} = -1$.

After each return from HDSFDJ, the user should examine the value of ICC to determine the course of action.

If $\text{ICC} = 1$, the user's program should evaluate the function using the $\mathbf{x}$ variable values returned in X, that is, set $\mathbf{F} = \mathbf{f}(\mathbf{x})$. It then must call HDSFDJ again to continue the finite difference computations.

If $\text{ICC} = 0$ and $\text{IER} = 0$, HDSFDJ has successfully computed the finite difference Jacobian and loaded it in CJAC.

If $\text{ICC} = 0$ and $\text{IER} < 0$, HDSFDJ has detected a usage error.

Additionally, note the following:

1. The IROW and JCOL arrays must be the same arrays as used in HJSFDI.
2. The variables ICC, IROW, JCOL, CJAC, PRTSYZ, PRTMIN, X, INDSET, and HOLD must not be changed by the user between successive calls to HDSFDJ (when $\text{ICC} = 1$).
3. The variable IFERR can be used to communicate a function error that occurred when the user evaluated $\mathbf{f}(\mathbf{x})$. In this case, HDSFDJ will reduce the perturbation sizes, where possible, and request another function evaluation at new X variable values.

EXAMPLE

The sparse finite difference software described can be used to compute the Jacobian matrix $\mathbf{J}_k$ for a given $\mathbf{x}^{(k)}$. As an example, suppose $\mathbf{f}(\mathbf{x})$ is given by:

$$\begin{aligned} f_1 &= x_1^2 + 4x_2 \\ f_2 &= 2x_1x_3 + 1 \\ f_3 &= x_4 + x_5 \\ f_4 &= \frac{2}{x_2} + x_4 \\ f_5 &= x_1^2 - x_5^2 \end{aligned}$$

where

$$\mathbf{f} = \begin{pmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \end{pmatrix} \quad \text{and} \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix}.$$

In order to use HDSFDJ to compute the Jacobian matrix, the sparsity pattern of the Jacobian, i.e., its nonzero locations, must be identified. From the definition of $\mathbf{f}(\mathbf{x})$ in (3.2), the location of nonzero partial derivatives in the Jacobian results in the following sparsity pattern:

$$\begin{pmatrix} * & * & & & \\ * & & * & & \\ & & & * & * \\ & * & & * & \\ * & & & & * \end{pmatrix}.$$

The example program given below illustrates how the Jacobian matrix $\mathbf{J}$ would be computed as one step of solving $\mathbf{f}(\mathbf{x}) = 0$ for $\mathbf{x}$, with $\mathbf{f}$ as defined in (3.2). The Jacobian is computed for three different values of $\mathbf{x}$, the third of which happens to be a solution to $\mathbf{f}(\mathbf{x}) = 0$.

After initialization, the subroutine HJSFDI is called to generate the index sets for the problem. HJSFDI uses the sparsity pattern given in (3.2) and loads the index set information into the INDSET array. A print level of 3 is used to display information about the computed index sets. Note that no output would be generated if a print level of 0 was used. After the index sets are generated, the subroutine HDSFDC is called to check for correctness of the Jacobian sparsity pattern.

The program then proceeds to compute the Jacobian for three different values of $\mathbf{x}$. The Jacobian is computed by calling the subroutine HDSFDJ initially with an ICC value of -1 , and then repeatedly until an ICC value of 0 is returned. The subroutine HDSFDP is called to print out the Jacobian matrix.

Also, note on the output that the index sets were computed to be $\{1, 4\}$ and $\{2, 3, 5\}$, as one would expect from looking at the sparsity pattern in (3.2). Here, columns 1 and 4, representing variables 1 and 4, do not have overlapping row elements. Therefore, variables 1 and 4 can be perturbed simultaneously. The same is true for the index set $\{2, 3, 5\}$.

SAMPLE PROGRAM (lr71ch4.cd1)³

PROGRAM SAMPLE

```

INTEGER  IDTYPE, IHESS, IOPT, IPU, MSGVLV, NCOL, NFDJNZ,
&        NHOLD, NINSET, NIWORK, NROW, NVEC
PARAMETER (IDTYPE=2, IHESS=0, IPU=6, MSGVLV=3, IOPT=1, NVEC=0,
&          NCOL=5, NROW=5, NFDJNZ=10, NHOLD=4*NROW+NCOL,
&          NIWORK=NCOL+NCOL, NINSET=21+3*(NCOL+1)+NFDJNZ)
DOUBLE PRECISION PRINIT, PRTMIN, TOLJAC
PARAMETER (PRINIT=1.0D-3, PRTMIN=5.0D-05, TOLJAC=1.0D-03)

INTEGER  ICC, IER, IFERR, ITER, J, JMISS, MAXHNZ, NCOLH, NFDHNZ
INTEGER  IPCOLH(1), IROW(NFDJNZ), IROWH(1), INDSET(NINSET),
&        IVEC(1), IWORK(NIWORK), JCOL(NFDJNZ)
DOUBLE PRECISION CJAC(NFDJNZ), F(NROW), HOLD(NHOLD), PRTSYZ(NCOL),
&        X(NCOL), X1(NCOL), X2(NCOL), X3(NCOL)
CHARACTER(LEN=30)  TITLE

DATA IROW /1, 1, 2, 2, 3, 3, 4, 4, 5, 5/
DATA JCOL /1, 2, 1, 3, 4, 5, 2, 4, 1, 5/
DATA X1   /1.0, -1.0, -1.0, 1.0, -1.0/
DATA X2   /2.3, -0.9, -0.1, 2.3, -2.3/
DATA X3   /2.0, -1.0, -0.25, 2.0, -2.0/
DATA TITLE /'JACOBIAN MATRIX BY COLUMNS' /

```

³sosexample.tar

```
C
C          Set the perturbation sizes.
C
PRTSYZ(1:NCOL) = PRINIT
C
C          Generate index sets; note that variables dealing
C          with the Hessian are not used.
C
WRITE (IPU,9000)
CALL HJSFDI (IHESS,NCOL,NROW,NFDJNZ,IROW,JCOL,NFDHNZ,MAXHNZ,
&           NCOLH,IPCOLH,IROWH,MSGVLV,IPU,INDSET,NINSET,IER)
C
C          Check the Jacobian sparsity pattern for correctness.
C
WRITE (IPU,9010)
X(1:NCOL) = X1(1:NCOL)

CALL FUN (X,F,IFERR)
ICC = -1

250 CONTINUE
CALL HDSFDC (ICC,IROW,JCOL,PRTSYZ,PRTMIN,X,F,IFERR,
&           MSGVLV,IPU,INDSET,HOLD,NHOLD,JMISS,IER)
IF (ICC.NE.0) THEN
    CALL FUN (X,F,IFERR)
    GO TO 250
ENDIF

IF (JMISS.NE.0) THEN
    WRITE(IPU,*)
&    ' >>>>ERROR: Jacobian sparsity pattern check failed.'
    STOP
ENDIF

C
C          Compute the Jacobian for the three X values.
C
ITER = 1
WRITE (IPU,9020)
300 CONTINUE
WRITE (IPU,9030) ITER
IF (ITER.EQ.1) THEN
    X(1:NCOL) = X1(1:NCOL)

ELSEIF (ITER.EQ.2) THEN
    X(1:NCOL) = X2(1:NCOL)

ELSE
    X(1:NCOL) = X3(1:NCOL)
```

```

ENDIF

CALL FUN (X,F,IFERR)
ICC = -1

500 CONTINUE
CALL HDSFDJ (ICC,IROW,JCOL,CJAC,PRTSYZ,PRTMIN,X,F,IFERR,
&           MSGLVL,IPU,IDTYPE,TOLJAC,INDSET,HOLD,NHOLD,IER)

IF (ICC.NE.0) THEN
  CALL FUN (X,F,IFERR)
  GO TO 500
ENDIF

IF (IER.EQ.0) THEN
C
C           Print out the Jacobian for this X value; note that
C           the IVEC vector is not used since the whole matrix
C           is being printed out.
C
  CALL HDSFDP (IOPT,NVEC,IVEC,IROW,JCOL,CJAC,TITLE,IPU,
&           INDSET,IER)
  ELSE
    WRITE(IPU,*) ' >>>>ERROR: Jacobian computation failed.'
    STOP
  ENDIF

  IF (ITER.LT.3) THEN
    ITER = ITER + 1
    GO TO 300
  ENDIF

  WRITE(IPU,*) '* NORMAL TERMINATION *'
  STOP
9000 FORMAT (//,' ==> STEP 1: GENERATE INDEX SETS',/)
9010 FORMAT (//,' ==> STEP 2: CHECK JACOBIAN SPARSITY PATTERN',/)
9020 FORMAT (//,' ==> STEP 3: COMPUTE JACOBIAN FOR X VALUES',/)
9030 FORMAT (//,' ==> ITER = ',I2,/)
END

SUBROUTINE FUN (X,F,IFERR)

INTEGER   NCOL, NROW
DOUBLE PRECISION ZERO
PARAMETER (NCOL=5,NROW=5,ZERO=0.0D0)
INTEGER   IFERR
DOUBLE PRECISION X(NCOL), F(NROW)

```

```
IF (X(2).EQ.ZERO) THEN
  IFERR = 1
  RETURN
ENDIF

IFERR = 0
F(1) = X(1)*X(1) + 4.0D0*X(2)
F(2) = 2.0D0*X(1)*X(3) + 1.0D0
F(3) = X(4) + X(5)
F(4) = 2.0D0/X(2) + X(4)
F(5) = X(1)*X(1) - X(5)*X(5)

RETURN
END
```

OUTPUT FROM SAMPLE PROGRAM

====> STEP 1: GENERATE INDEX SETS

CHECKING JACOBIAN SPARSITY PATTERN

.....NO INPUTS OUT OF RANGE

SUMMARY STATISTICS:

MATRIX	NROWS	NCOLS	NFDNZ	%NZ
-----	-----	-----	-----	-----
JACOBIAN	5	5	10	40.0

NUMBER OF INDEX SETS =	2	LOWER BOUND =	2
NUMBER OF FUNCTIONS AT L.B. =	5	1ST SUCH FUNCTION =	1

INDEX SET #	COUNT OF VARIABLES IN EACH INDEX SET
-----	-----
1 TO 2	2 3

INDEX SET 1 : 1 4

```

FE PER JACOBIAN (2-SIDED) =      4
FE PER JACOBIAN/HESSIAN   =      5

SPARSE DIFFERENCE SPEEDUP PERCENTAGE =    60.0

```

INPUT SPARSITY PATTERN CHECKS O.K.

====> ITER = 1

MATRIX ELEMENT NORM = 4.000000000

JACOBIAN MATRIX BY COLUMNS

COL 5 - - - - -
 3, 1.000 5, 4.000

HDSFDH: Sparse Finite Difference Jacobian/Hessian—Reverse Communication Format**PURPOSE**

HDSFDH computes a finite difference approximation to the sparse Jacobian matrix and the Hessian matrix of a set of dependent functions with respect to a set of independent variables.

RELATED SUBPROGRAMS

HJSFDI	Generate Sparse Finite Difference Index Sets
HDSFDJ	Compute Sparse Finite Difference Jacobian—Reverse Communication Format
HDDFDH	Compute Dense Finite Difference Jacobian and the Hessian—Reverse Communication Format
HDSFDC	Check Sparsity Pattern for Missing Elements—Reverse Communication Format
HDSFDP	Print Out Sparse Matrix

METHOD

The index sets generated by HJSFDI are used to efficiently compute the finite difference Jacobian, given by

$$J = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix}.$$

Each variable in an index set is perturbed simultaneously from the given nominal values, and the corresponding function values are obtained. The finite differences are computed using central differences as defined by

$$\frac{\partial f_k}{\partial x_j} \approx \frac{f_k(\mathbf{x} + \vec{\delta}_j) - f_k(\mathbf{x} - \vec{\delta}_j)}{2\delta_j}$$

for $k = 1, \dots, m$ and $j = 1, \dots, n$, where $\mathbf{x} = (x_1, x_2, \dots, x_n)$ and $\vec{\delta}_j = \delta_j \hat{\mathbf{e}}_j$ with $\hat{\mathbf{e}}_j$ a unit vector in the j^{th} coordinate axis.

While the Jacobian is being computed, the diagonal of the Hessian matrix is also being computed and quantities needed for the computation of off-diagonal Hessian elements are being stored in the Hessian matrix. The Hessian matrix is formed from the second partial derivatives of the function $L = \sum_{k=1}^m \lambda_k f_k$. The diagonal elements of the Hessian, h_{ii} , are computed using:

$$h_{ii} = \frac{\partial^2 L}{\partial x_i^2} = \sum_{k=1}^m \lambda_k \frac{\partial^2 f_k}{\partial x_i^2},$$

where

$$\frac{\partial^2 f_k}{\partial x_i^2} \approx \frac{f_k(\mathbf{x} + \vec{\delta}_i) + f_k(\mathbf{x} - \vec{\delta}_i) - 2f_k(\mathbf{x})}{\delta_i^2}.$$

The number of perturbations which will be performed during this computation is $2n_I$, where n_I is the number of index sets. To complete the Hessian, an additional $n_I(n_I - 1)/2$ perturbations will be required. The off-diagonal Hessian elements are similarly given by:

$$h_{ij} = \frac{\partial^2 L}{\partial x_i \partial x_j} = \sum_{k=1}^m \lambda_k \frac{\partial^2 f_k}{\partial x_i \partial x_j},$$

and are determined by pairwise perturbation of variables via:

$$\frac{\partial^2 f_k}{\partial x_i \partial x_j} \approx \frac{f_k(\mathbf{x} + \vec{\delta}_i + \vec{\delta}_j) + f_k(\mathbf{x}) - f_k(\mathbf{x} + \vec{\delta}_i) - f_k(\mathbf{x} + \vec{\delta}_j)}{\delta_i \delta_j}.$$

USAGE

```

INTEGER MAXHNZ, NCOL, NCOLH, NFDJNZ, NINSET, NROW
INTEGER ICC, IER, IFERR, IPU, MSGLVL, NHOLD, NIHOLD
INTEGER IHOLD(NIHOLD), INDSET(NINSET), IPCOLH(NCOLH), IROW(NFDJNZ),
1      IROWH(MAXHNZ), JCOL(NFDJNZ)
DOUBLE PRECISION      PRTMIN, TOLJAC
DOUBLE PRECISION      CJAC(NFDJNZ), CMULT(NROW), F(NROW), HMAT(MAXHNZ),
1      HOLD(NHOLD), PRTSYZ(NCOL), X(NCOL)
CALL HDSFDH(ICC, IROW, JCOL, CJAC, IPCOLH, IROWH, HMAT, CMULT,
1      PRTSYZ, PRTMIN, X, F, IFERR, MSGLVL, IPU, TOLJAC, INDSET,
2      HOLD, NHOLD, IHOLD, NIHOLD, IER)

```

where MAXHNZ, NCOL, NCOLH, NFDJNZ, NINSET, and NROW have the same values as used in HJSFDI. MAXHNZ is a dimensional constant for IROWH; NCOL is the number of columns, n , in the Jacobian matrix; NCOLH is a dimensional constant for IPCOLH; NFDJNZ is the number of nonzeros in the Jacobian matrix; NINSET is a dimensional constant for INDSET; and NROW is the number of rows, m , in the Jacobian matrix. These values were stored in the INDSET array by HJSFDI and are therefore not repeated in the calling sequence to HDSFDH.

ARGUMENTS

ICC [INPUT/OUTPUT, INTEGER]
Initial/continue call code; see USAGE REMARKS for explanation. ICC is set to -1 by the user on the initialization call and must not be changed by the user subsequently. On output, ICC contains process status information and directions to the user:

ICC=1 Continue flag; the Jacobian and the diagonal of the Hessian are being computed. Evaluate $\mathbf{f}(\mathbf{x})$ for the $\mathbf{x}$ variable values returned in X and call HDSFDH again with ICC = 1.

ICC=-2	Continue flag; the Jacobian and the diagonal of the Hessian have been computed. Call HDSFDH again with ICC = -2 to continue the Hessian computations.
ICC=2	Continue flag; the off-diagonal Hessian elements are being computed. Evaluate $\mathbf{f}(\mathbf{x})$ for the $\mathbf{x}$ variable values returned in X and call HDSFDH again with ICC = 2.
ICC=0	Termination flag; check the value of IER for success/error status.
IROW	[INPUT, INTEGER, ARRAY] Array of row numbers for each nonzero element of the Jacobian matrix. IROW(k) gives the row number of the k^{th} nonzero element, for $k = 1, \dots, \text{NFDJNZ}$. IROW must be the same array as in HJSFDI and must not be changed by the user after the call to HJSFDI.
JCOL	[INPUT, INTEGER, ARRAY] Array of column numbers for each nonzero element of the Jacobian matrix, in correspondence to the IROW array. JCOL(k) gives the column number of the k^{th} nonzero element, for $k = 1, \dots, \text{NFDJNZ}$. JCOL must be the same array as in HJSFDI and must not be changed by the user after the call to HJSFDI.
CJAC	[OUTPUT, DOUBLE PRECISION, ARRAY] If $ \text{ICC} \neq 1$ and $\text{IER} \geq 0$, CJAC contains the finite difference approximation to the Jacobian matrix, stored in correspondence to the IROW and JCOL arrays. CJAC(k) gives the finite difference value for row IROW(k) and column JCOL(k) of the Jacobian, $\partial f_{\text{IROW}(k)} / \partial x_{\text{JCOL}(k)}$. CJAC must not be modified by the user during the Jacobian iteration process.
IPCOLH	[INPUT, INTEGER, ARRAY] Array containing pointers to locations in the IROWH array corresponding to the beginning of each column of the Hessian. IPCOLH(j) gives the location in the IROWH array of the beginning of the j^{th} column, for $j = 1, \dots, \text{NCOL}$. IPCOLH must be the same array as in HJSFDI and must not be changed by the user after the call to HJSFDI.
IROWH	[INPUT, INTEGER, ARRAY] Array of row numbers of the Hessian nonzero locations. IROWH(k) for $k = \text{IPCOLH}(j), \dots, \text{IPCOLH}(j+1)-1$ gives the row numbers in ascending order, starting with the diagonal, for the nonzero entries in column j of the Hessian lower triangular matrix. IROWH must be the same array as in HJSFDI and must not be changed by the user after the call to HJSFDI.
HMAT	[OUTPUT, DOUBLE PRECISION, ARRAY] If ICC = 0 and $\text{IER} \geq 0$, HMAT contains the Hessian values, stored in correspondence to the IROWH array and IPCOLH pointer array. HMAT(k) gives the Hessian value for row IROWH(k) and column j of the Hessian matrix, where j is the largest index in $1, \dots, \text{NCOL}$ with $\text{IPCOLH}(j) \leq k$. HMAT must not be modified by the user during the Hessian iteration process.

CMULT	[INPUT, DOUBLE PRECISION, ARRAY] Array of multipliers, λ_k, used in forming the Hessian function. The Hessian matrix will be the second partial derivatives of the function $L = \sum_{k=1}^m \lambda_k f_k$. At least one of the CMULT values must be nonzero. CMULT must be input by the user on the initialization call and not subsequently changed.
PRTSYZ	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Perturbation sizes, $\tilde{\delta}$, for the $\mathbf{x}$ variable values. PRTSYZ(j) is the perturbation size for X(j); $\text{PRTSYZ}(j) \geq \text{PRTMIN}$. The perturbation sizes are only reduced when a function error, signalled by IFERR $\neq 0$, prevents a successful Jacobian finite difference computation. Furthermore, a reduction in perturbation sizes is done only for variables in the current index set. PRTSYZ must be specified by the user on the initialization call and not subsequently changed by the user. A recommended initial value for PRTSYZ(j) is $10^{-5} \max[1, x(j)]$.
PRTMIN	[INPUT, DOUBLE PRECISION] Minimum absolute perturbation size for any variable x_j; $10 * \text{HDMCON}(5) \leq \text{PRTMIN} \leq \min_{1 \leq j \leq \text{NCOL}} \text{PRTSYZ}(j)$. HDSFDH will not reduce any $\text{PRTSYZ}(j)$ below PRTMIN during the Jacobian computations. PRTMIN must be specified by the user on the initialization call and not subsequently changed. A recommended initial value for PRTMIN is $\sqrt{\text{HDMCON}(5)}$. Setting PRTMIN less than this value will result in a warning message.
X	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Independent variable values $\mathbf{x}$. The current use of X depends on the value of ICC: ICC=-1 X must be initialized to the nominal point, i.e., the $\mathbf{x}$ value for which the Jacobian and Hessian are to be computed. ICC>0 X contains a perturbed point at which a function evaluation is required. ICC=0 or -2X contains the nominal point, restored by HDSFDH. X must be specified by the user on the initialization call and not subsequently changed by the user. See USAGE REMARKS.
F	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Dependent function values $\mathbf{f}(\mathbf{x})$. The current use of F depends on the value of ICC (see USAGE REMARKS): ICC=-1, F must be set by the user to the values of the function at the 1, or 2 current value of X. ICC=0 or -2F contains the values of the function at the nominal point, restored by HDSFDH.
IFERR	[INPUT, INTEGER] Function error flag. This variable is used to communicate a function error that occurred when the user evaluated $\mathbf{f}(\mathbf{x})$.

IFERR=0 The evaluation of $\mathbf{f}(\mathbf{x})$ was successful.

IFERR $\neq$ 0 An error occurred in evaluating $\mathbf{f}(\mathbf{x})$.

IFERR must be 0 on the initialization call. This implies that the function must be valid at the nominal point. See USAGE REMARKS.

MSGLVL [INPUT, INTEGER]
Level of diagnostic print.

MSGLVL $\leq$ 0 Suppress all output.

MSGLVL=1 Error messages and some diagnostic information about reduced perturbation sizes.

MSGLVL $\geq$ 2 Complete diagnostic output, including additional perturbation information and the norm of the Jacobian.

Note: Jacobian matrix norms are computed for output purposes only and result in some extra computation.

IPU [INPUT, INTEGER]

Fortran logical unit number to which diagnostic output will be written. It is assumed that this unit has been opened by the user prior to the call to HDSFDH if MSGLVL > 0.

TOLJAC [INPUT, DOUBLE PRECISION]

Comparison tolerance for Jacobian row and column norms; $\text{TOLJAC} \geq 0$. If $\text{MSGLVL} \geq 2$, any rows for which $\max_{1 \leq j \leq n} |\frac{\partial f_k}{\partial x_j}| \leq \text{TOLJAC}$ and any columns for which $\max_{1 \leq k \leq m} |\frac{\partial f_k}{\partial x_j}| \leq \text{TOLJAC}$ will be identified in the output file. If $\text{MSGLVL} < 2$, TOLJAC is not used by HDSFDH.

INDSET [INPUT/OUTPUT, INTEGER, ARRAY]

An array containing the index set information generated by HJSFDI. This array must not be changed by the user between the call to HJSFDI and the initial call to HDSFDH and also must not be changed by the user between successive calls to HDSFDH.

HOLD [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]

Hold array of length NHOLD. This array must not be changed by the user between successive calls to HDSFDH when the Jacobian and Hessian matrices are being computed. After the Jacobian and Hessian computations are complete (ICC = 0), this array can be used elsewhere as a work array.

NHOLD [INPUT, INTEGER]

The length of the HOLD array. $\text{NHOLD} \geq 2 \cdot (\text{NROW} + \text{NCOL})$, where NROW and NCOL are the same values used in HJSFDI.

IHOLD [INPUT/OUTPUT, INTEGER, ARRAY]

Integer hold array of length NIHOLD. This array must not be changed by the user

between successive calls to HDSFDH when the Jacobian and Hessian matrices are being computed. After the Jacobian and Hessian computations are complete ($ICC = 0$), this array can be used elsewhere as a work array.

NIHOLD	[INPUT, INTEGER] The length of the IHOLD array. $NIHOLD \geq NROW$, where NROW is the same value used in HJSFDI.
IER	[OUTPUT, INTEGER] Success/error code. For $IER < 0$, ICC has been set to 0, and at least part of the CJAC and HMAT arrays have been clobbered with HDMCON(1). See Appendix B for HDMCON. Note, however, that for $IER = -21, -22$, and -23 , the CJAC array is not clobbered and contains the computed Jacobian matrix.
IER=0	Success, results computed.
IER=1	$PRTMIN < \sqrt{HDMCON(5)}$.
IER=2	$MSGLVL \geq 2$ and $TOLJAC < 0$. No matrix norms were computed.
IER=3	$MSGLVL \geq 2$ and at least one row or column norm of the Jacobian is $\leq TOLJAC$.
IER=4	The CMULT array is all zeros; no need to compute the Hessian matrix. If only the Jacobian matrix is needed, use HDSFDJ.
IER=-1	Invalid initial/continue call code; $ICC \neq -2, -1, 1$, or 2 .
IER=-2	Invalid INDSET array—the INDSET array must not be changed between the call to HJSFDI and the initial call to HDSFDH and also must not be changed between successive calls to HDSFDH.
IER=-3	NIHOLD too small; the number of words needed is given by the printed error message.
IER=-4	NHOLD too small; the number of words needed is given by the printed error message.
IER=-5	For $ICC = -1$ or -2 , $IFERR \neq 0$. There must not be a function error at the nominal point.
IER=-6	The minimum perturbation size is too small; $PRTMIN < 10 * HDMCON(5)$.
IER=-7	$\min_{1 \leq j \leq NCOL} PRTSYZ(j) < PRTMIN$.

IER=-10	Incorrect use of ICC code; make sure that an initial call to HDSFDH with ICC = -1 is made prior to the Jacobian and Hessian diagonal computations (ICC = 1). See USAGE REMARKS.
IER=-11	Smallest allowable perturbation caused a function error.
IER=-20	Incorrect use of ICC code; make sure that a call to HDSFDH with ICC = -2 is made after the Jacobian and Hessian diagonal computations (ICC = 1) and prior to the off-diagonal Hessian computations (ICC = 2). See USAGE REMARKS.
IER=-21	Inconsistency in nominal variable values detected. The same nominal values must be used in X for ICC = -1 and -2, and the HOLD array must not be changed between successive calls to HDSFDH. In spite of this, the Jacobian matrix was successfully computed for the nominal values supplied when HDSFDH was called with ICC = -1.
IER=-22	Inconsistency in nominal function values detected. The same nominal values must be used in F for ICC = -1 and -2, and the HOLD array must not be changed between successive calls to HDSFDH. In spite of this, the Jacobian matrix was successfully computed for the nominal values supplied when HDSFDH was called with ICC = -1.
IER=-23	A function error occurred while computing the off-diagonal Hessian elements. The computation of the Hessian was not completed. The Jacobian matrix, on the other hand, was successfully computed and is stored in CJAC.

USAGE REMARKS

HDSFDH uses reverse communication to compute the Jacobian and Hessian matrices. A user calls HDSFDH repeatedly to formulate the Jacobian and Hessian values, where each call to HDSFDH performs computations based on a particular index set. Before the initial call to HDSFDH, the nominal variable values must be stored in X, and the corresponding nominal function values must be stored in F. The function error flag IFERR must equal 0 at the nominal X variable values. Also, the perturbation sizes to be used in perturbing the variable values must be stored in PRTSYZ. The user can then make an initial call to HDSFDH with ICC = -1.

After each return from HDSFDH, the user should examine the value of ICC to determine the course of action.

If ICC = 1 or 2, the user's program should evaluate the function using the $\mathbf{x}$ variable values returned in X, that is, set $F = \mathbf{f}(\mathbf{x})$. It then must call HDSFDH again with the same value of ICC to continue the Jacobian and Hessian computations.

If $\text{ICC} = -2$, the Jacobian and the diagonal elements of the Hessian have been computed. To continue the Hessian computation, the user's program can simply call HDSFDH again with $\text{ICC} = -2$.

If $\text{ICC} = 0$ and $\text{IER} = 0$, HDSFDH has successfully computed the Jacobian and Hessian matrices and loaded them in CJAC and HMAT, respectively.

If $\text{ICC} = 0$ and $\text{IER} < 0$, HDSFDH has detected a usage error.

Additionally, note the following:

1. The IROW, JCOL, IPCOLH, and IROWH arrays must be the same arrays as used in HJSFDI.
2. The variables ICC, IROW, JCOL, CJAC, IPCOLH, IROWH, HMAT, CMULT, PRTSYZ, PRTMIN, X, INDSET, HOLD, and IHOLD must not be changed by the user between successive calls to HDSFDH (when $\text{ICC} = 1, 2$, or -2).
3. The variable IFERR can be used to communicate a function error that occurred when the user evaluated $\mathbf{f}(\mathbf{x})$. When $\text{ICC} = 1$ and $\text{IFERR} \neq 0$, HDSFDH will reduce the perturbation sizes, where possible, and request another function evaluation at new X variable values. When $\text{ICC} = 2$ and $\text{IFERR} \neq 0$, HDSFDH cannot proceed with the Hessian computations and returns with $\text{IER} = -23$.

EXAMPLE

As an example, suppose $\mathbf{f}(\mathbf{x})$ is given by:

$$\begin{aligned} f_1 &= x_1^2 + 4x_2 \\ f_2 &= 2x_1x_3 + 1 \\ f_3 &= x_4 + x_5 \\ f_4 &= \frac{2}{x_2} + x_4 \\ f_5 &= x_1^2 - x_5^2 \end{aligned}$$

where

$$\mathbf{f} = \begin{pmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \end{pmatrix} \quad \text{and} \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix}.$$

and suppose a related function is given by

$$L(\mathbf{x}, \lambda) = \sum_{i=1}^M \lambda_i f_i.$$

This example illustrates how the Hessian matrix $\mathbf{H}$ of the function L is evaluated at the point $\mathbf{x} = (2, -1, -.25, 2, -2)$ using $\lambda_i = 1$ for $i = 1, \dots, 5$.

Note that HJSFDI is called to generate the index sets for the problem and is only given the Jacobian sparsity pattern. Using $\text{IHES} = 2$, the Hessian sparsity pattern is generated by HJSFDI based on the Jacobian sparsity pattern.

Also note that the double precision version HDSFDH is used to gain the additional accuracy needed for computing the Hessian matrix. The 2-sided Jacobian matrix is computed in the process of computing the Hessian, but the Jacobian is not used in this example.

SAMPLE PROGRAM (hdsfdh.cd1)⁴

```
PROGRAM SAMPLE

  INTEGER   IHESS, IOPT, IPU, MAXHNZ, MSGVLV, NCOL, NCOLH,
&          NFDJNZ, NHOLD, NINSET, NIWORK, NROW, NVEC
  PARAMETER (IHESS=2, IOPT=2, IPU=6, MAXHNZ=10, MSGVLV=3,
&          NVEC=0, NCOL=5, NCOLH=NCOL+1, NROW=5, NFDJNZ=10,
&          NHOLD=2*(NROW+NCOL), NIWORK=NCOL+NCOL,
&          NINSET=21+(NROW+1)+4*(NCOL+1)+2*NFDJNZ+2*MAXHNZ)
  DOUBLE PRECISION PRINIT, PRTMIN, TOLJAC
  PARAMETER (PRINIT=1.0D-3, PRTMIN=5.0D-05, TOLJAC=1.0D-03)

  INTEGER ICC, IER, IFERR, J, NFDHNZ
  INTEGER IPCOLH(NCOLH), IROW(NFDJNZ), IROWH(MAXHNZ),
&          INDSET(NINSET), IVEC(1), IWORK(NIWORK), JCOL(NFDJNZ)
  DOUBLE PRECISION CJAC(NFDJNZ), CMULT(NROW), F(NROW),
&          HMAT(MAXHNZ), HOLD(NHOLD), PRTSYZ(NCOL),
&          X(NCOL)
  CHARACTER(LEN=30) TITLE1, TITLE2

  DATA IROW  /1, 1, 2, 2, 3, 3, 4, 4, 5, 5/
  DATA JCOL  /1, 2, 1, 3, 4, 5, 2, 4, 1, 5/
  DATA CMULT /1.0, 1.0, 1.0, 1.0, 1.0/
  DATA X     /2.0, -1.0, -0.25, 2.0, -2.0/
  DATA TITLE1 /'JACOBIAN MATRIX BY COLUMNS  '/
  DATA TITLE2 /'HESSIAN MATRIX BY COLUMNS  '/

C
C          Set the perturbation sizes.
C
  PRTSYZ(1:NCOL) = PRINIT

C
C          Generate index sets; note that the Hessian sparsity
C          pattern is generated.
C
  WRITE (IPU,9000)
  CALL HJSFDI (IHESS,NCOL,NROW,NFDJNZ,IROW,JCOL,NFDHNZ,MAXHNZ,
&          NCOLH,IPCOLH,IROWH,MSGVLV,IPU,INDSET,NINSET,IER)
C
C          Compute the Hessian for the X value.
```

⁴sosexample.tar


```

C
  WRITE (IPU,9010)
  CALL FUN (X,F,IFERR)
  ICC = -1

200 CONTINUE
  CALL HDSFDH (ICC,IROW,JCOL,CJAC,IPCOLH,IROWH,HMAT,CMULT,
&              PRTSYZ,PRTMIN,X,F,IFERR,MSGVLV,IPU,TOLJAC,INDSET,
&              HOLD,NHOLD,IWORK,NIWORK,IER)

  IF (ICC.NE.0) THEN
    CALL FUN (X,F,IFERR)
    GO TO 200
  ENDIF

  IF (IER.EQ.0) THEN
C
C          Print out the Hessian matrix for this X value;
C          note that the IVEC vector is not used since
C          the whole matrix is being printed out.
C
    CALL HDSFDP (IOPT,NVEC,IVEC,IROWH,IPCOLH,HMAT,TITLE2,IPU,
&              INDSET,IER)

    ELSE
      WRITE(IPU,*) ' >>>>ERROR: Hessian computation failed.'
      STOP
    ENDIF

    WRITE(IPU,*) '* NORMAL TERMINATION *'
    STOP
9000 FORMAT (//,' ==> STEP 1: GENERATE INDEX SETS',/)
9010 FORMAT (//,' ==> STEP 2: COMPUTE HESSIAN FOR X VALUE',/)
END

SUBROUTINE FUN (X,F,IFERR)

  INTEGER   NCOL, NROW
  DOUBLE PRECISION  ZERO
  PARAMETER (NCOL=5,NROW=5,ZERO=0.0D0)
  INTEGER   IFERR
  DOUBLE PRECISION  X(NCOL), F(NROW)

  IF (X(2).EQ.ZERO) THEN
    IFERR = 1
    RETURN
  ENDIF

```

```
IFERR = 0
F(1) = X(1)*X(1) + 4.0D0*X(2)
F(2) = 2.0D0*X(1)*X(3) + 1.0D0
F(3) = X(4) + X(5)
F(4) = 2.0D0/X(2) + X(4)
F(5) = X(1)*X(1) - X(5)*X(5)

RETURN
END
```

OUTPUT FROM SAMPLE PROGRAM

====> STEP 1: GENERATE INDEX SETS

CHECKING JACOBIAN SPARSITY PATTERN

.....NO INPUTS OUT OF RANGE

GENERATING HESSIAN SPARSITY PATTERN

SUMMARY STATISTICS:

MATRIX	NROWS	NCOLS	NFDNZ	%NZ
-----	-----	-----	-----	-----
JACOBIAN	5	5	10	40.0
HESSIAN	5	5	10	66.7

NUMBER OF INDEX SETS =	2	LOWER BOUND =	2
NUMBER OF FUNCTIONS AT L.B. =	5	1ST SUCH FUNCTION =	1

INDEX SET #	COUNT OF VARIABLES IN EACH INDEX SET
-----	-----
1 TO 2	2 3

INDEX SET 1 : 1 4

INDEX SET 2 : 2 3 5

SPARSE DIFFERENCE SPEEDUP PERCENTAGE = 60.0

MATRIX ELEMENT NORM = 4.000000000

COL	1	- - - - -
	1,	4.000 2, 0. 3, 2.000 5, 0.
COL	2	- - - - -
	2,	-4.000 4, 0.
COL	3	- - - - -
	3,	0.
COL	4	- - - - -
	4,	0. 5, 0.
COL	5	- - - - -
	5,	-2.000

HDSFDC: Check Sparsity Pattern for Missing Elements—Reverse Communication Format

PURPOSE

HDSFDC checks user input of Jacobian nonzero locations for potential missing elements.

RELATED SUBPROGRAMS

HJSFDI	Generate Sparse Finite Difference Index Sets
HDSFDJ	Compute Sparse Finite Difference Jacobian—Reverse Communication Format
HDSFDH	Compute Sparse Finite Difference Jacobian and the Hessian—Reverse Communication Format
HDSFDP	Print Out Sparse Matrix

METHOD

The index sets generated by HJSFDI are used to efficiently check the sparsity pattern of the Jacobian. A set of nominal variable values and the corresponding function values are provided on the initial call to HDSFDC. For each index set, two evaluations are made using different multiples of the given perturbation sizes. By differencing the corresponding multiples of the dependent function values, a determination is made whether the sparsity pattern for each row is correct for the current index set. If not, another perturbation is made to estimate the missing element(s).

USAGE

```
INTEGER NCOL, NFDJNZ, NINSET, NROW
INTEGER ICC, IER, IFERR, IPU, JMISS, MSGLVL, NHOLD
INTEGER INDSET(NINSET), IROW(NFDJNZ), JCOL(NFDJNZ)
DOUBLE PRECISION    PRTMIN
DOUBLE PRECISION    F(NROW), HOLD(NHOLD), PRTSYZ(NCOL), X(NCOL)
CALL HDSFDC(ICC, IROW, JCOL, PRTSYZ, PRTMIN, X, F, IFERR,
1          MSGLVL, IPU, INDSET, HOLD, NHOLD, JMISS, IER)
```

where NCOL, NFDJNZ, NINSET, and NROW have the same values as used in HJSFDI. NCOL is the number of columns, n , in the Jacobian matrix; NFDJNZ is the number of nonzeros in the Jacobian matrix; NINSET is a dimensional constant for INDSET; and NROW is the number of rows, m , in the Jacobian matrix. These values were stored in the INDSET array by HJSFDI and are therefore not repeated in the calling sequence to HDSFDC.

ARGUMENTS

ICC [INPUT/OUTPUT, INTEGER]
Initial/continue call code; see USAGE REMARKS for explanation. ICC is set to -1 by the user on the initialization call and must not be changed by the user

subsequently. On output, ICC contains process status information and directions to the user:

ICC=1 Continue flag; evaluate $\mathbf{f}(\mathbf{x})$ for the $\mathbf{x}$ variable values returned in X and call HDSFDC again with ICC = 1.

ICC=0 Termination flag; check the value of IER for success/error status.

IROW [INPUT, INTEGER, ARRAY]
 Array of row numbers for each nonzero element of the Jacobian matrix. IROW(k) gives the row number of the k^{th} nonzero element, for $k = 1, \dots, \text{NFDJNZ}$. IROW must be the same array as in HJSFDI and must not be changed by the user after the call to HJSFDI.

JCOL [INPUT, INTEGER, ARRAY]
 Array of column numbers for each nonzero element of the Jacobian matrix, in correspondence to the IROW array. JCOL(k) gives the column number of the k^{th} nonzero element, for $k = 1, \dots, \text{NFDJNZ}$. JCOL must be the same array as in HJSFDI and must not be changed by the user after the call to HJSFDI.

PRTSYZ [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
 Perturbation sizes, $\vec{\delta}$, for the $\mathbf{x}$ variable values. PRTSYZ(j) is the perturbation size for X(j); $|\text{PRTSYZ}(j)| \geq \text{PRTMIN}$. The perturbation sizes are only reduced when a function error, signalled by IFERR $\neq 0$, prevents a successful finite difference computation. Furthermore, a reduction in perturbation sizes is done only for variables in the current index set. PRTSYZ must be specified by the user on the initialization call and not subsequently changed by the user. A recommended initial value for PRTSYZ(j) is $10^{-5} \max[1, x(j)]$.

PRTMIN [INPUT, DOUBLE PRECISION]
 Minimum absolute perturbation size for any variable x_j ;
 $10 * \text{HDMCON}(5) \leq \text{PRTMIN} \leq \min_{1 \leq j \leq \text{NCOL}} |\text{PRTSYZ}(j)|$. HDSFDC will not reduce any $|\text{PRTSYZ}(j)|$ below PRTMIN during its computations. PRTMIN must be specified by the user on the initialization call and not subsequently changed. A recommended initial value for PRTMIN is $\sqrt{\text{HDMCON}(5)}$. Setting PRTMIN less than this value will result in a warning message.

X [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
 Independent variable values $\mathbf{x}$. The current use of X depends on the value of ICC:

ICC=-1 X must be initialized to the nominal point, that is, the $\mathbf{x}$ value at which the Jacobian will be evaluated for missing elements.

ICC=1 X contains a perturbed point at which a function evaluation is required.

ICC=0 X contains the nominal point, restored by HDSFDC.

X must be specified by the user on the initialization call and not subsequently changed by the user. See USAGE REMARKS.

F	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Dependent function values $\mathbf{f}(\mathbf{x})$. The current use of F depends on the value of ICC (see USAGE REMARKS): ICC =1 F must be set by the user to the values of the function at the current value of X. ICC=0 F contains the values of the function at the nominal point, restored by HDSFDC.
IFERR	[INPUT, INTEGER] Function error flag. This variable is used to communicate a function error that occurred when the user evaluated $\mathbf{f}(\mathbf{x})$. IFERR=0 The evaluation of $\mathbf{f}(\mathbf{x})$ was successful. IFERR$\neq$0 An error occurred in evaluating $\mathbf{f}(\mathbf{x})$. IFERR must be 0 on the initialization call. This implies that the function must be valid at the nominal point. See USAGE REMARKS.
MSGLVL	[INPUT, INTEGER] Level of diagnostic print. MSGLVL$\leq$0 Suppress all output. MSGLVL=1 Error messages and some diagnostic information about reduced perturbation sizes and potential missing elements in the sparsity pattern. MSGLVL$\geq$2 Complete diagnostic output, including additional perturbation information.
IPU	[INPUT, INTEGER] Fortran logical unit number to which diagnostic output will be written. It is assumed that this unit has been opened by the user prior to the call to HDSFDC if MSGLVL > 0.
INDSET	[INPUT/OUTPUT, INTEGER, ARRAY] An array containing the index set information generated by HJSFDI. This array must not be changed by the user between the call to HJSFDI and the initial call to HDSFDC and also must not be changed by the user between successive calls to HDSFDC.
HOLD	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Hold array of length NHOLD. This array must not be changed by the user between successive calls to HDSFDC when the Jacobian sparsity pattern is being evaluated. After the evaluation is complete (ICC = 0), this array can be used elsewhere as a work array.

NHOLD	[INPUT, INTEGER] The length of the HOLD array. $\text{NHOLD} \geq (4*\text{NROW}+\text{NCOL})$, where NROW and NCOL are the same values used in HJSFDI.
JMISS	[OUTPUT, INTEGER] Code for potential missing elements in the Jacobian sparsity pattern. JMISS=-1 Error condition; $\text{IER} < 0$. JMISS=0 No missing elements found in the sparsity pattern. JMISS>0 Potential missing elements found in the sparsity pattern. The value of JMISS gives a lower bound on the number of potential missing elements. HDSFDC can find at most one potential missing element per index set per row of the Jacobian, so there could be more than JMISS missing elements. Additional diagnostic information can be obtained with MSGVLV > 0.
IER	[OUTPUT, INTEGER] Success/error code. For $\text{IER} < 0$, ICC has been set to 0 and JMISS has been set to -1. IER=0 Success, results computed. IER=1 $\text{PRTMIN} < \sqrt{\text{HDMCON}(5)}$. See Appendix B for HDMCON. IER=-1 Invalid initial/continue call code; $\text{ICC} \neq -1$ or 1. IER=-2 Invalid INDSET array—the INDSET array must not be changed between the call to HJSFDI and the initial call to HDSFDC and also must not be changed between successive calls to HDSFDC. IER=-4 NHOLD too small; the number of words needed is given by the printed error message. IER=-5 For $\text{ICC} = -1$, $\text{IFERR} \neq 0$. There must not be a function error at the nominal point. IER=-6 The minimum perturbation size is too small; $\text{PRTMIN} < 10*\text{HDMCON}(5)$. IER=-7 $\min_{1 \leq j \leq \text{NCOL}} \text{PRTSYZ}(j) < \text{PRTMIN}$. IER=-10 Incorrect use of ICC code; need to make initial call to HDSFDC with $\text{ICC} = -1$. See USAGE REMARKS. IER=-11 Smallest allowable perturbation caused a function error.

USAGE REMARKS

HDSFDC uses reverse communication to check the Jacobian sparsity pattern. A user calls HDSFDC repeatedly to perform the checks, where each call to HDSFDC performs computations based on a particular index set. Before the initial call to HDSFDC, the nominal variable values must be stored in $\mathbf{X}$, and the corresponding nominal function values must be stored in $\mathbf{F}$. The function error flag IFERR must equal 0 at the nominal $\mathbf{X}$ variable values. Also, the perturbation sizes to be used in perturbing the variable values must be stored in PRTSYZ. The user can then make an initial call to HDSFDC with $\text{ICC} = -1$.

After each return from HDSFDC, the user should examine the value of ICC to determine the course of action.

If $\text{ICC} = 1$, the user's program should evaluate the function using the $\mathbf{x}$ variable values returned in $\mathbf{X}$, that is, set $\mathbf{F} = \mathbf{f}(\mathbf{x})$. It then must call HDSFDC again to continue the sparsity pattern check.

If $\text{ICC} = 0$ and $\text{JMISS} = 0$, HDSFDC has checked the Jacobian sparsity pattern and did not find any missing elements.

If $\text{ICC} = 0$ and $\text{JMISS} > 0$, HDSFDC has checked the Jacobian sparsity pattern and has determined that there are at least JMISS missing elements.

If $\text{ICC} = 0$ and $\text{JMISS} < 0$, HDSFDC has detected a usage error. Check the value of IER for the error number.

Additionally, note the following:

1. The IROW and JCOL arrays must be the same arrays as used in HJSFDI.
2. The variables ICC, IROW, JCOL, PRTSYZ, PRTMIN, $\mathbf{X}$, INDSET, and HOLD must not be changed by the user between successive calls to HDSFDC (when $\text{ICC} = 1$).
3. The variable IFERR can be used to communicate a function error that occurred when the user evaluated $\mathbf{f}(\mathbf{x})$. In this case, HDSFDC will reduce the perturbation sizes, where possible, and request another function evaluation at new $\mathbf{X}$ variable values.

EXAMPLE

See HDSFDJ for an example illustrating the use of HDSFDC.

HDSFDP: Print Out Sparse Matrix

PURPOSE

HDSFDP writes the specified rows or columns of a given Jacobian or Hessian matrix, previously processed by HJSFDI, to the output unit.

RELATED SUBPROGRAMS

HJSFDI	Generate Sparse Finite Difference Index Sets
HDSFDJ	Compute Sparse Finite Difference Jacobian—Reverse Communication Format
HDSFDH	Compute Sparse Finite Difference Jacobian and the Hessian—Reverse Communication Format
HDSFDC	Check Sparsity Pattern for Missing Elements—Reverse Communication Format

METHOD

The index sets and row/column ordering generated by HJSFDI are used to efficiently produce either a row- or column-oriented printout of the specified matrix. For the case where only the index sets for the Jacobian were computed and no Hessian sparsity pattern information was retained, a print by rows requires an exhaustive search in order to find the required elements for printing. It is recommended that only selected rows, rather than the entire matrix, be printed in this case.

USAGE

```

INTEGER NINSET
INTEGER IER, IOPT, IPU, MAXIR, MAXJC, NVDIM, NVEC
INTEGER ICONUM(MAXJC), INDSET(NINSET), IRONUM(MAXIR), IVEC(NVDIM)
DOUBLE PRECISION    AMAT(MAXIR)
CHARACTER$(*) TITLE
CALL HDSFDP(IOPT,NVEC,IVEC,IRONUM,ICONUM,AMAT,TITLE,IPU,INDSET,IER)

```

where $\text{MAXIR} = \text{MAXJC} = \text{NFDJNZ}$ when a Jacobian matrix is to be printed out,
 $\text{MAXIR} = \text{NFDHNZ}$ and $\text{MAXJC} = \text{NCOLH}$ when a Hessian matrix is to be printed out,
 $\text{NVDIM} \geq \max(1, \text{NVEC})$,

and NCOLH , NFDHNZ , NFDJNZ , and NINSET have the same values as used in HJSFDI. These values were stored in the INDSET array by HJSFDI and are therefore not repeated in the calling sequence to HDSFDP.

REMARKS

In the following description of the arguments for HDSFDP, NROW is the number of rows and NCOL is the number of columns in the Jacobian matrix. These are the same values as used in HJSFDI. Note that NCOL is also the size of the symmetric Hessian matrix.

ARGUMENTS

- IOPT** [INPUT, INTEGER]
Print matrix option; $|\text{IOPT}| = 1$ or 2 .
- IOPT=-1 Print the given Jacobian matrix by rows.
IOPT=1 Print the given Jacobian matrix by columns.
IOPT=-2 Print the given Hessian matrix by rows.
IOPT=2 Print the given Hessian matrix by columns.
- NVEC** [INPUT, INTEGER]
Number of rows or columns to print—see the description of IOPT. If $\text{IOPT} = -1$, $\text{NVEC} \leq \text{NROW}$. Otherwise, $\text{NVEC} \leq \text{NCOL}$. If $\text{NVEC} \leq 0$, the whole matrix is printed.
- IVEC** [INPUT, INTEGER, ARRAY]
Index array of the row or column numbers to print. Its use depends on the value of IOPT:
- IOPT<0 IVEC contains the matrix row numbers to print. Printout is only generated for valid IVEC values, $1 \leq \text{IVEC}(k) \leq \text{NROW}$ for $\text{IOPT} = -1$ and $1 \leq \text{IVEC}(k) \leq \text{NCOL}$ for $\text{IOPT} = -2$. Other values are ignored.
- IOPT>0 IVEC contains the matrix column numbers to print. Printout is only generated for valid IVEC values, $1 \leq \text{IVEC}(k) \leq \text{NCOL}$. Other values are ignored.
- IRONUM** [INPUT, INTEGER, ARRAY]
Array of row numbers for each nonzero element of the matrix. See USAGE REMARKS. Its contents depend on the value of IOPT:
- $|\text{IOPT}|=1$ IRONUM is an array of row numbers for each nonzero element of the Jacobian matrix. IRONUM must be the same as the IROW array used by HJSFDI.
- $|\text{IOPT}|=2$ IRONUM is an array of row numbers of the Hessian nonzero locations. IRONUM must be the same as the IROWH array used by HJSFDI.
- ICONUM** [INPUT, INTEGER, ARRAY]
Array containing column information about the nonzero elements of the matrix. See USAGE REMARKS. Its contents depend on the value of IOPT:
- $|\text{IOPT}|=1$ ICONUM is an array of column numbers for each nonzero element of the Jacobian matrix, in correspondence to the IRONUM array. ICONUM must be the same as the JCOL array used by HJSFDI.
- $|\text{IOPT}|=2$ ICONUM is an array containing pointers to locations in the IRONUM array corresponding to the beginning of each column of the Hessian matrix. ICONUM must be the same as the IPCOLH array used by HJSFDI.

AMAT	[INPUT, DOUBLE PRECISION, ARRAY] Array containing the Jacobian or Hessian matrix values, stored in correspondence to the IRONUM and ICONUM arrays. See USAGE REMARKS. The order of values in AMAT depends on IOPT: IOPT =1 AMAT(k) is the value for row IRONUM(k) and column ICONUM(k) of the Jacobian matrix. IOPT =2 AMAT(k) is the value for row IRONUM(k) and column j of the Hessian matrix, where j is the largest index in 1, ..., NCOL with ICONUM(j) $\leq k$.
TITLE	[INPUT, CHARACTER] Title to print with the output.
IPU	[INPUT, INTEGER] Fortran logical unit number to which output will be written. It is assumed that this unit has been opened by the user prior to the call to HDSFDP.
INDSET	[INPUT, INTEGER, ARRAY] An array containing the index set information generated by HJSFDI. This array must not be changed by the user between the call to HJSFDI and the call to HDSFDP.
IER	[OUTPUT, INTEGER] Success/error code. For IER < 0, no output is generated. IER=0 Success, the matrix was printed as requested. IER=-1 Invalid print option; IOPT $\neq$ -2, -1, 1, or 2. IER=-2 Invalid INDSET array—the INDSET array must not be changed between the call to HJSFDI and the call to HDSFDP. IER=-3 Either IOPT = -1 and NVEC > NROW or IOPT $\neq$ -1 and NVEC > NCOL. IER=-4 IRONUM and ICONUM are not compatible with the sparsity pattern information in INDSET. See USAGE REMARKS.

USAGE REMARKS

HDSFDP uses the arrays IRONUM, ICONUM, and AMAT in a dual capacity, depending on the value of IOPT.

|IOPT|=1 These arrays are assumed to contain the Jacobian matrix information. IRONUM and ICONUM must then be the same as the IROW and JCOL arrays used by HJSFDI. The AMAT array must contain the Jacobian values in correspondence to the IRONUM and ICONUM arrays. The AMAT array is therefore equivalent to the CJAC array computed in HDSFDJ and HDSFDH.

|IOPT|=2 These arrays are assumed to contain the Hessian matrix information. IRONUM and ICONUM must then be the same as the IROWH and IPCOLH arrays used by HJSFDI. The AMAT array must contain the Hessian values in correspondence to the IRONUM and ICONUM arrays. The AMAT array is therefore equivalent to the HMAT array computed in HDSFDH.

EXAMPLE

See HDSFDJ and HDSFDH for an example illustrating the use of HDSFDC.

Chapter 4

Advanced Usage Examples

4.1 Introduction—Advanced Application Examples

The optimization and derivative software described in the preceding chapters can be utilized to solve very complex problems. While it is not feasible to anticipate all possible uses for the suite of routines, a number of typical situations are illustrated in this chapter. The first three examples in Sections 4.2, 4.3, and 4.4 illustrate how the optimization software can be used in conjunction with the finite difference derivative software. The last two examples in Sections 4.5 and 4.6 illustrate how the diagnostic capabilities of the algorithm can be exploited to interpret the results.

4.2 Reverse Communication Dense NLP with Finite Difference Derivatives

This example illustrates the use of the reverse communication format dense nonlinear programming algorithm DNLPRV, used in conjunction with the reverse communication finite difference derivative routine HDDFDJ and HDDFDH.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2, x_3)$ to minimize the objective function

$$f(x) = (x_1 - 1)^2 + (x_1 - x_2)^2 + (x_2 - x_3)^4$$

subject to the constraint

$$x_1(1 + x_2^2) + x_3^4 - 4 - 3\sqrt{2} = 0$$

beginning at the initial point $x = (2, 2, 2)$.

SAMPLE PROGRAM (adv1.cd1)¹

```

PROGRAM EXAMPL
C
C      PROGRAM TO DEMONSTRATE THE USE OF DNLPRV WITH THE FINITE
C      DIFFERENCE ROUTINES HDDFDJ AND HDDFDH
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C-----
C
C      PARAMETER (NDIM = 3, MCON = 1)
C      PARAMETER (NHOLD=1000, NIHOLD=1000 ,NHOLDQ = 1000)
C      PARAMETER (LNHMAT = (NDIM*(NDIM+1))/2 )
C      PARAMETER (LNQMAT = (MCON+1)*NDIM )
C      DIMENSION HMAT(LNHMAT)
C      DIMENSION QMAT(LNQMAT), QUANT(MCON+1), CMULT(MCON+1)
C
C      DIMENSION XBAR(NDIM),XLWR(NDIM),XUPR(NDIM),
C      $      ISTATV(NDIM),VECNU(NDIM),
C      $      CBAR(MCON),CLWR(MCON),
C      $      CUPR(MCON),ISTATC(MCON),VECLAM(MCON)
C      DIMENSION PRSZF(NDIM),PRSZC(NDIM)
C      DIMENSION HOLD(NHOLD),IHOLD(NIHOLD),HOLDQ(NHOLDQ)
C
C      INTERNAL ARRAYS
C

```

¹sosexample.tar

```

        DIMENSION IREVRS(5)
C
        EXTERNAL FUNBOX
C
C -----
C
        IPU = 6
C
        LOCATION OF GRADIENT VECTOR IN QMAT ARRAY
C
        LCDELF = MCON*NDIM + 1
        ISTOP = 1
C
        BIGBND = 1.D0/HDMCON(5)
C
        INITIALIZE NLP REVERSE COMMUNICATION CONTROL FLAG
C
        IRVCOM = -1
C
        INDEPENDENT VARIABLE DATA
C
        ----INITIAL GUESS
        xbar(1:ndim) = 2.d0
C
        ----BOUNDS
        xlwr(1:ndim) = -bigbnd
        xupr(1:ndim) = bigbnd
C
        ----VARIABLE STATUS
        istatv(1:ndim) = 0
C
C -----
C
        CONSTRAINT DATA
C
        ----CONSTRAINT STATUS
        ISTATC(1) = 3
C
        ----BOUNDS
        CLWR(1) = 0.D0
        CUPR(1) = 0.D0
C
C
        DEFINE PERTURBATION SIZES
C
        PRMTIN = SQRT(HDMCON(5))
        EPSRLF = HDMCON(5)**.9
        CNTINT = EPSRLF**.333
        FWDINT = SQRT(EPSRLF)
        DO 110 II = 1,NDIM
            VARSIZ = 1.D0 + ABS(XBAR(II))

```

```

        PRTSZC(II) = CNTINT*VARSIZ
        PRTSZF(II) = FWDINT*VARSIZ
110  CONTINUE
C
C -----
C
C      INITIALIZE FUNCTION EVALUATION COUNT
C
C      NFEVAL = 0
C
C      SET OPTIONAL INPUTS
C
C      CALL INSNLP('DENSE DEFAULT')
C      CALL INSNLP('IOFLAG=1')
C
C -----
C
120  CONTINUE
C
C      CALL DNLPRV( IRVCOM, IREVRS, XBAR, XLWR, XUPR,
$      ISTATV, VECNU, NDIM, FBAR, QMAT(LCDELF),
$      HMAT, CBAR, CLWR, CUPR, ISTATC, MCON,
$      VECLAM, QMAT, IFERR, NFEVAL, HOLD,
$      NHOLD, IHOLD, NIHOLD, NEEDED, IERNLP )
C
C      REVERSE COMMUNICATION TERMINATION TEST
C
C      IF(IRVCOM.EQ.0) GO TO 170
C
C -----
C
C      SET FUNCTION EVALUATION ERROR FLAG
C
C      IFERR = 0
C
C      >>>>FUNCTION EVALUATION REQUEST
C
C      IF(IREVRS(1).EQ.1) THEN
C
C      EVALUATE FUNCTIONS
C
C      NFEVAL = NFEVAL + 1
C      CALL FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
C      IF(IFERR.EQ.0) THEN
C          cbar(1:mcon) = quant(1:mcon)
C          FBAR = QUANT(MCON+1)
C      ENDIF
C

```



```
      ENDIF
C
C      >>>>GRADIENT EVALUATION REQUEST
C
      IF(IREVRS(2).EQ.1) THEN
C
C          ICCD = -1
C
130      CONTINUE
C
C          FORWARD DIFFERENCE GRADIENTS
C
C          IDTYPE = 1
C          CALL HDDFDJ(ICCD,MCON+1,NDIM,QMAT,
$           PRTSZF,PRTMIN,XBAR,QUANT,IFERR,ISTOR,MSGVLV,
$           IPU,IDTYPE,HOLDQ,NHOLDQ,IERD)
C
C          IF(ICCD.NE.0) THEN
C
C              NFEVAL = NFEVAL + 1
C              CALL FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
C
C              GO TO 130
C
C          ENDIF
C
      ELSEIF(IREVRS(2).EQ.2.AND.IREVRS(3).EQ.0) THEN
C
C          ICCD = -1
C
140      CONTINUE
C
C          CENTRAL DIFFERENCE GRADIENTS
C
C          IDTYPE = 2
C          CALL HDDFDJ(ICCD,MCON+1,NDIM,QMAT,
$           PRTSZC,PRTMIN,XBAR,QUANT,IFERR,ISTOR,MSGVLV,
$           IPU,IDTYPE,HOLDQ,NHOLDQ,IERD)
C
C          IF(ICCD.NE.0) THEN
C
C              NFEVAL = NFEVAL + 1
C              CALL FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
C
C              GO TO 140
C
C          ENDIF
C
```

```

        ENDIF
C
C      >>>>HESSIAN EVALUATION REQUEST
C
      IF(IREVRS(2).EQ.2.AND.IREVRS(3).EQ.1) THEN
C
      ICCH = -1
      cmult(1:mcon) = -veclam(1:mcon)
      CMULT(MCON+1) = 1.DO
C
      150    CONTINUE
C
C          DIAGONAL ELEMENTS
C
      CALL HDDFDH(ICCH,MCON+1,NDIM,QMAT,HMAT,
$          CMULT,PRTSZC,PRTMIN,XBAR,QUANT,
$          IFERR,ISTOR,MSGLVL,IPU,HOLDQ,NHOLDQ,IERH)
C
      IF(ICCH.GT.0) THEN
C
      NFEVAL = NFEVAL + 1
      CALL FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
C
      GO TO 150
C
      ENDIF
C
      ELSEIF(IREVRS(3).EQ.2) THEN
C
      ICCH = -2
C
      160    CONTINUE
C
C          OFF-DIAGONAL ELEMENTS
C
      CALL HDDFDH(ICCH,MCON+1,NDIM,QMAT,HMAT,
$          CMULT,PRTSZC,PRTMIN,XBAR,QUANT,
$          IFERR,ISTOR,MSGLVL,IPU,HOLDQ,NHOLDQ,IERH)
C
      IF(ICCH.GT.0) THEN
C
      NFEVAL = NFEVAL + 1
      CALL FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
      GO TO 160
C
      ENDIF
C
      ENDIF

```

```
C
C      >>>>>SYSTEM PRINT (OUTPUT) REQUEST
C
C      IF(IREVRS(4).EQ.1) THEN
C
C          WRITE(6,1003) FBAR,NFEVAL
C          WRITE(6,1001) 'VARIABLES'
C          WRITE(6,1002) (XBAR(II),II=1,NDIM)
C          IF(MCON.GT.0) THEN
C              WRITE(6,1001) 'CONSTRAINTS'
C              WRITE(6,1002) (CBAR(II),II=1,MCON)
C          ENDIF
C          WRITE(6,1004)
C
C      ENDIF
C
C      -----
C
C      END OF REVERSE COMMUNICATION PROCESSING.  RETURN TO SNLPMN
C
C      GO TO 120
C
C      170 CONTINUE
C
C
C      TERMINATE PROCESSING
C
C      IF(IERNLP.EQ.0) THEN
C
C          NORMAL TERMINATION---SOLUTION FOUND
C
C          PRINT *, 'SOLUTION      '
C          WRITE(6,1003) FBAR,NFEVAL
C          WRITE(6,1001) 'VARIABLES'
C          WRITE(6,1002) (XBAR(II),II=1,NDIM)
C          IF(MCON.GT.0) THEN
C              WRITE(6,1001) 'CONSTRAINTS'
C              WRITE(6,1002) (CBAR(II),II=1,MCON)
C          ENDIF
C          WRITE(6,1004)
C
C      ELSEIF(IERNLP.LT.0) THEN
C
C          ABNORMAL TERMINATION---FATAL ERROR
C
C          PRINT *, 'FATAL ERROR; IERNLP =', IERNLP
C
C      ELSE
```

```

C
C      ABNORMAL TERMINATION---WARNING.  EVALUATION OF RESULTS RECOMMENDED!
C
C      PRINT *, 'WARNING MESSAGE; IERNLP =', IERNLP
C
C      ENDIF
C
C-----
C
1001 FORMAT(/,T20,A,/)
1002 FORMAT(T5,5G16.6)
1003 FORMAT(/,T20,'OBJECTIVE =',G16.6,T60,'NFEVAL =',I5)
1004 FORMAT(/)
C
C      STOP
C      END
C
C      SUBROUTINE FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      DIMENSION XBAR(NDIM),QUANT(MCON+1)
C
C      IFERR = 0
C      QUANT(1) = XBAR(1)*(1.D0 + XBAR(2)**2) + XBAR(3)**4
C      $          - 4.D0 - 3.D0*SQRT(2.D0)
C      QUANT(MCON+1) = (XBAR(1) - 1.D0)**2 + (XBAR(1) - XBAR(2))**2
C      $          + (XBAR(2) - XBAR(3))**4
C
C      RETURN
C      END

```

OUTPUT FROM SAMPLE PROGRAM

Feasible Point							
It	Qit	Nkt	Ndof	Cond(G)	Step	Norm p	Violtn
1	1	1	2	1.0E+00	1.0E+00	5.3E-01	1.8E+01
2	2	1	2	1.0E+00	1.0E+00	3.4E-01	5.3E+00
3	2	1	2	1.0E+00	1.0E+00	1.0E-01	1.1E+00
4	2	1	2	1.0E+00	1.0E+00	6.4E-03	5.8E-02
5	2	1	2	1.0E+00	1.0E+00	2.2E-05	2.0E-04
6	0	0	2	1.0E+00	1.0E+00	2.2E-05	2.3E-09

Optimization											
It	Qit	Nkt	Ndof	PH Cond	Step	Norm p	Violtn	Levnbrg	Penalty	Norm Pg	Merit Function
1	2	1	2	1.0E+00	1.0E+00	1.8E+00	2.3E-09	0.0E+00	1.5E-08	1.6E+00	6.988735E-01
2	2	1	2	4.7E+00	3.4E-01	6.5E-01	8.9E-01	0.0E+00	1.5E-08	8.4E-01	1.919590E-01
3	2	1	2	3.4E+00	1.0E+00	3.5E-01	1.1E+00	0.0E+00	1.5E-08	8.8E-01	1.033876E-01
4	2	1	2	2.5E+00	1.0E+00	1.3E-01	3.8E-01	0.0E+00	1.5E-08	2.8E-01	6.628421E-02
5	2	1	2	5.1E+00	1.0E+00	1.4E-01	3.3E-02	0.0E+00	1.5E-08	1.5E-01	4.029617E-02

6	2	1	2	3.1E+00	1.0E+00	6.4E-02	5.4E-02	0.0E+00	1.5E-08	1.2E-01	3.616363E-02
7	2	1	2	2.7E+00	1.0E+00	9.7E-03	1.1E-02	0.0E+00	1.5E-08	2.2E-02	3.269229E-02
8	2	1	2	3.0E+00	1.0E+00	1.3E-03	2.1E-04	0.0E+00	1.5E-08	2.4E-03	3.256990E-02
9	2	1	2	2.9E+00	1.0E+00	2.9E-05	4.1E-06	0.0E+00	1.5E-08	5.4E-05	3.256820E-02
(10)	0	0	2	2.9E+00	1.0E+00	2.9E-05	2.3E-09	0.0E+00	1.5E-08	3.9E-07	3.256820E-02

 SPRNLP ALGORITHM PERFORMANCE STATISTICS

Total CPU Time.....+1.119742E-02

Number of Function Calls.....16

Number of Gradient Calls.....15

Number of Hessian Calls.....9

Total Number of Function Evaluations.....64

Storage Needed in HOLD Array.....973

Storage Needed in IHOLD Array.....800

 SOLUTION

OBJECTIVE = 0.325682E-01 NFEVAL = 64

VARIABLES

1.10486 1.19667 1.53526

CONSTRAINTS

0.229365E-08

4.3 Sparse Finite Differences with Sparse NLP

This example illustrates the use of the sparse finite difference derivative evaluation subroutines HDSFDJ and HDSFDH, which require using the sparse index set routine HJSFDI. The derivatives are evaluated as required by the sparse optimization routine SNLPMN.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2, x_3, x_4)$ to minimize the objective function

$$f(x) = (x_3 - \sin(x_4))^2$$

subject to the constraints

$$\begin{aligned} x_1 + x_2 &= 0 \\ x_2 + x_3 &= 0 \end{aligned}$$

beginning at the initial point $x = (-1, 2, 2, 1)$. The sparse finite difference subroutine is applied to an augmented Jacobian of the form

$$\mathbf{Q} = \begin{pmatrix} \mathbf{G} \\ \mathbf{g}^T \end{pmatrix}$$

consisting of the constraint Jacobian in the first m rows with the objective gradient as the last row. For this example the sparsity pattern of $\mathbf{Q}$ is

$$\begin{pmatrix} * & * & & \\ & * & * & \\ & & * & * \end{pmatrix}.$$

Notice also that the sparsity pattern of the Hessian matrix $\mathbf{H}$ is

$$\begin{pmatrix} * & & & \\ & * & & \\ & & * & * \\ & & * & * \end{pmatrix}$$

whereas the sparsity pattern of the normal matrix is

$$\begin{pmatrix} * & x & & \\ x & * & x & \\ & x & * & * \\ & & * & * \end{pmatrix}.$$

The elements denoted by “x” are zero because the constraints are linear, and consequently are not included in the sparsity pattern input to subroutine HJSFDI. The absence of these elements in the Hessian structure is detected by HJSFDI and this information is communicated with the warning message “IERIND = 5” in the sample output.

SAMPLE PROGRAM (adv2.cd1)²

²sosexample.tar

```

PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE SPARSE NONLINEAR
C      PROGRAMMING ALGORITHM SNLPMN
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      PARAMETER (NDIM=4, MCON=2)
C      PARAMETER (NONZH=5, NONZG=4)
C      PARAMETER (NQNT = MCON + 1)
C      PARAMETER (NHOLD=2000,NIHOLD=2000)
C      PARAMETER (NHOLDQ=2000,NIHLDQ=1000,NINSET=1000)
C
C      DIMENSION IREVRS(5),XBAR(NDIM),XLWR(NDIM),XUPR(NDIM),
$      ISTATV(NDIM),VECNU(NDIM),DELF(NDIM),HMAT(NONZH),
$      IROWH(NONZH),JSTRH(NDIM+1),CBAR(MCON),CLWR(MCON),
$      CUPR(MCON),ISTATC(MCON),VECLAM(MCON),
$      GMAT(NONZG),IROWG(NONZG),JCOLG(NONZG)
C      DIMENSION HOLD(NHOLD),IHOLD(NIHOLD)
C
C      DIMENSION CMULT(NQNT),QUANT(NQNT),QMAT(NONZG+2)
C      DIMENSION IROWQ(NONZG+2),JCOLQ(NONZG+2)
C      DIMENSION HOLDQ(NHOLDQ),IHOLDQ(NIHLDQ),INDSET(NINSET)
C      DIMENSION PRSZC(NDIM),PRSZF(NDIM)
C
C      EXTERNAL FUNBOX
C      IPU = 6
C
C      -----
C
C      INDEPENDENT VARIABLE DATA
C
C      ----INITIAL GUESS
C      XBAR(1) = -1.0D0
C      XBAR(2) = 2.0D0
C      XBAR(3) = 2.0D0
C      XBAR(4) = 1.0D0
C
C      ----BOUNDS
C      BIGBND = 1.D0/HDMCON(5)
C      xlwr(1:ndim) = -bigbnd
C      xupr(1:ndim) = +bigbnd
C
C      ----VARIABLE STATUS
C      istatv(1:ndim) = 0
C
C      -----
C
C      OBJECTIVE FUNCTION DATA
C

```

```

C      ----HESSIAN SPARSITY PATTERN (ROW INDICES)
C      IROWH(1) = 1
C      IROWH(2) = 2
C      IROWH(3) = 3
C      IROWH(4) = 4
C      IROWH(5) = 4
C      ----HESSIAN SPARSITY PATTERN (COLUMN START INDICES)
C      JSTRH(1) = 1
C      JSTRH(2) = 2
C      JSTRH(3) = 3
C      JSTRH(4) = 5
C      JSTRH(5) = 6
C
C      -----
C
C      CONSTRAINT DATA
C
C      ----BOUNDS
C      CLWR(1) = 0.DO
C      CUPR(1) = 0.DO
C      CLWR(2) = 0.DO
C      CUPR(2) = 0.DO
C      ----CONSTRAINT STATUS
C      istatc(1:mcon) = 3
C      ----JACOBIAN SPARSITY PATTERN (ROW INDICES)
C      IROWG(1) = 1
C      IROWG(2) = 1
C      IROWG(3) = 2
C      IROWG(4) = 2
C      ----JACOBIAN SPARSITY PATTERN (COLUMN INDICES)
C      JCOLG(1) = 1
C      JCOLG(2) = 2
C      JCOLG(3) = 2
C      JCOLG(4) = 3
C
C      -----
C
C      CONSTRUCT SPARSITY PATTERN FOR AUGMENTED JACOBIAN ARRAY.
C      BY CONSTRUCTION THE FIRST MCON ROWS CONTAIN THE CONSTRAINT
C      JACOBIAN MATRIX.  THE LAST ROW OF THE AUGMENTED JACOBIAN
C      CONTAINS THE OBJECTIVE GRADIENT (STORED AS A ROW VECTOR).
C
C      ----JACOBIAN SPARSITY PATTERN
C
C      irowq(1:nonzg) = irowg(1:nonzg)
C      jcolq(1:nonzg) = jcolg(1:nonzg)
C      NFDQNZ = NONZG
C

```



```

C      ----GRADIENT VECTOR
C
      NFDQNZ = NFDQNZ + 1
      JCOLQ(NFDQNZ) = 3
      IROWQ(NFDQNZ) = MCON+1
      NFDQNZ = NFDQNZ + 1
      JCOLQ(NFDQNZ) = 4
      IROWQ(NFDQNZ) = MCON+1
C
C      GENERATE INDEX SETS
C
      IHESS = 1
      NFDHNZ = NONZH
      MAXHNZ = NONZH
      MSGLVL = 0
      CALL HJSFDI(IHESS,NDIM,NQNT,NFDQNZ,IROWQ,JCOLQ,
$  NFDHNZ,MAXHNZ,NDIM+1,JSTRH,IROWH,MSGLVL,IPU,
$  INDSET,NINSET,IERIND)
C
      IF(IERIND.LT.0) THEN
        PRINT *, 'INDEX SET ERROR; IERIND = ',IERIND
        STOP
      ELSEIF(IERIND.GT.0) THEN
        PRINT *, 'INDEX SET WARNING; IERIND = ',IERIND
      ENDIF
C
C      DEFINE PERTURBATION SIZES
C
      PRTMIN = SQRT(HDMCON(5))
      EPSRLF = HDMCON(5)**.9
      CNTINT = EPSRLF**.333
      FWDINT = SQRT(EPSRLF)
      DO 110 II = 1,NDIM
        VARSIZ = 1.DO + ABS(XBAR(II))
        PRTSZC(II) = CNTINT*VARSIZ
        PRTSZF(II) = FWDINT*VARSIZ
110  CONTINUE
C
C      -----
C
C      INITIALIZE FUNCTION EVALUATION COUNT
C
      NFEVAL = 0
C
C      INITIALIZE REVERSE COMMUNICATION CONTROL FLAG
C
      ICC = -1
C

```

```
C          SET OPTIONAL INPUTS
C
C          CALL INSNLP('IOFLAG=1')
C
C -----
C
C 120  CONTINUE
C
C          CALL SNLPMN( ICC ,IREVRS ,XBAR ,XLWR ,XUPR ,ISTATV
$           ,VEGNU ,NDIM ,FBAR ,DELF ,HMAT ,IROWH
$           ,JSTRH ,NONZH ,CBAR ,CLWR ,CUPR ,ISTATC
$           ,MCON ,VECLAM ,GMAT ,IROWG ,JCOLG ,NONZG
$           ,IFERR ,NFEVAL ,HOLD ,NHOLD ,IHOLD ,NIHOLD
$           ,NEEDED ,IERNLP )
C
C          REVERSE COMMUNICATION TERMINATION TEST
C
C          IF(ICC.EQ.0) GO TO 170
C
C -----
C
C          SET FUNCTION EVALUATION ERROR FLAG
C
C          IFERR = 0
C
C          >>>>FUNCTION EVALUATION REQUEST
C
C          IF(IREVRS(1).EQ.1) THEN
C
C          EVALUATE FUNCTIONS
C
C          CALL FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
C          NFEVAL = NFEVAL + 1
C          IF(IFERR.EQ.0) THEN
C              cbar(1:mcon) = quant(1:mcon)
C              FBAR = QUANT(MCON+1)
C          ENDIF
C
C          ENDIF
C
C          >>>>GRADIENT EVALUATION REQUEST
C
C          IF(IREVRS(2).EQ.1) THEN
C
C              ICCD = -1
C
C 130  CONTINUE
C
```

```

C          FORWARD DIFFERENCE GRADIENTS
C
C          IDTYPE = 1
C          CALL HDSFDJ(ICCD,IROWQ,JCOLQ,QMAT,
$           PRTSZF,PRTMIN,XBAR,QUANT,IFERR,MSGVLV,
$           IPU,IDTYPE,PRTMIN,INDSET,HOLDQ,NHOLDQ,IERD)
C
C          IF(ICCD.NE.0) THEN
C
C              CALL FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
C              NFEVAL = NFEVAL + 1
C
C              GO TO 130
C
C          ENDIF
C          gmat(1:nonzg) = qmat(1:nonzg)
C          delf(1:ndim) = 0.d0
C          delf(3:4) = qmat(nonzg+1:nonzg+2)
C
C          ELSEIF(IREVRS(2).EQ.2.AND.IREVRS(3).EQ.0) THEN
C
C              ICCD = -1
C
C          140 CONTINUE
C
C          CENTRAL DIFFERENCE GRADIENTS
C
C          IDTYPE = 2
C          CALL HDSFDJ(ICCD,IROWQ,JCOLQ,QMAT,
$           PRTSZC,PRTMIN,XBAR,QUANT,IFERR,MSGVLV,
$           IPU,IDTYPE,PRTMIN,INDSET,HOLDQ,NHOLDQ,IERD)
C
C          IF(ICCD.NE.0) THEN
C
C              CALL FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
C              NFEVAL = NFEVAL + 1
C
C              GO TO 140
C
C          ENDIF
C          gmat(1:nonzg) = qmat(1:nonzg)
C          delf(1:ndim) = 0.d0
C          delf(3:4) = qmat(nonzg+1:nonzg+2)
C
C          ENDIF
C
C          >>>>HESSIAN EVALUATION REQUEST

```

```

C
      IF (IREVRS(2).EQ.2.AND.IREVRS(3).EQ.1) THEN
C
      ICCH = -1
      cmult(1:mcon) = -veclam(1:mcon)
      CMULT(MCON+1) = 1.D0
C
150    CONTINUE
C
      DIAGONAL ELEMENTS
C
      CALL HDSFDH(ICCH,IROWQ,JCOLQ,QMAT,
$       JSTRH,IROWH,HMAT,CMULT,
$       PRSZC,PRTMIN,XBAR,QUANT,IFERR,MSGGLVL,IPU,
$       PRTMIN,INDSET,HOLDQ,NHOLDQ,IHOLDQ,
$       NIHLQ,IERH)
C
      IF(ICCH.GT.0) THEN
C
      CALL FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
      NFEVAL = NFEVAL + 1
C
      GO TO 150
C
      ENDIF
      gmat(1:nonzg) = qmat(1:nonzg)
      delf(1:ndim) = 0.d0
      delf(3:4) = qmat(nonzg+1:nonzg+2)
C
      ELSEIF (IREVRS(3).EQ.2) THEN
C
      ICCH = -2
C
160    CONTINUE
C
      OFF-DIAGONAL ELEMENTS
C
      CALL HDSFDH(ICCH,IROWQ,JCOLQ,QMAT,
$       JSTRH,IROWH,HMAT,CMULT,
$       PRSZC,PRTMIN,XBAR,QUANT,IFERR,MSGGLVL,IPU,
$       PRTMIN,INDSET,HOLDQ,NHOLDQ,IHOLDQ,
$       NIHLQ,IERH)
C
      IF(ICCH.GT.0) THEN
C
      CALL FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
      NFEVAL = NFEVAL + 1
      GO TO 160

```

```

C          ENDIF
C
C          ENDIF
C
C          >>>>>SYSTEM PRINT (OUTPUT) REQUEST
C
C          IF(IREVRS(4).EQ.1) THEN
C
C              WRITE(6,1003) FBAR,NFEVAL
C              WRITE(6,1001) 'VARIABLES'
C              WRITE(6,1002) (XBAR(II),II=1,NDIM)
C              IF(MCON.GT.0) THEN
C                  WRITE(6,1001) 'CONSTRAINTS'
C                  WRITE(6,1002) (CBAR(II),II=1,MCON)
C              ENDIF
C              WRITE(6,1004)
C
C          ENDIF
C
C          -----
C
C          END OF REVERSE COMMUNICATION PROCESSING.  RETURN TO SNLPMN
C
C          GO TO 120
C
C          170  CONTINUE
C
C          TERMINATE PROCESSING
C
C          IF(IERNLP.EQ.0) THEN
C
C              NORMAL TERMINATION---SOLUTION FOUND
C
C              WRITE(6,1001) 'SOLUTION    '
C
C              WRITE(6,1003) FBAR,NFEVAL
C              WRITE(6,1001) 'VARIABLES'
C              WRITE(6,1002) (XBAR(II),II=1,NDIM)
C              IF(MCON.GT.0) THEN
C                  WRITE(6,1001) 'CONSTRAINTS'
C                  WRITE(6,1002) (CBAR(II),II=1,MCON)
C              ENDIF
C              WRITE(6,1004)
C
C          ELSEIF(IERNLP.LT.0) THEN
C
C              ABNORMAL TERMINATION---FATAL ERROR

```

```

C
      PRINT *, 'FATAL ERROR; IERNLP = ', IERNLP
C
      ELSE
C
      ABNORMAL TERMINATION---WARNING.  EVALUATION OF RESULTS RECOMMENDED!
C
      PRINT *, 'WARNING MESSAGE; IERNLP = ', IERNLP
C
      ENDIF
C
1001 FORMAT(/, T20, A, /)
1002 FORMAT(T5, 5G16.6)
1003 FORMAT(/, T20, 'OBJECTIVE = ', G16.6, T60, 'NFEVAL = ', I5)
1004 FORMAT(/)
C
      STOP
      END

      SUBROUTINE FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
      DIMENSION XBAR(NDIM),QUANT(MCON+1)
C
      IFERR = 0
C
      QUANT(1) = XBAR(1) + XBAR(2)
      QUANT(2) = XBAR(2) + XBAR(3)
C
      QUANT(MCON+1) = (XBAR(3) - sin(XBAR(4)))**2
C
      RETURN
      END

```

OUTPUT FROM SAMPLE PROGRAM

INDEX SET WARNING; IERIND = 5

Feasible Point							
It	Qit	Nkt	Ndof	KT Cond	Step	Norm p	Violtn
1	3	1	2	3.1E+00	1.0E+00	2.9E+00	4.1E+00
2	0	0	2	3.1E+00	1.0E+00	2.9E+00	3.1E-09

Optimization											
It	Qit	Nkt	Ndof	KT Cond	Step	Norm p	Violtn	Levnbrg	Penalty	Norm Pg	Merit Function
(1)	3	9	2	3.4E+01	1.0E+00	5.9E-01	3.0E-09	1.0E+00	1.5E-08	1.3E+00	1.380165E+00
(2)	1	1	2	9.0E+00	1.0E+00	5.5E-01	6.5E-14	5.0E-01	1.5E-08	1.1E+00	4.073135E-01
(3)	1	1	2	1.1E+02	1.0E+00	4.5E-02	6.5E-14	4.6E-02	1.5E-08	1.0E-01	2.729025E-03

[4]	1	1	2	5.2E+03	1.0E+00	9.1E-04	2.8E-17	9.0E-04	1.5E-08	2.1E-03	1.074082E-06
[5]	1	2	2	2.8E+06	1.0E+00	4.9E-05	0.0E+00	3.9E-06	1.5E-08	8.9E-07	2.003607E-13
(6)	0	0	2	2.8E+06	1.0E+00	4.9E-05	0.0E+00	3.9E-06	1.5E-08	9.7E-11	3.257546E-21

```

-----
|
|               SPRNLP ALGORITHM PERFORMANCE STATISTICS
|
| Total CPU Time.....+2.210767E-02
|
| Number of Function Calls.....7
| Number of Gradient Calls.....8
| Number of Hessian Calls.....5
| Total Number of Function Evaluations.....40
|
| Storage Needed in HOLD Array.....385
| Storage Needed in IHOLD Array.....27
|
|
|-----

```

SOLUTION

OBJECTIVE = 0.325755E-20 NFEVAL = 40

VARIABLES

-0.106500 0.106500 -0.106500 -0.106702

CONSTRAINTS

0. 0.

4.4 Sparse Finite Differences with Sparse Least Squares

This example illustrates the use of the sparse finite difference derivative evaluation subroutines HDSFDJ and HDSFDH, which require using the sparse index set routine HJSFDI. The derivatives are evaluated as required by the sparse optimization routine SNLPLS.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2, x_3, x_4)$ to minimize the least squares objective function

$$f(x) = \sum_{i=1}^4 (x_3 - x_4 - i)^2$$

subject to the constraints

$$\begin{aligned}\sin(x_1) + x_2 &= 0 \\ \cos(x_2) + x_3 &= 0\end{aligned}$$

beginning at the initial point $x = (-1, 2, 2, 1)$.

SAMPLE PROGRAM (adv3.cd1)³

```

PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE SPARSE NONLINEAR
C      LEAST SQUARES ALGORITHM SNLPLS
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      PARAMETER (NDIM=4, MCON=2, NRES = 4)
C      PARAMETER (NONZH=7, NONZG=4, NONZR=8)
C      PARAMETER (NQNT = MCON + NRES)
C      PARAMETER (NHOLD=5000,NIHOLD=5000)
C      PARAMETER (NHOLDQ=5000,NIHLDQ=1000,NINSET=1000)
C
C      DIMENSION IREVRS(5),XBAR(NDIM),XLWR(NDIM),XUPR(NDIM),
$      ISTATV(NDIM),VECNU(NDIM),HMAT(NONZH),
$      IROWH(NONZH),JSTRH(NDIM+1),CBAR(MCON),CLWR(MCON),
$      CUPR(MCON),ISTATC(MCON),VECLAM(MCON),
$      GMAT(NONZG),IROWG(NONZG),JCOLG(NONZG)
C      DIMENSION RESVEC(NRES),RMAT(NONZR),IROWR(NONZR),JCOLR(NONZR)
C      DIMENSION HOLD(NHOLD),IHOLD(NIHOLD)
C
C      DIMENSION CMULT(NQNT),QUANT(NQNT),QMAT(NONZG+NONZR)
C      DIMENSION IROWQ(NONZG+NONZR),JCOLQ(NONZG+NONZR)
C      DIMENSION HOLDQ(NHOLDQ),IHOLDQ(NIHLDQ),INDSET(NINSET)

```

³sosexample.tar


```

      DIMENSION PRTSZC(NDIM),PRTSZF(NDIM)
C
      EXTERNAL FUNBOX
      IPU = 6
C
C -----
C
C      INDEPENDENT VARIABLE DATA
C
C      ----INITIAL GUESS
      XBAR(1) = -1.0D0
      XBAR(2) = 2.0D0
      XBAR(3) = 2.0D0
      XBAR(4) = 1.0D0
C
C      ----BOUNDS
      BIGBND = 1.D0/HDMCON(5)
      xlwr(1:ndim) = -bigbnd
      xupr(1:ndim) = +bigbnd
C
C      ----VARIABLE STATUS
      istatv(1:ndim) = 0
C
C -----
C
C      OBJECTIVE FUNCTION DATA
C
C      ----HESSIAN SPARSITY PATTERN (ROW INDICES)
      IROWH(1) = 1
      IROWH(2) = 2
      IROWH(3) = 2
      IROWH(4) = 3
      IROWH(5) = 3
      IROWH(6) = 4
      IROWH(7) = 4
C
C      ----HESSIAN SPARSITY PATTERN (COLUMN START INDICES)
      JSTRH(1) = 1
      JSTRH(2) = 3
      JSTRH(3) = 5
      JSTRH(4) = 7
      JSTRH(5) = 8
C
C      ----RESIDUAL JACOBIAN SPARSITY PATTERN (ROW INDICES)
      IROWR(1) = 1
      IROWR(2) = 2
      IROWR(3) = 3
      IROWR(4) = 4
      IROWR(5) = 1
      IROWR(6) = 2
      IROWR(7) = 3
      IROWR(8) = 4

```

```

C      ----RESIDUAL JACOBIAN SPARSITY PATTERN (COLUMN INDICES)
C      JCOLR(1) = 3
C      JCOLR(2) = 3
C      JCOLR(3) = 3
C      JCOLR(4) = 3
C      JCOLR(5) = 4
C      JCOLR(6) = 4
C      JCOLR(7) = 4
C      JCOLR(8) = 4
C
C      -----
C
C      CONSTRAINT DATA
C
C      ----BOUNDS
C      CLWR(1) = 0.DO
C      CUPR(1) = 0.DO
C      CLWR(2) = 0.DO
C      CUPR(2) = 0.DO
C
C      ----CONSTRAINT STATUS
C      istatc(1:mcon) = 3
C
C      ----JACOBIAN SPARSITY PATTERN (ROW INDICES)
C      IROWG(1) = 1
C      IROWG(2) = 1
C      IROWG(3) = 2
C      IROWG(4) = 2
C
C      ----JACOBIAN SPARSITY PATTERN (COLUMN INDICES)
C      JCOLG(1) = 1
C      JCOLG(2) = 2
C      JCOLG(3) = 2
C      JCOLG(4) = 3
C
C      -----
C
C      CONSTRUCT SPARSITY PATTERN FOR AUGMENTED JACOBIAN ARRAY.
C      BY CONSTRUCTION THE FIRST MCON ROWS CONTAIN THE CONSTRAINT
C      JACOBIAN MATRIX. THE LAST NRES ROWS OF THE AUGMENTED JACOBIAN
C      CONTAIN THE RESIDUALS
C
C      ----AUGMENTED JACOBIAN SPARSITY PATTERN
C
C      irowq(1:nonzg) = irowg(1:nonzg)
C      jcolq(1:nonzg) = jcolg(1:nonzg)
C
C      NFDQNZ = NONZG
C      DO 110 I = 1,NONZR
C          NFDQNZ = NFDQNZ + 1
C          IROWQ(NFDQNZ) = IROWR(I) + MCON

```

```

        JCOLQ(NFDQNZ) = JCOLR(I)
110  CONTINUE
C
C        GENERATE INDEX SETS
C
        LCRMAT = NFDQNZ - NONZR + 1
        IHESS = 1
        NFDHNZ = NONZH
        MAXHNZ = NONZH
        MSGLVL = 0
        CALL HJSFDI(IHESS,NDIM,NQNT,NFDQNZ,IROWQ,JCOLQ,
$  NFDHNZ,MAXHNZ,NDIM+1,JSTRH,IROWH,MSGLVL,IPU,
$  INDSET,NINSET,IERIND)
C
        IF(IERIND.LT.0) THEN
            PRINT *, 'INDEX SET ERROR; IERIND =', IERIND
            STOP
        ELSEIF(IERIND.GT.0) THEN
            PRINT *, 'INDEX SET WARNING; IERIND =', IERIND
        ENDIF
C
C        DEFINE PERTURBATION SIZES
C
        PRTMIN = SQRT(HDMCON(5))
        EPSRLF = HDMCON(5)**.9
        CNTINT = EPSRLF**.333
        FWDINT = SQRT(EPSRLF)
        DO 120 II = 1,NDIM
            VARSIZ = 1.DO + ABS(XBAR(II))
            PRSZC(II) = CNTINT*VARSIZ
            PRSZF(II) = FWDINT*VARSIZ
120  CONTINUE
C
C -----
C
C        INITIALIZE FUNCTION EVALUATION COUNT
C
        NFEVAL = 0
C
C        INITIALIZE REVERSE COMMUNICATION CONTROL FLAG
C
        ICC = -1
C
C        SET OPTIONAL INPUTS
C
        CALL INSNLP('IOFLAG=1')
C
C -----

```

```

C
130  CONTINUE
C
      CALL SNLPLS( ICC ,IREVRS ,XBAR   ,XLWR   ,XUPR   ,ISTATV
$          ,VEGNU  ,NDIM   ,RESVEC  ,NRES   ,RMAT   ,IROWR
$          ,JCOLR  ,NONZR  ,HMAT   ,IROWH  ,JSTRH  ,NONZH
$          ,CBAR   ,CLWR   ,CUPR   ,ISTATC  ,MCON   ,VECLAM
$          ,GMAT   ,IROWG  ,JCOLG  ,NONZG  ,IFERR  ,NFEVAL
$          ,HOLD   ,NHOLD  ,IHOLD  ,NIHOLD ,NEEDED ,IERNLP )
C
C      REVERSE COMMUNICATION TERMINATION TEST
C
      IF(ICC.EQ.0) GO TO 180
C
C -----
C
C      SET FUNCTION EVALUATION ERROR FLAG
C
      IFERR = 0
C
C      >>>>FUNCTION EVALUATION REQUEST
C
      IF(IREVRS(1).EQ.1) THEN
C
C      EVALUATE FUNCTIONS
C
      CALL FUNBOX(XBAR,NDIM,NQNT,QUANT,IFERR)
      NFEVAL = NFEVAL + 1
      IF(IFERR.EQ.0) THEN
          cbar(1:mcon) = quant(1:mcon)
          resvec(1:nres) = quant(mcon+1:mcon+nres)
      ENDIF
C
      ENDIF
C
C      >>>>GRADIENT EVALUATION REQUEST
C
      IF(IREVRS(2).EQ.1) THEN
C
C      ICCD = -1
C
140  CONTINUE
C
C      FORWARD DIFFERENCE GRADIENTS
C
      IDTYPE = 1
      CALL HDSFDJ(ICCD,IROWQ,JCOLQ,QMAT,
$          PRSZF,PRTMIN,XBAR,QUANT,IFERR,MSGGLVL,

```

```

C      $      IPU, IDTYPE, PRTMIN, INDSET, HOLDQ, NHOLDQ, IERD)
C
C      IF(ICCD.NE.0) THEN
C
C          CALL FUNBOX(XBAR,NDIM,NQNT,QUANT,IFERR)
C          NFEVAL = NFEVAL + 1
C
C          GO TO 140
C
C      ENDIF
C      if(mcon.gt.0) gmat(1:nonzg) = qmat(1:nonzg)
C      kk = lcrmat-1
C      do ii = 1,nonzr
C          kk = kk+1
C          rmat(ii) = qmat(kk)
C      enddo
C
C      ELSEIF(IREVRS(2).EQ.2.AND.IREVRS(3).EQ.0) THEN
C
C          ICCD = -1
C
C          150 CONTINUE
C
C          CENTRAL DIFFERENCE GRADIENTS
C
C          IDTYPE = 2
C          CALL HDSFDJ(ICCD,IROWQ,JCOLQ,QMAT,
C      $      PRTSZC,PRTMIN,XBAR,QUANT,IFERR,MSGVLV,
C      $      IPU, IDTYPE, PRTMIN, INDSET, HOLDQ, NHOLDQ, IERD)
C
C
C      IF(ICCD.NE.0) THEN
C
C          CALL FUNBOX(XBAR,NDIM,NQNT,QUANT,IFERR)
C          NFEVAL = NFEVAL + 1
C
C          GO TO 150
C
C      ENDIF
C      if(mcon.gt.0) gmat(1:nonzg) = qmat(1:nonzg)
C      kk = lcrmat-1
C      do ii = 1,nonzr
C          kk = kk+1
C          rmat(ii) = qmat(kk)
C      enddo
C
C      ENDIF
C

```

```

C      >>>>HESSIAN EVALUATION REQUEST
C
C      IF(IREVRS(2).EQ.2.AND.IREVRS(3).EQ.1) THEN
C
C          ICCH = -1
C          cmult(1:mcon) = -veclam(1:mcon)
C          cmult(mcon+1:mcon+nres) = resvec(1:nres)
C
C      160      CONTINUE
C
C          DIAGONAL ELEMENTS
C
C          CALL HDSFDH(ICCH,IROWQ,JCOLQ,QMAT,
C          $          JSTRH,IROWH,HMAT,CMULT,
C          $          PRSZC,PRTMIN,XBAR,QUANT,IFERR,MSGLVL,IPU,
C          $          PRTMIN,INDSET,HOLDQ,NHOLDQ,IHOLDQ,
C          $          NIHLQ,IERH)
C
C          IF(ICCH.GT.0) THEN
C
C              CALL FUNBOX(XBAR,NDIM,NQNT,QUANT,IFERR)
C              NFEVAL = NFEVAL + 1
C
C              GO TO 160
C
C          ENDIF
C          if(mcon.gt.0) gmat(1:nonzg) = qmat(1:nonzg)
C          kk = lcrmat-1
C          do ii = 1,nonzr
C              kk = kk+1
C              rmat(ii) = qmat(kk)
C          enddo
C
C      ELSEIF(IREVRS(3).EQ.2) THEN
C
C          ICCH = -2
C
C      170      CONTINUE
C
C          OFF-DIAGONAL ELEMENTS
C
C          CALL HDSFDH(ICCH,IROWQ,JCOLQ,QMAT,
C          $          JSTRH,IROWH,HMAT,CMULT,
C          $          PRSZC,PRTMIN,XBAR,QUANT,IFERR,MSGLVL,IPU,
C          $          PRTMIN,INDSET,HOLDQ,NHOLDQ,IHOLDQ,
C          $          NIHLQ,IERH)
C
C          IF(ICCH.GT.0) THEN

```

```

C          CALL FUNBOX(XBAR,NDIM,NQNT,QUANT,IFERR)
C          NFEVAL = NFEVAL + 1
C          GO TO 170
C
C      ENDIF
C
C      if(mcon.gt.0) gmat(1:nonzg) = qmat(1:nonzg)
C      kk = lcrmat-1
C      do ii = 1,nonzr
C          kk = kk+1
C          rmat(ii) = qmat(kk)
C      enddo
C
C      ENDIF
C
C      >>>>SYSTEM PRINT (OUTPUT) REQUEST
C
C      IF(IREVRS(4).EQ.1) THEN
C
C          WRITE(6,1003) .5D0*dot_product(RESVEC,RESVEC),NFEVAL
C          WRITE(6,1001) 'VARIABLES'
C          WRITE(6,1002) (XBAR(II),II=1,NDIM)
C          IF(MCON.GT.0) THEN
C              WRITE(6,1001) 'CONSTRAINTS'
C              WRITE(6,1002) (CBAR(II),II=1,MCON)
C          ENDIF
C          WRITE(6,1004)
C
C      ENDIF
C
C      -----
C
C      END OF REVERSE COMMUNICATION PROCESSING.  RETURN TO SNLPLS
C
C      GO TO 130
C
C      180  CONTINUE
C
C      TERMINATE PROCESSING
C
C      IF(IERNLP.EQ.0) THEN
C
C          NORMAL TERMINATION---SOLUTION FOUND
C
C          WRITE(6,1001) 'SOLUTION      '
C
C          WRITE(6,1003) .5D0*dot_product(RESVEC,RESVEC),NFEVAL

```

```

        WRITE(6,1001) 'VARIABLES'
        WRITE(6,1002) (XBAR(II),II=1,NDIM)
        IF(MCON.GT.0) THEN
            WRITE(6,1001) 'CONSTRAINTS'
            WRITE(6,1002) (CBAR(II),II=1,MCON)
        ENDIF
        WRITE(6,1004)
C
        ELSEIF(IERNLP.LT.0) THEN
C
C            ABNORMAL TERMINATION---FATAL ERROR
C
            PRINT *, 'FATAL ERROR; IERNLP =', IERNLP
C
        ELSE
C
C            ABNORMAL TERMINATION---WARNING.  EVALUATION OF RESULTS RECOMMENDED!
C
            PRINT *, 'WARNING MESSAGE; IERNLP =', IERNLP
C
        ENDIF
C
1001 FORMAT(/,T20,A,/)
1002 FORMAT(T5,5G16.6)
1003 FORMAT(/,T20,'OBJECTIVE =',G16.6,T60,'NFEVAL =',I5)
1004 FORMAT(/)
C
        STOP
        END

        SUBROUTINE FUNBOX(XBAR,NDIM,NQNT,QUANT,IFERR)
C
        IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
        DIMENSION XBAR(NDIM),QUANT(NQNT)
C
        IFERR = 0
C
        QUANT(1) = SIN(XBAR(1)) + XBAR(2)
        QUANT(2) = COS(XBAR(2)) + XBAR(3)
C
        QUANT(3) = (XBAR(3) - XBAR(4) - 1.DO)
        QUANT(4) = (XBAR(3) - XBAR(4) - 2.DO)
        QUANT(5) = (XBAR(3) - XBAR(4) - 3.DO)
        QUANT(6) = (XBAR(3) - XBAR(4) - 4.DO)
C
        RETURN
        END

```


OUTPUT FROM SAMPLE PROGRAM

Feasible Point								
It	Qit	Nkt	Ndof	KT Cond	Step	Norm p	Violtn	
1	3	1	2	3.4E+00	1.0E+00	2.4E+00	2.0E+00	
2	3	1	2	4.9E+00	1.0E+00	8.0E-01	7.4E-01	
3	3	1	2	5.8E+00	1.0E+00	1.5E-01	1.2E-01	
4	3	1	2	5.4E+00	1.0E+00	3.0E-03	3.4E-03	
5	3	1	2	5.4E+00	1.0E+00	4.7E-08	6.7E-08	
6	0	0	2	5.4E+00	1.0E+00	4.7E-08	2.4E-15	

Optimization											
It	Qit	Nkt	Ndof	KT Cond	Step	Norm p	Violtn	Levnbrg	Penalty	Norm Pg	Merit Function
(1)	1	1	2	1.1E+02	1.0E+00	4.0E+00	2.4E-15	0.0E+00	1.5E-08	1.6E+01	3.524696E+01
(2)	0	0	2	1.1E+02	1.0E+00	4.0E+00	2.4E-15	0.0E+00	1.5E-08	5.7E-11	2.500000E+00

SPRNLP ALGORITHM PERFORMANCE STATISTICS											
Total CPU Time.....+1.000000E-03											
Number of Function Calls.....7											
Number of Gradient Calls.....8											
Number of Hessian Calls.....1											
Total Number of Function Evaluations.....28											
Storage Needed in HOLD Array.....430											

SOLUTION

OBJECTIVE = 2.50000 NFEVAL = 28

VARIABLES

-1.69156 0.992718 -0.546416 -3.04642

CONSTRAINTS

0.111022E-15 -0.244249E-14

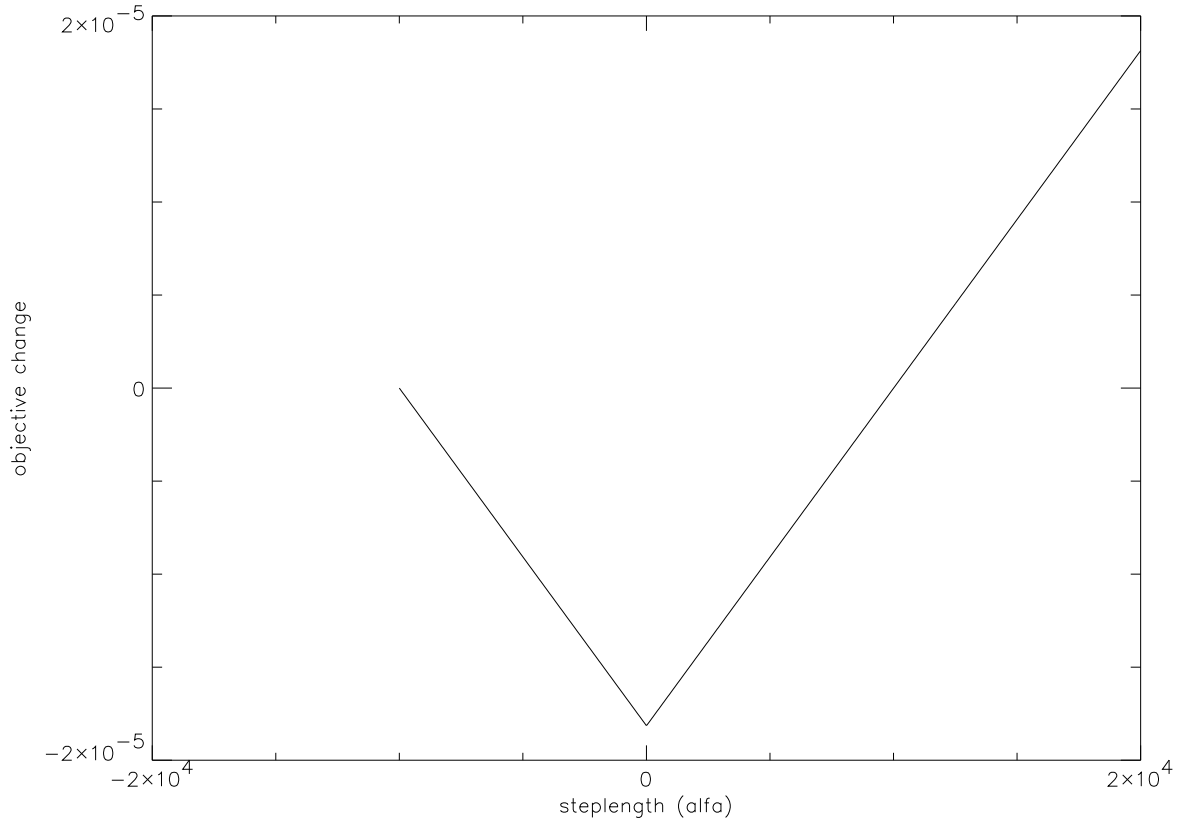


Figure 4.1: Line Search Diagnostic Plot

4.5 Diagnostic Line Search Output

This example illustrates the use of the line search output capability to diagnose problem formulation difficulties. The code demonstrates the application on an example intentionally formulated with a discontinuity at the solution.

EXAMPLE

Find the values of the independent variables $x = (x_1, x_2, x_3, x_4)$ to minimize the objective function

$$f(x) = \sum_{i=1}^4 |x_i - i|$$

beginning at the initial point $x = (1, -1, 1, 1)$. It is easily demonstrated that the objective function is not differentiable at the minimum $(1, 2, 3, 4)$, and because of this the optimization algorithm will terminate abnormally. In this instance a previous run indicates that after iteration number 33, the NLP will terminate with an error indicating that no further progress can be made because the steps are too small. This occurs in the neighborhood of the solution. This example illustrates how to obtain diagnostic output from the linesearch at iteration 33. This example creates a file containing the values of the objective function along the search direction which is displayed below.

SAMPLE PROGRAM (adv4.cd1)⁴

```

      PROGRAM EXAMPL
C
C      PROGRAM TO ILLUSTRATE THE USE OF THE NONLINEAR
C      PROGRAMMING ALGORITHM DNLPFW
C
C-----
C
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
      PARAMETER ( NHOLD=10000, NIHOLD=1000 )
      PARAMETER ( ZERO=0.D0, ONE=1.D0, POINT5 = .5D0, TWO = 2.D0)
      PARAMETER (MAXDIM = 5, MAXCON = 10)
C
C      NLP CALLING ARGUMENTS
C
      DIMENSION XBAR(MAXDIM),XLWR(MAXDIM),XUPR(MAXDIM),
$      ISTATV(MAXDIM),VECNU(MAXDIM),
$      CBAR(MAXCON),CLWR(MAXCON),
$      CUPR(MAXCON),ISTATC(MAXCON),VECLAM(MAXCON)
      DIMENSION HOLD(NHOLD),IHOLD(NIHOLD)
      EXTERNAL FUNBOX
C
C      DEFINE INDEPENDENT VARIABLE DATA
C
      NDIM = 4
      BIGBND = 1.D0/HDMCON(5)
      XBAR(1) = 1.0D0
      XBAR(2) = -1.0D0
      XBAR(3) = 1.0D0
      XBAR(4) = 1.0D0
C
      ----BOUNDS
      xlwr(1:ndim) = -bigbnd
      xupr(1:ndim) = +bigbnd
C
      ----VARIABLE STATUS
      istatv(1:ndim) = 0
C
C      DEFINE CONSTRAINT DATA
C
      MCON = 0
C
C-----
C
C      CALL NLP TO SOLVE THE PROBLEM
C
      CALL INSNLP('DENSE DEFAULT')

```

⁴sosexample.tar

```

CALL INSNLP('IOFLAG=1')
CALL INSNLP('LYNOUT=30')
CALL INSNLP('LYNPLT=33')
CALL INSNLP('LYNPNT=1001')
CALL INSNLP('ALFLWR=-1.D4')
CALL INSNLP('ALFUPR=2.D4')
C
CALL DNLPCFW(FUNBOX, XBAR, XLWR, XUPR, ISTATV, VECNU, NDIM,
$   FBAR, CBAR, CLWR, CUPR, ISTATC, VECLAM, MCON,
$   HOLD, NHOLD, IHOLD, NIHOLD, NEEDED, IERNLP)
C
STOP
END

SUBROUTINE FUNBOX(XBAR,NDIM,MCON,QUANT,IFERR)
C
IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
DIMENSION XBAR(NDIM),QUANT(MCON+1)
C
IFERR = 0
C
QUANT(MCON+1) = ABS(XBAR(1) - 1.DO) + ABS(XBAR(2) - 2.DO)
$              + ABS(XBAR(3) - 3.DO) + ABS(XBAR(4) - 4.DO)
C
RETURN
END

```

OUTPUT FROM SAMPLE PROGRAM

It	Qit	Nkt	Ndof	PH Cond	Step	Optimization						
						Norm p	Violtn	Levnbrg	Penalty	Norm Pg	Merit	Function
1	2	1	4	1.0E+00	1.0E+00	2.0E+00	0.0E+00	0.0E+00	1.5E-08	1.0E+00	8.000000E+00	
2	2	2	4	*****	1.0E+00	3.4E+08	0.0E+00	5.0E-01	1.5E-08	1.0E+00	6.000000E+00	
3	2	1	4	2.0E+00	5.3E-09	2.0E+00	0.0E+00	1.0E+00	1.5E-08	1.0E+00	2.365071E+00	
4	2	1	4	2.6E+00	1.0E+00	8.9E-01	0.0E+00	1.0E+00	1.5E-08	1.0E+00	2.000000E+00	
5	2	1	4	5.0E+00	3.9E-01	3.7E-01	0.0E+00	1.0E+00	1.5E-08	1.0E+00	1.949436E+00	
6	2	1	4	6.5E+00	1.0E+00	1.6E+00	0.0E+00	5.0E-01	1.5E-08	1.0E+00	1.295372E+00	
7	2	1	4	1.1E+01	4.4E-01	9.4E-01	0.0E+00	5.0E-01	1.5E-08	1.0E+00	4.788027E-01	
8	2	1	4	7.6E+00	3.0E-01	1.6E-01	0.0E+00	1.0E+00	1.5E-08	1.0E+00	4.255959E-01	
9	2	1	4	5.5E+00	1.0E+00	2.6E-02	0.0E+00	5.0E-01	1.5E-08	1.0E+00	1.234991E-01	
10	2	1	4	5.6E+00	1.0E+00	2.3E-03	0.0E+00	5.0E-01	1.5E-08	1.0E+00	1.080893E-01	
11	2	1	4	1.5E+01	1.0E+00	2.5E-01	0.0E+00	2.5E-01	1.5E-08	1.0E+00	1.039200E-01	
12	2	1	4	6.6E+01	3.2E-01	4.0E-02	0.0E+00	2.5E-01	1.5E-08	1.0E+00	4.461692E-02	
13	2	1	4	1.4E+01	1.0E+00	1.1E-02	0.0E+00	2.5E-01	1.5E-08	1.0E+00	3.420700E-02	
14	2	2	4	1.0E+09	1.0E+00	4.2E+05	0.0E+00	9.8E-01	1.5E-08	1.0E+00	2.002572E-02	
15	2	1	4	3.0E+00	5.3E-09	2.1E-02	0.0E+00	1.0E+00	1.5E-08	1.0E+00	1.553435E-02	
16	2	1	4	7.1E+00	3.9E-01	6.9E-03	0.0E+00	1.0E+00	1.5E-08	1.0E+00	1.353079E-02	
17	2	1	4	1.3E+01	1.0E+00	2.9E-03	0.0E+00	5.0E-01	1.5E-08	1.0E+00	9.273834E-03	
18	2	1	4	1.2E+01	1.0E+00	7.2E-04	0.0E+00	2.5E-01	1.5E-08	1.0E+00	3.682817E-03	
19	2	1	4	9.4E+00	1.0E+00	2.9E-05	0.0E+00	2.5E-01	1.5E-08	1.0E+00	3.589105E-03	
20	2	1	4	2.9E+01	1.0E+00	1.4E-02	0.0E+00	1.2E-01	1.5E-08	1.0E+00	3.537767E-03	

21	2	1	4	3.3E+01	1.3E-01	2.8E-03	0.0E+00	2.5E-01	1.5E-08	1.0E+00	5.934168E-04
22	2	1	4	3.6E+01	9.3E-02	7.2E-03	0.0E+00	5.0E-01	1.5E-08	1.0E+00	2.507637E-04
23	2	1	4	4.5E+02	5.2E-03	2.2E-04	0.0E+00	1.0E+00	1.5E-08	1.0E+00	2.189913E-04
24	2	1	4	1.5E+01	1.0E+00	7.4E-05	0.0E+00	1.0E+00	1.5E-08	1.0E+00	1.121328E-04
25	2	1	4	1.4E+01	1.0E+00	2.8E-05	0.0E+00	5.0E-01	1.5E-08	1.0E+00	9.163464E-06
26	2	1	4	3.2E+00	3.1E-01	2.6E-06	0.0E+00	1.0E+00	1.5E-08	1.0E+00	6.301593E-06
27	2	1	4	5.1E+00	1.0E+00	1.5E-04	0.0E+00	5.0E-01	1.5E-08	1.0E+00	1.061976E-06
28	2	1	4	3.0E+00	5.4E-03	2.7E-07	0.0E+00	1.0E+00	1.5E-08	1.0E+00	5.766247E-07
29	2	1	4	1.0E+01	1.0E+00	2.5E-07	0.0E+00	3.8E-01	1.5E-08	3.8E-01	2.553404E-07
(30)	2	1	4	2.0E+03	2.0E+00	2.3E-08	0.0E+00	1.8E-03	1.5E-08	1.8E-03	2.553404E-07
(31)	2	1	4	4.9E+03	1.0E+00	1.0E-08	0.0E+00	8.8E-04	1.5E-08	2.1E-03	2.512176E-07
(32)	2	1	4	5.4E+03	1.0E+00	1.1E-07	0.0E+00	4.4E-04	1.5E-08	1.9E-03	2.510239E-07
(33)	2	1	4	1.4E+04	1.0E+00	1.0E-06	0.0E+00	1.8E-04	1.5E-08	8.0E-04	9.006959E-08

0***** WARNING REPORTED BY SUBROUTINE HDSNLP
SEE HDSNLP ABSTRACT (IER = 119)

***** NLP ERROR from subroutine HDSNLP , IER = 119
TERMINATE AFTER DIAGNOSTIC LINE SEARCH

```

-----
|
|               SPRNLP ALGORITHM PERFORMANCE STATISTICS
|
| Total CPU Time.....+0.114299
|
| Number of Function Calls.....1076
| Number of Gradient Calls.....33
| Number of Hessian Calls.....33
| Total Number of Function Evaluations.....1224
|
| Storage Needed in HOLD Array.....9925
| Storage Needed in IHOLD Array.....800
|
-----

```

```

***** FINAL POINT *****
*****
*
* Objective Function =      8.00000          IERNLP = 119
*
* -----
*
* Variable   Status   Value      Lower bound      Upper bound      Lagrange Mlt.      Slack
*
*      1      FR      1.00000     -4.503600E+15     4.503600E+15          0.      4.5036E+15
*      2      FR     -1.00000     -4.503600E+15     4.503600E+15          0.      4.5036E+15
*      3      FR      1.00000     -4.503600E+15     4.503600E+15          0.      4.5036E+15
*      4      FR      1.00000     -4.503600E+15     4.503600E+15          0.      4.5036E+15
*
*****
0***** PROCESS ERROR REPORTED BY SUBROUTINE HDNLPD
SEE HDNLPD ABSTRACT (IER = 119)

```


Chapter 5

Optimal Control Background

Problem Definition

An optimal control problem can be formulated as a collection of N *phases*. In general, the independent variable t for phase k is defined in the region $t_I^{(k)} \leq t \leq t_F^{(k)}$. For many applications the independent variable t is *time*, and the phases are sequential, that is $t_I^{(k+1)} = t_F^{(k)}$, however neither of these assumptions is required. Within phase k the dynamics of the system are described by a set of *dynamic* variables

$$z = \begin{bmatrix} y^{(k)}(t) \\ u^{(k)}(t) \end{bmatrix}$$

made up of the $n_y^{(k)}$ *state variables* and the $n_u^{(k)}$ *control variables* respectively. In addition the dynamics may incorporate the $n_p^{(k)}$ *parameters* $p^{(k)}$ which are not dependent on t . For clarity we drop the phase-dependent notation from the remaining discussion, however it is important to remember that many complex problem descriptions require different dynamics and/or constraints within each phase, and the approach accommodates this requirement.

Typically the dynamics of the system are defined by a set of ordinary differential equations written in explicit form, which are referred to as the *state equations*

$$\dot{y} = f[y(t), u(t), p, t]$$

where y is the n_y dimension state vector. Initial conditions at time t_I are defined by

$$\psi_{I\ell} \leq \psi[y(t_I), u(t_I), p, t_I] \leq \psi_{Iu}$$

where $\psi[y(t_I), u(t_I), p, t_I] \equiv \psi_I$ and terminal conditions at the final time t_F are defined by

$$\psi_{F\ell} \leq \psi[y(t_F), u(t_F), p, t_F] \leq \psi_{Fu}$$

where $\psi[y(t_F), u(t_F), p, t_F] \equiv \psi_F$. In addition the solution must satisfy *algebraic path constraints* of the form

$$g_\ell \leq g[y(t), u(t), p, t] \leq g_u,$$

where g is a vector of size n_g . The algebraic constraints are of the form

$$g[y(t), u(t), p, t] = \alpha^\top v + \beta^\top a[v, t]$$

where

$$v = \begin{bmatrix} y(t) \\ u(t) \\ p \end{bmatrix}.$$

The constraint definition can include *analytic* terms involving $\alpha^\top v$ where the $(n_y + n_u + n_p)$ vector α is constant, as well as linear combinations of the n_a *auxiliary functions* $a_k(v)$ for $k = 0, \dots, n_a$ where the coefficients β_k are nonzero constants. By convention, a path constraint with a single nonlinear term $a_0(y, u, p, t)$ has no auxiliary functions ($n_a = 0$). Observe that each individual path constraint may have a different number of auxiliary functions and analytic terms. Simple bounds can be imposed on the state variables

$$y_\ell \leq y(t) \leq y_u,$$

the control variables

$$u_\ell \leq u(t) \leq u_u,$$

and the parameters

$$p_\ell \leq p \leq p_u.$$

Note that an equality constraint can be imposed if the upper and lower bounds are equal, e.g. $(g_\ell)_k = (g_u)_k$ for some k .

Finally it may be convenient to evaluate expressions of the form

$$\int_{t_I}^{t_F} q[y(t), u(t), p, t] dt$$

which involve the *quadrature* functions q . Collectively we refer to those functions evaluated during the phase, namely,

$$F(t) = \begin{bmatrix} f[y(t), u(t), p, t] \\ g[y(t), u(t), p, t] \\ q[y(t), u(t), p, t] \\ a[y(t), u(t), p, t] \end{bmatrix}$$

as the vector of *continuous functions*. Similarly functions evaluated at a specific point (i.e. the phase boundary) such as the boundary conditions $\psi[y(t_I), u(t_I), p, t_I]$ and $\psi[y(t_F), u(t_F), p, t_F]$ are referred to as *point functions*.

The basic optimal control problem is to determine the $n_u^{(k)}$ -dimensional control vectors $u^{(k)}(t)$ and parameters $p^{(k)}$ to minimize the performance index

$$\begin{aligned} J = & \phi \left[y(t_I^{(1)}), t_I^{(1)}, y(t_F^{(1)}), p^{(1)}, t_F^{(1)}, \dots, y(t_I^{(N)}), t_I^{(N)}, y(t_F^{(N)}), p^{(N)}, t_F^{(N)} \right] \\ & + \sum_{k=1}^N \int_{t_I^{(k)}}^{t_F^{(k)}} q^{(k)}[y^{(k)}(t), u^{(k)}(t), p^{(k)}, t] dt. \end{aligned}$$

Notice that the objective function may depend on quantities computed in each of the N phases.

The above formulation can also accommodate implicit differential equations of the form,

$$f[y(t), \dot{y}(t), u(t), p, t] = 0.$$

A simple substitution turns the implicit equation above into the semi-explicit differential-algebraic (DAE) system,

$$\begin{aligned}\dot{y} &= w \\ 0 &= f[y(t), w(t), u(t), p, t]\end{aligned}$$

In the semi-explicit form, w are considered another set of algebraic variables, and the functions defining the implicit differential equations now become algebraic path conditions.

Transcription Formulation

The basic approach for solving the optimal control problem by transcription has been presented in detail elsewhere (cf. Ref [5, 6, 7]). All approaches divide the time interval into n_s segments

$$t_I < t_1 < t_2 < \dots < t_F = t_{n_s},$$

where the points are referred to as node, mesh or grid points. Define the number of mesh points as $M \equiv n_s + 1$. Let us introduce the notation $y_k \equiv y(t_k)$ to indicate the value of the state variable at a grid point. In like fashion denote the control at a grid point by $u_k \equiv u(t_k)$. In addition some discretization schemes require values for the variables at the midpoint of an interval, and we denote these quantities by $y_{k+1/2} \equiv y(\bar{t})$ and $u_{k+1/2} \equiv u(\bar{t})$ with $\bar{t} = \frac{1}{2}(t_k + t_{k-1})$. The SOS software provides a number of different discretization schemes. Each scheme produces a distinct set of nonlinear programming (NLP) variables and constraints.

For the trapezoidal discretization, the NLP variables are

$$x = [y_0, u_0, y_1, u_1, \dots, y_F, u_F, p, t_I, t_F]^\top.$$

The state equations are approximately satisfied by setting the *defects*

$$\zeta_k = y_k - y_{k-1} - \frac{h_k}{2} [f_k + f_{k-1}]$$

to zero for $k = 1, \dots, n_s$. The step size is denoted by $h_k \equiv t_k - t_{k-1}$, and the right hand side of the differential equations are given by $f_k \equiv f[y(t_k), u(t_k), p, t_k]$.

For the Compressed Hermite-Simpson discretization scheme, the NLP variables are

$$x = [y_0, u_0, u_{1/2}, y_1, u_1, u_{3/2}, \dots, y_F, u_F, p, t_I, t_F]^\top.$$

The defects for this discretization are given by

$$\zeta_k = y_k - y_{k-1} - \frac{h_k}{6} [f_k + 4f_{k+1/2} + f_{k-1}]$$

where

$$f_{k+1/2} = f[y_{k+1/2}, u_{k+1/2}, p, \bar{t}]$$

with

$$y_{k+1/2} = \frac{1}{2} [y_{k-1} + y_k] + \frac{h_k}{8} [f_{k-1} - f_k]$$

for $k = 1, \dots, n_s$.

An alternate form of this discretization scheme referred to as Separated Hermite-Simpson, has the following NLP variables

$$x = [y_0, u_0, y_{1/2}, u_{1/2}, y_1, u_1, y_{3/2}, u_{3/2}, \dots, y_F, u_F, p, t_I, t_F]^\top.$$

The defects for this discretization are given by

$$\zeta_k = y_k - y_{k-1} - \frac{h_k}{6} [f_k + 4f_{k+1/2} + f_{k-1}]$$

and

$$\bar{\zeta}_k = y_{k+1/2} - \frac{1}{2} [y_{k-1} + y_k] - \frac{h_k}{8} [f_{k-1} - f_k]$$

where

$$f_{k+1/2} = f[y_{k+1/2}, u_{k+1/2}, p, \bar{t}]$$

for $k = 1, \dots, n_s$.

For the fourth order Runge-Kutta discretization scheme, the NLP variables are the same as the Compressed Hermite-Simpson method. The defects for this discretization are given by

$$\zeta_k = y_k - y_{k-1} - \frac{1}{6} [k_1 + 2k_2 + 2k_3 + k_4]$$

where

$$\begin{aligned} k_1 &= h_k f(y_{k-1}, u_{k-1}, p, t_{k-1}) \\ k_2 &= h_k f(y_{k-1} + k_1/2, u_{k+1/2}, p, \bar{t}) \\ k_3 &= h_k f(y_{k-1} + k_2/2, u_{k+1/2}, p, \bar{t}) \\ k_4 &= h_k f(y_{k-1} + k_3, u_k, p, t_k) \end{aligned}$$

for $k = 1, \dots, n_s$.

As a result of the transcription the optimal control constraints are replaced by the NLP constraints

$$c_\ell \leq c(x) \leq c_u,$$

where

$$c(x) = [\zeta_1, \zeta_2, \dots, \zeta_F, \psi_I, \psi_F, \bar{g}_0, \bar{g}_1, \dots, \bar{g}_F]^\top$$

with

$$c_\ell = [0, \dots, 0, \psi_{I\ell}, \psi_{F\ell}, g_\ell, \dots, g_\ell]^\top$$

and a corresponding definition of c_u . The Separated Hermite-Simpson discretization also includes $\bar{\zeta}_k$ as constraints. The first $n_y n_s$ equality constraints require that the defect vectors from each of the n_s segments be zero thereby approximately satisfying the differential equations. The boundary conditions are enforced directly by the equality constraints on ψ , and the nonlinear path constraints are imposed at the grid points. By default the path constraints are imposed at the first and last grid point, however, this can be altered as described in the remarks (p. 441). Note that nonlinear equality path constraints are enforced by setting $c_\ell = c_u$.

In a similar fashion the state and control variable bounds become simple bounds on the NLP variables. The path constraints and variable bounds are always imposed at the grid points for all discretization schemes. For the Hermite-Simpson, and Runge-Kutta discretization methods the path constraints and variable bounds are also imposed at the interval midpoints.

The nonlinear programming (NLP) problem can be stated as follows: Find the n -vector x which minimizes the objective function J subject to the constraints on c . This large, sparse NLP can be solved efficiently using either a sequential quadratic programming (SQP) method or an interior point (barrier) method.

To summarize, there are three primary operations that are performed when solving an optimal control problem using a transcription method. Briefly the approach is as follows:

Direct Transcription *Transcribe* the optimal control problem into a nonlinear programming (NLP) problem by discretization;

Sparse Nonlinear Program Solve the sparse NLP using either a sequential quadratic programming or an interior-point (barrier) method.

Mesh Refinement Assess the accuracy of the approximation (i.e. the finite dimensional problem), and if necessary refine the discretization, and then repeat the optimization steps.

It is worth noting that the method is a “direct” transcription because it is not necessary to explicitly construct the optimal control necessary conditions, i.e. the adjoint equations and the Pontryagin Maximum Principle. However, if the adjoint equations are available the SOS software can be used to efficiently solve the boundary value problem.

The Parameter Estimation Problem

The parameter estimation problem has a different objective function than the optimal control problem. The basic parameter estimation problem is to determine the n_p -dimensional vector p to minimize the least squares performance index

$$J = \frac{1}{2} r^\top r.$$

where r is the ℓ -dimensional *residual* vector. Components of the residual vector can be of two forms. State residuals are of the form

$$r_k = w_{ij} [y_i(\theta_{ij}) - \hat{y}_{ij}]$$

where $y_i(\theta_{ij})$ is the value of state variable i computed at time θ_{ij} and $\hat{y}_{ij}$ is the observed value at the same point. Algebraic residuals are of the form

$$r_k = w_{ij} [u_i(\theta_{ij}) - \hat{u}_{ij}]$$

where $u_i(\theta_{ij})$ is the value of algebraic variable i computed at time θ_{ij} and $\hat{u}_{ij}$ is the observed value at the same point. The quantities w_{ij} are residual scale weights. It is required that data evaluation points satisfy

$$t_I \leq \theta_{ij} \leq t_F.$$

We refer to $\hat{y}_{ij}$ and $\hat{u}_{ij}$ as the *discrete data functions*. Often the evaluation points are arranged monotonically, that is $\theta_{i,j} \leq \theta_{i,j+1}$. It is also common to have many variables evaluated at the same time, e.g. $\theta_{i,j} = \theta_{i+1,j}$. Neither of these assumptions are necessary for our software however the initial and final (phase) times t_I and t_F must be fixed.

It is worth noting that more complicated problem descriptions can be accommodated by the formulation given. For example, suppose it is required to minimize the expression

$$F = \frac{1}{2} \sum_{k=1}^N [\omega(y, u, p, \theta_k) - \hat{\omega}_k]^\top \Lambda [\omega(y, u, p, \theta_k) - \hat{\omega}_k]$$

where $\hat{\omega}_k$ are the observed values of the function ω at the times θ_k and Λ is the inverse covariance matrix of these quantities. Since the positive definite matrix can be factored as $\Lambda = Q^\top Q$ we can define a new set of algebraic variables

$$\begin{aligned} v(t) &= Q\omega(y, u, p, t) \\ \hat{v}_k &= Q\hat{\omega}_k. \end{aligned}$$

The objective function then becomes

$$F = \frac{1}{2} \sum_{k=1}^N [v_k - \hat{v}_k]^\top [v_k - \hat{v}_k]$$

where the residuals are expressed in terms of the algebraic variables and the transformation can be treated as an equality path constraint.

Computing The Residuals

In order to evaluate the residuals it is necessary to construct the value of the state variable at the data evaluation time θ_{ij} as illustrated in Figure 5.1. This quantity can be constructed from the Hermite interpolating polynomial. Thus for any particular residual k there is an interval $t_j \leq \theta_k \leq t_{j+1}$, and a particular state $\nu = i(k)$. Then the value of the state needed in the residual calculation is

$$y_\nu(\theta_k) = (1 - 3\delta^2 + 2\delta^3)y_{\nu j} + (3\delta^2 - 2\delta^3)y_{\nu, j+1} + (h_j\delta - 2h_j\delta^2 + h_j\delta^3)f_{\nu j} + (-h_j\delta^2 + h_j\delta^3)f_{\nu, j+1}$$

where $h_j = t_{j+1} - t_j$ is the length of the discretization interval and $\delta = (\theta_k - t_j)/h_j$ defines the location of the evaluation time relative to the beginning of the interval. In this expression, $y_{\nu j}$ is the value of state variable ν at grid point j and $f_{\nu j}$ is the corresponding value of the right hand side of the differential equations at the same grid point.

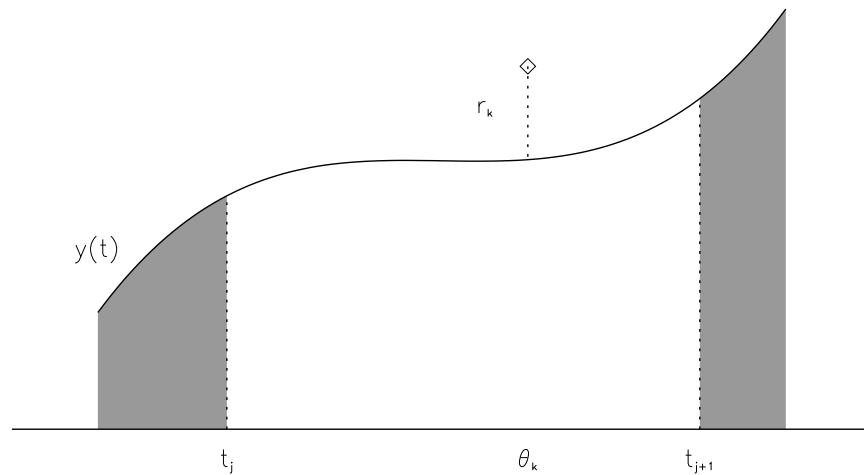


Figure 5.1: Residual Evaluation

When the residual requires the value of the algebraic variable it can be constructed from a quadratic interpolant when the Hermite-Simpson discretization is used, i.e. according to:

$$u_\nu(\theta_k) = (1 - \delta)(1 - 2\delta)u_{\nu j} + 4\delta(1 - \delta)\bar{u}_{\nu,j+1} - \delta(1 - 2\delta)u_{\nu,j+1}$$

where $\delta = (\theta_k - t_j)/h_j$. Similarly, when a trapezoidal discretization is used, linear interpolation between the grid points yields:

$$u_\nu(\theta_k) = (1 - \delta)u_{\nu j} + \delta u_{\nu,j+1}.$$

It is important to observe that the residuals are computed by interpolation and do not have any direct affect on the location of the discretization grid points. This is often referred to as *dense output* in methods for numerical integration (c.f. [20]).

Delay Differential Equations

The usual formulation of a dynamic system involves a differential-algebraic equation model

$$\dot{\mathbf{y}} = \mathbf{f}[\mathbf{y}(t), \mathbf{u}(t), \mathbf{p}, t] \quad (5.1a)$$

$$\mathbf{0} = \mathbf{g}[\mathbf{y}(t), \mathbf{u}(t), \mathbf{p}, t] \quad (5.1b)$$

where the algebraic inequalities

$$\mathbf{g}_L \leq \mathbf{g}[\mathbf{y}(t), \mathbf{u}(t), \mathbf{p}, t] \leq \mathbf{g}_U \quad (5.1c)$$

can also be treated. The differential or state variables are denoted by $\mathbf{y}$, the algebraic or control variables by $\mathbf{u}$, and the (non-dynamic) parameters $\mathbf{p}$.

The correct mathematical formulation of many physical processes involves “time lags” or “time leads.”

These systems ... arise in a wide variety of scientific and engineering applications including circuit analysis, computer-aided design and real-time simulation of mechanical (multibody) systems, power systems, chemical process simulation, optimal control, etc. In some situations, for example in real-time simulation, where time delays can be introduced by the computer time needed to compute an output after the input has been sampled, and where additional delays can be introduced by the operator-in-the-loop, differential equations with delays must be included in the model. Delays arise also in circuit simulation and power systems, due for example to interconnects for computer chips and transmission lines, and in chemical process simulation when modeling pipe flow. [2]

Let us define a set of ν *delay functions*, ω_k that are prescribed functions of time for $k = 1, \dots, \nu$. Although we generically refer to the functions as “delay” functions, as in

$$\omega_1(t) = t - \tau \quad (5.2)$$

for a delay time $\tau > 0$, the approach can also be used to model “advances” such as

$$\omega_2(t) = t + 5\tau. \quad (5.3)$$

Let us denote the *delayed state vector* by

$$\hat{\mathbf{y}}(\omega_k) = \mathbf{\Phi}(\omega_k)\mathbf{y}(\omega_k) \quad (5.4)$$

where the $\hat{n}_y \times n_y$ matrix

$$\Phi_{i,j} = \begin{cases} 1 & \text{if } y_j(\omega_k) \neq 0, \\ 0 & \text{otherwise} \end{cases} \quad (5.5)$$

for $i = 1, \dots, \hat{n}_y$. Thus the vector $\hat{\mathbf{y}}(\omega_k)$ of length $\hat{n}_y$ ($\hat{n}_y \leq n_y$) denotes the subset of the n_y state variables that are functions of the delay argument ω_k .

Similarly let us denote the *delayed control vector* by

$$\hat{\mathbf{u}}(\omega_k) = \mathbf{\Theta}(\omega_k)\mathbf{u}(\omega_k) \quad (5.6)$$

where the $\hat{n}_u \times n_u$ matrix

$$\Theta_{i,j} = \begin{cases} 1 & \text{if } u_j(\omega_k) \neq 0, \\ 0 & \text{otherwise} \end{cases} \quad (5.7)$$

for $i = 1, \dots, \hat{n}_u$.

The problem description then entails a model such as

$$\dot{\mathbf{y}} = \mathbf{f}[\mathbf{y}(t), \mathbf{u}(t), \hat{\mathbf{y}}(\omega_1), \dots, \hat{\mathbf{y}}(\omega_\nu), \hat{\mathbf{u}}(\omega_1), \dots, \hat{\mathbf{u}}(\omega_\nu), \mathbf{p}, t] \quad (5.8)$$

$$\mathbf{0} = \mathbf{g}[\mathbf{y}(t), \mathbf{u}(t), \hat{\mathbf{y}}(\omega_1), \dots, \hat{\mathbf{y}}(\omega_\nu), \hat{\mathbf{u}}(\omega_1), \dots, \hat{\mathbf{u}}(\omega_\nu), \mathbf{p}, t]. \quad (5.9)$$

For simplicity let us consider a problem with no parameters $\mathbf{p}$ and a single delay function

$$\omega_1(t) = t - \tau. \quad (5.10)$$

In this case (5.8)-(5.9) becomes

$$\dot{\mathbf{y}} = \mathbf{f}[\mathbf{y}(t), \mathbf{u}(t), \hat{\mathbf{y}}(\omega_1(t)), \hat{\mathbf{u}}(\omega_1(t)), t] \quad (5.11)$$

$$\mathbf{0} = \mathbf{g}[\mathbf{y}(t), \mathbf{u}(t), \hat{\mathbf{y}}(\omega_1(t)), \hat{\mathbf{u}}(\omega_1(t)), t] \quad (5.12)$$

When $\tau > 0$, the right hand sides of (5.11)-(5.12) involve values of the state $\hat{\mathbf{y}}[\omega_1(t)] = \hat{\mathbf{y}}(t - \tau)$ at an earlier time, and as such are called *delay* equations. On the other hand when $\tau < 0$ the state at a future time enters the equation, and as such are referred to as *advances*. When there are no algebraic equations, the system is referred to as a *delay-differential equation* (DDE), and when algebraic equations do appear the system is referred to as a *delay-differential-algebraic equation* or DDAE [2]. We will consider problems with prescribed arguments $\omega_k(t)$ which implies the delay times τ are fixed.

Delay equations introduce an issue not encountered in a standard DAE problem. For a system defined by the DAE model (5.1a)-(5.1c) a complete description usually requires boundary conditions at the initial time t_I and/or the final time t_F . The boundary conditions can involve specified values for the states or nonlinear conditions $b(\mathbf{y}, \mathbf{u}, t) = 0$ and are enforced at specific points, namely at t_I and/or t_F . In contrast, for a delay equation such as (5.11)-(5.12), there must be a start up region. Specifically when $0 \leq t \leq t_I + \tau$, we must evaluate $\hat{\mathbf{y}}[\omega_1(t)]$. Thus we require

$$\hat{\mathbf{y}}[\omega_1(t)] = \boldsymbol{\alpha}(t) \quad \text{for} \quad t_I - \tau \leq t \leq t_I \quad (5.13)$$

where $\alpha(t)$ must be a defined function on the startup region $\tau \leq t \leq t_I$. A similar function may be required at the final time for problems with advances.

Direct Transcription Approach

Let us address a DDAE system of the form (5.11)-(5.12). Define new algebraic variables $\mathbf{v}(t)$, and $\mathbf{w}(t)$, and require they be consistent with the delayed variables. The original DDAE system can now be written as

$$\dot{\mathbf{y}} = \mathbf{f}[\mathbf{y}(t), \mathbf{u}(t), \mathbf{v}(t), \mathbf{w}(t), t] \quad (5.14)$$

$$\mathbf{0} = \mathbf{g}[\mathbf{y}(t), \mathbf{u}(t), \mathbf{v}(t), \mathbf{w}(t), t] \quad (5.15)$$

$$\mathbf{0} = \mathbf{v}(t) - \hat{\mathbf{y}}[\omega_1(t)] \quad (5.16)$$

$$\mathbf{0} = \mathbf{w}(t) - \hat{\mathbf{u}}[\omega_1(t)] \quad (5.17)$$

The basic idea is to first discretize the problem thereby creating a finite dimensional approximation. Large scale optimization methods can then be used to adjust the variables that define the discretization. The direct transcription approach introduces a discretization of the problem by subdividing the time domain into n_s segments or intervals

$$t_I = t_1 < t_2 < \dots < t_M = t_F, \quad (5.18)$$

where the points are referred to as node, mesh, or grid points. We use $\mathbf{y}_k \equiv \mathbf{y}(t_k)$ to indicate the value of the state variable at a grid point, with a similar notation for the algebraic variables. Thus one treats

$$\mathbf{x}^T = (\mathbf{y}_1, \mathbf{u}_1, \mathbf{v}_1, \mathbf{w}_1, \dots, \mathbf{y}_M, \mathbf{u}_M, \mathbf{v}_M, \mathbf{w}_M) \quad (5.19)$$

as optimization variables in a nonlinear programming problem. We then approximate the differential equation using a nonlinear algebraic constraint. When the discretization is based on an implicit Runge-Kutta (IRK) scheme as in the SOS software, the control problem is *transcribed* into a finite dimensional nonlinear program.

Thus for the simplest IRK scheme, the trapezoidal method, we approximate the differential equations (5.14) and algebraic constraints (5.15) by a set of algebraic constraints given by

$$\mathbf{0} = \mathbf{y}_{k+1} - \mathbf{y}_k - \frac{h_k}{2} (\mathbf{f}_k + \mathbf{f}_{k+1}) = \boldsymbol{\zeta}_k \quad k = 1, \dots, M-1 \quad (5.20)$$

$$\mathbf{0} = \mathbf{g}_k \quad k = 1, \dots, M \quad (5.21)$$

where (5.20) are referred to as the *defect* constraints $\boldsymbol{\zeta}_k = \mathbf{0}$. This discretization is implicit because the optimization variables $(\mathbf{y}_k, \mathbf{u}_k, \mathbf{v}_k, \mathbf{w}_k)$ appear as arguments in the nonlinear functions $\mathbf{f}_k$ and $\mathbf{g}_k$. Now in order to evaluate the right hand side functions in (5.14) we must express the consistency relationships (5.16) and (5.17) in terms of the NLP optimization variables.

Now the offset time $\omega_1(t_k)$ will lie in the interval

$$t_j \leq t_k - \tau \leq t_{j+1} \quad (5.22)$$

and because IRK methods are collocation methods we can express the delayed arguments as a linear combination of the interval endpoints, that is

$$\hat{\mathbf{y}}[\omega_1(t_k)] = \hat{\mathbf{y}}(t_k - \tau) = \begin{pmatrix} \hat{\mathbf{y}}_j & \dot{\hat{\mathbf{y}}}_j & \hat{\mathbf{y}}_{j+1} & \dot{\hat{\mathbf{y}}}_{j+1} \end{pmatrix} \mathbf{a} = \begin{pmatrix} \hat{\mathbf{y}}_j & \hat{\mathbf{f}}_j & \hat{\mathbf{y}}_{j+1} & \hat{\mathbf{f}}_{j+1} \end{pmatrix} \mathbf{a} \quad (5.23)$$

where the coefficient vector $\mathbf{a}$ is defined such that the interpolant is a Hermite cubic polynomial. Using this expression for the delayed state, we can impose the consistency condition (5.16) as an additional set of NLP constraints

$$\mathbf{0} = \mathbf{v}_k - \begin{pmatrix} \hat{\mathbf{y}}_j & \hat{\mathbf{f}}_j & \hat{\mathbf{y}}_{j+1} & \hat{\mathbf{f}}_{j+1} \end{pmatrix} \mathbf{a} \quad k = 1, \dots, M \quad (5.24)$$

Since $\hat{\mathbf{f}}_j = \hat{\mathbf{f}}(\mathbf{y}_j, \mathbf{u}_j, \mathbf{v}_j, \mathbf{w}_j, t_j)$ these equations define an implicit nonlinear relationship between the optimization variables at grid points k , j , and $j + 1$. The same approach can be applied to insure consistency in the algebraic variables. When using a trapezoidal discretization (5.17) is enforced using a piecewise linear control interpolant and imposition of the NLP constraints

$$\mathbf{0} = \mathbf{w}_k - (\hat{\mathbf{u}}_j \quad \hat{\mathbf{u}}_{j+1}) \mathbf{a} \quad k = 1, \dots, M. \quad (5.25)$$

To summarize, for the trapezoidal discretization the NLP variables (5.19) must be chosen to satisfy the NLP constraints (5.20), (5.21), (5.24) and (5.25)

$$\mathbf{0} = \mathbf{y}_{k+1} - \mathbf{y}_k - \frac{h_k}{2} (\mathbf{f}_k + \mathbf{f}_{k+1}) = \boldsymbol{\zeta}_k \quad k = 1, \dots, M - 1 \quad (5.26)$$

$$\mathbf{0} = \mathbf{g}_k \quad k = 1, \dots, M \quad (5.27)$$

$$\mathbf{0} = \mathbf{v}_k - \begin{pmatrix} \hat{\mathbf{y}}_j & \hat{\mathbf{f}}_j & \hat{\mathbf{y}}_{j+1} & \hat{\mathbf{f}}_{j+1} \end{pmatrix} \mathbf{a} \quad k = 1, \dots, M \quad (5.28)$$

$$\mathbf{0} = \mathbf{w}_k - (\hat{\mathbf{u}}_j \quad \hat{\mathbf{u}}_{j+1}) \mathbf{a} \quad k = 1, \dots, M. \quad (5.29)$$

When using a Hermite-Simpson discretization three things change. First

$$\mathbf{x}^\top = (\mathbf{y}_1, \mathbf{u}_1, \mathbf{v}_1, \mathbf{w}_1, \mathbf{u}_{3/2}, \mathbf{v}_{3/2}, \mathbf{w}_{3/2}, \dots, \mathbf{y}_M, \mathbf{u}_M, \mathbf{v}_M, \mathbf{w}_M) \quad (5.30)$$

where $\mathbf{u}_{j+1/2}$ is an additional NLP variable representing the control at the midpoint $\mathbf{u}_{j+1/2} = \mathbf{u}[\frac{1}{2}(t_j + t_{j+1})]$. Second the discretization constraints (5.20)-(5.21) are replaced by

$$\mathbf{0} = \mathbf{y}_{k+1} - \mathbf{y}_k - \frac{h_k}{6} (\mathbf{f}_k + 4\mathbf{f}_{k+1/2} + \mathbf{f}_{k+1}) = \boldsymbol{\zeta}_k \quad k = 1, \dots, M - 1 \quad (5.31)$$

$$\mathbf{0} = \mathbf{g}_k \quad k = 1, \dots, M \quad (5.32)$$

$$\mathbf{0} = \mathbf{g}_{k+1/2} \quad k = 1, \dots, M - 1 \quad (5.33)$$

and finally, to ensure a piecewise quadratic control interpolant satisfies the consistency condition (5.17) we impose the NLP constraints

$$\mathbf{0} = \mathbf{v}_k - \begin{pmatrix} \hat{\mathbf{y}}_j & \hat{\mathbf{f}}_j & \hat{\mathbf{y}}_{j+1} & \hat{\mathbf{f}}_{j+1} \end{pmatrix} \mathbf{a} \quad k = 1, \dots, M \quad (5.34)$$

$$\mathbf{0} = \mathbf{v}_{k+1/2} - \begin{pmatrix} \hat{\mathbf{y}}_j & \hat{\mathbf{f}}_j & \hat{\mathbf{y}}_{j+1} & \hat{\mathbf{f}}_{j+1} \end{pmatrix} \mathbf{a} \quad k = 1, \dots, M - 1 \quad (5.35)$$

$$\mathbf{0} = \mathbf{w}_k - (\hat{\mathbf{u}}_j \quad \hat{\mathbf{u}}_{j+1/2} \quad \hat{\mathbf{u}}_{j+1}) \mathbf{a} \quad k = 1, \dots, M \quad (5.36)$$

$$\mathbf{0} = \mathbf{w}_{k+1/2} - (\hat{\mathbf{u}}_j \quad \hat{\mathbf{u}}_{j+1/2} \quad \hat{\mathbf{u}}_{j+1}) \mathbf{a} \quad k = 1, \dots, M - 1 \quad (5.37)$$

To summarize, for the Hermite-Simpson discretization the NLP variables (5.30) must be chosen to satisfy the NLP constraints (5.31)-(5.37).

In order to efficiently solve the resulting large nonlinear programming problem it is imperative to exploit sparsity in the Jacobian and Hessian matrix. In particular the Jacobian will involve

derivatives of the constraints (5.26)-(5.29) or (5.31)-(5.37) and (5.24) with respect to the variables $\mathbf{x}$ given by (5.19) or (5.30). These matrices will be sparse, because the subset

$$(\mathbf{y}_k, \mathbf{y}_j, \mathbf{y}_{j+1}, \mathbf{u}_k, \mathbf{u}_j, \mathbf{u}_{j+1}, \mathbf{v}_k, \mathbf{v}_j, \mathbf{v}_{j+1}, \mathbf{w}_k, \mathbf{w}_j, \mathbf{w}_{j+1}, t_k)$$

are the only variables that appear in (5.26)-(5.29). In order to fully exploit sparsity it is important to exploit separability. This can be achieved by writing the constraint functions so as to isolate the linear terms from the nonlinear ones. Specifically using Eq. 4.114 in Reference [6], (Eq. 4.84 [7]) we write

$$\begin{bmatrix} \mathbf{c}(\mathbf{x}) \\ F(\mathbf{x}) \end{bmatrix} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{q}(\mathbf{x}), \quad (5.38)$$

where the vector $\mathbf{q}$ consists of the nonlinear right hand side functions $\mathbf{f}_k$ and $\mathbf{g}_k$ evaluated at the grid points. Sparse finite difference estimates can be used to compute the Jacobian and Hessian matrices as described in [6, 7]. Furthermore because the functions $\mathbf{f}$ and $\mathbf{g}$ appear linearly in the transcribed formulation, sparsity in these right hand side functions can be exploited (cf Eq. 4.115 in [6]).

Problem Scaling

The optimal control problem variables and constraints in an attempt to improve the conditioning and efficiency of the underlying nonlinear programming problem. Thus the NLP solves a problem in terms of the scaled variables:

$$\tilde{x}_i = w_i x_i + s_i$$

and the scaled functions:

$$\tilde{c}_i = w_i c_i$$

$$\tilde{F} = w_o F.$$

Although there are many ways to compute the weights one possibility is to use scaling to improve the Jacobian conditioning. When the original constraint is $0 = g(y, u, t)$ and $\|\nabla g\| \neq 0$ then one can set $w = 1/\|\nabla g\|$ and define the scaled constraint $0 = \tilde{g}(y, u, t) = wg(y, u, t)$. With this choice, the gradient of the scaled constraint is $\|\nabla \tilde{g}\| = 1$. Unfortunately, gradient information can be significantly different between the initial iterate and the solution. The automatic scaling procedure constructs the scale weights at the initial point, and then recomputes the scaling at the solution to the first mesh refinement problem. In addition this technique is used in conjunction with the following SOS scaling heuristics:

Rule 1: Scale from a *control perspective*, e.g. state variable $y_4(t)$ has the same scale for all grid points.

Rule 2: *Variable Scaling*

1. estimate the largest/smallest variable value from
 - (a) user input upper/lower bound, or if not available
 - (b) user specified initial guess
2. normalize and shift the variables. When (a) and (b) provide no information default scaling to 1.

Rule 3: *ODE Defect Scaling*

1. Set ODE defect scaling to state variable scaling, or optionally
2. choose it to normalize the defect gradients

Rule 4: *Algebraic and Boundary Constraint Scaling*

1. estimate the largest/smallest constraint value from user input upper/lower bound
2. optionally, estimate and/or compute the Jacobian
3. normalize the constraint.
4. When bounds provide no information set scaling to 1.

Chapter 6

Optimal Control Software

6.1 Overview of Sparse Optimal Control Software

The software for solving optimal control and parameter estimation problems can be grouped into four distinct classes:

1. the *optimal control routines* namely SOSCTL, SOSDLY or SOSEST, which are called by the user to solve an optimal control or parameter estimation problem, and the input routine INPSOS;
2. the *user supplied subroutines* needed to define the optimal control or parameter estimation problem, and;
3. the *optimization software* needed to solve a sparse nonlinear programming subproblem, and;
4. the *optimal control utility software* available for special analysis and applications.

The first three sections of this chapter describe the optimal control routines in the SOS package. The primary SOS routine SOSCTL (p. 382) and a procedure for defining optional algorithm parameters INPSOS (p. 401) are both discussed. The delay equation software SOSDLY (p. 388) and parameter estimation software SOSEST (p. 395) are also described. The next six sections provide a description of the user supplied optimal control routines. Specifically the user must define the problem using a routine generically referred to as ODEINP (p. 426). An initial guess can be supplied in the a routine referred to as ODEIGS (p. 422). Subroutine ODERHS (p. 455) permits the user to define the right hand sides of the relevant differential-algebraic equations, and nonlinear boundary conditions can be constructed in subroutine ODEPTF (p. 450). Optional output can be constructed in subroutine ODEPRT (p. 446). Delay equation problems require input of the delay functions using subroutine EVALDL (p. 419). Parameter estimation problems require input of the measurement data using subroutine DDLOAD (p. 416). The SOS program utilizes the sparse NLP and sparse finite difference software described in Chapters 2 and 3. The SOS user *does not* need to call the optimization and/or finite difference software, however, optional input to this software may be useful. The final sections of this chapter describe a collection of useful utility procedures available in the SOS library, that are commonly needed for many applications. In particular subroutines AUXOUT (p. 464), OCSEVL (p. 487), OCSRNG (p. 493) and OCSTAU (p. 495) are output utility routines. Subroutine CTLSTA (p. 474) can be used to monitor the status of the software and subroutine LINKST (p. 481) is useful for linking dynamic variables across a phase boundary.

Subroutines FYLFMT (p. 477), and RST2SP (p. 509), are utility routines for manipulation of the B-spline solution files produced by SOS. Subroutines PHSLNG (p. 498), PNTCON (p. 500), and PTHCON (p. 506), are utility routines to simplify the specification of phase duration constraints, point functions, and path constraints, respectively. The SOS software and all lower level support routines are written in ANSI-Standard FORTRAN 90, and is available on most major computational hardware.

In addition to calling the main program SOSCTL, SOSDLY or SOSEST, the user must provide at least two of the subroutines in class 2, which will be called repeatedly throughout the solution process. One routine provides detailed input which sets up the optimal control problem to be solved, and the second evaluates the differential and algebraic equations occurring in the problem. Three additional subroutines may be supplied as required by the application. One optional subroutine may be supplied if it is necessary to evaluate functions at discrete points, namely the phase boundaries in the problem. This routine can be used to construct nonlinear boundary conditions. A second optional routine may be supplied to permit flexibility when computing the initial guess. A third optional routine can be utilized to accommodate special user supplied auxiliary output requirements. The organization of the software suite is illustrated in Figures 6.1, 6.2 and 6.3.

The optimal values for the dynamic variables (i.e. the solution to the problem), are represented as a B-spline vector. In particular the coefficients that define a cubic spline representation of each dynamic variable are returned to the user. This representation permits the user to evaluate the functions $[y(t), u(t)]$ at arbitrarily many points. The knots of the spline correspond to the grid points used in discretizing the continuous problem. An array of the discrete parameters occurring in the problem is also available.

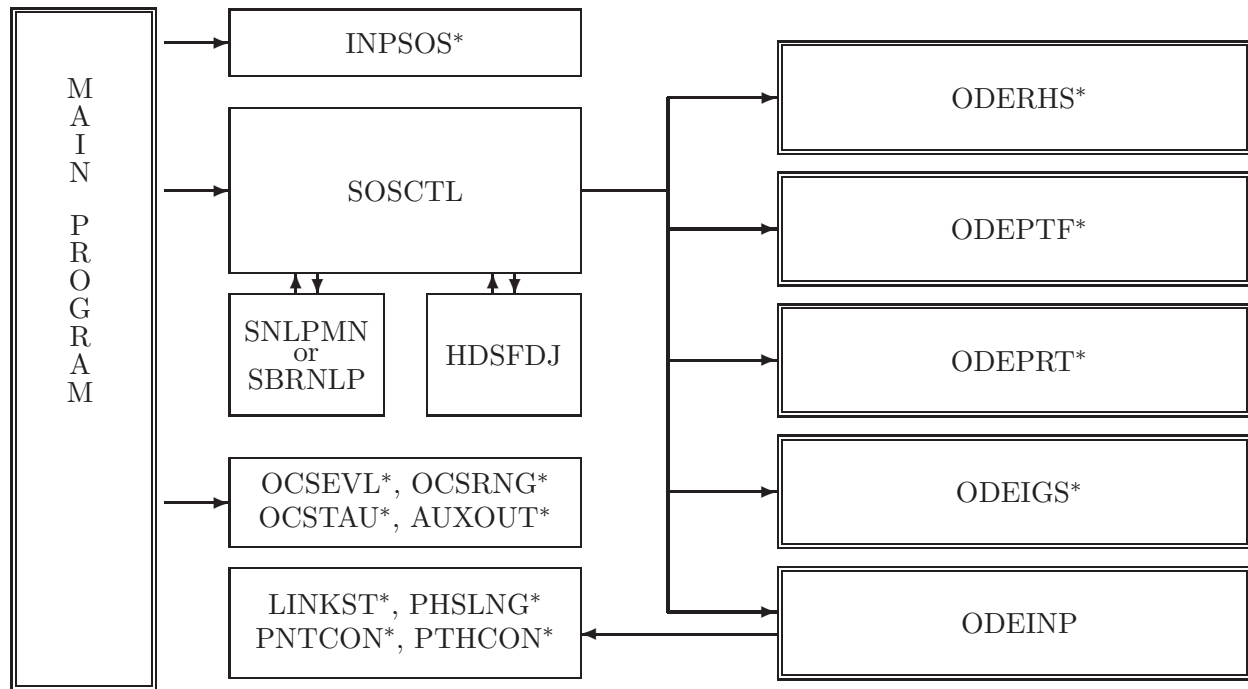


Figure 6.1: Sparse Optimal Control Software

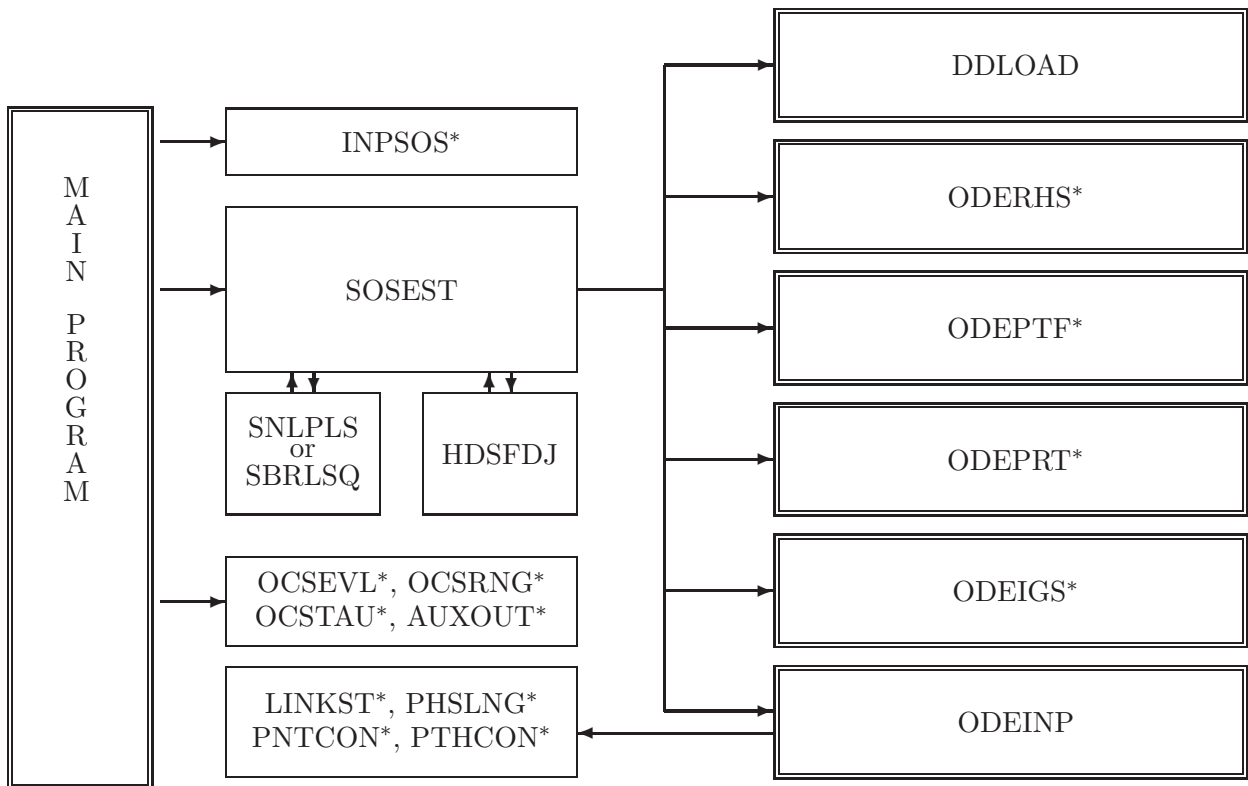


Figure 6.2: Sparse Optimal Parameter Estimation

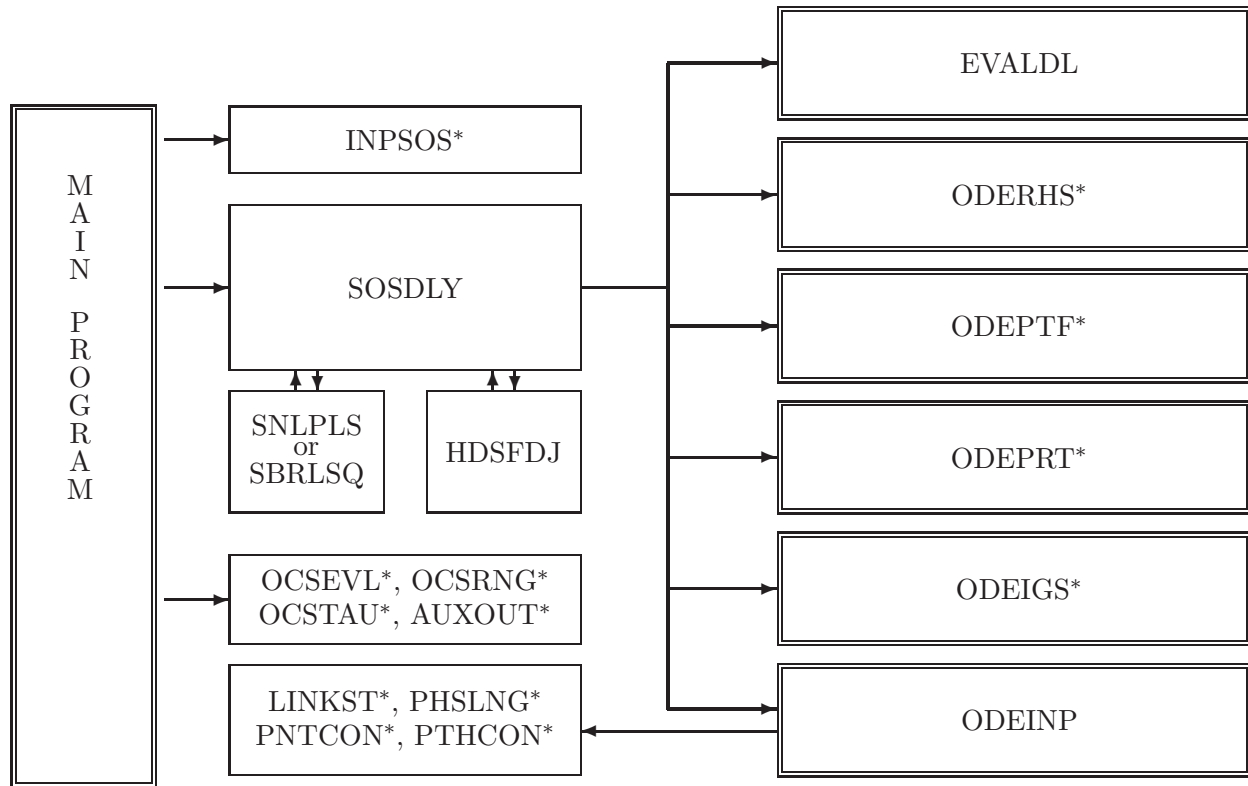


Figure 6.3: Sparse Optimal Control of Delay Systems

6.2 Subprograms for Optimal Control

SOSCTL: Sparse Optimal Control	382
SOSDLY: Sparse Optimal Control of Delay Systems	388
SOSEST: Sparse Optimal Parameter Estimation	395
INPSOS: Sparse Optimization Suite Input Procedure	401
INSOCX: Sparse Optimal Control Input Procedure	403
DDLOAD: Discrete Data Load	416
EVALDL: Delay Equation Evaluation	419
ODEIGS: Optimal Control Initial Guess	422
ODEINP: Optimal Control Problem Definition	426
ODEPRT: Optimal Control Print	446
ODEPTF: Optimal Control Point Function Evaluation	450
ODERHS: Optimal Control Right Hand Side Evaluation	455
AUTOLK: Auto Link Variable Utility	460
AUTOPL: Auto Phase Length Constraint Utility	462
AUXOUT: Auxiliary Output Utility	464
BSPDEF: B-Spline Algebraic Variable Utility	468
CTLFIL: Optimal Control Input From a File	472
CTLSTA: Optimal Control Software Status Utility	474
FETCH: Variable Retrieval Utility	475
FYLFMT: Restart File Format Conversion Utility	477
ICSTPH: Optimal Control Solution Evaluation Utility	479
LINKST: Optimal Control Linkage Utility	481
NUMGRD: Gridpoint Evaluation Utility	483
OCDGRD: Optimal Control Solution and Derivative Grid Evaluation Utility	484
OCSEVL: Optimal Control Solution Evaluation Utility	487
OCSGRD: Optimal Control Solution Grid Evaluation Utility	491
OCSRNG: Optimal Control Solution Range Utility	493
OCSTAU: Optimal Control Solution Evaluation Utility	495
PHSLNG: Optimal Control Phase Duration Utility	498

PNTCON: Optimal Control Point Constraint Utility	500
PTHAUX: Path Constraint with Auxiliary Functions	503
PTHCON: Path Constraint Utility	506
RST2SP: Restart File to Spline Array Utility	509
SCTFIL: Save an Optimal Control Input File	510
SETMRF: Set Mesh Refinement Option	512
STMSEQ: Set Discretization Refinement Sequence	514
TRMAUX: Terminate Auxiliary Function Construction	516
WATCH: Variable Monitor Utility	518

SOSCTL: Sparse Optimal Control

PURPOSE

The purpose of SOSCTL is to determine the $n_u^{(k)}$ -dimensional control vectors $u^{(k)}(t)$ and parameters $p^{(k)}$ to minimize the performance index

$$J = \phi \left[y(t_I^{(1)}), t_I^{(1)}, y(t_F^{(1)}), p^{(1)}, t_F^{(1)}, \dots, y(t_I^{(N)}), t_I^{(N)}, y(t_F^{(N)}), p^{(N)}, t_F^{(N)} \right] \\ + \sum_{k=1}^N \int_{t_I^{(k)}}^{t_F^{(k)}} q^{(k)}[y^{(k)}(t), u^{(k)}(t), p^{(k)}, t] dt$$

where N is the number of phases. Notice that the objective function may depend on quantities computed in each of the N phases.

The independent variable t for phase k is defined in the region $t_I^{(k)} \leq t \leq t_F^{(k)}$. Within phase k the dynamics of the system are described by a set of *dynamic* variables

$$z = \begin{bmatrix} y^{(k)}(t) \\ u^{(k)}(t) \end{bmatrix}$$

made up of the $n_y^{(k)}$ *state variables* and the $n_u^{(k)}$ *control variables* respectively. In addition the dynamics may incorporate the $n_p^{(k)}$ *parameters* $p^{(k)}$ which are not dependent on t . Within phase k the dynamics of the system are defined by a set of ordinary differential equations written in explicit form, which are referred to as the *state equations*

$$\dot{y} = f[y(t), u(t), p, t]$$

where y is the n_y dimension state vector. Initial conditions at t_I are defined by

$$\psi[y(t_I), u(t_I), p, t_I] \equiv \psi_I = 0,$$

and terminal conditions at t_F are defined by

$$\psi[y(t_F), u(t_F), p, t_F] \equiv \psi_F = 0.$$

In addition the solution must satisfy *algebraic path constraints* of the form

$$g_\ell \leq g[y(t), u(t), p, t] \leq g_u,$$

where g is a vector of size n_g . The algebraic constraints are of the form

$$g[y(t), u(t), p, t] = \alpha^\top v + \beta^\top a[v, t]$$

where

$$v = \begin{bmatrix} y(t) \\ u(t) \\ p \end{bmatrix}.$$

The constraint definition can include *analytic* terms involving $\alpha^\top v$ where the $(n_y + n_u + n_p)$ vector α is constant, as well as linear combinations of the n_a *auxiliary functions* $a_k(v)$ for $k = 0, \dots, n_a$

where the coefficients β_k are nonzero constants. By convention, a path constraint with a single nonlinear term $a_0(y, u, p, t)$ has no auxiliary functions ($n_a = 0$). Observe that each individual path constraint may have a different number of auxiliary functions and analytic terms. Simple bounds can be imposed on the state variables

$$y_\ell \leq y(t) \leq y_u,$$

the control variables

$$u_\ell \leq u(t) \leq u_u,$$

and the parameters

$$p_\ell \leq p \leq p_u.$$

Note that a path variable equality constraint can be imposed if the upper and lower bounds are equal, e.g. $(g_\ell)_k = (g_u)_k$ for some k .

Finally it may be convenient to evaluate expressions of the form

$$\int_{t_I}^{t_F} q[y(t), u(t), p, t] dt$$

which involve the *quadrature* functions q . Collectively we refer to those functions evaluated during the phase, namely,

$$F(t) = \begin{bmatrix} f[y(t), u(t), p, t] \\ g[y(t), u(t), p, t] \\ q[y(t), u(t), p, t] \\ a[y(t), u(t), p, t] \end{bmatrix}$$

as the vector of *continuous functions*. Similarly functions evaluated at the beginning or end of a phase such as the boundary conditions $\psi[y(t_I), u(t_I), p, t_I]$ and $\psi[y(t_F), u(t_F), p, t_F]$ are referred to as *point functions*.

RELATED SUBPROGRAMS

INSNLP	Sparse Nonlinear Programming Input Procedure
SNLPMN	Sparse Nonlinear Programming
SBRNLP	Sparse Barrier Nonlinear Programming
SOSEST	Sparse Optimal Parameter Estimation
INSOCX	Sparse Optimal Control Input Procedure
INPSOS	Sparse Optimization Suite Input Procedure

METHOD

The optimal control problem is solved using a *direct transcription* method. The basic steps in the approach are as follows:

1. **Direct Transcription.** *Transcribe* the optimal control problem into a nonlinear programming (NLP) problem by discretization;

2. **Sparse Nonlinear Program.** Solve the sparse NLP using either a sequential quadratic programming (SQP) or an interior-point (barrier) method;
3. **Mesh Refinement.** Assess the accuracy of the approximation (i.e. the finite dimensional problem), and if necessary refine the discretization, and then repeat the optimization steps.

The SOS software provides a number of discretization techniques including, trapezoidal, Hermite-Simpson, and Runge-Kutta methods. The sparse nonlinear programming problem can be solved using either a *sequential quadratic programming* (SQP) approach or an *interior-point* (barrier) method. There are a number of issues which determine the efficiency of the NLP. First, the Jacobian matrix (of first derivatives) and the Hessian matrix (of second derivatives) needed to solve the NLP are large and sparse, and SOS automatically constructs the sparsity structure. Second, it is necessary to compute the Hessian matrix H_L and within SOS sparse finite difference techniques are utilized. Third, the sparse NLP subproblem can be solved efficiently using a sparse Schur-Complement Method or a primal-dual barrier method. The SOS software also incorporates a technique for computing the discretization error and if necessary refining the mesh to reduce this error.

WARNING

The direct transcription method implemented in the SOS software yields approximate solutions for the state and control functions. The technique is appropriate when the control variables are continuous and the state variables have continuous first derivatives. If the solution to the optimal control problem cannot be represented in this form, unpredictable performance in the numerical methods may be observed. Furthermore the technique may be inappropriate for solving differential-algebraic systems with global index greater than one. This situation can occur either because of the presence of state variable path constraints and/or singular arcs in the solution. Many of these situations can be circumvented by reformulation of the underlying problem to exploit the multi-phase capability of the SOS software.

The user supplied functions which define the objective function, path constraints, point functions, and differential equations are all assumed to be continuous and have continuous first and second derivatives. Non-differentiable functions can cause unpredictable performance in the optimization algorithm. It should be emphasized that discontinuities in the second derivatives can significantly degrade the speed of convergence without producing any other obvious difficulties. The following common sources of error *should be avoided* when evaluating the objective and constraint functions:

- (1) non-smooth interpolation of data;
- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;
- (3) discontinuous behavior caused by branching for IF tests;
- (4) non-differentiable functions such as ABS, MAX, and MIN.

The user must provide procedures to evaluate the continuous and point functions. If it is not possible to evaluate the functions the user can communicate this information to the algorithm using the function error flag IFERR as described below.

USAGE

```

INTEGER          MAXCS,NEEDED,IER
DOUBLE PRECISION CSTAT(MAXCS)
EXTERNAL  ODEINP,ODEIGS,ODERHS,ODEPTF,ODEPRT

CALL SOSCTL(ODEINP,ODEIGS,ODERHS,ODEPTF,ODEPRT,
$          CSTAT,MAXCS,NEEDED,IER)

```

ARGUMENTS

USER-SUPPLIED SUBPROGRAMS

- ODEINP** [INPUT, SUBROUTINE]
The name of the user routine which defines the specifics of the optimal control problem (p. 426).
- ODEIGS** [INPUT, SUBROUTINE]
The name of the user routine which can be called to obtain the initial guess (p. 422). If a sophisticated initial guess is not required, the system supplied dummy routine DUMYIG may be used instead.
- ODERHS** [INPUT, SUBROUTINE]
The name of the user routine which evaluates the right hand sides of the differential-algebraic equations (p. 455). If there are no differential-algebraic equations, the system supplied dummy routine DUMYDE may be used instead.
- ODEPTF** [INPUT, SUBROUTINE]
The name of the user routine which evaluates functions at the ends of each phase (p. 450). If there are no point functions, the system supplied dummy routine DUMYPF may be used instead.
- ODEPRT** [INPUT, SUBROUTINE]
The name of the user routine which evaluates auxiliary print at each point (p. 446). If sophisticated output is not required, the system supplied dummy routine DUMYPR may be used instead.

OPTIMAL CONTROL ARGUMENTS

- CSTAT** [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
Array containing the solution data for all phases in the optimal control problem. An initial guess for the continuous functions $[y(t), u(t)]$ can be specified if CSTAT is *input* and the initialization flag $INIT = 3$ is set in subroutine ODEINP. After terminating the final value for the continuous variables $[y(t), u(t)]$ is represented in the array CSTAT which is *output*.
- MAXCS** [INPUT, INTEGER]
Maximum length of the array, CSTAT.

NEEDED [OUTPUT, INTEGER]

Storage needed in the array CSTAT. When $IER \neq 0$ to indicate that SOSCTL has failed to compute a solution, an error message will be printed. Section 6.5 describes the optimal control error messages. NEEDED contains the required value for the quantity MAXCS.

IER [OUTPUT, INTEGER]

Error return flag; $IER = 0$ for a successful termination. When $IER \neq 0$ to indicate that SOSCTL has failed to compute a solution, an error message will be printed. Section 6.5 describes the optimal control error messages.

USAGE REMARKS

Optimal Solution Representation

All information that defines the optimal solution is contained in the array CSTAT. The optimal values for the continuous functions are represented as either a B-spline or piecewise polynomial.

The utility routine OCSEVL (p. 487) can be used to evaluate the solution at an arbitrary point. The utility routine AUXOUT (p. 464) can be used to create an output file from the information in the CSTAT array. For a multiphase problem the CSTAT array contains the coefficients for all phases stored in a single contiguous array. The utility routine ICSTPH (p. 479) can be used to extract the relevant information that defines the solution.

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine SOSCTL. There are three ways that optional inputs can be specified to the SOS software. The first method is to call the input subroutine INPSOS (p. 401). The second method which can be used to define input for the sparse NLP is to call the input routine INSNLP (p. 226). The third approach is to read an input file by utilizing subroutine CTLFIL (p. 472). It should be emphasized that it is *not necessary* to call INPSOS, INSNLP or CTLFIL if the default values are appropriate.

Usage of the routine INPSOS is described on page 401. The sole argument of the routine is a character string of the form '*symbol* = *value*', where '*symbol*' is the mnemonic for a variable name which is to be set, and '*value*' is the corresponding integer, floating point, or character constant to be used. Exceptions to this form are the character strings, 'DEFAULT', 'OPTIONS', and 'FULL OPTIONS'. The preceding strings, or those representing mnemonics, can be in upper, lower, or mixed case.

Sample calls to INPSOS are of the form:

```
CALL INPSOS('IPUOCP = 11')
CALL INPSOS('odetol = 1.e-04')
```

The sparse nonlinear programming algorithms SNLPMN (p. 67) and SBRNLP (p. 171) which are called by SOSCTL utilize a similar input mechanism. In fact, there are three functionally

equivalent methods for setting the sparse NLP options. The first approach is to directly call the sparse NLP input procedure INSNLP (p. 226) from the main program. The second approach is to call the SOCX input procedure INSOCX (p. 403), with the special string 'INSNLP:symbol = value'. The third approach is to call the SOS input procedure INPSOS (p. 401). The following examples illustrate the options:

```
C
C   THE FOLLOWING CALL TO THE SPARSE NLP
C
C   CALL INSNLP('ALGOPT = FM')
C
C   IS EQUIVALENT TO
C
C   CALL INSOCX('INSNLP:ALGOPT = FM')
C
C-----
C
C   THE FOLLOWING CALL TO THE SPARSE NLP
C
C   CALL INSNLP('MAXNFE = 100')
C
C   IS EQUIVALENT TO
C
C   CALL INPSOS('MAXNFE = 100')
C
```

Another way of setting optimal control values is through the routine, CTLFIL. This routine will read a user specified file where each line corresponds to a character string like those described above.

```
CHARACTER*80  FILNAM
INTEGER  INUNIT

CALL CTLFIL(INUNIT,FILNAM)
```

OPTIONAL PRINTOUT

The level of printed output from SOSCTL is controlled by the variables IPGRD, IPNLP, IPODE and IPUOCP which are set by calls to INPSOS. A complete description of the iteration output options is found in Section 6.6.

SOSDLY: Sparse Optimal Control of Delay Systems

PURPOSE

The purpose of SOSDLY is to determine the $n_u^{(k)}$ -dimensional control vectors $u^{(k)}(t)$ and parameters $p^{(k)}$ to minimize the performance index

$$J = \sum_{k=1}^N \int_{t_I^{(k)}}^{t_F^{(k)}} q^{(k)}[z^{(k)}(t), p^{(k)}, t] dt \\ + \phi \left[y(t_I^{(1)}), t_I^{(1)}, y(t_F^{(1)}), p^{(1)}, t_F^{(1)}, \dots, y(t_I^{(N)}), t_I^{(N)}, y(t_F^{(N)}), p^{(N)}, t_F^{(N)} \right]$$

where N is the number of phases. Notice that the objective function may depend on quantities computed in each of the N phases. For simplicity in presentation, we drop the phase dependent notation as in $z^{(k)}(t)$ from the subsequent discussion.

Let us define a set of ν *delay functions*, ω_k that are prescribed functions of time for $k = 1, \dots, \nu$. Although we generically refer to the functions as “delay” functions, as in

$$\omega_1(t) = t - \tau$$

for a delay time $\tau > 0$, the approach can also be used to model “advances” such as

$$\omega_2(t) = t + 5\tau.$$

Let us denote the *delayed state vector* by

$$\hat{\mathbf{y}}(\omega_k) = \mathbf{\Phi}(\omega_k) \mathbf{y}(\omega_k)$$

where the $\hat{n}_y \times n_y$ matrix

$$\Phi_{i,j} = \begin{cases} 1 & \text{if } y_j(\omega_k) \neq 0, \\ 0 & \text{otherwise} \end{cases}$$

for $i = 1, \dots, \hat{n}_y$. Thus the vector $\hat{\mathbf{y}}(\omega_k)$ of length $\hat{n}_y$ ($\hat{n}_y \leq n_y$) denotes the subset of the n_y state variables that are functions of the delay argument ω_k .

Similarly let us denote the *delayed control vector* by

$$\hat{\mathbf{u}}(\omega_k) = \mathbf{\Theta}(\omega_k) \mathbf{u}(\omega_k)$$

where the $\hat{n}_u \times n_u$ matrix

$$\Theta_{i,j} = \begin{cases} 1 & \text{if } u_j(\omega_k) \neq 0, \\ 0 & \text{otherwise} \end{cases}$$

for $i = 1, \dots, \hat{n}_u$.

The independent variable t for phase k is defined in the region $t_I^{(k)} \leq t \leq t_F^{(k)}$. Within phase k the

dynamics of the system are described by a set of *dynamic* variables

$$\mathbf{z} = \begin{bmatrix} \mathbf{y}(t) \\ \hat{\mathbf{y}}(\omega_1(t)) \\ \vdots \\ \hat{\mathbf{y}}(\omega_\nu(t)) \\ \mathbf{u}(t) \\ \hat{\mathbf{u}}(\omega_1(t)) \\ \vdots \\ \hat{\mathbf{u}}(\omega_\nu(t)) \end{bmatrix}.$$

In addition the dynamics may incorporate the n_p *parameters* $\mathbf{p}$ which are not dependent on t . Within phase k the dynamics of the system are defined by a set of ordinary delay-differential equations written in explicit form, which are referred to as the *state equations*

$$\dot{\mathbf{y}} = \mathbf{f}[\mathbf{y}(t), \mathbf{u}(t), \hat{\mathbf{y}}(\omega_1), \dots, \hat{\mathbf{y}}(\omega_\nu), \hat{\mathbf{u}}(\omega_1), \dots, \hat{\mathbf{u}}(\omega_\nu), \mathbf{p}, t]$$

where y is the n_y dimension state vector. Initial conditions at the fixed time t_I are defined by

$$\psi[y(t_I), u(t_I), p, t_I] \equiv \psi_I = 0,$$

and terminal conditions at the fixed time t_F are defined by

$$\psi[y(t_F), u(t_F), p, t_F] \equiv \psi_F = 0.$$

The solution must satisfy *algebraic path constraints* of the form

$$g_\ell \leq \mathbf{g}[\mathbf{y}(t), \mathbf{u}(t), \hat{\mathbf{y}}(\omega_1), \dots, \hat{\mathbf{y}}(\omega_\nu), \hat{\mathbf{u}}(\omega_1), \dots, \hat{\mathbf{u}}(\omega_\nu), \mathbf{p}, t] \leq g_u,$$

where g is a vector of size n_g . Simple bounds can be imposed on the state variables

$$y_\ell \leq y(t) \leq y_u,$$

the control variables

$$u_\ell \leq u(t) \leq u_u,$$

and the parameters

$$p_\ell \leq p \leq p_u.$$

Note that a path variable equality constraint can be imposed if the upper and lower bounds are equal, e.g. $(g_\ell)_k = (g_u)_k$ for some k .

RELATED SUBPROGRAMS

EVALDL	Delay Equation Evaluation
INSNLP	Sparse Nonlinear Programming Input Procedure
SNLPMN	Sparse Nonlinear Programming
SBRNLP	Sparse Barrier Nonlinear Programming

SOSCTL	Sparse Optimal Control
SOSEST	Sparse Optimal Parameter Estimation
INSOCX	Sparse Optimal Control Input Procedure
INPSOS	Sparse Optimization Suite Input Procedure

METHOD

The optimal control problem is solved using a *direct transcription* method. The basic steps in the approach are as follows:

1. **Direct Transcription.** *Transcribe* the optimal control problem into a nonlinear programming (NLP) problem by discretization;
2. **Sparse Nonlinear Program.** Solve the sparse NLP using either a sequential quadratic programming (SQP) or an interior-point (barrier) method;
3. **Mesh Refinement.** Assess the accuracy of the approximation (i.e. the finite dimensional problem), and if necessary refine the discretization, and then repeat the optimization steps.

The SOS software provides a number of discretization techniques including, trapezoidal, Hermite-Simpson, and Runge-Kutta methods. The sparse nonlinear programming problem can be solved using either a *sequential quadratic programming* (SQP) approach or an *interior-point* (barrier) method. There are a number of issues which determine the efficiency of the NLP. First, the Jacobian matrix (of first derivatives) and the Hessian matrix (of second derivatives) needed to solve the NLP are large and sparse, and SOS automatically constructs the sparsity structure. Second, it is necessary to compute the Hessian matrix H_L and within SOS sparse finite difference techniques are utilized. Third, the sparse NLP subproblem can be solved efficiently using a sparse Schur-Complement Method or a primal-dual barrier method. The SOS software also incorporates a technique for computing the discretization error and if necessary refining the mesh to reduce this error.

WARNING

The direct transcription method implemented in the SOS software yields approximate solutions for the state and control functions. The technique is appropriate when the control variables are continuous and the state variables have continuous first derivatives. If the solution to the optimal control problem cannot be represented in this form, unpredictable performance in the numerical methods may be observed. Furthermore the technique may be inappropriate for solving differential-algebraic systems with global index greater than one. This situation can occur either because of the presence of state variable path constraints and/or singular arcs in the solution. Many of these situations can be circumvented by reformulation of the underlying problem to exploit the multi-phase capability of the SOS software.

The user supplied functions which define the objective function, path constraints, point functions, and differential equations are all assumed to be continuous and have continuous first and second derivatives. Non-differentiable functions can cause unpredictable performance in the optimization algorithm. It should be emphasized that discontinuities in the second derivatives can significantly

degrade the speed of convergence without producing any other obvious difficulties. The following common sources of error *should be avoided* when evaluating the objective and constraint functions:

- (1) non-smooth interpolation of data;
- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;
- (3) discontinuous behavior caused by branching for IF tests;
- (4) non-differentiable functions such as ABS, MAX, and MIN.

The user must provide procedures to evaluate the continuous and point functions. If it is not possible to evaluate the functions the user can communicate this information to the algorithm using the function error flag IFERR as described below.

USAGE

```

INTEGER          MAXCS,NEEDED,IER
DOUBLE PRECISION CSTAT(MAXCS)
EXTERNAL  ODEINP,ODEIGS,ODERHS,ODEPTF,ODEPRT,EVALDL

CALL SOSDLY(ODEINP,ODEIGS,ODERHS,ODEPTF,ODEPRT,EVALDL,
$          CSTAT,MAXCS,NEEDED,IER)

```

ARGUMENTS

USER-SUPPLIED SUBPROGRAMS

- | | |
|---------------|--|
| ODEINP | [INPUT, SUBROUTINE]
The name of the user routine which defines the specifics of the optimal control problem (p. 426). |
| ODEIGS | [INPUT, SUBROUTINE]
The name of the user routine which can be called to obtain the initial guess (p. 422). If a sophisticated initial guess is not required, the system supplied dummy routine DUMYIG may be used instead. |
| ODERHS | [INPUT, SUBROUTINE]
The name of the user routine which evaluates the right hand sides of the delay-differential-algebraic equations (p. 455). If there are no delay-differential-algebraic equations, the system supplied dummy routine DUMYDE may be used instead. |
| ODEPTF | [INPUT, SUBROUTINE]
The name of the user routine which evaluates functions at the ends of each phase (p. 450). If there are no point functions, the system supplied dummy routine DUMYPF may be used instead. |

ODEPRT [INPUT, SUBROUTINE]
The name of the user routine which evaluates auxiliary print at each point (p. 446). If sophisticated output is not required, the system supplied dummy routine DUMYPR may be used instead.

EVALDL [INPUT, SUBROUTINE]
The name of the user routine which evaluates the delay arguments $\omega(t)$ (p. 419). If there are no delayed dynamic variables, the system supplied dummy routine DUMYDL may be used instead.

OPTIMAL CONTROL ARGUMENTS

CSTAT [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
Array containing the solution data for all phases in the optimal control problem. An initial guess for the continuous functions $[y(t), u(t)]$ can be specified if CSTAT is *input* and the initialization flag $\text{INIT} = 3$ is set in subroutine ODEINP. After terminating the final value for the continuous variables $[y(t), u(t)]$ is represented in the array CSTAT which is *output*.

MAXCS [INPUT, INTEGER]
Maximum length of the array, CSTAT.

NEEDED [OUTPUT, INTEGER]
Storage needed in the array CSTAT. When $\text{IER} \neq 0$ to indicate that SOSDLY has failed to compute a solution, an error message will be printed. Section 6.5 describes the optimal control error messages. NEEDED contains the required value for the quantity MAXCS.

IER [OUTPUT, INTEGER]
Error return flag; $\text{IER} = 0$ for a successful termination. When $\text{IER} \neq 0$ to indicate that SOSDLY has failed to compute a solution, an error message will be printed. Section 6.5 describes the optimal control error messages.

USAGE REMARKS

Optimal Solution Representation

All information that defines the optimal solution is contained in the array CSTAT. The optimal values for the continuous functions are represented as either a B-spline or piecewise polynomial.

The utility routine OCSEVL (p. 487) can be used to evaluate the solution at an arbitrary point. The utility routine AUXOUT (p. 464) can be used to create an output file from the information in the CSTAT array. For a multiphase problem the CSTAT array contains the coefficients for all phases stored in a single contiguous array. The utility routine ICSTPH (p. 479) can be used to extract the relevant information that defines the solution.

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine SOSDLY. There are three ways that optional inputs can be specified to the SOS software. The first method is to call the input subroutine INPSOS (p. 401). The second

method which can be used to define input for the sparse NLP is to call the input routine INSNLP (p. 226). The third approach is to read an input file by utilizing subroutine CTLFIL (p. 472). It should be emphasized that it is *not necessary* to call INPSOS, INSNLP or CTLFIL if the default values are appropriate.

Usage of the routine INPSOS is described on page 401. The sole argument of the routine is a character string of the form '*symbol = value*', where '*symbol*' is the mnemonic for a variable name which is to be set, and '*value*' is the corresponding integer, floating point, or character constant to be used. Exceptions to this form are the character strings, 'DEFAULT', 'OPTIONS', and 'FULL OPTIONS'. The preceding strings, or those representing mnemonics, can be in upper, lower, or mixed case.

Sample calls to INPSOS are of the form:

```
CALL INPSOS('IPUOCP = 11')
CALL INPSOS('odetol = 1.e-04')
```

The sparse nonlinear programming algorithms SNLPMN (p. 67) and SBRNLP (p. 171) which are called by SOSDLY utilize a similar input mechanism. In fact, there are three functionally equivalent methods for setting the sparse NLP options. The first approach is to directly call the sparse NLP input procedure INSNLP (p. 226) from the main program. The second approach is to call the SOCX input procedure INSOCX (p. 403), with the special string 'INSNLP:*symbol = value*'. The third approach is to call the SOS input procedure INPSOS (p. 401). The following examples illustrate the options:

```
C
C   THE FOLLOWING CALL TO THE SPARSE NLP
C
C   CALL INSNLP('ALGOPT = FM')
C
C   IS EQUIVALENT TO
C
C   CALL INSOCX('INSNLP:ALGOPT = FM')
C
C-----
C
C   THE FOLLOWING CALL TO THE SPARSE NLP
C
C   CALL INSNLP('MAXNFE = 100')
C
C   IS EQUIVALENT TO
C
C   CALL INPSOS('MAXNFE = 100')
C
```

Another way of setting optimal control values is through the routine, CTLFIL. This routine will

read a user specified file where each line corresponds to a character string like those described above.

```
CHARACTER*80 FILNAM
INTEGER INUNIT

CALL CTLFIL(INUNIT,FILNAM)
```

OPTIONAL PRINTOUT

The level of printed output from SOSDLY is controlled by the variables IPGRD, IPNLP, IPODE and IPUOCP which are set by calls to INPSOS. A complete description of the iteration output options is found in Section [6.6](#).

SOSEST: Sparse Optimal Parameter Estimation

PURPOSE

The purpose of SOSEST is to determine the $n_u^{(k)}$ -dimensional control vectors $u^{(k)}(t)$ and parameters $p^{(k)}$ to minimize the performance index

$$J = \frac{1}{2} r^\top r$$

where r is the L -dimensional *residual* vector. The form of the residual vector is defined below. Notice that the objective function may depend on quantities computed in each of the N phases.

The independent variable t for phase k is defined in the region $t_I^{(k)} \leq t \leq t_F^{(k)}$. Within phase k the dynamics of the system are described by a set of *dynamic* variables

$$z = \begin{bmatrix} y^{(k)}(t) \\ u^{(k)}(t) \end{bmatrix}$$

made up of the $n_y^{(k)}$ *state variables* and the $n_u^{(k)}$ *control variables* respectively. In addition the dynamics may incorporate the $n_p^{(k)}$ *parameters* $p^{(k)}$ which are not dependent on t . Within phase k the dynamics of the system are defined by a set of ordinary differential equations written in explicit form, which are referred to as the *state equations*

$$\dot{y} = f[y(t), u(t), p, t]$$

where y is the n_y dimension state vector. Initial conditions at t_I are defined by

$$\psi[y(t_I), u(t_I), p, t_I] \equiv \psi_I = 0,$$

and terminal conditions at t_F are defined by

$$\psi[y(t_F), u(t_F), p, t_F] \equiv \psi_F = 0.$$

In addition the solution must satisfy *algebraic path constraints* of the form

$$g_l \leq g[y(t), u(t), p, t] \leq g_u,$$

where g is a vector of size n_g . The algebraic constraints are of the form

$$g[y(t), u(t), p, t] = \alpha^\top v + \beta^\top a[v, t]$$

where

$$v = \begin{bmatrix} y(t) \\ u(t) \\ p \end{bmatrix}.$$

The constraint definition can include *analytic* terms involving $\alpha^\top v$ where the $(n_y + n_u + n_p)$ vector α is constant, as well as linear combinations of the n_a *auxiliary functions* $a_k(v)$ for $k = 0, \dots, n_a$ where the coefficients β_k are nonzero constants. By convention, a path constraint with a single nonlinear term $a_0(y, u, p, t)$ has no auxiliary functions ($n_a = 0$). Observe that each individual path constraint may have a different number of auxiliary functions and analytic terms. Simple bounds can be imposed on the state variables

$$y_L \leq y(t) \leq y_U,$$

the control variables

$$u_L \leq u(t) \leq u_U,$$

and the parameters

$$p_L \leq p \leq p_U.$$

Note that a path variable equality constraint can be imposed if the upper and lower bounds are equal, e.g. $(g_L)_k = (g_U)_k$ for some k .

Components of the residual vector can be of two forms. State residuals on phase k are of the form

$$r_\ell = w_{ij} \left[y_i^{(k)}(\theta_{ij}) - \hat{y}_{ij}^{(k)} \right]$$

where $y_i^{(k)}(\theta_{ij})$ is the value of state variable i computed at time θ_{ij} and $\hat{y}_{ij}$ is the observed value at the same point. Algebraic residuals are of the form

$$r_\ell = w_{ij} \left[u_i^{(k)}(\theta_{ij}) - \hat{u}_{ij}^{(k)} \right]$$

where $u_i^{(k)}(\theta_{ij})$ is the value of algebraic variable i computed at time θ_{ij} and $\hat{u}_{ij}^{(k)}$ is the observed value at the same point. The quantities w_{ij} are residual scale weights. It is required that data evaluation points satisfy

$$t_I^{(k)} \leq \theta_{ij} \leq t_F^{(k)}.$$

Often the evaluation points are arranged monotonically, that is $\theta_{i,j} \leq \theta_{i,j+1}$. It is also common to have many variables evaluated at the same time, e.g. $\theta_{i,j} = \theta_{i+1,j}$. Neither of these assumptions are necessary for the software, however the initial and final phase times $t_I^{(k)}$ and $t_F^{(k)}$ must be fixed.

RELATED SUBPROGRAMS

INSNLP	Sparse Nonlinear Programming Input Procedure
SNLPLS	Sparse Constrained Nonlinear Least Squares
SBRLSQ	Sparse Barrier Constrained Nonlinear Least Squares
INSOCX	Sparse Optimal Control Input Procedure
INPSOS	Sparse Optimization Suite Input Procedure

METHOD

The parameter estimation problem is solved using a *direct transcription* method. The basic steps in the approach are as follows:

1. **Direct Transcription.** *Transcribe* the parameter estimation problem into a nonlinear programming (NLP) problem by discretization;
2. **Sparse Nonlinear Program.** Solve the sparse NLP using a sequential quadratic programming (SQP) method;
3. **Mesh Refinement.** Assess the accuracy of the approximation (i.e. the finite dimensional problem), and if necessary refine the discretization, and then repeat the optimization steps.

The SOS software provides a number of discretization techniques including, trapezoidal, Hermite-Simpson, and Runge-Kutta methods. The sparse nonlinear programming problem is solved using either a *sequential quadratic programming* (SQP) approach or an *interior-point* (barrier) method. There are a number of issues which determine the efficiency of the NLP. First, the Jacobian matrix (of first derivatives) and the Hessian matrix (of second derivatives) needed to solve the NLP are large and sparse, and SOS automatically constructs the sparsity structure. Second, it is necessary to compute the Hessian matrix H_L and within SOS sparse finite difference techniques are utilized. Third, the sparse NLP subproblem can be solved efficiently using a sparse Schur-Complement Method. The SOS software also incorporates a technique for computing the discretization error and if necessary refining the mesh to reduce this error.

WARNING

The direct transcription method implemented in the SOS software yields approximate solutions for the state and control functions. The technique is appropriate when the control variables are continuous and the state variables have continuous first derivatives. If the solution to the parameter estimation problem cannot be represented in this form, unpredictable performance in the numerical methods may be observed. Furthermore the technique may be inappropriate for solving differential-algebraic systems with global index greater than one. This situation can occur either because of the presence of state variable path constraints and/or singular arcs in the solution. Many of these situations can be circumvented by reformulation of the underlying problem to exploit the multi-phase capability of the SOS software.

The user supplied functions which define the objective function, path constraints, point functions, and differential equations are all assumed to be continuous and have continuous first and second derivatives. Non-differentiable functions can cause unpredictable performance in the optimization algorithm. It should be emphasized that discontinuities in the second derivatives can significantly degrade the speed of convergence without producing any other obvious anomalies. The following common sources of error *should be avoided* when evaluating the objective and constraint functions:

- (1) non-smooth interpolation of data;
- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;
- (3) discontinuous behavior caused by branching for IF tests;
- (4) non-differentiable functions such as ABS, MAX, and MIN.

The user must provide procedures to evaluate the continuous and point functions. If it is not possible to evaluate the functions the user can communicate this information to the algorithm using the function error flag IFERR as described below.

USAGE

```

INTEGER          MAXCS, NEEDED, IER
DOUBLE PRECISION CSTAT(MAXCS)
EXTERNAL         ODEINP, ODEIGS, ODERHS, ODEPTF, ODEPRT, DDLOAD

CALL SOSEST(ODEINP, ODEIGS, ODERHS, ODEPTF, ODEPRT, DDLOAD,
```

\$ CSTAT, MAXCS, NEEDED, IER)

ARGUMENTS

USER-SUPPLIED SUBPROGRAMS

- ODEINP** [INPUT, SUBROUTINE]
The name of the user routine which defines the specifics of the parameter estimation problem to solve (p. 426).
- ODEIGS** [INPUT, SUBROUTINE]
The name of the user routine which can be called to obtain the initial guess (p. 422). If a sophisticated initial guess is not required, the system supplied dummy routine DUMYIG may be used instead.
- ODERHS** [INPUT, SUBROUTINE]
The name of the user routine which evaluates the right hand sides of the differential-algebraic equations (p. 455). If there are no differential-algebraic equations, the system supplied dummy routine DUMYDE may be used instead.
- ODEPTF** [INPUT, SUBROUTINE]
The name of the user routine which evaluates functions at the ends of each phase (p. 450). If there are no point functions, the system supplied dummy routine DUMYPF may be used instead.
- ODEPRT** [INPUT, SUBROUTINE]
The name of the user routine which evaluates auxiliary print at each point (p. 446). If sophisticated output is not required, the system supplied dummy routine DUMYPR may be used instead.
- DDLOAD** [INPUT, SUBROUTINE]
The name of the user routine which loads the discrete data values (p. 416).

PARAMETER ESTIMATION ARGUMENTS

- CSTAT** [INPUT/OUTPUT, DOUBLE PRECISION, ARRAY]
Array containing the solution data for all phases in the optimal control problem. An initial guess for the continuous functions $[y(t), u(t)]$ can be specified if CSTAT is *input* and the initialization flag $INIT = 3$ is set in subroutine ODEINP. After terminating the final value for the continuous variables $[y(t), u(t)]$ is represented in the array CSTAT which is *output*.
- MAXCS** [INPUT, INTEGER]
Maximum length of the array, CSTAT.
- NEEDED** [OUTPUT, INTEGER]
Storage needed in the array CSTAT. When $IER \neq 0$ to indicate that SOSCTL has failed to compute a solution, an error message will be printed. Section 6.5 describes the optimal control error messages. NEEDED contains the required value for the quantity MAXCS.

IER [OUTPUT, INTEGER]
 Error return flag; IER = 0 for a successful termination. When IER $\neq$ 0 to indicate that SOSEST has failed to compute a solution, an error message will be printed. Section 6.5 describes the parameter estimation error messages.

USAGE REMARKS

Optimal Solution Representation

All information that defines the optimal solution is contained in the array CSTAT. The optimal values for the continuous functions are represented as either a B-spline or piecewise polynomial.

The utility routine OCSEVL (p. 487) can be used to evaluate the solution at an arbitrary point. The utility routine AUXOUT (p. 464) can be used to create an output file from the information in the CSTAT array. For a multiphase problem the CSTAT array contains the coefficients for all phases stored in a single contiguous array. The utility routine ICSTPH (p. 479) can be used to extract the relevant information that defines the solution.

OPTIONAL INPUT

In addition to the required arguments, there are a number of optional arguments that can be specified for subroutine SOSEST. There are three ways that optional inputs can be specified to the SOS software. The first method is to call the input subroutine INPSOS (p. 401). The second method which can be used to define input for the sparse NLP is to call the input routine INSNLP (p. 226). The third approach is to read an input file by utilizing subroutine CTLFIL (p. 472). It should be emphasized that it is *not necessary* to call INPSOS, INSNLP or CTLFIL if the default values are appropriate.

Usage of the routine INPSOS is described on page 401. The sole argument of the routine is a character string of the form '*symbol* = *value*', where '*symbol*' is the mnemonic for a variable name which is to be set, and '*value*' is the corresponding integer, floating point, or character constant to be used. Exceptions to this form are the character strings, 'DEFAULT', 'OPTIONS', and 'FULL OPTIONS'. The preceding strings, or those representing mnemonics, can be in upper, lower, or mixed case.

Sample calls to INPSOS are of the form:

```
CALL INPSOS('IPUOCP = 11')
CALL INPSOS('odetol = 1.e-04')
```

The sparse nonlinear programming algorithms SNLPLS (p. 86) and SBRLSQ (p. 191) which are called by SOSEST utilize an input mechanism similar to the other optimization algorithms in the library. In fact, there are three functionally equivalent methods for setting the sparse NLP options. The first approach is to directly call the sparse NLP input procedure INSNLP (p. 226) from the main program. The second approach is to call the SOCX input procedure INSOCX (p. 403), with the special string 'INSNLP:*symbol* = *value*'. The third approach is to call the SOS input procedure INPSOS (p. 401). The following examples illustrate the options:

```
C
C   THE FOLLOWING CALL TO THE SPARSE NLP
C
C   CALL INSNLP('ALGOPT = FM')
C
C   IS EQUIVALENT TO
C
C   CALL INSOCX('INSNLP:ALGOPT = FM')
C
C-----
C
C   THE FOLLOWING CALL TO THE SPARSE NLP
C
C   CALL INSNLP('MAXNFE = 100')
C
C   IS EQUIVALENT TO
C
C   CALL INPSOS('MAXNFE = 100')
C
```

Another way of setting parameter estimation values is through the routine, CTLFIL. This routine will read a user specified file where each line corresponds to a character string like those described above.

```
CHARACTER*80  FILNAM
INTEGER  INUNIT

CALL CTLFIL(INUNIT,FILNAM)
```

OPTIONAL PRINTOUT

The level of printed output from SOEST is controlled by the variables IPGRD, IPNLP, IPODE and IPUOCP which are set by calls to INPSOS. A complete description of the iteration output options is found in Section [6.6](#).

INPSOS: Sparse Optimization Suite Input Procedure

PURPOSE

INPSOS is a utility subroutine used to define optional inputs for any of the software modules in the sparse optimization suite SOS.

RELATED SUBPROGRAMS

INSNLP Sparse Nonlinear Programming Input Procedure

INSOCX Sparse Optimal Control Input Procedure

METHOD

The sparse optimization suite consists of a collection of algorithms organized into two groups. The optimal control group referred to collectively as the SOCX group, consists of the following algorithms:

SOSCTL Sparse Optimal Control
SOSDLY Sparse Optimal Control of Delay Systems
SOSEST Sparse Optimal Parameter Estimation

The nonlinear programming group referred to collectively as the SNLP group, consists of the following algorithms

DNLPFW Dense Nonlinear Programming
DNLPGF Dense Nonlinear Programming–User Supplied Gradients
DNLPRV Dense Nonlinear Programming–Reverse Communication Format
SNLPMN Sparse Nonlinear Programming
SNLPLS Sparse Constrained Nonlinear Least Squares
SHURQP Sparse Quadratic Programming
DBNLPF Dense Barrier Nonlinear Programming
DBNPGF Dense Barrier Nonlinear Programming–User Supplied Gradients
DBNLPR Dense Barrier Nonlinear Programming–Reverse Communication Format
SBRNLP Sparse Barrier Nonlinear Programming
SBRLSQ Sparse Barrier Constrained Nonlinear Least Squares
SBRLPQ Sparse Barrier Quadratic Programming

Subroutine INPSOS can be used to specify optional input to any algorithm in either group. Thus suppose it is desired to set the differential equation accuracy ODETOL, and the maximum number of NLP iterations NITMAX. There are three equivalent ways to do so:

```
CALL INSOCX('ODETOL = 1.D-7')  
CALL INSNLP('NITMAX = 150')
```

or

```
CALL INSOCX('ODETOL = 1.D-7')  
CALL INSOCX('INSNLP:NITMAX = 150')
```

or

```
CALL INPSOS('ODETOL = 1.D-7')  
CALL INPSOS('NITMAX = 150').
```

Subroutine INPSOS calls the appropriate lower level routine INSOCX (p. 403) or INSNLP (p. 226). Acceptable values for the input character string can be is found in sections 6.2 and 2.2.

INSOCX: Sparse Optimal Control Input Procedure

PURPOSE

INSOCX is a utility subroutine used to define optional inputs for the sparse optimal control extended software `SOCX` implemented in subroutine `SOSCTL`, `SOSDLY`, and `SOSEST`.

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control
SOSEST	Sparse Optimal Parameter Estimation
SOSDLY	Sparse Optimal Control of Delay Systems
SNLPMN	Sparse Nonlinear Programming
SBRNLP	Sparse Barrier Nonlinear Programming
SNLPLS	Sparse Constrained Nonlinear Least Squares
SBRLSQ	Sparse Barrier Constrained Nonlinear Least Squares
INSNLP	Sparse Nonlinear Programming Input Procedure
INPSOS	Sparse Optimization Suite Input Procedure

METHOD

Subroutine INSOCX must be called once for every optional parameter to be set. The input to INSOCX must be a single character variable of length not to exceed 80 characters. The string is case insensitive and is assumed to have the following format:

' symbol = value '

The symbol must be one of the valid mnemonics defined below; see ARGUMENTS. Information contained in the character string is communicated to the sparse optimal control algorithm `SOSCTL`, the sparse optimal parameter estimation algorithm `SOSEST`, and the sparse optimal delay algorithm `SOSDLY` via three internal commons. The character string must have an equals sign “=” to separate the *symbol* from the *value*. The only exceptions to this rule are when *symbol* = `'DEFAULT'`, or `'OPTIONS'`. The *value* portion of the string can contain integer, real (single or double precision), or character values.

The sparse nonlinear programming algorithms called by `SOSCTL`, `SOSDLY`, and `SOSEST` utilize a similar input mechanism. In fact, there are two functionally equivalent methods for setting the sparse NLP options. The first approach is to directly call the sparse NLP input procedure `INSNLP` (p. 226) from the main program. The second approach is to call the `SOCX` input procedure `INSOCX`, with the special string

'INSNLP:symbol = value'.

All parameters not specified by the user are automatically set to their default values. Any optional parameters that are set by the user are not altered by algorithms `SOSCTL`, `SOSEST`, `SOSDLY`,

SBRNLP, SBRLSQ, SNLPLS and SNLPMN, and hence changes to the options are cumulative. Therefore any input changes made by the user will remain in effect until they are explicitly reset by another call to INSOCX (or INSNLP).

There are two possible sources for error when using INSOCX;

- (1) The character string input to INSOCX is invalid,
- (2) The character string input to INSOCX is valid, but the *value* is incorrect for algorithms SOSCTL, SOSEST, SOSDLY, SBRNLP, SBRLSQ, SNLPLS and SNLPMN.

Errors encountered by INSOCX as a result of incorrect character strings (class 1) are communicated to the user by a call to the error handler HHERR, followed by a call to HHERRX to display the incorrect string. A list of the character string errors is given below. Input errors caused by an incorrect *value* (i.e. class 2) are detected by algorithms SOSCTL, SOSEST, SOSDLY, SBRNLP, SBRLSQ, SNLPLS and SNLPMN when called and the information is communicated to the user by the appropriate error return flag.

USAGE

```
CHARACTER STRING*k
CALL INSOCX (STRING)
```

ARGUMENTS

STRING [INPUT, CHARACTER]
 Character variable of length k where $k \leq 80$, having one of the following forms:
 STRING(1:k) = '*symbol = value*'

 STRING(1:k) = 'INSNLP:*symbol = value*'

SOCX options set with '*symbol = value*'

SOCX Convergence and Termination Tolerances

The following optional inputs can be used to control convergence and termination: Symbol

<u>Default</u>	<u>Description</u>	
OBJTOL	10^{-5}	Relative error in the performance index between final value and predicted optimum.
ODETOL	10^{-3}	Relative error in the solution of the differential equations on any phase. ODETOL is ignored if METHOD = 5 on the phase.
SMLTOL	$\epsilon^{\frac{1}{2}}$	ODE variable absolute tolerance, where $\epsilon = \text{HDMCON}(5)$. Discretization error is a relative test based on ODETOL, when the magnitude of the variable exceeds SMLTOL.

SOCX Algorithm Control Parameters

The following optional inputs can be used to specialize the numerical processes in the SOCX algorithm:

<u>Symbol</u>	<u>Default</u>	<u>Description</u>
IDTSFD	1	Default derivative type and index set construction algorithm for sparse finite differences (may be superceded by NLP).
		IDTSFD > 0 Curtis-Powell-Reid (greedy) algorithm.
		IDTSFD < 0 Coleman-Moré (graph coloring) algorithm.
		IDTSFD = 1 Forward difference (1-sided) derivatives.
		IDTSFD = 2 Central difference (2-sided) derivatives.
IPRSFD	1	Perturbation size option.
		IPRSFD = 0 Perturbation size fixed at PRTSFD.
		IPRSFD = +1 Perturbation size between PRTMSD and PRTMXD computed to minimize truncation and roundoff error .
IRSTRT	0	Flag determining usage of restart file RSTFIL.
		IRSTRT = -1 Save file during solution process (write only).
		IRSTRT = 0 Do not use file.
		IRSTRT = +1 Use file to start problem and update during solution process (read and write).
		IRSTRT = +2 Use file to start problem but do not update during solution process (read only).

ISCALE	0	Automatic Scaling Option; see usage remarks (p. 413). The "ones" digit of ISCALE = 0 Automatic Row and Column Scaling. = 1 Automatic Row, Simple Column Scaling. = 2 Simple Row and Column Scaling. = 3 Automatic Row/Column Scaling Except Obj. The "tens" digit of ISCALE = 0 Ignore scalewgt.fil = 1 Read scalewgt.fil (if it exists) = 2 Write scalewgt.fil = 3 Read/Write scalewgt.fil
M5DTYP	0	Method 5 Derivative Type. M5DTYP= 0 Internal (Variational) Derivatives. M5DTYP= +1 External (Finite Difference) Derivatives.
MITODE	5	Maximum number of mesh refinement iterations (grids). MITODE=0 Do data check to size problem.
NLPALG	SNLPMN	Nonlinear Programming Algorithm (SNLPMN, SBRLSQ, SNLPLS or SBRNLP).
NLPOMR	M	Nonlinear Programming option (cf ALGOPT (p. 234)) for mesh refinement. This option controls the NLP algorithm during mesh refinement iterations, but does not alter the first refinement iteration.
NIWOPI	10000000	Initial integer storage allocation for optimization. Dynamic reallocation will be done as necessary.
NRWOPI	20000000	Initial real storage allocation for optimization. Dynamic reallocation will be done as necessary.
PRTMSD	10^{-8}	Minimum perturbation size for sparse finite difference derivatives.
PRTMXD	10^{-3}	Maximum perturbation size for sparse finite difference derivatives.
PRTSFD	10^{-5}	Relative perturbation size for sparse finite difference derivatives.

SPRTHS	SPARSE	Right Hand Side Sparsity Option. SPRTHS=SPARSE Compute and use sparsity for right hand sides of the user supplied functions.
		SPRTHS=DENSE Assume user supplied functions are dense.
		SPRTHS=FILE Right Hand Side Sparsity Defined by User Supplied File RHSTMP; see usage remarks (p. 411).
TOLJSD	10^{-6}	Jacobian row and column tolerance for print.
TOLM5A	$\epsilon^{\frac{1}{2}}$	Absolute integration tolerance ($\epsilon = \text{HDMCON}(5)$). Only used when METHOD = 5 and $ \text{NSTG}(1) \neq 6$.
TOLM5R	$\epsilon^{\frac{1}{2}}$	Relative integration tolerance ($\epsilon = \text{HDMCON}(5)$). Only used when METHOD = 5 and $ \text{NSTG}(1) \neq 6$.

SOCX Output Control

Output from the numerical processes in the SOCX software is controlled by setting one or more of the flags listed below.

<u>Symbol</u>	<u>Default</u>	<u>Description</u>
IDSCPH	0	Discontinuity diagnostic phase number. Display diagnostic information on phase IDSCPH, where $1 \leq \text{IDSCPH} \leq \text{NPHS}$. No diagnostic information is displayed unless IPGRD ≥ 10 , and the inputs IDSCND, IDSCVR, and IDSCFN are also set.
IDSCND	0	Discontinuity diagnostic phase end. Display diagnostic information at the beginning of the phase when IDSCND = - 1, during the phase when IDSCND = 0, and at the end of the phase when IDSCND = + 1. No diagnostic information is displayed unless IPGRD ≥ 10 , and the inputs IDSCPH, IDSCVR, and IDSCFN are also set.
IDSCVR	0	Discontinuity diagnostic variable number. Display diagnostic information for variable IDSCVR, where $1 \leq \text{IDSCVR} \leq \text{NCF}(1) + \text{NAV} + \text{NPV}$. To display state variable k, set IDSCVR = k. To display control variable k, set IDSCVR = k + NCF(1). To display parameter k, set IDSCVR = k + NAV + NCF(1). No diagnostic information is displayed unless IPGRD ≥ 10 , and the inputs IDSCPH, IDSCND, and IDSCFN are also set.

IDSCFN	0	Discontinuity diagnostic function number. When IDSCND = +1, or -1, set IDSCND = k to display point function k. When IDSCND = 0, set IDSCFN = k to display differential equation k. When IDSCND = 0, set IDSCFN = NCF(1) + k to display algebraic equation k. When IDSCND = 0, set IDSCFN = NCF(1) + NCF(2) + k to display quadrature equation k. No diagnostic information is displayed unless IPGRD $\geq$ 10, and the inputs IDSCPH, IDSCND, and IDSCVR are also set.
IPFSFD	0	Sparse finite difference output level (equivalent to MSGGLVL in HDSFDH, HDSFDJ, and HJSFDI); $0 \leq \text{IPFSFD} \leq 3$. 0=IPFSFD No Output. 1=IPFSFD Error messages and summary statistics. 2=IPFSFD Complete statistics. 3=IPFSFD Diagnostic output
IPGRD	10	Optimal control output level; $0 \leq \text{IPGRD} \leq 20$. IPGRD=0 No Output. 0<IPGRD<10 Terse Output. 10 $\leq$ IPGRD<20 Phase level grid and errors. 20 $\leq$ IPGRD State level errors, mathematical input interpretation, NLP variable and constraint maps.
IPNLP	10	Sparse NLP output level (equivalent to IOFLAG in INSNLP (p. 226)); $0 \leq \text{IPNLP} \leq 30$. IPNLP=0 No Output. 0<IPNLP<10 Terse Output. 10 $\leq$ IPNLP<20 Standard Output. 20 $\leq$ IPNLP<30 Interpretive Output. 30=IPNLP Diagnostic Output.

IPODE	0	Differential equation output level; $0 \leq \text{IPODE} \leq 40$. $0 \leq \text{IPODE} < 10$ No Output. $10 \leq \text{IPODE} < 20$ Phase end states, max constraint error. $20 \leq \text{IPODE} < 30$ States/Constraints at every grid point. $30 \leq \text{IPODE} < 40$ Gradient, Jacobian, and Hessian. $40 \leq \text{IPODE}$ Debug print at initial point.
IPUOCP	6	Output unit number for iteration output from algorithm SOSCTL, SOSDLY, or SOSEST.
ISFHES	41	FORTTRAN I/O unit for scratch Hessian storage.
ISFINP	42	FORTTRAN I/O unit for scratch file.
ISFRST	43	FORTTRAN I/O unit for restart file.
ISFSCL	44	FORTTRAN I/O unit for scale weight input.
KEYDPL	.lueiLUE	Dynamic Status Display Key. A character*8 variable that defines the key for the dynamic status display. KEYDPL(1:1) defines the symbol that will be displayed when an inequality constraint is satisfied. KEYDPL(2:2) defines the symbol that will be displayed when an inequality is active at its lower bound. Thus, to denote active upper bounds by a 'U', and display nothing for all other constraints, the input string should be 'KEYDPL=uuUuuuuu'. The default display key has the following interpretation: . Satisfied Inequality l Active Lower Bound ('z' denotes a near zero multiplier) u Active Upper Bound ('Z' denotes a near zero multiplier) e Equality i Ignored L Violated Lower Bound U Violated Upper Bound E Violated Equality
RHSTMP	RHSTMPLT	Character variable name of right hand side sparsity template file prefix. On input a file "RHSTMPLT.inp" must be supplied. On output a file "RHSTMPLT.out" will be created.
RSTFIL	socx.restart	Character variable name of restart file used when IRSTRT $\neq 0$.
SCLFIL	scalewgt.fil	Character variable name of scale weight file used when ISCALE ≥ 10 .

MRFMET	MINMAX	Mesh Refinement Method. =MINMAX Minimize Maximum Error (mrfopt=0). =BISECT Interval Bisection (mrfopt=1). =EQUIDS Equidistribute Error (mrfopt=2).
--------	--------	--

SOCOUT Character variable defining the SOCX output control menu. The complete SOCX output is divided into distinct portions. Each portion of the SOCX output is defined by an alphabetic character. The level of output for each portion is specified by a number from 0 to 9, where 0 specifies no output, and more detailed output is obtained by a larger number. A default output configuration is defined by the quantities IPGRD and IPODE. For example the statements

```
CALL INSOX('IPGRD=20')
```

```
CALL INSOX('IPODE=20')
```

are equivalent to the single statement

```
CALL INSOX('SOCOUT=a0b0c1d1e1f1g0h1i2j2k2l1m1n1o1p1q1r1s1').
```

Thus, to alter the default output such that the variables, constraints, and objective function are printed at every iteration, the following sequence can be used:

```
CALL INSOX('IPGRD=20')
```

```
CALL INSOX('IPODE=20')
```

```
CALL INSOX('SOCOUT=I4J4K4')
```

All output from SOCX can be eliminated using the special command

```
CALL INSOX('SOCOUT=NONE')
```

The distinct portions of the SOCX output are identified as follows:

- A Mathematical Optimal Control Interpretation.
- B Right Hand Side Sparsity Pattern.
- C Transcription Summary.
- D SOCX I/O Parameters.
- E Variable Grid Map.
- F Constraint Grid Map.
- G Optimal Control Scale Information.
- H Index Sets and Sparsity Summary.
- I Variables.
- J Objective Function.
- K Constraints.
- L Control Analysis Constraint Error.
- M Relative DAE Error.
- N Dynamic Status Display.
- O Dynamic Constraint Adjoints.
- P Dynamic Variable Adjoints.

- Q Control Analysis Lagrangian Errors.
- R Control Analysis Optimality Error.
- S Grid Refinement Summary.

Auxiliary Output Control

The ability to produce auxiliary output is implemented using the AUXOUT (p. 464) subroutine. The following quantities can be used to control the auxiliary output:

<u>Symbol</u>	<u>Default</u>	<u>Description</u>
DTAUX	0.	Step size taken between calls to auxiliary routine ODEPRT during output. Used only if NPTAUX = 0.
IPFAUX	0	Output level for auxiliary print.
IPUAUX	0	FORTTRAN unit number for auxiliary print. Opened by user prior to SOSCTL, SOSDLY, or SOSEST call.
NPTAUX	100	Number of auxiliary print points per phase (see also DTAUX in the description of AUXOUT (p. 464)).

Sparse NLP options set with 'INSNLP:symbol = value'

All optional input for the sparse nonlinear programming algorithm can be set using INSOCX. For example, the maximum number of iterations for the (SQP) NLP algorithm can be set to 150 using either

```
CALL INSNLP('NITMAX = 150')
```

or

```
CALL INSOCX('INSNLP:NITMAX = 150')
```

For a complete list of the nonlinear programming algorithm inputs consult INSNLP (p. 226).

SOCX Special Options

Two special options can be obtained from INSOCX by setting STRING(1:k) = '*symbol*' (with no "value"). The following inputs describe the special options available:

<u>Symbol</u>	<u>Description</u>
DEFAULT	Set default values for all optional inputs.
OPTIONS	Write a description of the optional parameters.

USAGE REMARKS

Sparse Right Hand Side Matrix Input

The sparsity pattern of the user supplied functions can be specified by the user when the input “SPRTHS = FILE.” Specifically the user must define the pattern for the following *sparsity template* matrix:

$$\mathcal{T} = \begin{pmatrix} f_y & f_u & f_p \\ g_y & g_u & g_p \\ q_y & q_u & q_p \\ a_y & a_u & a_p \\ \psi_y^{(I)} & \psi_u^{(I)} & \psi_p^{(I)} \\ \psi_y^{(F)} & \psi_u^{(F)} & \psi_p^{(F)} \end{pmatrix}.$$

where

$$f_y \equiv \begin{pmatrix} \frac{\partial f_1}{\partial y_1} & \cdots & \frac{\partial f_1}{\partial y_{n_y}} \\ \vdots & & \\ \frac{\partial f_{n_y}}{\partial y_1} & \cdots & \frac{\partial f_{n_y}}{\partial y_{n_y}} \end{pmatrix}$$

with a similar definition for the other blocks of the template. The sparsity template may be specified using four different options. The following example illustrates all four possibilities:

```

phase = 1
option = 1
x xx x
  xx x
xx   x
phase = 2
option = 2
x xx x
  xx x
xx   x
phase = 3
option = 3
1 1
3 1
2 2
3 2
1 3
2 3
1 4
2 5
1 6
3 6
phase = 4
option = 4
1 1
1 3
2 3
1 4
2 5
1 6

```



```

3 6
3 1
2 2
3 2

```

The sparsity template is specified for each phase following the string “phase =”. The data can be input as characters with an “x” used to denote a nonzero, and a blank otherwise. A second alternative is to specify the pattern using sparse matrix notation “irow, jcol.” The row and column number for each nonzero element is specified as a “double” on a single line. When the matrix format is used the elements can be specified in any order (e.g. phase 3 and 4 have the same pattern in the example). Regardless of the format of the input, the sparsity pattern can be used either to augment the finite difference pattern or instead of the finite difference pattern.

Option	Description
1	Character format, augment finite difference pattern
2	Character format, ignore finite difference pattern
3	Nonzero format, augment finite difference pattern
4	Nonzero format, ignore finite difference pattern

The character variable RHSTMP contains the prefix for the right hand side template file. On input, the user must supply a file with the suffix “.inp”, as in “RHSTMPLT.inp”. On output, another file with the suffix “.out”, will be created, that is, “RHSTMPLT.out”. When either option 1 or 3 is used, the output file created may be different than the original user supplied input file. Thus, when constructing the right hand side sparsity template for a new problem, it may be useful to first use option 1 or 3 to construct a template file, and then manually edit the file for subsequent applications.

Problem Scaling

The natural variables and constraints as they appear in the optimal control problem can be scaled to improve the numerical conditioning and efficiency of the underlying nonlinear programming problem. Scale weights can be defined by the user and/or computed by the SOCX algorithm. Scale weights must be positive, and are defined for each variable and constraint present on a phase. For illustration consider the typical scale weight information as displayed in the iteration output:

	DEFAULT	SCALE WEIGHT
T	0.121417	0.121372
Y01	0.153940E-03	0.153868E-03
Y02	0.552714	0.555094
Y03	3.11648	3.11514
U01	0.518017E-02	0.518017E-02
D01	0.153868E-03	0.199987E-03
D02	0.555094	0.139343E-02
D03	3.11514	1.50000
G01	0.518017E-02	0.518017E-02
B01	0.222222E-01	0.222222E-01
B02	0.153868E-03	0.200000E-03
B03	0.126632E-02	0.142857E-02
B04	0.595597	1.50000

B05	0.968217E-01	0.113766E-01
B06	0.121372	0.666667E-01

All variable scale weights appear first—in this case the time T, the states Y01, . . . , Y03 and the control U01. Scaling for the defect constraints D01, . . . , D03, is followed by values for the path constraint, G01, and boundary conditions B01, . . . , B06, with an objective weight (if present) appearing last. When $ISCALE \geq 10$ the user specified file SCLFIL, is used to access the scale weights. Specifically, when $ISCALE = 10, 11, 12, 30, 31$, or 32 , scale weights will be read from the scale weight file. If the value on the file is positive it will be used in lieu of the value computed by SOCX. On the other hand if the value on the file is negative it will not be used, and SOCX will compute a scale weight. When $ISCALE \geq 20$ the scale weight file will be written to the name specified by SCLFIL.

For this example the scale file corresponding to information on phase 2, has the following format:

0.2000000000000000D+01	0
0.1213720499418485D+00	0
0.1538678927401882D-03	0
0.5550941893601847D+00	0
0.3115141899462238D+01	0
0.5180166179731046D-02	0
0.1999873988891675D-03	0
0.1393433363285093D-02	0
0.1500000000000000D+01	0
0.5180166179731033D-02	0
0.2222222222222222D-01	0
0.2000000000000000D-03	0
0.1428571428571429D-02	0
0.1500000000000000D+01	0
0.1137657086431346D-01	0
0.6666666666666667D-01	0

The first item contains the phase number in double precision format followed by an unused integer value. Subsequent values in column one contain the scale weights as they appear in the iteration output. Column two contains an integer value which must be either 0 or 1. When the integer is zero, the scale weight has been constructed automatically and may be altered by subsequent scaling. When the integer value is one, the scale weight is input and will not be changed.

Character String Input Errors

When an invalid value is input for the character variable STRING(1:k) subroutine INSOCX calls the standard error handler HHERR with an error flag $IER < 0$, followed by a call to HHERRX which displays the invalid string. The nonstandard error returns are defined as follows:

- | | |
|---------|--|
| IER= -1 | String too long (more than 80 characters). |
| IER= -2 | Replacement error during capitalization. |
| IER= -3 | Equal sign “=” missing from string. |

IER= -4	Invalid real value.
IER= -5	Invalid integer value.
IER= -6	Invalid character value.
IER= -7	Invalid symbol name.

6.3 User-Supplied Subprograms for Optimal Control

DDLOAD: Discrete Data Load

PURPOSE

DDLOAD is a user-supplied subroutine that loads values for the discrete data functions needed to define the parameter estimation objective function. Specifically the routine supplies the values $\hat{y}_{ij}$, the weights w_{ij} and the corresponding evaluation times θ_{ij} used to construct the residual

$$r_k = w_{ij} [y_i(\theta_{ij}) - \hat{y}_{ij}]$$

in addition to the values θ_{ij} and $\hat{u}_{ij}$ which define the residual

$$r_k = w_{ij} [u_i(\theta_{ij}) - \hat{u}_{ij}].$$

For applications that use the parameter estimation software SOSEST but have no discrete data, the system supplied dummy routine, DUMMYDE may be used in the calling arguments to SOSEST. This routine is not used by the optimal control algorithms SOSCTL, or SOSDLY.

RELATED SUBPROGRAMS

SOSEST	Sparse Optimal Parameter Estimation
ODEINP	Optimal Control Problem Definition
ODEIGS	Optimal Control Initial Guess
ODEPTF	Optimal Control Point Function Evaluation
ODEPRT	Optimal Control Print
INSOCX	Sparse Optimal Control Input Procedure
INPSOS	Sparse Optimization Suite Input Procedure

METHOD

DDLOAD is a generic name for this routine, and the actual routine name must be declared as an external in the main program and placed in the calling list to subroutine SOSEST.

On a specific phase the fundamental input to this routine is the number of the discrete data function being loaded. DDLOAD must load either the values θ_{ij} , w_{ij} and $\hat{y}_{ij}$ or the values θ_{ij} , w_{ij} and $\hat{u}_{ij}$. In either case DDLOAD must also supply the total number of discrete data values (NDATA) and the corresponding number of the dynamic variable. Data is always supplied as a triple, i.e.

$$(\theta_{ij}, \hat{y}_{ij}, w_{ij})$$

for a fixed value of i , and $j = 1, \dots, N_d$. The function evaluation error flag, IFERR, should be set to zero if the data is supplied successfully. Otherwise, IFERR can be set to any nonzero value to designate that the data can not be supplied for the current inputs.

WARNING

If a phase contains discrete data, then both the initial phase time $t_I^{(k)}$ and the final phase time $t_F^{(k)}$ must be fixed. Furthermore, the evaluation times loaded by DDLOAD must lie strictly interior to the phase, that is

$$t_I^{(k)} \leq \theta_{ij} \leq t_F^{(k)}$$

assuming $t_I^{(k)} < t_F^{(k)}$.

USER-SUPPLIED SUBPROGRAM

The user must supply a subroutine with the following format:

```
SUBROUTINE DDLOAD(IPHASE,NDD,NDDST,MXDATA,TDATA,YDATA,WDATA,NDATA,IFERR)
```

```
INTEGER IPHASE,NDD,NDDST,MXDATA,NDATA,IFERR
```

```
DOUBLE PRECISION TDATA(MXDATA),YDATA(MXDATA),WDATA(MXDATA)
```

User supplied code to evaluate the requested arguments (see below).

```
      .
      .
      .
RETURN
END
```

ARGUMENTS

IPHASE [INPUT, INTEGER]
The current phase number.

NDD [INPUT, INTEGER]
The number of the discrete data function to be loaded. This quantity must correspond to the value specified in ODEINP (p. 426) by the value of ITERM(4,j).

NDDST [OUTPUT, INTEGER]
The index $NDDST = i$ that defines the dynamic variable in the residual. For $1 \leq i \leq NDE$, the residual is $r_k = w_{ij} [y_i(\theta_{ij}) - \hat{y}_{ij}]$, and for $NDE+1 \leq i \leq NDE+NAV$, the residual is $r_k = w_{ij} [u_i(\theta_{ij}) - \hat{u}_{ij}]$.

MXDATA [INPUT, INTEGER]
The maximum dimension of the discrete data arrays. The value must be defined by a call to INPSOS (p. 401)

TDATA [OUTPUT, DOUBLE PRECISION, ARRAY]
Real array of length NDATA containing the specified evaluation times for the discrete data. For $1 \leq i \leq NDE+NAV$, $TDATA(j) = \theta_{ij}$.

YDATA	[OUTPUT, DOUBLE PRECISION, ARRAY] Real array of length NDATA containing the specified discrete data. For $1 \leq i \leq \text{NDE}$, $\text{YDATA}(j) = \hat{y}_{ij}$, and for $\text{NDE}+1 \leq i \leq \text{NDE}+\text{NAV}$, $\text{YDATA}(j) = \hat{u}_{ij}$.
WDATA	[OUTPUT, DOUBLE PRECISION, ARRAY] Real array of length NDATA containing the specified residual weights. Typically $w_{ij} > 0$, although this is not required.
NDATA	[OUTPUT, INTEGER] The number of discrete data values N_d for data function NDD.
IFERR	[OUTPUT, INTEGER] Discrete Data Load Error Flag.
	IFERR = 0 Successful Load of Discrete Data Functions.
	IFERR $\neq$ 0 Error in Load of Discrete Data Functions.

EVALDL: Delay Equation Evaluation

PURPOSE

EVALDL is a user-supplied subroutine that loads values for the delay functions needed to define the optimal control of delay systems. Specifically the routine supplies the values $\omega_k(t)$, for $k = 1, \dots, \nu$, the startup function(s) $\alpha(t)$, and defines which dynamic variables are functions of $\omega_k(t)$. For applications that use the delay system software SOSDLY but have no delayed quantities, the system supplied dummy routine, DUMYDL may be used in the calling arguments to SOSDLY. This routine is used only by SOSDLY.

RELATED SUBPROGRAMS

SOSDLY	Sparse Optimal Control of Delay Systems
ODEINP	Optimal Control Problem Definition
ODEIGS	Optimal Control Initial Guess
ODEPTF	Optimal Control Point Function Evaluation
ODEPRT	Optimal Control Print
INSOCX	Sparse Optimal Control Input Procedure
INPSOS	Sparse Optimization Suite Input Procedure

METHOD

EVALDL is a generic name for this routine, and the actual routine name must be declared as an external in the main program and placed in the calling list to subroutine SOSDLY.

WARNING

If a phase contains a delay function ($\text{NCF}(6) > 0$), then both the initial phase time $t_I^{(k)}$ and the final phase time $t_F^{(k)}$ must be fixed.

USER-SUPPLIED SUBPROGRAM

The user must supply a subroutine with the following format:

```

SUBROUTINE EVALDL(IPHASE,TIME,Y,NY,P,NP,TI,TF,IDELAY,TDELAY,IYDLAY,YDELAY,IFERR)

INTEGER  IPHASE,IDELAY,IYDLAY,NY,NP,IFERR
DOUBLE PRECISION TIME,TI,TF,TDELAY
INTEGER  IYDLAY(NY)
DOUBLE PRECISION YDELAY(NY)
```

User supplied code to evaluate the requested arguments (see below).

```

      .
      .
      .
RETURN
END

```

ARGUMENTS

IPHASE	[INPUT, INTEGER] The current phase number.
TIME	[INPUT, DOUBLE PRECISION] The current time t .
Y	[INPUT, DOUBLE PRECISION] State vector at the current time t .
NY	[INPUT, INTEGER] Length of state vector.
P	[INPUT, DOUBLE PRECISION] Parameter values on current phase.
NP	[INPUT, INTEGER] Number of parameters.
TI	[INPUT, DOUBLE PRECISION] The initial time for phase IPHASE.
TF	[INPUT, DOUBLE PRECISION] The final time for phase IPHASE.
IDELAY	[INPUT, INTEGER] Number of delay function about which data is requested.
TDELAY	[OUTPUT, DOUBLE PRECISION] The delay time, i.e. the value of the delay function $\omega_k(t)$ at the current time t . A value for TDELAY must be computed for all t .
IYDLAY	[OUTPUT, INTEGER] Vector defining state delays needed and prehistory evaluation. $K = -1$ State K is delayed with user specified prehistory. $K = 0$ State K not delayed. $K = J$ State K is delayed with value specified by state J at TDELAY.

YDELAY	[OUTPUT, DOUBLE PRECISION, ARRAY] Double precision array of length NY. If $\omega_k(t) < TI$, YDELAY(j) = $\alpha(t)$ where $\alpha(t)$ is the startup (prehistory) function for delayed state j. If $\omega_k(t) > TF$, YDELAY(j) = $\alpha(t)$ where $\alpha(t)$ is the termination (posthistory) function for advanced state j. If the delay is zero at the initial time $\omega_k(TI) = TI$ and YDELAY(j) must be supplied even though it will not be used. Similarly if the delay is zero at the final time $\omega_k(TF) = TF$, YDELAY(j) must be supplied. If delay function $\omega_k(t)$ does not change dynamic variable j, then YDELAY(j) must not be modified.				
IFERR	[OUTPUT, INTEGER] Delay Function Error Flag. <table style="width: 100%; border: none;"> <tr> <td style="width: 30%;">IFERR = 0</td> <td>Successful Delay Function Evaluation.</td> </tr> <tr> <td>IFERR $\neq$ 0</td> <td>Error in Delay Function Evaluation.</td> </tr> </table>	IFERR = 0	Successful Delay Function Evaluation.	IFERR $\neq$ 0	Error in Delay Function Evaluation.
IFERR = 0	Successful Delay Function Evaluation.				
IFERR $\neq$ 0	Error in Delay Function Evaluation.				

ODEIGS: Optimal Control Initial Guess

PURPOSE

ODEIGS is an *optional* user-supplied subroutine which defines the initial guess of the solution to an optimal control problem for SOCX. This subroutine evaluates an initial guess for the time dependent functions $z(t)$ (states plus controls) and the value of the independent variable t . This routine is only called if the user has set $INIT = 2$ or $INIT = 10$ in subroutine ODEINP. For applications that use other initialization options (i.e. $INIT \neq 2, 10$) the system supplied dummy routine, DUMMYIG may be used in the calling arguments to SOSCTL.

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control Software
ODEINP	Optimal Control Problem Definition
ODERHS	Optimal Control Right Hand Side Evaluation
ODEPTF	Optimal Control Point Function Evaluation
ODEPRT	Optimal Control Print
INSOCX	Sparse Optimal Control Input Procedure
INPSOS	Sparse Optimization Suite Input Procedure

METHOD

ODEIGS is a generic name for this routine, and the actual routine name must be declared as an external in the main program and placed in the calling list to subroutine SOSCTL, SOSDLY, or SOSEST.

On a specific phase the fundamental input to this routine is the grid point number k , where $k = 1, \dots, N$ and N is the number of grid points NGRID defined in ODEINP. The ODEIGS routine is called sequentially N times beginning with $k = 1$, followed by a call with $k = 2$, etc. The purpose of the routine is to supply values for the dynamic variables $z(t_k)$ as well as the independent variable t_k . If the application requires an equally spaced mesh of N points, the user can construct $t_k = (k - 1)h + t_I$ where $h = (t_F - t_I)/(N - 1)$. Notice however that the initial mesh distribution does not need to be equally spaced.

When using $INIT = 10$, the input grid point number is negative ($k < 0$). In this case the values of the state $y(t)$, parameters p and time t are input, and the routine must compute the control vector

$$\hat{u}(t) = g[y(t), p, t].$$

Since an optimization problem is usually sensitive to the initial guess, this routine allows the user to have complete control over what the initial guess will be. A common alternative is to construct an initial guess by interpolation of a previously constructed solution. Interpolation and file manipulation procedures needed to implement this approach can be incorporated into the ODEIGS routine.

WARNING

The ODEIGS routine is called sequentially N times on all phases with $\text{INIT} = 2$. Thus, the first call will be made with $k = 1$, the second with $k = 2$, etc. The user specified grid sequence must either be monotonically increasing ($t_k < t_{k+1}$) or monotonically decreasing ($t_k > t_{k+1}$) for all k . When $\text{INIT} = 10$ the numerical integration algorithm will define the location of the grid point t_k and the values for the state $y(t_k)$.

If file manipulation is required to construct the initial guess the user must exercise caution to OPEN, CLOSE, and REWIND the appropriate files.

USER-SUPPLIED SUBPROGRAM

The user must supply a subroutine with the following format:

```
SUBROUTINE ODEIGS(IPHASE,NGRDPT,T,Z,NZ,IFERR)
```

```
INTEGER  IPHASE,NGRDPT,NZ,IFERR
```

```
DOUBLE PRECISION  T,Z(NZ)
```

User supplied code to evaluate the requested arguments (see below).

```
      .  
      .  
      .
```

```
RETURN
```

```
END
```

ARGUMENTS

IPHASE [INPUT, INTEGER]
The current phase number.

NGRDPT [INPUT, INTEGER]
When $\text{INIT}(1) = 2$, the number of the grid point, k , where $k = 1, \dots, N$ and N is the number of grid points NGRID defined in ODEINP. When $\text{INIT}(1) = 10$, the input value $k < 0$.

T [INPUT/OUTPUT, DOUBLE PRECISION]
When $\text{INIT}(1) = 2$, the value of the independent variable t_k at grid point k . The values of t_k must be monotonic functions of the grid point k . When $\text{INIT}(1) = 10$, the value of t_k is input.

Z	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] When $\text{INIT}(1) = 2$, a real array of length NZ containing an initial guess for the dynamic variables $z_k(t_k)$ at grid point k . When $\text{INIT}(1) = 10$, the state variables $y_k(t_k)$ at grid point k are input, and the control variables $\hat{u}_k(t_k) = g[y(t), p, t]$ must be output.
NZ	[INPUT, INTEGER] The number of time dependent variables.
IFERR	[OUTPUT, INTEGER] Initial Guess Evaluation Error Flag. IFERR = 0 Successful Evaluation of Initial Guess. IFERR $\neq$ 0 Error in Evaluation of Initial Guess.

EXAMPLE Computed Initial Guess

Consider an example with one phase, two state variables, and one control variable, so $z(t) = [y_1(t), y_2(t), u(t)]$. Let us suppose that a “good guess” for the optimal control solution is

$$\begin{aligned} y_1(t) &= \sin(2\pi t) \\ y_2(t) &= \cos^2(2\pi t) \\ u(t) &= \exp[y_1(t) + 3y_2(t)] \end{aligned}$$

for $t_I \leq t \leq t_F$. Furthermore let us suppose that previous analysis suggests that the discretization error is small if grid points are located according to the formula

$$t_k = (k - 1)^2 h + t_I$$

where $h = (t_F - t_I)/(N - 1)^2$ for $k = 1, \dots, N$. This asymmetric grid distribution tends to put more grid points near the end of the phase. To implement this when $\text{INIT}(1) = 2$ the user could supply the following code fragment in the ODEIGS subroutine:

```

C
C      DEFINE THE PARAMETERS FOR THE PHASE
C      (COULD BE PASSED FROM ODEINP IN A USER COMMON)
C
      TWOPI = 2.D0*HDMCON(12)
      NGRID = 20
      TZERO = 0.D0
      TFINAL = 1.D0
      HSTEP = (TFINAL - TZERO)/DBLE((NGRID - 1)**2)
C
C      COMPUTE THE INDEPENDENT VARIABLE FROM THE GRID POINT NUMBER
C
      T = HSTEP*DBLE((NGRDPT-1)**2) + TZERO

```

```
C
C      COMPUTE THE STATE VARIABLES AT CURRENT GRID POINT
C
      Z(1) = SIN(TWOPI*T)
      Z(2) = COS(TWOPI*T)**2
C
C      COMPUTE THE CONTROL VARIABLE AT THE CURRENT GRID POINT
C
      Z(3) = EXP(Z(1) + 3.D0*Z(2))
C
```

On the other hand suppose initial values for the state are known, and the option `INIT(1) = 10` is used. In this case the user could supply the following code fragment in the `ODEIGS` subroutine:

```
      IF(NGRDPT.LT.0) THEN
C
C      COMPUTE THE CONTROL VARIABLE AT THE CURRENT GRID POINT
C
      Z(3) = EXP(Z(1) + 3.D0*Z(2))
C
      ENDIF
C
```

ODEINP: Optimal Control Problem Definition

PURPOSE

ODEINP is a user-supplied subroutine which defines the optimal control problem for SOCX.

RELATED SUBPROGRAMS

BSPDEF	B-Spline Algebraic Variable Utility
SOSCTL	Sparse Optimal Control Software
SOSEST	Sparse Optimal Parameter Estimation
LINKST	Optimal Control Linkage Utility
ODEIGS	Optimal Control Initial Guess
ODEPRT	Optimal Control Print
ODEPTF	Optimal Control Point Function Evaluation
ODERHS	Optimal Control Right Hand Side Evaluation
PHSLNG	Optimal Control Phase Duration Utility
PNTCON	Optimal Control Point Constraint Utility
PTHCON	Optimal Control Path Constraint Utility
INSOCX	Sparse Optimal Control Input Procedure
INPSOS	Sparse Optimization Suite Input Procedure

METHOD

This subroutine defines the phase dependent problem input. It will be called *once* for each phase. ODEINP is a generic name for this routine, and the actual routine name must be declared as an external in the main program and placed in the calling list to subroutine SOSCTL, SOSDLY, or SOSEST. The number of phases is defined by the quantity NPHS or the maximum number of phases MAXPHS defined in the call to SOSCTL, SOSDLY, or SOSEST. Quantities defined by this routine include the transcription method, the initial grid and guess, the number of continuous and discrete user defined functions, linkage conditions between phases, and output scales and titles. Except for the quantities MAXMIN, MXPARM, MXSTAT, MXPCON, MXTERM, and NPHS, all inputs to ODEINP described below can be changed from phase to phase (e.g. the number of grid points NGRID can be different in each phase). Prior to calling ODEINP the SOCX software sets default values for many of the arguments, and therefore it may not be necessary to assign values to all arguments listed as outputs for the routine.

The definition of a complicated optimal control problem can be simplified by using one or more utility routines in the SOS library. For problems with many phases, linking the phases together can be expedited by using the utility routines LINKST (p. 481) and/or AUTOLK (p. 460).

Constraints on the duration of a phase are defined using subroutines PHSLNG (p. 498) and/or AUTOPL (p. 462). Boundary conditions are defined using subroutine PNTCON (p. 500), and the utility PTHCON (p. 506) can be used to define path constraints.

WARNING

The only quantities that *must be the same on all phases* are: MAXMIN, MXPARM, MXSTAT, MXPCON, MXTERM, and NPHS. The quantities MXPARM, MXSTAT, MXPCON, and MXTERM are inputs to ODEINP and can only be modified by calling INPSOS or INSOCX prior to calling SOSCTL, SOSDLY, or SOSEST.

USER-SUPPLIED SUBPROGRAM

The user must supply a subroutine with the following format:

```
SUBROUTINE ODEINP(IPHASE,NPHS,METHOD,NSTG,NCF,NPF,NPV,NAV,NGRID,
                  INIT,MAXMIN,MXPARM,PO,PLB,PUB,PLBL,
                  MXSTAT,YO,Y1,YLB,YUB,STSKL,STLBL,MXPCON,CLB,CUB,
                  CLBL,MXTERM,COEF,ITERM,TITLE,IER)

INTEGER          IPHASE,NPHS,METHOD,NSTG(2),NCF(6),NPF(2),NPV,NAV,NGRID,
                  INIT(2),MAXMIN,MXPARM,MXSTAT,MXPCON,MXTERM,
                  ITERM(4,MXTERM),IER

DOUBLE PRECISION PO(MXPARM),PLB(MXPARM),PUB(MXPARM),YO(0:MXSTAT),
                  Y1(0:MXSTAT),YLB(-1:1,0:MXSTAT),YUB(-1:1,0:MXSTAT),
                  STSKL(0:MXSTAT+MXPARM,2),CLB(MXPCON),CUB(MXPCON),COEF(MXTERM)

CHARACTER(LEN=80) PLBL(MXPARM+2),STLBL(0:MXSTAT),CLBL(0:MXPCON)
CHARACTER(LEN=60) TITLE(3)
```

User supplied code to evaluate the requested arguments (see below).

```
      .
      .
      .

RETURN
END
```

Since subroutine ODEINP is supplied by the user, all “OUTPUT” arguments described below are defined by the user. Conversely, all “INPUT” arguments below are specified by the SOS software when the user routine is called.

ARGUMENTS

IPHASE	[INPUT, INTEGER] The current phase number, k .
NPHS	[OUTPUT, INTEGER, <i>Default</i> = 1] Number of phases in the problem.
METHOD	[OUTPUT, INTEGER, <i>Default</i> = 2] Discretization/integration method, used in conjunction with NSTG. Valid combinations are:

METHOD	NSTG(1)	Discretization.
0	1	ANL: Analytic Transformation
1	1	EUL: Euler's Method
1	2	RK2: Runge-Kutta 2-stage
1	3	RK3: Runge-Kutta 3-stage
1	4	RK4: Runge-Kutta 4-stage
2	1	TRP: Trapezoidal
2	2	HSS: Hermite-Simpson (Separated)
3	1	HSC: Hermite-Simpson (Compressed)
4	2	LA2: Lobatto IIIA 2-stage
4	3	LA3: Lobatto IIIA 3-stage
4	4	LA4: Lobatto IIIA 4-stage
4	5	LA5: Lobatto IIIA 5-stage
5	1	RKT: Runge-Kutta-Taylor Variable Step
5	2	DOP: Dormand-Prince Variable Step
5	3	CWG: Gear Stiff BDF, Variable Order and Step
5	4	ADM: Adams Predictor-Corrector, Variable Order and Step
5	5	DP5: Dormand-Prince 5(4) Variable Step
5	6	RKF: Runge-Kutta Fixed Step

If METHOD = 5, it is required that

- $\text{NCF}(2) = \text{NCF}(3) = \text{NCF}(4) = \text{NCF}(5) = \text{NCF}(6) = \text{NAV} = 0$, and
- $\text{NGRID} = 2$.

NSTG	[OUTPUT, INTEGER, ARRAY, <i>Default</i> = (1, 0)] Discretization stages (see METHOD). When METHOD = 5, if;
NSTG(1) > 0	Forward propagation (from the beginning of the phase to the end), that is from $Y_0(0)$ to $Y_1(0)$.
NSTG(1) < 0	Backward propagation (from phase end to phase start), that is from $Y_1(0)$ to $Y_0(0)$.
When $ \text{NSTG}(1) = 6$ (Runge-Kutta Fixed Step)	
NSTG(2) > 1	NSTG(2) is the number of equidistributed integration steps on the phase.

See the initial value problem (p. 601) for an illustration of METHOD = 5.

NCF

[OUTPUT, INTEGER, ARRAY, *Default* = 0]
Number of continuous functions (see usage remarks (p. 444)).

NCF(1) = n_y Number of Differential Equations.

NCF(2) = n_g Number of Algebraic Equations.

NCF(3) = n_q Number of Quadrature Equations.

NCF(4) = n_a Number of Auxiliary Functions.

NCF(5) = NDD Number of Discrete Data Functions.

NCF(6) = ν Number of Delay Functions.

NPF

[OUTPUT, INTEGER, ARRAY, *Default* = 0]
Number of computed point functions.

NPF(1) = $\dim[\psi_I]$ Number of point functions at phase start, t_I .

NPF(2) = $\dim[\psi_F]$ Number of point functions at phase end, t_F .

NPV

[OUTPUT, INTEGER, *Default* = 0]
Number of parameters, p .

NAV

[OUTPUT, INTEGER, *Default* = 0]
Number of algebraic or control variables, u .

NGRID

[OUTPUT, INTEGER, *Default* = 10]
Number of grid points in initial grid.

INIT

[OUTPUT, INTEGER, ARRAY *Default* = 1, 0]
Initial guess type for internal states (see usage remarks (p. 444)). INIT(2) is not used when INIT(1) ≤ 5 .

INIT(1) = 1 Construct a linear initial guess between Y0 and Y1. For grid point j the independent variable is

$$t_j = (j - 1) \left[\frac{Y1(0) - Y0(0)}{NGRID - 1} \right] + Y0(0),$$

and dynamic variable k is

$$z_k(t_j) = (j - 1) \left[\frac{Y1(k) - Y0(k)}{NGRID - 1} \right] + Y0(k),$$

where $1 \leq k \leq [NCF(1) + NAV]$ and $1 \leq j \leq NGRID$.

INIT(1) = 2 Construct an initial guess by calling subroutine ODEIGS at each of the NGRID points between t_I and t_F ; Y0 and Y1 are ignored.

INIT(1) = 3 Construct an initial guess from a previously computed solution saved on file RSTFIL as described in INPSOS (p. 401). The values of Y0(0), Y1(0), and NGRID are ignored.

INIT(1) = 4 Construct an initial guess from a previously computed solution saved on file RSTFIL as described in INPSOS (p. 401). Equally space NGRID points between the initial time Y0(0) and the final time Y1(0) as described for INIT(1) = 1. Interpolate/extrapolate if necessary.

INIT(1) = 5 Construct an initial guess from a previously computed solution saved on file RSTFIL as described in INPSOS (p. 401). Use the grid distribution defined by file RSTFIL to place points between the initial time Y0(0) and the final time Y1(0). The value of NGRID is ignored. Interpolate/extrapolate if necessary.

INIT(1) = 6

Construct an initial guess by solving an initial value problem with a linear control approximation. Approximate the control variable by

$$\hat{u}(t) = \frac{(t_f - t)}{(t_f - t_0)}u_0 + \frac{(t - t_0)}{(t_f - t_0)}u_f$$

where $t_0 = Y0(0)$, $t_f = Y1(0)$, $u_{0,k} = Y0(k)$, and $u_{f,k} = Y1(k)$, with $NCF(1) + 1 \leq k \leq [NCF(1) + NAV]$. Approximate the state variable by the solution of the ODE system

$$\dot{y}(t) = f[y(t), \hat{u}(t), p, t]$$

solved as an initial value problem (IVP). When $INIT(2) > 0$ propagate in the forward direction from t_0 to t_f with initial conditions $y_{0,k} = Y0(k)$ for $1 \leq k \leq NCF(1)$. When $INIT(2) < 0$ propagate in the reverse direction from t_f to t_0 with terminal conditions $y_{f,k} = Y1(k)$ for $1 \leq k \leq NCF(1)$.

|INIT(2)| Propagation Method

- | | |
|---|---|
| 1 | RKT: Runge-Kutta-Taylor Variable Step |
| 2 | DOP: Dormand-Prince Variable Step |
| 3 | CWG: Gear Stiff BDF, Variable Order and Step |
| 4 | ADM: Adams Predictor-Corrector, Variable Order and Step |
| 5 | DP5: Dormand-Prince 5(4) Variable Step |

When $INIT(1) = 6$, each integration step is used as a grid point; the user specified value of NGRID is not used. If $INIT(2)$ is not specified, the default $INIT(2) = 1$ is used. The maximum number of integration steps is limited to 10,000.

INIT(1) = 7

Construct an initial guess with NGRID equally spaced grid points by solving an initial value problem with a linear control approximation. All other inputs for $INIT(1) = 7$ are the same as $INIT(1) = 6$. If $INIT(2)$ is not specified, the default $INIT(2) = 1$ is used.

INIT(1) = 8

Construct an initial guess with NGRID equally spaced grid points, using linear interpolation/extrapolation of the discrete data $\{\hat{z}_{ij}^{(k)}\}$ defined in DDLOAD (p. 416) augmented by the values Y0 and Y1. Three sets of data can be interpolated.

1. $\{Y0(i), \hat{z}_{ij}^{(k)}, Y1(i)\}$ when $NDDST = i$, $INIT(2) > 0$
2. $\{\hat{z}_{ij}^{(k)}\}$ when $NDDST = i$, $INIT(2) \leq 0$
3. $\{Y0(i), Y1(i)\}$ when $NDDST \neq i$

The default value for $INIT(2) = 0$.

INIT(1) = 9

Construct an initial guess using linear interpolation/extrapolation of the discrete data $\{\hat{z}_{ij}^{(k)}\}$ defined in DDLOAD (p. 416) augmented by the values Y0 and Y1. Grid points are chosen to coincide with the unique set of discrete data times $\{\theta_{ij}^{(k)}\}$. The number of grid points, NGRID, is not used. Three sets of data can be interpolated.

1. $\{Y0(i), \hat{z}_{ij}^{(k)}, Y1(i)\}$ when $NDDST = i$, $INIT(2) > 0$
2. $\{\hat{z}_{ij}^{(k)}\}$ when $NDDST = i$, $INIT(2) \leq 0$
3. $\{Y0(i), Y1(i)\}$ when $NDDST \neq i$

The default value for $INIT(2) = 0$.

INIT(1) = 10

Construct an initial guess by solving an initial value problem with a prescribed (closed-loop) control function and specified boundary condition. Approximate the control variables by

$$\hat{u}(t) = g[y(t), p, t]$$

where the control functions g are defined in subroutine ODEIGS (p. 422). Approximate the state variable by the solution of the ODE system

$$\dot{y}(t) = f[y(t), \hat{u}(t), p, t] = f\{y(t), g[y(t), p, t], p, t\}$$

solved subject to the boundary (event) condition

$$E[y(t_f), \hat{u}(t_f), p, t_f] = 0.$$

In general, the event condition may contain multiple roots, and consequently there are two mechanisms to isolate the desired condition. First, it is necessary to bound the region containing the event condition $t_L \leq t_f \leq t_U$ where the bounds t_L and t_U are defined using the input YLB($\pm 1, 0$) and YUB($\pm 1, 0$). Second, it is required that the root satisfy the slope condition

$$\frac{dE}{dt}[y(t_f), \hat{u}(t_f), p, t_f] \leq 0.$$

If it is desired to obtain a root of E with a positive slope, the user must define a new event condition $\hat{E} = -E$. The event detection procedure will compute the first root located within the bounded region. For sensitive event conditions it may be necessary to tighten the integration tolerance(s) TOLM5A and/or TOLM5R in subroutine INPSOS (p. 401). When INIT(2) > 0 the propagation is in the forward direction from t_0 to t_f with initial conditions $y_{0,k} = Y0(k)$ for $1 \leq k \leq \text{NCF}(1)$. When INIT(2) < 0 propagation is in the reverse direction from t_f to t_0 and the boundary condition is imposed at t_0 with terminal conditions $y_{f,k} = Y1(k)$ for $1 \leq k \leq \text{NCF}(1)$. The boundary condition must be defined in subroutine ODEPTF (p. 450) and can be imposed at only one end of the phase, that is, either t_0 or t_f but not both. Each integration step is used as a grid point; the user specified value of NGRID is not used. If INIT(2) is not specified, the default INIT(2) = 1 is used. Otherwise the propagation method is defined by |INIT(2)| =;

- 1 RKT: Runge-Kutta-Taylor Variable Step
- 2 DOP: Dormand-Prince Variable Step
- 3 CWG: Gear Stiff BDF, Variable Order and Step
- 4 ADM: Adams Predictor-Corrector, Variable Order and Step
- 5 DP5: Dormand-Prince 5(4) Variable Step

INIT(1) = 11 Construct an initial guess for a delay differential equation using linear interpolation/extrapolation of the dynamic states on a grid constructed using the delay times $\{\omega_k\}$ defined in EVALDL (p. 419), augmented by the values Y0 and Y1. Grid points are chosen to coincide with integral multiples and integral divisions of the delay times. The number of grid points, NGRID, is used as a target for the number of intervals created in this manner, and will not be in a one to one correspondence with the actual number of grid points created. The default value for INIT(2) = 0. For delay equations, INIT(1) = 1, and INIT(1) = 2 are also legal options for defining the initial grid and corresponding initial guess. For INIT(1) = 1, the uniform initial grid created using the input value of NGRID will be augmented by the delay times $\{\omega_k\}$ defined in EVALDL. For INIT(1) = 2, the user is entirely responsible for the initial grid and the values of the dynamic variables at the grid points.

MAXMIN [OUTPUT, INTEGER, *Default* = -1]
Optimization Type Flag.

MAXMIN = -1 Minimize objective.

MAXMIN = 0 Search for feasibility.

MAXMIN = +1 Maximize objective.

MAXMIN = +2 Minimize least squares objective.

MXPARM [INPUT, INTEGER, *Default* = 5]
Maximum number of parameters per phase.

P0 [OUTPUT, DOUBLE PRECISION, ARRAY]
Array of length NPV containing the initial guess for parameters, p .

PLB	[OUTPUT, DOUBLE PRECISION, ARRAY, <i>Default</i> = $-.01/\text{HDMCON}(5)$] Array of length NPV containing the lower bounds for parameters, p .	
PUB	[OUTPUT, DOUBLE PRECISION, ARRAY, <i>Default</i> = $.01/\text{HDMCON}(5)$] Array of length NPV containing the upper bounds for parameters, p .	
PLBL	[OUTPUT, CHARACTER(LEN=80), ARRAY, <i>Default</i> = '␣'] Symbol and description for parameters, p .	
	$\text{PLBL}(k)(1:8) = \text{'variable'}$	Name of parameter k . If the user does not supply a value, SOS will choose the name "TZERO", "TFINAL", or "PARM" as appropriate.
	$\text{PLBL}(k)(9:80) = \text{'descriptive label'}$	Description of parameter k . A generic description will be displayed if none is supplied.
MXSTAT	[INPUT, INTEGER, <i>Default</i> = 20] Maximum number of dynamic variables per phase.	
Y0	[OUTPUT, DOUBLE PRECISION, ARRAY] Real array of length $\text{NCF}(1) + \text{NAV} + 1$ containing an initial guess for the dynamic variables at the beginning of the phase.	
	$Y0(0) = t_I$	Guess for independent variable.
	$Y0(k) = z_k(t_I)$	Guess for dynamic variable, k .
Y1	[OUTPUT, DOUBLE PRECISION, ARRAY] Real array of length $\text{NCF}(1) + \text{NAV} + 1$ containing an initial guess for the dynamic variables at the end of the phase.	
	$Y1(0) = t_F$	Guess for independent variable.
	$Y1(k) = z_k(t_F)$	Guess for dynamic variable, k .
YLB	[OUTPUT, DOUBLE PRECISION, ARRAY, <i>Default</i> = $-.01/\text{HDMCON}(5)$] Real array of length $\text{NCF}(1) + \text{NAV} + 1$ containing the lower bounds for the independent and dynamic variables in the interior and boundaries of the phase.	
	For $k > 0$	
	$\text{YLB}(-1,k) \leq z_k(t_I)$	Lower bound for dynamic variable k at beginning of phase.
	$\text{YLB}(0,k) \leq z_k(t)$	Lower bound for dynamic variable k during phase.

	$YLB(+1,k) \leq z_k(t_F)$	Lower bound for dynamic variable k at end of phase.
	For $k = 0$	
	$YLB(-1,0) \leq t_I$	Lower bound for independent variable at beginning of phase.
	$YLB(+1,0) \leq t_F$	Lower bound for independent variable at end of phase.
YUB	[OUTPUT, DOUBLE PRECISION, ARRAY, <i>Default</i> = .01/HDMCON(5)] Real array of length NCF(1)+NAV+1 containing the upper bounds for the independent and dynamic variables in the interior and boundaries of the phase.	
	For $k > 0$	
	$YUB(-1,k) \geq z_k(t_I)$	Upper bound for dynamic variable k at beginning of phase.
	$YUB(0,k) \geq z_k(t)$	Upper bound for dynamic variable k during phase.
	$YUB(+1,k) \geq z_k(t_F)$	Upper bound for dynamic variable k at end of phase.
	For $k = 0$	
	$YUB(-1,0) \geq t_I$	Upper bound for independent variable at beginning of phase.
	$YUB(+1,0) \geq t_F$	Upper bound for independent variable at end of phase.
STSKL	[OUTPUT, DOUBLE PRECISION, ARRAY, <i>Default</i> = 1.0] Real array of length 2*(NCF(1)+NAV+NPV+1) containing scale information (see usage remarks (p. 444)). Column one, that is, STSKL(*,1) defines the output quantities $\hat{t}$ and $\hat{z}$.	
	$\hat{t} = \text{STSKL}(0,1)*t$	Scaled independent variable.
	$\hat{z}_k = \text{STSKL}(k,1)*z_k$	Scaled dynamic variable, k .
	Column two, that is, STSKL(*,2) defines the variable scales for the NLP problem, overriding the automatic scaling procedure. The sparse NLP is posed in terms of the quantities $\hat{t}$ and $\hat{z}$.	
	$\hat{t} = \text{STSKL}(0,2)*t$	Scaled independent variable.
	$\hat{z}_k = \text{STSKL}(k,2)*z_k$	Scaled dynamic variable, k .

STLBL	[OUTPUT, CHARACTER(LEN=80), ARRAY, <i>Default</i> = '␣'] Character array of length NCF(1)+NAV+1 containing symbol and description for independent variable t and dynamic variables, z. STLBL(k)(1:8) = '<i>variable</i>' Name of variable k. If the user does not supply a value, SOS will choose the name "TIME", "STATE", or "CNTRL" as appropriate. STLBL(k)(9:80) = '<i>descriptive label</i>' Description of variable k. A generic description will be displayed if none is supplied.
MXPCON	[INPUT, INTEGER, <i>Default</i> = 20] Maximum number of user defined constraints per phase.
CLB	[OUTPUT, DOUBLE PRECISION, ARRAY, <i>Default</i> = $-.01/\text{HDMCON}(5)$] Array containing the lower bounds for constraints.
CUB	[OUTPUT, DOUBLE PRECISION, ARRAY, <i>Default</i> = $.01/\text{HDMCON}(5)$] Array containing the upper bounds for constraints.
CLBL	[OUTPUT, CHARACTER(LEN=80), ARRAY, <i>Default</i> = '␣'] Array containing the symbol and description for constraints. CLBL(k)(1:8) = '<i>symbol</i>' Name of the constraint k. If the user does not supply a value, SOS will choose the name "PHSCON" if appropriate. CLBL(k)(9:80) = '<i>descriptive label</i>' Description of the constraint. A generic description will be displayed if none is supplied. Note, the last character can be used to alter the domain of the path constraints as described in the usage remarks (p. 441).
MXTERM	[INPUT, INTEGER, <i>Default</i> = 50] Maximum number of terms allowed in ITERM per phase.
COEF	[OUTPUT, DOUBLE PRECISION, <i>Default</i> = 1.0] Multipliers for user defined terms. Used for terms referencing the independent variable, the dynamic variables, the parameters, and all user defined functions except quadrature functions.
ITERM	[OUTPUT, INTEGER, ARRAY, <i>Default</i> = -1] Integer array containing a description of the user defined term j used in forming general constraints and/or objective. For a more complete discussion see the usage remarks (p. 439) and the utility routines LINKST (p. 481), PHSLNG (p. 498), PNTCON (p. 500), and PTHCON (p. 506). ITERM(1,j)

$= 0$ Term j is part of NLP objective function.

$= k$ Term j is part of NLP constraint k .

ITERM(2, j)

$= k$ Term j is computed in phase k .

ITERM(3, j)

$= -1$ Term j is computed at phase start

$= 0$ Term j is computed during phase

$= +1$ Term j is computed at phase end

$= +2$ Term j is assigned to discrete data

$= +3$ Term j is assigned to algebraic rate constraint

ITERM(4, j)

$= k$ $k < 0$. Term j is user defined function number $|k|$.

$0 < k \leq \text{NDE} + \text{NAV}$. Term j is dynamic variable k .

$\text{NDE} + \text{NAV} < k \leq \text{NDE} + \text{NAV} + \text{NPV}$. Term j is parametric variable $k - (\text{NDE} + \text{NAV})$.

$0 < k$ and ITERM(3, j) = 2. Term j is discrete data function k .

$\text{NDE} < k$ and ITERM(3, j) = 3. Term j is rate constrained algebraic variable k .

$= 0$ Term j is the independent variable.

TITLE [OUTPUT, CHARACTER(LEN=60), ARRAY, *Default* = '␣']
Array containing the titles describing the problem.

TITLE(1) = 'Optimal Control Problem Title '

TITLE(2) = 'Optimal Control Problem Subtitle '

TITLE(3) = *'Phase Title '* for phase k

IER [OUTPUT, INTEGER]
Input Evaluation Error Flag.

IER = 0 Successful Evaluation of Input.

IER $\neq$ 0 Error in Evaluation of Input.

USAGE REMARKS

Input Using ITERM

For most optimal control applications a mechanism is needed to construct complicated expressions that occur in the problem statement. This functionality is supplied using the input array ITERM. Specifically, it is useful to form *quantities* that are constructed as a linear combination of *terms*, i.e.

$$Quantity = Term_1 + Term_2 + \dots + Term_n.$$

When a quantity is constructed in this manner it is necessary to specify information about each of the terms. In particular, for term j

ITERM(1, j) Defines what quantity term j belongs to.

ITERM(2, j) Defines which phase term j is computed in.

ITERM(3, j) Defines where in the phase term j is computed.

ITERM(4, j) Defines what kind of function term j is.

The values assigned to *ITERM*(3, j) and *ITERM*(4, j) are also related to the user supplied subroutines ODERHS, ODEPTF and DDLOAD. In particular, if a function is computed at the beginning of the phase the user must supply that information in the subroutine ODEPTF when IPHEND = -1. The correct assignment of *ITERM*(4, j) is summarized in Table 6.1.

Table 6.1: Setting ITERM(4,j) when ITERM(3,j) = -1

ITERM	Point Functions Computed in ODEPTF	
ITERM(4,j) = -1	$\psi_1[y(t_I), u(t_I), p, t_I]$	Point Fun. 1
ITERM(4,j) = -2	$\psi_2[y(t_I), u(t_I), p, t_I]$	Point Fun. 2
	$\vdots$	
ITERM(4,j) = - n_p	$\psi_{n_p}[y(t_I), u(t_I), p, t_I]$	Point Fun. n_p

A similar referencing scheme applies when a point function is evaluated at the end of the phase—the functions $\psi_k[y(t_F), u(t_F), p, t_F]$ must be evaluated in ODEPTF when IPHEND = +1 and ITERM(4,j) must be set as in Table 6.1. In many cases, the utility routine PNTCON (p. 500) can be used to construct point constraints.

On the other hand, if a function is computed during the phase the user must supply that information in subroutine ODERHS. The correct assignment of ITERM(4,j) is summarized in Table 6.2.

Table 6.2: Setting ITERM(4,j) when ITERM(3,j) = 0

ITERM	Right Hand Side Functions Computed in ODERHS	
	$f_1[y(t), u(t), p, t]$	Differential Eq. 1
	$\vdots$	
	$f_{n_y}[y(t), u(t), p, t]$	Differential Eq. n_y
ITERM(4,j) = -1	$g_1[y(t), u(t), p, t]$	Algebraic Eq. 1
ITERM(4,j) = -2	$g_2[y(t), u(t), p, t]$	Algebraic Eq. 2
	$\vdots$	
ITERM(4,j) = - n_g	$g_{n_g}[y(t), u(t), p, t]$	Algebraic Eq. n_g
ITERM(4,j) = - $n_g - 1$	$q_1[y(t), u(t), p, t]$	Quadrature Fun. 1
ITERM(4,j) = - $n_g - 2$	$q_2[y(t), u(t), p, t]$	Quadrature Fun. 2
	$\vdots$	
ITERM(4,j) = - $n_g - n_q$	$q_{n_q}[y(t), u(t), p, t]$	Quadrature Fun. n_q
ITERM(4,j) = - $n_g - n_q - 1$	$a_1[y(t), u(t), p, t]$	Auxiliary Fun. 1
ITERM(4,j) = - $n_g - n_q - 2$	$a_2[y(t), u(t), p, t]$	Auxiliary Fun. 2
	$\vdots$	
ITERM(4,j) = - $n_g - n_q - n_a$	$a_{n_a}[y(t), u(t), p, t]$	Auxiliary Fun. n_a

In many cases, the utility routine PTHCON (p. 506) can be used to construct path constraints.

Path Constraint Domain

An algebraic path constraint of the form

$$g[y(t), u(t), p, t] = 0$$

can be specified when $\text{ITERM}(1,j) > 0$ and $\text{ITERM}(3,j) = 0$. By default the constraint is assumed to apply on the closed domain $t \in [t_I, t_F]$, i.e. for

$$t_I \leq t \leq t_F$$

where t_I is the initial time on the phase and t_F is the final time on the phase. The domain can be altered to exclude the initial time t_I and/or the final time t_F by using a special symbol as the last character in the constraint label $\text{CLBL}(k)$ as summarized in Table 6.3.

Table 6.3: Setting Path Domain

Last Character	Domain
<	$t_I < t \leq t_F$
>	$t_I \leq t < t_F$
#	$t_I < t < t_F$
Any character except <, >, #	$t_I \leq t \leq t_F$

Consider the following four different ways to define the constraint label:

```
CLBL(NKON) = 'PATHCON Path Constraint Closed Domain [t0,tf]'
CLBL(NKON) = 'PATHCON Path Constraint Open Left Domain (t0,tf]<'
CLBL(NKON) = 'PATHCON Path Constraint Open Right Domain [t0,tf)>'
CLBL(NKON) = 'PATHCON Path Constraint Open Domain (t0,tf)#'
```

EXAMPLE 1: General Boundary Condition

Consider a problem with two phases and assume that on the first phase there are two state variables, and one control variable, so $z^{(1)} = (y_1^{(1)}, y_2^{(1)}, u_1^{(1)})$. On the second phase assume the dynamics are characterized by one state and one control, namely $z^{(2)} = (y_1^{(2)}, u_1^{(2)})$. Finally, let us suppose that the third boundary condition required to define the optimal control problem on the second phase has the form

$$2.4 \leq \psi_3^{(2)} \leq 3.7$$

where the constraint function is given by

$$\psi_3^{(2)} = \left[y_1^{(1)}(t_F^{(1)}) + y_2^{(1)}(t_F^{(1)}) \right]^2 - 4u_1^{(2)}(t_I^{(2)})$$

The first term in the constraint function must be computed in a point function routine ODEPTF (see the example in ODEPTF (p. 450)).

Definition of this constraint is achieved with the following code fragment in the ODEINP subroutine:

```
C
      IF(IPHASE.EQ.1) THEN
C
C          EVALUATE THE SQUARED TERM (Y1 + Y2)**2 IN A POINT FUNCTION ROUTINE
C
C          NPF(2) = 1
C
C      ELSEIF(IPHASE.EQ.2) THEN
C
C          DEFINE CONSTRAINT NUMBER 3
C
C          DEFINE THE FIRST TERM (COMPUTED FUNCTION AT END OF PHASE 1)
C
C          ITERM(1,1) = 3
C          ITERM(2,1) = 1
C          ITERM(3,1) = +1
C          ITERM(4,1) = -1
C
C          DEFINE THE SECOND TERM (CONTROL AT THE BEGINNING OF PHASE 2)
C
C          ITERM(1,2) = 3
C          ITERM(2,2) = 2
C          ITERM(3,2) = -1
C          ITERM(4,2) = 2
C          COEF(2) = -4.D0
C          CLB(3) = 2.4D0
C          CUB(3) = 3.7D0
C
C      ENDIF
C
```

EXAMPLE 2: General Path Constraints

As a second illustration of the ITERM input mechanism consider a single phase problem with two state variables, and one control variable, so $z = (y_1, y_2, u_1)$. Let us suppose that the optimal control problem requires the following path constraint

$$-4.1 \leq g_1(t) \leq 1.9$$

for $t_I \leq t \leq t_F$ where the constraint function is given by

$$g_1(t) = \frac{1}{2} \exp[-y_1(t) - 3y_2(t)] + 3u_1(t).$$

This example requires evaluation of the constraint function $g_1(t)$ in the subroutine ODERHS. The following code fragment which should appear in ODEINP illustrates this construction:

```
C
```

```

C          SPECIFY THE NUMBER OF CONTINUOUS FUNCTIONS ON THE PHASE,
C          I.E. 2 DIFFERENTIAL EQUATIONS, 1 ALGEBRAIC CONSTRAINT, AND
C          NO QUADRATURE FUNCTIONS
C
C          NCF(1) = 2
C          NCF(2) = 1
C          NCF(3) = 0
C
C          DEFINE CONSTRAINT NUMBER 1
C
C          DEFINE THE FIRST (AND ONLY) TERM
C
C          ITERM(1,1) = 1
C          ITERM(2,1) = 1
C          ITERM(3,1) = 0
C          ITERM(4,1) = -1
C          CLB(1) = -4.1D0
C          CUB(1) = 1.9D0
C

```

EXAMPLE 3: Linking Variables

One of the most common requirements for multiphase optimal control problems is to require continuity in the dynamic variables across phase boundaries. Mathematically the desired constraints are:

$$z \left[t_F^{(k)} \right] = z \left[t_I^{(k+1)} \right]$$

Because these constraints are so common a special utility routine LINKST (p. 481) is available to simplify the specification of the ITERM array. The code fragment which should appear in ODEINP illustrates how to link state variables:

```

C
C          IF(IPHASE.GT.1) THEN
C
C          LINK (NST) STATE VARIABLES ACROSS THE BOUNDARY BETWEEN
C          PHASE (IPHASE-1) AND PHASE (IPHASE)
C
C          DEFINE THE NUMBER OF CONSTRAINTS AND TERMS FOR PHASE IPHASE
C
C          NCONS = 0
C          NTERMS = 0
C
C          DO 110 I = 1,NST
C
C          CALL LINKST(IPHASE-1,+1,I,IPHASE,-1,I,NCONS,NTERMS,ITERM,COEF,CLB,CUB)
C
C          110 CONTINUE

```

```
C
      ENDIF
C
```

FORTRAN Array Conventions

It is often possible to simplify the user input routine by exploiting how FORTRAN treats arrays. Specifically the user routine ODEINP is *called* by SOS, and the SOS library allocates storage for all arrays that appear in the argument list. However, all operations performed inside of the user routine ODEINP are *relative* to the first array location, and assume a column oriented storage scheme. To illustrate consider code fragments for two different implementations involving the arrays NCF, STSKL, and INIT. In the first implementation the arrays NCF, STSKL, and INIT are explicitly dimensioned and defined by executable statements in the code.

Full Format Implementation

```
C
      INTEGER      NCF(6),INIT(2),
      $           .....
      DIMENSION    STSKL(0:MXSTAT+MXPARM,2),
      $           .....
C
C      DEFINE PROBLEM WITH 4 DIFFERENTIAL EQUATIONS, 1 ALGEBRAIC
C      EQUATION, AND 1 QUADRATURE EQUATION.  NO AUXILLIARY FUNCTIONS
C      AND NO DISCRETE DATA.
C
      NCF(1) = 4
      NCF(2) = 1
      NCF(3) = 1
C
C      INITIALIZE THE PROBLEM USING AN ODE INTEGRATION WITH
C      RKT PROPAGATION ALGORITHM
C
      INIT(1) = 6
      INIT(2) = 1
C
      STSKL(3,2) = 1.D0
C
```

In the second implementation, NCF is dimensioned of length 3 (not 5), INIT is treated as a scalar (not an array), and STSKL is an array with one (not two) dimensions.

Abbreviated Format Implementation

```
C
```



```

      INTEGER      NCF(3),INIT,
$              .....
      DIMENSION    STSKL(0:MXSTAT+MXPARM),
$              .....
C
C      DEFINE PROBLEM WITH 4 DIFFERENTIAL EQUATIONS, 1 ALGEBRAIC
C      EQUATION, AND 1 QUADRATURE EQUATION.  NO AUXILLIARY FUNCTIONS
C      AND NO DISCRETE DATA.
C
      NCF(1) = 4
      NCF(2) = 1
      NCF(3) = 1
C
C      INITIALIZE THE PROBLEM USING AN ODE INTEGRATION WITH
C      RKT PROPAGATION ALGORITHM
C
      INIT = 6
C
C      USE DEFAULT SCALING (STSKL NOT SPECIFIED)
C
```

ODEPRT: Optimal Control Print

PURPOSE

ODEPRT is an optional user-supplied subroutine which evaluates any auxiliary functions and prints the quantities to an auxiliary output file. Subroutine ODEPRT is an input for both SOSCTL and the auxiliary output utility AUXOUT. If auxiliary output is not desired, the user may specify the system supplied subroutine, DUMYPR, in the calling sequence to SOSCTL. The dummy print routine will print the output scaled dynamic variables to the auxiliary print unit if one is specified.

RELATED SUBPROGRAMS

SOSCTL Sparse Optimal Control Software

AUXOUT Auxiliary Output Utility

METHOD

ODEPRT is a generic name for this routine, and the actual routine name must be declared as an external in the main program and placed in the calling list to subroutine SOSCTL and/or AUXOUT. The subroutine specified as input to SOSCTL will produce output during the SOCX iterations. The subroutine specified as input to AUXOUT will produce output *after* the SOCX iteration process has terminated.

The STSKL array is the same as that loaded in ODEINP, and acts as a unit conversion between internal units and print units. The frequency with which ODEPRT is called from SOSCTL will be determined from the settings for the print flags IPFAUX and IPODE. When ODEPRT is called from AUXOUT the frequency of the output is determined from the AUXOUT input parameters.

USER-SUPPLIED SUBPROGRAM

The user must supply a subroutine with the following format:

```
SUBROUTINE ODEPRT(IPHASE,IPHEND,T,Z,NZ,P,NP,STSKL,IPUAUX,IPFAUX)
INTEGER   IPHASE,IPHEND,NZ,NP,IPUAUX,IPFAUX
DOUBLE PRECISION  T,Z(NZ),P(NP),STSKL(0:NZ)
```

User supplied code to evaluate the requested arguments (see below).

```
      .
      .
      .
RETURN
END
```

ARGUMENTS

IPHASE	[INPUT, INTEGER] The current phase number.
IPHEND	[INPUT, INTEGER] The phase location indicator.
	IPHEND = -1 Current value of t corresponds to the beginning of the phase, i.e. at t_I .
	IPHEND = 0 Current value of t corresponds to the interior of the phase, i.e. $t_I < t < t_F$
	IPHEND = +1 Current value of t corresponds to the end of the phase, i.e. at t_F .
T	[INPUT, DOUBLE PRECISION] Value of the independent variable t .
Z	[INPUT, DOUBLE PRECISION, ARRAY] Real array of length NZ containing the dynamic variables $z(t) = [y(t), u(t)]$.
NZ	[INPUT, INTEGER] The number of time dependent variables.
P	[INPUT, DOUBLE PRECISION, ARRAY] Real array of length NP containing the discrete parameters p , and the times t_I and/or t_F if they are free (see usage remarks for ODERHS).
NP	[INPUT, INTEGER] The length of the array P. In general, $NP = NPV + k_t$ where NPV is the number of user defined discrete parameters and
	$k_t = \begin{cases} 0, & \text{if both } t_I \text{ and } t_F \text{ are fixed;} \\ 1, & \text{if either } t_I \text{ or } t_F \text{ are free;} \\ 2, & \text{if both } t_I \text{ and } t_F \text{ are free.} \end{cases}$
STSKL	[INPUT, DOUBLE PRECISION, ARRAY] Real array of length NZ + 1 containing scales for output quantities $\hat{t}$ and $\hat{z}$.
	$\hat{t} = \text{STSKL}(0) * t$ Scaled independent variable.
	$\hat{z}_k = \text{STSKL}(k) * z_k$ Scaled dynamic variable, k .
IPUAUX	[INPUT, INTEGER] FORTRAN unit number for auxiliary print, which must be opened by user prior to calling SOSCTL.
IPFAUX	[INPUT, INTEGER] Flag controlling the level of auxiliary print.

EXAMPLE Linear Tangent Steering Analytic Solution

Section 4.1 describes the use of SOCX to solve a trajectory optimization problem which results in *linear tangent steering*. This particular example can be solved analytically resulting in the expression

$$\tan u = \tan u_0 - ct$$

where u_0 and c are constants. It may be desirable to compare the analytic expression for $u(t)$ with the numerical approximation $\hat{u}(t)$ constructed by SOCX. For this example the control is the fifth dynamic variable and the scale factor is used to convert the internal values from radians to degrees. The following code fragment could be used in subroutine ODEPRT:

```
C
C          DEFINE CONSTANTS FOR CLOSED FORM SOLUTION.
C
      U0 = 54.6263551908E0/STSKL(5)
      TANU0 = TAN(U0)
      T1 = .554570878337E0
      C = 2.E0*TANU0/T1
C
C          COMPUTE ANALYTIC EXPRESSION FOR CONTROL, BETA.
C
      TANU = TANU0 - C*T
      BETA = ATAN(TANU)
C
C          COMPUTE THREE QUANTITIES FOR OUTPUT;
C          (1) THE ANALYTIC SOLUTION IN DEGREES
C          (2) THE SOCX APPROXIMATION IN DEGREES
C          (3) THE ERROR BETWEEN THE ANALYTIC AND APPROXIMATE SOLUTION
C
      EXACT = BETA*STSKL(5)
      SOCXU = Z(5)*STSKL(5)
      ERROR = EXACT - SOCXU
C
C          CHECK OUTPUT FLAG AND THEN DO THE FOLLOWING:
C
C          IF (0 .LT. IPFAUX .LT. 10) PRINT RESULTS AT THE ENDS
C                                     OF THE PHASE
C
C          IF (10 .LE. IPFAUX)          PRINT RESULTS AT ALL GRID
C                                     POINTS
C
      IF( 0.LT.IPFAUX .AND. IPFAUX.LT.10 ) THEN
        IF(IPHEND.NE.0) THEN
          WRITE(IPUAUX,1001) EXACT,SOCXU,ERROR
        ENDIF
      ELSEIF( 10 .LE. IPFAUX ) THEN
        WRITE(IPUAUX,1001) EXACT,SOCXU,ERROR
```

```
      ENDIF  
C  
1001 FORMAT(T5,'EXACT =',G16.6,T40,'SOCXU =',G16.6,T75,'ERROR =',G16.6)  
C
```

ODEPTF: Optimal Control Point Function Evaluation

PURPOSE

ODEPTF is an optional user-supplied subroutine which evaluates the point functions which occur at either the beginning or end of each phase. This routine is only called if the user has input a nonzero value for either NPF(1) or NPF(2) in subroutine ODEINP. For applications that do not require evaluation of point functions the system supplied dummy routine, DUMYPF may be used in the calling arguments to SOSCTL, SOSDLY, or SOSEST.

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control Software
ODEINP	Optimal Control Problem Definition
ODEIGS	Optimal Control Initial Guess
ODERHS	Optimal Control Right Hand Side Evaluation
ODEPRT	Optimal Control Print
INSOCX	Sparse Optimal Control Input Procedure
INPSOS	Sparse Optimization Suite Input Procedure

METHOD

ODEPTF is a generic name for this routine, and the actual routine name must be declared as an external in the main program and placed in the calling list to subroutine SOSCTL, SOSDLY, or SOSEST.

At the beginning of a specific phase the fundamental inputs to this routine are values for the independent variable t_I , the dependent functions $z(t_I)$ (states plus controls), and the constant parameters, p . From this information, ODEPTF must evaluate the point functions

$$\psi[y(t_I), u(t_I), p, t_I].$$

Similarly, at the end of the phase, using input values for the independent variable t_F , the dependent functions $z(t_F)$ (states plus controls), and the constant parameters, p , ODEPTF must evaluate the point functions

$$\psi[y(t_F), u(t_F), p, t_F].$$

Note that the number of point functions computed at the beginning of the phase NPF(1) and the number of point functions at the end of the phase NPF(2) are both defined in ODEINP. The computed values for the point functions must be stored in the output array PSI in the order consistent with the definition of the ITERM array which is also defined in ODEINP. Thus a term which was input as (ITERM(2,K)=2, ITERM(3,K)=1, ITERM(4,K)=-5), should be loaded into PSI(5) when ODEPTF is called with IPHASE=2 and IPHEND=1. Subroutine ODEPTF is also used to compute the event condition $E[y(t_F), u(t_F), p, t_F]$ when initialization option INIT(1) = 10.

There can be only one event condition enforced with option $\text{INIT}(1) = 10$ and consequently the values of $\text{NPF}(1)$ and/or $\text{NPF}(2)$ are not altered.

It may not be possible to evaluate the point functions for some arguments $[y(t_I), u(t_I), p, t_I]$ and/or $[y(t_F), u(t_F), p, t_F]$. The function evaluation error flag, IFERR, should be set to zero if the right hand side evaluation is successful. Otherwise, IFERR can be set to any nonzero value to designate that the point function can not be evaluated for the current inputs. It may be useful for the user to set the flag to different values depending on the type of input error so that the system print of the error number will have meaning in the user's context.

WARNING

The functions loaded in PSI should only depend on computations involving input values of T,Z,P and constants of the problem.

The functions computed in ODEPTF as supplied by the user are assumed to be continuous and have continuous first and second derivatives. Nondifferentiable functions can cause unpredictable performance in the optimization algorithm. It should be emphasized that discontinuities in the second derivatives of these functions can significantly degrade the speed of convergence without producing any other obvious difficulties. The following common sources of error *should be avoided* when evaluating the point functions:

- (1) nonsmooth interpolation of data;
- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;
- (3) discontinuous behavior caused by branching for IF tests;
- (4) nondifferentiable functions such as ABS, MAX, and MIN.

The most common source of difficulty encountered when using SOCX is nonsmooth interpolation of tabular data. *DO NOT USE LINEAR INTERPOLATION OF TABULAR DATA!*

USER-SUPPLIED SUBPROGRAM

The user must supply a subroutine with the following format:

```
SUBROUTINE ODEPTF(IPHASE,IPHEND,T,Z,NZ,P,NP,PSI,NPSI,IFERR)
INTEGER  IPHASE,IPHEND,NZ,NP,NPSI,IFERR
DOUBLE PRECISION      T,Z(NZ),P(NP),PSI(NPSI)
```

User supplied code to evaluate the requested arguments (see below).

```
      .
      .
      .
RETURN
END
```

ARGUMENTS

IPHASE	[INPUT, INTEGER] The current phase number.
IPHEND	[INPUT, INTEGER] The phase end indicator.
	IPHEND = -2 Event function evaluated at the beginning of the phase, i.e. at t_I . Used only during initialization with INIT(1) = 10 and INIT(2) < 0.
	IPHEND = -1 Point function evaluated at the beginning of the phase, i.e. at t_I .
	IPHEND = +1 Point function evaluated at the end of the phase, i.e. at t_F .
	IPHEND = +2 Event function evaluated at the end of the phase, i.e. at t_F . Used only during initialization with INIT(1) = 10 and INIT(2) > 0.
T	[INPUT, DOUBLE PRECISION] Value of the independent variable t , (either t_I or t_F).
Z	[INPUT, DOUBLE PRECISION, ARRAY] Real array of length NZ containing the dynamic variables $z(t) = [y(t), u(t)]$.
NZ	[INPUT, INTEGER] The number of time dependent variables.
P	[INPUT, DOUBLE PRECISION, ARRAY] Real array of length NP containing the discrete parameters p , and the times t_I and/or t_F if they are free (see usage remarks for ODERHS).
NP	[INPUT, INTEGER] The length of the array P. In general, NP = NPV + k_t where NPV is the number of user defined discrete parameters and
	$k_t = \begin{cases} 0, & \text{if both } t_I \text{ and } t_F \text{ are fixed;} \\ 1, & \text{if either } t_I \text{ or } t_F \text{ are free;} \\ 2, & \text{if both } t_I \text{ and } t_F \text{ are free.} \end{cases}$
PSI	[OUTPUT, DOUBLE PRECISION, ARRAY] Real array of length NPSI containing the computed values for the point functions $\psi(t)$.
	$E(t) = g[y(t_I), \hat{u}(t_I), p, t_I].$ When IPHEND = -2
	$\psi(t) = \psi[y(t_I), u(t_I), p, t_I].$ When IPHEND = -1
	$\psi(t) = \psi[y(t_F), u(t_F), p, t_F].$ When IPHEND = +1

$$E(t) = g[y(t_F), \hat{u}(t_F), p, t_F]. \quad \text{When IPHEND} = +2$$

NPSI	[INPUT, INTEGER] The number of point functions.	
NPSI = 1		When IPHEND = -2
NPSI = NPF(1)		When IPHEND = -1
NPSI = NPF(2)		When IPHEND = +1
NPSI = 1		When IPHEND = +2
IFERR	[OUTPUT, INTEGER] Point Function Evaluation Error Flag.	
IFERR = 0		Successful Evaluation of Point Functions.
IFERR $\neq$ 0		Error in Evaluation of Point Functions.

EXAMPLE 1: General Boundary Condition

Reconsider the example introduced in ODEINP for a problem with two phases, and assume that on the first phase there are two state variables, and one control variable, so $z^{(1)} = (y_1^{(1)}, y_2^{(1)}, u_1^{(1)})$. On the second phase assume the dynamics are characterized by one state and one control, namely $z^{(2)} = (y_1^{(2)}, u_1^{(2)})$. Finally, let us suppose that the third boundary condition required to define the optimal control problem on the second phase has the form

$$2.4 \leq \psi_3^{(2)} \leq 3.7$$

where the constraint function is given by

$$\psi_3^{(2)} = \left[y_1^{(1)}(t_F^{(1)}) + y_2^{(1)}(t_F^{(1)}) \right]^2 - 4u_1^{(2)}(t_I^{(2)})$$

The complete definition of this boundary condition requires specification of the ITERM array as described in ODEINP. The first term in the constraint function must be computed in a point function routine ODEPTF. Definition of the term $\left[y_1^{(1)}(t_F^{(1)}) + y_2^{(1)}(t_F^{(1)}) \right]^2$ is achieved with the following code fragment in the ODEPTF subroutine:

```

C
      IF(IPHASE.EQ.1.AND.IPHEND.EQ.1) THEN
C
C          EVALUATE THE SQUARED TERM (Y1 + Y2)**2 IN A POINT FUNCTION ROUTINE

```

```
C
    PSI(1) = (Y(1) + Y(2))**2
C
ENDIF
C
```

ODERHS: Optimal Control Right Hand Side Evaluation

PURPOSE

ODERHS is a user-supplied subroutine which evaluates the right hand sides of the differential equations, the algebraic equations, the quadrature functions, and the auxiliary functions for each input value of the independent variable t , the dependent functions $z(t)$ (states plus controls), and the constant parameters, p . For applications that do not require evaluation of differential equations, algebraic equations, quadrature functions or auxiliary functions the system supplied dummy routine, DUMMYDE may be used in the calling arguments to SOSCTL, SOSDLY or SOSEST.

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control Software
SOSEST	Sparse Optimal Parameter Estimation
SOSDLY	Sparse Optimal Control of Delay Systems
ODEINP	Optimal Control Problem Definition
ODEIGS	Optimal Control Initial Guess
ODEPTF	Optimal Control Point Function Evaluation
ODEPRT	Optimal Control Print
INSOCX	Sparse Optimal Control Input Procedure
INPSOS	Sparse Optimization Suite Input Procedure

METHOD

ODERHS is a generic name for this routine, and the actual routine name must be declared as an external in the main program and placed in the calling list to subroutine SOSCTL, SOSDLY or SOSEST.

On a specific phase the fundamental inputs to this routine are values for the independent variable t , the dependent functions $z(t)$ (states plus controls), and the constant parameters, p . From this information, ODERHS must evaluate the continuous functions

$$F(t) = \begin{bmatrix} f[y(t), u(t), p, t] \\ g[y(t), u(t), p, t] \\ q[y(t), u(t), p, t] \\ a[y(t), u(t), p, t] \end{bmatrix}.$$

The continuous functions are made up of the right hand sides of the differential equations

$$\dot{y} = f[y(t), u(t), p, t]$$

and the functions $\bar{g}$ and a defining the path constraints

$$g_l \leq \bar{g}[y(t), u(t), p, t] \leq g_u,$$

where $\bar{g}$ is a vector of size n_g . The algebraic constraints are of the form

$$\bar{g}[y(t), u(t), p, t] = \alpha_0 g[v, t] + \alpha^\top v + \beta^\top a[v, t]$$

where

$$v = \begin{bmatrix} y(t) \\ u(t) \\ p \end{bmatrix}.$$

The constraint definition can include *analytic* terms involving $\alpha^\top v$ where the $(n_y + n_u + n_p)$ vector α is constant, as well as linear combinations of the n_a *auxiliary functions* $a_k(v)$ where β is constant. The continuous functions also include the quadrature functions appearing as integrands in the expression

$$\int_{t_I}^{t_F} q[y(t), u(t), p, t] dt.$$

Note that the array of continuous functions $F(t)$ must be defined with the values of NCF(1) differential equations stored first, followed by NCF(2) path constraints. The NCF(3) quadrature functions are stored next and the NCF(4) auxiliary functions last. The partitioning of the continuous function array $F(t)$ is defined in subroutine ODEINP by the array NCF. For problems with delay variables NCF(6) is nonzero, and the delayed variables are stored after the real control variables.

The SOCX software may request an evaluation of the continuous functions for any value of t in the range $t_I \leq t \leq t_F$. The function evaluation error flag, IFERR, should be set to zero if the right hand side evaluation is successful. Otherwise, IFERR can be set to any nonzero value to designate that the differential-algebraic system can not be evaluated for the current inputs. It may be useful for the user to set the flag to different values depending on the type of input error so that the system print of the error number will have meaning in the user's context.

USAGE REMARKS

For some applications it is convenient to define the continuous functions in terms of *relative* times τ from the beginning or end of the phase, e.g. $\tau = t - t_I$ or $\tau = t_F - t$. Suppose the number of discrete parameters is n_p . If the initial or final times have been specified as variables in ODEINP, then they will also appear in the parameter array after the user defined parameters. Thus, if t_I is variable, then $t_I = P(n_p + 1)$. If the final time of the phase, t_F , is also variable, then $t_F = P(n_p + 2)$. In cases where t_I is fixed, but t_F is free, $t_F = P(n_p + 1)$. This information in the parameter array permits the user to compute the relative time τ from the input value of t , thereby allowing construction of differential equations depending on the relative time since the beginning of the phase, or the time to go until the end of the phase.

WARNING

The system of differential-algebraic equations may be different on each phase, but may not be changed within a phase. Thus the number of dependent functions, $z(t)$, and parameters, p , can vary from phase to phase.

The functions computed in ODERHS supplied by the user are assumed to be continuous and have continuous first and second derivatives. Nondifferentiable functions can cause unpredictable performance in the optimization algorithm. It should be emphasized that discontinuities in the

second derivatives of these functions can significantly degrade the speed of convergence without producing any other obvious difficulties. The following common sources of error *should be avoided* when evaluating the right hand side functions:

- (1) nonsmooth interpolation of data;
- (2) iterative procedures inside the function evaluation process, such as adaptive quadrature or root solving;
- (3) discontinuous behavior caused by branching for IF tests;
- (4) nondifferentiable functions such as ABS, MAX, and MIN.

The most common source of difficulty encountered when using SOCX is nonsmooth interpolation of tabular data. *DO NOT USE LINEAR INTERPOLATION OF TABULAR DATA!*

USER-SUPPLIED SUBPROGRAM

The user must supply a subroutine with the following format:

```
SUBROUTINE ODERHS(IPHASE,T,Z,NZ,P,NP,F,NF,IFERR)
```

```
INTEGER   IPHASE,NZ,NP,NF,IFERR
```

```
DOUBLE PRECISION    T,Z(NZ),P(NP),F(NF)
```

User supplied code to evaluate the requested arguments (see below).

```
      .
      .
      .
RETURN
END
```

ARGUMENTS

- | | |
|---------------|---|
| IPHASE | [INPUT, INTEGER]
The current phase number. |
| T | [INPUT, DOUBLE PRECISION]
Value of the independent variable t . |
| Z | [INPUT, DOUBLE PRECISION, ARRAY]
Real array of length NZ containing the dynamic variables $z(t) = [y(t), u(t)]$. When $NCF(6) > 0$, $z(t) = [y(t), u(t), \hat{\mathbf{v}}(\omega_1), \hat{\mathbf{w}}(\omega_1), \dots, \hat{\mathbf{v}}(\omega_\nu), \hat{\mathbf{w}}(\omega_\nu)]$ (cf (5.8)). |
| NZ | [INPUT, INTEGER]
The number of time dependent variables. |

P	[INPUT, DOUBLE PRECISION, ARRAY] Real array of length NP containing the discrete parameters p , and the times t_I and/or t_F if they are free (see usage remarks).
NP	[INPUT, INTEGER] The length of the array P. In general, $NP = NPV + k_t$ where NPV is the number of user defined discrete parameters and $k_t = \begin{cases} 0, & \text{if both } t_I \text{ and } t_F \text{ are fixed;} \\ 1, & \text{if either } t_I \text{ or } t_F \text{ are free;} \\ 2, & \text{if both } t_I \text{ and } t_F \text{ are free.} \end{cases}$
F	[OUTPUT, DOUBLE PRECISION, ARRAY] Real array of length NF containing the computed values for the continuous functions $F(t)$.
NF	[INPUT, INTEGER] The number of continuous functions, $NF = NCF(1) + NCF(2) + NCF(3) + NCF(4) + NCF(5) + NCF(6)$.
IFERR	[OUTPUT, INTEGER] Continuous Function Evaluation Error Flag. IFERR = 0 Successful Evaluation of Continuous Functions. IFERR $\neq$ 0 Error in Evaluation of Continuous Functions.

EXAMPLE 2: General Path Constraints

As an illustration let us reconsider the problem introduced in ODEINP. For this single phase example there are two state variables, and one control variable, so $z = (y_1, y_2, u_1)$. The dynamics of the system are described by the differential equations

$$\begin{aligned} \dot{y}_1 &= -uy_1 \\ \dot{y}_2 &= uy_1 - \rho u^k y_2 \end{aligned}$$

where the constants $\rho > 0$ and $k > 0$. As before suppose that the optimal control problem requires the following path constraint

$$-4.1 \leq g_1(t) \leq 1.9$$

for $t_I \leq t \leq t_F$ where the constraint function is given by

$$g_1(t) = \frac{1}{2} \exp[-y_1(t) - 3y_2(t)] + 3u_1(t).$$

For this example the user supplied subroutine ODERHS must evaluate the right hand side of the differential equations and the constraint function $g_1(t)$. The following code fragment illustrates this construction:

```
C
C      DEFINE THE PROBLEM PARAMETERS
C
      RHO    = 2.5E0
      EXPON  = 1.5E0
      Y1     = Z(1)
      Y2     = Z(2)
      U      = Z(3)
C
C      CONSTRUCT THE RIGHT HAND SIDES OF THE DIFFERENTIAL EQUATIONS
C
      F(1)   = - U*Y1
      F(2)   = U*Y1 - RHO*Y2*(U**EXPON)
C
C      CONSTRUCT THE PATH CONSTRAINT FUNCTION
C
      F(3)   = EXP(-Y1 - 3.E0*Y2)/2.E0 + 3.E0*U
C
```

6.4 Utility Subprograms for Optimal Control

AUTOLK: Auto Link Variable Utility

PURPOSE

AUTOLK is a utility routine which can be used in the user-supplied ODEINP subroutine to link variables from one phase to another.

RELATED SUBPROGRAMS

ODEINP Optimal Control Problem Definition

LINKST Optimal Control Linkage Utility

METHOD

Subroutine AUTOLK is a utility routine which is called from the user supplied input routine ODEINP. A single call to the routine will construct the inputs necessary to define a *linkage* constraint in one of the forms

'name1 < name2'

'name1 = name2'

'name1 > name2'

Note that the character string “<” is used to denote the mathematical operation “ $\leq$ ”, and similarly “ $\geq$ ” is denoted by “>”. Both *name1* and *name2* are character variables having the following format

name1 = “*symbol@phaseE*”.

The character substring *'symbol'* defines the name of the variable that is to be linked. The substring *'symbol'* of maximum length 8 is

- case sensitive (i.e. **TIME** is not the same as **time**)
- must exactly match a user specified label quantity STLBL or PLBL.

The substring *'phase'* is the numeric phase number for the desired variable and must be preceded by the symbol @.

The substring *'E'* denotes which end of the phase is linked.

- The beginning of a phase is denoted by $E = i$ or $E = I$.
- The end of a phase is denoted by $E = f$ or $E = F$.

For example the statement

```
call autolk('VEL@1f = VEL@2I')
```


executed in the user input routine ODEINP, will link the user defined variable VEL at the end of phase 1, to the same variable at the beginning of phase 2. Each call to AUTOLK will construct a constraint using the ITERM array. Multiple calls to the utility will create successive linkage constraints in the order same order AUTOLK is called. For example four successive calls to AUTOLK will construct four linkage constraints, e.g.

(B02)	LINK1	TIME@2I > TIME@1F	
(B03)	LINK2	HALT@1F = HALT@2I	
(B04)	LINK3	VEL@1F = VEL@2I	
(B05)	LINK4	MASS@1F = MASS@2I	

USAGE

```
CALL AUTOLK('string')
```

ARGUMENTS

STRING [INPUT, CHARACTER*80]
Character string containing the linkage condition of the form '*name1* = *name2*' as described in method.

AUTOPL: Auto Phase Length Constraint Utility

PURPOSE

AUTOPL is a utility routine which can be used in the user-supplied ODEINP subroutine to constrain the length of a phase.

RELATED SUBPROGRAMS

ODEINP	Optimal Control Problem Definition
PHSLNG	Optimal Control Phase Duration Utility

METHOD

Subroutine AUTOPL is a utility routine which is called from the user supplied input routine ODEINP. A single call to the routine will construct the inputs necessary to constrain the length (duration) of a phase. Specifically bounds of the following form can be imposed:

$$\begin{aligned}
\Delta t &= a \\
\Delta t &\leq a \\
\Delta t &\geq a \\
a &= \Delta t \\
a &\leq \Delta t \\
a &\geq \Delta t \\
a &\leq \Delta t \leq b \\
a &\geq \Delta t \geq b
\end{aligned}$$

where $\Delta t = t_F - t_I$ and t_I is the time (i.e. independent variable) at the beginning of the phase, and t_F is the time at the end of the phase. Typically $t_I \leq t_F$ however the procedure can also be applied when $t_I \geq t_F$. The phase duration or length Δt is denoted by the case insensitive character string PHSLNG. The fixed bounds a and b can be specified using any character format that has a numeric representation, i.e. integer, real, or double precision. The following examples are all valid:

```

call autopl('1 < PHSLNG')
call autopl('14.d0 > PHSLNG > .1')
call autopl('PHSLNG = 27.35e-2')

```

Note that the character string “<” is used to denote the mathematical operation “ $\leq$ ”, and similarly “ $\geq$ ” is denoted by “>”. Regardless of the format, the routine AUTOPL cannot be called more than once per phase from the user input routine ODEINP. A call to AUTOPL will construct a constraint using the ITERM array. If either the initial or final time is fixed by user specified bounds, the constraint limits will be shifted accordingly. For example suppose that $t_I = 2$, which is implemented in ODEINP by setting

```

YLB(-1,0)=2.
YUB(-1,0)=2.

```

Furthermore suppose the phase length is constrained using the following statement in ODEINP

```
call autop1('5 < PHSLNG')
```

These conditions can be simplified as follows

$$\begin{aligned}a &\leq \Delta t \\5 &\leq t_F - t_I \\5 &\leq t_F - 2 \\7 &\leq t_F\end{aligned}$$

and the final constraint construction will reflect this simplification.

USAGE

```
CALL AUTOPL('string')
```

ARGUMENTS

STRING [INPUT, CHARACTER*80]
Character string containing the linkage condition of the form '*name1* = *name2*' as described in method.

AUXOUT: Auxiliary Output Utility

PURPOSE

AUXOUT is an auxiliary output utility which can be used to display the optimal control solution produced by SOCX. The routine permits the user to print the optimal solution and any auxiliary data desired at either a fixed step size during the phase or at a specified number of points. This utility calls a user-supplied subroutine ODEPRT.

RELATED SUBPROGRAMS

SOSCTL Sparse Optimal Control Software

OCSEVL Optimal Control Solution Evaluation Utility

METHOD

Subroutine AUXOUT is a utility routine which is called by the user after SOSCTL. The optimal solution produced by SOSCTL is represented using B-spline information contained in the array CSTAT. This information is input to the AUXOUT utility. Using one option of AUXOUT permits the solution to be displayed at a specified number of points equally distributed over the phase. A second option permits the solution to be displayed at a specified step length. AUXOUT calls a user-supplied ODEPRT routine which must be declared as an external in the main program and placed in the calling list to subroutine AUXOUT. The STSKL array is the same as that loaded in ODEINP, and acts as a unit conversion between internal units and print units.

It should be emphasized that the user-supplied subroutine which is an input to AUXOUT *does not* have to be the same as the user-supplied subroutine which is input to SOSCTL. The format for this subroutine is unique, and is described in the section on ODEPRT. However the *name* supplied to AUXOUT may differ from the *name* supplied to SOSCTL. The subroutine supplied as input to SOSCTL will produce auxiliary output *during* the SOCX iterations. In contrast the subroutine supplied as input to AUXOUT will produce auxiliary output *after* the SOCX solution has been computed. Thus it may be desirable to use one routine (e.g. ODEPR1) during the SOCX iterations, and a second routine (e.g. ODEPR2) to produce an output file for graphical display of the solution.

Finally, subroutine AUXOUT is designed to display the complete optimal solution. In contrast the auxiliary routine OCSEVL should be used to evaluate the solution at one or more specific points.

USAGE

```
INTEGER  IPHASE, IOUNIT, IOFLAG, NPTAUX, MAXCS, MXSTAT
DOUBLE PRECISION  CSTAT(MAXCS), STSKL(0:MXSTAT), DTAUX
EXTERNAL  ODEPRT

CALL AUXOUT(IPHASE, CSTAT, MAXCS, STSKL, IOUNIT, IOFLAG,
```

NPTAUX,DTAUX,ODEPRT)

ARGUMENTS

IPHASE	[INPUT, INTEGER] The current phase number.
CSTAT	[INPUT, DOUBLE PRECISION, ARRAY] The spline array for the current phase.
MAXCS	[INPUT, INTEGER] Length of CSTAT array.
STSKL	[INPUT, DOUBLE PRECISION, ARRAY] Real array containing scales for output quantities $\hat{t}$ and $\hat{z}$. $\hat{t} = \text{STSKL}(0) * t$ Scaled independent variable. $\hat{z}_k = \text{STSKL}(k) * z_k$ Scaled dynamic variable, k .
IOUNIT	[INPUT, INTEGER] FORTRAN unit number for auxiliary print, which must be opened by user prior to calling AUXOUT.
IOFLAG	[INPUT, INTEGER] Flag controlling the level of auxiliary print.
NPTAUX	[INPUT, INTEGER] The number of output points in the interval $t_I \leq t \leq t_F$. There are three types of output possible. NPTAUX > 0 Subroutine ODEPRT will be called at NPTAUX equally spaced points in the phase. NPTAUX = 0 and DTAUX > 0 Subroutine ODEPRT will be called at points spaced DTAUX apart in the phase. NPTAUX < 0 Subroutine ODEPRT will be called at the grid points in the phase.
DTAUX	[INPUT, INTEGER] The step length between print points used only when NPTAUX = 0. Subroutine ODEPRT will be called for all values of $k = 0, 1, \dots$ such that $t_k \leq t_F$ where $t_k = k * \text{DTAUX} + t_I$, and the final point printed will always be t_F .

ODEPRT [INPUT, SUBROUTINE]

The name of the user routine which evaluates auxiliary print at each point. If sophisticated output is not required, the system supplied dummy routine DUMYPR may be used instead. See Section 3.3

EXAMPLE Creating a Plot File of the Solution

To illustrate the usage of the AUXOUT procedure, suppose that an optimal control problem has been solved using SOSCTL. Most graphics software requires as input a file containing the quantities to be displayed. The system supplied output routine DUMYPR must be declared using an external statement, and will produce a file containing the independent variable followed by all dynamic variables. The following code fragment which must be executed after calling SOSCTL will produce a file suitable for plotting:

```
C
C          OPEN A PLOT FILE ON UNIT 31 CALLED 'TSTSPR.PLT'
C
C      IPLT = 31
C      OPEN(IPLT,FILE='TSTSPR.PLT',STATUS='UNKNOWN')
C
C          DEFINE THE NUMBER OF PLOT POINTS AND SET STEPSIZE ZERO
C
C      NPLPTS = 100
C      DTAUX = ZERO
C
C          SET THE AUXILIARY PRINT FLAG (VALUE CONSISTENT WITH
C          OUTPUT SUBROUTINE)
C
C      IPFAUX = 10
C
C          SET THE PHASE TO ONE.
C
C      IPH = 1
C
C          SET THE PRINT SCALE WEIGHTS TO ONE FOR THE NDYN DYNAMIC
C          VARIABLES
C
C      NDYN = 20
C      STSKL(0:NDYN) = 1.DO
C
C          CREATE THE FILE BY CALLING AUXOUT
C
C      CALL AUXOUT(IPH,CSTAT,MAXCS,STSKL,IPLT,IPFAUX,
C      &          NPLPTS,DTAUX,DUMYPR)
C
C          CLOSE THE PLOT FILE
C
C      CLOSE(IPLT)
```


BSPDEF: B-Spline Algebraic Variable Utility

PURPOSE

BSPDEF is a utility routine which can be used to construct a cubic B-spline representation for an algebraic variable during a phase. It must be called from the user supplied input routine ODEINP (p. 426), and will define the required SOCX input for the construction.

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control Software
SOSEST	Sparse Optimal Parameter Estimation
ODEINP	Optimal Control Problem Definition
ODERHS	Optimal Control Right Hand Side Evaluation

METHOD

Subroutine BSPDEF is a utility routine that is called by the user from ODEINP. BSPDEF will construct the necessary SOCX information to represent an algebraic variable by a cubic B-spline. Specifically,

$$u(t) = c_0 B_0(t) + c_1 B_1(t) + \dots + c_n B_n(t)$$

where $B_j(t)$ are the cubic B-spline basis functions.

This requires four distinct operations:

1. $u(t)$ is defined as an algebraic variable
2. $c_0, c_1, \dots, c_n$ are defined as parameters
3. $c_0 B_0(t) \dots c_n B_n(t)$ are defined as analytic terms
4. The following path constraint is constructed

$$0 = u(t) - [c_0 B_0(t) + c_1 B_1(t) + \dots + c_n B_n(t)] = g(t)$$

The user can also supply a label to identify the variable. The B-spline representation is a continuous, differentiable (C^1) cubic with internal knots equally spaced between the initial phase time $t_I^{(k)}$ and the final phase time $t_F^{(k)}$. Since the B-spline terms $c_j B_j(t)$ are treated analytically, derivatives with respect to the optimization parameters c_j are also computed analytically, and the number of finite difference index sets is not affected.

Since a number of SOCX input quantities are *incremented* when BSPDEF is called, it is important that the user properly initializes these quantities in ODEINP (p. 426). Also since a B-spline construction introduces a path constraint the user supplied routine ODERHS (p. 455) must compute a (trivial) path function $g_j[y(t), u(t), p, t] = 0$ (cf Table 6.2) in the *same order* as defined by calls to BSPDEF. For a B-spline construction with N_s segments, BSPDEF introduces $n_p = (2N_s + 2)$ parameters, and the user must ensure that the value of MXPARM is sufficiently large.

The utility BSPDEF increments the number of algebraic variables NAV, however, it does not construct an initial guess. The user must supply an initial guess $u^{(0)}(t)$ for the algebraic variable using any of the standard initialization options specified by value of INIT as described in ODEINP. Initial values for the B-spline parameters $c_0, c_1, \dots, c_n$ will be computed automatically such that the path constraint

$$0 = u^{(0)}(t_j) - [c_0 B_0(t_j) + c_1 B_1(t_j) + \dots + c_n B_n(t_j)]$$

is satisfied at the B-spline knots t_j for $j = 1, \dots, n$, where $u^{(0)}(t)$ is the user supplied guess for $u(t)$.

BSPDEF also constructs labels for the B-spline construction using the character variables AVRLB1 and AVRLB2. Since each array of B-spline parameters is associated with a corresponding path constraint, the primary label AVRLB1 appears in both the parameter label and the path constraint label. Description information is supplied in the character variable AVRLB2. Default character strings will be utilized if the user does not supply values for AVRLB1 and AVRLB2.

USAGE

```

INTEGER  IPHASE,MXSTAT,MXPARM,NAV,NBAV,NPV,NSEG
INTEGER  NTERM,NKON,IPHASE,NCF(5),ITERM(4,MXTERM),MXTERM,MXPCON,IERBSP
DOUBLE PRECISION PO(MXPARM),CLB(MXPCON),CUB(MXPCON),COEF(MXTERM)
CHARACTER CLBL(0:MXPCON)*80,PLBL(MXPARM)*80,STLBL(0:MXSTAT)*80
CHARACTER*8  AVRLB1
CHARACTER*72 AVRLB2
      .
      .
      .
NTERM = 0
NKON = 0
NAV = 0
NBAV = 0
NPV = 0

```

User supplied code to set ODEINP quantities for current phase

```

NSEG          = 2
AVRLB1(1:8)   = 'BspVar  '
AVRLB2(1:34)  = 'First B-spline Algebraic Variable'
CALL BSPDEF(IPHASE,NAV,NBAV,NPV,NSEG,AVRLB1,AVRLB2,
$   NTERM,NKON,NCF,ITERM,PO,PLBL,
$   STLBL,COEF,CLB,CUB,CLBL,IERBSP)
IF(IERBSP.NE.0) THEN
  PRINT *, 'ERROR IN BSPDEF'
ENDIF

```

ARGUMENTS

IPHASE	[INPUT, INTEGER] Phase number. (Valid Input: $0 < \text{IPHASE} \leq 100$)
NAV	[INPUT/OUTPUT, INTEGER] Number of algebraic variables on the current phase (incremented). (Valid Input: $0 \leq \text{NAV}$)
NBAV	[INPUT/OUTPUT, INTEGER] Number of B-spline algebraic variables on the current phase (incremented). Each B-spline construction introduces $(2 \cdot \text{NSEG} + 2)$ parameters. (Valid Input: $0 \leq \text{NBAV} < 25$)
NPV	[INPUT/OUTPUT, INTEGER] Number of parameters on the current phase (incremented). Each B-spline construction introduces $(2 \cdot \text{NSEG} + 2)$ parameters. (Valid Input: $0 \leq \text{NPV}$)
NSEG	[INPUT, INTEGER] Number of equally spaced segments on the current phase. (Valid Input: $0 < \text{NSEG} \leq 200$)
AVRLB1	[INPUT, CHARACTER*8] Primary algebraic variable label
AVRLB2	[INPUT, CHARACTER*72] Algebraic variable description label
NTERM	[INPUT/OUTPUT, INTEGER] Number of terms on the current phase (incremented). (Valid Input: $0 < \text{NTERM} < \text{MXTERM}$)
NKON	[INPUT/OUTPUT, INTEGER] Number of constraints on the current phase (incremented). (Valid Input: $0 < \text{NKON} < \text{MXPCON}$)
NCF	[INPUT/OUTPUT, INTEGER] Number of path functions on the current phase. NCF(1) must equal the number of differential variables/equations on input. NCF(2) is incremented. (Valid Input: $\text{NCF}(2) \geq 0$)
ITERM	[OUTPUT, INTEGER, ARRAY] Term description array as described in ODEINP.
P0	[OUTPUT, DOUBLE PRECISION, ARRAY] Parameter variable initial values; set to zero.
PLBL	[OUTPUT, CHARACTER*80, ARRAY] Array containing the symbol and description for B-spline parameter variables.
STLBL	[OUTPUT, CHARACTER*80, ARRAY] Character array of length $\text{NCF}(1) + \text{NAV} + 1$ containing symbol and description for B-spline algebraic variable.

COEF	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of term coefficients as described in ODEINP.
CLB	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of constraint lower bounds as described in ODEINP.
CUB	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of constraint upper bounds as described in ODEINP.
CLBL	[OUTPUT, CHARACTER*80, ARRAY] Array containing the symbol and description for constraints.
IERBSP	[OUTPUT, INTEGER] Error Flag. IERBSP = 0 Successful B-spline construction. IERBSP = -1 NTERM < 0. IERBSP = -2 NKON < 0. IERBSP = -3 IPHASE < 1 or IPHASE > 100. IERBSP = -4 NCF(2) < 0. IERBSP = -5 NKON +1 > MXPCON IERBSP = -6 NTERM ≥ MXTERM IERBSP = -7 NCF(1) + NAV +1 > MXSTAT. IERBSP = -8 NBAV < 0 or NBAV ≥ 25. IERBSP = -9 NSEG < 1 or NSEG > 200. IERBSP = -10 NAV < 0. IERBSP = -11 NPV < 0. IERBSP = -12 MXPARM too small. IERBSP = -13 Allocation problem.

CTLFIL: Optimal Control Input From a File

PURPOSE

CTLFIL is a utility routine which can be used set optional input for the SOCX software that is contained in an external file. The utility is useful when SOSCTL and/or SOSEST are called with a predefined or saved set of algorithm parameters.

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control Software
SOSEST	Sparse Optimal Parameter Estimation
SCTFIL	Save an Optimal Control Input File
INSOCX	Sparse Optimal Control Input Procedure
INPSOS	Sparse Optimization Suite Input Procedure

METHOD

Subroutine CTLFIL is a utility routine which is called before SOSCTL and/or SOSEST. CTLFIL will open and read information from a user specified file and then call the SOCX input utility routine INSOCX (p. 403).

USAGE

```
INTEGER  INUNIT
CHARACTER*(*) FILNAM

CALL CTLFIL(INUNIT,FILNAM)
```

ARGUMENTS

INUNIT	[INPUT, INTEGER] FORTRAN unit number for opening file.
FILNAM	[INPUT, CHARACTER] Character string with the name of the file containing the INSOCX (p. 403) input.

EXAMPLE SOCX Input From a File

Suppose that SOCX is to be executed using the barrier algorithm with a modified output level. One way to achieve this is as follows:

```
C
      CALL INSOCX('NLPALG=SBRNLP')
      CALL INSOCX('SBRNLP:BARRIER DEFAULT')
      CALL INSOCX('SBRNLP:IOFLAG=20')
C
      CALL SOSCTL(.....
C
```

An equivalent technique is as follows:

```
C
      CALL CTLFIL(35,'/home/socxinput.fil')
C
      CALL SOSCTL(.....
C
```

where the file “/home/socxinput.fil” contains the following information:

```
nlpalg=SBRNLP
SBRNLP:barrier default
SBRNLP:ioflag = 20
```

The file “/home/socxinput.fil” can be created manually or by using the companion utility routine SCTFIL (p. 510).

CTLSTA: Optimal Control Software Status Utility

PURPOSE

CTLSTA is a utility routine which can be used to retrieve and optionally display the status of the SOS software following execution. The utility is useful when INPSOS has been called with the iteration output turned off, but SOS returns with an error. In this case CTLSTA can be called to retrieve the complete error message, without rerunning the application.

RELATED SUBPROGRAMS

SOSCTL Sparse Optimal Control Software

METHOD

Subroutine CTLSTA is a utility routine which is called after SOSCTL. CTLSTA will retrieve and optionally display information contained in commons used by the SOS library.

USAGE

```
INTEGER  IPF,IPU,IERCTL
```

```
CALL CTLSTA(IPF,IPU,IERCTL)
```

ARGUMENTS

IPF [INPUT, INTEGER]
The print level flag (> 0 for output).

IPU [INPUT, INTEGER]
FORTRAN unit number for printing messages.

IERCTL [INPUT, INTEGER]
The SOS error return flag.

EXAMPLE Diagnostic Messages

If SOSCTL is executed with all output turned off, no error messages will be displayed. To assure retrieval of error messages the following statement can be executed after SOSCTL has terminated:

```
C
      IF (IER.NE.0) CALL CTLSTA(10,IPU,IERCTL)
C
```

FETCH: Variable Retrieval Utility

PURPOSE

FETCH is a utility routine which can be used to retrieve the numerical value(s) of variables specified by the companion utility WATCH (p. 518). It must be called from the main program after termination of SOSCTL, SOSEST, or SOSDLY.

RELATED SUBPROGRAMS

WATCH Variable Monitor Utility

METHOD

Subroutine FETCH is a utility routine that is called by the user from the main program. Unlike the companion utility WATCH, it cannot be called from the user input routine ODEINP. FETCH has a single input argument, a character string, which is the name of the variable to be retrieved. The input string to FETCH must identical to the input string monitored by WATCH. The input string also must be identical to the desired output name which is derived from one of the user specified label quantities STLBL, PLBL, or CLBL. The string must not exceed 80 characters and is of the following form:

'symbol@phase'

The character substring *'symbol'* defines the name of the variable that is to be monitored. The substring *'symbol'* is:

- case sensitive (i.e. TIME is not the same as time)
- must exactly match the desired output name which is derived from a user specified label quantity STLBL, PLBL, or CLBL.

The substring *'phase'* is the numeric phase number for the desired variable and must be preceded by the symbol @. A global quantity with no explicit phase number can be specified by a character string of the form *'symbol'* or *'symbol@0'*.

The FETCH utility retrieves the numerical values of the requested quantity at the solution (or termination) of a run. As such, it must be called after SOSCTL or SOSEST. An attempt to fetch a particular quantity must be preceded by a call to the WATCH (p. 518) utility. The values retrieved are stored in an array and can be used for additional calculations or processing.

USAGE

```
INTEGER  LNVRVL, NUMVAL, IERGET
DOUBLE PRECISION VRVAL(LNVRVL)
CHARACTER*80 VRNAME
```

```
  .
  .
  .
```

User main program call to SOSCTL or SOSEST

```
VRNAME = 'Variable'
CALL FETCH(VRNAME, LNVRVL, VRVAL, NUMVAL, IERGET)
IF (IERGET.NE.0) THEN
  PRINT *, 'ERROR IN FETCH'
ENDIF
```

ARGUMENTS

VRNAME	[INPUT, CHARACTER*80] Character string containing variable name of the form ' <i>symbol@phase</i> '
LNVRVL	[INPUT, INTEGER] Maximum length of the array VRVAL
VRVAL	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of of length NUMVAL containing numerical values of the requested variable.
NUMVAL	[OUTPUT, INTEGER] Length of the array VRVAL ($\text{NUMVAL} \geq 1$)
IERGET	[OUTPUT, INTEGER] Error Flag. IERGET = 0 Successful retrieval. VRVAL contains the requested values. IERGET = -1 VRNAME is too short. IERGET = -2 Input string in VRNAME is not found. IERGET = -3 Input phase number is invalid. IERGET = -4 Valid name not currently implemented in SOS. IERGET = -5 NUMVAL > LNVRVL IERGET = -6 NUMVAL < 1

FYLFMT: Restart File Format Conversion Utility

PURPOSE

FYLFMT is a utility routine that can be used to convert data contained in a SOS restart spline file from binary format to ASCII, or vice versa.

METHOD

When the solution of a SOS application is saved on a restart file, the information is saved in a binary format. This is the default format because data is compactly represented to machine precision. However, when it is necessary to transport data between one or more different hardware platforms, it is desirable to represent the data in a portable ASCII format. Subroutine FYLFMT is a utility routine which can be used to read a restart file in binary format, and write a new file in ASCII format, or conversely read an ASCII file and write a binary file.

USAGE

```
INTEGER  MAXCS,LNRWRK,LNIWRK,IPUOCP,IERFYL
INTEGER  IWORK(LNIWRK)
DOUBLE PRECISION  CSTAT(MAXCS),RWORK(LNRWRK)
```

```
CHARACTER*60 OLDFIL,NEWFIL
LOGICAL A2B
```

```
CALL FYLFMT(CSTAT,MAXCS,RWORK,LNRWRK,IWORK,LNIWRK,
&  ISFRST,OLDFIL,NEWFIL,A2B,IPUOCP,IERFYL)
```

ARGUMENTS

CSTAT	[OUTPUT, DOUBLE PRECISION, ARRAY] The real array containing spline coefficients for the optimal control solution.
MAXCS	[INPUT, INTEGER] The length of CSTAT.
RWORK	[OUTPUT, DOUBLE PRECISION, ARRAY] Real work array
LNRWRK	[INPUT, INTEGER] The length of RWORK
IWORK	[OUTPUT, INTEGER, ARRAY] Integer work array
LNIWRK	[INPUT, INTEGER] The length of IWORK

ISFRST	[INPUT, INTEGER] FORTRAN I/O unit for restart file
OLDFIL	[INPUT, CHARACTER*60] Character variable name of restart file in old format
NEWFIL	[INPUT, CHARACTER*60] Character variable name of restart file in new format
A2B	[INPUT, LOGICAL] Logical format conversion flag. A2B = .TRUE. Convert OLDFIL in ASCII format to NEWFIL in binary format. A2B = .FALSE. Convert OLDFIL in binary format to NEWFIL in ASCII format.
IPUOCP	[INPUT, INTEGER] Output unit number for output from algorithm SOSCTL or SOSEST.
IERFYL	[OUTPUT, INTEGER] Integer error return flag. IERFYL = 0 Successful format conversion. IERFYL = -1 Old restart file name exceeds 60 characters IERFYL = -2 New restart file name exceeds 60 characters IERFYL = -4 Number of CSTAT quantities exceeds MAXCS IERFYL = -5 Number of RWORK quantities exceeds LNRWRK IERFYL = -6 Number of IWORK quantities exceeds LNIWRK

ICSTPH: Optimal Control Solution Evaluation Utility

PURPOSE

ICSTPH is a utility routine which can be used to evaluate the optimal control solution produced by SOS. The routine permits the user to specify the phase number and extract pointers to extract solution data from the CSTAT array.

RELATED SUBPROGRAMS

OCDGRD	Optimal Control Solution and Derivative Grid Evaluation Utility
OCSEVL	Optimal Control Solution Evaluation Utility
OCSTAU	Optimal Control Solution Evaluation Utility
NUMGRD	Gridpoint Evaluation Utility

METHOD

Subroutine ICSTPH is a utility routine which is called after SOSCTL, SOSEST, or SOSDLY. The optimal solution produced by SOSCTL, SOSEST, or SOSDLY is represented using information contained in the array CSTAT. The user must supply the phase number IPHASE and CSTAT as input to the ICSTPH utility. The utility computes the information needed to evaluate both the continuous solution quantities $z(t) = [y(t), u(t)]$, as well as the discrete parameters p .

USAGE

```
INTEGER  IPHASE, MAXCS, IPDPH, NUMDP, IPCPH, IBSPLY, IER
DOUBLE PRECISION  CSTAT(MAXCS)

CALL ICSTPH(IPHASE, CSTAT, MAXCS, IPDPH, NUMDP, IPCPH, IBSPLY, IER)
```

ARGUMENTS

IPHASE	[INPUT, INTEGER] The phase number.
CSTAT	[INPUT, DOUBLE PRECISION, ARRAY] The real array containing discrete and continuous data for the optimal control solution.
MAXCS	[INPUT, INTEGER] The length of CSTAT.

IPDPH	[OUTPUT, INTEGER] Starting location in CSTAT of discrete variables.	
NUMDP	[OUTPUT, INTEGER] Number of discrete parameters p on the phase.	
IPCPH	[OUTPUT, INTEGER] Starting location in CSTAT of coefficients that define continuous variables $z(t) = [y(t), u(t)]$ on the phase.	
IBSPLY	[OUTPUT, INTEGER] Continuous representation indicator; IBSPLY = -1 for piecewise polynomial, IBSPLY = $+1$ for B-spline.	
IER	[OUTPUT, INTEGER] Integer error return flag.	
	IER = 0	Success.
	IER = -1	Phase, IPHASE not found in CSTAT
	IER = -2	Length of CSTAT array exceeded before phase found.
	IER = -3	Error in CSTAT; phase did not start with discrete data header.

LINKST: Optimal Control Linkage Utility

PURPOSE

LINKST is a utility routine which can be used in the user-supplied ODEINP subroutine to link variables from one phase to another.

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control Software
ODEINP	Optimal Control Problem Definition
AUTOLK	Auto Link Variable Utility

METHOD

Subroutine LINKST is a utility routine which is called from the user supplied input routine ODEINP. A single call to the routine will construct the inputs necessary to define a *linkage* constraint of the form

$$z_j \left[t_F^{(k)} \right] = z_\ell \left[t_I^{(m)} \right].$$

The utility can link either the dynamic or independent variables at the beginning or end of a phase.

The usage of subroutine LINKST is illustrated in example 3 of the documentation for subroutine ODEINP (p. 426). Similar functionality is provided using subroutine AUTOLK (p. 460).

USAGE

```

INTEGER  IPHS1, IEND1, ISTAT1, IPHS2, IEND2, ISTAT2, NCPHS, NTPHS, ITERM(4,*)
DOUBLE PRECISION  COEF(*), CLB(*), CUB(*)
CALL LINKST(IPHS1, IEND1, ISTAT1, IPHS2, IEND2, ISTAT2,
            NCPHS, NTPHS, ITERM, COEF, CLB, CUB)

```

ARGUMENTS

IPHS1	[INPUT, INTEGER] The phase number of the first dynamic variable.
IEND1	[INPUT, INTEGER] The phase end indicator of the first dynamic variable.
IEND1 = -1	The first dynamic variable evaluated at the beginning of the phase, i.e. at t_I .
IEND1 = +1	The first dynamic variable evaluated at the end of the phase, i.e. at t_F .

ISTAT1	[INPUT, INTEGER] The number of the first dynamic variable.
IPHS2	[INPUT, INTEGER] The phase number of the second dynamic variable.
IEND2	[INPUT, INTEGER] The phase end indicator of the second dynamic variable. IEND2 = -1 The second dynamic variable evaluated at the beginning of the phase, i.e. at t_I. IEND2 = +1 The second dynamic variable evaluated at the end of the phase, i.e. at t_F.
ISTAT2	[INPUT, INTEGER] The number of the second dynamic variable.
NCPHS	[INPUT/OUTPUT, INTEGER] Running number of constraints on the current phase.
NTPHS	[INPUT/OUTPUT, INTEGER] Running number of terms on the current phase.
ITERM	[INPUT/OUTPUT, INTEGER, ARRAY] Term description array as described in ODEINP.
COEF	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Array of term coefficients as described in ODEINP.
CLB	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Array of constraint lower bounds as described in ODEINP.
CUB	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Array of constraint upper bounds as described in ODEINP.

NUMGRD: Gridpoint Evaluation Utility

PURPOSE

NUMGRD is a utility function which can be used to evaluate the number of grid points in an optimal control solution produced by SOS. The function permits the user to specify the phase number and computes the number of grid points.

RELATED SUBPROGRAMS

OCSEVL Optimal Control Solution Evaluation Utility

OCSGRD Optimal Control Solution Grid Evaluation Utility

METHOD

Subroutine NUMGRD is a utility function which is called after SOSCTL, SOSEST, or SOSDLY. The optimal solution produced by SOSCTL, SOSEST, or SOSDLY is represented using information contained in the array CSTAT. This information is input to the NUMGRD utility. The user must specify the phase number IPHASE. NUMGRD will compute the number of grid points on the phase. This is often useful when constructing an initial guess from another solution.

USAGE

```
INTEGER  MAXCS,IPHASE,NUMPNT
DOUBLE PRECISION  CSTAT(MAXCS)
INTEGER  NUMGRD

NUMPNT = NUMGRD(CSTAT,MAXCS,IPHASE)
```

ARGUMENTS

CSTAT	[INPUT, DOUBLE PRECISION, ARRAY] The real array containing the optimal control solution.
MAXCS	[INPUT, INTEGER] The length of CSTAT.
IPHASE	[INPUT, INTEGER] The phase number.
NUMPNT	[OUTPUT, INTEGER] The number of grid points on phase IPHASE

OCDGRD: Optimal Control Solution and Derivative Grid Evaluation Utility

PURPOSE

OCDGRD is a utility routine which can be used to evaluate the optimal control solution and its time derivatives produced by SOS. The routine permits the user to specify the grid point number and compute the optimal state vector $y(t)$ and control vector $u(t)$. In addition the routine computes the derivatives

$$\frac{dy}{dt}, \quad \frac{d^2y}{dt^2}, \quad \frac{d^3y}{dt^3} \dots$$

and

$$\frac{du}{dt}, \quad \frac{d^2u}{dt^2}, \quad \frac{d^3u}{dt^3} \dots$$

RELATED SUBPROGRAMS

OCSGRD	Optimal Control Solution Grid Evaluation Utility
OCSEVL	Optimal Control Solution Evaluation Utility
OCSTAU	Optimal Control Solution Evaluation Utility
NUMGRD	Gridpoint Evaluation Utility

METHOD

Subroutine OCDGRD is a utility routine which is called after SOSCTL or SOSEST. The optimal solution produced by SOSCTL or SOSEST is represented using information contained in the array CSTAT. This information is input to the OCDGRD utility. The user must specify the phase number IPHASE, the number of derivatives NDER, and the desired grid point number IGRDPT. If solution derivatives are not needed, subroutine OCSGRD can be used.

The primary function of OCDGRD is to evaluate the solution and time derivatives at the gridpoints of a previous solution. This is often useful when constructing an initial guess from another solution.

The solution is stored in the array ZVEC(*,*) as follows:

$$\begin{bmatrix} t_k & * & * & * \\ y_1(t_k) & \dot{y}_1(t_k) & \ddot{y}_1(t_k) & \cdots \\ \vdots & \vdots & \vdots & \\ u_m(t_k) & \dot{u}_m(t_k) & \ddot{u}_m(t_k) & \cdots \end{bmatrix}$$

USAGE

INTEGER MAXCS, NWORK, IPHASE, IGRDPT, LNZVEC, IEROCS


```

DOUBLE PRECISION      CSTAT(MAXCS),WORK(NWORK)
DOUBLE PRECISION      ZVEC(0:LNZVEC,0:NDER)

CALL OCDGRD(CSTAT,MAXCS,WORK,NWORK,IPHASE,
$   IGRDPT,ZVEC,LNZVEC,NDER,IEROCS)

```

ARGUMENTS

CSTAT	[INPUT, DOUBLE PRECISION, ARRAY] The real array containing coefficients for the optimal control solution.
MAXCS	[INPUT, INTEGER] The length of CSTAT.
WORK	[INPUT, DOUBLE PRECISION, ARRAY] Real work array.
NWORK	[INPUT, INTEGER] Length of work array. $NWORK \geq 28 + 4*NDER$
IPHASE	[INPUT, INTEGER] The phase number.
IGRDPT	[INPUT, INTEGER] The grid point number. $1 \leq IGRDPT \leq NUMGRD$, where NUMGRD is the number of grid points on the phase.
ZVEC	[OUTPUT, DOUBLE PRECISION, ARRAY] Real array containing the independent variable t_k and the dynamic variables $z(t_k) = [y(t_k), u(t_k)]$ evaluated at grid point $k = IGRDPT$ when IEROCS = 0. Column one contains the function values, column two contains the first derivative values, etc.
LNZVEC	[INPUT, INTEGER] The maximum length of the augmented array ZVEC. Note that $ZVEC(0,0) = t_k$.
NDER	[INPUT, INTEGER] The number of derivatives, $NDER \geq 0$.
IEROCS	[OUTPUT, INTEGER] Integer error return flag.
	IEROCS = 0 Successful evaluation of $z(t_k)$, NDER derivatives $\dot{z}(t_k), \ddot{z}(t_k), \dots$ and t_k .
	IEROCS = -1 Error finding phase data in CSTAT
	IEROCS = -2 NUMZV > LNZVEC

IEROCS = -3

IPHASE < 0 or IPHASE > MAXPHS

IEROCS = -4

NWORK < 28 + 4*NDER

IEROCS = -5

IGRDPT < 1 or IGRDPT > NUMGRD

$-100 \geq \text{IEROCS} \geq -152$

Evaluation error at t_I . Consult HDSPVL documentation, with IER = IEROCS + 100

OCSEVL: Optimal Control Solution Evaluation Utility

PURPOSE

OCSEVL is a utility routine which can be used to evaluate the optimal control solution produced by SOS. The routine permits the user to specify the value of the independent variable t and compute the optimal state vector $y(t)$ and control vector $u(t)$. The routine also computes the beginning and end of the phase, as well as the discrete parameter values.

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control Software
OCSGRD	Optimal Control Solution Grid Evaluation Utility
OCSTAU	Optimal Control Solution Evaluation Utility
AUXOUT	Auxiliary Output Utility

METHOD

Subroutine OCSEVL is a utility routine which is called after SOSCTL, SOSEST or SOSDLY. The optimal solution produced is represented using information contained in the array CSTAT. This information is input to the OCSEVL utility. The user must specify the phase number IPHASE and the desired evaluation point t . From the phase number OCSEVL first computes the initial time t_I and final time t_F for the phase. If the specified time is not within the phase i.e. either $t < t_I$ or $t > t_F$ the state and control are not evaluated and an error return is set. On the other hand if $t_I \leq t \leq t_F$ the specified time is within the phase, and OCSEVL will evaluate the representation for the dynamic variables and return the computed values for the optimal state vector $y(t)$, control vector $u(t)$, and parameters p .

The primary function of OCSEVL is to evaluate the solution at a few points. Subroutine AUXOUT may be more appropriate when a complete time history of the solution is desired for display purposes.

USAGE

```

INTEGER  MAXCS, NWORK, IPHASE, NUMZV, LNZVEC, NUMPV, LNPVEC, IEROCS
DOUBLE PRECISION  CSTAT(MAXCS), WORK(NWORK)
DOUBLE PRECISION  ZVEC(LNZVEC), PVEC(LNPVEC)
DOUBLE PRECISION  TIME, TZERO, TFINAL

CALL OCSEVL(CSTAT, MAXCS, WORK, NWORK, IPHASE, TIME, TZERO, TFINAL,
            ZVEC, NUMZV, LNZVEC, PVEC, NUMPV, LNPVEC, IEROCS)

```

ARGUMENTS

CSTAT	[INPUT, DOUBLE PRECISION, ARRAY] The real array containing coefficients for the optimal control solution.
MAXCS	[INPUT, INTEGER] The length of CSTAT.
WORK	[INPUT, DOUBLE PRECISION, ARRAY] Real work array.
NWORK	[INPUT, INTEGER] Length of work array. $NWORK \geq 2*LNZVEC + 19$
IPHASE	[INPUT, INTEGER] The phase number.
TIME	[INPUT, DOUBLE PRECISION] The independent variable value t .
TZERO	[OUTPUT, DOUBLE PRECISION] The phase start time t_I .
TFINAL	[OUTPUT, DOUBLE PRECISION] The phase end time t_F .
ZVEC	[OUTPUT, DOUBLE PRECISION, ARRAY] Real array containing the dynamic variables $z(t) = [y(t), u(t)]$ evaluated at t when $IEROCS = 0$.
NUMZV	[OUTPUT, INTEGER] The length of ZVEC.
LNZVEC	[INPUT, INTEGER] The maximum length of ZVEC.
PVEC	[OUTPUT, DOUBLE PRECISION, ARRAY] Real array containing the discrete parameters p evaluated when $IEROCS = 0$.
NUMPV	[OUTPUT, INTEGER] The length of PVEC.
LNPMVE	[INPUT, INTEGER] The maximum length of PVEC.
IEROCS	[OUTPUT, INTEGER] Integer error return flag. $IEROCS = 0$ $IEROCS = +1$ Successful evaluation of $z(t)$, t_I and t_F. Input value of t is outside the phase, i.e. either $t > t_F$ and $z(t)$ is linearly extrapolated from $z(t_F)$ or $t < t_I$ and $z(t)$ is linearly extrapolated from $z(t_I)$.

$\text{IEROCS} = -1$	Error finding phase data in CSTAT.
$\text{IEROCS} = -2$	$\text{NUMZV} > \text{LNZVEC}$
$\text{IEROCS} = -3$	$\text{Numpv} > \text{LNPVEC}$
$\text{IEROCS} = -4$	$\text{IPHASE} < 0$ or $\text{IPHASE} > \text{MAXPHS}$
$\text{IEROCS} = -5$	$\text{NWORK} < 2 * \text{LNZVEC} + 19$
$-100 \geq \text{IEROCS} \geq -152$	Evaluation error at t_I . Consult HDSPVL documentation with $\text{IER} = \text{IEROCS} + 100$
$-200 \geq \text{IEROCS} \geq -252$	Evaluation error at t_F . Consult HDSPVL documentation with $\text{IER} = \text{IEROCS} + 200$
$-300 \geq \text{IEROCS} \geq -352$	Evaluation error at t . Consult HDSPVL documentation with $\text{IER} = \text{IEROCS} + 300$

EXAMPLE Evaluating the Solution

To illustrate the usage of the OCSEVL procedure, suppose that an optimal control problem has been solved using SOSCTL or SOSEST. It is then desirable to perform additional calculations which require the values the dynamic variables at a specific point. The following code fragment which must be executed after calling SOSCTL or SOSEST will evaluate the solution:

```

C
C      THE FOLLOWING SEQUENCE CAN BE USED TO EVALUATE THE
C      SOLUTION AT THE SPECIFIED VALUE OF TIME IN PHASE 6
C
C      TIME = 8700.E0
C      IPHASE = 6
C
C      CALL OCSEVL(CSTAT,MAXCS,WORK,NWORK,IPHASE,TIME,TZERO,
&      TFINAL,ZVEC,NUMZV,LNZVEC,PVEC,Numpv,LNPVEC,IEROCS)
C
C      IF(IEROCS.EQ.0) THEN
C
C          SOLUTION EVALUATED AT THE REQUESTED POINT;  PRINT RESULTS
C
C          PRINT *, 'IEROCS =', IEROCS
C          PRINT *, 'TZERO,TIME,TFINAL',TZERO,TIME,TFINAL
C          PRINT *, 'ZVEC', (ZVEC(IZ),IZ=1,NUMZV)
C          PRINT *, 'PVEC', (PVEC(IZ),IZ=1,Numpv)
C
C      ELSEIF(IEROCS.EQ.1) THEN

```

```
C
C      REQUESTED POINT IS NOT IN PHASE 6; PRINT PHASE BOUNDARIES
C
C      PRINT *, 'IEROCS =', IEROCS
C      PRINT *, 'TZERO,TFINAL',TZERO,TFINAL
C
C      ELSE
C
C      INPUT AND/OR EVALUATION ERROR
C
C      PRINT *, 'IEROCS =', IEROCS
C
C      ENDIF
```

OCSGRD: Optimal Control Solution Grid Evaluation Utility

PURPOSE

OCSGRD is a utility routine which can be used to evaluate the optimal control solution produced by SOS. The routine permits the user to specify the grid point number and compute the optimal state vector $y(t)$ and control vector $u(t)$.

RELATED SUBPROGRAMS

OCDGRD	Optimal Control Solution and Derivative Grid Evaluation Utility
OCSEVL	Optimal Control Solution Evaluation Utility
OCSTAU	Optimal Control Solution Evaluation Utility
NUMGRD	Gridpoint Evaluation Utility

METHOD

Subroutine OCSGRD is a utility routine which is called after SOSCTL, SOSEST, or SOSDLY. The optimal solution produced by SOSCTL, SOSEST, or SOSDLY is represented using information contained in the array CSTAT. This information is input to the OCSGRD utility. The user must specify the phase number IPHASE and the desired grid point number IGRDPT.

The primary function of OCSGRD is to evaluate the solution at the gridpoints of a previous solution. This is often useful when constructing an initial guess from another solution. If time derivatives are required the subroutine OCDGRD can be utilized.

USAGE

```
INTEGER  MAXCS,IPHASE,IGRDPT,LNZVEC,IEROCS
DOUBLE PRECISION  CSTAT(MAXCS),WORK(NWORK)
DOUBLE PRECISION  ZVEC(0:LNZVEC)

CALL OCSGRD(CSTAT,MAXCS,WORK,NWORK,IPHASE,
$  IGRDPT,ZVEC,LNZVEC,IEROCS)
```

ARGUMENTS

CSTAT [INPUT, DOUBLE PRECISION, ARRAY]
The real array containing coefficients for the optimal control solution.

MAXCS	[INPUT, INTEGER] The length of CSTAT.
WORK	[INPUT, DOUBLE PRECISION, ARRAY] Real work array.
NWORK	[INPUT, INTEGER] Length of work array. $NWORK \geq LNZVEC + 19$
IPHASE	[INPUT, INTEGER] The phase number.
IGRDPT	[INPUT, INTEGER] The grid point number. $1 \leq IGRDPT \leq NUMGRD$, where $NUMGRD$ is the number of grid points on the phase.
ZVEC	[OUTPUT, DOUBLE PRECISION, ARRAY] Real array containing the independent variable t_k and the dynamic variables $z(t_k) = [y(t_k), u(t_k)]$ evaluated at grid point $k = IGRDPT$ when $IEROCS = 0$.
LNZVEC	[INPUT, INTEGER] The maximum length of the augmented array ZVEC. Note that $ZVEC(0) = t$.
IEROCS	[OUTPUT, INTEGER] Integer error return flag.
	$IEROCS = 0$ Successful evaluation of $z(t_k)$, and t_k .
	$IEROCS = -1$ Error finding phase data in CSTAT
	$IEROCS = -2$ $NUMZV > LNZVEC$
	$IEROCS = -3$ $IPHASE < 0$ or $IPHASE > MAXPHS$
	$IEROCS = -4$ $NWORK < LNZVEC + 19$
	$IEROCS = -5$ $IGRDPT < 1$ or $IGRDPT > NUMGRD$
	$-100 \geq IEROCS \geq -152$ Evaluation error at t_I . Consult HDSPVL documentation with $IER = IEROCS + 100$

OCSRNG: Optimal Control Solution Range Utility

PURPOSE

OCSRNG is a utility routine which can be used to evaluate the upper and lower bounds for the dynamic variables produced by SOS. The routine permits the user to specify the method used to construct the dynamic variable bounds using the information available in the B-spline solution.

RELATED SUBPROGRAMS

SOSCTL Sparse Optimal Control Software

AUXOUT Auxiliary Output Utility

METHOD

Subroutine OCSRNG is a utility routine which is called after SOSCTL has terminated successfully. The optimal solution produced by SOSCTL is represented using B-spline information contained in the array CSTAT. This information is input to the OCSRNG utility for the desired phase. Specifically the routine will compute the bounds z_L and z_U where

$$z_{L,k} \leq z_k(t) \leq z_{U,k}$$

for $t_I \leq t \leq t_F$ and $k = 1, \dots, (n_y + n_u)$. There are three methods for computing the range based on the input values NPTAUX and DTAUX:

1. NPTAUX > 0: compute $z(t)$ at nptaux equally spaced points in the phase.
2. NPTAUX = 0, and DTAUX > 0: compute $z(t)$ at points equally spaced of length DTAUX.
3. NPTAUX < 0: compute $z(t)$ at the grid points in the phase

The primary purpose of OCSRNG is to construct estimates for the upper and lower bounds. This information is often useful when constructing scale information as well as for display purposes.

USAGE

```
INTEGER  LNZVAR, MAXCS, IPHASE, NPTAUX, IERRNG
DOUBLE PRECISION  CSTAT(MAXCS)
DOUBLE PRECISION  ZUPR(LNZVAR), ZLWR(LNZVAR)
DOUBLE PRECISION  DTAUX
```

```
CALL OCSRNG(CSTAT, MAXCS, IPHASE, ZUPR, ZLWR, LNZVAR, NPTAUX,
$          DTAUX, IERRNG)
```

ARGUMENTS

CSTAT	[INPUT, DOUBLE PRECISION, ARRAY] The real array containing spline coefficients for the optimal control solution (see SOSCTL (p. 382) or SOSEST (p. 395)).
MAXCS	[INPUT, INTEGER] The length of solution array, CSTAT.
IPHASE	[INPUT, INTEGER] The phase on which the bounds are desired.
ZUPR	[OUTPUT, DOUBLE PRECISION, ARRAY] Real array containing the estimated upper bound for the dynamic variables $z(t) = [y(t), u(t)]$ evaluated over the duration of the phase.
ZLWR	[OUTPUT, DOUBLE PRECISION, ARRAY] Real array containing the estimated lower bound for the dynamic variables $z(t) = [y(t), u(t)]$ evaluated over the duration of the phase.
LNZVAR	[INPUT, INTEGER] The maximum length of ZUPR and ZLWR.
NPTAUX	[INPUT, INTEGER] The number of evaluation points in the phase.
DTAUX	[INPUT, DOUBLE PRECISION, ARRAY] The evaluation step length.
IERRNG	[OUTPUT, INTEGER] Integer error return flag.
	IERRNG = 0 Successful evaluation of ZUPR and ZLWR.
	IERRNG = -1 NST > LNZVAR or NST < 1
	IERRNG = -3 Inconsistent values for NPTAUX and DTAUX
	IERRNG ≤ -100 Spline evaluation error. Consult HDSPVL documentation with IER = IERRNG + 100

OCSTAU: Optimal Control Solution Evaluation Utility

PURPOSE

OCSTAU is a utility routine which can be used to evaluate the optimal control solution produced by SOS. The routine permits the user to specify the value of the independent parameter τ where $0 \leq \tau \leq 1$ and compute the value of time t , the optimal state vector $y(t)$ and control vector $u(t)$. The routine also computes the discrete parameter values.

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control Software
OCSEVL	Optimal Control Solution Evaluation Utility
OCSGRD	Optimal Control Solution Grid Evaluation Utility
AUXOUT	Auxiliary Output Utility

METHOD

Subroutine OCSTAU is a utility routine which is called after SOSCTL, SOSEST, or SOSDLY. The optimal solution produced by SOSCTL, SOSEST, or SOSDLY is represented using information contained in the array CSTAT. This information is input to the OCSTAU utility. The user must specify the phase number IPHASE and the desired evaluation point τ . By construction the parameter τ is defined on the domain $0 \leq \tau \leq 1$. Thus, when $\tau = 0$, subroutine OCSTAU will compute the initial time t_I and the corresponding state $y(t_I)$ and control $u(t_I)$. The final time t_F , state $y(t_F)$, and control $u(t_F)$ are computed by calling OCSTAU with $\tau = 1$. In general, for $0 \leq \tau \leq 1$, OCSTAU computes $t(\tau)$, $y[t(\tau)]$, and $u[t(\tau)]$. It is important to note that in general, the time $t(\tau)$ is a *nonlinear* function of the parameter τ , because of the mesh refinement algorithm in SOS. If the specified evaluation point is not within the phase, i.e. either $\tau < 0$ or $\tau > 1$, a warning is set, and the time, state, and control will be constructed using linear extrapolation.

USAGE

```

INTEGER  MAXCS,NWORK,IPHASE,NUMZV,LNZVEC,NUMPV,LNPVEC,IEROCS
DOUBLE PRECISION  CSTAT(MAXCS),WORK(NWORK)
DOUBLE PRECISION  ZVEC(LNZVEC),PVEC(LNPVEC)
DOUBLE PRECISION  TAU,TIME
```

```

CALL OCSTAU(CSTAT,MAXCS,WORK,NWORK,IPHASE,TAU,TIME,ZVEC,NUMZV,LNZVEC,
&           PVEC,NUMPV,LNPVEC,IEROCS)
```

ARGUMENTS

CSTAT	[INPUT, DOUBLE PRECISION, ARRAY] The real array containing coefficients for the optimal control solution.
MAXCS	[INPUT, INTEGER] The length of CSTAT.
WORK	[INPUT, DOUBLE PRECISION, ARRAY] Real work array.
NWORK	[INPUT, INTEGER] Length of work array. $NWORK \geq 2*LNZVEC + 19$
IPHASE	[INPUT, INTEGER] The phase number.
TAU	[INPUT, DOUBLE PRECISION] The parameter value τ .
TIME	[OUTPUT, DOUBLE PRECISION] The independent variable value t .
ZVEC	[OUTPUT, DOUBLE PRECISION, ARRAY] Real array containing the dynamic variables $z(t) = [y(t), u(t)]$ evaluated at t when $IEROCS = 0$.
NUMZV	[OUTPUT, INTEGER] The length of ZVEC.
LNZVEC	[INPUT, INTEGER] The maximum length of ZVEC.
PVEC	[OUTPUT, DOUBLE PRECISION, ARRAY] Real array containing the discrete parameters p evaluated when $IEROCS = 0$.
NUMPV	[OUTPUT, INTEGER] The length of PVEC.
LNPVEC	[INPUT, INTEGER] The maximum length of PVEC.
IEROCS	[OUTPUT, INTEGER] Integer error return flag.
	$IEROCS = 0$ Successful evaluation of $z(t)$, p , and t .
	$IEROCS = +1$ Input value of τ is outside the phase, i.e. either $\tau > 1$ and $z(t)$ is extrapolated linearly from $z(t_F)$ or $\tau < 0$ and $z(t)$ is extrapolated linearly from $z(t_I)$.
	$IEROCS = -1$ Error finding data in CSTAT
	$IEROCS = -2$ $NUMZV > LNZVEC$

IEROCS = -3

NUMPV > LNPVEC

IEROCS = -4

IPHASE < 0 or IPHASE > MAXPHS

IEROCS = -5

NWORK < 2*LNZVEC + 19

$-300 \geq \text{IEROCS} \geq -352$

Evaluation error at t . Consult HDSPVL documentation with IER = IEROCS + 300

PHSLNG: Optimal Control Phase Duration Utility

PURPOSE

PHSLNG is a utility routine which can be used to construct constraints on the duration of a phase. It is typically called from the user supplied input routine ODEINP (p. 426). Specifically, the utility can be used to construct constraints of the form:

$$p_L \leq t_F^{(k)} - t_I^{(k)} \leq p_U$$

where $t_F^{(k)}$ is the final time for phase k , and $t_I^{(k)}$ is the initial time for phase k . Often the phase duration p_L will be positive to ensure that $t_F^{(k)} > t_I^{(k)}$.

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control Software
SOSEST	Sparse Optimal Parameter Estimation
ODEINP	Optimal Control Problem Definition
AUTOPL	Auto Phase Length Constraint Utility

METHOD

Subroutine PHSLNG is a utility routine which is called by the user (usually from ODEINP). PHSLNG will construct a constraint on the phase duration by proper definition of the SOS input quantities. Similar functionality is available using routine AUTOPL (p. 462).

The Goddard Rocket test problem (p. 596) illustrates the use of this utility routine.

USAGE

```

INTEGER  NTERM,NKON,IPHASE,ITERM(4,MXTERM),MXTERM,MXPCON,IERLNG
DOUBLE PRECISION CLB(MXPCON),CUB(MXPCON),COEF(MXTERM),PHSLWR,PHSUPR
CHARACTER CLBL(0:MXPCON)*80

```

```

CALL PHSLNG(NTERM,NKON,IPHASE,ITERM,MXTERM,COEF,
$  CLB,CUB,CLBL,MXPCON,PHSLWR,PHSUPR,IERLNG)

```

ARGUMENTS

NTERM [INPUT/OUTPUT, INTEGER]
Running number of terms on the current phase.

NKON	[INPUT/OUTPUT, INTEGER] Running number of constraints on the current phase.
IPHASE	[INPUT, INTEGER] Phase number
ITERM	[INPUT/OUTPUT, INTEGER, ARRAY] Term description array as described in ODEINP.
MXTERM	[INPUT, INTEGER] Dimension of term arrays as described in ODEINP.
COEF	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Array of term coefficients as described in ODEINP.
CLB	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Array of constraint lower bounds as described in ODEINP.
CUB	[INPUT/OUTPUT, DOUBLE PRECISION, ARRAY] Array of constraint upper bounds as described in ODEINP.
CLBL	[OUTPUT, CHARACTER*80, ARRAY, <i>Default</i> = '␣'] Array containing the symbol and description for constraints.
MXPCON	[INPUT, INTEGER] Dimension of phase constraint arrays as described in ODEINP.
PHSLWR	[INPUT, DOUBLE PRECISION] Desired lower bound on phase duration.
PHSUPR	[INPUT, DOUBLE PRECISION] Desired upper bound on phase duration.
IERLNG	[OUTPUT, INTEGER] Error Flag. IER = 0 Successful Evaluation of Input. IER ≠ 0 Error in Evaluation of Input.

PNTCON: Optimal Control Point Constraint Utility

PURPOSE

PNTCON is a utility routine which can be used to construct point constraints at the beginning or end of a phase. It is typically called from the user supplied input routine ODEINP (p. 426). Specifically, the utility can be used to construct constraints of the form:

$$\psi_L \leq c \cdot \psi[y(t_I^{(k)}), u(t_I^{(k)}), p, t_I^{(k)}] \leq \psi_U$$

or

$$\psi_L \leq c \cdot \psi[y(t_F^{(k)}), u(t_F^{(k)}), p, t_F^{(k)}] \leq \psi_U$$

where $t_F^{(k)}$ is the final time for phase k , and $t_I^{(k)}$ is the initial time for phase k .

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control Software
SOSEST	Sparse Optimal Parameter Estimation
ODEINP	Optimal Control Problem Definition
ODEPTF	Optimal Control Point Function Evaluation

METHOD

Subroutine PNTCON is a utility routine which is called by the user (usually from ODEINP). PNTCON will construct a point constraint at the either end of a phase by proper definition of the SOS input quantities. It can be used in conjunction with other more complicated constraint definitions. The number of terms (NTERM), the number of constraints (NKON), and the number of point functions (NPF(*)) are all *incremented* when PNTCON is called. Consequently, it is important that the user properly initializes these quantities in ODEINP (p. 426). The point functions must be computed in the user supplied routine ODEPTF (p. 450) in the *same order* they are defined by calls to PNTCON. Thus, the first call to PNTCON will define the first point constraint; the second call will define the second constraint, etc.

The Goddard Rocket test problem (p. 596) illustrates the use of this utility routine.

USAGE

```

INTEGER  NTERM,NKON,IPHASE,NPF(2),ITERM(4,MXTERM),MXTERM,MXPCON,IERLNG
DOUBLE PRECISION  CLB(MXPCON),CUB(MXPCON),COEF(MXTERM),PNTLWR,PNTUPR,PNTCOF
CHARACTER  CLBL(0:MXPCON)*80
CHARACTER*8  PNTLB1
CHARACTER*72 PNTLB2

CALL PNTCON(NTERM,NKON,NPF,IPHASE,IPHEND,ITERM,MXTERM,COEF,
$  CLB,CUB,CLBL,MXPCON,PNTLWR,PNTUPR,PNTCOF,PNTLB1,PNTLB2,IERPNT)
```


ARGUMENTS

NTERM	[INPUT/OUTPUT, INTEGER] Running number of terms on the current phase.
NKON	[INPUT/OUTPUT, INTEGER] Running number of constraints on the current phase.
NPF	[INPUT/OUTPUT, INTEGER] Number of point functions on the current phase. When IPHEND=-1, NPF(1) is incremented. When IPHEND=+1, NPF(2) is incremented.
IPHASE	[INPUT, INTEGER] Phase number
IPHEND	[INPUT, INTEGER] Phase end where point constraint is imposed. When IPHEND=-1, constraint is imposed at $t_I^{(k)}$. When IPHEND=+1, constraint is imposed at $t_F^{(k)}$.
ITERM	[OUTPUT, INTEGER, ARRAY] Term description array as described in ODEINP.
MXTERM	[INPUT, INTEGER] Dimension of term arrays as described in ODEINP.
COEF	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of term coefficients as described in ODEINP.
CLB	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of constraint lower bounds as described in ODEINP.
CUB	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of constraint upper bounds as described in ODEINP.
CLBL	[OUTPUT, CHARACTER*80, ARRAY, <i>Default</i> = '␣'] Array containing the symbol and description for constraints.
MXPCON	[INPUT, INTEGER] Dimension of phase constraint arrays as described in ODEINP.
PNTLWR	[INPUT, DOUBLE PRECISION] Desired lower bound ψ_L on point constraint.
PNTUPR	[INPUT, DOUBLE PRECISION] Desired upper bound ψ_U on point constraint.
PNTCOF	[INPUT, DOUBLE PRECISION] Desired coefficient c on point function.
PNTLB1	[INPUT, CHARACTER*8] Primary point constraint label.

PNTLB2	[INPUT, CHARACTER*72] Point constraint description label.
IERPNT	[OUTPUT, INTEGER] Error Flag.
	IER = 0 Successful construction of point constraint.
	IER $\neq$ 0 Error in construction of point constraint.

PTHAUX: Path Constraint with Auxiliary Functions

PURPOSE

PTHAUX is a utility routine which can be used to construct path constraints involving auxiliary functions during a phase. It is typically called from the user supplied input routine ODEINP (p. 426). Specifically, the utility can be used to construct a constraint for phase k defined in the region $t_I^{(k)} \leq t \leq t_F^{(k)}$ of the form:

$$g_L \leq g[y(t), u(t), p, t] \leq g_U,$$

where

$$g[y(t), u(t), p, t] = \alpha^\top v + \beta^\top a[v, t]$$

and

$$v = \begin{bmatrix} y(t) \\ u(t) \\ p \end{bmatrix}.$$

The utility can be used to construct linear combinations of the n_a *auxiliary functions* $a_k(v)$ for $k = 0, \dots, n_a$ where the coefficients β_k are nonzero constants. By convention, a path constraint with a single nonlinear term $a_0(y, u, p, t)$ has no auxiliary functions ($n_a = 0$) and in this case it is simpler to use the utility routine PTHCON (p. 506). The complete constraint definition can include *analytic* terms involving $\alpha^\top v$ where the $(n_y + n_u + n_p)$ vector α is constant. Analytic terms must be constructed in ODEINP immediately after calling the PTHAUX utility, and the construction process is terminated by calling a second utility TRMAUX (p. 516). Observe that each individual path constraint may have a different number of auxiliary functions and analytic terms.

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control Software
SOSEST	Sparse Optimal Parameter Estimation
ODEINP	Optimal Control Problem Definition
ODERHS	Optimal Control Right Hand Side Evaluation

METHOD

Subroutine PTHAUX is a utility routine which is called by the user (usually from ODEINP). The constraint definition must be completed by a call to the routine TRMAUX (p. 516). The PTHAUX/TRMAUX call pairs will construct a path constraint involving auxiliary functions during the phase by proper definition of the SOS input quantities. They can be used in conjunction with other more complicated constraint definitions. The number of terms (NTERM), the number of constraints (NKON), and the number of algebraic functions (NCF(2)), are all *incremented* by the PTHAUX calls. Consequently, it is important that the user properly initializes these quantities in ODEINP (p. 426). The user supplied routine ODERHS (p. 455) must compute the path and auxiliary functions in the *same order* as they are defined by calls to PTHAUX. Thus, the first call to PTHAUX will define the first path constraint and define which auxiliary functions are linked to it; the second call will define the second constraint and associated auxiliary functions, etc.

The Van der Pol Oscillator test problem (p. 597) illustrates the use of this utility routine.

USAGE

```
INTEGER NTERM,NKON,IPHASE,NCF(5),ITERM(4,MXTERM),MXTERM,MXPCON,
$ IERPTH,NUMAUX
DOUBLE PRECISION CLB(MXPCON),CUB(MXPCON),COEF(MXTERM),AUXCOF(NUMAUX),
$ PTHLWR,PTHUPR,PTHCOF
CHARACTER CLBL(0:MXPCON)*80
CHARACTER*8 PTHLB1
CHARACTER*72 PTHLB2

CALL PTHAUX(NTERM,NKON,NCF,IPHASE,ITERM,MXTERM,COEF,
$ CLB,CUB,CLBL,MXPCON,NUMAUX,AUXCOF,PTHLWR,PTHUPR,PTHCOF,
$ PTHLB1,PTHLB2,IERPTH)
```

ARGUMENTS

NTERM	[INPUT/OUTPUT, INTEGER] Running number of terms on the current phase.
NKON	[INPUT/OUTPUT, INTEGER] Running number of constraints on the current phase.
NCF	[INPUT/OUTPUT, INTEGER] Number of path functions on the current phase. NCF(2) is incremented. NCF(4) is <u>not</u> incremented by PTHAUX.
IPHASE	[INPUT, INTEGER] Phase number
ITERM	[OUTPUT, INTEGER, ARRAY] Term description array as described in ODEINP.
MXTERM	[INPUT, INTEGER] Dimension of term arrays as described in ODEINP.
COEF	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of term coefficients as described in ODEINP.
CLB	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of constraint lower bounds as described in ODEINP.
CUB	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of constraint upper bounds as described in ODEINP.

CLBL	[OUTPUT, CHARACTER*80, ARRAY, <i>Default</i> = '␣'] Array containing the symbol and description for constraints.
MXPCON	[INPUT, INTEGER] Dimension of phase constraint arrays as described in ODEINP.
NUMAUX	[INPUT, INTEGER] Number of auxiliary functions linked to constraint NKON. NUMAUX < 0 Set all coefficients to AUXCOF(1). NUMAUX > 0 Use coefficient vector AUXCOF.
AUXCOF	[INPUT, DOUBLE PRECISION, ARRAY] Array of auxiliary function coefficients β_k .
PTHLWR	[INPUT, DOUBLE PRECISION] Desired lower bound g_L on path constraint.
PTHUPR	[INPUT, DOUBLE PRECISION] Desired upper bound g_U on path constraint.
PTHCOF	[INPUT, DOUBLE PRECISION] Desired coefficient β_0 on path function.
PTHLB1	[INPUT, CHARACTER*8] Primary path constraint label.
PTHLB2	[INPUT, CHARACTER*72] Path constraint description label.
IERPTH	[OUTPUT, INTEGER] Error Flag. IER = 0 Successful construction of path constraint. IER $\neq$ 0 Error in construction of path constraint.

PTHCON: Path Constraint Utility

PURPOSE

PTHCON is a utility routine which can be used to construct path constraints during a phase. It is typically called from the user supplied input routine ODEINP (p. 426). Specifically, the utility can be used to construct constraints of the form:

$$g_L \leq c \cdot g[y(t), u(t), p, t] \leq g_U,$$

for phase k defined in the region $t_I^{(k)} \leq t \leq t_F^{(k)}$. This utility should not be used when the problem definition requires the use of auxiliary functions.

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control Software
SOSEST	Sparse Optimal Parameter Estimation
ODEINP	Optimal Control Problem Definition
ODERHS	Optimal Control Right Hand Side Evaluation

METHOD

Subroutine PTHCON is a utility routine that is called by the user (usually from ODEINP). PTHCON will construct a path constraint during the phase by proper definition of the SOS input quantities. It can be used in conjunction with other more complicated constraint definitions. The number of terms (NTERM), the number of constraints (NKON), and the number of algebraic functions (NCF(2)) are all *incremented* when PTHCON is called. Consequently, it is important that the user properly initializes these quantities in ODEINP (p. 426). The user supplied routine ODERHS (p. 455) must compute the path functions in the *same order* they are defined by calls to PTHCON. Thus, the first call to PTHCON will define the first path constraint; the second call will define the second constraint, etc.

WARNING This utility should not be used if any path constraint involves an auxiliary function.

The Goddard Rocket test problem (p. 596) illustrates the use of this utility routine.

USAGE

```

INTEGER  NTERM,NKON,IPHASE,NCF(5),ITERM(4,MXTERM),MXTERM,MXPCON,IERPHTH
DOUBLE PRECISION  CLB(MXPCON),CUB(MXPCON),COEF(MXTERM),PTHLWR,PTHUPR,PTHCOF
CHARACTER  CLBL(0:MXPCON)*80
CHARACTER*8  PTHLB1
CHARACTER*72  PTHLB2

CALL PTHCON(NTERM,NKON,NCF,IPHASE,ITERM,MXTERM,COEF,
$  CLB,CUB,CLBL,MXPCON,PTHLWR,PTHUPR,PTHCOF,PTHLB1,PTHLB2,IERPHTH)

```

ARGUMENTS

NTERM	[INPUT/OUTPUT, INTEGER] Running number of terms on the current phase.
NKON	[INPUT/OUTPUT, INTEGER] Running number of constraints on the current phase.
NCF	[INPUT/OUTPUT, INTEGER] Number of path functions on the current phase. NCF(2) is incremented.
IPHASE	[INPUT, INTEGER] Phase number
ITERM	[OUTPUT, INTEGER, ARRAY] Term description array as described in ODEINP.
MXTERM	[INPUT, INTEGER] Dimension of term arrays as described in ODEINP.
COEF	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of term coefficients as described in ODEINP.
CLB	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of constraint lower bounds as described in ODEINP.
CUB	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of constraint upper bounds as described in ODEINP.
CLBL	[OUTPUT, CHARACTER*80, ARRAY, <i>Default</i> = '␣'] Array containing the symbol and description for constraints.
MXPCON	[INPUT, INTEGER] Dimension of phase constraint arrays as described in ODEINP.
PTHLWR	[INPUT, DOUBLE PRECISION] Desired lower bound g_L on path constraint.
PTHUPR	[INPUT, DOUBLE PRECISION] Desired upper bound g_U on path constraint.
PTHCOF	[INPUT, DOUBLE PRECISION] Desired coefficient c on path function.
PTHLB1	[INPUT, CHARACTER*8] Primary path constraint label.
PTHLB2	[INPUT, CHARACTER*72] Path constraint description label.

IERPTH [OUTPUT, INTEGER]

Error Flag.

$IER = 0$ Successful construction of path constraint.

$IER \neq 0$ Error in construction of path constraint.

RST2SP: Restart File to Spline Array Utility

PURPOSE

RST2SP is a utility routine that can be used to read information from a SOS restart spline file, into the spline storage arrays.

METHOD

Subroutine RST2SP is a utility routine which is called to read information from a previously created SOS restart file. The information is stored into the arrays used to represent the optimal control solution in B-spline format. This is often useful when constructing an initial guess from another solution.

USAGE

```
INTEGER  MAXCS, IERRST
DOUBLE PRECISION      CSTAT(MAXCS)
CHARACTER*60 RSTFIL

CALL RST2SP(RSTFIL, ISFRST, CSTAT, MAXCS, IERRST)
```

ARGUMENTS

RSTFIL	[INPUT, CHARACTER*60] Character variable name of restart file
ISFRST	[INPUT, INTEGER] FORTRAN I/O unit for restart file
CSTAT	[OUTPUT, DOUBLE PRECISION, ARRAY] The real array containing spline coefficients for the optimal control solution.
MAXCS	[INPUT, INTEGER] The length of CSTAT.
IERRST	[OUTPUT, INTEGER] Integer error return flag.
	IERRST = 0 Successful restoration of restart spline information.
	IERRST = -1 Restart file name exceeds 60 characters
	IERRST = -3 Number of CSTAT quantities exceeds MAXCS

SCTFIL: Save an Optimal Control Input File

PURPOSE

SCTFIL is a utility routine which can be used to save a file containing a particular configuration of optional inputs for the SOCX software. The utility is useful when it is desirable to save a particular set of algorithm parameters for a subsequent application of SOSCTL and/or SOSEST.

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control Software
SOSEST	Sparse Optimal Parameter Estimation
CTLFIL	Optimal Control Input From a File
INSOCX	Sparse Optimal Control Input Procedure
INPSOS	Sparse Optimization Suite Input Procedure

METHOD

Subroutine SCTFIL is a utility routine which is called from the users main program to save a particular configuration of algorithm parameters. SCTFIL is intended to be used in conjunction with subroutine CTLFIL (p. 472) for a subsequent call to SOSCTL and/or SOSEST. SCTFIL will write information from the SOCX parameter common to a user specified file.

USAGE

```
INTEGER  INUNIT
CHARACTER*(*) FILNAM

CALL SCTFIL(INUNIT,FILNAM)
```

ARGUMENTS

INUNIT	[INPUT, INTEGER] FORTRAN unit number for opening file.
FILNAM	[INPUT, CHARACTER] Character string with the name of the file containing the INPSOS (p. 401) input.

EXAMPLE Save SOCX Input Parameters on a File

Suppose that SOCX is to be executed using the barrier algorithm with a modified output level. One way to achieve this is as follows:

```
C
      CALL INSOCX('NLPALG=SBRNLP')
      CALL INSOCX('SBRNLP:BARRIER DEFAULT')
      CALL INSOCX('SBRNLP:IOFLAG=20')
C
      CALL SOSCTL(.....
C
```

This collection of algorithm inputs can be saved for subsequent use as follows:

```
C
      CALL INSOCX('NLPALG=SBRNLP')
      CALL INSOCX('SBRNLP:BARRIER DEFAULT')
      CALL INSOCX('SBRNLP:IOFLAG=20')
C
      CALL SCTFIL(35,'/home/socxinput.fil')
C
      CALL SOSCTL(.....
C
```

where the file “/home/socxinput.fil” contains the following information:

```
nlpalg=SBRNLP
SBRNLP:barrier default
SBRNLP:ioflag = 20
```

A subsequent run with the same algorithm settings can then be invoked as follows:

```
C
      CALL CTLFIL(35,'/home/socxinput.fil')
C
      CALL SOSCTL(.....
C
```

SETMRF: Set Mesh Refinement Option

PURPOSE

SETMRF is a utility routine which can be used in the user-supplied input routine to set the mesh refinement option on one or more phases.

METHOD

Subroutine SETMRF is a utility routine which is called from the user supplied input routine ODEINP. A single call to the routine of the form

```
call setmrf(mrfopt,ibeg,iend)
```

will utilize mesh refinement option mrfopt for all phases in the range $[ibeg, iend]$. The following values are valid:

- mrfopt = 0 Minimize Maximum Error (MINMAX)
- mrfopt = 1 Interval Bisection (BISECT)
- mrfopt = 2 Equidistribute Error (EQUIDS)

The default optimal point placement mesh refinement algorithm will be used if:

- The SETMRF utility is not called and
- for all phases outside of the range $[ibeg, iend]$.

When using the same mesh refinement option on all phases the variable MRFMET can be set using either INPSOS or INSOCX.

The following conditions are *invalid*:

1. IBEG < 1
2. IEND > NPHS
3. IBEG > IEND

The SETMRF utility can be used one or more times, so for example

```
call setmrf(0,2,4)
call setmrf(1,6,6)
```

will minimize the maximum error in phases 2, 3, 4 and utilize bisection in phase 6.

ARGUMENTS

MRFOPT [INPUT, INTEGER]
The mesh refinement method option.

IBEG [INPUT, INTEGER]
The first phase number to use refinement option mrfopt.

IEND [INPUT, INTEGER]
The last phase number to use refinement option mrfopt.

STMSEQ: Set Discretization Refinement Sequence

PURPOSE

STMSEQ is a utility routine that can be called from the user-supplied input routine to set the discretization option during the mesh refinement iteration sequence.

METHOD

Subroutine STMSEQ is a utility routine which is called from the user supplied input routine ODEINP. A single call to the routine of the form

```
call stmseq('string')
```

will define the sequence of discretization methods used during the mesh refinement sequence. The character variable *string* is used to define the sequence. There are two valid formats

refinement iteration order sequence which is of the form

```
(dc1), 2; (dc2), 3; (dc3), 5; (dc4), 20
```

discretization error order sequence which is of the form

```
(dc1),-1; (dc2),-3; (dc3),-20
```

The names dc1, dc2, ... define the discretization method using either the alphanumeric abbreviation as in

```
(dc1) ---> (TRP)
```

or the numeric specification as in

```
(dc1) ---> (2,1)
```

The order of the methods must be non-decreasing, that is

```
(TRP), 1; (HSC), 3
```

is correct because TRP is of order 2, and HSC of order 4. However

```
(LA4), 1; (LA2), 5
```

is NOT correct, because LA4 is of order 6, and LA2 of order 2. The numeric values must all have the same algebraic sign. When the values are positive, the sequence defines an iteration order sequence. For the first example above, mesh refinement iterations 1 and 2 will use discretization "dc1" and method "dc2" will be used for iteration number 3. When the values are all negative, the sequence defines an error order sequence. For the second example above, discretization method "dc1" will be used for all iterations resulting in a discretization error $\epsilon > 10^{-1}$. When $10^{-1} > \epsilon > 10^{-3}$ dc2 is used, and when $10^{-3} > \epsilon > 10^{-5}$ dc3 is used.

The following example illustrates the correct usage of the routine for an application with two phases:

```
IF(IPHASE.EQ.1) THEN
  CALL STMSEQ(' (TRP) ,2; (HSS) ,20')
ELSEIF(IPHASE.EQ.2) THEN
  CALL STMSEQ(' (4,2) ,-2; (LA3) ,-4; (LA4) ,-20')
ENDIF
```

In this example on phase 1, the trapezoidal discretization will be used for the first 2 refinement iterations, followed by the separated Simpson method. On phase 2, the Lobatto IIIA 2-stage (method=4,nstg=2) will be used as long as the discretization error is greater than 10^{-2} . Thereafter if the discretization error is in the range $10^{-2} > \epsilon > 10^{-4}$ the LA3 method will be used, and if the discretization error is in the range $10^{-4} > \epsilon > 10^{-20}$ the LA4 method will be used. Notice the final numeric value in either case is meant to be an upper bound. Presumably the maximum number of refinement iterations $\text{MITODE} \leq 20$ and the error tolerance $\text{ODETOL} \geq 10^{-20}$.

USAGE

```
CALL STMSEQ('string')
```

ARGUMENTS

STRING [INPUT, CHARACTER*80]
Character string containing the discretization sequence used during mesh refinement as described in method.

TRMAUX: Terminate Auxiliary Function Construction

PURPOSE

TRMAUX is a utility routine that is used to complete the construction of path constraints involving auxiliary functions during a phase. It is typically called from the user supplied input routine ODEINP (p. 426) after calling the utility PTHAUX (p. 503). Specifically, the utility can be used to complete construction for path constraints of the form:

$$g_L \leq g[y(t), u(t), p, t] \leq g_U,$$

where

$$g[y(t), u(t), p, t] = \alpha^\top v + \beta^\top a[v, t]$$

and

$$v = \begin{bmatrix} y(t) \\ u(t) \\ p \end{bmatrix}.$$

The utility can be used to complete constructions that involve linear combinations of the n_a *auxiliary functions* $a_k(v)$ for $k = 0, \dots, n_a$ where the coefficients β_k are nonzero constants. By convention, a path constraint with a single nonlinear term $a_0(y, u, p, t)$ has no auxiliary functions ($n_a = 0$) and in this case it is simpler to use the utility routine PTHCON (p. 506). The total constraint definition can include *analytic* terms involving $\alpha^\top v$ where the $(n_y + n_u + n_p)$ vector α is constant. Analytic terms must be constructed in ODEINP immediately after calling the PTHAUX utility and before calling TRMAUX.

RELATED SUBPROGRAMS

SOSCTL	Sparse Optimal Control Software
SOSEST	Sparse Optimal Parameter Estimation
ODEINP	Optimal Control Problem Definition
ODERHS	Optimal Control Right Hand Side Evaluation

METHOD

Subroutine TRMAUX is a utility routine which is called by the user (usually from ODEINP) after one or more calls to the utility PTHAUX (p. 503). PTHAUX must first be called to construct the primary path constraint definition and define the auxiliary functions linked to each path constraint. TRMAUX then completes the construction of the path constraint involving auxiliary functions by proper definition of the SOS input quantities. It can be used in conjunction with other more complicated constraint definitions. The number of terms (NTERM), and the number of auxiliary functions (NCF(4)) are both *incremented* when TRMAUX is called. Consequently, it is important that the user properly initializes these quantities in ODEINP (p. 426). The user supplied routine ODERHS (p. 455) must compute the path and auxiliary functions in the *same order* they are defined by calls to PTHAUX.

The Van der Pol Oscillator test problem (p. 597) illustrates the use of this utility routine.

USAGE


```
INTEGER  NTERM, IPHASE, NCF(5), ITERM(4, MXTERM), MXTERM, IERPTH
DOUBLE PRECISION COEF(MXTERM)
```

```
CALL TRMAUX(NTERM, NCF, IPHASE, ITERM, MXTERM, COEF, IERPTH)
```

ARGUMENTS

NTERM	[INPUT/OUTPUT, INTEGER] Running number of terms on the current phase.
NCF	[INPUT/OUTPUT, INTEGER] Number of auxiliary functions on the current phase. NCF(4) is incremented.
IPHASE	[INPUT, INTEGER] Phase number
ITERM	[OUTPUT, INTEGER, ARRAY] Term description array as described in ODEINP.
MXTERM	[INPUT, INTEGER] Dimension of term arrays as described in ODEINP.
COEF	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of term coefficients as described in ODEINP.
IERPTH	[OUTPUT, INTEGER] Error Flag.
	IER = 0 Successful completion of path constraint with auxiliary function.
	IER $\neq$ 0 Error in completion of path constraint with auxiliary function.

WATCH: Variable Monitor Utility

PURPOSE

WATCH is a utility routine that is used to monitor a variable for subsequent retrieval by the fetch utility, FETCH (p. 475). It must be called from the user supplied input routine ODEINP (p. 426).

RELATED SUBPROGRAMS

FETCH Variable Retrieval Utility

METHOD

Subroutine WATCH is a utility routine that is called by the user from ODEINP. WATCH has a single argument, a character string, which is the name of the variable to be retrieved. The string must not exceed 80 characters and is of the following form:

'symbol@phase'

The character substring *'symbol'* defines the name of the variable that is to be monitored. The substring *'symbol'* is:

- case sensitive (i.e. **TIME** is not the same as **time**)
- must exactly match the desired output name which is derived from a user specified label quantity STLBL, PLBL, or CLBL.

The substring *'phase'* is the numeric phase number for the desired variable and must be preceded by the symbol @. A global quantity with no explicit phase number can be specified by a character string of the form *'symbol'* or *'symbol@0'*. Note, since the objective function is unique, it must be specified in the form *'symbol'*.

Referring to the Goddard Rocket Problem (p. 596) the following examples are valid uses of the WATCH utility:

Command	Description
<code>call watch('VEL@2')</code>	Velocity $v(t)$ during phase 2
<code>call watch('T3BEG@3')</code>	Parameter t_3 (beginning) phase 3
<code>call watch('SINGLR_a@2')</code>	Singular arc algebraic constraint $g(t)$ during phase 2
<code>call watch('TIME_M@2')</code>	Time at interval midpoints during phase 2
<code>call watch('L_HALT@1')</code>	Discrete altitude adjoint during phase 1

The FETCH (p. 475) utility which retrieves data, must be preceded by a call to the WATCH utility.

ARGUMENTS

STRING [INPUT, CHARACTER*80]
Monitor variable label

6.5 Optimal Control Error Messages

The SOS library may detect errors as it attempts to produce a solution to the optimal control problem. These errors are usually caused by erroneous input in one or more of the user-supplied subroutines. When an error is detected, an error flag is set and processing is terminated. In addition, an error message is printed containing a more detailed description of the difficulty. This information is written from the subroutine in which the error is detected and is displayed provided the user-specified output level is nonzero.

The basic format for error messages is:

```
***** SOCX ERROR: IER =           number           procedure
>>>>> error description
```

where

<i>number</i>	Number assigned to the specific error
<i>procedure</i>	Description of SOS procedure that detected the error
<i>error description</i>	Explanation of the error

One or more messages of this form may appear either because there are multiple errors in the SOS input and/or the error is detected by a lower level routine and produces an error response from the higher level calling procedure. It is not uncommon for an erroneous input to produce a sequence of messages, which reflect the calling chain. In general, the user should correct the *first* error which is described since this may eliminate one or more messages. For example, suppose the number of grid points has *incorrectly* been set to $\text{NGRID} = -5$, in the user-supplied routine ODEINP. The following information will be displayed:

```
***** SOCX ERROR: IER = -1012      ODEINP Input Check Procedure
>>>>> Phase  1: NGRID = -5 < 2

***** SOCX ERROR: IER = -1282      Optimal Control Transcription Procedure
>>>>> Phase  1: Routine CHKINP set ERROR = -1012
```

This message indicates that the ODEINP input checking procedure set an error flag ($\text{IER} = -1012$), because it detected the erroneous input for NGRID. Since the error checking is done by subroutine CHKINP which is a lower level routine, the calling procedure (in this case routine SPRINP) also sets an error flag ($\text{IER} = -1282$). Obviously correcting the single error ($\text{NGRID} = -5$) will eliminate the error response. Note that the value of the error return flag IER in the calling argument to SOSCTL will correspond to the *last* or highest level error, i.e. in this case $\text{IER} = -1282$.

The following is a list of error messages which may be printed to the optimal control output file. The symbol # will be replaced by a numeric value in the actual output. The symbol _ will be replaced by a character string.

Messages from subroutine CHKINP (ODEINP Input Check Procedure)

$\text{IER} = -1001$ Number of phases = # < 1 or > MAXPHS = #

IER = -1002 Phase #: ODE hold array length = # need #

IER = -1003 Phase #: METHOD = # is invalid.

IER = -1004 Phase #: METHOD = # NSTG = # is invalid.

IER = -1005 Phase #: NCF(1) = # < 0

IER = -1006 Phase #: NCF(2) = # < 0

IER = -1007 Phase #: NCF(3) = # < 0

IER = -1008 Phase #: NPF(1) = # < 0

IER = -1009 Phase #: NPF(2) = # < 0

IER = -1010 Phase #: NPV = # < 0 or > MXPARM = #

IER = -1011 Phase #: NAV = # < 0

IER = -1012 Phase #: NGRID = # < 2

IER = -1013 Phase #: INIT = # < 1 or > 3

IER = -1014 Phase #: MAXMIN = # < -1 or > +1

IER = -1015 Phase #: Parm #: Upper bound # less than lower bound #

IER = -1016 Phase #: More than # parameters loaded.

IER = -1017 Phase #: Parm #, not initialized.

IER = -1018 Phase #: Quadrature equations not allowed with RK integration.

IER = -1019 Phase #: NDE+NAV = # > MXSTAT = #

IER = -1020 Phase #, state #: Upper bound # less than lower bound #

IER = -1021 Phase #: More than # states loaded.

IER = -1022 Phase #: More than # state scales loaded.

IER = -1023 Phase #: term # refers to future phase #

IER = -1024 Phase #: term # END = # must be -1,0,1

IER = -1025 Phase #: term # STATE = # does not match any of # states on phase #

IER = -1026 Phase #: term # STATE = # does not match any of # user functions on phase #

IER = -1027 Phase #: term # TIME on phase # end # is fixed.

IER = -1028 Phase #: term # STATE = # does not match any of # algebraic eqns on phase #

IER = -1029 Phase #: NCON = # > MXPCON

IER = -1030 Phase #: constraint #: Upper bound # less than lower bound #

IER = -1031 Phase #: term # refers to objective, yet MAXMIN is zero.

IER = -1033 Phase #: METHOD = # NGRID must be 2.

IER = -1035 Phase #: Initial/Final lower/upper bound for state # = # inconsistent with interior bound = #

IER = -1036 Phase #: No initial value for state #.

IER = -1037 Phase #: No final value for state #.

IER = -1038 Incorrect parameter in SOS common.

IER = +1001 Phase #: algebraic equation # not referenced

Messages from subroutine CHKSUB (ODERHS and ODEPTF Consistency Check)

IER = -1101 Phase # start. Point function # not loaded by user point function routine.

IER = -1102 Phase # start. More than # point functions loaded by user point function routine.

IER = -1103 Phase # Dependent function # not loaded by user diff. eq routine.

IER = -1104 Phase # More than # dependent functions loaded by user diff. eq routine.

IER = -1107 Phase # end. Point function # not loaded by user point function routine.

IER = -1108 Phase # end. More than # point functions loaded by user point function routine.

Messages from subroutine INPSOS (SOS Parameter Definition Procedure)

IER = -1151 Length of input string = # greater than 80.

IER = -1152 Error in capitalization process.

IER = -1153 Optimization routine — not recognized.
IER = -1154 Input string does not contain equal sign.
IER = -1155 Error in converting input value string to Integer.
IER = -1156 Error in converting input value string to Real.
IER = -1157 Error in converting input value string to Character.
IER = -1158 Unrecognized symbol for optimal control.

Messages from subroutine ODEIN (Initial Guess Procedure)

IER = -1176 Phase #: ODE hold array problem. ODEINQ set ERROR = #
IER = -1177 Real work array too small. Need = #
IER = -1178 Phase #: Initial time not loaded.
IER = -1179 Phase #: Final time not loaded.
IER = -1180 Phase #: More than # states loaded in user initial guess routine.
IER = -1181 Phase #: User set initial guess routine ERROR = #
IER = -1182 Phase #: Spline initial guess failed. SPL2X set ERROR = #
IER = -1183 Phase #: grid point # not monotone. T0,TK,TK+1 = #
IER = +1176 Phase #: INITIAL/FINAL time guess of # reset to fixed value #

Messages from subroutine ODESP (DAE Sparsity Pattern Procedure)

IER = -1201 Phase #: Variable array too small, need = #
IER = -1202 Phase #: Constraint array too small, need = #
IER = -1203 Phase #: Jacobian array too small, need = #
IER = -1204 Phase #: Constants array too small, need = #

Messages from subroutine OPTCTL (Optimal Control Analysis Procedure)

IER = -1226 Input error in call to HSSNLP. IER = #

IER = -1227 Error in grid resolution call to GRDRES. IER = #

IER = -1228 Phase #: Real work array too small. NEED = #

IER = -1229 Phase #: Conversion of NLP variables to spline error = #

IER = -1230 Error in re-gridding process in REGRID. IER = #

IER = -1231 Process error in call to HSSNLP. IER = #

IER = -1232 Integer working storage too small. NEED = #

IER = -1233 Real working storage too small. NEED = #

IER = -1234 Error computing discretization accuracy in LAGDER. Phase = #

IER = -1235 CSTAT array too small. NEED = #

Messages from subroutine SPL2X (Construct Dynamic Variables from B-Splines)

IER = -1251 Phase #: Initial time before start of spline.

IER = -1252 Phase #: Final time after end of spline.

IER = -1253 Phase #: Real working storage too small. Need = #

IER = -1254 Phase #: Spline array contains fewer dependent variables than number of ODE states.

IER = -1255 Phase #: Mismatch in number of discrete parameters.

IER = -1256 Phase #: Spline evaluation error # at TAU = #.

Messages from subroutine SPRINP (Optimal Control Transcription Procedure)

IER = -1276 Errors occurred in previous use of INPSOS.

IER = -1277 Discrete parameter array too small for restart. Need = #

IER = -1278 Spline array too small for restart. Need = #

IER = -1279 Integer storage array too small. Need = #

IER = -1280 Real storage array too small. Need = #

IER = -1281 Phase #: User set input routine ERROR = #

IER = -1282 Phase #: Routine CHKINP set ERROR = #

IER = -1283 Phase #: Routine ODESP set ERROR = #

IER = -1284 Phase #: Routine TERMDF set ERROR = #

IER = -1285 No objective function specified, but gradient nonzeros exist.

IER = -1286 Phase #: Routine ODEIN set ERROR = #

IER = -1287 Routine CHKSUB set ERROR = #

IER = -1288 Routine HJSFDI set ERROR = #

IER = -1289 Restart file unit ISFRST already used for file: __

IER = -1290 Can not find restart file: __

IER = -1291 Scratch file unit ISFINP already used for file: __

IER = -1292 Scratch file unit ISFHES already used for file: __

IER = -1293 Point functions declared, but not used.

IER = -1294 ODETOL too small. Must be $\geq$ #

IER = -1295 Thru phase #: number of degrees of freedom < 0

IER = -1296 Phase #: Routine X2SPL set ERROR = #

IER = -1297 Phase #: Algebraic equations with no controls.

IER = +1276 Thru phase #: number of degrees of freedom = 0. Setting MAXMIN = 0

IER = +1277 Data check run completed.

IER = +1278 Stopping after detailed print at initial point.

IER = +1279 Optimal control problem reset for compatibility with NLP option ALGOPT = F.

IER = +1280 Phase #: Ran out of internal character string storage, need = #.

IER = +1281 Phase #: RHS sparsity failed, continuing with dense RHS.

IER = +1282 Routine HJSFDI set warning ERROR = #.

Messages from subroutine TERMDF (User Constraint Sparsity Pattern Procedure)

IER = -1351 Term #: Time must occur at a phase end.

IER = -1352 Term #: User defined function number # not valid.

IER = -1353 Term #: IEND not equal to -1,0,+1.

IER = -1354 Term #: Invalid state number.

Messages from subroutine X2SPL (Construct B-Splines from Dynamic Variables)

IER = -1376 Number of discrete parameters = # exceeds length of DPARM array.

IER = -1377 Real working storage too small. Need = #

IER = -1378 Integer working storage too small. Need = #

IER = -1379 Spline array length needed = # exceeds length of CSTAT array.

IER = -1380 Spline fit to data failed with ERROR = #

IER = +1375 Phase #: Spline fit to state warning = #

Messages from subroutine SPL2T (Extract grid points from B-spline knots)

IER = -1401 Phase #: Spline length less than 0.

IER = -1402 Phase #: Spline initial/final time wrong.

Messages from subroutine CTLFIL (Read INPSOS input from file)

IER = -1426 Error opening file = _.

IER = -1427 Error reading file after line #.

Messages from subroutine SPRFUN (Evaluate optimal control function for NLP)

IER = -1451 Error writing Hessian to file, no. elements = #.

Messages from subroutine SPHSFN (Compute Sparsity Pattern for User Supplied Functions)

IER = -1476 Phase #: RHS/Endpoint function evaluation error, assuming dense dependency.

IER = -1477 Phase #: Algebraic equations with no controls.

IER = +1476 Phase #: No dependence on any variable for function #.

IER = +1477 Phase #: No control dependency for algebraic equation number #.

IER = +1478 Phase #: No function depends on variable #.

IER = +1479 Phase #: Discontinuity in function # with respect to variable #.

Messages from subroutine ODEIN0 (Compute User Supplied Initial Guess)

IER = -1501 Phase #: ODE hold array problem. ODEINQ set ERROR = #.

IER = -1502 Real work array too small. Need = #.

IER = -1503 Phase #: Initial time not loaded.

IER = -1504 Phase #: Final time not loaded.

IER = -1505 Phase #: More than # states loaded in user initial guess routine.

IER = -1506 Phase #: User set initial guess routine ERROR = #.

Messages from subroutine CKSOCX (Check SOS Input Parameters for Consistency)

IER = -1526 ODETOL too small. Must be $\geq$ #.

IER = -1527 PRTMSD too small. Must be $\geq$ #.

IER = -1528 PRTSFD too small. Must be $\geq$ #.

IER = -1529 TOLJSD less than zero.

IER = -1530 SMLTOL less than zero.

IER = -1531 MITODE greater than #.

IER = -1532 MXPARM less than zero.

IER = -1533 MXPCON less than zero.

IER = -1534 MXSTAT less than one.

IER = -1535 MXTERM less than one.

IER = -1536 IDTSFD less than one or greater than two.

IER = -1537 MORDOC greater than one.

IER = -1538 Invalid input: SPRTHS = #.

IER = -1539 Invalid input: (ITSWCH,MTSWCH,NSSWCH) = (#,#,#).

Messages from subroutine IQ2IR (Construct right hand side to constraint map)

IER = -1551 Quantity # does not depend on any variable.

IER = -1552 Quantity # declared, but not used.

6.6 Optimal Control Iteration Output

There are two mechanisms for producing output from SOCX. The first mechanism is enabled by setting one or more of the output control flags IPNLP, IPGRD, IPODE, and/or IPFSFD. These flags control output from the underlying numerical algorithms, and the next four subsections describe this information. The second mechanism is enabled via user-supplied subroutines. These routines can be called directly by the SOCX library, or the user may produce auxiliary output using the SOCX utility routine AUXOUT.

6.6.1 Sparse NLP Iteration Output

The level of printed output from the sparse nonlinear optimization routine SNLPMN is controlled by the variables IOFLAG and IPUNLP which are set by calls to INSNLP. Note that

```
CALL INSNLP('IOFLAG = 10')
```

is functionally equivalent to the approach

```
CALL INSOCX('IPNLP = 10')
```

or

```
CALL INSOCX('INSNLP:IOFLAG = 10')
```

For clarity in presentation the discussion in this subsection will only refer to IOFLAG. For a more complete understanding of the NLP output the reader should consult Section 2.3.

When IOFLAG = 0 and/or IPUNLP ≤ 0 , no output is printed by SNLPMN. Four different levels of iteration output are provided, when IPUNLP > 0 and IOFLAG > 0. These levels are referred to as Terse, Standard, Interpretive, and Diagnostic and are defined in the following subsections. The *default* level of output is referred to as Standard Output. For a complete discussion of NLP iteration output consult Section 2.3.

6.6.2 Optimal Control Output

When IPGRD = 0 and/or IPUOCP ≤ 0 , no output is printed by SOSCTL. Three different levels of optimal control output are provided, when IPUOCP > 0 and IPGRD > 0. These levels are referred to as Terse, Standard, and Interpretive and are defined in the following subsections. The *default* level of output is IPGRD = 10 and is referred to as Standard Output.

Terse Output

Terse Output is obtained by setting $0 < \text{IPGRD} < 10$, and all values of IPGRD in this range will produce the same result. The first block of output which appears after the SOCX logo, is of the form:

```
ODE = title(1)
GOAL = title(2)
```

where

<i>title(1)</i>	The user defined problem title as input in TITLE(1) in ODEINP.
<i>title(2)</i>	The user defined problem subtitle as input in TITLE(2) in ODEINP.

The beginning of a particular mesh refinement iteration is denoted by a delimiting banner of the form:

```
=====
=====
=====
===== REFINEMENT ITERATION 1 =====
=====
=====
=====
```

The next block of output describes the particulars for the specific transcription used on the current refinement iteration. A typical output summary is:

```
TRANSCRIPTION SUMMARY:
```

Phase	Discretization	Abrv	Method	Stage	Order	Grid Pts
1	Trapezoidal	TRP	2	1	2	10

VARIABLES:

PHASE	NT	NP	NST	GRID	NVAR
1	1	0	5	10	51

CONSTRAINTS:

PHASE	PT	CONT	NRHS	NCON
1	2	4	10	38

The first two lines summarize the discretization case by displaying the values for the ODEINP quantities METHOD, and NSTG. In addition to the order of the discretization scheme a brief description is also printed. The next three lines summarize the characteristics of the NLP variables for the discretization where

PHASE	The phase number,
NT	The number of free times, t_I and/or t_F ,
NP	The number of user-defined parameters p ,
NST	The number of continuous variables $z(t)$,
GRID	The number of grid points,
NVAR	The number of NLP variables.

The next three lines summarize the characteristics of the NLP constraints for the discretization where

PHASE	The phase number,
PT	The number of point constraints, ψ_I and/or ψ_F ,
CONT	The number of continuous constraints, $f(y, u, t)$ and $g(y, u, t)$,
NRHS	The number of right hand side evaluations per NLP function evaluation,
NCON	The number of NLP constraints.

The next major block of output summarizes the NLP solution obtained for the current mesh refinement iteration. A typical block of this output follows:

OPTIMAL CONTROL ANALYSIS GRID 1.0:

NLP SOLUTION:

```

      FUNCTION EVALUATION NUMBER      164
=====
|||||-----|||||
||||| Phase 1 |||||
|||||-----|||||
=====
Parameters =====
-----
      TFINAL
-----
      5.5028814E-01
=====
Dynamic Variables =====
-----
Gridpt  TIME      RANGE      ALTITUDE      RNG-RATE      ALT-RATE      PITCH

```

```

-----
1  0.0000000E+00  0.0000000E+00  0.0000000E+00  0.0000000E+00  0.0000000E+00  4.2375986E+01
2  6.1143127E-02  1.4056177E-01  1.2316216E-01  4.5977945E+00  4.0286510E+00  4.0074594E+01
3  1.2228625E-01  5.7044103E-01  4.8228152E-01  9.4636147E+00  7.7181923E+00  3.4268350E+01
4  1.8342938E-01  1.3101747E+00  1.0480832E+00  1.4733176E+01  1.0789257E+01  2.6198342E+01
5  2.4457251E-01  2.3852234E+00  1.7729303E+00  2.0431813E+01  1.2920592E+01  1.4814027E+01
6  3.0571563E-01  3.8182964E+00  2.5855060E+00  2.6444200E+01  1.3658868E+01  -8.1311992E-01
7  3.6685876E-01  5.6164726E+00  3.3874059E+00  3.2374386E+01  1.2571387E+01  -1.9969831E+01
8  4.2800189E-01  7.7562803E+00  4.0651507E+00  3.7619016E+01  9.5977377E+00  -3.9135724E+01
9  4.8914501E-01  1.0182818E+01  4.5166424E+00  4.1753361E+01  5.1706154E+00  -5.4781445E+01
10 5.5028814E-01  1.2835004E+01  4.6747162E+00  4.5000000E+01  0.0000000E+00  -6.0968986E+01
=====
===== Objective Function =====
=====
OBJ =  5.5028814E-01
=====
===== Differential Constraints =====
=====
Midpnt    TIME_M      RANGE_d      ALTITUDE_d    RNG-RATE_d    ALT-RATE_d
-----
1.5  3.0571563E-02  5.1063320E-13 -5.7254548E-13  2.7562015E-10  2.0804111E-10
2.5  9.1714690E-02  1.4698243E-12 -1.7231355E-12  2.6390140E-10  1.7256126E-10
3.5  1.5285782E-01  2.1502800E-12 -2.6875724E-12  1.3479967E-10  7.1718631E-11
4.5  2.1400094E-01  2.3281377E-12 -2.8316238E-12  1.5442647E-10  5.2069408E-11
5.5  2.7514407E-01  2.2435387E-12 -1.5676349E-12  6.7914174E-10  6.3659196E-11
6.5  3.3628720E-01  2.4167335E-12  4.9737992E-13  7.1059159E-10  -3.4517330E-11
7.5  3.9743032E-01  2.6525449E-12  1.7830182E-12  1.9274082E-10  -7.2640247E-11
8.5  4.5857345E-01  2.1875834E-12  1.6109336E-12  1.6765589E-10  -2.5940379E-10
9.5  5.1971658E-01  8.4732221E-13  5.9618976E-13  3.0056957E-10  -5.4913439E-10
=====
===== Point Constraints =====
=====
FINLBC
-----
6.0000000E+01

```

The first line in this block presents the number of function evaluations required to solve the NLP. The final values for the parameters are printed below the user-supplied labels (in this case TFINAL). The final values for the dynamic variables are printed in columns below the user-supplied labels. For the above example the independent variable is TIME and the values at the grid points are displayed in the first column. The next five columns display the values for the continuous variables below the user-supplied labels. The value of the objective function is displayed followed by the values of the continuous constraints (i.e. the differential constraint defects, followed by the algebraic path constraints). The final values for the point constraint functions ψ are displayed below the corresponding labels (in this case FINLBC).

The final line of output presents a summary of the performance at the end of the refinement iteration as in the example below:

```

=====
GRID  NPT  NFC  NGC  NHC  NFE  NRHS  ERRODE  ERRDAE  CPU TIME
1      10   31   31   23  164  3227  7.1610E-04  7.1610E-04  4.0000E-02

```

The parameters summarized are as follows:

GRID The mesh refinement iteration number,
NPT The number of grid points,
NFC The number of function calls,
NGC The number of gradient calls,

NHC	The number of Hessian calls,
NFE	The number of function evaluations including perturbations,
NRHS	The number of right hand side evaluations,
ERRODE	The relative error in the ordinary differential equations,
ERRDAE	The relative error in the DAE system
CPU	The total CPU time in seconds.

After the final mesh refinement iteration a summary of the SOCX iteration history is presented using the same format as the information presented after each refinement iteration. For this example the summary appears as:

```
=====
```

OPTIMAL CONTROL PROGRAM SUMMARY									
GRID	NPT	NFC	NGC	NHC	NFE	NRHS	ERRODE	ERRDAE	CPU TIME
1	10	31	31	23	164	3227	7.1610E-04	7.1610E-04	4.0000E-02
2	10	5	5	3	104	4382	1.8636E-05	1.8636E-05	1.0000E-02
3	19	3	3	1	48	6522	1.1942E-06	1.1942E-06	3.0000E-02
4	37	3	3	1	48	12930	7.5737E-08	7.5737E-08	6.0000E-02
---	---	---	---	---	---	-----			-----
	37	42	42	28	364	27061			1.4000E-01

```
=====
```

Standard Output

Standard Output is obtained by setting $10 \leq \text{IPGRD} < 20$, and all values of IPGRD in this range will produce the same result. All output produced with the Terse format is also produced with the standard format. This section describes the additional information displayed in the standard output format.

The following summary of SOCX input parameters appears after the output describing the transcription method:

SOCX OPTIONS (also see "FULL OPTIONS")			
SYMBOL	DEFAULT	VALUE	DESCRIPTION
-----	-----	-----	-----
IDTSFD	1	1	Derivative type (1 or 2-sided) for SFD.
IPFAUX	0	0	Print level for AUX print.
IPFSFD	0	0	Print level for SFD print.
IPGRD	10	10	Print level for OCP print.
IPNLP	10	10	Print level for NLP print.
IPODE	0	0	Print level for ODE print.
IPUAUX	0	0	FORTTRAN unit number for auxiliary print.
IPOCP	6	6	FORTTRAN unit number for OCP print (user opened).
IRSTR	0	0	Restart flag.
ISFHES	41	41	FORTTRAN unit number for scratch Hessian file.
ISFINP	42	42	FORTTRAN unit number for scratch input file.
ISFRST	43	43	FORTTRAN unit number for restart file output.
ITSWCH	2	2	Mesh refinement iteration to switch order.
MITODE	5	5	Maximum number of grids used by OCP.
MORDOC	1	1	Maximum order of OCP solution analysis.
MTSWCH	-1	-1	High order method for mesh refinement.
MXPARM	5	5	Maximum number of discrete parameters.
MXPCON	20	20	Maximum number of phase constraints.
MXSTAT	20	20	Maximum number of states.
MXTERM	50	50	Maximum number of terms per phase.
NPTAUX	100	100	Number of AUX print points per phase.
NSSWCH	-1	-1	High order stage number for mesh refinement.
-----	-----	-----	-----
DTAUX	0.	0.	Independent variable increment for AUX print.
OBJTOL	0.100E-04	0.100E-04	Performance index relative tolerance.
ODETOL	0.100E-02	0.100E-06	ODE solution relative tolerance.
PGDTOL	0.100E-02	0.100E-02	Projected gradient relative tolerance.
PRTMSD	0.149E-07	0.149E-07	Minimum perturbation size for sparse finite diff.
PRTSFD	0.100E-04	0.100E-04	Relative perturbation size for sparse finite diff.
RESTOL	0.100E-04	0.100E-04	State resolution tolerance.
SMLTOL	0.149E-07	0.149E-07	ODE small absolute tolerance.
TOLJSD	0.100E-05	0.100E-05	Jacobian row and column tolerance for print.
-----	-----	-----	-----
SPRTHS	SPARSE	SPARSE	Right Hand Side Sparsity Option.
RSTFIL	socx.restart	socx.restart	Restart file name.
-----	-----	-----	-----

The output is self explanatory and essentially produces a summary of all parameters described in the section on INSOCX. Any parameter which appears in this summary list, can be altered using a call to INSOCX.

A second block of output appears after the summary of the NLP variables and constraints. This output has the form:

```

NUMBER OF INDEX SETS      =   11
FE PER JACOBIAN (2-SIDED) =   22
FE PER JACOBIAN/HESSIAN   =   77

```


SPARSITY:

MATRIX	NROWS	NCOLS	NANNZ	NFDNZ	NZERO	%NZ
JACOBIAN	171	174	302	768	1070	3.60
GRADIENT	1	174	1	0	1	0.57
HESSIAN	174	174	0	850	850	5.58

```

-----
| Index Set Analysis:
|
| Phase 1      Function      Nonzeros |
|   Differential      2         4      |
|   Differential      1         2      |
|   Differential      3         1      |
| Phase 2      Function      Nonzeros |
|   Differential      1        11      |
|   Differential      2        11      |
|   Algebraic         1        11      |
|   Differential      3         5      |
|   L. Endpoint       1         3      |
| Phase 3      Function      Nonzeros |
|   Differential      2         5      |
|   Differential      1         3      |
|   Differential      3         2      |
-----

```

and summarizes the sparsity information for the SOCX problem. The first three lines are produced by the sparse finite difference operator and describe information related to this process. The parameters summarized under the sparsity heading are as follows:

MATRIX	The matrix of interest,
NROWS	The number of rows in the matrix,
NCOLS	The number of columns in the matrix,
NANNZ	The number of analytic nonzero elements,
NFDNZ	The number of finite difference nonzero elements,
NZERO	The total number of nonzero elements,
%NZ	The percentage of nonzero elements in the matrix where $\%NZ = 100 * NZERO / (NROWS * NCOLS)$.

The block labeled “Index Set Analysis” summarizes which functions contribute to the determination of the finite difference index sets. For this illustration the right hand side of differential equation number 1 on phase 2 had 11 nonzeros, which then determines the need for 11 index sets.

A third block of output information appears after the solution is displayed. Typical output of this type is

```
=====
```

```

===== Discretization Error Analysis =====
-----
|||||-----|||||
|||||Phase 1|||||
|||||-----|||||
-----
Interval      TIME_M      Differential      Algebraic      Quadrature
-----
    1  5.000000E-01      (1) <3.19E-01>      (-)  0.00E+00      (1)  5.10E-02
   20  1.950000E+01      (1)  9.66E-02      (-)  0.00E+00      (1) <2.84E-01>
    2  1.500000E+00      (1) <1.28E-01>      (-)  0.00E+00      (1)  1.82E-02
   17  1.650000E+01      (4) <8.40E-02>      (1)  1.04E-07      (1)  1.27E-02
   18  1.750000E+01      (4) <8.27E-02>      (-)  0.00E+00      (1)  1.31E-02
    3  2.500000E+00      (1) <7.53E-02>      (1)  4.95E-06      (1)  3.18E-02
   19  1.850000E+01      (3) <7.01E-02>      (-)  0.00E+00      (1)  1.09E-02
   16  1.550000E+01      (4) <6.51E-02>      (1)  4.64E-06      (1)  2.21E-02
    4  3.500000E+00      (1) <4.07E-02>      (1)  1.41E-03      (1)  9.80E-03
   15  1.450000E+01      (1) <2.55E-02>      (1)  2.20E-03      (1)  7.46E-03
-----
Maximum Differential Equation Error      3.1899685E-01
Maximum Algebraic Equation Error          2.2007461E-03
Maximum Quadrature Function Error          2.8410504E-01
-----
Intervals      Dominated By:
-----
    7      Differential Equation.....4
    7      Differential Equation.....1
    3      Differential Equation.....3
    2      Differential Equation.....2
    1      Quadrature Function.....1
    0      Algebraic Equation.....1
-----

```

The first portion of the discretization error analysis output presents information about the error distribution with respect to the independent variable. The ten intervals with the largest error are presented in descending order. In the example, the largest error occurs in interval number 1, with midpoint at 5.000000E-01. In this interval the largest error denoted <3.19E-01> is caused by differential equation number 1. The corresponding error in the algebraic equation was 0.00E+00, and quadrature equation number 1 had an error of 5.10E-02. The dominant error on each interval is enclosed in brackets. The second portion of the output presents information about the error distribution with respect to the dependent functions. First the maximum differential equation error over all intervals is displayed, followed by the corresponding information for the algebraic, and quadrature functions. Finally the distribution of the dependent variable errors by interval is shown. In the example 7 intervals were dominated by the error in differential equation 4.

The final block of output produced by the standard format presents significant information about the discretization error analysis on the phase. The following is typical:

```

ANALYSIS OF GRID  2 ERRORS:
RELATIVE ERROR ESTIMATES
MINIMUM      LOCAL      MAXIMUM
PHASE  1      0.46762E-05  0.10474E-04  0.15271E-04
(MAX ERROR)/(AVG ERROR) =  1.4579
MAXIMUM PREDICTED ERROR =  0.47721E-06
MAXIMUM NUMBER OF INTERVAL SUBDIVISIONS =  1

```

The mesh refinement procedure attempts to control the relative error in the dynamic variables.

Four quantities of interest are displayed. The first is the minimum relative error observed on any interval in the phase. The second is an average over all intervals on the phase and the third quantity is the maximum relative error on any phase. The ratio of maximum relative error to average error which is also shown, is a measure of *equidistribution* and ideally should be near one for a well-chosen grid distribution. If the error on a single interval is large compared to the average error, the mesh refinement procedure will attempt to place more grid points in the intervals with large errors, in order to reduce the maximum error. When the maximum number of interval subdivisions is large (4 or 5) this is an indication of some nonlinear behavior in a particular region, and is often an indication of discontinuous functions.

Interpretive Output

Interpretive Output is obtained by setting $20 \leq \text{IPGRD}$, and all values of IPGRD in this range will produce the same result. All output produced with the Standard format is also produced with the Interpretive format. This section describes the additional information displayed in the Interpretive output format.

The first significant difference from lower levels of output is the appearance of a quasi-mathematical description of the optimal control problem. This represents an interpretation of the problem definition in ODEINP, in terms that are more meaningful to the user. The following output is typical

```

      ODE   =  GODDARD ROCKET PROBLEM
      GOAL  =  SINGULAR ARC--THREE PHASE
-----
PHASE  1          0. < t < T1
-----

PARAMETERS:
          1.00 < T1 < 45.0
-----
          | T1      | TFINAL   | Final Phase 1 Time (sec) |
          -----
          |
STATES:
          Y01(T0) = 0.
          Y02(T0) = 0.
          Y03(T0) = 3.00
-----
          | Y01    | HALT     | Altitude (ft)  |
          | Y02    | VEL      | Velocity (fps)  |
          | Y03    | MASS     | Mass (slugs)   |
          -----

OPTIMAL CONTROL SPARSITY TEMPLATE:

=====
          dF/dY: Differential Equations wrt States
=====
          <--- COLUMN 1

          123
          1  X
          2  XXX
          3
=====

```

```

dF/dP: Differential Equations wrt Parameters

=

<--- COLUMN 1

1
1 X
2 X
3 X
-----
PHASE 2          T0 < t < T1
-----

PARAMETERS:
      T0 FREE
      T1 FREE
-----
| T0      | T2BEG    | Initial Phase 2 Time (sec) |
| T1      | T2END    | Final Phase 2 Time (sec)   |
-----

STATES:
                                     Y01 FREE
                                     Y02 FREE
                                     Y03 FREE
-----
| Y01      | HALT      | Altitude (ft) |
| Y02      | VEL       | Velocity (fps) |
| Y03      | MASS      | Mass (slugs)   |
-----

CONTROLS:
      0. < U01(T0) < 193.          0. < U01(t) < 193.          0. < U01(T1) < 193.
-----
| U01      | THRUST     | Thrust (lbs) |
-----

PATH CONSTRAINTS:
(G01)                                     1.00*C01(t:PHS02) = 0.
-----
| (G01)    | SINGLR     | Singular Arc Path Condition |
-----

BOUNDARY CONDITIONS:
(B01)                                     1.00*t(T1:PHS01) - 1.00*t(T0:PHS02) = 0.
(B02)                                     1.00*Y01(T1:PHS01) - 1.00*Y01(T0:PHS02) = 0.
(B03)                                     1.00*Y02(T1:PHS01) - 1.00*Y02(T0:PHS02) = 0.
(B04)                                     1.00*Y03(T1:PHS01) - 1.00*Y03(T0:PHS02) = 0.
(B05)                                     1.00*E01(T0:PHS02) = 0.
(B06)                                     1.00 < 1.00*t(T1:PHS02) - 1.00*t(T0:PHS02)
-----
| (B01)    | LINK0      | Time Continuity |
| (B02)    | LINK1      | Altitude Continuity |
| (B03)    | LINK2      | Velocity Continuity |
| (B04)    | LINK3      | Mass Continuity |
| (B05)    | ARCBEG     | Singular Arc Initialization |
| (B06)    | PHSLNG     | Phase Duration |
-----

OPTIMAL CONTROL SPARSITY TEMPLATE:

=====

dF/dY: Differential Equations wrt States

===

```

```

<--- COLUMN 1

123
1 X
2 XXX
3

=====

dF/dU: Differential Equations wrt Controls

=

<--- COLUMN 1

1
1
2 X
3 X

=====

dF/dP: Differential Equations wrt Parameters

==

<--- COLUMN 1

12
1 XX
2 XX
3 XX

=====

dC/dY: Continuous Functions wrt States

===

<--- COLUMN 1

123
1 XXX

=====

dC/dU: Continuous Functions wrt Controls

=

<--- COLUMN 1

1
1 X

=====

dE/dY: Endpoint Functions wrt States

===

<--- COLUMN 1

123
1 XXX

```

PHASE 3 T0 < t < T1

PARAMETERS:

T0 FREE
T1 FREE

T0	T3BEG	Initial Phase 3 Time (sec)
T1	T3END	Final Phase 3 Time (sec)

STATES:

Y01 FREE
Y02 FREE
0.100 < Y03(t)

Y03(T1) = 1.00

Y01	HALT	Altitude (ft)
Y02	VEL	Velocity (fps)
Y03	MASS	Mass (slugs)

BOUNDARY CONDITIONS:

(B01) $1.00*t(T1:PHS02) - 1.00*t(T0:PHS03) = 0.$
 (B02) $1.00*Y01(T1:PHS02) - 1.00*Y01(T0:PHS03) = 0.$
 (B03) $1.00*Y02(T1:PHS02) - 1.00*Y02(T0:PHS03) = 0.$
 (B04) $1.00*Y03(T1:PHS02) - 1.00*Y03(T0:PHS03) = 0.$
 (B05) $1.00 < 1.00*t(T1:PHS03) - 1.00*t(T0:PHS03)$

(B01)	LINK0	Time Continuity
(B02)	LINK1	Altitude Continuity
(B03)	LINK2	Velocity Continuity
(B04)	LINK3	Mass Continuity
(B05)	PHSLNG	Phase Duration

OBJECTIVE: MAXIMIZE

1.00*Y01(T1:PHS03)

OBJ	ALTF	Final Altitude (ft)
-----	------	---------------------

OPTIMAL CONTROL SPARSITY TEMPLATE:

```
=====
dF/dY: Differential Equations wrt States

===

<--- COLUMN 1

123
1 X
2 XXX
3

=====

dF/dP: Differential Equations wrt Parameters

==

<--- COLUMN 1

12
1 XX
2 XX
3 XX
```

This particular problem description was produced by the Goddard Rocket Problem (p. 596). The description begins with the phase number and displays the user supplied phase label defined in TITLE(3). The domain of the phase is described using the generic symbols “T0” and “T1” to denote the beginning and end of the phase. Next a summary of the parameters on the phase is given including a table which incorporates the user-supplied mnemonic and description along with the SOCX generic name. The next set of output relates to the state variables which are denoted by the symbols “Y01”, “Y02”, etc. and includes a key describing the user-supplied mnemonic and description of each quantity. Bounds on the state at the beginning of the phase are displayed using the format “Y01(T0)”. A bound imposed at the end of the phase is displayed using the format “Y01(T1)”, and if the bound occurs *during* the phase it is displayed as “Y01(t)”. After printing this information for the state variables a similar approach displays the control variable information using the generic symbol “U” to denote a control. After the control variable summary information, the algebraic (path) constraints are displayed. The generic symbol for an algebraic constraint is “G”, and it typically involves continuous functions denoted by “C” so for example

$$(G01) \qquad 1.00 * C01(t:PHS02) = 0.$$

indicates that the first algebraic path constraint is formed by setting the first continuous function (computed in ODERHS) to zero. Information on the point constraints ψ is displayed next. Since these constraints can be constructed by user specified functions using the ITERM input, the expressions displayed contain more information. Specifically the symbol “E” is used to refer to a user computed function (i.e. a function computed in ODEPTF). These quantities can be evaluated at either end of the phase, so the construct “E01(T1:PHS01)” describes user function number 1 evaluated at the end of phase 1.

After the description of the optimal control problem, the sparsity pattern for the user supplied functions is displayed. This information is computed numerically by SOCX, and should be corroborated by the users knowledge of the problem. The user may override the right hand side sparsity computed by SOCX by setting the flag “SPRTHS=DENSE”.

The next major block of output that appears describes the mapping between the optimal control variables and constraints, and the corresponding sparse NLP variables and constraints. This display is best explained with an example extracted from a multiple stage problem. The NLP variables are displayed first, and for this example the NLP variables on phase 4 are described by:

PHASE		4			

T0 =		216			
T1 =		217			
STATE GRID	1	2	3	4	5
1	218	224	230	236	242
2	219	225	231	237	243
3	220	226	232	238	244
4	221	227	233	239	245

5	222	228	234	240	246
6	223	229	235	241	247

In this example the initial and final time for phase 4 are both NLP variables and there are 5 grid points distributed between t_I and t_F . There are 6 dynamic variables (5 state and 1 control), which is information that cannot be extracted from this display. The equivalent mathematical representation for this mapping is

$$\left. \begin{array}{ccccc} y_1(t_1) & y_1(t_2) & y_1(t_3) & y_1(t_4) & y_1(t_5) \\ y_2(t_1) & y_2(t_2) & y_2(t_3) & y_2(t_4) & y_2(t_5) \\ y_3(t_1) & y_3(t_2) & y_3(t_3) & y_3(t_4) & y_3(t_5) \\ y_4(t_1) & y_4(t_2) & y_4(t_3) & y_4(t_4) & y_4(t_5) \\ y_5(t_1) & y_5(t_2) & y_5(t_3) & y_5(t_4) & y_5(t_5) \\ u_1(t_1) & u_1(t_2) & u_1(t_3) & u_1(t_4) & u_1(t_5) \end{array} \right\} \Rightarrow \begin{array}{ccccc} x_{218} & x_{224} & x_{230} & x_{236} & x_{242} \\ x_{219} & x_{225} & x_{231} & x_{237} & x_{243} \\ x_{220} & x_{226} & x_{232} & x_{238} & x_{244} \\ x_{221} & x_{227} & x_{233} & x_{239} & x_{245} \\ x_{222} & x_{228} & x_{234} & x_{240} & x_{246} \\ x_{223} & x_{229} & x_{235} & x_{241} & x_{247} \end{array}$$

where the dynamic variables $z = (y, u)$ are equivalent to the NLP variables x . Continuing with this example the NLP constraint mapping is also presented, and would appear as

PHASE		4				

EQN GRID		1	2	3	4	5
1		210	216	222	228	234
2		211	217	223	229	
3		212	218	224	230	
4		213	219	225	231	
5		214	220	226	232	
6		215	221	227	233	
POINT CONSTR		235	236	237	238	239 240

This particular example which has one algebraic equation and 5 differential equations serves to illustrate another point. The algebraic equation is evaluated at all of the grid points, whereas the defect constraints are evaluated at the interval *midpoints*—observe the “missing” column in the above display. Thus this output displays the mapping from optimal control quantities to NLP constraints, i.e.

$$\left. \begin{array}{ccccc} g_1(t_1) & g_1(t_2) & g_1(t_3) & g_1(t_4) & g_1(t_5) \\ \zeta_1(\bar{t}_1) & \zeta_1(\bar{t}_1) & \zeta_1(\bar{t}_3) & \zeta_1(\bar{t}_4) & \\ \zeta_2(\bar{t}_1) & \zeta_2(\bar{t}_2) & \zeta_2(\bar{t}_3) & \zeta_2(\bar{t}_4) & \\ \zeta_3(\bar{t}_1) & \zeta_3(\bar{t}_2) & \zeta_3(\bar{t}_3) & \zeta_3(\bar{t}_4) & \\ \zeta_4(\bar{t}_1) & \zeta_4(\bar{t}_2) & \zeta_4(\bar{t}_3) & \zeta_4(\bar{t}_4) & \\ \zeta_5(\bar{t}_1) & \zeta_5(\bar{t}_2) & \zeta_5(\bar{t}_3) & \zeta_5(\bar{t}_4) & \end{array} \right\} \Rightarrow \begin{array}{ccccc} c_{210} & c_{216} & c_{222} & c_{228} & c_{234} \\ c_{211} & c_{217} & c_{223} & c_{229} & \\ c_{212} & c_{218} & c_{224} & c_{230} & \\ c_{213} & c_{219} & c_{225} & c_{231} & \\ c_{214} & c_{220} & c_{226} & c_{232} & \\ c_{215} & c_{221} & c_{227} & c_{233} & \end{array}$$

where $\bar{t}_k = \frac{1}{2}(t_{k+1} + t_k)$ denotes the interval midpoint. The output mapping from optimal control quantities to the NLP variables and constraints, is complete when a trapezoidal discretization method is used. However for Runge-Kutta and Hermite-Simpson (Compressed) discretizations, the

NLP variables include values for the control variables at interval midpoints. The NLP variable numbers for the control at the midpoints *are not shown*. Similarly, the algebraic constraints are evaluated at interval midpoints for these discretizations and the corresponding constraint numbers are not shown. Currently, this information must be inferred from the available data. On the other hand when a Separated Hermite-Simpson discretization is used (“METHOD=2”, “NSTG=2”), the information is displayed as though an additional “pseudo-grid” point is present at each interval midpoint. Thus for a problem with 10 grid points, additional variables are introduced at each interval midpoint, and this is reported as though the problem had 19 grid points. A similar approach is used to summarize the additional constraints introduced by the separated Simpson discretization.

The control to NLP mapping can often be useful for diagnosing scaling problems. When the non-linear programming algorithm constructs the Jacobian matrix the largest elements are computed. This information is displayed using output such as this:

```

.....LARGEST JACOBIAN ELEMENTS

      dG1[t(1)=+1.50000E+01:PHS2]
      ----- =  +193.04400
      dU1[t(1)=+1.50000E+01:PHS2]

      dG1[t(6.5)=+2.41667E+01:PHS2]
      ----- =  +193.04400
      dU1[t(6.5)=+2.41667E+01:PHS2]
```

In this illustration, the largest NLP Jacobian element has the value +193.04400. It corresponds to the partial derivative of the first path constraint “G1” evaluated at the gridpoint number 1 in phase number 2, which is located at the time “t(1)=+1.50000E+01”. The partial derivative is computed with respect to control variable number 1, also at gridpoint 1 in phase 2, which is denoted by “dU1[t(1)=+1.50000E+01:PHS2]”. For quantities evaluated at the midpoint of an interval, we use the notation “t(6.5)=+2.41667E+01:PHS2”.

After the NLP solution is obtained an estimate of the active constraints is available. This information can be displayed (e.g by using the input CALL INSOCX(‘SOCOUT=N1’)) and is controlled by the input variable KEYDPL described in the documentation for INSOCX (p. 403). A typical summary produced using the default display key KEYDPL=.lueiLUE for the Goddard Rocket Problem (p. 596) is shown below.

Parameters

P

12

..

GRID	T	Y	U	G	D
PT		123	1	1	123
1	1.37501E+01	 e	. e	eee	

2	1.46656E+01		...		.		e		
					.		e		eee
3	1.55811E+01		...		.		e		
					.		e		eee
4	1.64965E+01		...		.		e		
					.		e		eee
5	1.74120E+01		...		.		e		
					.		e		eee
6	1.83274E+01		...		.		e		
					.		e		eee
7	1.92429E+01		...		.		e		
					.		e		eee
8	2.01584E+01		...		.		e		
					.		e		eee
9	2.10738E+01		...		.		e		
					.		e		eee
10	2.19893E+01		...		.		e		

Boundary Conditions

B

123456

eeeeee.

For this example which illustrates the solution on phase 2, there were two parameters and both were inactive (unconstrained) at the solution. There were 10 grid points during the phase with grid times running from 1.37501E+01 to 2.19893E+01. There were 3 state variables (Y1, Y2, Y3), one control variable (U1), one path constraint (G1), and 3 differential equations (D1, D2, D3). The state variables were free throughout the phase as indicated by the symbols ... at all grid points. The control variables were also free at the gridpoints and midpoints. The path constraint was satisfied as an equality at gridpoints and midpoints. Finally, the differential equations were satisfied as equalities at the interval midpoints. Of the six boundary conditions imposed on the phase, the first five were satisfied as equalities, and the last was free.

The third major block of output appears after a solution is obtained for the current discretization, i.e. at the end of the mesh refinement iteration. At the NLP solution the NLP constraint values are within tolerance of zero, however if the constraints are evaluated between the current grid points they will be violated. This information is used to construct a relative error for each interval in the current mesh. This relative error is displayed under the heading **RELATIVE EQUATION ERROR, PHASE 1**, and includes the number of the specific differential or algebraic equation with the largest error. After displaying the relative errors for each interval, the minimum, average, and maximum errors for the phase are tabulated.

The NLP Lagrange multipliers can be used to construct discrete approximations to the adjoint variables. Thus for the second phase of the Goddard Rocket Problem (p. 596) the following output is produced:

=====

Phase 2

===== Dynamic Constraints =====						
Gridpt Midpnt	TIME TIME_M	m_SINGLR M_SINGLR	L_HALT	L_VEL	L_MASS	
1	1.3716656E+01	7.0755904E-03				
1.5	1.4179380E+01	7.4674251E-03	1.1832834E+00	1.7470042E+01	2.1183365E+04	
2	1.4642105E+01	7.2119356E-03				
2.5	1.5104829E+01	7.6097309E-03	1.1688041E+00	1.7252185E+01	2.1583855E+04	
3	1.5567553E+01	7.3495526E-03				
3.5	1.6030277E+01	7.7547531E-03	1.1545044E+00	1.7037029E+01	2.1991888E+04	
4	1.6493002E+01	7.4898021E-03				
4.5	1.6955726E+01	7.9025437E-03	1.1403820E+00	1.6824542E+01	2.2407605E+04	
5	1.7418450E+01	7.6327347E-03				
5.5	1.7881175E+01	8.0531558E-03	1.1264348E+00	1.6614690E+01	2.2831151E+04	
6	1.8343899E+01	7.7784022E-03				
6.5	1.8806623E+01	8.2066437E-03	1.1126605E+00	1.6407440E+01	2.3262672E+04	
7	1.9269348E+01	7.9268575E-03				
7.5	1.9732072E+01	8.3630631E-03	1.0990571E+00	1.6202761E+01	2.3702317E+04	
8	2.0194796E+01	8.0781553E-03				
8.5	2.0657521E+01	8.5224717E-03	1.0856223E+00	1.6000620E+01	2.4150238E+04	
9	2.1120245E+01	8.2323450E-03				
9.5	2.1582969E+01	8.6849147E-03	1.0723542E+00	1.5800986E+01	2.4606590E+04	
10	2.2045694E+01	8.3910233E-03				
===== Point Constraints =====						
p_LINK0		p_LINK1	p_LINK2	p_LINK3	p_ARCBEG	p_PHS LNG
-5.3237356E-12		-1.1985276E+00	-1.7022762E+01	-2.1128284E+04	-4.4347524E+00	0.0000000E+00
===== Dynamic Variables =====						
Gridpt Midpnt	TIME TIME_M	n_HALT N_THRUST	n_VEL	n_MASS	n_THRUST	
1	1.3716656E+01	0.0000000E+00	0.0000000E+00	0.0000000E+00	0.0000000E+00	
1.5	1.4179380E+01	0.0000000E+00				
2	1.4642105E+01	0.0000000E+00	0.0000000E+00	0.0000000E+00	0.0000000E+00	
2.5	1.5104829E+01	0.0000000E+00				
3	1.5567553E+01	0.0000000E+00	0.0000000E+00	0.0000000E+00	0.0000000E+00	
3.5	1.6030277E+01	0.0000000E+00				
4	1.6493002E+01	0.0000000E+00	0.0000000E+00	0.0000000E+00	0.0000000E+00	
4.5	1.6955726E+01	0.0000000E+00				
5	1.7418450E+01	0.0000000E+00	0.0000000E+00	0.0000000E+00	0.0000000E+00	
5.5	1.7881175E+01	0.0000000E+00				
6	1.8343899E+01	0.0000000E+00	0.0000000E+00	0.0000000E+00	0.0000000E+00	
6.5	1.8806623E+01	0.0000000E+00				
7	1.9269348E+01	0.0000000E+00	0.0000000E+00	0.0000000E+00	0.0000000E+00	
7.5	1.9732072E+01	0.0000000E+00				
8	2.0194796E+01	0.0000000E+00	0.0000000E+00	0.0000000E+00	0.0000000E+00	
8.5	2.0657521E+01	0.0000000E+00				
9	2.1120245E+01	0.0000000E+00	0.0000000E+00	0.0000000E+00	0.0000000E+00	
9.5	2.1582969E+01	0.0000000E+00				
10	2.2045694E+01	0.0000000E+00	0.0000000E+00	0.0000000E+00	0.0000000E+00	

Estimates are computed at gridpoints and midpoints. A separate print key is displayed which summarizes the information. In this example, the equality constraint labeled “SINGLR” is imposed at gridpoints and midpoints. SOCX constructs the label “m_SINGLR” to denote the adjoint variable associated with this path constraint at a gridpoint. The prefix for the quantity evaluated at a midpoint is capitalized in “M_SINGLR”. By convention a prefix or suffix corresponding to a quantity at a gridpoint is lower case, and upper case for midpoints. Gridpoints are numbered consecutively from 1 to M . The midpoint of interval k , i.e. the point $(t_k + t_{k+1})/2$ is displayed as

point $k + .5$. In the example the midpoint of the second interval denoted 2.5 occurs at the time 1.5104829E+01. In general the adjoint variables associated with a constraint labeled “xxx” will be

Notation	Description
<code>l_xxx</code>	Differential constraint adjoints $\lambda(t)$
<code>m_xxx</code>	Path constraint adjoints $\mu(t)$
<code>n_xxx</code>	Dynamic variable adjoints $\nu(t)$
<code>p_xxx</code>	Boundary (point) constraint adjoints σ

Thus for this example adjoint estimates are computed at the gridpoints for the path constraint, and at the midpoint for both the path constraint and differential equations. Boundary condition adjoints are shown at the end of the display. Adjoint variable output is controlled by setting either `CALL INSOCX('SOCOUT=01')` and/or `CALL INSOCX('SOCOUT=P1')`. The error in the Lagrangian can be used to construct a prediction for the minimum objective function, which is displayed as `OBJPRED` in comparison to the current computed value for the objective which is labeled `OBJ`. The estimated error in the objective function is $ERROBJ = |OBJ - OBJPRED|$.

After the discretization error is computed and displayed the mesh refinement procedure can utilize this information to compute a new distribution of grid points prior to the next iteration. The final block of output displays both the old and new grid distributions. The display format for the new grid is designed to emphasize that new grid points always subdivide the old intervals (and hence are interleaved between the old points). The maximum number of subdivisions of any interval on the phase is displayed as well. Typically when this number is large, it indicates that points are clustered in a specific region in response to a large discretization error.

6.6.3 Differential Equation Output

The output flag `IPODE` controls the level of print obtained from the ordinary differential equations, that is the *trajectory print*. Output produced by setting `IPODE` is conditional, and depends on the values of the flags `IPGRD` and/or `IPNLP`. It is designed to control the level of output produced by a *function evaluation* and consequently will only appear during a function evaluation initiated by a call from the NLP or the optimal control interface. The *default* level of output is `IPODE = 0` which produces no trajectory level print.

The most common use of this output is in conjunction with the NLP iteration output flag `IOFLAG` (or `IPNLP`). The *frequency* of the output (i.e. the number of trajectories) is controlled by the value of `IOFLAG`. When `IOFLAG = 10`, the trajectories will be displayed at significant points in the optimization iterative process. The trajectory display will appear more often when `IOFLAG = 20`. The *amount* of information displayed in an individual trajectory print is controlled by the flag `IPODE`. There are four incremental amounts of information available described as follows:

$10 \leq IPODE < 20$	An abbreviated print of the trajectory will be produced. This includes the values of the dynamic variables z at the beginning and end of the phase, as well as the maximum error in the constraints evaluated on all intervals in the phase.
$20 \leq IPODE < 30$	A normal print of the trajectory consisting of the state and control variables at all grid points, and the values of all defect constraints on each interval in the phase.

$30 \leq \text{IPODE} < 40$	The normal trajectory print obtained with $\text{IPODE} = 20$, is augmented by the gradient vector, Jacobian matrix, and Hessian matrix displayed in sparse format.
$40 \leq \text{IPODE}$	All of the output obtained with $\text{IPODE} = 30$, is augmented by extensive diagnostic information about each variable and constraint at the initial point. After information is displayed, SOCCX terminates with an error condition.

6.6.4 Sparse Finite Difference Output

The output flag **IPFSFD** controls the level of print obtained from the sparse finite difference subroutines. This flag is identical to the quantity **MSGVLV** described in the documentation for subroutines **HDSFDH**, **HDSFDJ**, and **HSSFDI** which is fully described in Section 3.2. The *default* level of output is $\text{IPFSFD} = 0$ which produces no print from these routines.

Chapter 7

Usage Examples

7.1 Linear Tangent Steering Example

7.1.1 Problem Definition

For the first example problem the objective is to minimize the time required for a vehicle to move from a fixed initial state to a specified terminal position. This problem can be described by four state variables which represent the horizontal and vertical position and the horizontal and vertical velocities respectively. The direction of the vehicle thrust is defined by one control variable. For this simple example it is known that the optimal solution for the control is

$$\tan u = \tan u_0 - ct$$

which is referred to as *linear tangent steering*. The dynamics of the system are described by the state equations

$$\begin{aligned} \dot{y}_1 &= y_3 \\ \dot{y}_2 &= y_4 \\ \dot{y}_3 &= a \cos u \\ \dot{y}_4 &= a \sin u \end{aligned}$$

with $a = 100$. The control variable is restricted by upper and lower bounds

$$-\frac{\pi}{2} \leq u(t) \leq \frac{\pi}{2},$$

and the boundary conditions are

$$\begin{aligned} y_1(0) &= 0 & y_2(t_F) &= 5 \\ y_2(0) &= 0 & y_3(t_F) &= 45 \\ y_3(0) &= 0 & y_4(t_F) &= 0. \\ y_4(0) &= 0 \end{aligned}$$

The independent variable t is defined in the range $0 \leq t \leq t_F$ (which is a single phase) and the goal is to minimize the value of t_F .

7.1.2 SOSCTL Implementation

The following code illustrates the use of SOS to solve the linear tangent steering example. Default values have been used whenever possible for this double precision version of the software.

SAMPLE PROGRAM (lntsex.cd1)¹

```
PROGRAM SOCXEX
C
C      Sample main program for sparse optimal control software.
C      This program can be used to solve a single optimal control problem
C
C      *****
C      IMPLICIT DOUBLE PRECISION  (A-H,O-Z)
C
C      PARAMETER  (MXC=40000)
C
C      DIMENSION  CSTAT(MXC)
C
C      EXTERNAL  LNTSIN, LNTSDE, DUMYIG, DUMYPF, DUMYPR
C
C      *****
C
C      CALL SOS WITH THE APPROPRIATE ARGUMENTS FOR THE
C      PROBLEM
C
C      ---  LINEAR TANGENT STEERING PROBLEM
C
C      CALL SOSCTL(LNTSIN,DUMYIG,LNTSDE,DUMYPF,DUMYPR,
&                CSTAT,MXC,NEEDED,IER)
C
C      *****
C
C      STOP
C      END
C
C      SUBROUTINE LNTSIN(IPHASE,NPHS,METHOD,NSTG,NCF,NPF,NPV,NAV,NGRID,
&                      INIT,MAXMIN,MXPARM,PO,PLB,PUB,PLBL,
&                      MXSTAT,YO,Y1,YLB,YUB,STSKL,STLBL,MXPCON,CLB,CUB,
&                      CLBL,MXTERM,COEF,ITERM,TITLE,IER)
C
C      LINEAR TANGENT STEERING EXAMPLE PROBLEM
C
C      REF:  This is Problem #9 in
C           "APPLIED OPTIMAL CONTROL", Arthur E. Bryson, Jr.,
C           Yu-Chi Ho, Hemisphere Publishing Corp., John Wiley
```

¹sosexample.tar


```

C          & Sons, 1975, p.82-83.
C
C          *****
C
C          IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C          Arguments:
C
C          INTEGER      IPHASE,NPHS,METHOD,NSTG(2),NCF(6),NPF(3),NPV,NAV,NGRID,
&          INIT,MAXMIN,MXPARM,MXSTAT,MXPCON,MXTERM,
&          ITERM(4,MXTERM),IER
C          DIMENSION    PO(MXPARM),PLB(MXPARM),PUB(MXPARM),YO(0:MXSTAT),
&          Y1(0:MXSTAT),YLB(-1:1,0:MXSTAT),YUB(-1:1,0:MXSTAT),
&          STSKL(0:MXSTAT+MXPARM,2),CLB(MXPCON),CUB(MXPCON),
&          COEF(MXTERM)
C          CHARACTER(LEN=60)  TITLE(3)
C          CHARACTER(LEN=80)  PLBL(MXPARM),STLBL(0:MXSTAT),CLBL(0:MXPCON)
C
C          PARAMETER (ATHRUS=100.DO)
C
C          *****
C
C          ----PROBLEM TITLES AND LABELS (OPTIONAL)
C
C          TITLE(1) = 'Linear Tangent Steering ascent'
C          TITLE(2) = 'Solve by controls'
C          STLBL(0) = 'TIME      Time'
C          STLBL(1) = 'RANGE      Range'
C          STLBL(2) = 'ALTITUDE Altitude'
C          STLBL(3) = 'RNG-RATE Range Rate'
C          STLBL(4) = 'ALT-RATE Altitude Rate'
C          STLBL(5) = 'PITCH      Pitch angle (deg)'
C          PLBL(1) = 'TFINAL      Final time'
C
C          ----NUMBER OF DIFFERENTIAL EQUATIONS (STATES)
C
C          NDE = 4
C          NCF(1) = NDE
C
C          ----NUMBER OF ALGEBRAIC VARIABLES (CONTROLS)
C
C          NAV = 1
C
C          ----GUESS FOR INITIAL TIME AND FIXED INITIAL CONDITION
C
C          YO(0) = 0.DO
C          YLB(-1,0) = YO(0)
C          YUB(-1,0) = YO(0)

```

```
C
C      ----GUESS FOR FINAL TIME AND LOWER BOUND
C
      T1 = 1.DO
      Y1(0) = T1
      YLB(1,0) = 0.DO
C
C      ----DEFINE INITIAL CONDITIONS FOR STATE VARIABLES, Y.
C
      Y(1) = RANGE.
      Y(2) = ALTITUDE.
      Y(3) = HORIZONTAL VELOCITY.
      Y(4) = VERTICAL VELOCITY.
C
      CR2D = HDMCON(16)
      CPI2 = HDMCON(12)/2.DO
      Y0(1) = 0.DO
      Y0(2) = 0.DO
      Y0(3) = 0.DO
      Y0(4) = 0.DO
C
C      ----FIX THE INITIAL STATES
C
      YLB(-1,1:4) = Y0(1:4)
      YUB(-1,1:4) = Y0(1:4)
C
C      ----DEFINE FINAL CONDITIONS FOR STATE VARIABLES, Y.
C
      Y1(1) = 12.DO
      Y1(2) = 5.DO
      Y1(3) = 45.DO
      Y1(4) = 0.DO
C
C      ----DEFINE INITIAL AND FINAL CONTROL ANGLES.
C
      Y0(5) = 1.DO
      Y1(5) = -1.DO
C
C      ----FIX THE FINAL VALUES FOR STATES 2-4.
C
      YLB(1,2:4) = Y1(2:4)
      YUB(1,2:4) = Y1(2:4)
C
C      ----BOUND CONTROL MAGNITUDE ON PHASE BY 90 DEG.
C
      YLB(-1:1,NDE+NAV) = -CPI2
      YUB(-1:1,NDE+NAV) =  CPI2
C
```

```

C      ----DEFINE FINAL TIME AS THE OBJECTIVE TO BE MINIMIZED
C
C      ITERM(1,1) = 0
C      ITERM(2,1) = 1
C      ITERM(3,1) = 1
C      ITERM(4,1) = 0
C      COEF(1) = 1.D0
C
C      MAXMIN = -1
C
C      ----LOAD SCALE WEIGHTS FOR OUTPUT OF CONTROL IN DEGREES
C
C      STSKL(NDE+NAV,1) = CR2D
C
C      RETURN
C      END

SUBROUTINE LNTSDE(IPHASE,T,Y,NY,P,NP,F,NF,IFERR)
C
C      COMPUTES THE RIGHT HAND SIDES OF THE LINEAR TANGENT
C      STEERING SYSTEM OF DIFFERENTIAL EQUATIONS.
C
C      *****
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      ARGUMENTS:
C
C      INTEGER      IPHASE,NY,NP,NF,IFERR
C      DIMENSION    Y(NY),P(NP),F(NF)
C
C      PARAMETER (ATHRUS=100.D0)
C
C      *****
C
C      ----SET FUNCTION ERROR FLAG.
C
C      IFERR = 0
C
C      ----COMPUTE FUNCTIONS OF CONTROL.
C
C      COSU = COS(Y(5))
C      SINU = SIN(Y(5))
C
C      ----COMPUTE STATE EQUATIONS.
C
C      F(1) = Y(3)
C      F(2) = Y(4)

```

```
***** SOCX WARNING: IER = 1478      Compute Sparsity Pattern for User Supplied Functions
>>>> Phase 1: No function depends on state 2
```

SOSOSOSOSOSOSOSOSOSOSO	SOSOSOSOSOSOSOSOSOSOSO	SOSOSOSOSOSOSOSOSOSOSO
SOSOSOSOSOSOSOSOSOSOSO	SOSOSOSOSOSOSOSOSOSOSO	SOSOSOSOSOSOSOSOSOSOSO
SOSOSOSOSOSOSOSOSOSOSO	SOSOSOSOSOSOSOSOSOSOSO	SOSOSOSOSOSOSOSOSOSOSO
SOSOS	SOSOS	OSOSO
SOSOS	SOSOS	OSOSO
SOSOS	SOSOS	OSOSO
SOSOS	SOSOS	OSOSO
SOSOS	SOSOS	OSOSO
SOSOSOSOSOSOSOSOSOSOSO	SOSOS	OSOSO
SOSOSOSOSOSOSOSOSOSOSO	SOSOS	OSOSO
SOSOSOSOSOSOSOSOSOSOSO	SOSOS	OSOSO
OSOSO	SOSOS	OSOSO
OSOSO	SOSOS	OSOSO
OSOSO	SOSOS	OSOSO
OSOSO	SOSOS	OSOSO
OSOSO	SOSOS	OSOSO
OSOSO	SOSOS	OSOSO
SOSOSOSOSOSOSOSOSOSOSO	SOSOSOSOSOSOSOSOSOSOSO	SOSOSOSOSOSOSOSOSOSOSO
SOSOSOSOSOSOSOSOSOSOSO	SOSOSOSOSOSOSOSOSOSOSO	SOSOSOSOSOSOSOSOSOSOSO
SOSOSOSOSOSOSOSOSOSOSO	SOSOSOSOSOSOSOSOSOSOSO	SOSOSOSOSOSOSOSOSOSOSO

Sparse Optimization Suite
Applied Mathematical Analysis, LLC
SOS Version 2018.12

```
ODE    =    Linear Tangent Steering ascent
GOAL   =    Solve by controls
```

```
=====
=====
=====
===== REFINEMENT ITERATION 1 =====
=====
=====
=====
```

TRANSCRIPTION SUMMARY:

Phase	Discretization	Abrv	Method	Stage	Order	Grid Pts
1	Trapezoidal INIT=(1, 0): Linear Between Y0 and Y1	TRP	2	1	2	10

SOCX OPTIONS (also see "FULL OPTIONS")			
SYMBOL	DEFAULT	VALUE	DESCRIPTION
-----	-----	-----	-----
IDSCPH	0	0	Discontinuity Diagnostic ---- phase number.
IDSCND	0	0	Discontinuity Diagnostic ---- phase end.
IDSCVR	0	0	Discontinuity Diagnostic ---- variable number.
IDSCFN	0	0	Discontinuity Diagnostic ---- function number.
IDTSFD	-1	-1	Derivative type (1 or 2-sided) for SFD.
IPFAUX	0	0	Print level for AUX print.
IPFSFD	0	0	Print level for SFD print.
IPRSFD	1	1	Perturbation Size Option
IPGRD	10	10	Print level for OCP print.
IPNLP	10	10	Print level for NLP print.
IPODE	0	0	Print level for ODE print.
IPUAUX	0	0	FORTRAN unit number for auxiliary print.
IPUOCP	6	6	FORTRAN unit number for OCP print (user opened).
IRSTRT	0	0	Restart flag.
ISCALE	0	0	Automatic Scaling Option.
ISFHES	41	41	FORTRAN unit number for scratch Hessian file.
ISFINP	42	42	FORTRAN unit number for scratch input file.
ISFRST	43	43	FORTRAN unit number for restart file output.
ISFSCL	44	44	FORTRAN unit number for scale weight input.
ITSWCH	2	2	Mesh refinement iteration to switch order.
M5DTYP	0	0	Method 5 Derivative Type.
MITODE	5	5	Maximum number of grids used by OCP.
MTSWCH	-1	-1	High order method for mesh refinement.
NSSWCH	-1	-1	High order stage number for mesh refinement.
NPTAUX	100	100	Number of AUX print points per phase.
NIWOPI	10000000	10000000	Initial integer storage alloc for optimization.
NRWOPI	20000000	20000000	Initial real storage alloc for optimization.
-----	-----	-----	-----
DTAUX	0.000E+00	0.000E+00	Independent variable increment for AUX print.
OBJCTL	1.000E-05	1.000E-05	Performance index relative tolerance.
ODETOL	1.000E-03	1.000E-03	Differential-algebraic equation relative tolerance
PRTMSD	1.490E-08	1.490E-08	Minimum perturbation size for sparse finite diff.
PRTMXD	1.000E-03	1.000E-03	Maximum perturbation size for sparse finite diff.
PRTSFD	1.000E-05	1.000E-05	Relative perturbation size for sparse finite diff.
SMLTOL	1.490E-11	1.490E-11	ODE small absolute tolerance.
TOLJSD	1.000E-06	1.000E-06	Jacobian row and column tolerance for print.
TOLMSA	1.490E-08	1.490E-08	Absolute integration tolerance (Method 5).
TOLM5R	1.490E-08	1.490E-08	Relative integration tolerance (Method 5).
-----	-----	-----	-----
SOCOUT			SOCX Output Control Menu
	A0	A0	Mathematical Optimal Control Interpretation.
	B0	B0	Right Hand Side Sparsity Pattern.
	C1	C1	Transcription Summary.
	D1	D1	SOCX I/O Parameters.
	E1	E1	Variable Grid Map.
	F1	F1	Constraint Grid Map.
	G0	G0	Optimal Control Scale Information.
	H1	H1	Index Sets and Sparsity Summary.
	I2	I2	Variables.
	J2	J2	Objective Function.
	K2	K2	Constraints.
	L1	L1	Control Analysis Constraint Error.
	M1	M1	Relative DAE Error.
	N0	N0	Dynamic Status Display.
	O0	O0	Dynamic Constraint Adjoints.

	P0	P0	Dynamic Variable Adjoints.
	Q1	Q1	Control Analysis Lagrangian Errors.
	R1	R1	Control Analysis Optimality Error.
	S1	S1	Grid Refinement Summary.
SPRTHS	SPARSE	SPARSE	Right Hand Side Sparsity Option.
NLPALG	SNLPMN	SNLPMN	Nonlinear Programming Algorithm.
NLPOMR	M	M	Nonlinear Programming Option for Mesh Refinement.
KEYDPL	.lueiLUE	.lueiLUE	Dynamic Status Display Key
		.	Satisfied Inequality
		l	Active Lower Bound
		u	Active Upper Bound
		e	Equality
		i	Ignored
		L	Violated Lower Bound
		U	Violated Upper Bound
		E	Violated Equality
		z	Active Lower Bound with Zero Multiplier
		Z	Active Upper Bound with Zero Multiplier
RHSTMP	RHSTMPLT	RHSTMPLT	Right Hand Side Sparsity Template File Prefix
RSTFIL	socx.restart	socx.restart	Restart File Name.
SCLFIL	scalewgt.fil	scalewgt.fil	Scale Weight File Name.
MRFMET	MINMAX	MINMAX	Mesh Refinement Method
			= MINMAX Minimize Maximum Error (mrfopt=0)
			= BISECT Interval Bisection (mrfopt=1)
			= EQUIDS Equidistribute Error (mrfopt=2)

VARIABLES:

PHASE	NT	NP	NST	GRID	NVAR
1	1	0	5	10	51

CONSTRAINTS:

PHASE	PT	CONT	NRHS	NCON
1	0	4	19	36

***** SOCX WARNING: IER = 1282 Optimal Control Transcription Procedure
 >>>> ROUTINE HJSFDI SET WARNING ERROR = 3

Curtis-Powell-Reid Algorithm
 NUMBER OF INDEX SETS = 2
 FE PER JACOBIAN (2-SIDED) = 4
 FE PER JACOBIAN/HESSIAN = 5

SPARSITY:

MATRIX	NROWS	NCOLS	NANNZ	NFDNZ	NZERO	%NZ
-----	-----	-----	-----	-----	-----	-----
JACOBIAN	36	51	72	108	180	9.80
GRADIENT	1	51	1	0	1	1.96
HESSIAN	51	51	0	81	81	6.11

```

-----
| Index Set Analysis:                                     |
| |                                                       |
| Phase 1          Function      Nonzeros |
| |-----|-----|
| | Differential    1             2 |
| | Differential    2             2 |
| | Differential    3             2 |
| | Differential    4             2 |
| |-----|-----|
-----

```

```

*****
*
*   PERTURBATION SIZES (INTERNAL)
*
*
*****

```

```

*      Smallest  5              Largest  5      *
*
*      (   26:  6.39327658E-06)   (   50:  1.00000000E-03)   *
*      (   31:  6.39327658E-06)   (   49:  1.00000000E-03)   *
*      (   21:  9.15211012E-06)   (   48:  1.00000000E-03)   *
*      (   36:  9.15225488E-06)   (   47:  1.00000000E-03)   *
*      (   16:  1.10765477E-05)   (   45:  1.00000000E-03)   *
*
*****
*
*      ....OPTIMIZATION OPERATOR SPRNLP....
*
*****
*
*      -----
*      Constraints          =   36      |   Variables          =   51      *
*      Equalities          =   36      |   Equalities          =    7      *
*      Inequalities        =    0      |   Bounds              =   18      *
*      Inactive            =    0      |   Free                 =   44      *
*      Fixed on Lower Bound =    0      |   Fixed on Lower Bound =    0      *
*      Fixed on Upper Bound =    0      |   Fixed on Upper Bound =    0      *
*      -----
*      Number of Active Constraints =   43      |   Number of Degrees of Freedom =    8      *
*      -----
*
*      ALGORITHM CONTROL PARAMETERS  (SPARSE DEFAULTS)
*
*      CONTOL   1.490E-08   Constraint Tolerance
*      OBJTOL   1.000E-07   Objective Function Tolerance
*      PGDTOL   1.000E-05   Projected Gradient Tolerance
*      SLPTOL    0.900     Slope Tolerance For SQP Line Search
*      IOFLAG    10        Output Level
*      MAXNFE   10000      Maximum Number of Function Evaluations
*      NITMAX    100       Maximum Number of Iterations
*      ALGOPT    FM        Algorithm Control Option
*
*****
*
*      CONSTRAINT SATISFACTION
*
*      ----- Iteration 1 -----
*      Constraint Error.....+16.2941 | Cond(K).....+67749.7
*      Active Constraints.....44 | Degrees of Freedom.....7
*      QP Iterations.....4 | Matrix Factorizations.....1
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      .....Error = 2.8503972          Change = -13.443678
*
*      ----- Iteration 2 -----
*      Constraint Error.....+2.85040 | Cond(K).....+46911.6
*      Active Constraints.....51 | Degrees of Freedom.....0
*      QP Iterations.....10 | Matrix Factorizations.....1
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      .....Error = 0.47597306          Change = -2.3744242
*
*      ----- Iteration 3 -----
*      Constraint Error.....+0.475973 | Cond(K).....+25521.4
*      Active Constraints.....44 | Degrees of Freedom.....7
*      QP Iterations.....12 | Matrix Factorizations.....1
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      .....Error = 2.0062869          Change = 1.5303138
*

```

```
*      Quadratic Line Search .....Step: 2.....ALFA = 5.32841823E-02      *
*      .....Error = 0.45335874          Change = -2.26143176E-02      *
*      Cubic Line Search .....Step: 3.....ALFA = 0.13972132      *
*      .....Error = 0.42989224          Change = -4.60808230E-02      *
*      Cubic Line Search .....Step: 4.....ALFA = 0.20391534      *
*      .....Error = 0.42677983          Change = -4.91932262E-02      *
*      ----- Iteration 4 -----      *
*      Constraint Error.....+0.426780 | Cond(K).....+7226.51      *
*      Active Constraints.....45 | Degrees of Freedom.....6      *
*      QP Iterations.....8 | Matrix Factorizations.....1      *
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000      *
*      .....Error = 0.99907494          Change = 0.57229511      *
*      Quadratic Line Search .....Step: 2.....ALFA = 0.15431865      *
*      .....Error = 0.37691298          Change = -4.98668565E-02      *
*      Cubic Line Search .....Step: 3.....ALFA = 0.23021436      *
*      .....Error = 0.36584706          Change = -6.09327708E-02      *
*      ----- Iteration 5 -----      *
*      Constraint Error.....+0.365847 | Cond(K).....+10709.4      *
*      Active Constraints.....45 | Degrees of Freedom.....6      *
*      QP Iterations.....3 | Matrix Factorizations.....1      *
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000      *
*      .....Error = 0.42484319          Change = 5.89961264E-02      *
*      Quadratic Line Search .....Step: 2.....ALFA = 0.42579964      *
*      .....Error = 0.28117529          Change = -8.46717704E-02      *
*      ----- Iteration 6 -----      *
*      Constraint Error.....+0.281175 | Cond(K).....+10295.0      *
*      Active Constraints.....43 | Degrees of Freedom.....8      *
*      QP Iterations.....5 | Matrix Factorizations.....1      *
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000      *
*      .....Error = 0.18881174          Change = -9.23635533E-02      *
*      ----- Iteration 7 -----      *
*      Constraint Error.....+0.188812 | Cond(K).....+8460.26      *
*      Active Constraints.....43 | Degrees of Freedom.....8      *
*      QP Iterations.....7 | Matrix Factorizations.....1      *
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000      *
*      .....Error = 9.50953636E-02      Change = -9.37163750E-02      *
*      ----- Iteration 8 -----      *
*      Constraint Error.....+9.509536E-02 | Cond(K).....+12247.6      *
*      Active Constraints.....43 | Degrees of Freedom.....8      *
*      QP Iterations.....3 | Matrix Factorizations.....1      *
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000      *
*      .....Error = 3.13802811E-03      Change = -9.19573355E-02      *
```



```

*
* ----- Iteration 9 -----
* Constraint Error.....+3.138028E-03 | Cond(K).....+13153.2
* Active Constraints.....43 | Degrees of Freedom.....8
* QP Iterations.....3 | Matrix Factorizations.....1
*
* Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
* .....Error = 2.99446104E-06 Change = -3.13503365E-03
*
* ----- Iteration 10 -----
* Constraint Error.....+2.994461E-06 | Cond(K).....+13172.0
* Active Constraints.....43 | Degrees of Freedom.....8
* QP Iterations.....3 | Matrix Factorizations.....1
*
* Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
* .....Error = 8.82827800E-11 Change = -2.99437276E-06
*
* ----- Iteration 11 -----
* Constraint Error.....+8.828278E-11 | Cond(K).....+13172.0
* Active Constraints.....43 | Degrees of Freedom.....8
* QP Iterations.....0 | Matrix Factorizations.....0
*
* CONSTRAINT SATISFACTION COMPLETED
*
*****
*
* CONSTRAINED OPTIMIZATION
*
* Number of Active Constraints: 43
* Number of Degrees of Freedom: 8
*
* Constraint Multipliers
*
* Smallest 5 Largest 5
*
* ( 2: -0.57846485 ) ( 36: 6.23366993E-03)
* ( 34: -0.57842234 ) ( 8: 2.80921969E-03)
* ( 6: -0.57840298 ) ( 28: 1.61920904E-03)
* ( 30: -0.57836629 ) ( 16: 5.84402168E-04)
* ( 10: -0.57832518 ) ( 20: 1.03469892E-04)
*
* Constraint Multiplier Magnitudes
*
* Smallest 5 Largest 5
*
* ( 20: 1.03469892E-04) ( 2: 0.57846485 )
* ( 16: 5.84402168E-04) ( 34: 0.57842234 )
* ( 24: 7.76913919E-04) ( 6: 0.57840298 )
* ( 33: 1.37760787E-03) ( 30: 0.57836629 )
* ( 12: 1.47999628E-03) ( 10: 0.57832518 )
*
* Bound Multipliers
*
* Smallest 5 Largest 5
*
* ( 3: -0.57846485 ) ( 48: 0.57842234 )
* ( 4: -3.93421964E-02) ( 49: 2.75618255E-02)
* ( 5: -1.17321893E-02) ( 51: 0.0000000 )
* ( 50: -1.10023922E-02) ( 47: 0.0000000 )
* ( 2: -6.18996903E-03) ( 46: 0.0000000 )
*
* Bound Multiplier Magnitudes
*
* Smallest 5 Largest 5
*
* ( 2: 6.18996903E-03) ( 3: 0.57846485 )

```

```

*      ( 50: 1.10023922E-02)      ( 48: 0.57842234 )
*      ( 5: 1.17321893E-02)      ( 4: 3.93421964E-02)
*      ( 49: 2.75618255E-02)      ( 49: 2.75618255E-02)
*      ( 4: 3.93421964E-02)      ( 5: 1.17321893E-02)
*
* -----
*
* ----- Iteration (1) -----
* Projected Gradient.....+1.394051E-02 | Objective Function.....+0.700771
* Equality Error.....+6.202061E-11 | Inequality Error.....+0.00000
* Merit Function.....+0.700771 | Penalty Weight Norm.....+1.490116E-08
* Levenberg Parameter.....+0.00000 | Cond(K).....+129428.
* Min. Eigenvalue.....-0.158589 | Max. Eigenvalue.....+0.256184
* Active Constraints.....43 | Degrees of Freedom.....8
* QP Iterations.....3 | Matrix Factorizations.....1
*
* Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
* .....Index = 0.46992306 Change = -0.23084758
*
* ----- Iteration (2) -----
* Projected Gradient.....+0.127344 | Objective Function.....+0.345635
* Equality Error.....+2.81630 | Inequality Error.....+0.00000
* Merit Function.....+1.19616 | Penalty Weight Norm.....+5.007969E-02
* Levenberg Parameter.....+0.00000 | Cond(K).....+55943.5
* Min. Eigenvalue.....-0.158589 | Max. Eigenvalue.....+0.256184
* Active Constraints.....43 | Degrees of Freedom.....8
* QP Iterations.....3 | Matrix Factorizations.....1
*
* Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
* .....Index = 1.4086056 Change = 0.21244447
*
* Quadratic Line Search .....Step: 2.....ALFA = 0.42783046
*
* .....Index = 0.96733421 Change = -0.22882695
*
* ----- Iteration (3) -----
* Projected Gradient.....+0.163378 | Objective Function.....+0.418290
* Equality Error.....+2.32191 | Inequality Error.....+0.00000
* Merit Function.....+0.882671 | Penalty Weight Norm.....+4.080268E-02
* Levenberg Parameter.....+2.980232E-08 | Cond(K).....+29890.4
* Min. Eigenvalue.....-0.295808 | Max. Eigenvalue.....+0.475704
* Active Constraints.....43 | Degrees of Freedom.....8
* QP Iterations.....1 | Matrix Factorizations.....1
*
* Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
* .....Index = 0.58506324 Change = -0.29760804
*
* ----- Iteration [4] -----
* Projected Gradient.....+2.006670E-02 | Objective Function.....+0.544395
* Equality Error.....+0.765452 | Inequality Error.....+0.00000
* Merit Function.....+0.559356 | Penalty Weight Norm.....+1.490116E-08
* Levenberg Parameter.....+2.980232E-08 | Cond(K).....+43998.3
* Min. Eigenvalue.....-0.255465 | Max. Eigenvalue.....+0.550333
* Active Constraints.....43 | Degrees of Freedom.....8
* QP Iterations.....1 | Matrix Factorizations.....1
*
* Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
* .....Index = 0.55755503 Change = -1.80124221E-03
*
* ----- Iteration [5] -----
* Projected Gradient.....+2.496413E-03 | Objective Function.....+0.554813
* Equality Error.....+4.955623E-02 | Inequality Error.....+0.00000
* Merit Function.....+0.557738 | Penalty Weight Norm.....+8.350296E-02
* Levenberg Parameter.....+0.00000 | Cond(K).....+25544.9

```

```

*      Min. Eigenvalue.....-3.989484E-02 |      Max. Eigenvalue.....+0.526459      *
*      Active Constraints.....43 |      Degrees of Freedom.....8      *
*      QP Iterations.....1 |      Matrix Factorizations.....1      *
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000      *
*
*      .....Index = 0.55757518          Change = -1.6236605E-04      *
*
*      ----- Iteration [6] -----      *
*      Projected Gradient.....+2.683132E-05 |      Objective Function.....+0.557564      *
*      Equality Error.....+3.756802E-04 |      Inequality Error.....+0.00000      *
*      Merit Function.....+0.557575 |      Penalty Weight Norm.....+5.166189E-02      *
*      Levenberg Parameter.....+0.00000 |      Cond(K).....+25656.4      *
*      Min. Eigenvalue.....-3.989484E-02 |      Max. Eigenvalue.....+0.526459      *
*      Active Constraints.....43 |      Degrees of Freedom.....8      *
*      QP Iterations.....1 |      Matrix Factorizations.....1      *
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000      *
*
*      .....Index = 0.55757517          Change = -7.70217279E-09      *
*
*      ----- Iteration [7] -----      *
*      Projected Gradient.....+2.512647E-07 |      Objective Function.....+0.557575      *
*      Equality Error.....+9.005317E-09 |      Inequality Error.....+0.00000      *
*      Merit Function.....+0.557575 |      Penalty Weight Norm.....+5.166189E-02      *
*      Levenberg Parameter.....+0.00000 |      Cond(K).....+25656.4      *
*      Min. Eigenvalue.....-3.989484E-02 |      Max. Eigenvalue.....+0.526459      *
*      Active Constraints.....43 |      Degrees of Freedom.....8      *
*      QP Iterations.....0 |      Matrix Factorizations.....0      *
*
*      CONSTRAINED OPTIMIZATION COMPLETED      *
*
*      *****      *
*      CONVERGENCE TESTS      *
*
*      -----      *
*      Constraints = 36 | Variables = 51      *
*      Equalities = 36 | Equalities = 7      *
*      Inequalities = 0 | Bounds = 18      *
*      Inactive = 0 | Free = 44      *
*      Fixed on Lower Bound = 0 | Fixed on Lower Bound = 0      *
*      Fixed on Upper Bound = 0 | Fixed on Upper Bound = 0      *
*
*      -----      *
*      Number of Active Constraints = 43 | Number of Degrees of Freedom = 8      *
*      -----      *
*
*      Predicted Optimum Objective Function = 0.55757517      *
*
*      Gradient Norm = 1.0000000      *
*
*      Projected Gradient Vector      *
*
*      Smallest 5          Largest 5      *
*
*      ( 26: -2.51264709E-07) ( 31: 1.53335677E-07)      *
*      ( 36: -1.42837214E-07) ( 11: 6.73590047E-08)      *
*      ( 41: -1.16450655E-07) ( 21: 5.88544583E-08)      *
*      ( 46: -6.47242258E-08) ( 6: 3.84448358E-08)      *
*      ( 16: -3.97285063E-08) ( 40: 3.17806444E-10)      *
*
*      Constraint Multipliers      *
*
*      Smallest 5          Largest 5      *
*
*      ( 35: -0.25805192 ) ( 36: 7.85913842E-03)      *
*      ( 31: -0.25805192 ) ( 32: 5.89435315E-03)      *
*      ( 27: -0.25805192 ) ( 28: 3.92956788E-03)      *

```

```

*      ( 23: -0.25805192 ) ( 24: 1.96478261E-03)
*      ( 19: -0.25805192 ) ( 29: 9.87404509E-16)
*
*      Constraint Multiplier Magnitudes
*
*      Smallest 5                Largest 5
*
*      ( 9: 7.17958679E-16) ( 35: 0.25805192 )
*      ( 13: 7.17958679E-16) ( 31: 0.25805192 )
*      ( 17: 7.17958679E-16) ( 27: 0.25805192 )
*      ( 21: 7.17958679E-16) ( 23: 0.25805192 )
*      ( 25: 7.17958679E-16) ( 19: 0.25805192 )
*
*      Bound Multipliers
*
*      Smallest 5                Largest 5
*
*      ( 4: -0.25805192 ) ( 49: 0.25805192 )
*      ( 3: -0.14976162 ) ( 48: 0.14976162 )
*      ( 5: -8.84153637E-03) ( 2: 7.18449714E-16)
*      ( 50: -8.84153106E-03) ( 51: 0.0000000 )
*      ( 1: 0.0000000 ) ( 47: 0.0000000 )
*
*      Bound Multiplier Magnitudes
*
*      Smallest 5                Largest 5
*
*      ( 2: 7.18449714E-16) ( 49: 0.25805192 )
*      ( 50: 8.84153106E-03) ( 4: 0.25805192 )
*      ( 5: 8.84153637E-03) ( 48: 0.14976162 )
*      ( 3: 0.14976162 ) ( 3: 0.14976162 )
*      ( 48: 0.14976162 ) ( 5: 8.84153637E-03)

```

```

*****
***** CONVERGENCE *****

```

```

|
|      SPRNLP ALGORITHM PERFORMANCE STATISTICS
|
|      Total CPU Time.....+6.000000E-03
|
|      Number of Function Calls.....24
|      Number of Gradient Calls.....18
|      Number of Hessian Calls.....4
|      Total Number of Function Evaluations.....78
|
|      Storage Needed in HOLD Array.....2084
|

```

=====

OPTIMAL CONTROL ANALYSIS GRID 1:

NLP SOLUTION:

```

      FUNCTION EVALUATION NUMBER      79
=====
|||||-----|||
||||| Phase 1 |||
|||||-----|||
=====
Parameters
=====
TFINAL
=====
5.5757516E-01

```

```

=====
===== Dynamic Variables =====
-----
Point      TIME      RANGE      ALTITUDE      RNG-RATE      ALT-RATE      PITCH
-----
1    0.0000000E+00  0.0000000E+00  0.0000000E+00  0.0000000E+00  0.0000000E+00  5.2311101E+01
2    6.1952796E-02  1.2217266E-01  1.4786075E-01  3.9440562E+00  4.7733359E+00  4.8557211E+01
3    1.2390559E-01  5.0462647E-01  5.7586013E-01  8.4025638E+00  9.0436152E+00  3.8972111E+01
4    1.8585839E-01  1.1861104E+00  1.2383865E+00  1.3597540E+01  1.2344487E+01  2.5891384E+01
5    2.4781118E-01  2.2095602E+00  2.0603872E+00  1.9442124E+01  1.4191868E+01  9.1904629E+00
6    3.0976398E-01  3.6034981E+00  2.9396130E+00  2.5557874E+01  1.4191868E+01 -9.1904637E+00
7    3.7171678E-01  5.3679241E+00  3.7616137E+00  3.1402459E+01  1.2344487E+01 -2.5891382E+01
8    4.3366957E-01  7.4743158E+00  4.4241400E+00  3.6597434E+01  9.0436140E+00 -3.8972143E+01
9    4.9562237E-01  9.8797377E+00  4.8521393E+00  4.1055942E+01  4.7733341E+00 -4.8557188E+01
10   5.5757516E-01  1.2545441E+01  5.0000000E+00  4.5000000E+01  0.0000000E+00 -5.2311072E+01
=====
===== Objective Function =====
-----
OBJ = 5.5757516E-01
=====
===== Differential Constraints =====
-----
Intrvl     RANGE_d     ALTITUDE_d     RNG-RATE_d     ALT-RATE_d
-----
1    2.2571944E-12 -7.5799089E-11  4.6676731E-09 -2.5215312E-10
2    -5.4970278E-11 -1.8354290E-10  1.7872478E-09  1.5182227E-09
3    -1.8958735E-10 -2.5442148E-10  1.4215029E-09  7.7461426E-10
4    -3.1935954E-10 -1.7067903E-10  2.0989001E-08  9.5350419E-09
5    -4.5063286E-10 -8.1457063E-12  3.0976771E-08  2.8481236E-09
6    -5.2827742E-10 -1.8720581E-11  2.2073158E-08 -7.3351640E-09
7    -4.1307091E-10  2.5791591E-11  2.0994231E-08 -1.7653380E-09
8    -1.8498270E-10  1.0933365E-10  1.5574931E-08 -2.9749784E-09
9    -3.6557424E-11  4.9040216E-11  7.5774736E-09 -8.4691407E-10
=====
===== Discretization Error Analysis =====
-----
|||||-----|||||
||||| Phase 1 |||||
|||||-----|||||
=====
Interval      TIME      Differential      Algebraic
-----
5    2.7878758E-01  (2)<1.20E-03>      (-) 0.00E+00
4    2.1683479E-01  (2)<1.04E-03>      (-) 0.00E+00
6    3.4074038E-01  (2)<1.04E-03>      (-) 0.00E+00
7    4.0269317E-01  (2)<7.22E-04>      (-) 0.00E+00
3    1.5488199E-01  (2)<7.22E-04>      (-) 0.00E+00
2    9.2929194E-02  (2)<4.53E-04>      (-) 0.00E+00
8    4.6464597E-01  (2)<4.53E-04>      (-) 0.00E+00
1    3.0976398E-02  (2)<1.57E-04>      (-) 0.00E+00
9    5.2659877E-01  (2)<1.57E-04>      (-) 0.00E+00
-----
Maximum Differential Equation Error  1.1986626E-03
Maximum Algebraic Equation Error    0.0000000E+00
-----
Intervals      Dominated By:
-----
9    Differential Equation.....2
-----

Optimal Objective Function  5.57575165E-01
=====

```

GRID	NPT	NFC	NGC	NHC	NFE	NRHS	ERREQN	CPU TIME
1	10	25	18	4	79	1308	1.1987E-03	9.0000E-03

ANALYSIS OF GRID 1 ERRORS:

RELATIVE ERROR ESTIMATES:

	MINIMUM	AVERAGE	MAXIMUM
PHASE 1	1.56571E-04	6.59884E-04	1.19866E-03
>>>(MAX ERROR)/(OLD ERROR) =	1.81647E+00		
(MAX ERROR)/(AVG ERROR) =	1.81647E+00		
MAXIMUM PREDICTED ERROR =	1.49833E-04		
MAXIMUM NUMBER OF INTERVAL SUBDIVISIONS =	1		
SMALLEST INTERVAL =	3.09764E-02	(3.40740E-01, 3.71717E-01)	
LARGEST INTERVAL =	3.09764E-02	(1.23906E-01, 1.54882E-01)	

```
=====
=====
=====
===== REFINEMENT ITERATION 2 =====
=====
=====
```

TRANSCRIPTION SUMMARY:

Phase	Discretization	Abrv	Method	Stage	Order	Grid Pts
1	Trapezoidal	TRP	2	1	2	19

VARIABLES:

PHASE	NT	NP	NST	GRID	NVAR
1	1	0	5	19	96

CONSTRAINTS:

PHASE	PT	CONT	NRHS	NCON
1	0	4	37	72

***** SOCX WARNING: IER = 1282 Optimal Control Transcription Procedure
>>>> ROUTINE HJSFDI SET WARNING ERROR = 3

Curtis-Powell-Reid Algorithm

NUMBER OF INDEX SETS	=	2
FE PER JACOBIAN (2-SIDED)	=	4
FE PER JACOBIAN/HESSIAN	=	5

SPARSITY:

MATRIX	NROWS	NCOLS	NANNZ	NFDNZ	NZERO	%NZ
JACOBIAN	72	96	144	216	360	5.21
GRADIENT	1	96	1	0	1	1.04
HESSIAN	96	96	0	153	153	3.29

```
-----
| Index Set Analysis:                |
|                                   |
| Phase 1      Function      Nonzeros |
|   Differential    1          2      |
|   Differential    2          2      |
|   Differential    3          2      |
|   Differential    4          2      |
|-----|
```

```
*****
*
* PERTURBATION SIZES (INTERNAL)
*
```

```

*
*      Smallest  5              Largest  5
*
*      (   51:  3.37755828E-06)   (   95:  1.00000000E-03)
*      (   46:  7.28499628E-06)   (   94:  1.00000000E-03)
*      (   56:  7.28506201E-06)   (   93:  1.00000000E-03)
*      (   61:  8.96068539E-06)   (   92:  1.00000000E-03)
*      (   41:  8.96068557E-06)   (   90:  1.00000000E-03)
*
*
*****
*
*      VARIABLE SCALE INFORMATION
*
*
*      ....SIGNIFICANT DISPARITY IN VARIABLE RANGES;  MEDIAN RANGE =  0.36827292
*
*
*      VARIABLE RANGE MAGNITUDES
*
*      Smallest  5              Largest  5
*
*      (   49:  1.13973820E-08)   (   96:  1.00000000 )
*      (   48:  1.86681183E-08)   (   91:  1.00000000 )
*      (   67:  8.62218788E-03)   (   86:  1.00000000 )
*      (   20:  1.53392541E-02)   (   81:  1.00000000 )
*      (   80:  1.53393984E-02)   (   76:  1.00000000 )
*
*****
*
*      ....OPTIMIZATION OPERATOR SPRNLP....
*
*****
*
*
*      -----
*      Constraints          =   72      |      Variables          =   96
*      Equalities          =   72      |      Equalities          =    7
*      Inequalities        =    0      |      Bounds              =   27
*      Inactive            =    0      |      Free                 =   82
*      Fixed on Lower Bound =    0      |      Fixed on Lower Bound =    3
*      Fixed on Upper Bound =    0      |      Fixed on Upper Bound =    4
*      -----
*      Number of Active Constraints =   86      |      Number of Degrees of Freedom =   10
*      -----
*
*      ALGORITHM CONTROL PARAMETERS  (SPARSE DEFAULTS)
*
*      CONTOL   1.490E-08   Constraint Tolerance
*      OBJTOL   1.000E-07   Objective Function Tolerance
*      PGDTOL   1.000E-05   Projected Gradient Tolerance
*      SLPTOL    0.900     Slope Tolerance For SQP Line Search
*      IOFLAG    10        Output Level
*      MAXNFE   10000      Maximum Number of Function Evaluations
*      NITMAX    100       Maximum Number of Iterations
*      *NITMIN    1        Minimum Number of Iterations
*      *ALGOPT    M        Algorithm Control Option
*
*****
*
*      CONSTRAINED OPTIMIZATION
*
*
*      Number of Active Constraints:    86
*      Number of Degrees of Freedom:    10
*
*
*      Constraint Multipliers
*
*
*      Smallest  5              Largest  5
*

```

```

*      ( 71: -0.25805192 ) ( 72: 8.35033474E-03)
*      ( 67: -0.25805192 ) ( 68: 7.36794210E-03)
*      ( 63: -0.25805192 ) ( 64: 6.38554947E-03)
*      ( 59: -0.25805192 ) ( 60: 5.40315683E-03)
*      ( 55: -0.25805192 ) ( 56: 4.42076420E-03)
*
*      Constraint Multiplier Magnitudes
*
*      Smallest 5          Largest 5
*
*      ( 69: 2.46851127E-16) ( 71: 0.25805192 )
*      ( 65: 2.46851127E-16) ( 67: 0.25805192 )
*      ( 21: 7.17958679E-16) ( 63: 0.25805192 )
*      ( 25: 7.17958679E-16) ( 59: 0.25805192 )
*      ( 29: 7.17958679E-16) ( 55: 0.25805192 )
*
*      Bound Multipliers
*
*      Smallest 5          Largest 5
*
*      ( 4: -0.25805192 ) ( 94: 0.25805192 )
*      ( 3: -0.14976162 ) ( 93: 0.14976162 )
*      ( 9: -0.12902596 ) ( 89: 0.12902596 )
*      ( 8: -7.48808105E-02) ( 88: 7.48808105E-02)
*      ( 5: -8.84153637E-03) ( 2: 7.18449714E-16)
*
*      Bound Multiplier Magnitudes
*
*      Smallest 5          Largest 5
*
*      ( 7: 3.59224857E-16) ( 94: 0.25805192 )
*      ( 2: 7.18449714E-16) ( 4: 0.25805192 )
*      ( 90: 4.42076553E-03) ( 3: 0.14976162 )
*      ( 10: 4.42076819E-03) ( 93: 0.14976162 )
*      ( 95: 8.84153106E-03) ( 89: 0.12902596 )
*
*      .....THE FOLLOWING CONSTRAINTS MAY BE EXTRANEIOUS OR POORLY SCALED
*
*      1      5      9      13      17      21      25      29      33      37
*      41     45     49     53     57     61     65     69
*
*      .....THE FOLLOWING BOUNDS MAY BE EXTRANEIOUS OR POORLY SCALED
*
*      2      7
*
*      -----
*
*      ----- Iteration (1) -----
*      Projected Gradient.....+0.345795 | Objective Function.....+0.772286
*      Equality Error.....+7.662715E-04 | Inequality Error.....+0.00000
*      Merit Function.....+0.769204 | Penalty Weight Norm.....+1.490116E-08
*      Levenberg Parameter.....+0.00000 | Cond(K).....+183.715
*      Min. Eigenvalue.....-0.699586 | Max. Eigenvalue.....+0.699586
*      Active Constraints.....79 | Degrees of Freedom.....17
*      QP Iterations.....3 | Matrix Factorizations.....1
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000
*
*      .....Index = 0.76928344 Change = 7.96606604E-05
*
*      Quadratic Line Search .....Step: 2.....ALFA = 0.36002616
*
*      .....Index = 0.76916700 Change = -3.67818527E-05
*
*      ----- Iteration (2) -----
*      Projected Gradient.....+4.574058E-04 | Objective Function.....+0.771102
*      Equality Error.....+4.908753E-04 | Inequality Error.....+0.00000
*      Merit Function.....+0.769167 | Penalty Weight Norm.....+1.29424

```



```

*      Levenberg Parameter.....+1.490116E-02 | Cond(K).....+69.4935      *
*      Min. Eigenvalue.....-0.327522 | Max. Eigenvalue.....+0.393364      *
*      Active Constraints.....79 | Degrees of Freedom.....17      *
*      QP Iterations.....15 | Matrix Factorizations.....7      *
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000      *
*
*      .....Index = 0.76916552          Change = -1.32617650E-06      *
*
*      ----- Iteration (3) -----      *
*      Projected Gradient.....+5.161257E-05 | Objective Function.....+0.769166      *
*      Equality Error.....+8.205559E-07 | Inequality Error.....+0.00000      *
*      Merit Function.....+0.769166 | Penalty Weight Norm.....+1.490116E-08      *
*      Levenberg Parameter.....+1.681411E-03 | Cond(K).....+61.9876      *
*      Min. Eigenvalue.....-0.323900 | Max. Eigenvalue.....+0.392514      *
*      Active Constraints.....79 | Degrees of Freedom.....17      *
*      QP Iterations.....3 | Matrix Factorizations.....1      *
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000      *
*
*      .....Index = 0.76916550          Change = -1.66331846E-08      *
*
*      ----- Iteration (4) -----      *
*      Projected Gradient.....+3.419096E-07 | Objective Function.....+0.769165      *
*      Equality Error.....+1.900270E-08 | Inequality Error.....+0.00000      *
*      Merit Function.....+0.769166 | Penalty Weight Norm.....+1.490116E-08      *
*      Levenberg Parameter.....+1.113858E-05 | Cond(K).....+96.4297      *
*      Min. Eigenvalue.....-0.323811 | Max. Eigenvalue.....+0.392290      *
*      Active Constraints.....79 | Degrees of Freedom.....17      *
*      QP Iterations.....1 | Matrix Factorizations.....1      *
*
*      Quadratic Line Search .....Step: 1.....ALFA = 1.0000000      *
*
*      .....Index = 0.76916550          Change = -9.72788516E-12      *
*
*      ----- Iteration [5] -----      *
*      Projected Gradient.....+6.048817E-11 | Objective Function.....+0.769166      *
*      Equality Error.....+1.269956E-11 | Inequality Error.....+0.00000      *
*      Merit Function.....+0.769166 | Penalty Weight Norm.....+1.490116E-08      *
*      Levenberg Parameter.....+1.113858E-05 | Cond(K).....+96.4297      *
*      Min. Eigenvalue.....-0.323811 | Max. Eigenvalue.....+0.392290      *
*      Active Constraints.....79 | Degrees of Freedom.....17      *
*      QP Iterations.....0 | Matrix Factorizations.....0      *
*
*      CONSTRAINED OPTIMIZATION COMPLETED      *
*
*      *****      *
*      CONVERGENCE TESTS      *
*
*      -----      *
*      Constraints          = 72 | Variables          = 96      *
*      Equalities          = 72 | Equalities          = 7      *
*      Inequalities        = 0 | Bounds              = 27      *
*      Inactive             = 0 | Free                 = 89      *
*      Fixed on Lower Bound = 0 | Fixed on Lower Bound = 0      *
*      Fixed on Upper Bound = 0 | Fixed on Upper Bound = 0      *
*
*      -----      *
*      Number of Active Constraints = 79 | Number of Degrees of Freedom = 17      *
*
*      -----      *
*
*      Predicted Optimum Objective Function = 0.76916550      *
*
*      Gradient Norm = 0.77228585      *
*
*      Projected Gradient Vector      *
*
*      Smallest 5          Largest 5      *

```

```

*
*      (   25: -6.04881700E-11)      (   60:  5.49742057E-11)
*      (   30: -5.92571964E-11)      (   55:  4.99852139E-11)
*      (   20: -5.24495447E-11)      (   65:  4.98104336E-11)
*      (    6: -4.81589386E-11)      (   46:  4.83065714E-11)
*      (   35: -4.51891996E-11)      (   41:  4.02966879E-11)
*
*      Constraint Multipliers
*
*      Smallest  5                      Largest  5
*
*      (    3: -0.35999193   )      (   72:  0.10307865   )
*      (    7: -0.35999193   )      (   68:  9.09517533E-02)
*      (   11: -0.35999192   )      (   64:  7.88248529E-02)
*      (   15: -0.35999192   )      (   60:  6.66979524E-02)
*      (   19: -0.35999192   )      (   56:  5.45710520E-02)
*
*      Constraint Multiplier Magnitudes
*
*      Smallest  5                      Largest  5
*
*      (   69:  2.39968113E-11)      (    3:  0.35999193   )
*      (   65:  4.78989173E-11)      (    7:  0.35999193   )
*      (   61:  7.14632708E-11)      (   11:  0.35999192   )
*      (   57:  9.44629988E-11)      (   15:  0.35999192   )
*      (   53:  1.16636550E-10)      (   19:  0.35999192   )
*
*      Bound Multipliers
*
*      Smallest  5                      Largest  5
*
*      (    4: -0.35999193   )      (   94:  0.35999192   )
*      (    3: -0.20458679   )      (   93:  0.20458679   )
*      (   95: -0.16123610   )      (   96:  0.00000000   )
*      (    5: -0.16123609   )      (   92:  0.00000000   )
*      (    2: -2.74962583E-10)      (   91:  0.00000000   )
*
*      Bound Multiplier Magnitudes
*
*      Smallest  5                      Largest  5
*
*      (    2:  2.74962583E-10)      (    4:  0.35999193   )
*      (    5:  0.16123609   )      (   94:  0.35999192   )
*      (   95:  0.16123610   )      (   93:  0.20458679   )
*      (    3:  0.20458679   )      (    3:  0.20458679   )
*      (   93:  0.20458679   )      (   95:  0.16123610   )
*
*****
***** CONVERGENCE *****

```

```

-----
|
|      SPRNLP ALGORITHM PERFORMANCE STATISTICS
|
|      Total CPU Time.....+7.000001E-03
|
|      Number of Function Calls.....6
|      Number of Gradient Calls.....6
|      Number of Hessian Calls.....4
|      Total Number of Function Evaluations.....32
|
|      Storage Needed in HOLD Array.....4133
|
-----

```

=====

OPTIMAL CONTROL ANALYSIS GRID 2:

NLP SOLUTION:

```

FUNCTION EVALUATION NUMBER      33
=====
|||||-----|||||
|||||Phase 1|||||
|||||-----|||||
===== Parameters =====
-----
TFINAL
-----
5.5532233E-01
=====
===== Dynamic Variables =====
-----
Point      TIME      RANGE      ALTITUDE      RNG-RATE      ALT-RATE      PITCH
-----
  1  0.000000E+00  0.000000E+00  0.000000E+00  0.000000E+00  0.000000E+00  5.3293356E+01
  2  3.0851241E-02  2.8997896E-02  3.7728503E-02  1.8798528E+00  2.4458338E+00  5.1615327E+01
  3  6.1702481E-02  1.1773867E-01  1.4947736E-01  3.8729646E+00  4.7985333E+00  4.7844835E+01
  4  9.2553722E-02  2.7047327E-01  3.3151755E-01  6.0283942E+00  7.0026250E+00  4.3434211E+01
  5  1.2340496E-01  4.9241646E-01  5.7865559E-01  8.3595644E+00  9.0186459E+00  3.8272989E+01
  6  1.5425620E-01  7.8912200E-01  8.8433158E-01  1.0875029E+01  1.0797477E+01  3.2259702E+01
  7  1.8510744E-01  1.1662589E+00  1.2403291E+00  1.3573707E+01  1.2280849E+01  2.5332516E+01
  8  2.1595868E-01  1.6292232E+00  1.6365511E+00  1.6438978E+01  1.3405120E+01  1.7515430E+01
  9  2.4680993E-01  2.1825820E+00  2.0609859E+00  1.9433729E+01  1.4109811E+01  8.9671879E+00
 10  2.7766117E-01  2.8294358E+00  2.5000000E+00  2.2500000E+01  1.4350248E+01  9.5007991E-08
 11  3.0851241E-01  3.5708878E+00  2.9390141E+00  2.5566271E+01  1.4109811E+01 -8.9671877E+00
 12  3.3936365E-01  4.4058349E+00  3.3634489E+00  2.8561022E+01  1.3405120E+01 -1.7515430E+01
 13  3.7021489E-01  5.3311764E+00  3.7596709E+00  3.1426293E+01  1.2280849E+01 -2.5332516E+01
 14  4.0106613E-01  6.3423453E+00  4.1156684E+00  3.4124971E+01  1.0797477E+01 -3.2259702E+01
 15  4.3191737E-01  7.4339456E+00  4.4213444E+00  3.6640436E+01  9.0186459E+00 -3.8272989E+01
 16  4.6276861E-01  8.6003083E+00  4.6684825E+00  3.8971606E+01  7.0026250E+00 -4.3434211E+01
 17  4.9361985E-01  9.8358795E+00  4.8505226E+00  4.1127035E+01  4.7985334E+00 -4.7844835E+01
 18  5.2447109E-01  1.1135445E+01  4.9622715E+00  4.3120147E+01  2.4458338E+00 -5.1615327E+01
 19  5.5532233E-01  1.2494752E+01  5.0000000E+00  4.5000000E+01  0.0000000E+00 -5.3293356E+01
=====
===== Objective Function =====
-----
OBJ =  5.5532233E-01
=====
===== Differential Constraints =====
-----
Intrvl  RANGE_d      ALTITUDE_d      RNG-RATE_d      ALT-RATE_d
-----
  1  8.0508517E-15 -6.1062266E-15  9.4106598E-11  1.2199960E-10
  2  2.2752633E-14 -1.7416624E-14  7.5290399E-11  8.8564962E-11
  3  3.4180991E-14 -2.8310687E-14  5.3233667E-11  5.5329237E-11
  4  4.1633363E-14 -3.5249581E-14  1.8666318E-11  1.7232193E-11
  5  4.3964832E-14 -3.7470027E-14  3.4772185E-12  2.3613341E-12
  6  4.1300297E-14 -3.2751579E-14  4.4856618E-11  2.2007122E-11
  7  3.6637360E-14 -2.0816682E-14  9.1247010E-11  3.6025810E-11
  8  3.3972825E-14 -4.4408921E-15  1.2653878E-10  2.8312546E-11
  9  2.8310687E-14  1.2767565E-14  1.1643686E-10  1.2800692E-11
 10  2.7311486E-14  2.6645353E-14  4.4229620E-11 -5.8873371E-13
 11  2.9976022E-14  3.3306691E-14  5.5805360E-12 -7.8924447E-13
 12  1.2656542E-14  3.2751579E-14  1.2050361E-11 -5.8020134E-12
 13  2.7977620E-14  2.7200464E-14  2.0433655E-11 -1.1320325E-11
 14  1.9984014E-14  2.1094237E-14  2.1742608E-11 -1.6138983E-11
 15  1.7319479E-14  1.4432899E-14  1.9674262E-11 -1.6933560E-11
 16  1.0658141E-14  9.4368957E-15  1.0296763E-11 -1.0678264E-11
 17  6.6613381E-15  5.5511151E-15  7.0893291E-12 -8.6358277E-12
 18  2.6645353E-15  1.1102230E-15  8.3633100E-12 -1.1284596E-11
=====
===== Discretization Error Analysis =====
=====

```

```

=====
|||||-----|||||
||||| Phase 1 |||||
|||||-----|||||
=====
Interval          TIME          Differential          Algebraic
-----
10  2.9308679E-01  (2)<1.45E-04>  (-) 0.00E+00
9   2.6223555E-01  (2)<1.45E-04>  (-) 0.00E+00
11  3.2393803E-01  (2)<1.35E-04>  (-) 0.00E+00
8   2.3138430E-01  (2)<1.35E-04>  (-) 0.00E+00
12  3.5478927E-01  (2)<1.18E-04>  (-) 0.00E+00
7   2.0053306E-01  (2)<1.18E-04>  (-) 0.00E+00
13  3.8564051E-01  (2)<9.88E-05>  (-) 0.00E+00
6   1.6968182E-01  (2)<9.88E-05>  (-) 0.00E+00
14  4.1649175E-01  (2)<7.99E-05>  (-) 0.00E+00
5   1.3883058E-01  (2)<7.99E-05>  (-) 0.00E+00
-----
Maximum Differential Equation Error  1.4547692E-04
Maximum Algebraic Equation Error    0.0000000E+00
-----
Intervals    Dominated By:
-----
18           Differential Equation.....2
-----

Differential Equation Error  1.4548E-04 < 1.0000E-03
Objective Function Change    2.2528E-03 > 1.0000E-05
Optimal Objective Function    5.55322332E-01
=====

GRID  NPT  NFC  NGC  NHC  NFE  NRHS  ERREQN  CPU TIME
2     19   7    6    4    33  1283  1.4548E-04  7.0000E-03
=====

OPTIMAL CONTROL PROGRAM SUMMARY

GRID  NPT  NFC  NGC  NHC  NFE  NRHS  ERREQN  CPU TIME
1     10  25   18   4    79  1308  1.1987E-03  9.0000E-03
2     19   7    6    4    33  1283  1.4548E-04  7.0000E-03
---  ---  ---  ---  ---  ---  ---  ---  ---
19   32   24   8   112  2591  1.6000E-02
=====

| Total Solution Time.....+1.700000E-02 |
| Total User Routine Time.....+0.00000   |
| Total SOCX Computation Time.....+1.700000E-02 |
|-----|

```

7.2 Generalized Linear Tangent Steering Example

7.2.1 Problem Definition

The second example is obtained by changing the first example to include nonlinear boundary conditions. In particular at the final time let us not specify the values of $y_3(t_F)$ and $y_4(t_F)$ and instead impose the following two conditions:

$$\psi_1 = \sqrt{y_3^2 + y_4^2} = 45$$

and

$$-5 \leq \psi_2 \leq 5$$

where $\psi_2 = \arctan(y_4, y_3)$. The first condition requires the magnitude of the velocity to be the same as example 1, while the second condition allows the flight path angle to vary ± 5 (deg) about zero.

7.2.2 SOSCTL Implementation

The SOS implementation of this example illustrates a number of features of the software. First note that the implementation requires a nonlinear point function subroutine (LNTSPF) which evaluates the boundary conditions. Secondly, observe that the input routine LNTSIN incorporates these boundary conditions using the ITERM mechanism. Finally, note that three nonstandard options have been set in the main program prior to calling SOSCTL, which alter the differential equation accuracy, and the output levels from SOS and the sparse NLP.

SAMPLE PROGRAM (lntsex2.cd1)²

```

PROGRAM SOCXEX
c
c      example lntsex2
C
C      Sample main program for sparse optimal control software.
C      This program can be used to solve a single optimal control problem
C
C      *****
C      IMPLICIT DOUBLE PRECISION  (A-H,O-Z)
C
C      PARAMETER  (MXC=40000)
C
C      DIMENSION  CSTAT(MXC)
C
C      EXTERNAL   LNTSIN,  LNTSDE,  DUMYIG,  LNTSPF,  DUMYPR
C
C      *****

```

²sosexample.tar

```
C
C      CALL SOS WITH THE APPROPRIATE ARGUMENTS FOR THE
C      PROBLEM
C
C      ----DIFFERENTIAL EQUATION ACCURACY
C
C      CALL inpsos('ODETOL=1.E-7')
C
C      ----SOCX OUTPUT LEVEL
C
C      CALL inpsos('IPGRD=5')
C
C      ----SPARSE NLP OUTPUT LEVEL
C
C      CALL inpsos('IOFLAG=1')
C
C      ---   LINEAR TANGENT STEERING PROBLEM
C
C      CALL SOSCTL(LNTSIN,DUMYIG,LNTSDE,LNTSPF,DUMYPR,
&                  CSTAT,MXC,NEEDED,IER)
C
C      *****
C
C      STOP
C      END
C
C      SUBROUTINE LNTSIN(IPHASE,NPHS,METHOD,NSTG,NCF,NPF,NPV,NAV,NGRID,
&                  INIT,MAXMIN,MXPARM,PO,PLB,PUB,PLBL,
&                  MXSTAT,YO,Y1,YLB,YUB,STSKL,STLBL,MXPCON,CLB,CUB,
&                  CLBL,MXTERM,COEF,ITERM,TITLE,IER)
C
C      LINEAR TANGENT STEERING EXAMPLE PROBLEM
C
C      *****
C
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      Arguments:
C
C      INTEGER      IPHASE,NPHS,METHOD,NSTG(2),NCF(6),NPF(3),NPV,NAV,NGRID,
&                  INIT,MAXMIN,MXPARM,MXSTAT,MXPCON,MXTERM,
&                  ITERM(4,MXTERM),IER
C      DIMENSION    PO(MXPARM),PLB(MXPARM),PUB(MXPARM),YO(0:MXSTAT),
&                  Y1(0:MXSTAT),YLB(-1:1,0:MXSTAT),YUB(-1:1,0:MXSTAT),
&                  STSKL(0:MXSTAT+MXPARM,2),CLB(MXPCON),CUB(MXPCON),
&                  COEF(MXTERM)
C      CHARACTER(LEN=60)  TITLE(3)
```

```

C      CHARACTER(LEN=80)  PLBL(MXPARM),STLBL(0:MXSTAT),CLBL(0:MXPCON)
C
C      *****
C
C      ----PROBLEM TITLES AND LABELS (OPTIONAL)
C
C      TITLE(1) = 'Generalized Linear Tangent Steering Ascent'
C      TITLE(2) = 'Nonlinear Boundary Conditions'
C      STLBL(0) = 'TIME      Time'
C      STLBL(1) = 'RANGE     Range'
C      STLBL(2) = 'ALTITUDE  Altitude'
C      STLBL(3) = 'RNG-RATE  Range Rate'
C      STLBL(4) = 'ALT-RATE  Altitude Rate'
C      STLBL(5) = 'PITCH     Pitch angle (deg)'
C      PLBL(1) = 'TFINAL     Final time'
C
C      ----NUMBER OF DIFFERENTIAL EQUATIONS (STATES)
C
C      NDE = 4
C      NCF(1) = NDE
C
C      ----NUMBER OF ALGEBRAIC VARIABLES (CONTROLS)
C
C      NAV = 1
C
C      ----GUESS FOR INITIAL TIME AND FIXED INITIAL CONDITION
C
C      Y0(0) = 0.D0
C      YLB(-1,0) = Y0(0)
C      YUB(-1,0) = Y0(0)
C
C      ----GUESS FOR FINAL TIME AND LOWER BOUND
C
C      T1 = 1.D0
C      Y1(0) = T1
C      YLB(1,0) = 0.D0
C
C      ----RADIANS TO DEGREES CONVERSION FACTOR
C
C      CR2D = HDMCON(16)
C
C      ----PI/2 FACTOR
C
C      CPI2 = HDMCON(12)/2.D0
C
C      ----DEFINE INITIAL CONDITIONS FOR STATE VARIABLES, Y.
C
C      Y(1) = RANGE.

```

```
C          Y(2) = ALTITUDE.
C          Y(3) = HORIZONTAL VELOCITY.
C          Y(4) = VERTICAL VELOCITY.
C
C          Y0(1) = 0.DO
C          Y0(2) = 0.DO
C          Y0(3) = 0.DO
C          Y0(4) = 0.DO
C
C          ----FIX THE INITIAL STATES
C
C          YLB(-1,1:4) = Y0(1:4)
C          YUB(-1,1:4) = Y0(1:4)
C
C          ----DEFINE FINAL CONDITIONS FOR STATE VARIABLES, Y.
C
C          Y1(1) = 12.DO
C          Y1(2) = 5.DO
C          Y1(3) = 45.DO
C          Y1(4) = 0.DO
C
C          ----DEFINE INITIAL AND FINAL CONTROL ANGLES.
C
C          Y0(5) = 1.DO
C          Y1(5) = -1.DO
C
C          ----FIX THE FINAL VALUES FOR STATE 2
C
C          YLB(1,2) = Y1(2)
C          YUB(1,2) = Y1(2)
C
C          ----INITIALIZE RUNNING COUNTER FOR THE NUMBER OF TERMS AND THE
C          NUMBER OF CONSTRAINTS ON THIS PHASE
C
C          NTERM = 0
C          NKON = 0
C
C          ----DEFINE CONSTRAINT ON FINAL VELOCITY MAGNITUDE
C
C          NTERM = NTERM + 1
C          NKON = NKON + 1
C          ITERM(1,NTERM) = NKON
C          ITERM(2,NTERM) = 1
C          ITERM(3,NTERM) = 1
C          ITERM(4,NTERM) = -NKON
C          COEF(NTERM) = 1.DO
C          CLB(NKON) = 45.DO
C          CUB(NKON) = 45.DO
```



```

C
C      ----DEFINE CONSTRAINT ON FINAL FLIGHT PATH DIRECTION
C
      NTERM = NTERM + 1
      NKON = NKON + 1
      ITERM(1,NTERM) = NKON
      ITERM(2,NTERM) = 1
      ITERM(3,NTERM) = 1
      ITERM(4,NTERM) = -NKON
      COEF(NTERM) = 1.D0
      CLB(NKON) = -5.D0
      CUB(NKON) = 5.D0

C
C      ----DEFINE THE NUMBER OF POINT FUNCTIONS AT THE END OF PHASE
C
      NPF(2) = NKON

C
C      ----BOUND CONTROL MAGNITUDE ON PHASE BY 90 DEG.
C
      YLB(-1:1,NDE+NAV) = -CPI2
      YUB(-1:1,NDE+NAV) =  CPI2

C
C      ----DEFINE FINAL TIME AS THE OBJECTIVE TO BE MINIMIZED
C
      NTERM = NTERM + 1
      ITERM(1,NTERM) = 0
      ITERM(2,NTERM) = 1
      ITERM(3,NTERM) = 1
      ITERM(4,NTERM) = 0
      COEF(NTERM) = 1.D0

C
      MAXMIN = -1

C
C      ----LOAD SCALE WEIGHTS FOR OUTPUT OF CONTROL IN DEGREES
C
      STSKL(NDE+NAV,1) = CR2D

C
      RETURN
      END

      SUBROUTINE LNTSDE(IPHASE,T,Y,NY,P,NP,F,NF,IFERR)

C
C      COMPUTES THE RIGHT HAND SIDES OF THE LINEAR TANGENT
C      STEERING SYSTEM OF DIFFERENTIAL EQUATIONS.
C
C      *****
C      IMPLICIT DOUBLE PRECISION (A-H,O-Z)

```

```
C
C  ARGUMENTS:
C
C      INTEGER    IPHASE,NY,NP,NF,IFERR
C      DIMENSION  Y(NY),P(NP),F(NF)
C
C      PARAMETER  (ATHRUS=100.DO)
C
C      *****
C
C      ----SET FUNCTION ERROR FLAG.
C
C      IFERR = 0
C
C      ----COMPUTE FUNCTIONS OF CONTROL.
C
C      COSU = COS(Y(5))
C      SINU = SIN(Y(5))
C
C      ----COMPUTE STATE EQUATIONS.
C
C      F(1) = Y(3)
C      F(2) = Y(4)
C      F(3) = ATHRUS*COSU
C      F(4) = ATHRUS*SINU
C
C      RETURN
C      END
C
C      SUBROUTINE LNTSPF(IPHASE,IPHEND,T,Y,NY,P,NP,PSI,NPSI,IFERR)
C
C          Computes terminal constraint for the linear tangent
C          steering problem.
C
C          *****
C          IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
C      Arguments:
C
C      INTEGER    IPHASE,IPHEND,NY,NP,NPSI,IFERR
C      DIMENSION  Y(NY),P(NP),PSI(NPSI)
C
C      *****
C
C      IFERR = 0
C
C      IF (IPHEND.EQ.1) THEN
C
```

```
***** SOCX WARNING: IER = 1478      Compute Sparsity Pattern for User Supplied Functions
>>>> Phase 1: No function depends on state 2
```

[illegible]

Sparse Optimization Suite
Applied Mathematical Analysis, LLC
SOS Version 2018.12

ODE = Generalized Linear Tangent Steering Ascent
GOAL = Nonlinear Boundary Conditions

```
=====
=====
===== REFINEMENT ITERATION 1 =====
=====
=====
```

TRANSCRIPTION SUMMARY:

Phase	Discretization	Abrv	Method	Stage	Order	Grid Pts
1	Trapezoidal	TRP	2	1	2	10
INIT=(1, 0): Linear Between Y0 and Y1						

SOCX OPTIONS (also see "FULL OPTIONS")

SYMBOL	DEFAULT	VALUE	DESCRIPTION
-----	-----	-----	-----
IDSCPH	0	0	Discontinuity Diagnostic ---- phase number.
IDSCND	0	0	Discontinuity Diagnostic ---- phase end.
IDSCVR	0	0	Discontinuity Diagnostic ---- variable number.
IDSCFN	0	0	Discontinuity Diagnostic ---- function number.
IDTSFD	-1	-1	Derivative type (1 or 2-sided) for SFD.
IPFAUX	0	0	Print level for AUX print.
IPFSFD	0	0	Print level for SFD print.
IPRSFD	1	1	Perturbation Size Option
*IPGRD	10	5	Print level for OCP print.
IPNLP	10	10	Print level for NLP print.
IPODE	0	0	Print level for ODE print.
IPUAUX	0	0	FORTTRAN unit number for auxiliary print.
IPOCP	6	6	FORTTRAN unit number for OCP print (user opened).
IRSTRT	0	0	Restart flag.
ISCALE	0	0	Automatic Scaling Option.
ISFHES	41	41	FORTTRAN unit number for scratch Hessian file.
ISFINP	42	42	FORTTRAN unit number for scratch input file.
ISFRST	43	43	FORTTRAN unit number for restart file output.
ISFSCL	44	44	FORTTRAN unit number for scale weight input.
ITSWCH	2	2	Mesh refinement iteration to switch order.
M5DTYP	0	0	Method 5 Derivative Type.
MITODE	5	5	Maximum number of grids used by OCP.
MTSWCH	-1	-1	High order method for mesh refinement.
NSSWCH	-1	-1	High order stage number for mesh refinement.
NPTAUX	100	100	Number of AUX print points per phase.
NIWOPI	10000000	10000000	Initial integer storage alloc for optimization.
NRWOPI	20000000	20000000	Initial real storage alloc for optimization.
-----	-----	-----	-----
DTAUX	0.000E+00	0.000E+00	Independent variable increment for AUX print.
OBJCTL	1.000E-05	1.000E-05	Performance index relative tolerance.
*ODETOL	1.000E-03	1.000E-07	Differential-algebraic equation relative tolerance
PRTMSD	1.490E-08	1.490E-08	Minimum perturbation size for sparse finite diff.
PRTMXD	1.000E-03	1.000E-03	Maximum perturbation size for sparse finite diff.
PRTSFD	1.000E-05	1.000E-05	Relative perturbation size for sparse finite diff.
SLMTOL	1.490E-11	1.490E-11	ODE small absolute tolerance.
TOLJSD	1.000E-06	1.000E-06	Jacobian row and column tolerance for print.
TOLMSA	1.490E-08	1.490E-08	Absolute integration tolerance (Method 5).
TOLMSR	1.490E-08	1.490E-08	Relative integration tolerance (Method 5).
-----	-----	-----	-----
SOCOUT			SOCX Output Control Menu
	A0	A0	Mathematical Optimal Control Interpretation.

	B0	B0	Right Hand Side Sparsity Pattern.
	C1	C1	Transcription Summary.
	D1	D1	SOCX I/O Parameters.
	E1	E1	Variable Grid Map.
	F1	F1	Constraint Grid Map.
	G0	G0	Optimal Control Scale Information.
	H1	H1	Index Sets and Sparsity Summary.
	I2	I2	Variables.
	J2	J2	Objective Function.
	K2	K2	Constraints.
	L1	L1	Control Analysis Constraint Error.
	M1	M1	Relative DAE Error.
	N0	N0	Dynamic Status Display.
	O0	O0	Dynamic Constraint Adjoints.
	P0	P0	Dynamic Variable Adjoints.
	Q1	Q1	Control Analysis Lagrangian Errors.
	R1	R1	Control Analysis Optimality Error.
	S1	S1	Grid Refinement Summary.
SPRTHS	SPARSE	SPARSE	Right Hand Side Sparsity Option.
NLPALG	SNLPMN	SNLPMN	Nonlinear Programming Algorithm.
NLPOMR	M	M	Nonlinear Programming Option for Mesh Refinement.
KEYDPL	.lueiLUE	.lueiLUE	Dynamic Status Display Key
			. Satisfied Inequality
			l Active Lower Bound
			u Active Upper Bound
			e Equality
			i Ignored
			L Violated Lower Bound
			U Violated Upper Bound
			E Violated Equality
			z Active Lower Bound with Zero Multiplier
			Z Active Upper Bound with Zero Multiplier
RHSTMP	RHSTMPLT	RHSTMPLT	Right Hand Side Sparsity Template File Prefix
RSTFIL	socx.restart	socx.restart	Restart File Name.
SCLFIL	scalewgt.fil	scalewgt.fil	Scale Weight File Name.
MRFMET	MINMAX	MINMAX	Mesh Refinement Method
			= MINMAX Minimize Maximum Error (mrfopt=0)
			= BISECT Interval Bisection (mrfopt=1)
			= EQUIDS Equidistribute Error (mrfopt=2)

VARIABLES:

PHASE	NT	NP	NST	GRID	NVAR
1	1	0	5	10	51

CONSTRAINTS:

PHASE	PT	CONT	NRHS	NCON
1	2	4	19	38

***** SOCX WARNING: IER = 1282 Optimal Control Transcription Procedure
 >>>> ROUTINE HJSFDI SET WARNING ERROR = 3

Curtis-Powell-Reid Algorithm
 NUMBER OF INDEX SETS = 3
 FE PER JACOBIAN (2-SIDED) = 6
 FE PER JACOBIAN/HESSIAN = 9

SPARSITY:

MATRIX	NROWS	NCOLS	NANNZ	NFDNZ	NZERO	%NZ
-----	-----	-----	-----	-----	-----	-----
JACOBIAN	38	51	72	112	184	9.49
GRADIENT	1	51	1	0	1	1.96
HESSIAN	51	51	0	82	82	6.18

```

-----
| Index Set Analysis:                                     |
|-----|
| Phase   1           Function           Nonzeros        |
|-----|
|         Differential      1             2              |
|         Differential      2             2              |
|         Differential      3             2              |
|         Differential      4             2              |
|         R. Endpoint       1             2              |
|         R. Endpoint       2             2              |
|-----|

*****
*
* PERTURBATION SIZES (INTERNAL)
*
* Smallest  5           Largest  5
*
* ( 26: 6.39327658E-06) ( 49: 1.00000000E-03)
* ( 31: 6.39327658E-06) ( 48: 1.00000000E-03)
* ( 21: 9.15211012E-06) ( 47: 1.00000000E-03)
* ( 36: 9.15225488E-06) ( 45: 1.00000000E-03)
* ( 16: 1.10765477E-05) ( 44: 1.00000000E-03)
*
*

Feasible Point
It  Qit Nkt Ndof  KT Cond  Step  Norm p  Violtn
1   4   1   8   8.3E+05  1.0E+00  3.2E+01  3.5E+01
2  17   1   2   3.7E+05  1.0E+00  2.2E+01  6.1E+00
3  18   1   7   1.1E+05  1.0E+00  3.6E+00  2.2E+00
4   7   1   5   5.0E+04  2.9E-01  2.2E+00  1.5E+00
5   4   1   6   1.1E+05  1.8E-01  2.0E+00  1.2E+00
6   5   1   8   8.1E+04  2.6E-01  1.7E+00  9.6E-01
7   4   1   9   7.5E+04  5.2E-01  1.2E+00  6.6E-01
8   5   1   7   4.2E+04  4.3E-01  7.4E-01  4.8E-01
9   5   1   9   9.9E+04  5.0E-01  5.4E-01  3.5E-01
10  6   1   8   6.7E+04  4.2E-01  3.1E-01  2.4E-01
11  4   1   9   1.0E+05  4.7E-01  2.2E-01  1.8E-01
12  3   1   9   5.7E+04  1.0E+00  4.3E-02  1.6E-01
13  3   1   9   1.0E+05  1.0E+00  1.2E-02  2.9E-02
14  3   1   9   1.1E+05  1.0E+00  3.7E-04  8.9E-04
15  3   1   9   1.1E+05  1.0E+00  2.9E-07  7.3E-07
16  0   0   9   1.1E+05  1.0E+00  2.9E-07  4.4E-11

Optimization
It  Qit Nkt Ndof  KT Cond  Step  Norm p  Violtn  Levnbrg  Penalty  Norm Pg  Merit Function
(1)  4   1   8   7.5E+06  1.0E+00  3.8E+01  4.1E-11  0.0E+00  1.5E-08  5.3E-03  6.246559E-01
(2)  5   1   6   6.8E+05  1.0E+00  1.1E+01  7.5E+00  0.0E+00  5.3E-02  1.9E-01  5.192432E+00
(3)  5   1   8   3.9E+05  3.7E-01  1.8E+01  7.1E+00  3.0E-08  2.6E-02  1.8E-01  2.498865E+00
(4)  3   1   8   3.0E+05  1.0E+00  3.0E+00  2.7E+00  3.0E-08  1.5E-08  7.5E-02  5.445220E-01
(5)  3   1   8   2.3E+05  1.0E+00  3.1E+00  4.9E-01  1.5E-08  3.4E-02  5.1E-02  5.357776E-01
(6)  1   1   8   2.0E+05  1.0E+00  6.8E-02  1.5E-08  1.5E-08  2.2E-03  5.272229E-01
[7]  1   1   8   2.1E+05  1.0E+00  4.8E-04  1.1E-04  0.0E+00  1.2E-01  1.3E-05  5.272196E-01
[8]  0   0   8   2.1E+05  1.0E+00  4.8E-04  2.4E-09  0.0E+00  1.2E-01  9.9E-11  5.272196E-01

-----
|
| SPRNLP ALGORITHM PERFORMANCE STATISTICS
|
| Total CPU Time.....+9.000000E-03
|
| Number of Function Calls.....39
| Number of Gradient Calls.....24
| Number of Hessian Calls.....6
| Total Number of Function Evaluations.....153
|
| Storage Needed in HOLD Array.....2194
|
|-----

```

=====

OPTIMAL CONTROL ANALYSIS GRID 1:

NLP SOLUTION:

FUNCTION EVALUATION NUMBER 154

=====

|||||-----|||||

||||| Phase 1 |||||

|||||-----|||||

=====

Parameters

TFINAL

5.2721958E-01

Dynamic Variables

Point	TIME	RANGE	ALTITUDE	RNG-RATE	ALT-RATE	PITCH
1	0.0000000E+00	0.0000000E+00	0.0000000E+00	0.0000000E+00	0.0000000E+00	4.9492941E+01
2	5.8579953E-02	1.1522824E-01	1.2702873E-01	3.9340502E+00	4.3369351E+00	4.6084478E+01
3	1.1715991E-01	4.7300980E-01	4.9542447E-01	8.2811032E+00	8.2406012E+00	3.7763383E+01
4	1.7573986E-01	1.1023403E+00	1.0697172E+00	1.3205104E+01	1.1366540E+01	2.7054416E+01
5	2.3431981E-01	2.0355887E+00	1.7951445E+00	1.8657277E+01	1.3400542E+01	1.3862918E+01
6	2.9289977E-01	3.2976002E+00	2.5992322E+00	2.4429526E+01	1.4052117E+01	-9.8228634E-01
7	3.5147972E-01	4.8970480E+00	3.3977195E+00	3.0177820E+01	1.3209336E+01	-1.5699605E+01
8	4.1005967E-01	6.8227819E+00	4.1072522E+00	3.5569378E+01	1.1015084E+01	-2.8591000E+01
9	4.6863963E-01	9.0484621E+00	4.6575041E+00	4.0418398E+01	7.7712722E+00	-3.8971059E+01
10	5.2721958E-01	1.1545349E+01	5.0000000E+00	4.4828761E+01	3.9220084E+00	-4.3256352E+01

Objective Function

OBJ = 5.2721958E-01

Differential Constraints

Intrvl	RANGE_d	ALTITUDE_d	RNG-RATE_d	ALT-RATE_d
1	-9.2356123E-12	5.2607224E-13	-9.0761149E-11	4.8251725E-10
2	-2.8611197E-11	3.5327991E-12	-1.4550861E-11	5.6787466E-10
3	-3.6054493E-11	1.8485907E-11	6.7145040E-10	1.2121236E-09
4	-2.9491076E-11	4.4170223E-11	7.2250206E-10	1.1003027E-09
5	-2.4859448E-11	5.6000760E-11	5.7712835E-10	-1.7525370E-10
6	-1.6594726E-11	4.5635162E-11	8.7336638E-10	-8.4545440E-10
7	1.4974688E-12	2.6043612E-11	5.7184757E-10	-9.5454632E-10
8	1.5273116E-11	8.8784535E-12	1.9160173E-10	-7.1802584E-10
9	9.2286179E-12	1.1612933E-12	-4.5148885E-11	-4.4747446E-10

Point Constraints

PHSCON01

PHSCON02

4.5000000E+01 5.0000000E+00

Discretization Error Analysis

|||||-----|||||

||||| Phase 1 |||||

|||||-----|||||

=====

Interval	TIME	Differential	Algebraic
----------	------	--------------	-----------

```

5  2.6360979E-01      (2)<9.27E-04>      (-) 0.00E+00
6  3.2218974E-01      (2)<9.15E-04>      (-) 0.00E+00
4  2.0502984E-01      (2)<7.77E-04>      (-) 0.00E+00
7  3.8076970E-01      (2)<7.51E-04>      (-) 0.00E+00
3  1.4644988E-01      (2)<5.69E-04>      (-) 0.00E+00
8  4.3934965E-01      (2)<5.43E-04>      (-) 0.00E+00
2  8.7869930E-02      (2)<3.90E-04>      (-) 0.00E+00
9  4.9792960E-01      (2)<2.03E-04>      (-) 0.00E+00
1  2.9289977E-02      (2)<1.44E-04>      (-) 0.00E+00
-----
Maximum Differential Equation Error      9.2725167E-04
Maximum Algebraic Equation Error      0.0000000E+00
-----
Intervals      Dominated By:
-----
9      Differential Equation.....2
-----

=====

GRID   NPT   NFC   NGC   NHC   NFE   NRHS   ERREQN   CPU TIME
1      10    40    24    6     154   2178   9.2725E-04  1.4000E-02

ANALYSIS OF GRID 1 ERRORS:
RELATIVE ERROR ESTIMATES:
              MINIMUM      AVERAGE      MAXIMUM

PHASE 1      1.44329E-04      5.80098E-04      9.27252E-04
>>>(MAX ERROR)/(OLD ERROR) = 1.59844E+00
(MAX ERROR)/(AVG ERROR) = 1.59844E+00
MAXIMUM PREDICTED ERROR = 1.15906E-04
MAXIMUM NUMBER OF INTERVAL SUBDIVISIONS = 1
SMALLEST INTERVAL = 2.92900E-02      ( 3.22190E-01, 3.51480E-01)
LARGEST INTERVAL = 2.92900E-02      ( 1.17160E-01, 1.46450E-01)

=====
=====
=====
===== REFINEMENT ITERATION 2 =====
=====
=====

TRANSCRIPTION SUMMARY:
Phase      Discretization      Abrv      Method      Stage      Order      Grid Pts
-----
1      Trapezoidal      TRP      2      1      2      19
-----

VARIABLES:
PHASE  NT   NP   NST   GRID   NVAR
1      1    0    5     19     96

CONSTRAINTS:
PHASE  PT   CONT   NRHS   NCON
1      2    4     37     74

***** SOCX WARNING: IER = 1282      Optimal Control Transcription Procedure
>>>> ROUTINE HJSFDI SET WARNING ERROR = 3

Curtis-Powell-Reid Algorithm
NUMBER OF INDEX SETS      = 3
FE PER JACOBIAN (2-SIDED) = 6

```


FE PER JACOBIAN/HESSIAN = 9

SPARSITY:

MATRIX	NROWS	NCOLS	NANNZ	NFDNZ	NZERO	%NZ
JACOBIAN	74	96	144	220	364	5.12
GRADIENT	1	96	1	0	1	1.04
HESSIAN	96	96	0	154	154	3.31

```

-----
| Index Set Analysis:                                     |
|-----|
| Phase 1          Function          Nonzeros          |
|-----|
|      Differential      1              2              |
|      Differential      2              2              |
|      Differential      3              2              |
|      Differential      4              2              |
|      R. Endpoint      1              2              |
|      R. Endpoint      2              2              |
|-----|

```

```

*****
*
*      PERTURBATION SIZES (INTERNAL)
*
*      Smallest 5          Largest 5
*
*      ( 56: 4.26516231E-06)  ( 93: 1.00000000E-03)
*      ( 51: 6.56937675E-06)  ( 92: 1.00000000E-03)
*      ( 61: 7.08842988E-06)  ( 90: 1.00000000E-03)
*      ( 46: 8.30189966E-06)  ( 89: 1.00000000E-03)
*      ( 66: 8.64540227E-06)  ( 88: 1.00000000E-03)
*
*

```

Optimization											
It	Qit	Nkt	Ndof	KT Cond	Step	Norm p	Violtn	Levnbrg	Penalty	Norm Pg	Merit Function
(1)	4	1	17	3.8E+02	1.0E+00	4.9E-02	5.5E-04	0.0E+00	1.5E-08	3.2E-01	7.198585E-01
(2)	3	1	17	4.9E+01	4.1E-01	8.0E-03	3.2E-04	0.0E+00	1.3E+00	5.4E-04	7.198223E-01
(3)	3	1	17	4.9E+01	1.0E+00	1.3E-05	9.4E-07	0.0E+00	1.5E-08	2.5E-06	7.198220E-01
(4)	0	0	17	4.9E+01	1.0E+00	1.3E-05	4.0E-12	0.0E+00	1.5E-08	8.9E-09	7.198220E-01

```

-----
|
|      SPRNLP ALGORITHM PERFORMANCE STATISTICS
|
|      Total CPU Time.....+1.000001E-03
|
|      Number of Function Calls.....5
|      Number of Gradient Calls.....5
|      Number of Hessian Calls.....2
|      Total Number of Function Evaluations.....38
|
|      Storage Needed in HOLD Array.....4175
|
|-----|

```

=====

OPTIMAL CONTROL ANALYSIS GRID 2:

NLP SOLUTION:

FUNCTION EVALUATION NUMBER 39

```

=====
|||||-----|||||
|||||Phase 1|||||
|||||-----|||||

```

```

=====
===== Parameters =====
-----
TFINAL
-----
5.2551660E-01
=====
===== Dynamic Variables =====
-----
Point      TIME      RANGE      ALTITUDE      RNG-RATE      ALT-RATE      PITCH
-----
1      0.0000000E+00  0.0000000E+00  0.0000000E+00  0.0000000E+00  0.0000000E+00  5.0352791E+01
2      2.9195366E-02  2.7631320E-02  3.2442454E-02  1.8928565E+00  2.2224386E+00  4.8804969E+01
3      5.8390733E-02  1.1189200E-01  1.2853380E-01  3.8793386E+00  4.3602049E+00  4.5396556E+01
4      8.7586099E-02  2.5606839E-01  2.8512926E-01  5.9973243E+00  6.3672140E+00  4.1521279E+01
5      1.1678147E-01  4.6410838E-01  4.9800802E-01  8.2542511E+00  8.2158369E+00  3.7119748E+01
6      1.4597683E-01  7.4012924E-01  7.6206860E-01  1.0654288E+01  9.8733750E+00  3.2140508E+01
7      1.7517220E-01  1.0882902E+00  1.0711876E+00  1.3196137E+01  1.1302517E+01  2.6552808E+01
8      2.0436756E-01  1.5125953E+00  1.4181096E+00  1.5870467E+01  1.2463041E+01  2.0363986E+01
9      2.3356293E-01  2.0166252E+00  1.7944124E+00  1.8657611E+01  1.3315212E+01  1.3638118E+01
10     2.6275830E-01  2.6032213E+00  2.1905946E+00  2.1526581E+01  1.3824863E+01  6.5080660E+00
11     2.9195366E-01  3.2741767E+00  2.5963235E+00  2.4436558E+01  1.3969192E+01  -8.2920383E-01
12     3.2114903E-01  4.0300125E+00  3.0008337E+00  2.7341237E+01  1.3741387E+01  -8.1395221E+00
13     3.5034440E-01  4.8699089E+00  3.3934167E+00  3.0195042E+01  1.3152126E+01  -1.5193730E+01
14     3.7953976E-01  5.7918128E+00  3.7638968E+00  3.2959085E+01  1.2227250E+01  -2.1807733E+01
15     4.0873513E-01  6.7926882E+00  4.1030000E+00  3.5604902E+01  1.1002687E+01  -2.7864494E+01
16     4.3793050E-01  7.8688324E+00  4.4025643E+00  3.8115305E+01  9.5186698E+00  -3.3314163E+01
17     4.6712586E-01  9.0161856E+00  4.6555957E+00  4.0482997E+01  7.8149972E+00  -3.8159650E+01
18     4.9632123E-01  1.0230583E+01  4.8562122E+00  4.2708126E+01  5.9280372E+00  -4.2437744E+01
19     5.2551660E-01  1.1508419E+01  5.0000000E+00  4.4828761E+01  3.9220084E+00  -4.4380622E+01
=====
===== Objective Function =====
-----
OBJ = 5.2551660E-01
=====
===== Differential Constraints =====
-----
Intrvl     RANGE_d      ALTITUDE_d      RNG-RATE_d      ALT-RATE_d
-----
1      2.3085700E-14  2.9165038E-14  2.7201366E-11  3.7071841E-11
2      5.2645388E-14  8.3821838E-14  1.4227369E-11  1.8105126E-11
3      5.0917603E-14  1.2833484E-13  1.3426760E-12  1.2359493E-12
4      3.3140157E-14  1.6021906E-13  8.1035179E-12  8.0040488E-12
5      1.2906343E-14  1.8970936E-13  1.4122453E-11  1.2433037E-11
6      1.4988011E-15  2.3189783E-13  4.7114812E-11  2.8171473E-11
7      -9.9920072E-16  2.8532732E-13  4.5371207E-11  2.5552487E-11
8      -3.3306691E-15  3.3362202E-13  3.0100922E-11  1.0580135E-11
9      -3.9968029E-15  3.6748382E-13  2.5884295E-11  7.8161028E-12
10     -2.6645353E-15  3.7414516E-13  7.1592732E-12  -1.1685339E-12
11     -1.9984014E-15  3.5138559E-13  1.8090529E-11  -3.8120355E-12
12     1.1324275E-14  3.0475622E-13  7.6069151E-11  -2.3321649E-11
13     3.0642155E-14  2.3925306E-13  8.0635498E-11  -2.8463607E-11
14     4.6629367E-14  1.7597035E-13  5.4646287E-11  -3.1019264E-11
15     5.8619776E-14  1.2156942E-13  4.4159676E-11  -2.7777195E-11
16     5.1958438E-14  8.5487173E-14  8.5731422E-12  -8.0479710E-12
17     3.3306691E-14  5.3845817E-14  3.1324943E-12  -3.3217415E-12
18     1.0658141E-14  1.8318680E-14  8.1934459E-13  -1.3345189E-12
=====
===== Point Constraints =====
-----
PHSCON01      PHSCON02
-----
4.5000000E+01  5.0000000E+00
=====
===== Discretization Error Analysis =====
-----
|||||-----|||||

```

```

||||||||||||||||||||||||||||||||||||||||| Phase 1 |||||||||||||||||||||||||||||||||||||||
|||||||||||||||||||||||||||||||||||||||||-----|||||||||||||||||||||||||||||||||||||
=====

```

Interval	TIME	Differential	Algebraic
10	2.7735598E-01	(2)<1.15E-04>	(-) 0.00E+00
11	3.0655135E-01	(2)<1.14E-04>	(-) 0.00E+00
9	2.4816061E-01	(2)<1.10E-04>	(-) 0.00E+00
12	3.3574671E-01	(2)<1.08E-04>	(-) 0.00E+00
8	2.1896525E-01	(2)<1.01E-04>	(-) 0.00E+00
13	3.6494208E-01	(2)<9.84E-05>	(-) 0.00E+00
7	1.8976988E-01	(2)<8.90E-05>	(-) 0.00E+00
14	3.9413745E-01	(2)<8.62E-05>	(-) 0.00E+00
6	1.6057452E-01	(2)<7.64E-05>	(-) 0.00E+00
15	4.2333281E-01	(2)<7.36E-05>	(-) 0.00E+00

```

-----
Maximum Differential Equation Error      1.1491040E-04
Maximum Algebraic Equation Error        0.0000000E+00
-----

```

```

Intervals      Dominated By:
-----

```

```

18      Differential Equation.....2
-----

```

```

=====
GRID  NPT  NFC  NGC  NHC  NFE  NRHS      ERREQN      CPU TIME
2      19    6    5    2   39   1549    1.1491E-04    3.0000E-03

```

```

ANALYSIS OF GRID 2 ERRORS:
RELATIVE ERROR ESTIMATES:

```

	MINIMUM	AVERAGE	MAXIMUM
PHASE 1	1.57473E-05	7.33919E-05	1.14910E-04
(MAX ERROR)/(AVG ERROR) =	1.56571E+00		
MAXIMUM PREDICTED ERROR =	-1.00000E+00		
MAXIMUM NUMBER OF INTERVAL SUBDIVISIONS =	0		
SMALLEST INTERVAL =	2.91954E-02	(3.21149E-01, 3.50344E-01)	
LARGEST INTERVAL =	2.91954E-02	(1.16781E-01, 1.45977E-01)	

```

=====
=====
=====
===== REFINEMENT ITERATION 3 =====
=====
=====
=====

```

```

TRANSCRIPTION SUMMARY:

```

Phase	Discretization	Abrv	Method	Stage	Order	Grid Pts
1	Hermite-Simpson (Compressed)	HSC	3	1	4	19

```

VARIABLES:

```

PHASE	NT	NP	NST	GRID	NVAR
1	1	0	5	19	114

```

CONSTRAINTS:

```

PHASE	PT	CONT	NRHS	NCON
1	2	4	55	74

```

***** SOCX WARNING: IER = 1282      Optimal Control Transcription Procedure
>>>>> ROUTINE HJSFDI SET WARNING ERROR = 3

```

```

Coleman-More Algorithm
NUMBER OF INDEX SETS      =    6
FE PER JACOBIAN (2-SIDED) =   12
FE PER JACOBIAN/HESSIAN   =   27

```

SPARSITY:

MATRIX	NROWS	NCOLS	NANNZ	NFDNZ	NZERO	%NZ
JACOBIAN	74	114	144	364	508	6.02
GRADIENT	1	114	1	0	1	0.88
HESSIAN	114	114	0	462	462	7.05

```

-----
| Index Set Analysis:                                     |
|-----|-----|-----|
| Phase 1          Function      Nonzeros |
|-----|-----|-----|
|      Differential      1          6      |
|      Differential      2          6      |
|      Differential      3          4      |
|      Differential      4          4      |
|      R. Endpoint      1          2      |
|      R. Endpoint      2          2      |
|-----|-----|-----|

```

```

*****
*
*      PERTURBATION SIZES (INTERNAL)
*
*      Smallest 5          Largest 5
*
*      ( 61: 1.08147334E-05)  ( 111: 1.00000000E-03)
*      ( 67: 1.10786392E-05)  ( 110: 1.00000000E-03)
*      ( 55: 1.19188132E-05)  ( 107: 1.00000000E-03)
*      ( 73: 1.21465601E-05)  ( 106: 1.00000000E-03)
*      ( 49: 1.28844963E-05)  ( 105: 1.00000000E-03)
*
*****

```

It	Qit	Nkt	Ndof	KT Cond	Step	Optimization					Merit Function
						Norm p	Violtn	Levnbrg	Penalty	Norm Pg	
(1)	4	1	35	1.3E+02	1.0E+00	1.5E-02	1.8E-04	0.0E+00	1.5E-08	1.2E-03	7.190490E-01
(2)	3	1	35	6.5E+01	1.0E+00	6.4E-05	2.8E-05	0.0E+00	1.6E+00	6.8E-06	7.190407E-01
(3)	0	0	35	6.5E+01	1.0E+00	6.4E-05	1.5E-10	0.0E+00	1.6E+00	4.1E-08	7.190407E-01

```

-----
|
|      SPRNLP ALGORITHM PERFORMANCE STATISTICS
|
|      Total CPU Time.....+3.000000E-03
|
|      Number of Function Calls.....3
|      Number of Gradient Calls.....4
|      Number of Hessian Calls.....1
|      Total Number of Function Evaluations.....60
|
|      Storage Needed in HOLD Array.....4794
|
|-----

```

=====

OPTIMAL CONTROL ANALYSIS GRID 3:

NLP SOLUTION:

FUNCTION EVALUATION NUMBER 61

```

=====
|||||-----|||||
||||| Phase 1 |||||
|||||-----|||||
=====
Parameters =====
-----
TFINAL
-----
5.2494620E-01
=====
Dynamic Variables =====
-----
Point      TIME      RANGE      ALTITUDE      RNG-RATE      ALT-RATE      PITCH
-----
1  0.0000000E+00  0.0000000E+00  0.0000000E+00  0.0000000E+00  0.0000000E+00  5.1579359E+01
2  2.9163678E-02  2.6985132E-02  3.2863426E-02  1.8701370E+00  2.2373735E+00  4.8575906E+01
3  5.8327356E-02  1.1026585E-01  1.2945200E-01  3.8622226E+00  4.3667600E+00  4.5167949E+01
4  8.7491034E-02  2.5353132E-01  2.8629692E-01  5.9851889E+00  6.3655045E+00  4.1297473E+01
5  1.1665471E-01  4.6071068E-01  4.9920088E-01  8.2463377E+00  8.2062227E+00  3.6906801E+01
6  1.4581839E-01  7.3590345E-01  7.6309514E-01  1.0649740E+01  9.8565387E+00  3.1946445E+01
7  1.7498207E-01  1.0832551E+00  1.0718981E+00  1.3194074E+01  1.1279526E+01  2.6387573E+01
8  2.0414575E-01  1.5067633E+00  1.4184029E+00  1.5870093E+01  1.2435364E+01  2.0238808E+01
9  2.3330942E-01  2.0100113E+00  1.7942379E+00  1.8658330E+01  1.3284670E+01  1.3564037E+01
10 2.6247310E-01  2.5958509E+00  2.1899483E+00  2.1528086E+01  1.3793510E+01  6.4937285E+00
11 2.9163678E-01  3.2660886E+00  2.5952397E+00  2.4438854E+01  1.3939163E+01 -7.7984442E-01
12 3.2080046E-01  4.0212557E+00  2.9993782E+00  2.7344625E+01  1.3714760E+01 -8.0285001E+00
13 3.4996414E-01  4.8605364E+00  3.3916840E+00  3.0200082E+01  1.3130815E+01 -1.5028614E+01
14 3.7912781E-01  5.7818772E+00  3.7620133E+00  3.2966546E+01  1.2212903E+01 -2.1599651E+01
15 4.0829149E-01  6.7822427E+00  4.1011280E+00  3.5615694E+01  1.0996607E+01 -2.7625644E+01
16 4.3745517E-01  7.8579354E+00  4.4009033E+00  3.8130390E+01  9.5217726E+00 -3.3055830E+01
17 4.6661885E-01  9.0049062E+00  4.6543792E+00  4.0503295E+01  7.8278200E+00 -3.7891134E+01
18 4.9578252E-01  1.0219006E+01  4.8557016E+00  4.2734443E+01  5.9507935E+00 -4.2166089E+01
19 5.2494620E-01  1.1496162E+01  5.0000000E+00  4.4828761E+01  3.9220084E+00 -4.5932816E+01
=====
Objective Function =====
-----
OBJ = 5.2494620E-01
=====
Differential Constraints =====
-----
Intrvl  RANGE_d      ALTITUDE_d      RNG-RATE_d      ALT-RATE_d
-----
1  4.6329529E-11  2.0355423E-11  2.0596618E-09  1.0317580E-09
2  4.4255641E-11  3.3788805E-11 -1.4567222E-10  5.1003799E-10
3  3.3936021E-11  3.5936774E-11 -1.2110313E-10  4.1833004E-10
4  2.4649449E-11  3.6992215E-11 -9.7017394E-11  3.2389738E-10
5  1.6038171E-11  3.6730341E-11 -7.2824247E-11  2.3588655E-10
6  7.5676132E-12  3.5211140E-11 -4.9522886E-11  1.6330057E-10
7 -1.3682389E-12  3.2915892E-11 -2.9941050E-11  1.1338865E-10
8 -1.1229684E-11  3.0787872E-11 -1.6122104E-11  9.0291717E-11
9 -2.2052027E-11  3.0042913E-11 -4.9035775E-12  9.1217147E-11
10 -3.3265390E-11  3.1602498E-11  9.0627506E-12  1.0339278E-10
11 -4.3847592E-11  3.5767500E-11  2.3601121E-11  1.0640605E-10
12 -5.2859717E-11  4.1754933E-11  2.7338132E-11  8.4140571E-11
13 -5.9994676E-11  4.8107074E-11  1.1585732E-11  3.0492198E-11
14 -6.5663475E-11  5.3303473E-11 -1.8869906E-11 -5.2682085E-11
15 -7.0728756E-11  5.6276650E-11 -5.1328941E-11 -1.6176228E-10
16 -7.6160411E-11  5.6589733E-11 -7.7093332E-11 -2.9140315E-10
17 -8.2764462E-11  5.4388716E-11 -9.6138097E-11 -4.3165480E-10
18 -7.2420736E-11  3.2602254E-11  3.1404729E-09 -1.4088352E-09
=====
Point Constraints =====
-----
PHSCON01  PHSCON02
-----
4.5000000E+01  5.0000000E+00
=====

```

```

===== Discretization Error Analysis =====
-----
|||||-----|||||
||||| Phase 1 |||||
|||||-----|||||
-----
Interval      TIME      Differential      Algebraic
-----
      8  2.1872758E-01  (4)<1.59E-08>  (-) 0.00E+00
     13  3.6454597E-01  (4)<1.59E-08>  (-) 0.00E+00
      7  1.8956391E-01  (4)<1.41E-08>  (-) 0.00E+00
     12  3.3538230E-01  (4)<1.39E-08>  (-) 0.00E+00
     14  3.9370965E-01  (4)<1.33E-08>  (-) 0.00E+00
      9  2.4789126E-01  (4)<1.27E-08>  (-) 0.00E+00
      6  1.6040023E-01  (4)<9.98E-09>  (-) 0.00E+00
     15  4.2287333E-01  (3)<9.61E-09>  (-) 0.00E+00
      5  1.3123655E-01  (3)<9.09E-09>  (-) 0.00E+00
     16  4.5203701E-01  (3)<8.82E-09>  (-) 0.00E+00
-----
Maximum Differential Equation Error  1.5910343E-08
Maximum Algebraic Equation Error    0.0000000E+00
-----
Intervals      Dominated By:
-----
     10  Differential Equation.....3
      8  Differential Equation.....4
-----

=====

GRID  NPT  NFC  NGC  NHC  NFE  NRHS      ERREQN      CPU TIME
  3    19   4   4    1   61  4695    1.5910E-08    4.0000E-03

=====

OPTIMAL CONTROL PROGRAM SUMMARY

GRID  NPT  NFC  NGC  NHC  NFE  NRHS      ERREQN      CPU TIME
  1    10   40  24   6   154  2178    9.2725E-04    1.4000E-02
  2    19   6   5   2   39   1549    1.1491E-04    3.0000E-03
  3    19   4   4   1   61   4695    1.5910E-08    4.0000E-03
-----
      19   50  33   9  254   8422                2.1000E-02

=====

| Total Solution Time.....+2.100000E-02 |
| Total User Routine Time.....+9.999992E-04 |
| Total SOCX Computation Time.....+2.000000E-02 |
-----

```

7.3 Workshop Self-Instructional Examples

The following subsections provide a series of instructive example problems designed to acquaint the user with many of the primary features of the SOS library. For each exercise a working implementation is available to use as a “starting point.” For example in

`wrkprb7.f``wrkprb7.soln`

wrkprb7.f contains the FORTRAN file that should be used to initiate exercise 7. A “solution” file corresponding to each of the reference cases, which is denoted as “wrkprbxx.soln.” Thus the “answer” to the exercise 7 is found in the file “wrkprb7.soln.” Pertinent page references are also given in the exercise instructions to assist the user. It is recommended that the exercises be solved by modifying the initial implementation. It is also suggested that the problems be done sequentially in order to become familiar with the software capability.

7.3.1 Problem 1: Simplified Usage Dense NLP–DNLPFW

`wrkprb1.f``wrkprb1.soln`

Problem: Minimize

$$f(\mathbf{x}) = x_1^2 + x_2^2 + \log(x_1 x_2)$$

subject to the constraint

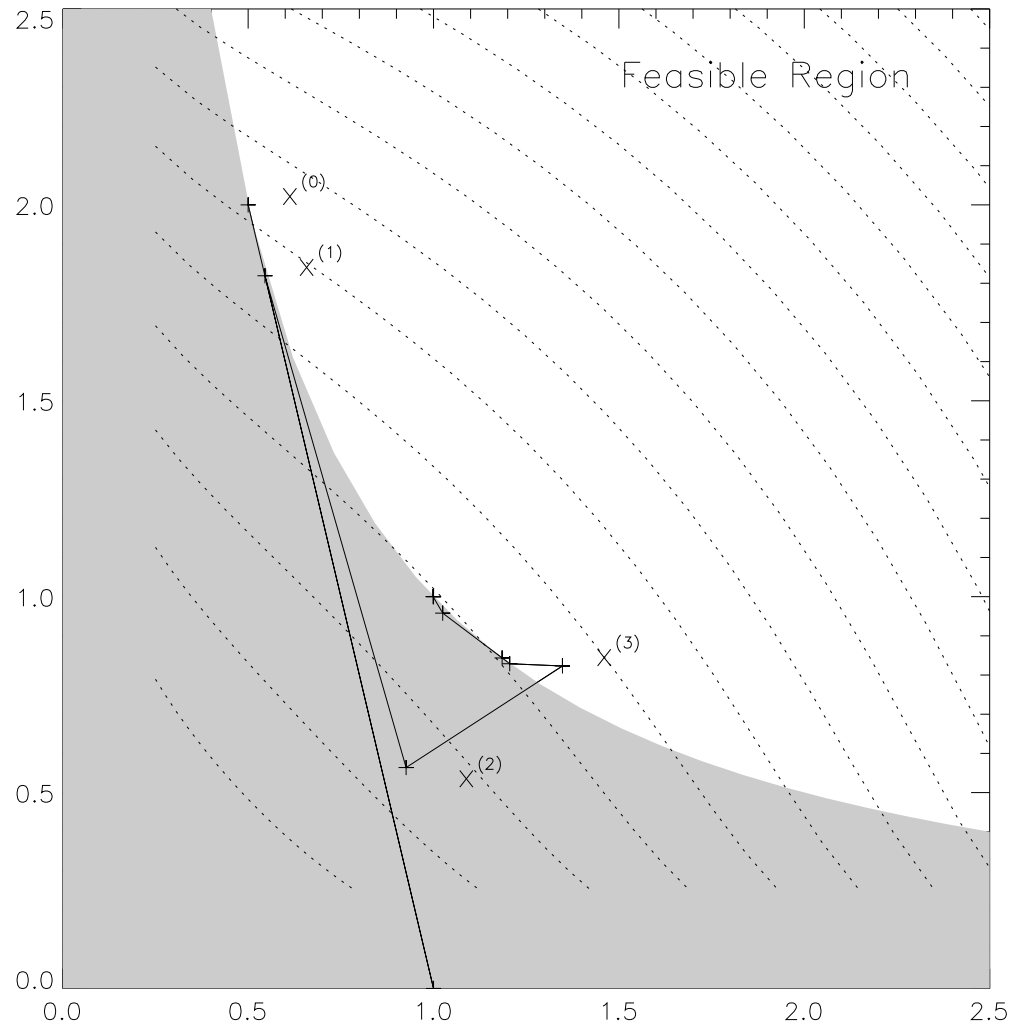
$$c_1(\mathbf{x}) = x_1 x_2 \geq 1$$

with bounds

$$0 \leq x_1 \leq 10$$

$$0 \leq x_2 \leq 10.$$

Initial guess: $\mathbf{x}^{(0)} = (.5, 2)$



Dense NLP Exercises

1. Modify the problem as follows:
 - (a) Change the initial guess to: $\mathbf{x}^{(0)} = (.1, .2)$
 - (b) Increase the NLP output level from “standard” to “interpretive” (p. 231).
 - (c) Use a BFGS quasi-Newton update instead of an SR1 update (p. 230).
2. Modify the previous case as follows:
 - (a) Use the Schur-complement sparse QP (p. 235).
 - (b) Use a finite difference Hessian (p. 230).
3. Use the default algorithm with “interpretive” line search output (p. 232) to solve the (modified) problem: Minimize

$$f(\mathbf{x}) = x_1^2 + x_2^2$$

subject to

$$x_1^2 x_2 - 1 \geq 0$$

$$x_2 \leq -.1$$

with no bounds on the variables. Use the initial guess: $\mathbf{x}^{(0)} = (.5, 2)$

7.3.2 Problem 2: Reverse Communication Dense NLP–DNLPRV

[wrkprb2.f](#)[wrkprb2.soln](#)

1. Modify the reference problem as follows:

- (a) Increase the NLP output level from “terse” to “interpretive” (p. 231).
- (b) Use the exact Hessian matrix (p. 230).
- (c) Change constraint number 1 from $x_1 x_2 \leq 1/2$ to

$$x_1 x_2^2 \leq 1/2$$

- (d) Display the quantity $x_1 x_2$ in addition to previous output using IREVRS(4)(p. 52).

2. Modify the previous case as follows:

- (a) Display the Jacobian and Hessian matrices (p. 231)
- (b) Set the iteration limit to one (p. 228).
- (c) Set the algorithm option to “M” (p. 234).

7.3.3 Problem 3: Sparse NLP–SNLPMN

[wrkprb3.f](#)[wrkprb3.soln](#)

A Sparse Jacobian Input Example

The Jacobian Matrix (3 constraints, 5 variables)

$$\mathbf{G} = \begin{pmatrix} 0 & 0 & 1.3 & 0 & 1.5 \\ 2.1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 3.3 & 0 & 0 \end{pmatrix}$$

The FORTRAN Input, (*Value, Row Index, Column Index*)

NONZG = 4

GMAT(1) = 2.1	IROWG(1) = 2	JCOLG(1) = 1
GMAT(2) = 1.3	IROWG(2) = 1	JCOLG(2) = 3
GMAT(3) = 3.3	IROWG(3) = 3	JCOLG(3) = 3
GMAT(4) = 1.5	IROWG(4) = 1	JCOLG(4) = 5

A Sparse Hessian Input Example

The Hessian Matrix (3 variables)

$$\mathbf{H} = \begin{pmatrix} 1.1 & 0 & \boxed{3.1} \\ 0 & 2.2 & 0 \\ 3.1 & 0 & 3.3 \end{pmatrix}$$

The FORTRAN Input, (*Value, Row Index, Column Start*)

Note: For NDIM variables, NONZH Hessian nonzeros,
JSTRH(NDIM+1) - 1 = NONZH

NONZH = 4

HMAT(1) = 1.1	IROWH(1) = 1	JSTRH(1) = 1
HMAT(2) = 3.1	IROWH(2) = 3	JSTRH(2) = 3
HMAT(3) = 2.2	IROWH(3) = 2	JSTRH(3) = 4
HMAT(4) = 3.3	IROWH(4) = 3	JSTRH(4) = 5

Sparse NLP Exercises

1. Modify the problem as follows:

(a) Display the Jacobian and Hessian sparsity pattern (p. 232).

2. Modify the previous case as follows:

(a) Increase the NLP output level from “terse” to “interpretive” (p. 231).

- (b) Change constraint number 1 from $x_1x_2 \leq 1/2$ to

$$x_1x_4 \leq 1/2$$

7.3.4 Problem 4: Sparse Least Squares–SNLPLS

[wrkprb4.f](#)
[wrkprb4.soln](#)

Sparse Least Squares Exercises

1. Modify the reference problem as follows:
 - (a) Display the Jacobian and Hessian sparsity pattern (p. 232)
 - (b) Increase the NLP output level from “terse” to “standard” (p. 231).
2. Modify the previous case as follows:
 - (a) Use the linear least squares algorithm option (p. 234)
 - (b) Change constraint number 1 from $x_1x_2 \leq 1/2$ to

$$x_1 + x_2 \leq 1/2$$

- (c) Change the residual vector to

$$\mathbf{r} = \begin{bmatrix} 10(x_2 - 2x_1) \\ (1 - x_1) \\ 10(x_4 - 2x_3) \\ (1 - x_3) \end{bmatrix}$$

7.3.5 Problem 5: Reverse Communication NLP, Finite Difference Gradients

[wrkprb5.f](#)
[wrkprb5.soln](#)

Exercises

1. Modify the problem as follows:
 - (a) Use the sparse QP (p. 235)
 - (b) Use a finite difference Hessian (p. 230)
 - (c) Set the KKT condition number tolerance to 1.d10 (p. 233)
 - (d) Increase the NLP output level from “terse” to “standard” (p. 231).
 - (e) Change constraint number 1 to

$$x_1 - x_3 = 0$$

- (f) Change the objective function to

$$f(x) = (x_1 - x_2)^2 + (x_2 - x_3)^4$$

7.3.6 Problem 6: Sparse NLP with Sparse Finite Differences

[wrkprb6.f](#)[wrkprb6.soln](#)

Exercises

1. Modify the problem as follows:
 - (a) Increase the sparse difference output level (i.e. set MSGVL=3) (p. 298)
 - (b) Increase the NLP output level from “terse” to “standard” (p. 231).
 - (c) Display the sparsity pattern (p. 232)

7.3.7 Problem 7: One Phase Example

[wrkprb7.f](#)[wrkprb7.soln](#)

Linear Tangent Steering Example Ref. [12]

Solve the following problem:

Objective: Minimize the final time t_f

State Equations:

$$\begin{aligned}\dot{y}_1 &= y_3 \\ \dot{y}_2 &= y_4 \\ \dot{y}_3 &= a \cos u \\ \dot{y}_4 &= a \sin u\end{aligned}$$

where $a = 100$ and

$$-\frac{\pi}{2} \leq u(t) \leq \frac{\pi}{2}$$

Boundary Conditions: $y_1(0) = y_2(0) = y_3(0) = y_4(0) = 0$ and $y_2(t_f) = 5$, $y_3(t_f) = 45$, $y_4(t_f) = 0$.

Setting SOS Parameters—Exercises

1. Modify the example problem as follows:
 - (a) Set the relative accuracy in the differential equations to 1.d-7 (p. 404).
 - (b) Change the optimal control output level to “interpretive” (i.e. level 20) (p. 408)
2. Modify the previous case as follows:
 - (a) Eliminate the SOCX output for menu items; (e, f, g, k, l, m, n, o, p, q, r) (p. 410).
 - (b) Display the NLP Jacobian and Hessian sparsity (p. 232)

7.3.8 Problem 8: User Routines Example

[wrkprb8.f](#)
[wrkprb8.soln](#)

The User Routines—Exercises

1. Modify the previous case to make the final differential equation (p. 455)

$$\dot{y}_4 = a \sin u - g_0 \left[\frac{h_0}{y_2 + h_0} \right]^2$$

where $g_0 = 32.2$ and $h_0 = 3500$

2. Modify the previous case to impose the boundary condition (p. 426)

$$y_2(t_f) = 6$$

3. Modify the previous case to impose the control path bounds (p. 426)

$$-\frac{\pi}{3} \leq u(t) \leq \frac{\pi}{3}$$

and display the inequality switching times (p. 410).

7.3.9 Problem 9: Path Constraint Example

[wrkprb9.f](#)
[wrkprb9.soln](#)

Path Constraints—Exercise (p. 455), (p. 506)

Solve the following problem:

Objective: Minimize the final time t_f

State Equations:

$$\begin{aligned} \dot{y}_1 &= y_3 \\ \dot{y}_2 &= y_4 \\ \dot{y}_3 &= a \cos u \\ \dot{y}_4 &= a \sin u \end{aligned}$$

where $a = 100$ and

$$-\frac{\pi}{2} \leq u(t) \leq \frac{\pi}{2}$$

Path Constraint:

$$a \sin u(t) \leq 60$$

Boundary Conditions: $y_1(0) = y_2(0) = y_3(0) = y_4(0) = 0$ and $y_2(t_f) = 5$, $y_3(t_f) = 45$, $y_4(t_f) = 0$.

7.3.10 Problem 10: Quadrature Example[wrkprb10.f](#)[wrkprb10.soln](#)**Quadrature Functions–Exercise** (p. 426), (p. 455)

Solve the following problem:

Objective: Minimize the “energy”

$$J = \int_0^{t_f} u^2(t) dt$$

State Equations:

$$\begin{aligned}\dot{y}_1 &= y_3 \\ \dot{y}_2 &= y_4 \\ \dot{y}_3 &= a \cos u \\ \dot{y}_4 &= a \sin u\end{aligned}$$

where $a = 100$ and

$$-\frac{\pi}{2} \leq u(t) \leq \frac{\pi}{2}$$

Boundary Conditions: $y_1(0) = y_2(0) = y_3(0) = y_4(0) = 0$ and $y_2(t_f) = 5$, $y_3(t_f) = 45$, $y_4(t_f) = 0$.**7.3.11 Problem 11: Point Function Example**[wrkprb11.f](#)[wrkprb11.soln](#)**Point Functions–Exercise** (p. 426), (p. 450), (p. 500)

Solve the following problem:

Objective: Minimize the final time t_f

State Equations:

$$\begin{aligned}\dot{y}_1 &= y_3 \\ \dot{y}_2 &= y_4 \\ \dot{y}_3 &= a \cos u \\ \dot{y}_4 &= a \sin u\end{aligned}$$

where $a = 100$ and

$$-\frac{\pi}{2} \leq u(t) \leq \frac{\pi}{2}$$

Boundary Conditions: $y_1(0) = y_2(0) = y_3(0) = y_4(0) = 0$ and $y_3(t_f) = 45$, $y_4(t_f) = 0$ and

$$y_1(t_f)y_2(t_f) \geq 60$$

7.3.12 Problem 12: Initial Guess Example

[wrkprb12.f](#)
[wrkprb12.soln](#)

Initial Guess–Exercise (p. 426), (p. 422)

Solve the previous problem with the following modifications:

1. Compute “exact” initial guess (provided in subroutine EXACKT) (p. 429).
2. Set the discretization method to Hermite-Simpson (Compressed) (p. 428)
3. Set the initial number of grid points to 15 (p. 429)
4. Use NLP option “M” (p. 234)

7.3.13 Problem 13: Auxilliary Output Example

[wrkprb13.f](#)
[wrkprb13.soln](#)

Auxilliary Output–Exercise

Modify the previous problem to display the analytic solution at the grid points.

1. Use subroutine EXACKT to compute the analytic solution.
2. Use IPUAUX=6 (p. 401), to display auxilliary output interspersed with the standard output (p. 446)
3. Force the NLP to be a Newton method–i.e. recompute the Hessian every step! (p. 234)

7.3.14 Problem 14: Two Phase Example

[wrkprb14.f](#)

Two Phase Example (p. 426), (p. 455), (p. 481)

Phase 1: (Burn) $0 \leq t \leq t_1$

State Equations:

$$\begin{aligned}\dot{y}_1 &= y_3 \\ \dot{y}_2 &= y_4 \\ \dot{y}_3 &= a \cos u \\ \dot{y}_4 &= a \sin u - g_0 \left[\frac{h_0}{y_2 + h_0} \right]^2\end{aligned}$$

where $a = 100$, $g_0 = 32.2$ and $h_0 = 3500$ with $-\frac{\pi}{2} \leq u(t) \leq \frac{\pi}{2}$

Boundary Conditions: $y_1(0) = y_2(0) = y_3(0) = y_4(0) = 0$ and $y_2(t_1) = 5$, $y_3(t_1) = 45$, $y_4(t_1) = 0$.

Phase 2: (Coast) $t_1 \leq t \leq 1$

State Equations:

$$\begin{aligned}\dot{y}_1 &= y_3 \\ \dot{y}_2 &= y_4 \\ \dot{y}_3 &= 0 \\ \dot{y}_4 &= -g_0 \left[\frac{h_0}{y_2 + h_0} \right]^2\end{aligned}$$

Boundary Conditions: Continuity for all states and time at t_1 (the boundary between phase 1 and 2). (p. 481).

Solve the following problem:

Objective: Minimize the final burn time t_1

Note: This example does not have a file “wrkprb14.soln.”

7.4 SOS Demonstration/Test Examples

A more extensive collection of test problems that illustrate many of the features of the SOS software are available in source code form. This section briefly describes the features of the problems.

7.4.1 Zermelo’s Problem

zermelo.f

This classical example illustrates how SOS can be applied to a very simple problem. It is described in Reference [12].

7.4.2 Goddard Rocket Problem

templt.f

This is a three phase problem containing a singular arc phase, which is modeled using an algebraic path constraint. This problem is described in Section 4.10.1 of Reference [5], Section 4.14.1 of Reference [6], and Example 4.12 [7].

7.4.3 Van der Pol Oscillator Problem

vpolex.f

This problem is formulated in eight mathematically equivalent ways to demonstrate various SOS implementation options using analytic terms and auxiliary functions. A discussion can be found in Example 4.8, Section 4.11.4 of Reference [6], and Example 4.11 [7].

Formulation 1

Consider a problem with two state variables and one control described by the system

$$\begin{aligned}\dot{y}_1 &= y_2 \\ \dot{y}_2 &= (1 - y_1^2)y_2 - y_1 + u.\end{aligned}\tag{7.1}$$

The goal is to minimize

$$J = \int_0^5 (y_1^2 + y_2^2 + u^2)dt$$

subject to the boundary conditions $y_1(0) = 1$ and $y_2(0) = 0$ and state bound

$$-4 \leq y_2(t).$$

Formulation 2

Introduce a new algebraic variable $v = (1 - y_1^2)y_2$ and replace (7.1) with

$$\begin{aligned}\dot{y}_1 &= y_2 \\ \dot{y}_2 &= v - y_1 + u \\ 0 &= v - y_2 + y_1^2 y_2.\end{aligned}\tag{7.2}$$

SOS Implementation

continuous function (NCF(2)=1), $\beta_0 a_0(y, u, p, t)$	v
analytic term	$-y_2$
auxiliary function (NCF(4)=1), $\beta_1 a_1(y, u, p, t)$	$y_1^2 y_2$

Formulation 3

Introduce a new algebraic variable $v = (1 - y_1^2)y_2$ and replace (7.1) with

$$\begin{aligned}\dot{y}_1 &= y_2 \\ \dot{y}_2 &= v - y_1 + u \\ 0 &= v - (1 - y_1^2)y_2.\end{aligned}\tag{7.3}$$

SOS Implementation

continuous function (NCF(2)=1), $\beta_0 a_0(y, u, p, t)$	v
auxiliary function (NCF(4)=1), $\beta_1 a_1(y, u, p, t)$	$-(1 - y_1^2)y_2$

Formulation 4

Introduce a new algebraic variable $v = (1 - y_1^2)y_2$ and replace (7.1) with

$$\begin{aligned}\dot{y}_1 &= y_2 \\ \dot{y}_2 &= v - y_1 + u \\ 0 &= v - (1 - y_1^2)y_2.\end{aligned}\tag{7.4}$$

SOS Implementation

continuous function (NCF(2)=1), $\beta_0 a_0(y, u, p, t)$	$v - (1 - y_1^2)y_2$
---	----------------------

Formulation 5

Introduce a new algebraic variable $v = (1 - y_1^2)y_2$ and replace (7.1) with

$$\begin{aligned}\dot{y}_1 &= y_2 \\ \dot{y}_2 &= v - y_1 + u \\ 0 &= v + y_1^2 y_2 - y_2.\end{aligned}\tag{7.5}$$

SOS Implementation

continuous function (NCF(2)=1), $\beta_0 a_0(y, u, p, t)$	$v + y_1^2 y_2$
analytic term	$-y_2$

Formulation 6

Introduce a new algebraic variable $v = (1 - y_1^2)y_2 - y_1 + u$ and replace (7.1) with

$$\begin{aligned}\dot{y}_1 &= y_2 \\ \dot{y}_2 &= v \\ 0 &= (1 - y_1^2)y_2 - y_1 + u - v.\end{aligned}\tag{7.6}$$

SOS Implementation

continuous function (NCF(2)=1), $\beta_0 a_0(y, u, p, t)$	$(1 - y_1^2)y_2 - y_1$
analytic term	u
analytic term	$-v$

Formulation 7

This formulation implements an indirect solution method using three distinct phases. In addition to the state equations defined by (7.1) the optimality conditions are formed explicitly. The problem has a first order state constraint $s(t) = -y_2(t) - 4 \leq 0$, so the Hamiltonian is

$$H = u^2 + y_1^2 + y_2^2 + \lambda_1 y_2 + \lambda_2 [(1 - y_1^2)y_2 - y_1 + u] + \mu [-(1 - y_1^2)y_2 + y_1 - u] \tag{7.7}$$

where the first derivative of the path constraint $\dot{s} = -\dot{y}_2 = -(1 - y_1^2)y_2 + y_1 - u$ appears in H . The adjoint equations are

$$\dot{\lambda}_1 = -2y_1 + 2y_1 y_2 \lambda_2 + \lambda_2 - \mu(2y_1 y_2 + 1) \tag{7.8}$$

$$\dot{\lambda}_2 = -2y_2 - \lambda_1 + \lambda_2(y_1^2 - 1) + \mu(1 - y_1^2). \tag{7.9}$$

The optimal control is

$$u(t) = \begin{cases} -\lambda_2/2 & 0 \leq t \leq t_1 \\ (y_1^2 - 1)y_2 + y_1 & t_1 \leq t \leq t_2 \\ -\lambda_2/2 & t_2 \leq t \leq 5 \end{cases} \quad (7.10)$$

with optimal multiplier given by

$$\mu(t) = \begin{cases} 0 & 0 \leq t \leq t_1 \\ 2(y_1^2 - 1)y_2 + 2y_1 + \lambda_2 & t_1 \leq t \leq t_2 \\ 0 & t_2 \leq t \leq 5. \end{cases} \quad (7.11)$$

The switching structure is determined by the tangency condition

$$\begin{aligned} 0 &= \dot{s} \\ &= -\dot{y}_2 \\ &= -(1 - y_1^2)y_2 + y_1 - u \\ &= (y_1^2 - 1)y_2 + y_1 + \lambda_2/2 \end{aligned} \quad (7.12)$$

which is enforced immediately before entering the boundary arc at $t = t_1^-$ and immediately after leaving the boundary arc at point $t = t_2^+$. The location of the exit point is also defined by enforcing the switching condition $\mu(t_2^-) = 0$.

Formulation 8

This formulation is identical to (7.1) except that the simple bound on the state is treated as the path constraint $s(t) = -y_2(t) - 4 \leq 0$.

7.4.4 Maximum Crossrange Space Shuttle Reentry Problem

shuttle.f

This single phase problem is described in Section 5.1 of Reference [5], Section 6.1 of Reference [6], and Section 8.1 of Reference [7].

7.4.5 Minimum Time to Climb Problem

clymex.files

This single phase problem is described in Section 5.2 of Reference [5] and Section 6.2 of Reference [6], and Section 8.2 of Reference [7] and illustrates the use of minimum curvature data fitting software. The file contains both the FORTRAN code and the relevant data files.

7.4.6 Low-Thrust Orbit Transfer

lthrst.f

This single phase problem using modified equinoctial coordinates is described in Section 5.3 of Reference [5], Section 6.3 of Reference [6], and Section 8.3 of Reference [7].

7.4.7 Two-Burn Orbit Transfer

`twobrn.f`

This multiple phase problem illustrates the use of analytic phase propagation, as well as modified equinoctial coordinates. A shooting method formulation is also demonstrated. It is described in Section 5.4 of Reference [5], Section 6.4 of Reference [6], and Section 8.4 of Reference [7].

7.4.8 Industrial Robot

`robot.f`

This nine phase problem illustrates how state constrained multibody problems can be solved. It is described in Section 5.5 of Reference [5] and Section 6.9 of Reference [6], and Section 8.9 of Reference [7].

7.4.9 Multibody Mechanism

`skwzer.f`

This example illustrates how SOS can be applied to an index-three differential-algebraic system. It is described in Section 5.6 of Reference [5] and Section 6.10 of Reference [6], and Section 8.10 of Reference [7].

7.4.10 Heat Flow–Parabolic PDE

`heatfl.f`

This example illustrates how SOS can be applied to a parabolic partial differential equation using the method of lines. It is described in Section 4.6.10 of Reference [5] and Section 4.6.10 of Reference [6].

7.4.11 Embedding (Homotopy) Example Problem

`embed.f`

This example demonstrates how to construct a solution to a series of related problems using an embedding or homotopy method. The approach can be especially useful when computing an initial guess for a very complex problem or when performing a parameter study that requires many solutions of a similar nature. The specific example mimics the behavior of a numerical integration algorithm, to compute the solution of an initial value problem.

7.4.12 Orbit Determination (Parameter Estimation) Example Problem

`lambrt.f`

This example demonstrates the use of the SOS parameter estimation software for a classical orbit determination problem. Three cases are illustrated including:

- determination of an orbit from two position vectors and time (Lambert's problem)
- determination of an orbit from position vectors with and without noise.

7.4.13 First-Order Irreversible Chain Reaction Parameter Estimation Problem

`foicex.files`

This problem illustrates the SOS parameter estimation software for a chemical process application. Three cases are illustrated including:

- A single phase formulation
- A multi-phase (multiple observation) formulation
- A multi-phase explicit model (curve fitting) solution.

The file contains both the FORTRAN code and the relevant data files.

7.4.14 Initial Value and Shooting Method Example Problems

`ivptst.f`

This example demonstrates the use of the SOS initial value and shooting method capability. A series of cases are illustrated including:

- forward and backward propagation of a simple initial value problem
- construction of an initial guess for a collocation method from an integrated (initial value) problem
- solution of Linear Tangent Steering problem using a multiphase shooting method.

Chapter 8

Spline Data Approximations

8.1 Overview of Data Fitting and Approximation

Many optimization, optimal control, and parameter estimation applications involve problems that are specified in terms of tabular data. All optimization algorithms require that the objective and constraints be sufficiently smooth to insure convergence and consequently it is recommended that tabular data be represented by functions with continuous first and second derivatives. For tabular functions of a single independent variable we consider

$$s(x) = \sum_{k=1}^n c_k B_k(x)$$

where $B_k(x)$ are referred to as B-spline basis functions. Tabular functions with more than one independent variable can be represented using a *tensor product* spline, e.g. in two dimensions

$$s(x, y) = \sum_{j=1}^m \sum_{k=1}^n c_{j,k} B_j(x) B_k(y).$$

When dealing with spline approximations, in general there are two distinct operations of interest to the user.

Construction of the spline coefficients $\{c_k\}$ or $\{c_{j,k}\}$ using the given tabular data, and;

Evaluation of the spline approximation at a specified value of the independent variable(s).

It is important to note that these operations are completely independent of each other. In fact it is usually preferable to construct the approximation (i.e. compute $\{c_{j,k}\}$) just once. Many evaluations of the spline models can then be performed very quickly. The subroutines described in this chapter focus on two particular aspects of interest for optimization applications. The first section describes software for the *construction* of smooth approximations from tabular data. In particular, the coefficients $\{c_{j,k}\}$ are computed by solving a large sparse optimization problem which is achieved using the software described in Chapter 2. The second section describes software for rapid evaluation of spline approximations, which is particularly important for many optimal control applications.

8.2 Subprograms for Tensor Product Spline Approximations of Multivariate Data

Spline Construction Subprograms

HDMVGI: Interpolation of Multivariate Gridded Data	605
HDMONO: Piecewise Monotonic Cubic Spline Interpolation	612
HDMGDI: Tensor Product Spline Interpolation of Gridded Data	616
HDGDWH: Tensor Product Spline Approximation of Gridded Data with Holes	625
HDCMVS: Constrained Tensor Product Approximation of Multivariate Data	636

Spline Evaluation Subprograms

HDNPVL: Evaluation of a Tensor Product Spline	655
HDENVL: Spline Evaluation with Extrapolation	662
HDFNVL: Fast Spline Evaluation without Error Checking	670
HDFEVL: Fast Spline Evaluation with Extrapolation, but without Error Checking	676

HDMVGI: Interpolation of Multivariate Gridded Data

PURPOSE

HDMVGI computes the coefficients of a tensor product spline that interpolates multivariate data which lies on a grid.

RELATED SUBPROGRAMS

HDNPVL	Evaluation of a Tensor Product Spline
HDNPDR	Evaluation of a Partial Derivative of a Tensor Product Spline
HDFNVL	Fast Spline Evaluation without Error Checking
HDFEVL	Fast Spline Evaluation with Extrapolation, but without Error Checking

METHOD

The user must provide values for the n independent variables which lie on a $m_1 \times m_2 \times \dots \times m_n$ grid and corresponding values of the dependent variable $y_{i_1, \dots, i_n}$ at the grid points. Optionally, spline representations for p dependent variables on the same grid can also be constructed. The subroutine HDMVGI generates multivariate spline functions $s_p(x), x \in R^n$ which satisfy

$$s_p(x_{i_1, \dots, i_n}) = y_{i_1, \dots, i_n}^p, \quad i_1 = 1, \dots, m_1, \quad \dots \quad , i_n = 1, \dots, m_n.$$

The index p refers to the fact that several sets of dependent variables can be interpolated at the same time. The spline is stored in the standard format and can be evaluated using any of the evaluation routines, HDNPVL, HDNPDR, HDFNVL, or HDFEVL.

USAGE

```
DOUBLE PRECISION X(NDIM,NDOM), Y(NYDIM(1),...,NYDIM(NDOM),NDEP), WORK(NWORK), C(MC)
INTEGER NPT(NDOM), NYDIM(NDOM)
```

```
CALL HDMVGI (X,NDIM,NDOM,NPT,Y,NYDIM,NDEP,NDEG,IOPT,WORK,NWORK,C,IER)
```

where

$$MC = 2 + NDOM * (K + K_a + 1) + \sum_{i=1}^{NDOM} NPT(i) + NDEP * \prod_{i=1}^{NDOM} (NPT(i) + K_a - 2),$$

$$K = NDEG + 1$$

$$K_a = \begin{cases} K & \text{for } K \text{ even,} \\ K + 1 & \text{for } K \text{ odd.} \end{cases}$$

ARGUMENTS

X	[INPUT, DOUBLE PRECISION, ARRAY] Array containing the values of the independent variables in each direction. The i^{th} value of the j^{th} independent variable is stored in $X(i, j)$. For each value of j , the X values must be in ascending order.
NDIM	[INPUT, INTEGER] Dimensional constant for the array X; $NDIM \geq \max_{i=1, \dots, NDOM} NPT(i)$.
NDOM	[INPUT, INTEGER] Number of independent variables (n in METHOD); $1 \leq NDOM$.
NPT	[INPUT, INTEGER, ARRAY] Array of NDOM values containing the number of data points in each variable. Thus, $NPT(i)$ is the number of data points in variable x_i , equivalent to m_i . $IOPT \neq 1 \quad NPT(i) \geq \max(2, K_a/2).$ $IOPT = 1 \quad NPT(i) \geq NDEG + 1.$
Y	[INPUT, DOUBLE PRECISION, ARRAY] Array containing the dependent variable values to be interpolated; $Y(i_1, \dots, i_n) = y_{i_1, \dots, i_n}^p$.
NYDIM	[INPUT, INTEGER, ARRAY] Array of dimensional constants for Y; $NYDIM(i) \geq NPT(i)$.
NDEP	[INPUT, INTEGER] The number of dependent variables represented in Y; $NDEP \geq 1$.
NDEG	[INPUT, INTEGER] The polynomial degree of the splines; $NDEG \geq 1$.
IOPT	[INPUT, INTEGER] The option flag. IOPT is used to specify the type of interpolation to perform by choice of boundary conditions. If unsure of the appropriate value of IOPT, the recommended value is $IOPT = 1$. $IOPT = 1 \quad \text{Fit the data without making assumptions about the derivatives at the boundaries. The number of interior knots is reduced to eliminate any extra degrees of freedom.}$

IOPT = 2 Use the natural spline interpolant. Thus, the boundary conditions will be chosen so that the last $(K_a - 2)/2$ normal derivatives to the boundary are zero. Thus,

$$\frac{\partial^\ell s_j}{\partial x_k^\ell}(x) = 0$$

whenever $x_k = X(k, 1)$ or $x_k = X(k, \text{NPT}(k))$, for $k = 1, \dots, \text{NDOM}$. Here, $\ell = (K_a - 2)/2, \dots, \text{NDEG} - 1$ and $j = 1, \dots, \text{NDEP}$.

IOPT = 3 Use divided differences to estimate the first $(K_a - 2)/2$ normal derivatives at the boundary. Thus

$$\frac{\partial^\ell s_j}{\partial x_k^\ell}(x) = \Delta_{k,\ell}$$

whenever $x_k = X(k, 1)$ or $x_k = X(k, \text{NPT}(k))$, for $k = 1, \dots, \text{NDOM}$ and x is a point on the grid. For this option, $\ell = 1, \dots, (K_a - 2)/2$, $j = 1, \dots, \text{NDEP}$ and $\Delta_{k,\ell}$ depends on the ℓ^{th} divided differences of the input data in the k^{th} independent variable. The divided differences are automatically computed by HDMVGI

WORK [OUTPUT, DOUBLE PRECISION, ARRAY]
Work array of length NWORK.

NWORK [INPUT, INTEGER]
Length of array WORK, depending on the value of IOPT.

IOPT = 1 NWORK $\geq 16 + 5 * M + 4 * \text{NDEG} + M * \text{NDEG}$.

IOPT > 1 NWORK $\geq 16 + 4 * M + 8 * K_a + (M + K_a - 2) * (3 * K_a - 2) + 3 * \max(K_a^2, M + K_a - 2)$,

where $M = \max_{i=1, \dots, \text{NDOM}} \text{NPT}(i)$.

C [OUTPUT, DOUBLE PRECISION, ARRAY]
Array containing the information needed to evaluate the spline.

IER [OUTPUT, INTEGER]
Success/error code. Results are not computed for IER < 0 and HDMVGI sets C(1) = -1.0.

IER = 0 Success, results computed.

IER = 2 Results computed, but they are strongly sensitive to inaccuracies in the entries of X and Y.

IER = 1	Results computed, but they are sensitive to inaccuracies in the entries of X and Y.
IER=-1	NDEG < 1.
IER=-2	If IOPT $\neq$ 1 NPT(<i>i</i>) < max(2, $K_a/2$). If IOPT = 1 NPT(<i>i</i>) < NDEG + 1 for $i = 1, \dots, \text{NDOM}$.
IER=-3	NWORK too small; the number of words needed is given by the printed error message.
IER=-4	NYDIM(<i>i</i>) < NPT(<i>i</i>), for $i = 1, \dots, \text{NDOM}$ or NDIM < max $_{i=1, \dots, \text{NDOM}}$ NPT(<i>i</i>).
IER=-5	The values of the independent variables given in the array X are not in strictly ascending order.
IER=-14	NDOM < 1.
IER=-25	IOPT is not 1, 2, or 3.
IER=-200	Unexpected error return.

EXAMPLE

This example computes the coefficients to define a bivariate cubic spline to approximate the function $f(x, y) = \cos(x) \exp(y)$. Data is constructed by evaluating the function on a 4 by 5 grid, that is four equally spaced points in the x-direction, where $0 \leq x \leq \pi/2$, and five equally spaced points in the y-direction, where $0 \leq y \leq \pi$. Subroutine HDNPVL is used to evaluate the spline, and a comparison of the spline fit to the original function at the grid points, and an arbitrary point inside the region is also displayed.

SAMPLE PROGRAM (exmvgi.cd1)¹

```

PROGRAM EXAMPLE
IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C
PARAMETER (IOPT=1, NDEG=3, NDEP=1, NDIM=4, NDOM=2)
C
PARAMETER (NXPNT = 4, NYPNT = 5, LNX = MAX(NXPNT,NYPNT))
DIMENSION XPNT(NXPNT), YPNT(NYPNT), XGRID(LNX,NDOM), YGRID(LNX,LNX,1)

```

¹sosexample.tar

```

    PARAMETER (K = NDEG+1, KA = K,
$           LNCOEF = 2 + NDOM*(K+KA+1) + NXPNT + NYPNT
$           + NDEP*(NXPNT+KA-2)*(NYPNT+KA-2),
$           LNWORK = 16 + 5*LNK + 4*NDEG + LNK*NDEG)

    DIMENSION WORK(LNWORK),COEF(LNCOEF)
C
    DIMENSION NPT(NDOM), NYDIM(NDOM)
    DIMENSION XARG(2)
C
    FUNCTION TO BE APPROXIMATED
C
    FNC(XVAL,YVAL) = COS(XVAL)*EXP(YVAL)
C
    GENERATE GRID POINTS FOR X IN [0,PI/2] AND Y IN [0,PI]
C
    PI = HDMCON(12)
C
    DELX = .5D0*PI/DBLE(NXPNT-1)
    DO I = 1,NXPNT
        XPNT(I) = DBLE(I-1)*DELX
    ENDDO
C
    DELY = PI/DBLE(NYPNT-1)
    DO J = 1,NYPNT
        YPNT(J) = DBLE(J-1)*DELY
    ENDDO
C
    LOAD INDEPENDENT VARIABLE GRID LOCATIONS INTO XGRID ARRAY
C
    XGRID(1:NXPNT,1) = XPNT(1:NXPNT)
    XGRID(1:NYPNT,2) = YPNT(1:NYPNT)
C
    COMPUTE THE FUNCTION VALUES F(X,Y) AT THE GRID POINTS
C
    DO I = 1,NXPNT
        DO J = 1,NYPNT
            XG = XGRID(I,1)
            YG = XGRID(J,2)
            YGRID(I,J,1) = FNC(XG,YG)
        ENDDO
    ENDDO
C
    FIT CUBIC SPLINE TO DATA
C
    NPT(1) = NXPNT
    NPT(2) = NYPNT
    NYDIM(1) = LNK

```

```
      NYDIM(2) = LNX
C
      CALL HDMVGI (XGRID, LNX, NDOM, NPT, YGRID, NYDIM, NDEP,
$               NDEG, IOPT, WORK, LNWORK, COEF, IER)
C
      CHECK FOR ERROR RETURN
C
      IF(IER.NE.0) THEN
        WRITE(6, '(" Error return from HDMVGI; IER = ",I5)') IER
        STOP
      ENDIF
C
      EVALUATE SPLINE AT ORIGINAL DATA POINTS AND COMPARE WITH SPLINE FIT
C
      WRITE(6, ' (T40,"Grid Point Comparison")')
      WRITE(6, ' (T10,"x",T25,"y",T39,"f(x,y)",T55,"s(x,y)",T73,"err")')
C
      DO J = 1,NYPNT
C
        XARG(2) = XGRID(J,2)
C
        DO I = 1,NXPNT
C
          XARG(1) = XGRID(I,1)
C
          CALL HDNPVL(XARG,1,COEF,WORK,LNWORK,FVAL,IER)
C
          CHECK FOR ERROR RETURN
C
          IF(IER.NE.0) THEN
            WRITE(6, '(" Error return from HDSPVL; IER = ",i5)') IER
            STOP
          ENDIF
C
          ERR = ABS(YGRID(I,J,1)-FVAL)
          WRITE(6, ' (5G16.6)') XARG(1:2),YGRID(I,J,1),FVAL,ERR
C
        ENDDO
C
      ENDDO
C
      EVALUATE THE SPLINE AT AN INTERIOR POINT
C
      XARG(1) = XPNT(1) + .43D0*(XPNT(NXPNT) - XPNT(1))
      XARG(2) = YPNT(1) + .72D0*(YPNT(NYPNT) - YPNT(1))
C
      CALL HDNPVL(XARG,1,COEF,WORK,LNWORK,FVAL,IER)
C
```

```

WRITE(6,'(//T35,"Interior Point Comparison")')
WRITE(6,'(T10,"x",T25,"y",T39,"f(x,y)",T55,"s(x,y)",T73,"err")')
FTRUE = FNC(XARG(1),XARG(2))
ERR = ABS(FTRUE-FVAL)
WRITE(6,'(5G16.6)') XARG(1:2),FTRUE,FVAL,ERR
C
STOP
END

```

OUTPUT FROM SAMPLE PROGRAM

Grid Point Comparison				
x	y	f(x,y)	s(x,y)	err
0.00000	0.00000	1.00000	1.00000	0.666134E-15
0.523599	0.00000	0.866025	0.866025	0.222045E-15
1.04720	0.00000	0.500000	0.500000	0.00000
1.57080	0.00000	0.612323E-16	0.902056E-16	0.289733E-16
0.00000	0.785398	2.19328	2.19328	0.222045E-14
0.523599	0.785398	1.89944	1.89944	0.222045E-15
1.04720	0.785398	1.09664	1.09664	0.222045E-15
1.57080	0.785398	0.134300E-15	0.124900E-15	0.939958E-17
0.00000	1.57080	4.81048	4.81048	0.888178E-15
0.523599	1.57080	4.16600	4.16600	0.888178E-15
1.04720	1.57080	2.40524	2.40524	0.177636E-14
1.57080	1.57080	0.294557E-15	0.166533E-15	0.128023E-15
0.00000	2.35619	10.5507	10.5507	0.355271E-14
0.523599	2.35619	9.13720	9.13720	0.177636E-14
1.04720	2.35619	5.27536	5.27536	0.888178E-15
1.57080	2.35619	0.646046E-15	0.499600E-15	0.146445E-15
0.00000	3.14159	23.1407	23.1407	0.710543E-14
0.523599	3.14159	20.0404	20.0404	0.355271E-14
1.04720	3.14159	11.5703	11.5703	0.355271E-14
1.57080	3.14159	0.141696E-14	0.266454E-14	0.124758E-14

Interior Point Comparison				
x	y	f(x,y)	s(x,y)	err
0.675442	2.26195	7.49351	7.46977	0.237377E-01

HDMONO: Piecewise Monotonic Cubic Spline Interpolation

PURPOSE

HDMONO computes the coefficients of a piecewise monotonic cubic spline interpolant to a set of user supplied data.

RELATED SUBPROGRAMS

HDNPVL	Evaluation of a Tensor Product Spline
HDNPDR	Evaluation of a Partial Derivative of a Tensor Product Spline
HDFNVL	Fast Spline Evaluation without Error Checking
HDFEVL	Fast Spline Evaluation with Extrapolation, but without Error Checking

METHOD

Given a set of points (x_i, y_i) , for $i = 1, 2, \dots, n$, HDMONO computes the coefficients c_j of the C^1 cubic spline

$$S(x) = \sum_{j=1}^{2n} c_j B_j(x)$$

where the $B_j(x)$'s are the B-splines defined by using x_i for $i = 2, \dots, n-1$ as knots of multiplicity 2 and x_1 and x_n as knots of multiplicity 4. The function $S(x)$ then satisfies the following conditions:

1. $S(x_i) = y_i$ for $i = 1, 2, \dots, n$.
2. For each x in the interval $[x_i, x_{i+1}]$, $\text{sign}(S'(x)) = \text{sign}(y_{i+1} - y_1)$.

That is, $S(x)$ is a piecewise monotonic spline.

USAGE

DOUBLE PRECISION X(N), Y(N), WORK(NWORK), C(4*N+9)

CALL HDMONO (N,X,Y,WORK,NWORK,C,IER)

ARGUMENTS

N	[INPUT, INTEGER] Number of data points; $N \geq 2$.
X	[INPUT, DOUBLE PRECISION, ARRAY] Array of N values of the independent variable in strictly ascending order.
Y	[INPUT, DOUBLE PRECISION, ARRAY] Array containing the dependent variable values to be interpolated.
WORK	[OUTPUT, DOUBLE PRECISION, ARRAY] Work array of length NWORK.
NWORK	[INPUT, INTEGER] Length of array WORK, $NWORK \geq 23 * N + 4 * \max(8, N) + 21$.
C	[OUTPUT, DOUBLE PRECISION, ARRAY] Array of length $4 * N + 9$ containing the information needed to evaluate the spline.
IER	[OUTPUT, INTEGER] Success/error code. Results are not computed for $IER < 0$ and HDMONO sets $C(1) = -1.0$. IER = 0 Success, results computed. IER = 2 Results computed, but they are strongly sensitive to inaccuracies in the entries of X and Y. IER = 1 Results computed, but they are sensitive to inaccuracies in the entries of X and Y. IER=-2 $N < 2$. IER=-3 NWORK too small; the number of words needed is given by the printed error message. IER=-5 The values of the independent variables given in the array X are not in strictly ascending order. IER=-100 The matrix representing the interpolation problem is singular. IER=-200 Unexpected error return.

EXAMPLE

This example computes the coefficients to define a monotonic cubic spline approximation to the function $f(x) = \sin(x)$. The angles x range from 0 to 180 degrees. Subroutine HDENVL is used

to evaluate the spline, and a comparison of the spline fit to the original function at equally spaced points between 0 and 180 degrees are displayed.

SAMPLE PROGRAM (exmono.cd1)²

```
PROGRAM SAMPLE
C
  INTEGER N, NWORK
  PARAMETER (N = 9, NWORK = 23*N + 4*(8+N) + 21)
  INTEGER I, IER
  DOUBLE PRECISION DEG2RA, W, XPOINT
  DOUBLE PRECISION C(4*N+9), V(1), WORK(NWORK),X(N),Y(N)
C
  DOUBLE PRECISION HDMCON
  EXTERNAL          HDMCON
C
  DATA X / 0.0,      30.0,      45.0,      60.0,  90.0,
$          120.0,      135.0,      150.0, 180.0 /
C
  DATA Y / 0.0,      0.5,  0.7071067,  0.8660254,  1.0,
$          0.8660254,  0.7071067,      0.5,  0.0 /
C
  DEG2RA = HDMCON(15)
C
  CALL HDMONO(N,X,Y,WORK,NWORK,C,IER)
C
  IF(IER .LT. 0) THEN
    WRITE(*,*) '* Error at call to HDMONO *'
    STOP
  ENDIF
C
  WRITE(*,*) '   Angle x           Interpolated           True'
  WRITE(*,*) ' in degrees       value of sin(x)   value of sin(x)'
  WRITE(*,*) ' -----          -----          -----'
C
  DO I = 0,180,10
C
    XPOINT = DBLE(I)
C
    CALL HDENVL( XPOINT, 1, C, WORK, NWORK, V, IER)
C
    IF(IER .LT. 0) THEN
      WRITE(*,*) '* Error at call to HDENVL *'
      STOP
    ENDIF
C
```

²sosexample.tar

```

      W = SIN( DEG2RA*XPPOINT )
      WRITE(*,'(4X,I3,13X,F6.4,12X,F6.4)') I, V(1), W

      ENDDO
C
      STOP
      END

```

OUTPUT FROM SAMPLE PROGRAM

Angle x in degrees	Interpolated value of sin(x)	True value of sin(x)
-----	-----	-----
0	0.0000	0.0000
10	0.1688	0.1736
20	0.3376	0.3420
30	0.5000	0.5000
40	0.6438	0.6428
50	0.7682	0.7660
60	0.8660	0.8660
70	0.9297	0.9397
80	0.9797	0.9848
90	1.0000	1.0000
100	0.9797	0.9848
110	0.9297	0.9397
120	0.8660	0.8660
130	0.7682	0.7660
140	0.6438	0.6428
150	0.5000	0.5000
160	0.3376	0.3420
170	0.1688	0.1736
180	0.0000	0.0000

HDMGDI: Tensor Product Spline Interpolation of Gridded Data

PURPOSE

HDMGDI interpolates multivariate data which lies on a grid. It computes a tensor product spline which has minimal variation of its second order partials, interpolates the data, and, optionally, is monotonic along grid lines.

RELATED SUBPROGRAMS

HDNPVL	Evaluation of a Tensor Product Spline
HDNPDR	Evaluation of a Partial Derivative of a Tensor Product Spline
HDMVGI	Interpolate Multivariate Gridded Data

METHOD

The user must provide values for the n independent variables which lie on a $m_1 \times m_2 \times \dots \times m_n$ grid defined as $(\gamma_{1,1}, \dots, \gamma_{m_1,1}) \times \dots \times (\gamma_{1,n}, \dots, \gamma_{m_n,n})$ and corresponding values of the dependent variable $y_{i_1, \dots, i_n}$.

HDMGDI defines the tensor product spline $s(X)$ for $X = (x_1, \dots, x_n)$ of degree $K_j - 1$ in x_j as

$$s(X) = \sum_{i_n=1}^{M_n+K_n} \cdots \sum_{i_1=1}^{M_1+K_1} \alpha_{i_1, \dots, i_n} \prod_{j=1}^n B_{i_j}(x_j|T_j) \quad (8.1)$$

where $B_{i_j}(x_j|T_j)$ is the value at x_j of the i_j -th B-spline defined by the knot vector T_j . If $x_l^j = \min_i \{x_{i,j}\}$, $x_u^j = \max_i \{x_{i,j}\}$, and $t_{k,j}$ for $k = 1, \dots, M_j$, represents the internal knots, then

$$T_j = (\overbrace{x_l^j, \dots, x_l^j}^{K_j \text{ times}}, t_{1,j}, t_{2,j}, \dots, t_{M_j,j}, \overbrace{x_u^j, \dots, x_u^j}^{K_j \text{ times}}). \quad (8.2)$$

Notice the end knots have multiplicity K_j . The number of B-splines, $B_{i_j}(x_j|T_j)$, in the x_j -th independent variable is $M_j + K_j$. The number of coefficients, $\alpha_{i_1, \dots, i_n}$, is $\prod_{j=1}^n (M_j + K_j)$.

To define the spline the user must specify the degree, $K_j - 1$, the local monotonicity condition, and the continuity condition in each of the independent variables. The local monotonicity condition specifies if the spline should be derivative sign perserving along grid lines. For example, suppose y_1 and y_2 represent the dependent variable values associated with the grid points $(\gamma_{k_1,1}, \dots, \gamma_{k_i,i}, \dots, \gamma_{k_n,n})$ and $(\gamma_{k_1,1}, \dots, \gamma_{k_i+1,i}, \dots, \gamma_{k_n,n})$, respectively. If the spline is derivative sign perserving, then

$$\frac{\partial s(X)}{\partial x_i} \begin{cases} > 0 & \text{if } y_1 < y_2; \\ = 0 & \text{if } y_1 = y_2; \\ < 0 & \text{if } y_1 > y_2; \end{cases} \quad (8.3)$$

for $x_i \in [\gamma_{k_i,i}, \gamma_{k_{i+1},i}]$ and $x_j = \gamma_{k_j,j}$ for $j = 1, \dots, n$ and $j \neq i$. Notice this condition is only in effect along grid lines.

The continuity condition specifies if the spline is of full or reduced continuity in x_j at the grid points. If the spline has full continuity, then its first $K_j - 2$ partial derivatives are continuous everywhere. If the spline has reduced continuity, then its $K_j - 2$ -th partial derivative is not continuous at the grid points. A spline which is linear in x_j can not have reduced continuity in x_j .

For a given set of data, HDMGDI determines the underlying grid on which the data resides and defines internal knots, $t_{1,j}, \dots, t_{M_j,j}$, for $j = 1, \dots, n$, which allows the spline to interpolate the data and satisfy the local monotonicity and continuity conditions. The internal knots are defined in the following manner.

If the user has not requested monotonicity, then HDMGDI places a knot at every internal grid point; that is, $t_{i,j} = \gamma_{i+1,j}$ for $i = 1, \dots, m_j - 2$, and the number of internal knots in x_j is $M_j = m_j - 2$.

If the user requests monotonicity and full continuity, then the internal knots are placed according to the degree of the spline. If the spline is linear in x_j , then HDMGDI places a knot at every internal grid point and the number of internal knots in x_j is $M_j = m_j - 2$. If the spline is quadratic in x_j , then HDMGDI places a knot at every internal grid point and at the midpoint of each interval. The number of internal knots in x_j is $M_j = 2m_j - 3$. If the spline is cubic in x_j , then HDMGDI places a knot on either side of and at every internal grid point. The number of internal knots in x_j is $3m_j - 6$.

If the user requests monotonicity and reduced continuity, then HDMGDI places a knot at every internal grid point, if the spline is linear. Otherwise, HDMGDI places two knots at every internal grid point. Hence, the number of internal knots in x_j is $M_j = m_j - 2$, if the spline is linear, and $M_j = 2m_j - 4$, otherwise.

After defining the internal knots HDMGDI employs HDCMVS to compute a tensor product spline which has minimal variation in its second order partials and satisfies the interpolation, monotonicity, and continuity conditions.

USAGE REMARKS

Under certain conditions HDCMVS may not be able to find a feasible point for the interpolation and monotonicity conditions. Such failures are due to poor numerical properties which can be related to the data. For example, if the underlying grid upon which the data resides contains grid lines which are closely placed (within ϵ of each other), then constraints which are redundant within machine precision are possible. In this case, HDMGDI will fail to find a feasible point. HDMGDI will produce an error message informing the user of this failure.

USAGE

```
INTEGER N(NDOM), NDIM, NDOM, NYDIM(NDOM), NDEG(NDOM), MONO(NDOM)
INTEGER CONT(NDOM), NWORK, IWORK(NIWORK), NIWORK, MCOU, NEEDED, IER
```

```
DOUBLE PRECISION X(NDIM,NDOM), Y(N), WORK(NWORK), C(MC)
```

```
CALL HDMGDI (N,X,NDIM,NDOM,Y,NYDIM,NDEG,MONO,CONT,WORK,NWORK,IWORK,NIWORK,
+           MCOU,NEEDED,C,IER)
```

where $MC = 2 + 3*NDOM + \sum_{j=1}^{NDOM}(M_j + 2 * K_j) + \prod_{j=1}^{NDOM}(M_j + K_j)$.

ARGUMENTS

- N** [INPUT, INTEGER, ARRAY]
 Array of NDOM values containing the number of data points in each variable. Thus, $N(j)$ is the number of data points in variable x_j , equivalent to m_j , and must satisfy; $N(j) \geq 2$, for $j = 1, \dots, NDOM$.
- X** [INPUT, DOUBLE PRECISION, ARRAY]
 Array containing the values of the independent variables in each direction. The i -th value of the j -th independent variable is stored in $X(i, j)$, that is $X(i, j) = \gamma_{i,j}$. For each value of j , the X values need not be ordered but they must satisfy the condition $\min_i \{X(i, j)\} < \max_i \{X(i, j)\}$.
- NDOM** [INPUT, INTEGER]
 Number of independent variables (n in METHOD); $1 \leq NDOM \leq 10$.
- NDIM** [INPUT, INTEGER]
 Dimensional constant for the array X; $NDIM \geq \max_{j=1, \dots, NDOM} N(j)$.
- Y** [INPUT, DOUBLE PRECISION, ARRAY]
 Array containing the dependent variable values to be interpolated; $Y(i_1, \dots, i_n) = y_{i_1, \dots, i_n}$.
- NYDIM** [INPUT, INTEGER, ARRAY]
 Array of NDOM values containing the dimensional constants for Y; $NYDIM(j) \geq N(j)$, for $j = 1, \dots, NDOM$.
- NDEG** [INPUT, INTEGER, ARRAY]
 Array of NDOM values where $NDEG(j)$ is the degree of the spline in x_j ; $1 \leq NDEG(j) \leq 3$.
- MONO** [INPUT, INTEGER, ARRAY]
 Monotonicity option. Array of NDOM values where $MONO(j)$ specifies if the spline is locally monotonic in x_j along the data grid lines.
 $MONO(j) = 0$ The spline will not be locally monotonic in x_j .
 $MONO(j) = 1$ The spline will be locally monotonic in x_j .
- CONT** [INPUT, INTEGER, ARRAY]
 Continuity option. Array of NDOM values where $CONT(j)$ specifies if the spline has full or reduced continuity at the data.

CONT(j) = 0 Spline should have full continuity in x_j at the grid points. If the spline is linear in x_j , then it will be continuous everywhere in x_j . If the spline is quadratic in x_j , then the first order partial in x_j will be continuous everywhere. If the spline is cubic in x_j , then the second order partial in x_j will be continuous everywhere.

CONT(j) = 1 Spline should have full continuity in x_j at the grid points. If the spline is linear in x_j , then it will be continuous everywhere in x_j . If the spline is quadratic in x_j ; then the first order partial in x_j will be continuous everywhere, except at grid points. If the spline is cubic in x_j ; then the second order partial in x_j will be continuous everywhere, except at grid points.

WORK [INPUT, DOUBLE PRECISION, ARRAY]
Work array of length NWORK.

NWORK [INPUT, INTEGER]
Length of array WORK, HDMGDI can not determine a priori the amount of real workspace needed. NWORK is checked at each stage of the algorithm to determine if sufficient workspace is provided. If sufficient workspace is not provided, then HDMGDI will report an error condition and return the amount of real workspace required in NEEDED.

IWORK [INPUT, INTEGER, ARRAY]
Work array of length NIWORK.

NIWORK [INPUT, INTEGER]
Length of array IWORK, HDMGDI can not determine a priori the amount of integer workspace needed. NIWORK is checked at each stage of the algorithm to determine if sufficient workspace is provided. If sufficient workspace is not provided, then HDMGDI will report an error condition and return the amount of integer workspace required in NEEDED.

MCOUT [INPUT, INTEGER]
Length of array C; MCOUT $\geq$ MC. If sufficient space is not provided, then HDMGDI will report an error condition and return the required value of MCOUT in NEEDED.

NEEDED [OUTPUT, INTEGER]
Specifies the minimum value of MC or the amount of integer or real workspace required to proceed.

C [OUTPUT, DOUBLE PRECISION, ARRAY]
Array containing the information needed to evaluate the spline.

IER [OUTPUT, INTEGER]
Success/error code. Results are not computed for IER < 0 and HDMGDI sets C(1) = -1.0.

IER = 0	Success, results computed.
IER = 2	The resulting spline satisfies the interpolation and monotonicity conditions but may not have minimal variation in its second order partials.
IER = 1	The resulting spline satisfies the interpolation and monotonicity conditions, has minimal variation in its second order partials, but may exhibit undesirable behavior.
IER=-1	NDEG(j) < 0 or NDEG(j) > 3 for some $j = 1, \dots, \text{NDOM}$.
IER=-2	N(j) < 2 for some $j = 1, \dots, \text{NDOM}$.
IER=-3	NWORK too small; the number of words of real workspace needed to continue is returned in NEEDED.
IER=-4	NIWORK too small; the number of words of integer workspace needed to continue is returned in NEEDED.
IER=-5	NDIM < $\max_{j=1, \dots, \text{NDOM}} \text{N}(j)$ or NYDIM(j) < N(j) for some $j = 1, \dots, \text{NDOM}$.
IER=-7	$\min_i \{X(i, j)\} = \max_i \{X(i, j)\}$ for some $j = 1, \dots, \text{NDOM}$.
IER=-12	NDOM < 1 or NDOM > 10.
IER=-14	MONO(j) $\neq$ 0 and MONO(j) $\neq$ 1 for some $j = 1, \dots, \text{NDOM}$.
IER=-15	CONT(j) $\neq$ 0 and CONT(j) $\neq$ 1 for some $j = 1, \dots, \text{NDOM}$.
IER=-18	MCOUT too small; the value of MCOUT required is returned in NEEDED.
IER=-40	The interpolation and monotonicity constraints do not have a feasible point. See discussion under USAGE REMARKS.
IER=-43	The interpolation and monotonicity constraints are singular. See discussion under USAGE REMARKS.
IER=-100	Unexpected error return, see explanation of MODE = 5 error message from HHERR.
IER=-200	Numerical error, see discussion under USAGE REMARKS.

EXAMPLE

This example computes a least squares approximation of data sampled from the function $z = x^2y^3$. The data is defined on a 6 by 6 grid over the region $0.0 \leq x \leq 1.0$, $0.0 \leq y \leq 1.0$. The spline interpolation will be quadratic in x and cubic in y . Monotonicity constraints will be imposed and the spline will have full continuity at the grid points in both independent variables.

SAMPLE PROGRAM (exmgdi.cd1)³

```

      PROGRAM EXAMPLE

C      -----
C      ... PROGRAM TO ILLUSTRATE EXAMPLE PROBLEM
C      -----

C ... HDMGDI PARAMETERS

      INTEGER          NDOM, NDIM  , NWORK, NIWORK, NEEDED, MCOUT, IER

      PARAMETER ( NDOM = 2, NDIM = 6, NWORK = 100000, NIWORK = 100000 )
      PARAMETER ( MCOUT = 305 )

      INTEGER          N(NDOM)    , NYDIM(NDOM), NDEG(NDOM), MONO(NDOM),
+                     CONT(NDOM), IWORK(NWORK)

      DOUBLE PRECISION C(MCOUT), WORK(NWORK), X(NDIM,NDOM), Y(6,6)

C ... LOCAL VARIABLES

      INTEGER          I          , J

      DOUBLE PRECISION VAL        , XTMP(2)

C      -----
C      ... DEFINE THE DATA TO APPROXIMATE. DATA REPRESENTS THE FUNCTION
C      ... Z = X**2Y**3
C      -----

C ... DEFINE FUNCTION DATA

      DO I = 1, 6

          X(I,1) = DBLE( I - 1 ) * 2.0D-1
          X(I,2) = DBLE( I - 1 ) * 2.0D-1

          END DO

      DO J = 1, 6

```

³sosexample.tar

```

      DO I = 1, 6

          Y(I,J) = X(I,1)**2*X(J,2)**3

      END DO

END DO

C -----
C ... SPECIFY THE NUMBER OF POINTS, DEGREE, MONOTONICITY AND CONTINUITY
C ... CONDITIONS, AND DIMENSIONAL CONSTANTS
C -----

N(1) = 6
N(2) = 6

NDEG(1) = 3
NDEG(2) = 3

MONO(1) = 1
MONO(2) = 1

CONT(1) = 0
CONT(2) = 0

NYDIM(1) = 6
NYDIM(2) = 6

C -----
C ... CALL HDMGDI
C -----

CALL HDMGDI ( N      , X      , NDIM  , NDOM  , Y      , NYDIM,
+           NDEG  , MONO  , CONT  , WORK  , NWORK , IWORK,
+           NIWORK, MCOU  , NEEDED, C      , IER  )

IF ( IER .NE. 0 ) THEN

    WRITE ( *, 1000 ) IER
    IF ( IER .LT. 0 ) STOP

END IF

C -----
C ... EVALUATE SPLINE APPROXIMATION
C -----

```

```

WRITE ( *, 1200 )

DO J = 1, 6

  DO I = 1, 6

    XTMP(1) = X(I,1)
    XTMP(2) = X(J,2)

    CALL HDNPVL ( XTMP, 1, C, WORK, NWORK, VAL, IER )

    IF ( IER .LT. 0 ) THEN

      WRITE ( *, 1100 ) IER
      STOP

    END IF

    WRITE ( *, 1300 ) XTMP(1), XTMP(2), VAL, Y(I,J),
+      ABS ( Y(I,J) - VAL )

  END DO

END DO

WRITE ( *, 1400 )

C -----

1000 FORMAT ( ' ERROR RETURN FROM HDMGDI - IER = ', I5 )
1100 FORMAT ( ' ERROR RETURN FROM HDNPVL - IER = ', I5 )
1200 FORMAT(/1X,'  X      Y      S(X,Y)    Z(X,Y)    ABS DIFF.',
+      /1X,'=====')
1300 FORMAT(2(1X,F6.3),3(1X,F9.6))
1400 FORMAT ( / '* NORMAL TERMINATION *' )
      STOP
      END

```

OUTPUT FROM SAMPLE PROGRAM

X	Y	S(X,Y)	Z(X,Y)	ABS DIFF.
0.000	0.000	0.000000	0.000000	0.000000
0.200	0.000	0.000000	0.000000	0.000000
0.400	0.000	0.000000	0.000000	0.000000
0.600	0.000	0.000000	0.000000	0.000000

0.800	0.000	0.000000	0.000000	0.000000
1.000	0.000	0.000000	0.000000	0.000000
0.000	0.200	0.000000	0.000000	0.000000
0.200	0.200	0.000320	0.000320	0.000000
0.400	0.200	0.001280	0.001280	0.000000
0.600	0.200	0.002880	0.002880	0.000000
0.800	0.200	0.005120	0.005120	0.000000
1.000	0.200	0.008000	0.008000	0.000000
0.000	0.400	0.000000	0.000000	0.000000
0.200	0.400	0.002560	0.002560	0.000000
0.400	0.400	0.010240	0.010240	0.000000
0.600	0.400	0.023040	0.023040	0.000000
0.800	0.400	0.040960	0.040960	0.000000
1.000	0.400	0.064000	0.064000	0.000000
0.000	0.600	0.000000	0.000000	0.000000
0.200	0.600	0.008640	0.008640	0.000000
0.400	0.600	0.034560	0.034560	0.000000
0.600	0.600	0.077760	0.077760	0.000000
0.800	0.600	0.138240	0.138240	0.000000
1.000	0.600	0.216000	0.216000	0.000000
0.000	0.800	0.000000	0.000000	0.000000
0.200	0.800	0.020480	0.020480	0.000000
0.400	0.800	0.081920	0.081920	0.000000
0.600	0.800	0.184320	0.184320	0.000000
0.800	0.800	0.327680	0.327680	0.000000
1.000	0.800	0.512000	0.512000	0.000000
0.000	1.000	0.000000	0.000000	0.000000
0.200	1.000	0.040000	0.040000	0.000000
0.400	1.000	0.160000	0.160000	0.000000
0.600	1.000	0.360000	0.360000	0.000000
0.800	1.000	0.640000	0.640000	0.000000
1.000	1.000	1.000000	1.000000	0.000000

* NORMAL TERMINATION *

HDGDWH: Tensor Product Spline Approximation of Gridded Data with Holes

PURPOSE

HDGDWH approximates multivariate data which lies on a grid but contains regions of missing data. It computes a tensor product spline which has minimal variation of its second order partials, approximates the data within a user specified tolerance and, optionally, is locally monotonic along grid lines.

RELATED SUBPROGRAMS

HDNPVL	Evaluation of a Tensor Product Spline
HDNPDR	Evaluation of a Partial Derivative of a Tensor Product Spline
HDMVGI	Interpolate Multivariate Gridded Data

METHOD

The user must provide values for the n independent variables $X_\ell = (x_{1,\ell}, \dots, x_{n,\ell})$, corresponding values of the dependent variable y_ℓ , and corresponding values of the absolute tolerance δ_ℓ , for $\ell = 1, \dots, N$. The points X_ℓ should be a subset of a grid defined by $(\gamma_{1,1}, \dots, \gamma_{1,m_1}) \times \dots \times (\gamma_{n,1}, \dots, \gamma_{n,m_n})$.

HDGDWH defines the tensor product spline $s(X)$ for $X = (x_1, \dots, x_n)$ of degree $K_i - 1$ in x_i as

$$s(X) = \sum_{j_n=1}^{M_n+K_n} \cdots \sum_{j_1=1}^{M_1+K_1} \alpha_{j_1, \dots, j_n} \prod_{i=1}^n B_{j_i}(x_i|T_i) \quad (8.4)$$

where $B_{j_i}(x_i|T_i)$ is the value at x_i of the j_i -th B-spline defined by the knot vector T_i . If $x_l^i = \min_\ell \{x_{i,\ell}\}$, $x_u^i = \max_\ell \{x_{i,\ell}\}$, and $t_{i,k}$ for $k = 1, \dots, M_i$, represents the internal knots, then

$$T_i = (\overbrace{x_l^i, \dots, x_l^i}^{K_i \text{ times}}, t_{i,1}, t_{i,2}, \dots, t_{i,M_i}, \overbrace{x_u^i, \dots, x_u^i}^{K_i \text{ times}}). \quad (8.5)$$

Notice the end knots have multiplicity K_i . The number of B-splines, $B_{j_i}(x_i|T_i)$, in the x_i -th independent variable is $M_i + K_i$. The number of coefficients, $\alpha_{j_1, \dots, j_n}$, is $\prod_{i=1}^n (M_i + K_i)$.

HDGDWH computes the tensor product spline coefficients $\alpha_{j_1, \dots, j_n}$, such that $s(X)$ satisfies the constraints

$$y_\ell - \delta_\ell \leq s(X_\ell) \leq y_\ell + \delta_\ell \quad (8.6)$$

for $\ell = 1, \dots, N$.

To define the spline the user must specify the degree, $K_i - 1$, the local monotonicity condition, and the continuity condition in each of the independent variables.

The local monotonicity condition specifies if the spline should be derivative sign perserving along grid lines. For example, suppose y_1 and y_2 represent the dependent variable values associated with the grid points $(\gamma_{1,k_1}, \dots, \gamma_{i,k_i}, \dots, \gamma_{n,k_n})$ and $(\gamma_{1,k_1}, \dots, \gamma_{i,k_i+1}, \dots, \gamma_{n,k_n})$, respectively. If the spline is derivative sign perserving, then

$$\frac{\partial s(X)}{\partial x_i} \begin{cases} > 0 & \text{if } y_1 < y_2; \\ = 0 & \text{if } y_1 = y_2; \\ < 0 & \text{if } y_1 > y_2; \end{cases} \quad (8.7)$$

for $x_i \in [\gamma_{i,k_i}, \gamma_{i,k_i+1}]$ and $x_j = \gamma_{j,k_j}$ for $j = 1, \dots, n$ and $j \neq i$. Note: This condition is only in effect along grid lines.

The continuity condition specifies if the spline is of full or reduced continuity in x_i at the grid points. If the spline has full continuity, then its first $K_i - 2$ partial derivatives are continuous everywhere. If the spline has reduced continuity, then its $K_i - 2$ -th partial derivative is not continuous at the grid points. A spline which is linear in x_i can not have reduced continuity in x_i .

For a given set of data, HDGDWH determines the underlying grid on which the data resides and defines internal knots, $t_{i,1}, \dots, t_{i,M_i}$, for $i = 1, \dots, n$, which allows the spline to approximate the data and satisfy the local monotonicity and continuity conditions. The internal knots are defined in the following manner.

If the user has not requested monotonicity, then HDGDWH places a knot at every internal grid point; that is, $t_{i,j} = \gamma_{i,j+1}$ for $j = 1, \dots, m_i - 2$, and the number of internal knots in x_i is $M_i = m_i - 2$.

If the user requests monotonicity and full continuity, then the internal knots are placed according to the degree of the spline. If the spline is linear in x_i , then HDGDWH places a knot at every internal grid point and the number of internal knots in x_i is $M_i = m_i - 2$. If the spline is quadratic in x_i , then HDGDWH places a knot at every internal grid point and at the midpoint of each interval. The number of internal knots in x_i is $M_i = 2m_i - 3$. If the spline is cubic in x_i , then HDGDWH places a knot on either side of and at every internal grid point. The number of internal knots in x_i is $3m_i - 6$.

If the user requests monotonicity and reduced continuity, then HDGDWH places a knot at every internal grid point, if the spline is linear. Otherwise, HDGDWH places two knots at every internal grid point. Hence, the number of internal knots in x_i is $M_i = m_i - 2$, if the spline is linear, and $M_i = 2m_i - 4$, otherwise.

In addition to the approximation, monotonicity and continuity conditions; the user can specify a lower bound s_l , and an upper bound s_u , on the spline approximation. HDGDWH will compute a spline $s(X)$ which satisfies the condition

$$s_l \leq s(X) \leq s_u \quad (8.8)$$

for values of X in regions without data. The only way to insure the spline approximation satisfies the above relation in independent variable regions which contain data is to impose monotonicity constraints in at least one of the independent variables.

After defining the internal knots and determining the constraints to satisfy the approximation and monotonicity conditions, HDGDWH employs HDCMVS to compute a tensor product spline which

has minimal variation in its second order partials and satisfies the approximation, monotonicity, and continuity conditions.

USAGE REMARKS

Under certain conditions HDCMVS may not be able to find a feasible point for the approximation and monotonicity conditions. Such failures are due to poor numerical properties which may be related to the data. For example, if the underlying grid upon which the data resides contains grid lines which are close (within ϵ of each other), then constraints which are redundant within machine precision are possible. Also, if the dependent variable has some extreme derivative variation, then it can be difficult to satisfy the local monotonicity conditions and the approximation conditions for small values of δ_ℓ . In either case HDGDWH may fail to find a feasible point or produce a singular set of constraints. HDGDWH will report these conditions with an error message.

USAGE

```

INTEGER N, NDM, NDM, ITOL, NDEG(NDM), MONO(NDM), CONT(NDM)
INTEGER IBND, NWORK, IWORK(NIWORK), NIWORK, MOUT, NEEDED, IER

DOUBLE PRECISION ABSTOL(N), BOUNDS(2), X(NDM,NDM), Y(N), WORK(NWORK), C(MC)

CALL HDGDWH (N,X,NDIM,NDOM,Y,ITOL,ABSTOL,NDEG,MONO,CONT,IBND,BOUNDS,
+           WORK,NWORK,IWORK,NIWORK,MOUT,NEEDED,C,IER)

```

where $MC = 2 + 3*NDOM + \sum_{i=1}^{NDOM} (M_i + 2 * K_i) + \prod_{i=1}^{NDOM} (M_i + K_i)$.

ARGUMENTS

N	[INPUT, INTEGER] Number of data points; $N \geq 2$.
X	[INPUT, DOUBLE PRECISION, ARRAY] Two-dimensional array containing the N values of the independent variables in the x_1 through x_{NDOM} directions which are stored in the first through NDOM columns of X, respectively. This array need not be ordered but it must represent N distinct points and satisfy the condition $\min_{\ell} \{X(\ell, i)\} < \max_{\ell} \{X(\ell, i)\}$ for $i = 1, \dots, NDOM$.
NDIM	[INPUT, INTEGER] Dimensional constant for the array X; $NDIM \geq N$.
NDOM	[INPUT, INTEGER] Number of independent variables (n in METHOD); $1 \leq NDOM \leq 10$.
Y	[INPUT, DOUBLE PRECISION, ARRAY] Array of N values of the dependent variable y in correspondence with the X array.
ITOL	[INPUT, INTEGER] Absolute tolerance option flag;

ITOL=0	HDGDWH uses $\delta_\ell = \text{ABSTOL}(1)$ for all ℓ .
ITOL=1	The user must specify the tolerances δ_ℓ , for $\ell = 1, \dots, N$. See description of ABSTOL below.
ABSTOL	[INPUT, DOUBLE PRECISION, ARRAY] Absolute error tolerance. The value of ABSTOL depends on the absolute tolerance option flag ITOL.
ITOL = 0	ABSTOL(1) specifies the error tolerances δ_ℓ , for $\ell = 1, \dots, N$, of the constraints in 8.6; $\text{ABSTOL}(1) \geq \sqrt{\text{HDMCON}(5)}$.
ITOL = 1	Array of N values in correspondence with the X array. $\text{ABSTOL}(\ell)$ specifies the error tolerances δ_ℓ , for $\ell = 1, \dots, N$, of the constraints in 8.6. If $\text{ABSTOL}(\ell) = \text{HDMCON}(2)$, then the ℓ -th point is not used. It must satisfy $\text{ABSTOL}(\ell) \geq \sqrt{\text{HDMCON}(5)}$ for all ℓ .
NDEG	[INPUT, INTEGER, ARRAY] Array of NDOM values where $\text{NDEG}(i)$ is the degree of the spline in x_i ; $1 \leq \text{NDEG}(i) \leq 3$.
MONO	[INPUT, INTEGER, ARRAY] Monotonicity option. Array of NDOM values where $\text{MONO}(i)$ specifies if the spline is locally monotonic in x_i along the data grid lines.
$\text{MONO}(i) = 0$	The spline will not be locally monotonic in x_i .
$\text{MONO}(i) = 1$	The spline will be locally monotonic in x_i .
CONT	[INPUT, INTEGER, ARRAY] Continuity option. Array of NDOM values where $\text{CONT}(i)$ specifies if the spline has full or reduced continuity at the grid points in x_i .
$\text{CONT}(i) = 0$	Spline has full continuity in x_i at the grid points. If the spline is linear in x_i , then it will be continuous everywhere in x_i . If the spline is quadratic in x_i , then the first order partial in x_i will be continuous everywhere. If the spline is cubic in x_i , then the second order partial in x_i will be continuous everywhere.
$\text{CONT}(i) = 1$	Spline has reduced continuity in x_i at the grid points. If the spline is linear in x_i , then it will be continuous everywhere in x_i . If the spline is quadratic in x_i ; then the first order partial in x_i will be continuous everywhere, except at grid points. If the spline is cubic in x_i ; then the second order partial in x_i will be continuous everywhere, except at grid points.

IBND	[INPUT, INTEGER] Bound options flag; Specifies if upper and lower bounds are imposed on the spline approximation. These bounds will only be in affect in independent variable regions without data. IBND=0 No bounds are imposed on the approximation. IBND=1 HDGDWH imposes the condition $s_l \leq s(X) \leq s_u$ where $s_l = \min_{\ell} \{Y(\ell) - \text{ABSTOL}(\ell)\}$ and $s_u = \max_{\ell} \{Y(\ell) + \text{ABSTOL}(\ell)\}$. IBND=2 HDGDWH imposes the condition $s_l \leq s(X) \leq s_u$ where $s_l = \text{BOUNDS}(1)$ and $s_u = \text{BOUNDS}(2)$.
BOUNDS	[INPUT, DOUBLE PRECISION, ARRAY] Array of two elements containing the lower and upper bounds. The contents of BOUNDS depends on the bound options flag IBND. IBND = 0 The contents of BOUNDS are not used but it must be present in the calling sequence. IBND = 1 HDGDWH sets $\text{BOUNDS}(1) = \min_{\ell} \{Y(\ell) - \text{ABSTOL}(\ell)\}$ and $\text{BOUNDS}(2) = \max_{\ell} \{Y(\ell) + \text{ABSTOL}(\ell)\}$ and imposes the condition $\text{BOUNDS}(1) \leq s(X) \leq \text{BOUNDS}(2)$. IBND = 1 The users must specify the values of BOUNDS. HDGDWH imposes the condition $\text{BOUNDS}(1) \leq s(X) \leq \text{BOUNDS}(2)$. The elements of BOUNDS must satisfy $\text{BOUNDS}(1) \leq \min_{\ell} \{Y(\ell) - \text{ABSTOL}(\ell)\}$ and $\text{BOUNDS}(2) \geq \max_{\ell} \{Y(\ell) + \text{ABSTOL}(\ell)\}$. If $\text{BOUNDS}(1) = \text{HDMCON}(2)$, then no lower bound is imposed on the spline. Similarly, no upper bound is imposed if $\text{BOUNDS}(2) = \text{HDMCON}(2)$.
WORK	[INPUT, DOUBLE PRECISION, ARRAY] Work array of length NWORK.
NWORK	[INPUT, INTEGER] Length of array WORK, HDGDWH can not determine a priori the amount of real workspace needed. NWORK is checked at each stage of the algorithm to determine if sufficient workspace is provided. If sufficient workspace is not provided, then HDGDWH will report an error condition and return the amount of real workspace required in NEEDED.
IWORK	[INPUT, INTEGER, ARRAY] Work array of length NIWORK.
NIWORK	[INPUT, INTEGER] Length of array IWORK, HDGDWH can not determine a priori the amount of

integer workspace needed. NIWORK is checked at each stage of the algorithm to determine if sufficient workspace is provided. If sufficient workspace is not provided, then HDGDWH will report an error condition and return the amount of integer workspace required in NEEDED.

MCOUT	[INPUT, INTEGER] Length of array C; $MCOUT \geq MC$. If sufficient space is not provided, then HDGDWH will report an error condition and return the minimum required value of MCOUT in NEEDED.
NEEDED	[OUTPUT, INTEGER] Specifies the minimum required value of MCOUT or the amount of integer or real workspace required.
C	[OUTPUT, DOUBLE PRECISION, ARRAY] Array containing the information needed to evaluate the spline.
IER	[OUTPUT, INTEGER] Success/error code. Results are not computed for $IER < 0$ and HDGDWH sets $C(1) = -1.0$.
IER = 0	Success, results computed.
IER = 2	The resulting spline satisfies the approximation and monotonicity conditions but may not have minimal variation in its second order partials.
IER = 1	The resulting spline satisfies the approximation and monotonicity conditions, has minimal variation in its second order partials, but may exhibit undesirable behavior.
IER=-1	$NDEG(i) < 0$ or $NDEG(i) > 3$ for some $i = 1, \dots, NDOM$.
IER=-2	$N < 2$.
IER=-3	NWORK too small; the number of words of real workspace needed to continue is returned in NEEDED.
IER=-4	NIWORK too small; the number of words of integer workspace needed to continue is returned in NEEDED.
IER=-5	$NDIM < N$.
IER=-7	$\min_{\ell}\{X(\ell, i)\} = \max_{\ell}\{X(\ell, i)\}$ for some $i = 1, \dots, NDOM$.
IER=-8	The array X does not contain N distinct points.
IER=-9	$ITOL \neq 0$ and $ITOL \neq 1$.

IER=-10	If ITOL = 0, then $\text{ABSTOL}(1) < \sqrt{\text{HDMCON}(5)}$. If ITOL = 1, then $\text{ABSTOL}(\ell) < \sqrt{\text{HDMCON}(5)}$ for some $\ell = 1, \dots, N$.
IER=-12	NDOM < 1 or NDOM > 10.
IER=-14	MONO(i) $\neq$ 0 and MONO(i) $\neq$ 1 for some $i = 1, \dots, \text{NDOM}$.
IER=-15	CONT(i) $\neq$ 0 and CONT(i) $\neq$ 1 for some $i = 1, \dots, \text{NDOM}$.
IER=-16	IBND < 0 or IBND > 2.
IER=-17	For IBND = 2, either $\text{BOUNDS}(1) > \min_{\ell} \{Y(\ell) - \text{ABSTOL}(\ell)\}$ or $\text{BOUNDS}(2) < \max_{\ell} \{Y(\ell) + \text{ABSTOL}(\ell)\}$.
IER=-18	MCOUT too small; the minimum required value of MCOUT is returned in NEEDED.
IER=-40	The approximation and monotonicity constraints do not have a feasible point. See discussion under USAGE REMARKS.
IER=-43	The approximation and monotonicity constraints are singular. The user should consider relaxing the absolute error tolerance. See discussion under USAGE REMARKS.
IER=-100	Unexpected error return, see explanation of MODE = 5 error message from HHERR.
IER=-200	Numerical error, the user should consider relaxing the absolute error tolerance. See discussion under USAGE REMARKS.
IER=-300	The user is using one of the input/output units required by the sparse optimization routine SNLPMN, see the discussion of IER = -137 error return in the SNLPMN documentation for additional information.

EXAMPLE

This example computes a least squares approximation of data sampled from the function $z = x^2y^3$. The data is defined on a 6 by 6 grid over the region $0.0 \leq x \leq 1.0$, $0.0 \leq y \leq 1.0$ such that $0.4 \leq \sqrt{x^2 + y^2} \leq 1.0$. The spline approximation will be quadratic in x and cubic in y and will approximate the data within 0.00001. Monotonicity constraints will be impose and the spline will have full continuity at the grid points in both independent variables.

SAMPLE PROGRAM (exgdwh.cd1)⁴

⁴sosexample.tar

```

PROGRAM EXAMPLE

C -----
C ... PROGRAM TO ILLUSTRATE EXAMPLE PROBLEM
C -----

C ... HDGDWH PARAMETERS

      INTEGER          N          , NDOM  , NDIM  , IBND  , ITOL  , NWORK,
+          NIWORK, MCOUT , NEEDED, IER

      PARAMETER ( NDOM = 2, NDIM = 36, NWORK = 100000, NIWORK = 100000 )
      PARAMETER ( MCOUT = 238 )

      INTEGER          NDEG(NDOM), MONO(NDOM), CONT(NDOM), IWORK(NWORK)

      DOUBLE PRECISION ABSTOL(1)   , C(MCOUT), WORK(NWORK),
+          X(NDIM,NDOM), Y(NDIM) , BOUNDS(2)

C ... LOCAL VARIABLES

      INTEGER          I          , J

      DOUBLE PRECISION RADIUS, VAL  , XTMP(2)

C -----
C ... DEFINE THE DATA TO APPROXIMATE. DATA REPRESENTS THE FUNCTION
C ... Z = X**2Y**3
C -----

C ... DEFINE FUNCTION DATA

      N = 0

      DO J = 1, 6

        DO I = 1, 6

          XTMP(1) = DBLE ( I - 1 ) * 2.0D-1
          XTMP(2) = DBLE ( J - 1 ) * 2.0D-1
          RADIUS  = SQRT ( XTMP(1)**2 + XTMP(2)**2 )

          IF ( 4.0D-1 .LE. RADIUS .AND. RADIUS .LE. 1.0D0 ) THEN

            N      = N + 1
            X(N,1) = XTMP(1)
            X(N,2) = XTMP(2)
            Y(N)   = XTMP(1)**2*XTMP(2)**3
          
```

```

        END IF

        END DO

    END DO

C  -----
C  ... SPECIFY THE DEGREE, MONOTONICITY, AND CONTINUITY CONDITIONS
C  -----

    ITOL      = 0
    ABSTOL(1) = 1.0D-5

    IBND = 1

    NDEG(1) = 2
    NDEG(2) = 3

    MONO(1) = 1
    MONO(2) = 1

    CONT(1) = 0
    CONT(2) = 0

C  -----
C  ... CALL HDGDWH
C  -----

    CALL HDGDWH ( N      , X      , NDIM , NDOM , Y      , ITOL ,
+               ABSTOL, NDEG  , MONO  , CONT , IBND , BOUNDS,
+               WORK   , NWORK , IWORK , NIWORK, MCOUT , NEEDED,
+               C      , IER )

    IF ( IER .NE. 0 ) THEN

        WRITE ( *, 1000 ) IER
        IF ( IER .LT. 0 ) STOP

    END IF

C  -----
C  ... EVALUATE SPLINE APPROXIMATION
C  -----

    N = 0
    WRITE ( *, 1200 )

```

```

DO J = 1, 6

    DO I = 1, 6

        XTMP(1) = DBLE ( I - 1 ) * 2.0D-1
        XTMP(2) = DBLE ( J - 1 ) * 2.0D-1
        RADIUS  = SQRT ( XTMP(1)**2 + XTMP(2)**2 )

        CALL HDNPVL ( XTMP, 1, C, WORK, NWORK, VAL, IER )

        IF ( IER .LT. 0 ) THEN

            WRITE ( *, 1100 ) IER
            STOP

        END IF

        IF ( 4.0D-1 .LE. RADIUS .AND. RADIUS .LE. 1.0D0 ) THEN

            N = N + 1
            WRITE ( *, 1300 ) XTMP(1), XTMP(2), VAL, Y(N),
+                               ABS ( Y(N) - VAL )

        ELSE

            WRITE ( *, 1300 ) XTMP(1), XTMP(2), VAL

        END IF

    END DO

END DO

WRITE ( *, 1400 )

C -----

1000 FORMAT ( ' ERROR RETURN FROM HDGDWH - IER = ', I5 )
1100 FORMAT ( ' ERROR RETURN FROM HDNPVL - IER = ', I5 )
1200 FORMAT(/1X,' X      Y      S(X,Y)    Z(X,Y)    ABS DIFF.',
+          /1X,'=====')
1300 FORMAT(2(1X,F6.3),3(1X,F9.6))
1400 FORMAT ( / ' * NORMAL TERMINATION *' )
      STOP
      END

```

OUTPUT FROM SAMPLE PROGRAM

X	Y	S(X,Y)	Z(X,Y)	ABS DIFF.
=====				
0.000	0.000	-0.000010		
0.200	0.000	-0.000010		
0.400	0.000	-0.000010	0.000000	0.000010
0.600	0.000	-0.000010	0.000000	0.000010
0.800	0.000	-0.000010	0.000000	0.000010
1.000	0.000	-0.000010	0.000000	0.000010
0.000	0.200	-0.000010		
0.200	0.200	0.000097		
0.400	0.200	0.001290	0.001280	0.000010
0.600	0.200	0.002870	0.002880	0.000010
0.800	0.200	0.005130	0.005120	0.000010
1.000	0.200	0.021279		
0.000	0.400	-0.000010	0.000000	0.000010
0.200	0.400	0.002570	0.002560	0.000010
0.400	0.400	0.010230	0.010240	0.000010
0.600	0.400	0.023030	0.023040	0.000010
0.800	0.400	0.040970	0.040960	0.000010
1.000	0.400	0.087951		
0.000	0.600	-0.000010	0.000000	0.000010
0.200	0.600	0.008630	0.008640	0.000010
0.400	0.600	0.034570	0.034560	0.000010
0.600	0.600	0.077770	0.077760	0.000010
0.800	0.600	0.138230	0.138240	0.000010
1.000	0.600	0.156389		
0.000	0.800	-0.000010	0.000000	0.000010
0.200	0.800	0.020490	0.020480	0.000010
0.400	0.800	0.081930	0.081920	0.000010
0.600	0.800	0.184310	0.184320	0.000010
0.800	0.800	0.184330		
1.000	0.800	0.184330		
0.000	1.000	-0.000010	0.000000	0.000010
0.200	1.000	0.062586		
0.400	1.000	0.147311		
0.600	1.000	0.184330		
0.800	1.000	0.184330		
1.000	1.000	0.181540		

* NORMAL TERMINATION *

HDCMVS: Constrained Tensor Product Approximation of Multivariate Data

PURPOSE

HDCMVS computes a tensor product spline approximation of multivariate data. The approximation can be either: a least squares approximation; a weighted least squares approximation; or an approximation with minimal variation of its first or second order partials, which is within an user specified tolerance of the data. Additionally, HDCMVS allows the user to define constraints on the spline and its partial derivatives.

RELATED SUBPROGRAMS

HDNPVL Evaluation of a Tensor Product Spline

HDNPDR Evaluation of a Partial Derivative of a Tensor Product Spline

METHOD

To specify the data the user must provide values for the n independent variables $X_\ell = (x_{1,\ell}, \dots, x_{n,\ell})$ and corresponding values of the dependent variable y_ℓ , for $\ell = 1, \dots, N$. The dependent variable data y_ℓ can represent either function values or partial derivative values. In the latter case the user must specify the indices of the partial derivative, $(\gamma_{1,\ell}, \dots, \gamma_{n,\ell})$, which y_ℓ represents.

HDCMVS defines the tensor product spline $s(X)$ for $X = (x_1, \dots, x_n)$ of degree $K_i - 1$ in x_i as

$$s(X) = \sum_{j_n=1}^{M_n+K_n} \cdots \sum_{j_1=1}^{M_1+K_1} A_{j_1, \dots, j_n} \prod_{i=1}^n B_{j_i}(x_i|T_i) \quad (8.9)$$

where $B_{j_i}(x_i|T_i)$ is the value at x_i of the j_i -th B-spline defined by the knot vector T_i . If $x_l^i = \min_\ell \{x_{i,\ell}\}$, $x_u^i = \max_\ell \{x_{i,\ell}\}$, and $t_{i,k}$ for $k = 1, \dots, M_i$, represents the internal knots, then

$$T_i = (\overbrace{x_l^i, \dots, x_l^i}^{K_i \text{ times}}, t_{i,1}, t_{i,2}, \dots, t_{i,M_i}, \overbrace{x_u^i, \dots, x_u^i}^{K_i \text{ times}}) \quad (8.10)$$

Notice the end knots have multiplicity K_i . The number of B-splines, $B_{j_i}(x_i|T_i)$, in the x_i -th independent variable is $M_i + K_i$. The number of coefficients, $A_{j_1, \dots, j_n}$, is $\prod_{i=1}^n (M_i + K_i)$.

To define the spline the user must specify the degree, $K_i - 1$, in each of the independent variables and, optionally, the internal knots. Then for a given set of data (X_ℓ, y_ℓ) and, optionally, spline partial derivative indices $(\gamma_{1,\ell}, \dots, \gamma_{n,\ell})$; HDCMVS will compute a tensor product spline approximation of the data. The spline approximation can be either: a least squares approximation of the data; a weighted least squares approximation of the data; an approximation with minimal variation of its first order partials, which is within an user specified tolerance of the data; or an approximation with minimal variation of its second order partials, which is within an user specified tolerance of the data. The user selects one of these options with the approximation option flag.

For the first two options HDCMVS computes the coefficients $A = (A_{j_1, \dots, j_n})^T$ which minimize

$$E(A) = \sum_{\ell=1}^N w_{\ell} \left(y_{\ell} - \frac{\partial s^{(\gamma_1, \ell + \dots + \gamma_n, \ell)}}{\partial x_1^{\gamma_1, \ell} \dots \partial x_n^{\gamma_n, \ell}}(X_{\ell}) \right)^2. \quad (8.11)$$

If the spline is a least squares approximation, then $w_{\ell} = 1.0$ for all ℓ . A weighted least squares approximation is obtained by specifying $w_{\ell} \geq 0.0$, for $\ell = 1, \dots, N$. For the latter two options the user specifies error tolerances, $\epsilon_{\ell} \geq 0$, and HDCMVS computes the coefficients A which minimize either

$$E(A) = \int \left(\frac{\partial s(X)}{\partial x_1} \right)^2 dx_1 + \dots + \int \left(\frac{\partial s(X)}{\partial x_n} \right)^2 dx_n \quad (8.12)$$

or

$$E(A) = \int \left(\frac{\partial^2 s(X)}{\partial^2 x_1} \right)^2 dx_1 + \dots + \int \left(\frac{\partial^2 s(X)}{\partial^2 x_n} \right)^2 dx_n \quad (8.13)$$

subject to the constraints

$$y_{\ell} - \epsilon_{\ell} \leq \frac{\partial s^{(\gamma_1, \ell + \dots + \gamma_n, \ell)}}{\partial x_1^{\gamma_1, \ell} \dots \partial x_n^{\gamma_n, \ell}}(X_{\ell}) \leq y_{\ell} + \epsilon_{\ell} \quad (8.14)$$

for $\ell = 1, \dots, N$.

Independent of which of the above four options is chosen, the user can impose constraints on the spline. These constraints are of the form

$$\frac{\partial s^{(\lambda_{1,j} + \dots + \lambda_{n,j})}(X)}{\partial x_1^{\lambda_{1,j}} \dots \partial x_n^{\lambda_{n,j}}} \quad \left\{ \begin{array}{l} \geq \\ \leq \\ = \end{array} \right\} \quad g_j(X) \quad (8.15)$$

for $j = 1, \dots, NCON$. The indices $\lambda_{1,j}$ through $\lambda_{n,j}$ specify the order of the partial derivative in x_1 through x_n , respectively, that is bounded by $g_j(X)$. The bounds $g_j(X)$ are either univariate or multivariate splines. If $g_j(X)$ is an univariate spline, which depends on x_i , then it must be of the form

$$g_j(x_i) = \sum_{l_i=1}^{m_{i,j}} \beta_{l_i} G_{l_i}(x_i | S_i) \quad (8.16)$$

where $G_{l_i}(x_i | S_i)$ are B-splines of order $k_{i,j}$ defined by the knot vector S_i . If $g_j(X)$ is a multivariate spline, then it must be of the form

$$g_j(X) = \sum_{l_{n_j}=1}^{m_{n_j,j}} \dots \sum_{l_1=1}^{m_{1,j}} \beta_{l_1, \dots, l_{n_j}} \prod_{i=1}^{n_j} G_{l_i}(x_i | S_i) \quad (8.17)$$

where $G_{l_i}(x_i|S_i)$ are B-splines of order $k_{i,j}$ defined by the knot vector S_i ; n_j is the number of independent variables for $g_j(X)$, which must satisfy $n_j \leq n$; and X is a vector of n_j elements.

Whenever $n_j < n$ the user must specify the indices of the independent variables for $g_j(X)$ and values for the remaining independent variables. For example, suppose the user wishes to impose a constraint on a spline $s(X)$ which is a function of three variables, (x, y, z) . If the bounding spline $g(X)$ associated with the constraint is a function of the variables (x, z) , then the user must specify the value of y at which the constraint is imposed.

For a given approximation option and set of constraints, HDCMVS formulates a linearly constrained quadratic programming problem in the following manner. First, HDCMVS defines a set of tensor product B-splines

$$\Phi_{j_1, \dots, j_n}(X) = \prod_{i=1}^n C_{j_i}(x_i|\Lambda_i) \quad (8.18)$$

which forms a common basis for the spline $s(X)$, the NCON partial derivatives of the spline, and the NCON constraint splines $g_j(X)$. The $C_{j_i}(x_i|\Lambda_i)$'s, for $j_i = 1, \dots, M_{c_i}$, are B-splines of order K_{c_i} where

$$K_{c_i} = \max\{K_i, \max_j \{k_{i,j}\}\} \quad (8.19)$$

and

$$M_{c_i} \leq \sum_{j=1}^{\text{NCON}} (m_{i,j} * (K_{c_i} - k_{i,j} + 1) + (M_i + K_i) * (K_{c_i} - K_i + 1)). \quad (8.20)$$

After defining the common tensor product basis the constraints represented by Equation (8.15) are converted into linear constraints of the form

$$\begin{pmatrix} l_1 \\ l_2 \\ \vdots \\ l_{\text{NCON}} \end{pmatrix} \leq \begin{pmatrix} D_1 \\ D_2 \\ \vdots \\ D_{\text{NCON}} \end{pmatrix} A \leq \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{\text{NCON}} \end{pmatrix} \quad (8.21)$$

where $D_j A$ are the coefficients of the partial derivative of the spline $s(X)$ with respect to the common tensor product B-splines $\Phi_{j_1, \dots, j_n}(X)$. The vectors l_j and u_j are the coefficients of the constraint splines $g_j(X)$ with respect to the common tensor product B-splines. If the j -th constraint is an equality constraint, then $l_j = u_j$.

Then, HDCMVS employs either the sparse nonlinear optimization package SNLPMN or SBRNLP to solve the quadratic programming problem, minimize $E(A)$ subject to the linear constraints defined by Equation (8.21). SNLPMN or SBRNLP are designed to locate the minimum of a quadratic function subject to linear constraints. They utilize sparse linear algebra techniques and are capable of solving large sparse optimization problems. See SNLPMN (p. 67) or SBRNLP (p. 171) for more information.

HDCMVS provides the user with an option to regularize rank deficient problems. A problem is considered rank deficient if the location of the data, the placement of the internal knots, and the selected approximation option do not uniquely define the spline coefficients. Three options are provided for handling rank deficient problems.

If the first option is chosen, then HDCMVS will not determine if the problem is rank deficient. If the second option is chosen, then HDCMVS will inform the user of any rank deficiencies via the error return code. In either case the problem is not regularized. If the third option is chosen, then the problem is regularized in the following manner.

First, HDCMVS determines the spline coefficients which are not uniquely defined by the data and approximation option. Then two optimizations are performed. During the first optimization, the spline coefficients which minimized the approximation option and satisfy the constraints are computed. The second optimization computes the coefficients which minimize the second partial derivative with respect to each independent variable and satisfies the constraints. During the second optimization the spline coefficients which were uniquely defined by the data remain fixed and equal to their previously computed values. If the second optimization fails, then HDCMVS returns the result of the first optimization.

USAGE REMARKS

Whether or not a set of spline coefficients which satisfies the constraints exists depends on the type of constraints, the constraint splines, and the internal knots of the spline approximation. That is, a feasible point may exist for some values of the internal knots but not for others. Therefore, the user must define a set of knots which allow sufficient flexibility for the spline to satisfy the constraints.

Also, the optimization routines SNLPMN and SBRNLP require upper and lower bounds for the spline coefficients. HDCMVS imposes the bounds

$$-10^5\delta_y \leq \alpha_{j_1, \dots, j_n} - \min_{\ell} \{y_{\ell}\} \leq 10^5\delta_y \quad (8.22)$$

where $\delta_y = \max_{\ell} \{y_{\ell}\} - \min_{\ell} \{y_{\ell}\}$. If a bound is active at the solution, then HDCMVS will inform the user via the error return code. In this case, the user should consider imposing additional constraints on the spline approximation.

WARNING

It may be necessary for HDCMVS to create scratch files to hold information required to define the constraint Jacobian. These binary files are associated with input/output units 18 and 19. These input/output units are reserved for HDCMVS and can not be utilized by the user. If an error occurs while accessing these input/output units, then HDCMVS reports an error condition.

USAGE

```

INTEGER N, NDIM, NDOM, IFUN, IDRV, IDIM, IREG, MDIM, NCON, LDIM, GDIM, NWORK
INTEGER NIWORK, MCOU, NEEDED, IER
INTEGER IGAMMA(IDIM,NDOM), NDEG(NDOM), NKNOTS(NDOM), LAMBDA(LDIM,NCON)
INTEGER TYPE(NCON), MASK(NCON), IWORK(NIWORK)

```

```
DOUBLE PRECISION X(NDIM,NDOM), Y(N), WTOL(N), T(MDIM,NDOM), GSPLN(GDIM,NCON)
DOUBLE PRECISION FIXVAL(LDIM,NCON), WORK(NWORK), C(MC)
```

```
CALL HDCMVS (N,X,NDIM,NDOM,Y,IFUN,WTOL,IDRV,IGAMMA,IDIM,IREG,NDEG,NKNOTS,T,MDIM,
+           NCON,LAMBDA,TYPE,MASK,GSPLN,GDIM,FIXVAL,LDIM,WORK,NWORK,IWORK,
+           NIWORK,MCOUT,NEEDED,C,IER)
```

where $MC = 2 + 3 \cdot NDOM + \sum_{i=1}^{NDOM} (M_i + 2 * K_i) + \prod_{i=1}^{NDOM} (M_i + K_i)$.

ARGUMENTS

- N** [INPUT, INTEGER]
Number of data points; $N \geq 2$.
- X** [INPUT, DOUBLE PRECISION, ARRAY]
Two-dimensional array containing the N values of the independent variables in the x_1 through x_{NDOM} directions which are stored in the first through $NDOM$ columns of X , respectively. This array need not be ordered but it must satisfy the condition $\min_{\ell} \{X(\ell, i)\} < \max_{\ell} \{X(\ell, i)\}$ for $i = 1, \dots, NDOM$.
- NDIM** [INPUT, INTEGER]
Dimensional constant for the array X ; $NDIM \geq N$.
- NDOM** [INPUT, INTEGER]
Number of independent variables (n in METHOD section); $1 \leq NDOM \leq 10$.
- Y** [INPUT, DOUBLE PRECISION, ARRAY]
Array of N values of the dependent variable y in correspondence with the X array.
- IFUN** [INPUT, INTEGER]
Approximation option flag; $0 \leq IFUN \leq 7$. When $IFUN \leq 3$, the sparse SQP algorithm SNLPMN (p. 67) is used, and when $IFUN > 3$, the sparse barrier algorithm SBRNLP (p. 171) is used.
- IFUN=0 or 4 HDCMVS computes a least squares spline approximation of the data. It uses $w_{\ell} = 1.0$ for all ℓ and the WTOL array is not used, but it must be present in the calling sequence.
- IFUN=1 or 5 HDCMVS computes a weighted least squares spline approximation of the data. The user must specify the weights, w_{ℓ} , for $\ell = 1, \dots, N$. See description of WTOL below.
- IFUN=2 or 6 HDCMVS computes a spline approximation which has minimal variation of its first order partials and is within an user specified tolerance of the data. The user must specify the error tolerances, ϵ_{ℓ} , for $\ell = 1, \dots, N$. See the description of WTOL below and the discussion under USAGE REMARKS.

IFUN=3 or 7 HDCMVS computes a spline approximation which has minimal variation of its second order partials and is within an user specified tolerance of the data. The user must specify the error tolerances, $\epsilon_\ell \geq 0.0$, for $\ell = 1, \dots, N$. See the description of WTOL below and the discussion under USAGE REMARKS.

WTOL [INPUT, DOUBLE PRECISION, ARRAY]
 Array of N values in correspondence to the X array. The contents of WTOL depends on the approximation option flag IFUN.

IFUN= 0 or 4 WTOL is not used but must be present in the calling sequence.

IFUN= 1 or 5 WTOL specifies the weights, w_ℓ , for $\ell = 1, \dots, N$; used for weighted least squares approximations. If $WTOL(\ell) = HDMCON(2)$, then HDCMVS will interpolate the ℓ -th point. If $WTOL(\ell) \leq 0.0$, then the ℓ -th point is not used.

IFUN= 2 or 6 WTOL specifies the error tolerances, ϵ_ℓ , for $\ell = 1, \dots, N$; used for the constraints in Equation (8.14). If $WTOL(\ell) = 0.0$, then HDCMVS will interpolate the ℓ -th point. If $WTOL(\ell) = HDMCON(2)$, then the ℓ -th point is not used.

IFUN= 3 or 7 WTOL specifies the error tolerances, ϵ_ℓ , for $\ell = 1, \dots, N$; used for the constraints in Equation (8.14). If $WTOL(\ell) = 0.0$, then HDCMVS will interpolate the ℓ -th point. If $WTOL(\ell) = HDMCON(2)$, then the ℓ -th point is not used.

IDRV [INPUT, INTEGER]
 Derivative option.

IDRV=0 HDCMVS assumes the data represents function values and the array IGAMMA is not used.

IDRV $\neq$ 0 HDCMVS uses the partial derivative indices provided in the array IGAMMA.

IGAMMA [INPUT, INTEGER, ARRAY]
 Two-dimensional array in correspondence with the X array which specifies the order of the partial derivative represented by Y. If IDRV = 0, IGAMMA is not used but must be present in the calling sequence. If IDRV $\neq$ 0, then $IGAMMA(\ell, i)$ must satisfy $0 \leq IGAMMA(\ell, i) \leq NDEG(i)$ for all ℓ and $i = 1, \dots, NDOM$.

IDIM [INPUT, INTEGER]
 Dimensional constant for the array IGAMMA; IDIM $\geq$ N.

IREG [INPUT, INTEGER]
 Regularization option; $0 \leq IREG \leq 2$.

IREG=0 HDCMVS does not determine if the problem is rank deficient and no regularization is performed.

	IREG=1	HDCMVS determines if the problem is rank deficient but no regularization is performed.
	IREG=2	HDCMVS determines if the problem is rank deficient and performs a second optimization which minimizes the second partial derivatives of the spline, see discussion under METHOD.
NDEG	[INPUT, INTEGER, ARRAY]	Array of NDOM values where NDEG(i) is the degree of the spline in x_i ; NDEG(i) ≥ 0 .
NKNOTS	[INPUT, INTEGER, ARRAY]	Array of NDOM values where NKNOTS(i) is the number of internal knots in x_i ; NKNOTS(i) ≥ 0 .
T	[INPUT, DOUBLE PRECISION, ARRAY]	Two-dimensional array containing the internal knots in the x_1 through x_{NDOM} variables which are stored in the first through NDOM columns, respectively. The i -th column of T must be in ascending order with multiplicity $\leq \text{NDEG}(i) + 1$; that is, no more than NDEG(i)+1 consecutive values of the i -th column of T may be equal. It is also required that $T(1, i) > \min_{\ell} \{X(\ell, i)\}$ and $T(\text{NKNOTS}(i), i) < \max_{\ell} \{X(\ell, i)\}$. If NKNOTS(1) through NKNOTS(NDOM) are equal to zero, T is not used but must be present in the calling sequence.
MDIM	[INPUT, INTEGER]	Dimensional constant for the array T; $\text{MDIM} \geq \max_{i=1, \dots, \text{NDOM}} \text{NKNOTS}(i)$.
NCON	[INPUT, INTEGER]	Number of constraints imposed on the spline; NCON ≥ 0 .
LAMBDA	[INPUT, INTEGER, ARRAY]	Two-dimensional array of NCON columns where the j -th column of LAMBDA specifies the partial derivative that is to be constrained by the spline defined in the j -th column of GSPLN. The first through NDOM rows of LAMBDA specify the order of the partial derivative in the x_1 through x_{NDOM} variables, respectively. LAMBDA(i, j) must satisfy $0 \leq \text{LAMBDA}(i, j) \leq \text{NDEG}(i)$ for all $i = 1, \dots, \text{NDOM}$ and $j = 1, \dots, \text{NCON}$.
TYPE	[INPUT, INTEGER, ARRAY]	Array of NCON values where TYPE(j) specifies the type of constraint represented by the j -th column of GSPLN. TYPE(j) = -1 The j -th column of GSPLN represents a lower bound. TYPE(j) = 0 The j -th column of GSPLN represents an equality constraint. TYPE(j) = 1 The j -th column of GSPLN represents an upper bound.

MASK	[INPUT, INTEGER, ARRAY] Array of NCON values where MASK(j) specifies the data mask of constraint represented by the j-th column of GSPLN. MASK(j) = -1 Impose constraint in regions without data. MASK(j) = 0 Impose constraint everywhere. MASK(j) = 1 Impose constraint in regions containing data.
GSPLN	[INPUT, DOUBLE PRECISION, ARRAY] Two-dimensional array which defines the NCON splines g_j where the j-th column of GSPLN represents the j-th constraint. These splines must be stored in the standard spline format. Additionally, for $j = 1, \dots, \text{NCON}$, the elements of GSPLN must satisfy the following conditions; $1 \leq \text{GSPLN}(1,j) \leq \text{NDOM}$, $\text{GSPLN}(2,j) = 1$, and the minimum and maximum knots must be between the minimum and maximum independent variable values.
GDIM	[INPUT, INTEGER] Dimensional constant for the array GSPLN; $\text{GDIM} \geq \max_{j=1, \dots, \text{NCON}} G_j$ where, if $\text{GDOM} = \text{GSPLN}(1,j)$, $G_j = \sum_{i=1}^{\text{GDOM}} (2 * \text{GSPLN}(2 + i, j) + \text{GSPLN}(2 + \text{GDOM} + i, j))$ $+ \prod_{i=1}^{\text{GDOM}} (\text{GSPLN}(2 + i, j) + \text{GSPLN}(2 + \text{GDOM} + i, j))$ $+ 2 + 3 * \text{GDOM}$
FIXVAL	[INPUT, DOUBLE PRECISION, ARRAY] Two-dimensional array which defines the independent variables of the NCON splines represented in GSPLN. If $\text{FIXVAL}(i, j) = \text{HDMCON}(2)$ then the spline represented by the j-th column of GSPLN is a function of the x_i-th variable. Otherwise $\text{FIXVAL}(i, j)$ must satisfy $\min_{\ell} \{X(\ell, i)\} \leq \text{FIXVAL}(i, j) \leq \max_{\ell} \{X(\ell, i)\}$ and $\text{FIXVAL}(i, j)$ defines the value of x_i at which the constraint is imposed.
LDIM	[INPUT, INTEGER] Dimensional constant for the arrays FIXVAL and LAMBDA; $\text{LDIM} \geq \text{NDOM}$.
WORK	[INPUT, DOUBLE PRECISION, ARRAY] Work array of length NWORK.
NWORK	[INPUT, INTEGER] Length of array WORK, HDCMVS can not determine a priori the amount of real

workspace needed. NWORK is checked at each stage of the algorithm to determine if sufficient workspace is provided. If sufficient workspace is not provided, then HDCMVS will report an error condition and return the amount of real workspace required in NEEDED.

IWORK	[INPUT, INTEGER, ARRAY] Work array of length NIWORK.
NIWORK	[INPUT, INTEGER] Length of array IWORK, HDCMVS can not determine a priori the amount of integer workspace needed. NIWORK is checked at each stage of the algorithm to determine if sufficient workspace is provided. If sufficient workspace is not provided, then HDCMVS will report an error condition and return the amount of integer workspace required in NEEDED.
MCOUT	[INPUT, INTEGER] Length of array C; $MCOUT \geq MC$. If sufficient space is not provided, then HDCMVS will report an error condition and return minimum required value of MCOUT in NEEDED.
NEEDED	[OUTPUT, INTEGER] Specifies the minimum required value of MCOUT or the amount of integer or real workspace required to proceed.
C	[OUTPUT, DOUBLE PRECISION, ARRAY] Array containing the information needed to evaluate the spline.
IER	[OUTPUT, INTEGER] Success/error code. Results are not computed for $IER < 0$ and HDCMVS sets $C(1) = -1.0$. $IER = 0$ Success, results computed. $IER = 6$ Second regularization optimization failed. $IER = 5$ A suboptimal feasible spline has been computed. $IER = 4$ Some spline coefficients are equal to their bounds but rank deficiency has not been determined. $IER = 3$ The problem is not rank deficient and some spline coefficients are equal to their bounds. $IER = 2$ The problem is rank deficient and some spline coefficients are equal to their bounds. $IER = 1$ The problem is rank deficient. $IER = -1$ $NDEG(i) < 0$ for some $i = 1, \dots, NDOM$.

IER=-2	$N < 2$.
IER=-3	NWORK too small; the number of words of real workspace needed to continue is specified by the printed error messages and returned in NEEDED.
IER=-4	NIWORK too small; the number of words of integer workspace needed to continue is specified by the printed error messages and returned in NEEDED.
IER=-5	$NDIM < N$, $MDIM < \max_{i=1,\dots,NDOM} NKNOTS(i)$, $IDIM < NDOM$, $LDIM < NDOM$, or $GDIM < \max_{j=1,\dots,NCON} G_j$. See the discussion of GDIM for the definition of G_j .
IER=-6	$NKNOTS(i) < 0$ for some $i = 1, \dots, NDOM$.
IER=-7	$\min_{\ell} \{X(\ell, i)\} = \max_{\ell} \{X(\ell, i)\}$ for some $i = 1, \dots, NDOM$.
IER=-8	Either the i -th column of T is not in ascending order or the multiplicity of a knot exceeds $NDEG(i)+1$ for some $i = 1, \dots, NDOM$.
IER=-9	$IFUN < 1$ or $IFUN > 4$.
IER=-10	$IREG < 0$ or $IREG > 2$.
IER=-11	$IDRV \neq 0$ and $IGAMMA(\ell, i) < 0$ or $IGAMMA(\ell, i) > NDEG(i)$ for some $\ell = 1, \dots, N$ and $i = 1, \dots, NDOM$.
IER=-12	$NDOM < 0$ or $NDOM > 10$.
IER=-13	$NCON < 0$.
IER=-14	$TYPE(j) \neq 1$ and $TYPE(j) \neq 0$ and $TYPE(j) \neq -1$ for some $j = 1, \dots, NCON$.
IER=-15	$MASK(j) \neq 1$ and $MASK(j) \neq 0$ and $MASK(j) \neq -1$ for some $j = 1, \dots, NCON$.
IER=-16	$LAMBDA(i, j) < 0$ or $LAMBDA(i, j) > NDEG(i)$ for some $i = 1, \dots, NDOM$ and $j = 1, \dots, NCON$.
IER=-17	Either $FIXVAL(i, j) < \min_{\ell} \{X(\ell, i)\}$ or $FIXVAL(i, j) > \max_{\ell} \{X(\ell, i)\}$ for some $i = 1, \dots, NDOM$ and $j = 1, \dots, NCON$.
IER=-18	The number of values of the j -th column of FIXVAL equal to $HDMCON(2)$ does not equal $GSPLN(1, j)$.

IER=-19	MCOUT too small; the value of MCOUT required is returned in NEEDED.
IER=-36	$T(1,i) \leq \min_{\ell}\{X(\ell,i)\}$ or $T(NKNOTS(i),i) \geq \max_{\ell}\{X(\ell,i)\}$ for some $i = 1, \dots, NDOM$.
IER=-40	Feasible spline does not exist, see discussion under USAGE REMARKS.
IER=-41	Inconsistent inequality constraints are defined. A lower bound is greater than an upper bound.
IER=-42	Inconsistent equality constraints are defined.
IER=-43	Redundant inequality constraints are defined.
IER=-50	The spline represented by the j -th column of GSPLN is defined for values of x_i which lie outside of the interval $[\min_{\ell} X(\ell,i), \max_{\ell} X(\ell,i)]$ for some $i = 1, \dots, NDOM$ and $j = 1, \dots, NCON$.
IER=-51	$GSPLN(1,j) < 1$ or $GSPLN(1,j) > NDOM$ for some $j = 1, \dots, NCON$.
IER=-52	$GSPLN(2,j) \neq 1$ for some $j = 1, \dots, NCON$.
IER=-53	$GSPLN(2+i,j) < 0$ for some $i = 1, \dots, GSPLN(1,j)$ and $j = 1, \dots, NCON$.
IER=-54	$GSPLN(2+GSPLN(1,j)+i,j) < GSPLN(2+i,j)$ for some $i = 1, \dots, GSPLN(1,j)$ and $j = 1, \dots, NCON$.
IER=-55	The spline represented by the j -th column of GSPLN has an invalid knot set for some $j = 1, \dots, NCON$.
IER=-60	Error opening the scratch file associated with unit 19. The user may have previously opened a file associated with unit 19.
IER=-61	Error opening the scratch file associated with unit 18. The user may have previously opened a file associated with unit 18.
IER=-62	Error writing to the scratch file associated with unit 19.
IER=-63	Error writing to the scratch file associated with unit 18.
IER=-64	Error reading from the scratch file associated with unit 19.
IER=-65	Error reading from the scratch file associated with unit 18.

IER=-66	Error closing the scratch file associated with unit 19.
IER=-67	Error closing the scratch file associated with unit 18.
IER=-100	Unexpected error return, see explanation of MODE = 5 error message from HHERR.
IER=-101	Unexpected error returned by a rank revealing QR routine during redundant equality constraint determination.
IER=-102	Unexpected error returned by a rank revealing QR routine during rank determination.
IER=-103	Unexpected error returned by a rank revealing QR routine during optimization initialization.
IER=-110	Unexpected process error during problem initialization.
IER=-120	Process error during optimization. This error is caused by numerical problems in the optimization software which should be brought to the attention of the software staff.
IER=-200	Unexpected error returned by SNLPMN or SBRNLP, see explanation of MODE = 5 error message from HHERR.
IER=-300	The user is using one of the input/output units required by the sparse optimization routine SNLPMN, see the discussion of IER = -137 error return in the SNLPMN or SBRNLP, documentation for additional information.

EXAMPLE

This example computes a least squares approximation of data sampled from the function $z = x^2y^3$. The data is defined on a 5 by 5 grid over the region $0.0 \leq x \leq 1.0$, $0.0 \leq y \leq 1.0$ such that $x^2 + y^2 = 1.0$. The spline approximation, $s(x, y)$, will be linear in x and quadratic in y . The internal knots are defined as $S_1 = S_2 = (0.25, 0.50, 0.75)^T$.

The interpolation condition

$$\frac{\partial^2 s(0.0, 0.0)}{\partial x \partial y} = 0.0 \quad (8.23)$$

is imposed. Also, the constraints

$$\begin{aligned} \frac{\partial s(x, y)}{\partial x} &\geq 0.0 \\ \frac{\partial s(x, y)}{\partial y} &\geq 0.0. \end{aligned}$$

are imposed along with the boundary conditions

$$s(0.0, y) = 0.0 \quad (8.24)$$

for $0.0 \leq y \leq 1.0$ and

$$s(x, 0.0) = 0.0 \quad (8.25)$$

for $0.0 \leq x \leq 1.0$.

SAMPLE PROGRAM (excmvs.cd1)⁵

```

PROGRAM EXAMPLE

C -----
C ... PROGRAM TO ILLUSTRATE HDCMVS EXAMPLE PROBLEM
C -----

C ... HDCMVS PARAMETERS

      INTEGER          N      , NDOM  , NDIM  , IFUN  , IREG  , IPAR  ,
+                    IDIM  , MDIM  , NCON  , LDIM  , GDIM  , NWORK,
+                    NIWORK, MCOUT , NEEDED, IER

      PARAMETER ( NDOM = 2, NDIM = 25, IDIM = 25, MDIM = 10 )
      PARAMETER ( LDIM = 2, GDIM = 13, NWORK = 100000, NIWORK = 100000 )
      PARAMETER ( MCOUT = 100 )

      INTEGER          IGAMMA(IDIM,NDOM), LAMBDA(LDIM,4), NDEG(NDOM) ,
+                    NKNOTS(NDOM)      , TYPE(4)          , MASK(4)      ,
+                    IWORK(NWORK)

      DOUBLE PRECISION C(MCOUT)          , FIXVAL(LDIM,4), GSPLN(GDIM,4),
+                    T(MDIM,NDOM)      , WTOL(NDIM)      , WORK(NWORK)  ,
+                    X(NDIM,NDOM)      , Y(NDIM)

C ... LOCAL VARIABLES

      INTEGER          I      , J

      DOUBLE PRECISION VAL      , XTMP(2)

C ... EXTERNALS

      DOUBLE PRECISION HDMCON

```

⁵sosexample.tar

```

C -----
C ... DEFINE THE DATA TO APPROXIMATE. IPAR = 1 SPECIFIES THE DATA
C REPRESENTS THE FUNCTION  $Z = X^{**2}Y^{**3}$ , AS WELL AS PARTIALS
C -----

      N      = 0
      IPAR = 1
      IFUN = 1
      IREG = 2

C ... DEFINE FUNCTION DATA

      DO I = 1, 5

        DO J = 1, 5

          XTMP(1) = DBLE ( I - 1 ) * 2.5D-1
          XTMP(2) = DBLE ( J - 1 ) * 2.5D-1

          IF ( XTMP(1)**2 + XTMP(2)**2 .LE. 1.0D0 ) THEN

            N          = N + 1
            X(N,1)     = XTMP(1)
            X(N,2)     = XTMP(2)
            Y(N)       = XTMP(1)**2*XTMP(2)**3
            IGAMMA(N,1) = 0
            IGAMMA(N,2) = 0
            WTOL(N)    = 1.0D0

          END IF

        END DO

      END DO

C -----
C ... ADD PARTIAL DERIVATIVE INTERPOLATION CONDITION
C -----

C ... MIXED PARTIAL AT (0.0,0.0)

      N          = N + 1
      X(N,1)     = 0.0D0
      X(N,2)     = 0.0D0
      Y(N)       = 0.0D0
      IGAMMA(N,1) = 1
      IGAMMA(N,2) = 1

```

```

        WTOL(N)      = HDMCON(2)

C      -----
C      ... SPECIFY THE DEGREE AND INTERNAL KNOTS
C      -----

        NDEG(1) = 1
        NDEG(2) = 2
        NKNOTS(1) = 2
        NKNOTS(2) = 2

        T(1,1) = 3.3D-1
        T(2,1) = 6.6D-1

        T(1,2) = 3.3D-1
        T(2,2) = 6.6D-1

C      -----
C      ... SPECIFY THE CONSTRAINTS
C      -----

        NCON = 4

C ... PARTIAL IN X IS POSITIVE CONSTRAINT

        TYPE(1) = -1
        MASK(1) = 0

        LAMBDA(1,1) = 1
        LAMBDA(2,1) = 0

        FIXVAL(1,1) = HDMCON(2)
        FIXVAL(2,1) = HDMCON(2)

        GSPLN(1,1) = 2.0D0
        GSPLN(2,1) = 1.0D0
        GSPLN(3,1) = 1.0D0
        GSPLN(4,1) = 1.0D0
        GSPLN(5,1) = 1.0D0
        GSPLN(6,1) = 1.0D0
        GSPLN(7,1) = 1.0D0
        GSPLN(8,1) = 1.0D0
        GSPLN(9,1) = 0.0D0
        GSPLN(10,1) = 1.0D0
        GSPLN(11,1) = 0.0D0
        GSPLN(12,1) = 1.0D0
        GSPLN(13,1) = 0.0D0

```

C ... PARTIAL IN Y IS POSITIVE CONSTRAINT

TYPE(2) = -1

MASK(2) = 0

LAMBDA(1,2) = 0

LAMBDA(2,2) = 1

FIXVAL(1,2) = HDMCON(2)

FIXVAL(2,2) = HDMCON(2)

GSPLN(1,2) = 2.0D0

GSPLN(2,2) = 1.0D0

GSPLN(3,2) = 1.0D0

GSPLN(4,2) = 1.0D0

GSPLN(5,2) = 1.0D0

GSPLN(6,2) = 1.0D0

GSPLN(7,2) = 1.0D0

GSPLN(8,2) = 1.0D0

GSPLN(9,2) = 0.0D0

GSPLN(10,2) = 1.0D0

GSPLN(11,2) = 0.0D0

GSPLN(12,2) = 1.0D0

GSPLN(13,2) = 0.0D0

C ... S(X,0.0) = 0.0 BOUNDARY CONDITION

TYPE(3) = 0

MASK(3) = 0

LAMBDA(1,3) = 0

LAMBDA(2,3) = 0

FIXVAL(1,3) = HDMCON(2)

FIXVAL(2,3) = 0.0D0

GSPLN(1,3) = 1.0D0

GSPLN(2,3) = 1.0D0

GSPLN(3,3) = 1.0D0

GSPLN(4,3) = 1.0D0

GSPLN(5,3) = 1.0D0

GSPLN(6,3) = 0.0D0

GSPLN(7,3) = 1.0D0

GSPLN(8,3) = 0.0D0

C ... S(0.0,Y) = 0.0 BOUNDARY CONDITION

TYPE(4) = 0

```

MASK(4) = 0

LAMBDA(1,4) = 0
LAMBDA(2,4) = 0

FIXVAL(1,4) = 0.0D0
FIXVAL(2,4) = HDMCON(2)

GSPLN(1,4) = 1.0D0
GSPLN(2,4) = 1.0D0
GSPLN(3,4) = 1.0D0
GSPLN(4,4) = 1.0D0
GSPLN(5,4) = 1.0D0
GSPLN(6,4) = 0.0D0
GSPLN(7,4) = 1.0D0
GSPLN(8,4) = 0.0D0

C -----
C ... CALL HDCMVS
C -----

CALL HDCMVS ( N      , X      , NDIM  , NDOM , Y      , IFUN  , WTOL  ,
+           IPAR  , IGAMMA, IDIM  , IREG  , NDEG  , NKNOTS, T      ,
+           MDIM  , NCON  , LAMBDA, TYPE  , MASK  , GSPLN , GDIM  ,
+           FIXVAL, LDIM  , WORK   , NWORK, IWORK, NIWORK, MCOUT,
+           NEEDED, C      , IER  )

IF ( IER .NE. 0 ) THEN

    WRITE ( *, 1000 ) IER
    IF ( IER .LT. 0 ) STOP

END IF

C -----
C ... EVALUATE SPLINE APPROXIMATION
C -----

N = 0
WRITE ( *, 1200 )

DO I = 1, 5

    DO J = 1, 5

        XTMP(1) = DBLE ( I - 1 ) * 2.5D-1
        XTMP(2) = DBLE ( J - 1 ) * 2.5D-1
    
```



```

CALL HDNPVL ( XTMP, 1, C, WORK, NWORK, VAL, IER )

IF ( IER .LT. 0 ) THEN

    WRITE ( *, 1100 ) IER
    STOP

END IF

IF ( XTMP(1)**2 + XTMP(2)**2 .LE. 1.0D0 ) THEN

    N = N + 1
    WRITE ( *, 1300 ) XTMP(1), XTMP(2), VAL, Y(N),
+               ABS ( Y(N) - VAL )

ELSE

    WRITE ( *, 1300 ) XTMP(1), XTMP(2), VAL

END IF

END DO

END DO

WRITE ( *, 1400 )

C -----

1000 FORMAT ( ' ERROR RETURN FROM HDCMVS - IER = ', I5 )
1100 FORMAT ( ' ERROR RETURN FROM HDNPVL - IER = ', I5 )
1200 FORMAT(/1X,'   X       Y       S(X,Y)   Z(X,Y)   ABS DIFF.',
+         /1X,'=====')
1300 FORMAT(2(1X,F6.3),3(1X,F9.6))
1400 FORMAT ( / '* NORMAL TERMINATION *' )
      STOP
      END

```

OUTPUT FROM SAMPLE PROGRAM

ERROR RETURN FROM HDCMVS - IER = 1

X	Y	S(X,Y)	Z(X,Y)	ABS DIFF.
0.000	0.000	0.000000	0.000000	0.000000
0.000	0.250	0.000000	0.000000	0.000000

0.000	0.500	0.000000	0.000000	0.000000
0.000	0.750	0.000000	0.000000	0.000000
0.000	1.000	0.000000	0.000000	0.000000
0.250	0.000	0.000000	0.000000	0.000000
0.250	0.250	0.001211	0.000977	0.000234
0.250	0.500	0.007717	0.007812	0.000096
0.250	0.750	0.026384	0.026367	0.000017
0.250	1.000	0.034765		
0.500	0.000	0.000000	0.000000	0.000000
0.500	0.250	0.004844	0.003906	0.000938
0.500	0.500	0.030867	0.031250	0.000383
0.500	0.750	0.105584	0.105469	0.000115
0.500	1.000	0.139716		
0.750	0.000	0.000000	0.000000	0.000000
0.750	0.250	0.008789	0.008789	0.000000
0.750	0.500	0.055828	0.070312	0.014484
0.750	0.750	0.192795		
0.750	1.000	0.283537		
1.000	0.000	0.000000	0.000000	0.000000
1.000	0.250	0.011262		
1.000	0.500	0.071099		
1.000	0.750	0.250064		
1.000	1.000	0.437744		

* NORMAL TERMINATION *

HDNPVL: Evaluation of a Tensor Product Spline

PURPOSE

HDNPVL computes values of a tensor product spline function with one or more independent variables and any number of dependent variables. If derivatives are required use HDNPDR. HDNPVL can also be used to calculate the values of a rational tensor product spline.

RELATED SUBPROGRAMS

HDENVL	Fast Spline evaluation without error checking
HDENVL	Spline Evaluation with extrapolation
HDENVL	Fast Spline evaluation with extrapolation, but without error checking
HDNPDR	Evaluation of a Partial Derivative of a Tensor Product Spline

METHOD

Usually, the function evaluated by HDNPVL is a tensor product spline, represented in a B-spline basis, defined by one of the spline construction subroutines of this chapter. The data defining the tensor product B-spline representation is stored in a standard data structure called the C array. HDNPVL computes the values of the dependent variables by evaluating the tensor product B-splines at the value of the independent variables. If there is only one independent variable, the tensor product B-spline is a univariate spline and it is evaluated in the same manner as HDSPVL evaluates a univariate spline. If there is more than one independent variable, the tensor product spline is evaluated by computing the value of a sequence of univariate splines. Each univariate spline is evaluated as the inner product of its B-spline coefficients with the values of the non-vanishing B-splines, obtained by the algorithm of Cox and de Boor.

HDNPVL may also be used to evaluate a rational tensor product spline, represented in a rational tensor product B-spline basis. Often, tensor product spline functions which must be represented by rational B-splines are defined by subroutines that convert CAD/CAM data, such as CATIA data, to the spline representation. In this case, the C array stores data defining the tensor product spline for the numerator for each dependent variable and for the common denominator—see Appendix A. HDNPVL first computes the values of the numerator and denominator tensor product B-splines as in the tensor product spline case above. Then each dependent variable is evaluated as the ratio of the values of its numerator and the common denominator.

USAGE

```
INTEGER INCX,NWORK,IER
DOUBLE PRECISION X(1+(NDOM-1)*INCX),C(MC),WORK(NWORK),V(NRNG)
CALL HDNPVL(X,INCX,C,WORK,NWORK,V,IER)
```

where

NDOM is the number of independent variables for the tensor product spline represented by the C array. For example, if HDMVGI were used to construct the C array, NDOM would have the same value as it had in the call to HDMVGI. $\text{NDOM} = \text{C}(1)$.

MC is the length of the C array as determined when C was created. If necessary, MC may be obtained by a call to HD1SVY, or MC may also be computed directly from the data stored in the C array:

$$\text{MC} = 2 + 3 * \text{C}(1) + \sum_{i=1}^{\text{C}(1)} [\text{C}(2+i) + \text{C}(2+\text{C}(1)+i)] + |\text{C}(2)| * \prod_{i=1}^{\text{C}(1)} \text{C}(2+\text{C}(1)+i).$$

NRNG is the number of dependent variables for the tensor product spline represented by the C array. For example, if HDMVGI were used to construct the C array, NRNG would have the same value as the argument NDEP in the call to HDMVGI. If $\text{C}(2) > 0$, then the C array represents a tensor product B-spline and $\text{NRNG} = \text{C}(2)$. If $\text{C}(2) < -1$, then the C array represents a rational tensor product B-spline and $\text{NRNG} = -(\text{C}(2)+1)$.

Note: The formula to calculate the required dimension NWORK of array WORK is complicated. It is very important that NWORK is at least of the size specified. If not, HDNPVL will write into areas that it should not—causing unpredictable side effects.

ARGUMENTS

X [INPUT, DOUBLE PRECISION , ARRAY]
 Array of length $1 + (\text{NDOM} - 1) * \text{INCX}$ containing a set of values of the independent variables at which the values of the dependent variables are desired. The values of the independent variables must be in the interval of definition of their corresponding splines.

INCX [INPUT, INTEGER]
 Increment parameter for array X. The i^{th} independent variable is stored in $\text{X}(1 + (i - 1) * \text{INCX})$. See USAGE REMARKS for an example of the usage of INCX.

C [INPUT, DOUBLE PRECISION , ARRAY]
 Spline definition array. See USAGE for dimension information. Usually, C will have been created by one of the spline construction subprograms of this chapter.

NWORK [INPUT, INTEGER]
 Length of array WORK. It depends on NRNG and NDOM—see USAGE.
 If $\text{NDOM} = 1$ and the C array represents a spline, then

$$\text{NWORK} \geq 5 * K_1 - 2.$$

If $\text{NDOM} \geq 2$ and the C array represents a tensor product spline, then

$$\text{NWORK} \geq \text{NZERO} * (\text{NRNG} + 1) + 3 * \text{KMAX} + \text{NDOM}.$$

If $\text{NDOM} = 1$ and the C array represents a rational spline, then

$$\text{NWORK} \geq 5 * \text{K}_1 + \text{NRNG} - 1.$$

If $\text{NDOM} \geq 2$ and the C array represents a rational tensor product spline, then

$$\text{NWORK} \geq \text{NZERO} * (\text{NRNG} + 2) + 3 * \text{KMAX} + \text{NDOM} + \text{NRNG} + 1,$$

where

$$\text{KMAX} = \max_{i=1, \text{NDOM}} \text{K}_i \quad \text{and} \quad \text{NZERO} = \prod_{i=2}^{\text{NDOM}} \text{K}_i$$

and $\text{K}_i = \text{C}(2 + i)$ is the order of the spline for the i^{th} independent variable.

V

[OUTPUT, DOUBLE PRECISION , ARRAY]

Array of length NRNG containing the values of the dependent variables at X. See USAGE for a description of NRNG.

IER

[OUTPUT, INTEGER]

Success/error code. For $\text{IER} < 0$, results have not been computed; HDNPVL has set $\text{V}(1) = \text{HDMCON}(1)$.

$\text{IER} = 0$ Success, results computed.

$\text{IER} = -1$ $\text{K}_i < 1$ for some i .

$\text{IER} = -3$ NWORK is too small; the number of words needed is given by the printed error message.

IER = -6	Number of spline coefficients with respect to the i^{th} independent variable is less than K_i for some i .
IER = -8	Invalid knot set.
IER = -10	The absolute value of the denominator of the spline (when evaluated at X) is less than HDMCON(6), the relative precision.
IER = -38	X is inside a knot interval that is too small. This error is extremely unlikely.
IER = -50	One (or more) of the independent variables is out of range.
IER = -51	C(1) < 1.
IER = -52	C(2) = -1 or 0.

USAGE REMARKS

When the input C array is output from a spline construction subroutine, the user is responsible only for setting X, INCX, and NWORK. If the construction subroutine was called correctly, errors other than IER=-3 and IER=-50 should not be returned by HDNPVL.

The following code fragment evaluates a tensor product spline at points x_i , $i = 1, 2, \dots, M$ where each x_i is entered in X as a row vector, $x_i = (x_{i,1}, x_{i,2}, \dots, x_{i,\text{NDOM}})$.

```
DOUBLE PRECISION  X(M,NDOM),C(MC),WORK(NWORK),V(NRNG)
INCX = M
DO 10 I=1,M
    CALL HDNPVL(X(I,1),INCX,C,WORK,NWORK,V,IER)
10 CONTINUE
```

However, if the x_i are stored as columns in array X, then one should write:

```
DOUBLE PRECISION  X(NDOM,M),C(MC),WORK(NWORK),V(NRNG)
INCX = 1
DO 10 J=1,M
    CALL HDNPVL(X(1,J),INCX,C,WORK,NWORK,V,IER)
10 CONTINUE
```

EXAMPLE

Subroutine HDMVGI is used in the following example to fit a bicubic spline to data on a 4 by 4 grid. Subroutine HDNPVL is then used to evaluate the spline at the grid points. A comparison of the spline fit with the original data points is displayed.

SAMPLE PROGRAM (hdnpvl.cd1)⁶

```

PROGRAM SAMPLE

  INTEGER      IOPT, MCDIM, NDEG, NDEP, NDIM, NDOM, NGRID, NWORK,
1             INCX

  PARAMETER ( IOPT = 1, NDEG = 3, NDEP = 1, NGRID = 4,
1             NDIM = 4, NDOM = 2, INCX = 1,
2             MCDIM = 2 + NDOM*(2*NDEG) + 2*NDEG
3             + NDEP*(NGRID+NDEG-1)**2,
4             NWORK = (NGRID+NDEG-1)*(3*NDEG+1)
5             + 3*(NDEG+1)**2 + 8*NDEG + 4*NGRID + 14 )

  INTEGER      I , IER, J
  INTEGER      NPT(NDOM), NYDIM(NDOM)
  DOUBLE PRECISION X(NDIM,NDOM), Y(NGRID,NGRID,1), WORK(NWORK),
1             C(MCDIM), XX(2), V(1)

  DATA      X / 0.000,   1.000,   2.000,   3.000,
1            0.000,   1.000,   2.000,   3.000 /

  DATA      Y / 0.000,   1.000,   0.000,  -1.000,
1            0.000,   3.718,   2.000,   0.282,
2            0.000,   9.389,   4.000,  -1.389,
3            0.000,  23.086,   6.000, -11.086 /

C      ... Fit cubic spline to data on a 4 by 4 grid of points.

  NPT   (1) = NGRID
  NPT   (2) = NGRID
  NYDIM (1) = NGRID
  NYDIM (2) = NGRID

  CALL HDMVGI ( X, NDIM, NDOM, NPT, Y, NYDIM, NDEP,
1             NDEG, IOPT, WORK, NWORK, C, IER )

C      ... Check for error return

  IF ( IER .NE. 0 ) THEN
    WRITE ( *, 9000 ) IER
    STOP
  END IF

C      ... Evaluate spline at original data points and compare with
C      spline fit

```

⁶sosexample.tar

```
      WRITE ( *, 9010 )

      DO J = 1, NGRID
        XX(2) = X(J,2)

        DO I = 1, NGRID
          XX(1) = X(I,1)

          CALL HDNPVL ( XX, INCX, C, WORK, NWORK, V, IER )

C          ... Check for error return

          IF ( IER .NE. 0 ) THEN
            WRITE ( *, 9020 ) IER
            STOP
          END IF

          WRITE ( *, 9030 ) XX(1), XX(2), Y(I,J,1), V(1),
1          ABS(Y(I,J,1)-V(1))
        ENDDO

      ENDDO

      WRITE ( *, 9040 )

      STOP
9000 FORMAT ( ' Error return from HDMVGI - IER = ', I5,
1          /, '* ERROR *' )
9010 FORMAT ( /10X, 'X1', 10X, 'X2', 6X, 'True Y', 5X,
1          'Comp. Y', 4X, 'Abs. Diff.' / )
9020 FORMAT ( ' Error return from HDNPVL - IER = ', I5,
1          /, '* ERROR *' )
9030 FORMAT ( 5F12.3 )
9040 FORMAT ( / '* NORMAL TERMINATION *' )
      END
```

OUTPUT FROM SAMPLE PROGRAM

X1	X2	True Y	Comp. Y	Abs. Diff.
0.000	0.000	0.000	0.000	0.000
1.000	0.000	1.000	1.000	0.000
2.000	0.000	0.000	0.000	0.000
3.000	0.000	-1.000	-1.000	0.000
0.000	1.000	0.000	0.000	0.000
1.000	1.000	3.718	3.718	0.000

2.000	1.000	2.000	2.000	0.000
3.000	1.000	0.282	0.282	0.000
0.000	2.000	0.000	0.000	0.000
1.000	2.000	9.389	9.389	0.000
2.000	2.000	4.000	4.000	0.000
3.000	2.000	-1.389	-1.389	0.000
0.000	3.000	0.000	0.000	0.000
1.000	3.000	23.086	23.086	0.000
2.000	3.000	6.000	6.000	0.000
3.000	3.000	-11.086	-11.086	0.000

* NORMAL TERMINATION *

HDENVL: Spline Evaluation with Extrapolation

PURPOSE

HDENVL computes values of a tensor product spline function with one or more independent variables and any number of dependent variables. This routine also uses linear extrapolation if the user tries to evaluate the tensor product spline outside of the range of the independent variables. HDENVL can also be used to calculate the values of a rational tensor product spline.

RELATED SUBPROGRAMS

HDNPVL	Evaluation of a Tensor Product Spline
HDENVL	Fast Spline Evaluation without Error Checking
HDFEVL	Fast Spline Evaluation with Extrapolation, but without Error Checking
HDNPDR	Evaluation of a Partial Derivative of a Tensor Product Spline

METHOD

This function is an extension of HDNPVL. If a point at which the tensor product spline needs to be evaluated lies within the range of the spline, this routine directly calls HDNPVL. Otherwise, a linear extrapolation is used. This is done by finding the appropriate point on the boundary of the range of the spline, and calculating the value and first derivative of the spline at that point. This information is then used to calculate a linear extrapolation.

USAGE

```
INTEGER INCX,NWORK,IER
DOUBLE PRECISION X(1+(NDOM-1)*INCX),C(MC),WORK(NWORK),V(NRNG)
CALL HDENVL(X,INCX,C,WORK,NWORK,V,IER)
```

where

NDOM is the number of independent variables for the tensor product spline represented by the C array. For example, if HDMVGI were used to construct the C array, NDOM would have the same value as it had in the call to HDMVGI. $NDOM = C(1)$. Extrapolation for splines with more than 6 independent variables is not permitted.

MC is the length of the C array as determined when C was created. If necessary, MC may be obtained by a call to HD1SVY or MC may also be computed directly from the data stored in the C array:

$$MC = 2 + 3 * C(1) + \sum_{i=1}^{C(1)} [C(2+i) + C(2+C(1)+i)] + |C(2)| * \prod_{i=1}^{C(1)} C(2+C(1)+i).$$

NRNG is the number of dependent variables for the tensor product spline represented by the C array. For example, if HDMVGI were used to construct the C array, NRNG would have the same value as the argument NDEP in the call to HDMVGI. If $C(2) > 0$, then the C array represents a tensor product B-spline and $NRNG = C(2)$. If $C(2) < -1$, then the C array represents a rational tensor product B-spline and $NRNG = -(C(2)+1)$. Extrapolation for splines with more than 20 dependent variables is not permitted.

Note: The formula to calculate the required dimension NWORK of array WORK is complicated. It is very important that NWORK is at least of the size specified. If not, HDENVL will write into areas that it should not—causing unpredictable side effects.

ARGUMENTS

X [INPUT, DOUBLE PRECISION , ARRAY]
Array of length $1+(NDOM-1)*INCX$ containing a set of values of the independent variables at which the values of the dependent variables are desired.

INCX [INPUT, INTEGER]
Increment parameter for array X. The i^{th} independent variable is stored in $X(1+(i-1)*INCX)$. See USAGE REMARKS for an example of the usage of INCX.

C [INPUT, DOUBLE PRECISION , ARRAY]
Spline definition array. See USAGE for dimension information. Usually, C will have been created by one of the spline construction subprograms of this chapter.

NWORK [INPUT, INTEGER]
 Length of array WORK. It depends on NRNG and NDOM—see USAGE.
 If NDOM = 1 and the C array represents a spline, then

$$\text{NWORK} \geq 5 * K_1 - 2.$$

If NDOM ≥ 2 and the C array represents a tensor product spline, then

$$\text{NWORK} \geq \text{NZERO} * (\text{NRNG} + 1) + 3 * \text{KMAX} + \text{NDOM}.$$

If NDOM = 1 and the C array represents a rational spline, then

$$\text{NWORK} \geq 5 * K_1 + \text{NRNG} - 1.$$

If NDOM ≥ 2 and the C array represents a rational tensor product spline, then

$$\text{NWORK} \geq \text{NZERO} * (\text{NRNG} + 2) + 3 * \text{KMAX} + \text{NDOM} + \text{NRNG} + 1,$$

where

$$\text{KMAX} = \max_{i=1, \text{NDOM}} K_i \quad \text{and} \quad \text{NZERO} = \prod_{i=2}^{\text{NDOM}} K_i$$

and $K_i = C(2 + i)$ is the order of the spline for the i^{th} independent variable.

V [OUTPUT, DOUBLE PRECISION, ARRAY]
 Array of length NRNG containing the values of the dependent variables at X. See USAGE for a description of NRNG.

IER [OUTPUT, INTEGER]
 Success/error code. For IER < 0, results have not been computed; HDENVL has set V(1) = HDMCON(1).

IER = 0 Success, results computed.

IER > 0 Success, results computed. Extrapolation was done, and the largest percentage by which one of the independent variables exceeded the range of the spline is larger than 10 %. Let DELTAX(i) be the amount by which variable i exceeds its range, and CL(i) and CU(i) the upper and lower bounds for the range of variable i . Then

$$\text{IER} = \max_{i=1, \dots, \text{NDOM}} \text{INT} \left(10 \frac{|\text{DELTAX}(i)|}{\text{CU}(i) - \text{CL}(i)} \right).$$

IER = -1 $K_i < 1$ for some i .

IER = -3	NWORK is too small; the number of words needed is given by the printed error message.
IER = -6	Number of spline coefficients with respect to the i^{th} independent variable is less than K_i for some i .
IER = -8	Invalid knot set.
IER = -10	The absolute value of the denominator of the spline (when evaluated at X) is less than HDMCON(6), the relative precision.
IER = -38	X is inside a knot interval that is too small. This error is extremely unlikely.
IER = -51	C(1) < 1.
IER = -52	C(2) = -1 or 0.
IER = -100	NDOM > 6. Extrapolation for splines with more than 6 independent variables is not permitted. Contact SOS support if this error occurs.
IER = -101	NRNG > 20. Extrapolation for splines with more than 20 dependent variables is not permitted. Contact SOS support if this error occurs.
IER = -200	An error occurred while trying to extrapolate. One reason might be insufficient workspace. Increase NWORK.

USAGE REMARKS

When the input C array is output from a spline construction subroutine, the user is responsible only for setting X, INCX, and NWORK. If the construction subroutine was called correctly, errors other than IER=-3, IER=-100, IER=-101, or IER=-200 should not be returned by HDENVL.

The following code fragment evaluates a tensor product spline at points x_i , $i = 1, 2, \dots, M$ where each x_i is entered in X as a row vector, $x_i = (x_{i,1}, x_{i,2}, \dots, x_{i,\text{NDOM}})$.

```
DOUBLE PRECISION  X(M,NDOM),C(MC),WORK(NWORK),V(NRNG)
INCX = M
DO 10 I=1,M
    CALL HDENVL(X(I,1),INCX,C,WORK,NWORK,V,IER)
10 CONTINUE
```

However, if the x_i are stored as columns in array X, then one should write:

```
DOUBLE PRECISION  X(NDOM,M),C(MC),WORK(NWORK),V(NRNG)
```

```
      INCX = 1
      DO 10 J=1,M
        CALL HDENVL(X(1,J),INCX,C,WORK,NWORK,V,IER)
10    CONTINUE
```

EXAMPLE

Subroutine HDMVGI is used in the following example to fit a bicubic spline to data on a 4 by 4 grid. Subroutine HDENVL is then used to evaluate the spline at the grid points. A comparison of the spline fit with the original data points is displayed. We also evaluate the spline at some points outside of the range of the data to test the extrapolation.

SAMPLE PROGRAM (hdenvl.cd1)⁷

```
      PROGRAM SAMPLE

      INTEGER      IOPT, MCDIM, NDEG, NDEP, NDM, NDOM, NGRID, NWORK,
1      INCX

      PARAMETER ( IOPT = 1, NDEG = 3, NDEP = 1, NGRID = 4,
1      NDM = 4, NDOM = 2, INCX = 1,
2      MCDIM = 2 + NDOM*(2*NDEG) + 2*NDEG
3      + NDEP*(NGRID+NDEG-1)**2,
4      NWORK = (NGRID+NDEG-1)*(3*NDEG+1)
5      + 3*(NDEG+1)**2 + 8*NDEG + 4*NGRID + 14 )

      INTEGER      I , IER, J
      INTEGER      NPT(NDOM), NYDIM(NDOM)
      DOUBLE PRECISION X(NDIM,NDOM), Y(NGRID,NGRID,1), WORK(NWORK),
1      C(MCDIM), XX(2), V(1)

      DATA      X / 0.000, 1.000, 2.000, 3.000,
1      0.000, 1.000, 2.000, 3.000 /

      DATA      Y / 0.000, 1.000, 0.000, -1.000,
1      0.000, 3.718, 2.000, 0.282,
2      0.000, 9.389, 4.000, -1.389,
3      0.000, 23.086, 6.000, -11.086 /

C      ... Fit cubic spline to data on a 4 by 4 grid of points.

      NPT (1) = NGRID
      NPT (2) = NGRID
      NYDIM (1) = NGRID
      NYDIM (2) = NGRID
```

⁷sosexample.tar

```

        CALL HDMVGI ( X, NDIM, NDOM, NPT, Y, NYDIM, NDEP,
1           NDEG, IOPT, WORK, NWORK, C, IER )

C      ... Check for error return

        IF ( IER .NE. 0 ) THEN
            WRITE ( *, 9000 ) IER
            STOP
        END IF

C      ... Evaluate spline at original data points and compare with
C      spline fit

        WRITE ( *, 9010 )

        DO J = 1, NGRID
            XX(2) = X(J,2)

            DO I = 1, NGRID
                XX(1) = X(I,1)

                CALL HDENVL ( XX, INCX, C, WORK, NWORK, V, IER )

C              ... Check for error return

                IF ( IER .NE. 0 ) THEN
                    WRITE ( *, 9020 ) IER
                    STOP
                END IF

                WRITE ( *, 9030 ) XX(1), XX(2), Y(I,J,1), V(1),
1              ABS(Y(I,J,1)-V(1))
            ENDDO

        ENDDO

C      ... Evaluate spline at data points outside of the region
C      defined by the data points to check extrapolation...

        WRITE ( *, 10010 )

        DO J = NGRID, NGRID
            XX(2) = X(J,2)+0.1D0*X(J,2)

            DO I = 1, NGRID
                XX(1) = X(I,1)

                CALL HDENVL ( XX, INCX, C, WORK, NWORK, V, IER )

```

```
C          ... Check for error return

          IF ( IER .NE. 0 ) THEN
              WRITE ( *, 9020 ) IER
              STOP
          END IF

          WRITE ( *, 10030 ) XX(1), XX(2), V(1)
      ENDDO

  ENDDO

  WRITE ( *, 9040 )

  STOP
9000 FORMAT ( ' Error return from HDMVGI - IER = ', I5,
1          /, '* ERROR *' )
9010 FORMAT ( /10X, 'X1', 10X, 'X2', 6X, 'True Y', 5X,
1          'Comp. Y', 4X, 'Abs. Diff.' / )
9020 FORMAT ( ' Error return from HDENVL - IER = ', I5,
1          /, '* ERROR *' )
9030 FORMAT ( 5F12.3 )
9040 FORMAT ( / '* NORMAL TERMINATION *' )
10010 FORMAT ( /10X, 'X1', 10X, 'X2', 6X, 'Comp. Y' / )
10030 FORMAT ( 3F12.3 )
  END
```

OUTPUT FROM SAMPLE PROGRAM

X1	X2	True Y	Comp. Y	Abs. Diff.
0.000	0.000	0.000	0.000	0.000
1.000	0.000	1.000	1.000	0.000
2.000	0.000	0.000	0.000	0.000
3.000	0.000	-1.000	-1.000	0.000
0.000	1.000	0.000	0.000	0.000
1.000	1.000	3.718	3.718	0.000
2.000	1.000	2.000	2.000	0.000
3.000	1.000	0.282	0.282	0.000
0.000	2.000	0.000	0.000	0.000
1.000	2.000	9.389	9.389	0.000
2.000	2.000	4.000	4.000	0.000
3.000	2.000	-1.389	-1.389	0.000
0.000	3.000	0.000	0.000	0.000
1.000	3.000	23.086	23.086	0.000
2.000	3.000	6.000	6.000	0.000

3.000	3.000	-11.086	-11.086	0.000
-------	-------	---------	---------	-------

X1	X2	Comp. Y
----	----	---------

0.000	3.300	0.000
-------	-------	-------

1.000	3.300	29.496
-------	-------	--------

2.000	3.300	6.600
-------	-------	-------

3.000	3.300	-16.296
-------	-------	---------

* NORMAL TERMINATION *

HDFNVL: Fast Spline Evaluation without Error Checking

PURPOSE

HDFNVL computes values of a tensor product spline function with one or more independent variables and any number of dependent variables. This method only performs limited error checking. It should only be used if a user is certain that the splines he passes are valid, and that the workspace is large enough. The user is advised to use the corresponding routine HDNPVL during product development. HDNPVL is slower than HDFNVL, but performs error checking.

Furthermore, if the number of independent variables is larger than 3, this routine calls HDNPVL internally. In this case the user is again advised to directly call HDNPVL. Note that both routines have the same calling sequence. If derivatives are required use HDNPDR. HDFNVL can also be used to calculate the values of a rational tensor product spline.

RELATED SUBPROGRAMS

HDNPVL	Evaluation of a Tensor Product Spline
HDENVL	Spline Evaluation with Extrapolation
HDFEVL	Fast Spline Evaluation with Extrapolation, but without Error Checking
HDNPDR	Evaluation of a Partial Derivative of a Tensor Product Spline

METHOD

HDFNVL is very similar to HDNPVL, except that it does not perform many error checks, and it uses a faster method if the number of independent variables is between one and three. If the number of independent variables is larger, this method calls HDNPVL. For more details on the method see the documentation to HDNPVL.

USAGE

```
INTEGER INCX,NWORK,IER
DOUBLE PRECISION X(1+(NDOM-1)*INCX),C(MC),WORK(NWORK),V(NRNG)
CALL HDFNVL(X,INCX,C,WORK,NWORK,V,IER)
```

where

NDOM is the number of independent variables for the tensor product spline represented by the C array. For example, if HDMVGI were used to construct the C array, NDOM would have the same value as it had in the call to HDMVGI. $NDOM = C(1)$.

MC is the length of the C array as determined when C was created. If necessary, MC may be obtained by a call to HD1SVY or MC may also be computed directly from the data stored in the C array:

$$MC = 2 + 3 * C(1) + \sum_{i=1}^{C(1)} [C(2+i)+C(2+C(1)+i)] + |C(2)| * \prod_{i=1}^{C(1)} C(2+C(1)+i).$$

NRNG is the number of dependent variables for the tensor product spline represented by the C array. For example, if HDMVGI were used to construct the C array, NRNG would have the same value as the argument NDEP in the call to HDMVGI. If $C(2) > 0$, then the C array represents a tensor product B-spline and $NRNG = C(2)$. If $C(2) < -1$, then the C array represents a rational tensor product B-spline and $NRNG = -(C(2)+1)$.

Note: The formula to calculate the required dimension NWORK of array WORK is complicated. It is very important that NWORK is at least of the size specified. If not, HDFNVL will write into areas that it should not—causing unpredictable side effects.

ARGUMENTS

- X** [INPUT, DOUBLE PRECISION , ARRAY]
 Array of length $1+(NDOM-1)*INCX$ containing a set of values of the independent variables at which the values of the dependent variables are desired. The values of the independent variables must be in the interval of definition of their corresponding splines.
- INCX** [INPUT, INTEGER]
 Increment parameter for array X. The i^{th} independent variable is stored in $X(1+(i-1)*INCX)$. See USAGE REMARKS for an example of the usage of INCX.
- C** [INPUT, DOUBLE PRECISION , ARRAY]
 Spline definition array. See USAGE for dimension information. Usually, C will have been created by one of the spline construction subprograms like HDGDWH.
- NWORK** [INPUT, INTEGER]
 Length of array WORK. It depends on NRNG and NDOM—see USAGE.
 If $NDOM = 1$ and the C array represents a spline, then

$$NWORK \geq 5 * K_1 - 2.$$

If $NDOM \geq 2$ and the C array represents a tensor product spline, then

$$NWORK \geq NZERO * (NRNG + 1) + 3 * KMAX + NDOM.$$

If $NDOM = 1$ and the C array represents a rational spline, then

$$NWORK \geq 5 * K_1 + NRNG - 1.$$

If $NDOM \geq 2$ and the C array represents a rational tensor product spline, then

$$\text{NWORK} \geq \text{NZERO} * (\text{NRNG} + 2) + 3 * \text{KMAX} + \text{NDOM} + \text{NRNG} + 1,$$

where

$$\text{KMAX} = \max_{i=1, \text{NDOM}} K_i \quad \text{and} \quad \text{NZERO} = \prod_{i=2}^{\text{NDOM}} K_i$$

and $K_i = C(2 + i)$ is the order of the spline for the i^{th} independent variable.

V

[OUTPUT, DOUBLE PRECISION , ARRAY]

Array of length NRNG containing the values of the dependent variables at X. See USAGE for a description of NRNG.

IER

[OUTPUT, INTEGER]

Success/error code. For $\text{IER} < 0$, results have not been computed; HDFNVL has set $\text{V}(1) = \text{HDMCON}(1)$. *Note that no errors except $\text{IER} = -50$ will be detected by this routine if the number of independent variables is less than 4.* The user should only use this method if he is sure that these errors cannot happen. Otherwise, he should use HDNPVL instead.

$\text{IER} = 0$ Success, results computed.

$\text{IER} = -1$ $K_i < 1$ for some i .

$\text{IER} = -3$ NWORK is too small; the number of words needed is given by the printed error message.

$\text{IER} = -6$ Number of spline coefficients with respect to the i^{th} independent variable is less than K_i for some i .

$\text{IER} = -8$ Invalid knot set.

$\text{IER} = -10$ The absolute value of the denominator of the spline (when evaluated at X) is less than $\text{HDMCON}(6)$, the relative precision.

$\text{IER} = -38$ X is inside a knot interval that is too small. This error is extremely unlikely.

$\text{IER} = -50$ One (or more) of the independent variables is out of range.

$\text{IER} = -51$ $C(1) < 1$.

$\text{IER} = -52$ $C(2) = -1$ or 0 .

USAGE REMARKS

When the input C array is output from a spline construction subroutine, the user is responsible only for setting X, INCX, and NWORK. If the construction subroutine was called correctly, errors other than IER=−3 and IER=−50 should not be returned by HDFNVL. If the number of independent variables is between 1 and 3, the error IER=−3 will not be detected, and unpredictable behavior of the program could result. The user is responsible to ensure that enough workspace is available.

The following code fragment evaluates a tensor product spline at points x_i , $i = 1, 2, \dots, M$ where each x_i is entered in X as a row vector, $x_i = (x_{i,1}, x_{i,2}, \dots, x_{i,\text{NDOM}})$.

```
DOUBLE PRECISION  X(M,NDOM),C(MC),WORK(NWORK),V(NRNG)
INCX = M
DO 10 I=1,M
    CALL HDFNVL(X(I,1),INCX,C,WORK,NWORK,V,IER)
10 CONTINUE
```

However, if the x_i are stored as columns in array X, then one should write:

```
DOUBLE PRECISION  X(NDOM,M),C(MC),WORK(NWORK),V(NRNG)
INCX = 1
DO 10 J=1,M
    CALL HDFNVL(X(1,J),INCX,C,WORK,NWORK,V,IER)
10 CONTINUE
```

EXAMPLE

Subroutine HDMVGI is used in the following example to fit a bicubic spline to data on a 4 by 4 grid. Subroutine HDFNVL is then used to evaluate the spline at the grid points. A comparison of the spline fit with the original data points is displayed.

SAMPLE PROGRAM (hdfnvl.cd1)⁸

```
PROGRAM SAMPLE

INTEGER          IOPT, MCDIM, NDEG, NDEP, NDIM, NDOM, NGRID, NWORK,
1               INCX

PARAMETER ( IOPT = 1, NDEG = 3, NDEP = 1, NGRID = 4,
1          NDIM = 4, NDOM = 2, INCX = 1,
2          MCDIM = 2 + NDOM*(2*NDEG) + 2*NDEG
3          + NDEP*(NGRID+NDEG-1)**2,
4          NWORK = (NGRID+NDEG-1)*(3*NDEG+1)
5          + 3*(NDEG+1)**2 + 8*NDEG + 4*NGRID + 14 )

INTEGER          I , IER, J
INTEGER          NPT(NDOM), NYDIM(NDOM)
DOUBLE PRECISION X(NDIM,NDOM), Y(NGRID,NGRID,1), WORK(NWORK),
```

⁸sosexample.tar

```

1          C(MCDIM), XX(2), V(1)

DATA      X / 0.000,   1.000,   2.000,   3.000,
1          0.000,   1.000,   2.000,   3.000 /

DATA      Y / 0.000,   1.000,   0.000,  -1.000,
1          0.000,   3.718,   2.000,   0.282,
2          0.000,   9.389,   4.000,  -1.389,
3          0.000,  23.086,   6.000, -11.086 /

C      ... Fit cubic spline to data on a 4 by 4 grid of points.

NPT      (1) = NGRID
NPT      (2) = NGRID
NYDIM    (1) = NGRID
NYDIM    (2) = NGRID

CALL HDMVGI ( X, NDIM, NDOM, NPT, Y, NYDIM, NDEP,
1            NDEG, IOPT, WORK, NWORK, C, IER )

C      ... Check for error return

IF ( IER .NE. 0 ) THEN
    WRITE ( *, 9000 ) IER
    STOP
END IF

C      ... Evaluate spline at original data points and compare with
C      spline fit

WRITE ( *, 9010 )

DO J = 1, NGRID
    XX(2) = X(J,2)

    DO I = 1, NGRID
        XX(1) = X(I,1)

        CALL HDFNVL ( XX, INCX, C, WORK, NWORK, V, IER )

C      ... Check for error return

        IF ( IER .NE. 0 ) THEN
            WRITE ( *, 9020 ) IER
            STOP
        END IF

        WRITE ( *, 9030 ) XX(1), XX(2), Y(I,J,1), V(1),

```

```

1              ABS(Y(I,J,1)-V(1))
      ENDDO

ENDDO

WRITE ( *, 9040 )

STOP
9000 FORMAT ( ' Error return from HDMVGI - IER = ', I5,
1          /, '* ERROR *' )
9010 FORMAT ( /10X, 'X1', 10X, 'X2', 6X, 'True Y', 5X,
1          'Comp. Y', 4X, 'Abs. Diff.' / )
9020 FORMAT ( ' Error return from HDFNVL - IER = ', I5,
1          /, '* ERROR *' )
9030 FORMAT ( 5F12.3 )
9040 FORMAT ( / '* NORMAL TERMINATION *' )
END

```

OUTPUT FROM SAMPLE PROGRAM

X1	X2	True Y	Comp. Y	Abs. Diff.
0.000	0.000	0.000	0.000	0.000
1.000	0.000	1.000	1.000	0.000
2.000	0.000	0.000	0.000	0.000
3.000	0.000	-1.000	-1.000	0.000
0.000	1.000	0.000	0.000	0.000
1.000	1.000	3.718	3.718	0.000
2.000	1.000	2.000	2.000	0.000
3.000	1.000	0.282	0.282	0.000
0.000	2.000	0.000	0.000	0.000
1.000	2.000	9.389	9.389	0.000
2.000	2.000	4.000	4.000	0.000
3.000	2.000	-1.389	-1.389	0.000
0.000	3.000	0.000	0.000	0.000
1.000	3.000	23.086	23.086	0.000
2.000	3.000	6.000	6.000	0.000
3.000	3.000	-11.086	-11.086	0.000

* NORMAL TERMINATION *

HDFEVL: Fast Spline Evaluation with Extrapolation, but without Error Checking

PURPOSE

HDFEVL computes values of a tensor product spline function with one or more independent variables and any number of dependent variables. This routine also uses linear extrapolation if the user tries to evaluate the tensor product spline outside of the range of the independent variables. This method only performs limited error checking. It should only be used if a user is certain that the splines are valid, and that the workspace is large enough. The user is advised to use the corresponding routine HDENVL during product development. HDENVL is slower than HDFEVL, but performs error checking.

Furthermore, if the number of independent variables is larger than 3, this routine calls HDENVL internally. In this case the user is again advised to directly call HDENVL. Note that both routines have the same calling sequence. HDFEVL can also be used to calculate the values of a rational tensor product spline.

RELATED SUBPROGRAMS

HDNPVL	Evaluation of a Tensor Product Spline
HDFNVL	Fast Spline Evaluation without Error Checking
HDENVL	Spline Evaluation with Extrapolation
HDNPDR	Evaluation of a Partial Derivative of a Tensor Product Spline

METHOD

HDFEVL is very similar to HDENVL, except that it does not perform many error checks, and it uses a faster method if the number of independent variables is between one and three. If the number of independent variables is larger, this method calls HDENVL. For more details on the method see the documentation to HDENVL.

USAGE

```
INTEGER INCX,NWORK,IER
DOUBLE PRECISION X(1+(NDOM-1)*INCX),C(MC),WORK(NWORK),V(NRNG)
CALL HDFEVL(X,INCX,C,WORK,NWORK,V,IER)
```

where

NDOM is the number of independent variables for the tensor product spline represented by the C array. For example, if HDMVGI were used to construct the C array, NDOM would have the same value as it had in the call to HDMVGI. $NDOM = C(1)$.

MC is the length of the C array as determined when C was created. If necessary, MC may be obtained by a call to HD1SVY or MC may also be computed directly from the data stored in the C array:

$$MC = 2 + 3 * C(1) + \sum_{i=1}^{C(1)} [C(2+i)+C(2+C(1)+i)] + |C(2)| * \prod_{i=1}^{C(1)} C(2+C(1)+i).$$

NRNG is the number of dependent variables for the tensor product spline represented by the C array. For example, if HDMVGI were used to construct the C array, NRNG would have the same value as the argument NDEP in the call to HDMVGI. If $C(2) > 0$, then the C array represents a tensor product B-spline and $NRNG = C(2)$. If $C(2) < -1$, then the C array represents a rational tensor product B-spline and $NRNG = -(C(2)+1)$.

Note: The formula to calculate the required dimension NWORK of array WORK is complicated. It is very important that NWORK is at least of the size specified. If not, HDFEVL will write into areas that it should not—causing unpredictable side effects.

ARGUMENTS

X [INPUT, DOUBLE PRECISION , ARRAY]
Array of length $1+(NDOM-1)*INCX$ containing a set of values of the independent variables at which the values of the dependent variables are desired.

INCX [INPUT, INTEGER]
Increment parameter for array X. The i^{th} independent variable is stored in $X(1+(i-1)*INCX)$. See USAGE REMARKS for an example of the usage of INCX.

C [INPUT, DOUBLE PRECISION , ARRAY]
Spline definition array. See USAGE for dimension information. Usually, C will have been created by one of the spline construction subprograms of this chapter.

NWORK [INPUT, INTEGER]
 Length of array WORK. It depends on NRNG and NDOM—see USAGE.
 If NDOM = 1 and the C array represents a spline, then

$$\text{NWORK} \geq 5 * K_1 - 2.$$

If NDOM ≥ 2 and the C array represents a tensor product spline, then

$$\text{NWORK} \geq \text{NZERO} * (\text{NRNG} + 1) + 3 * \text{KMAX} + \text{NDOM}.$$

If NDOM = 1 and the C array represents a rational spline, then

$$\text{NWORK} \geq 5 * K_1 + \text{NRNG} - 1.$$

If NDOM ≥ 2 and the C array represents a rational tensor product spline, then

$$\text{NWORK} \geq \text{NZERO} * (\text{NRNG} + 2) + 3 * \text{KMAX} + \text{NDOM} + \text{NRNG} + 1,$$

where

$$\text{KMAX} = \max_{i=1, \dots, \text{NDOM}} K_i \quad \text{and} \quad \text{NZERO} = \prod_{i=2}^{\text{NDOM}} K_i$$

and $K_i = C(2 + i)$ is the order of the spline for the i^{th} independent variable.

V [OUTPUT, DOUBLE PRECISION, ARRAY]
 Array of length NRNG containing the values of the dependent variables at X. See USAGE for a description of NRNG.

IER [OUTPUT, INTEGER]
 Success/error code. For $\text{IER} < 0$, results have not been computed; HDFEVL has set $\text{V}(1) = \text{HDMCON}(1)$. *Note that no errors will be detected by this routine if the number of independent variables is less than 4.* The user should only use this method if he is sure that these errors cannot happen. Otherwise, he should use HDNPVL instead.

$\text{IER} = 0$ Success, results computed.

$\text{IER} > 0$ Success, results computed. Extrapolation was done, and the largest percentage by which one of the independent variables exceeded the range of the spline is larger than 10 %. Let $\text{DELTA}(i)$ be the amount by which variable i exceeds its range, and $\text{CL}(i)$ and $\text{CU}(i)$ the upper and lower bounds for the range of variable i . Then

$$\text{IER} = \max_{i=1, \dots, \text{NDOM}} \text{INT} \left(10 \frac{|\text{DELTA}(i)|}{\text{CU}(i) - \text{CL}(i)} \right).$$

IER = -1	$K_i < 1$ for some i .
IER = -3	NWORK is too small; the number of words needed is given by the printed error message.
IER = -6	Number of spline coefficients with respect to the i^{th} independent variable is less than K_i for some i .
IER = -8	Invalid knot set.
IER = -10	The absolute value of the denominator of the spline (when evaluated at X) is less than HDMCON(6), the relative precision.
IER = -38	X is inside a knot interval that is too small. This error is extremely unlikely.
IER = -51	$C(1) < 1$.
IER = -52	$C(2) = -1$ or 0 .
IER = -100	NDOM > 6. Extrapolation for splines with more than 6 independent variables is not permitted. Contact SOS support if this error occurs.
IER = -101	NRNG > 20. Extrapolation for splines with more than 20 dependent variables is not permitted. Contact SOS support if this error occurs.
IER = -200	An error occurred while trying to extrapolate. One reason might be insufficient workspace. Increase NWORK.

USAGE REMARKS

When the input C array is output from a spline construction subroutine, the user is responsible only for setting X, INCX, and NWORK. If the construction subroutine was called correctly, errors other than IER=-3, IER=-100, IER=-101, or IER=-200 should not be returned by HDFEVL.

The following code fragment evaluates a tensor product spline at points x_i , $i = 1, 2, \dots, M$ where each x_i is entered in X as a row vector, $x_i = (x_{i,1}, x_{i,2}, \dots, x_{i,\text{NDOM}})$.

```

      DOUBLE PRECISION  X(M,NDOM),C(MC),WORK(NWORK),V(NRNG)
      INCX = M
      DO 10 I=1,M
         CALL HDFEVL(X(I,1),INCX,C,WORK,NWORK,V,IER)
10  CONTINUE

```

However, if the x_i are stored as columns in array X, then one should write:

```
DOUBLE PRECISION X(NDOM,M),C(MC),WORK(NWORK),V(NRNG)
INCX = 1
DO 10 J=1,M
    CALL HDFEVL(X(1,J),INCX,C,WORK,NWORK,V,IER)
10 CONTINUE
```

EXAMPLE

Subroutine HDMVGI is used in the following example to fit a bicubic spline to data on a 4 by 4 grid. Subroutine HDFEVL is then used to evaluate the spline at the grid points. A comparison of the spline fit with the original data points is displayed. We also evaluate the spline at some points outside of the range of the data to test the extrapolation.

SAMPLE PROGRAM

```
PROGRAM SAMPLE

INTEGER      IOPT, MCDIM, NDEG, NDEP, NDIM, NDOM, NGRID, NWORK,
1            INCX

PARAMETER ( IOPT = 1, NDEG = 3, NDEP = 1, NGRID = 4,
1            NDIM = 4, NDOM = 2, INCX = 1,
2            MCDIM = 2 + NDOM*(2*NDEG) + 2*NDEG
3            + NDEP*(NGRID+NDEG-1)**2,
4            NWORK = (NGRID+NDEG-1)*(3*NDEG+1)
5            + 3*(NDEG+1)**2 + 8*NDEG + 4*NGRID + 14 )

INTEGER      I , IER, J
INTEGER      NPT(NDOM), NYDIM(NDOM)
DOUBLE PRECISION X(NDIM,NDOM), Y(NGRID,NGRID,1), WORK(NWORK),
1            C(MCDIM), XX(2), V(1)

DATA      X / 0.000, 1.000, 2.000, 3.000,
1          0.000, 1.000, 2.000, 3.000 /

DATA      Y / 0.000, 1.000, 0.000, -1.000,
1          0.000, 3.718, 2.000, 0.282,
2          0.000, 9.389, 4.000, -1.389,
3          0.000, 23.086, 6.000, -11.086 /

C      ... Fit cubic spline to data on a 4 by 4 grid of points.

NPT      (1) = NGRID
NPT      (2) = NGRID
NYDIM    (1) = NGRID
NYDIM    (2) = NGRID

CALL HDMVGI ( X, NDIM, NDOM, NPT, Y, NYDIM, NDEP,
```

```

1          NDEG, IOPT, WORK, NWORK, C, IER )

C      ... Check for error return

      IF ( IER .NE. 0 ) THEN
        WRITE ( *, 9000 ) IER
        STOP
      END IF

C      ... Evaluate spline at original data points and compare with
C      spline fit

      WRITE ( *, 9010 )

      DO J = 1, NGRID
        XX(2) = X(J,2)

        DO I = 1, NGRID
          XX(1) = X(I,1)

          CALL HDFEVL ( XX, INCX, C, WORK, NWORK, V, IER )

C          ... Check for error return

          IF ( IER .NE. 0 ) THEN
            WRITE ( *, 9020 ) IER
            STOP
          END IF

          WRITE ( *, 9030 ) XX(1), XX(2), Y(I,J,1), V(1),
1          ABS(Y(I,J,1)-V(1))
        ENDDO

      ENDDO

C      ... Evaluate spline at data points outside of the region
C      defined by the data points to check extrapolation...

      WRITE ( *, 10010 )

      DO J = NGRID, NGRID
        XX(2) = X(J,2)+0.1D0*X(J,2)

        DO I = 1, NGRID
          XX(1) = X(I,1)

          CALL HDFEVL ( XX, INCX, C, WORK, NWORK, V, IER )

```

```
C          ... Check for error return

          IF ( IER .NE. 0 ) THEN
              WRITE ( *, 9020 ) IER
              STOP
          END IF

          WRITE ( *, 10030 ) XX(1), XX(2), V(1)
          ENDDO

          ENDDO

          WRITE ( *, 9040 )

          STOP
9000 FORMAT ( ' Error return from HDMVGI - IER = ', I5,
1          /, '* ERROR *' )
9010 FORMAT ( /10X, 'X1', 10X, 'X2', 6X, 'True Y', 5X,
1          'Comp. Y', 4X, 'Abs. Diff.' / )
9020 FORMAT ( ' Error return from HDNPVL - IER = ', I5,
1          /, '* ERROR *')
9030 FORMAT ( 5F12.3 )
9040 FORMAT ( / '* NORMAL TERMINATION *' )
10010 FORMAT ( /10X, 'X1', 10X, 'X2', 6X, 'Comp. Y' / )
10030 FORMAT ( 3F12.3 )
          END
```

OUTPUT FROM SAMPLE PROGRAM

X1	X2	True Y	Comp. Y	Abs. Diff.
0.000	0.000	0.000	0.000	0.000
1.000	0.000	1.000	1.000	0.000
2.000	0.000	0.000	0.000	0.000
3.000	0.000	-1.000	-1.000	0.000
0.000	1.000	0.000	0.000	0.000
1.000	1.000	3.718	3.718	0.000
2.000	1.000	2.000	2.000	0.000
3.000	1.000	0.282	0.282	0.000
0.000	2.000	0.000	0.000	0.000
1.000	2.000	9.389	9.389	0.000
2.000	2.000	4.000	4.000	0.000
3.000	2.000	-1.389	-1.389	0.000
0.000	3.000	0.000	0.000	0.000
1.000	3.000	23.086	23.086	0.000
2.000	3.000	6.000	6.000	0.000
3.000	3.000	-11.086	-11.086	0.000

X1	X2	Comp. Y
0.000	3.300	0.000
1.000	3.300	29.496
2.000	3.300	6.600
3.000	3.300	-16.296

* NORMAL TERMINATION *

Chapter 9

Getting Started

The SOS library provides considerable flexibility both in problem formulation and solution options. This flexibility is achieved by incorporating user supplied subroutines to define the problem and solution characteristics. Although there is no obvious way to relieve the user of this burden there are some guidelines that are generally helpful.

Perhaps the most important first step is to carefully *formulate* the optimal control problem. It is usually extremely helpful to write down the mathematical description of the problem, even if some of the details are omitted. Special attention should be given to identifying the variables and constraints.

With a mathematical formulation in hand the next step is to implement the necessary software. The user must supply subroutines ODEINP and ODERHS and possibly may need the routines ODEPTF, ODEIGS, and ODEPRT. It is suggested that the subroutines be written in the following order:

1. ODERHS—right hand side of differential-algebraic equations
2. ODEINP—problem description, method, options, etc.
3. ODEPTF—nonlinear point functions, computed quantities, etc.
4. ODEIGS—special purpose initial guess
5. ODEPRT—special purpose output.

Usually by writing the subroutines to evaluate the differential and algebraic equations, the user is forced to identify the fundamental quantities (i.e. state and control variables). Once this information has been established constructing the problem description in the ODEINP routine is relatively straightforward. Often a simplified version of the desired problem can be tested using the SOS dummy subroutines DUMYPF, DUMYIG, and DUMYPR in lieu of the other required software.

If possible *modify a working example*, e.g. (p. 596).

For applications with many phases *write one phase at a time!*

For applications with many constraints *write one constraint at a time!*

DO NOT USE LINEAR INTERPOLATION OF TABULAR DATA. (p. 603)

Do not use *embedded iterations* when evaluating the point functions and/or differential equations.

Become familiar with the SOS library by solving the *workshop problems* (p. 586).

Appendix A

The Index Set Array

The following is a description of the index set array INDSET which is output by HJSFDI and used by HSSFDC, HSSFDH, HSSFDJ, and HSSFDP.

<u>Index Range</u>	<u>Name</u>	<u>Description</u>
1	l_{INDSET}	The length of INDSET, which depends on whether the Hessian information is stored along with the Jacobian information. $n_H=0$ $l_{INDSET} = 21 + n_J + 3 * (n + 1).$ $n_H \neq 0$ $l_{INDSET} = 21 + 2 * n_J + 4 * (n + 1) + (m + 1) + 2 * n_H.$
2	m	Number of finite difference functions; NROW.
3	n	Number of independent variables; NCOL.
4	n_I	Number of index sets.
5	n_J	Number of finite difference nonzeros in the Jacobian matrix; NFDJNZ.
6	n_H	Number of finite difference Hessian nonzeros; NFDHZN.
7...15	—	Starting locations in the INDSET array for key subarrays. The locations will be called: $j_{IS}, j_{ISP}, j_{CP}, j_C, j_{RP}, j_R, h_{RP}, h_R,$ and $h_C.$
16	n_S	Number of separable elements found in the Hessian sparsity pattern.

17	i_{com}	Set and used by HSSFDC, HSSFDJ, and HSSFDH for keeping track of initial vs. continue calls in the reverse communication.
18...20	—	Future expansion.
$j_{IS} \dots j_{IS} + n - 1$	$\vec{J}_{IS}$	The indices of the independent variables ordered by index set.
$j_{ISP} \dots j_{ISP} + n_I$	$\vec{J}_{ISP}$	Pointers into the $\vec{J}_{IS}$ subarray for the beginning of each index set.
$j_{CP} \dots j_{CP} + n$	$\vec{J}_{CP}$	Pointers into the $\vec{J}_C$ subarray locating the start of the indices corresponding to each column of the Jacobian.
$j_C \dots j_C + n_J - 1$	$\vec{J}_C$	The indices of the nonzero locations in the Jacobian matrix ordered by column.
$j_{RP} \dots j_{RP} + m$	$\vec{J}_{RP}$	Pointers into the $\vec{J}_R$ subarray locating the start of the indices corresponding to each row of the Jacobian.
$j_R \dots j_R + n_J - 1$	$\vec{J}_R$	The indices of the nonzero locations in the Jacobian matrix ordered by row.

<u>Index Range</u>	<u>Name</u>	<u>Description</u>
$h_{RP} \dots h_{RP} + n$	$\vec{H}_{RP}$	Pointers into the $\vec{H}_R$ subarray locating the start of the indices corresponding to each row of the Hessian.
$h_R \dots h_R + n_H - 1$	$\vec{H}_R$	The indices of the nonzero locations in the Hessian matrix ordered by row.
$h_C \dots h_C + n_H - 1$	$\vec{H}_C$	The column indices for the Hessian matrix.
l_{INDSET}	CS	A check sum; must be equal to $m + n + n_J$ for a valid INDSET array.

Appendix B

Standard Math and Machine Constants

A number of standard math and machine constants are used by the SOS library. The double precision constant values can be obtained using the function `HDMCON(k)` where the integer $k = 1, \dots, 16$. Similarly, integer constant values can be obtained using the function `JHMCN(k)` where the integer $k = 1, \dots, 15$. The following tables summarize the constants. For the sake of illustration the second column presents “typical” double precision and integer values obtained using an INTEL FORTRAN 90 compiler.

Double Precision Constants, HDMCON(k)

Index	Value	Description
1	1.797693134862316E+308	Clobber constant. A number used by SOS when errors are detected to clobber continued computations. If special computer numbers such as “out of range”, “not a number”, or “indefinite” exist and are printable, then the clobber constant is set to one of them. Otherwise, the clobber constant is the same as the overflow threshold.
2	4.494232837155790E+307	Symmetric range. The largest positive x such that x , $-x$, $1/x$, and $-1/x$ are floating point numbers.
3	1.797693134862316E+308	Overflow threshold. The largest positive x such that x , and $-x$, are floating point numbers.
4	2.225073858507201E-308	Underflow threshold. The smallest positive x such that x , and $-x$, are floating point numbers.
5	2.220446049250313E-016	Relative spacing. The maximum relative spacing between adjacent floating point numbers; that is, $b^{(1-n)}$ where b is the base (see index 7) and n is the mantissa length (see index 8). This is the conservative definition of machine epsilon.
6	2.220446049250313E-016	Relative precision. The smallest positive floating point x such that $1 - x < 1 < 1 + x$ where $1 - x$ and $1 + x$ are the stored values of the computed results. This is the IFIP definition of machine epsilon.
7	2.000000000000000	Radix. The base of the floating point number system.
8	53.00000000000000	Mantissa length. The number of radix digits in the mantissa of a floating point number.
9	709.0000000000000	Exponential overflow. The largest positive floating point integer x such that e^x is a floating point number (does not overflow).
10	-708.0000000000000	Exponential underflow. The largest (in magnitude) negative floating point integer x such that e^x is a floating point number (does not underflow).
11	2147483647.00000	Maximum float-able integer. The largest positive floating point integer such that it, and all smaller such integers, can be truncated to INTEGER and converted back to the same floating point number.
12	3.14159265358979	π .
13	2.71828182845905	The base of the natural logarithm, e .
14	0.577215664901533	Euler’s constant, γ .
15	1.745329251994330E-002	Number of radians in one degree, $\pi/180$.
16	57.2957795130823	Number of degrees in one radian, $180/\pi$.

Integer Constants, JHMCN(k)

Index	Value	Description
1	2147483647	Clobber constant. A number used by SOS when errors are detected to clobber continued computations. If special computer numbers such as “out of range”, “not a number”, or “indefinite” exist and are printable, then the clobber constant is set to one of them. Otherwise, the clobber constant is the same as the overflow threshold.
2	2147483647	Symmetric range. The largest integer j such that a full arithmetic capability is supported for all integers in $[-j, j]$.
3	2147483647	Overflow threshold. The largest integer j such that all integers in $[-j, j]$ are machine integers.
4	6	Default error message unit. The SOS convention identifies the default error message unit with the default output unit (see index 6).
5	5	Default input unit. The logical unit number that is preconnected to the standard input unit of the system.
6	6	Default output unit. The logical unit number that is preconnected to the standard output unit of the system.
7	80	Number of input characters. The default number of characters, including blanks, which can be read from a single record on the default input unit (see index 5).
8	132	Number of output characters. The default number of characters, including blanks and carriage control, which can be output to a single record on the default output unit (see index 6).
9	9	Number of decimal digits for INTEGER. The largest number of decimal digits always allowed and converted correctly by the compiler when compiling INTEGER constants.
10	7	Number of decimal digits for REAL. The largest number of decimal digits always allowed and converted correctly by the compiler when compiling REAL constants.
11	16	Number of decimal digits for DOUBLE PRECISION. The largest number of decimal digits always allowed and converted correctly by the compiler when compiling DOUBLE PRECISION constants.
13	4	Characters in INTEGER. The maximum number of characters that will fit in one INTEGER storage unit.
14	1	Size of REAL word. The maximum number of INTEGER storage units that will fit in one REAL storage unit.
15	2	Size of DOUBLE PRECISION word. The maximum number of INTEGER storage units that will fit in one DOUBLE PRECISION storage unit.

Bibliography

- [1] BCSLIB-EXT Mathematical Library, The Boeing Extended Mathematical Subprogram Library. Mathematics and Engineering Analysis Library Report 20462-0520-R3, Boeing Information and Support Services, The Boeing Company, PO Box 3707, Seattle, WA 98124-2207, December 1993.
- [2] Uri M. Ascher and Linda R. Petzold. The Numerical Solution of Delay-Differential-Algebraic Equations of Retarded and Neutral Type. *SIAM Journal on Numerical Analysis*, 32(5):1635–1657, 1995.
- [3] John T. Betts. Trajectory Optimization Using Sparse Sequential Quadratic Programming. In Roland Bulirsch, Angelo Miele, Josef Stoer, and Klaus H. Well, editors, *Optimal Control*, volume 111 of *International Series of Numerical Mathematics*, pages 115–128, Basel, 1993. Birkhäuser Verlag.
- [4] John T. Betts. Issues in the Direct Transcription of Optimal Control Problems to Sparse Nonlinear Programs. In Roland Bulirsch and Dieter Kraft, editors, *Computational Optimal Control*, volume 115 of *International Series of Numerical Mathematics*, pages 3–18. Birkhäuser Verlag, Basel, 1994.
- [5] John T. Betts. *Practical Methods for Optimal Control using Nonlinear Programming*. Society for Industrial and Applied Mathematics, Philadelphia, PA., 2001.
- [6] John T. Betts. *Practical Methods for Optimal Control and Estimation using Nonlinear Programming*. Second Edition. Society for Industrial and Applied Mathematics, Philadelphia, PA., 2010.
- [7] John T. Betts. *Practical Methods for Optimal Control using Nonlinear Programming*. Third Edition. Society for Industrial and Applied Mathematics, Philadelphia, PA., 2020.
- [8] John T. Betts, Samuel K. Eldersveld, Paul D. Frank, and John G. Lewis. An Interior-Point Nonlinear Programming Algorithm for Very Large Scale Optimization. In Lorenz T. Biegler, Omar Ghattas, Matthias Heinkenschloss, and Bart van Bloemen Waanders, editors, *Large-Scale PDE-Constrained Optimization*, pages 184–198. Springer-Verlag, Berlin, 2003.
- [9] John T. Betts and Paul D. Frank. A Sparse Nonlinear Optimization Algorithm. *Journal of Optimization Theory and Applications*, 82(3):519–541, September 1994.
- [10] John T. Betts and William P. Huffman. Application of Sparse Nonlinear Programming to Trajectory Optimization. *AIAA Journal of Guidance, Control, and Dynamics*, 15(1):198–206, January–February 1992.

- [11] John T. Betts and William P. Huffman. Path Constrained Trajectory Optimization Using Sparse Sequential Quadratic Programming. *AIAA Journal of Guidance, Control, and Dynamics*, 16(1):59–68, January–February 1993.
- [12] Arthur E. Bryson, Jr. and Yu-Chi Ho. *Applied Optimal Control*. John Wiley & Sons, New York, NY, 1975.
- [13] Thomas F. Coleman and Jorge J. Moré. Estimation of Sparse Jacobian Matrices and Graph Coloring Problems. *SIAM Journal on Numerical Analysis*, 20:187–209, 1983.
- [14] A. R. Curtis, M. J. D. Powell, and J. K. Reid. On the Estimation of Sparse Jacobian Matrices. *Journal of the Institute of Mathematics and Applications*, 13:117–120, 1974.
- [15] Ernst D. Dickmanns. Efficient Convergence and Mesh Refinement Strategies for Solving General Ordinary Two-point Boundary Value Problems by Collocated Hermite Approximation. 2nd IFAC Workshop on Optimisation, Oberpfaffenhofen, September 1980.
- [16] Paul J. Enright and Bruce A. Conway. Optimal Finite-thrust Spacecraft Trajectories Using Collocation and Nonlinear Programming. *AIAA Journal of Guidance, Control, and Dynamics*, 14(5):981–985, 1991.
- [17] Philip E. Gill, Walter Murray, and Michael A. Saunders. Users Guide for QPOPT (Version 1): a Fortran Package for Quadratic Programming. Technical Report Report NA 95-1, Department of Mathematics, University of California, San Diego.
- [18] Philip E. Gill, Walter Murray, Michael A. Saunders, and Margaret H. Wright. Some Theoretical Properties of an Augmented Lagrangian Merit Function. Technical Report Report SOL 86-6, Department of Operations Research, Stanford University, April 1986.
- [19] Philip E. Gill, Walter Murray, Michael A. Saunders, and Margaret H. Wright. A Schur-complement Method for Sparse Quadratic Programming. Technical Report Report SOL 87-12, Department of Operations Research, Stanford University, October 1987.
- [20] E. Hairer, S. P. Norsett, and G. Wanner. *Solving Ordinary Differential Equations I, Nonstiff Problems*. Springer-Verlag, New York, New York, 1993.
- [21] Charles R. Hargraves and Stephen W. Paris. Direct Trajectory Optimization Using Nonlinear Programming and Collocation. *AIAA Journal of Guidance, Control, and Dynamics*, 10(4):338–342, July–August 1987.
- [22] Oskar von Stryk. Numerical Solution of Optimal Control Problems by Direct Collocation. In Roland Bulirsch, Angelo Miele, Josef Stoer, and Klaus H. Well, editors, *Optimal Control*, volume 111 of *International Series of Numerical Mathematics*, pages 129–143, Basel, 1993. Birkhäuser Verlag.